C++ Programlama Dili ile
NESNE YÖNELİMLİ PROGRAMLAMA

Modern Standartlar, Kavramlar ve Algoritma Mantığı

C++

Elektronik Yük. Müh.
İlhan ÖZKAN

Üçüncü Sürüm, Nisan 2026, Ankara

Önsöz

Ülkemizde her alanda olduğu gibi yazılım alanında da bir terminoloji karmaşasının yaşandığı aşikardır. Son dönemde yazılanlar adeta çeviri yapan programların bir gecede ürettiği çıktılar birleştirilerek hazırlanmaktadır. Birçok programlama dili kullanarak yazılım geliştiriyor olmama rağmen, yazılım alanında içeriği anlama konusunda oldukça zorlanıyorum. Bu kitapla birlikte tarihsel gelişimle birlikte programlama mantığının yapısal programlamaya geçişi sürecini de içerecek şekilde yazılımım gelişme süreci anlatılmıştır.

Yapısal programlamanın en çok uygulandığı Pascal dilidir. Pascal diline yakın zamanda ortaya çıkan C dili, kısa sürede sistem dili olarak tarihte yerini almıştır. Günümüzde sistem programlama ve yüksek hız gerektiren gömülü sistemlerde vazgeçilmez bir dil olmuştur. Yapısal programlamada, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler.

Yapısal dillerde çok büyük projelerde değişim yönetiminin zorlaşması, hata ayıklamanın zor olması ve aynı kodların benzer projelerde tekrar tekrar yazılması gibi problemlerden dolayı yazılımcıların imdadına Simula ve Smalltalk programlama dillindeki yaklaşım yetişmiştir. Bu diller oldukça yavaş olmasına rağmen getirdiği çözümler oldukça yenilikçiydi. Smalltalk dilinde nesneler temel yapıtaşlarıdır. Nesneler; durum ve davranışlara sahiptir ve programlama, nesnelerin birbirlerine ileti göndermesiyle yapılır.

C++ programlama dili ise C diline yapılan nesne yönelimli programlama eklentileri yapılarak geliştirilmiş bir programlama dili olup ilk sürümü 1985 yılında yayınlanmıştır. Hala popüler olarak yüksek performans isteyen projelerde çokça kullanılmaktadır.

Nesne yönelimli programlama mantığını oluşturan kavramların programcıların kafasında aynı ışığı yakması için bu kitap hazırlanmıştır. Bu dilin eğitimini verecek tüm kişiler için serbestçe kullanılabilir.

Bu kitabın güncel sürümüne https://github.com/HoydaBre/cplusplus adresinden erişilebilir. Amacım Türkçe düşünen ve konuşan öğrencilerimize kavramları doğru bir şekilde aktarmaktır. Katkılarınızı bekler iyi çalışmalar dilerim.

Elektronik Yüksek Mühendisi, İlhan ÖZKAN
ilhanozkan[at]outlook.com
linkedin.com/in/ilhanozkan
Şubat 2025

Sürümler

İkinci Sürüm

İlk sürümden farklı olarak aşağıdaki değişiklikler yapılmıştır.

- ◇ Aşırı yükleme kavramı fonksiyonlar altına alındı.
- ◇ Fonksiyonların aşırı yüklenmesi kısmı düzeltildi.
- ◇ Sınıflar Arası İlişkiler ve Sınıf Tasarlama İlkeleri bölümü eklendi.
- ◇ Yapılar C diline göre anlatılmıştı. C++ diline daha uygun hale getirildi.
- ◇ Özellikle Tasarım Desenlerinde gösterici kullanımından ziyade güvenli kod yazma örnekleri eklendi.
- ◇ SOLID ilkelerine örnekler verildi.

Mayıs 2025

Üçüncü Sürüm

İkinci sürümden farklı olarak;

- ◇ Her bölüme düşün ve çöz kısmı eklendi.
- ◇ Kodlar renklendirildi.
- ◇ Yapılar ve birlikler bölümü, Göstericiler sonrasına alındı.
- ◇ Göstericiler bölümü sonunda metinlere giriş yapıldı.
- ◇ İsim Uzayları, Yapılar ve Birlikler bölümüne taşındı.
- ◇ Numaralandırma sınıfı, Sınıf Çeşitleri başlığına alındı.
- ◇ Akıllı gösterici kullanımı kitabın geneline yayıldı.
- ◇ Metinler kısmı Standart Şablon Kütüphanesi altına taşındı.
- ◇ Algoritma Şablonlarına örnek Verildi.
- ◇ Faydalı şablonlar bölümü eklendi
- ◇ İstisnalar kısmına Yığın Çözülmesi başlığı eklendi

Nisan 2026

İçindekiler

Önsöz i

Sürümler ii

1 Temel Bilgisayar Kavramları 1
- 1.1 Mühendis ve Teknisyen . . . 1
- 1.2 Bilgisayara Niçin İhtiyaç Duyulur? . . . 1
- 1.3 Veri ve Bilgi . . . 1
- 1.4 En Basit Bilgisayar . . . 1
- 1.5 Emri Alma, Çözme ve İcra Çevrimi . . . 3
- 1.6 İşlemci Tasarlama Stratejileri . . . 4
- 1.7 Onaltılı Sayı Sisteminin Kullanılması . . . 5
- 1.8 Program ve Programlama . . . 5
- 1.9 Genel Amaçlı Bilgisayarlar ve İşletim Sistemleri . . . 6
- 1.10 Düşün ve Çöz . . . 7

2 Temel Yazılım Kavramları 8
- 2.1 Düşük Düzey Programlama . . . 8
- 2.2 Genel Amaçlı Yüksek Düzey Diller . . . 9
- 2.3 Arapsaçı Kod . . . 9
- 2.4 Değişken . . . 11
- 2.5 Fonksiyon . . . 12
- 2.6 Yapısal Programlama . . . 12
- 2.7 Yapısal Programlama Genel Çerçevesi . . . 13
- 2.8 Nesne Yönelimli Programlama İhtiyacı . . . 14
- 2.9 Düşün ve Çöz . . . 14

3 Derleyiciler ve Çalıştırma Ortamı 16
- 3.1 Derleyici . . . 16
- 3.2 C++ derleyicileri . . . 16
- 3.3 Entegre Geliştirme Ortamı . . . 17
- 3.4 Çevrimiçi Derleyiciler . . . 18
- 3.5 Derleme Zamanı . . . 18
- 3.6 Hatalar . . . 19
- 3.7 C++ Dilinin Kullanım Alanları . . . 19
- 3.8 Düşün ve Çöz . . . 19

4 C++ Programlama Diline Giriş 21
- 4.1 En Basit C++ Programı . . . 21
- 4.2 Söz Dizim Kuralları . . . 21
  - 4.2.1 Kaynak Kodumuzu Oluşturacak Karakterler . . . 21
  - 4.2.2 Talimatlar ve Anahtar Kelimeler . . . 22
  - 4.2.3 Açıklamalar . . . 23
- 4.3 Değişken Tanımlama . . . 24
  - 4.3.1 Tam Sayı Veri Tipleri . . . 24
  - 4.3.2 Kayan Noktalı Sayı Veri Tipleri . . . 24
  - 4.3.3 Diğer Veri Tipleri . . . 24
  - 4.3.4 Değişken Tanımlama . . . 24
  - 4.3.5 Değişken Kimliklendirme Kuralları . . . 25
  - 4.3.6 Veri Tipi Değiştiricileri . . . 26
- 4.4 Değişmezler . . . 26
- 4.5 Sabit Değişkenler . . . 28
- 4.6 Sabit İfadeler . . . 29
- 4.7 Ön İşlemci Makrolarıyla Değişmez Tanımı . . . 29

4.8 Yazdığımız Kod Nasıl İcra Edilir? . . . 30
4.9 İşleçler . . . 30
4.9.1 Aritmetik İşleçler . . . 30
4.9.2 Tekli İşleçler . . . 31
4.9.3 İlişkisel İşleçler . . . 32
4.9.4 Bit Düzeyi İşleçler . . . 33
4.9.5 Mantıksal İşleçler . . . 33
4.9.6 Atama işleçleri . . . 34
4.9.7 İşleç Öncelikleri . . . 34
4.10 Kayan Noktalı Sayı Hesapları . . . 35
4.11 Düşün ve Çöz . . . 36

5 Giriş Çıkış İşlemleri 37
5.1 Modüler Programlama . . . 37
5.2 Klavyeden Veri Okuma ve Konsola Veri Yazma . . . 37
5.3 Giriş ve Çıkışı Biçimlendirme . . . 39
5.4 Uluslar Araşı Karakterlerle Çalışma . . . 40
5.5 Düşün ve Çöz . . . 41

6 Kontrol İşlemleri 42
6.1 Ardışık İşlem ve Kontrol İşlemleri . . . 42
6.2 Akış Diyagramları . . . 42
6.3 Duruma Göre Seçimler . . . 43
6.3.1 If Talimatı . . . 43
6.3.2 If-Else Talimatı . . . 45
6.3.3 Sarkan Else . . . 47
6.3.4 Switch Talimatı . . . 48
6.3.5 Üçlü Koşul İşleci . . . 50
6.4 İlişkisel Döngüler . . . 50
6.4.1 Sayaç Kontrollü Döngüler . . . 51
6.4.2 While Talimatı . . . 52
6.4.3 Do-While Talimatı . . . 52
6.4.4 For Talimatı . . . 53
6.4.5 İç İçe Döngü Kodu . . . 56
6.4.6 Gözcü Kontrollü Döngüler . . . 57
6.5 Dallanmalar . . . 57
6.5.1 Continue ve Break Talimatları . . . 58
6.5.2 Return Talimatı . . . 60
6.5.3 Goto Talimatı . . . 60
6.6 Düşün ve Çöz . . . 60

7 Fonksiyonlar 61
7.1 Fonksiyon Nedir? . . . 61
7.2 Hazır Fonksiyonlar . . . 61
7.2.1 cmath Başlık Dosyası . . . 62
7.2.2 cstdlib Başlık Dosyası . . . 62
7.3 Kullanıcı Tanımlı Fonksiyonlar . . . 63
7.3.1 Fonksiyon Tanımı Nasıl Yapılır? . . . 64
7.3.2 Fonksiyon Bildirimi Nasıl Yapılır? . . . 66
7.4 Çağrı Kuralı . . . 69
7.5 Depolama Sınıfları . . . 69
7.5.1 auto Depolama Sınıfı . . . 70
7.5.2 static Depolama Sınıfı . . . 70
7.5.3 extern Depolama Sınıfı . . . 71
7.5.4 register Depolama Sınıfı . . . 72
7.6 Evrensel ve Yerel Değişken . . . 72
7.7 Değerle Çağırma . . . 73
7.8 Özyinelemeli Fonksiyonlar . . . 74
7.9 Sabit İfadelerin Fonksiyonlarda Kullanımı . . . 76
7.10 Satır İçi Fonksiyonlar . . . 76
7.11 Varsayılan Argümanlar . . . 77

7.12 Aşırı Yükleme . . . 77
7.13 Düşün ve Çöz . . . 78

8 Diziler **79**
8.1 Dizi Nedir . . . 79
8.2 Tek boyutlu Diziler . . . 79
8.3 İki Boyutlu Diziler . . . 83
8.4 Çok Boyutlu Diziler . . . 85
8.5 Parametre Olarak Diziler . . . 87
8.6 Aralık Tabanlı For Döngüsü . . . 88
8.7 Düşün ve Çöz . . . 88

9 Göstericiler ve Referanslar **89**
9.1 Gösterici nedir? Niçin İhtiyaç Duyarız? . . . 89
9.2 Gösterici Tanımlama . . . 89
9.3 Dinamik Değişken Kullanımı . . . 91
9.4 Gösterici Aritmetiği . . . 92
9.5 Gösterici Dizileri . . . 94
9.6 Referanslar . . . 95
9.7 Referansla Çağırma . . . 96
9.8 Göstericilerin Dizileri İşaret Etmesi . . . 97
9.9 Fonksiyonlardan Bir Diziyi Geri Döndürme . . . 98
9.10 Fonksiyon Göstericisi . . . 99
9.11 Dinamik Hafıza Yönetimi . . . 99
9.12 Metinler ve Göstericiler . . . 101
9.12.1 Karakter Dizisi . . . 101
9.12.2 Gösterici ile Metinlerin İlişkisi . . . 101
9.12.3 Metinlerle İşlemler . . . 102
9.12.4 Metinler İçin Tavsiye Edilen Yöntem . . . 103
9.13 Düşün ve Çöz . . . 103

10 Yapılar, Birlikler ve İsim Uzayları **104**
10.1 Yapılar . . . 104
10.2 Yapı Elemanlarına Erişim . . . 104
10.3 Yapı Göstericileri . . . 105
10.4 Yapılar ve Fonksiyonlar . . . 106
10.5 İç İçe Yapılar . . . 107
10.6 Öz Referanslı Yapılar . . . 108
10.7 Yapı Dolgusu . . . 108
10.8 Yapılarda Bit Alanları . . . 109
10.9 Birlikler . . . 110
10.10 İsim Uzayları . . . 111
10.10.1 İsim Uzayı Tanımlama . . . 111
10.10.2 Bağımlı Argüman Arama . . . 113
10.10.3 İsim Uzaylarını Genişletme . . . 113
10.10.4 using Anahtar Kelimesinin İsim Uzaylarında Kullanılması . . . 114
10.11 Düşün ve Çöz . . . 115

11 Nesne Yönelimli Programlama **116**
11.1 Nesne Yönelimli Programlama Paradigması . . . 116
11.2 Sınıf ve Nesne Nedir? Nasıl Tanımlanır? . . . 116
11.3 Erişim Değiştiricileri . . . 119
11.4 Nesne Yapıcısı . . . 119
11.5 Dinamik Nesne İmalatı . . . 122
11.6 this Göstericisi . . . 123
11.7 Soyut, Somut ve Bilgi Gizleme . . . 124
11.8 Kapsülleme . . . 124
11.9 Kalıtım . . . 127
11.10 Yöntemlerin Aşırı Yüklenmesi . . . 134
11.11 Yapıcıların Aşırı Yüklenmesi . . . 135
11.11.1 Varsayılan Yapıcı . . . 135

11.11.2 Parametreli Yapıcılar . . . 136
11.11.3 Devredilen Yapıcı . . . 136
11.11.4 Kopya Yapıcı . . . 137
11.11.5 Atama İşlecinin Aşırı Yüklenmesi . . . 138
11.11.6 Varsayılan Yapıcının Değiştirilmesi . . . 138
11.11.7 Taşıma Yapıcısı . . . 139
11.11.8 Yapıcılarda Varsayılan Argümanlar . . . 140
11.12 using Anahtar Kelimesi ile Yapıcıların Miras Alınması . . . 141
11.13 Nesne Yıkıcı . . . 141
11.14 friend Anahtar Kelimesi . . . 142
11.15 Statik Sınıf Üyeleri . . . 143
11.16 const Sınıf Üyeleri . . . 144
11.17 mutable Depolama Sınıfı . . . 144
11.18 Geçersiz Kılma . . . 145
11.19 Ara Yüzler . . . 146
11.20 Çok Biçimlilik . . . 148
11.21 Sınıf Çeşitleri . . . 149
11.22 Numaralandırma Sınıfı . . . 151
11.23 Düşün ve Çöz . . . 153

12 Tip Dönüşümleri ve Lamda İfadeleri 154
12.1 Tip Dönüşümü Nedir? . . . 154
12.2 Üstü Kapalı Tip Dönüşümleri . . . 154
12.3 Bilinçli Tip Dönüşümü . . . 155
12.3.1 Derleme Zamanı Kasıtlı Tip Dönüşümü . . . 155
12.3.2 Dinamik Dönüşüm . . . 155
12.3.3 Sabit Dönüşüm . . . 156
12.3.4 Yeniden Yorumlama Dönüşümü . . . 157
12.3.5 Kasıtlı Tip Dönüşümü Örneği . . . 157
12.4 Tip Dönüşümünün Üstünlük ve Zayıflıkları . . . 158
12.5 Tip Çıkarımı . . . 158
12.6 Lamda İfadeleri . . . 159
12.6.1 Yakalama Listesi . . . 160
12.6.2 Parametre Listesi . . . 160
12.6.3 Gövde . . . 162
12.6.4 Lamda Örnekleri . . . 162
12.7 explicit Sıfatı . . . 164
12.8 Takma İsimler . . . 164
12.9 using Anahtar Kelimesinin Takma İsim Olarak Kullanılması . . . 165
12.10 Düşün ve Çöz . . . 166

13 Sınıflar Arası İlişkiler ve SOLID İlkeleri 167
13.1 Unified Modeling Language . . . 167
13.2 Sınıf Diyagramları . . . 168
13.3 Sınıflar Arası İlişkiler . . . 169
13.3.1 Genelleştirme İlişkisi . . . 169
13.3.2 Bağımlılık İlişkisi . . . 169
13.3.3 Sahiplik ve Ortaklık İlişkisi . . . 170
13.3.4 Biriktirme İlişkisi . . . 172
13.4 Sınıf Tasarlama İlkeleri . . . 173
13.4.1 Tek Sorumluluk İlkesi . . . 174
13.4.2 Açık-Kapalı İlkesi . . . 174
13.4.3 Liskov İkame İlkesi . . . 175
13.4.4 Ara Yüzleri Ayırma İlkesi . . . 175
13.4.5 Bağımlılıkları Ters Çevirme İlkesi . . . 175
13.5 Düşün ve Çöz . . . 176

14 İstisna Yönetimi 177
14.1 Yapısal Programlamada Hata Yönetimi . . . 177
14.2 İstisna Yönetimi . . . 177
14.3 Standart İstisna Tipleri . . . 178

14.4 std::exception ve what() Yöntemi . . . 181
14.5 noexcept Anahtar Kelimesi . . . 181
14.5.1 noexcept İşleci . . . 181
14.5.2 noexcept Belirleyicisi . . . 182
14.6 Kullanıcı Tanımlı İstisna Tipleri . . . 182
14.7 Yığın Çözülmesi . . . 183
14.8 Düşün ve Çöz . . . 184

15 Şablonlar **185**
15.1 Şablon Nedir? . . . 185
15.2 Foksiyon Şablonları . . . 185
15.3 Değişken Sayıda Argüman Alabilen Fonksiyon Şablonları . . . 186
15.4 Katlama İfadeleri . . . 187
15.5 Sınıf Şablonları . . . 188
15.6 Faydalı Şablonlar . . . 190
15.6.1 std::pair . . . 190
15.6.2 std::move . . . 191
15.6.3 std::swap . . . 191
15.6.4 std::forward . . . 191
15.6.5 Değişmez Değerler ve Diğerleri . . . 191
15.7 Akıllı Göstericiler . . . 191
15.7.1 Kaynak Edinimi İlk Değer Atamasıdır! Prensibi . . . 191
15.7.2 std::unique_ptr ile Tekil Sahiplik . . . 192
15.7.3 std::shared_ptr ile Paylaşılan Sahiplik . . . 193
15.7.4 std::weak_ptr ile Zayıf Takipçi . . . 193
15.7.5 Akıllı Gösterici Örnekleri . . . 194
15.8 Standart Şablon Kütüphanesi . . . 196
15.9 Konteyner Şablonları . . . 196
15.9.1 std::array . . . 197
15.9.2 std::vector . . . 199
15.9.3 std::map, std::multimap ve std::pair . . . 201
15.9.4 std::list ve std::forwardlist . . . 204
15.9.5 std::set ve std::multiset . . . 205
15.9.6 std::optional . . . 205
15.9.7 std::function ve std::bind . . . 206
15.9.8 std::tupple . . . 208
15.9.9 std::string . . . 208
15.10 Algoritma Şablonları . . . 213
15.11 Yineleme Şablonları . . . 214
15.12 Fonksiyon Nesneleri . . . 215
15.13 using Anahtar Kelimesinin Şablonlarda Kullanımı . . . 216
15.14 Düşün ve Çöz . . . 217

16 Akışlar **218**
16.1 Akış Nedir? . . . 218
16.2 Akış Hiyerarşisi ve Temel Sınıflar . . . 218
16.3 Standart Akış Nesneleri . . . 218
16.4 Akış İşleçleri . . . 218
16.5 Standart olmayan Akış Nesneleri . . . 218
16.6 Akış Biçimlendirme . . . 221
16.7 Metin Akışları . . . 221
16.8 Akış Durum Kontrolleri . . . 221
16.9 İkili Akışlar . . . 222
16.10 std::cerr ve std:clog Nesnelerini Dosyaya Yönlendirme . . . 225
16.11 Düşün ve Çöz . . . 225

17 Tasarım Desenleri **227**
17.1 Desen Nedir? . . . 227
17.2 Nesne İmalatı ile İlgili Desenler . . . 227
17.2.1 Soyut Fabrika . . . 228
17.2.2 Fabrika Yöntemi . . . 230

17.2.3 Tekil Nesne . . . 231
17.2.4 Kurucu . . . 232
17.2.5 Prototip . . . 234
17.3 Yapısal Desenler . . . 236
17.3.1 Önyüz . . . 236
17.3.2 Adaptör . . . 239
17.3.3 Bileşik . . . 241
17.3.4 Süsleyici . . . 245
17.3.5 Sinek Sıklet . . . 248
17.3.6 Vekil . . . 249
17.3.7 Köprü . . . 251
17.4 Davranışla İlgili Desenler . . . 253
17.4.1 Sorumluluk Zinciri . . . 254
17.4.2 Komut . . . 255
17.4.3 Yineleyici . . . 259
17.4.4 Gözlemci . . . 261
17.4.5 Strateji . . . 263
17.4.6 Şablon Yöntem . . . 266
17.4.7 Ziyaretçi . . . 268
17.4.8 Hatıra . . . 270
17.4.9 Ara Bulucu . . . 273
17.4.10 Durum . . . 277
17.4.11 Yorumlayıcı . . . 281
17.5 MVC Mimari Deseni . . . 284
17.6 Düşün ve Çöz: Tasarım Desenleri Antrenmanı . . . 286
17.6.1 Yaratımsal (Creational) Desenler . . . 286
17.6.2 Yapısal (Structural) Desenler . . . 287
17.6.3 Davranışsal (Behavioral) Desenler . . . 287

18 Değişken Argümanlı Fonksiyonlar **288**
18.1 Değişken Sayıda Parametre Tanımı . . . 288
18.2 Değişken Sayıda Argüman Alan Ana Fonksiyon . . . 289

19 Ön İşlemci Yönergeleri **292**
19.1 Ön İşlemci Yönergesi Nasıl Çalışır? . . . 292
19.2 Ön işlemci Makroları . . . 292
19.3 Parametreli Ön İşlemci Makroları . . . 293
19.4 Ön Tanımlı Ön İşlemci Makroları . . . 293
19.5 Başlık Koruması . . . 294
19.6 Derleme Sürecinin Detayı . . . 295
19.7 Düşün ve Çöz . . . 295

Dizin **297**

Şekiller Listesi

1.1 Motorola 6802, Intel 8086 İşlemciler ile 2732 EPROM Belleği . . . 2
1.2 Frekansı 1 olan Kare Dalga İşareti . . . 3
1.3 Basit Bir İşlemcinin Yapısı . . . 3
1.4 İşlemcinin Emri Alma, Çözme ve İcra Çevrimi . . . 4

6.1 Akış Diyagramları Genel Şekilleri . . . 42
6.2 Dik Üçgenin Alana Hesabını Gösteren Akış Diyagramı . . . 43
6.3 If Talimatı İcra Akışı . . . 43
6.4 Bir Sayının İşaretini Bulan Akış Diyagramı . . . 45
6.5 If-Else Talimatı İcra Akışı . . . 45
6.6 Bir Sayının İşaretini Bulan If-Else Akış Diyagramı . . . 47
6.7 Switch Talimatı İcra Akışı . . . 48
6.8 Sayaç Kontrollü Döngü İcra Akışı . . . 51
6.9 While Talimatı İcra Akışı . . . 52
6.10 Do-While Talimatı İcra Akışı . . . 53
6.11 For Talimatı İcra Akışı . . . 54
6.12 While ve Do-While Döngülerinde Break ve Continue Talimatları İcra Akışı . . . 58
6.13 For Döngüsünde Break ve Continue Talimatları İcra Akışı . . . 59

7.1 Fonksiyon Tanımı Örneği . . . 65
7.2 Fonksiyon Bildirimi Örneği . . . 66
7.3 Fonksiyon Çağırma Sürecinde Yığın Bellek . . . 74

8.1 Tek Boyutlu Dizi Tanımlama . . . 79
8.2 İki Boyutlu Dizi Tanımlama . . . 83

11.1 Okul Projesi Sınıfları UML Diyagramı . . . 129
11.2 Kalıtım Uygulanmış Okul Projesi Sınıfları UML Diyagramı . . . 131

13.1 Örnek Bir Sınıf Diyagramı . . . 168
13.2 Sınıf Diyagramlarında Arayüz Gerçekleştirme . . . 168
13.3 Sınıf Diyagramlarında Sınırlama ve Çokluk . . . 168
13.4 Genelleştirme İlişkisi Örneği . . . 169
13.5 Bağımlılık İlişkisi Örneği . . . 169
13.6 Sahiplik ve Ortaklık İlişkisi Örneği . . . 171
13.7 Biriktirme İlişkisi Örneği . . . 172

17.1 Soyut Fabrika Deseni UML Diyagramı . . . 228
17.2 Fabrika Yöntemi Deseni UML Diyagramı . . . 230
17.3 Tekil Deseni UML Diyagramı . . . 231
17.4 Kurucu Deseni UML Diyagramı . . . 233
17.5 Prototip Deseni UML Diyagramı . . . 235
17.6 Önyüz Deseni UML Diyagramı . . . 236
17.7 Adaptör Deseni UML Diyagramı . . . 239
17.8 Bileşik Deseni UML Diyagramı . . . 241
17.9 Süsleyici Deseni UML Diyagramı . . . 245
17.10 Sinek Sıklet Deseni UML Diyagramı . . . 248
17.11 Vekil Deseni UML Diyagramı . . . 250
17.12 Köprü Deseni UML Diyagramı . . . 252
17.13 Sorumluluk Zinciri Deseni UML Diyagramı . . . 254
17.14 Komut Deseni UML Diyagramı . . . 255
17.15 Yineleyici Deseni UML Diyagramı . . . 259
17.16 Gözlemci Deseni UML Diyagramı . . . 261
17.17 Strateji Deseni UML Diyagramı . . . 263
17.18 Şablon Yöntem Deseni UML Diyagramı . . . 266

17.19 Ziyaretçi Deseni UML Diyagramı 268
17.20 Hatıra Deseni UML Diyagramı 271
17.21 Ara Bulucu Deseni UML Diyagramı 273
17.22 Durum Deseni UML Diyagramı 277
17.23 Yorumlayıcı Deseni UML Diyagramı 281

19.1 Derleme Süreci 295

Tablolar Listesi

1.1 Bazı İşlemcilerin Adres ve Veri Yolu Genişlikleri . . . 2
1.2 Onaltılı Sayı Rakamları ve İkili ve Onlu Karşılıkları . . . 5
1.3 6800 Emir Seti Örneği . . . 6

2.1 Tam Sayılar ve Sayı Sınırları . . . 11
2.2 Kayan Noktalı Sayılar ve Sınırları . . . 12
2.3 İki İle Çarpma Fonksiyonu . . . 12

4.1 Anahtar Kelimeler . . . 22
4.2 Tüm Veri Tipleri İçin Değişmez Örnekleri . . . 27
4.3 C++ const ve constexpr Karşılaştırması . . . 29
4.4 Aritmetik İşleçler . . . 31
4.5 Tekli İşleçler . . . 32
4.6 İlişkisel İşleçler . . . 32
4.7 VE, VEYA, ÖZEL VEYA ve DEĞİL İşlemlerinin Doğruluk Tabloları . . . 33
4.8 Bit Düzeyi İşleçler . . . 33
4.9 Mantıksal İfadelerin Doğruluk Tablosu . . . 34
4.10 Mantıksal İşleçler . . . 34
4.11 Atama İşleçleri . . . 34
4.12 İşleç Öncelikleri . . . 35

5.1 En Çok Kullanılan Başlık Dosyaları . . . 37
5.2 iostream Biçimlendirme Bayrakları . . . 40

7.1 Sık Kullanılan Standart C++ Fonksiyonlarından Bazıları . . . 61
7.2 <cmath> Kütüphanesi Temel Fonksiyonları . . . 62
7.3 Stdlib.h Başlık Dosyasının Rastgele Sayı Fonksiyonları . . . 63
7.4 Depolama Sınıfları . . . 72

11.1 Kalıtımda Sınıf Üyelerine Erişim . . . 132

15.1 Katlama Türleri . . . 187
15.2 std::string ile Karakter Dizilerinin Karşılaştırması . . . 209
15.3 Metin İşleme Yöntemleri . . . 212
15.4 Yineleme Fonksiyonları . . . 215

16.1 Standart Akış Nesneleri . . . 218
16.2 Dosya Açma Modları . . . 219

18.1 Stdarg.h Fonksiyonları . . . 288

19.1 Ön İşlemci Yönergeleri . . . 292
19.2 Standart Ön İşlemci Makroları . . . 293

1. Bölüm

Temel Bilgisayar Kavramları

1.1 Mühendis ve Teknisyen

Mühendis (engineer), ihtiyaç duyulan ürünü; ekonomik (cost), güvenli (safe) ve kullanışlı (functional) olarak zamanında (on time) ortaya koyan kişilerdir. Bunu yapmak için; keşif (invent), tasarım (design), analiz (analysis), uygulama (build) ve sınama (test) yaparlar [1].

Teknisyenler (technician) ise genellikle laboratuvarda teknik ekipmanlarla ilgilenen veya bu ekipmanlar ile pratik çalışmalar yapan kişilerdir. Genel olarak amacı ekipmanın bakım ve kullanımına ilişkindir.

Konumuz yazılım olduğundan, yazılım mühendisleri istenilen bilgisayar yazılımını ekonomik, güvenli ve kullanışlı olarak zamanında ortaya koyan kişilerdir.

Programcılar ise teknisyenlere karşılık gelir. Yani geliştirme aşamasında bazı yazılım kısımlarının kodlanmasını veya geliştirilen yazılımın çalışır tutulmasını sağlarlar.

1.2 Bilgisayara Niçin İhtiyaç Duyulur?

Bir çiftçi niçin traktöre ihtiyaç duyar? Bir berber niçin elektrikli tıraş makinesi kullanır? Benzer şekilde bir mühendis niçin bilgisayara ihtiyaç duyar? İşte bütün bu soruların cevabı yapılacak işleri daha kısa sürede yapmaktır. Yani bütün araç gereç ve makinalara olan ihtiyacımız işlerimizi **daha hızlı** (speed) yapabilmek içindir. Bütün makineler gibi bilgisayar da veriden hızlıca bilgi elde etmemizi ve tekrarlanan hesaplamaları hatasız yapmamızı hızlıca sağlayan makinelerdir.

Veri sorumlusu, veri hazırlama kontrol işletmeni gibi unvanlar bu sebeple ortaya çıkmıştır. Bu unvanlara ihtiyaç duyulmasının sebebi verinin belli bir şekle uygun olarak bilgisayara girilmesi ve işlenen verinin saklanması ve raporlanmasının bilgisayar dünyasında önemli bir yer tutmasıdır. Bilgisayarlar ilk zamanlarda daha çok cebirsel ifadeleri hatasız ve hızlı bir şekilde yapmak için kullanılmışlardır. Bunu daha sonradan ALGOL (ALGOritmic Language) adını alacak IAL (International Algebraic Language) ya da FORTRAN (FORmula TRANslation) dilinden de anlayabiliriz.

1.3 Veri ve Bilgi

Veriler (data), genellikle bir ölçüm ya da sayım sonucu elde edilen ve hakkında az şey bilinen varlıklardır. Örneğin ölçülen hava sıcaklığı, derse giren öğrenci sayısı, tartılan sebzenin ağırlığı gibi şeyler.

Bilgi (information) ise verinin işlenerek otaya çıkan anlamlı verilerdir. Örneğin ortalama hava sıcaklığı, ölçülen yerde bir ölçüm yapmaya gerek kalmadan sıcaklığı yaklaşık olarak bilmeye yarayan bir bilgidir. Bir sınıftaki not ortalaması, o sınıftan rastgele seçilen bir öğrencinin notunu yaklaşık belirleyen bir bilgidir. Kısacası bilgi, işlenmiş veridir

1.4 En Basit Bilgisayar

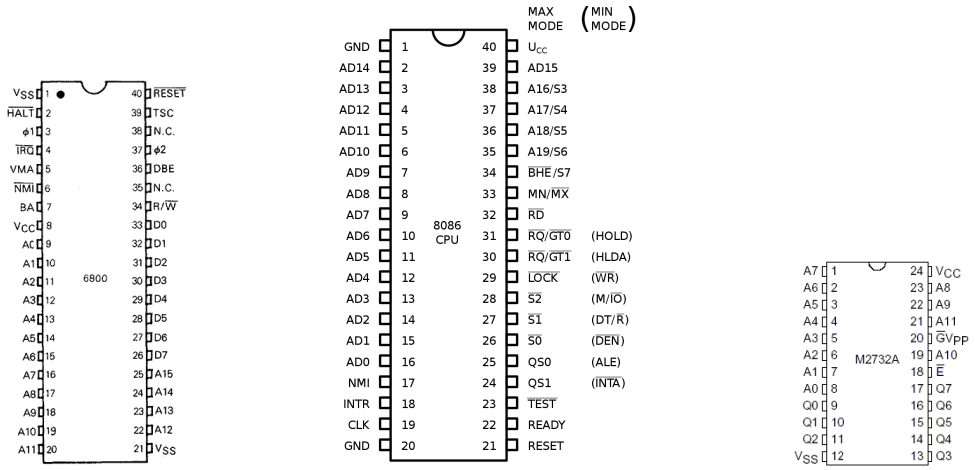
En basit bilgisayar üç bileşenden oluşur;

1. Merkezi İşlem Birimi CPU (Central Processing Unit): Kısaca **işlemci** (processor) olarak adlandırılır.
2. Bellek (memory): İşlemcinin işleyeceği **emirleri** (instruction) ve verileri barındırır.
3. Yol (bus): İşlemci ile belleği birbirine bağlayan elektrik yollardır.

Hem belleğin hem de işlemcinin adres (address) veri (data) bacakları bulunur. Bu bacakları karşılıklı olarak yollar (bus) birbirine bağlar. Bunların yanında hem işlemcinin hem de belleğin okuma (read), yazma (write) bacakları bulunur. Ayrıca işlemciye özel kesme (interrupt) veya kristal bağlanan bacakları gibi kontrol (control) bacakları da bulunur. İşlemci ve bellek, entegre devre (integrated circuit) kısaca **yonga** şeklinde üretilirler.

Şekil 1.1'de görüldüğü üzere 6800 işlemcisinin 15 adet adres bacağı, 8 adet veri bacağı bulunmaktadır. 8086 işlemcisinin ise 20 adet adres bacağı bulunakta olup bu bacakların 16 adedi aynı zamanda veri bacağı olarak kullanılmaktadır. İşlemciye karşılık olarak kullanılacak 2732 belleğin ise 12 adet adres bacağı, Q ile gösterilen 8 adet veri bacağı bulunaktadır.

[1] www.bls.gov/ooh/architecture-and-engineering



(a) Motorola 6800 (b) Intel 8086 (c) 2732 EPROM

Şekil 1.1: Motorola 6802, Intel 8086 İşlemciler ile 2732 EPROM Belleği

Görüldüğü üzere işlemci ile bellek arasında bağlantı sağlayan elektrik **yolları** (bus) adres veya veri taşır. Bu yolların veri taşıyanlarına **veri yolu** (data bus), adres taşıyanına ise **adres yolu** (address bus) adı verilir. Veri yolu; hem işlemci tarafından belleğe veri yazmak veya bellekten işlemciye taşınacak veriler için kullanıldığından çift yönlüdür. Adres yolu ise belleğin hangi bölgesindeki veriye ulaşılacağını belirleyen hatlar olup tek yönlüdür. 8086 işlemcisinde AD ile işaretli veri ve adres için kullanılan ortak yolların ne zaman adres ne zaman veri göstereceği S0 bacağından belli olur.

Bilgisayarlar elektrikle çalışan aletlerdir. İşlemci ile bellek arasında bir hatta elektrik varsa `1` yoksa `0` olarak anlamlandırılır. Bunlar **ikili** (binary) sayı sisteminin rakamlarıdır ve kısaca bit (BInary DigiT) olarak adlandırılır. Dolayısıyla; bir elektrik hattının taşıyacağı veri, bir haneli ikili sayı ile, iki elektrik hattının taşıyacağı veri, iki haneli ikili sayı ile ... ifade edilebilir.

İşlemcinin veri yolunun genişliği **kelime uzunluğudur** (word length). ve işlemcinin aynı anda işlem yaptığı bit sayısını da verir. Sekiz (8) bitlik veriye **bayt** (byte) adı verilir ve bellek miktarı genellikle bayt olarak ölçülür.

İşlemcinin veri yolu, 8 bit ise BYTE işlemci, 16 bit ise WORD işlemci, 32 bit ise DWORD (Double WORD) işlemci, 64 bit ise QWORD (Quad WORD) işlemci olarak adlandırılır.

İşlemcinin adreslenebilir bellek miktarı ise işlemcinin adres hatlarının sayısı yani adres yolu ile belirlenir. **Adres yoluna konulan her bir adres ile 1 bellek baytı adreslenir**. Adres yolu ne kadar geniş ise adreslenebilir bellek miktarı da o kadar artar. Örneğin 64 bit adres bacağına sahip işlemci 2^{64}=4.294.967.296 kadar bellek baytını adresler.

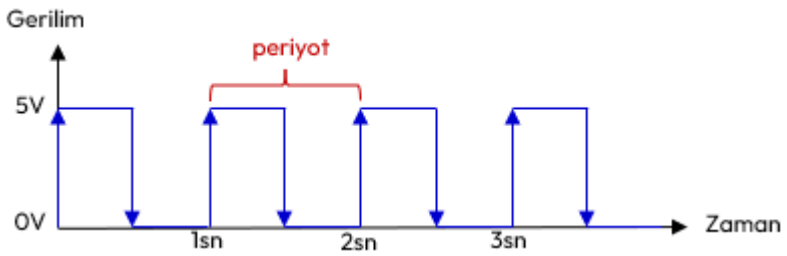
İşlemci	**Veri Yolu**	**Adres Yolu**	**AdreslenebilirBellek**	**Kilo bayt**	**Mega bayt**	**Giga bayt**
Motorola 6802	8 Bit	16 Bit	2^{16}=65.536	64		
Intel 8086	16 Bit	20 Bit	2^{20}=1.048.576	1024	1	
Intel 80286	16 Bit	24 Bit	2^{24}=16.777.216		16	
Intel Pentium	32 Bit	32 Bit	2^{32}=4.294.967.296		4096	4
Intel i7	64 Bit	36 Bit	2^{36}=68.719.476.736			64

Tablo 1.1: Bazı İşlemcilerin Adres ve Veri Yolu Genişlikleri

Bilgisayarlarda kablodaki gerilimin değeri genelde 5V, 3.3V, 2.8V, 2V, 1.5V veya 1V gibi değerler alır. **Programcılar için gerilimin ne olduğu değil varlığı önemlidir**. Gerilim değerin yüksek olması fazla akım çekme ve ısınma anlamına gelir. Bu nedenle günümüzde daha düşük gerilimle çalışan işlemciler tasarlanmaktadır.

İşlemciler kare dalga olarak oluşturulmuş elektriksel bir işareti saat (clock) olarak kullanır. Şekil 1.2'de gösterilen bu elektrik işareti işlemciye ϕ ya da CLK bacaklarından uygulanır. Saniyede 1 kez bir dalga üreten işaretin frekansı 1/1sn = 1 Hertz= 1 Hz. olarak hesaplanır. Bir mikro saniyede bir kez dalga üreten işaretin frekansı; $1/(1\mu s) = 1/(10^{-6}s) = 10^6$ Hertz = 1.000.000 Hertz = 1 MHz olarak hesaplanır.

İşlemciye uygulanan saatin frekansı ne kadar yüksek olursa işlemci o kadar hızlanır. Ancak işlemcilerin de tasarımından ötürü karşılayabileceği belli bir frekans değeri bulunmaktadır. Günümüzde 5 GHz frekansını karşılayan işlemciler bulunmaktadır.

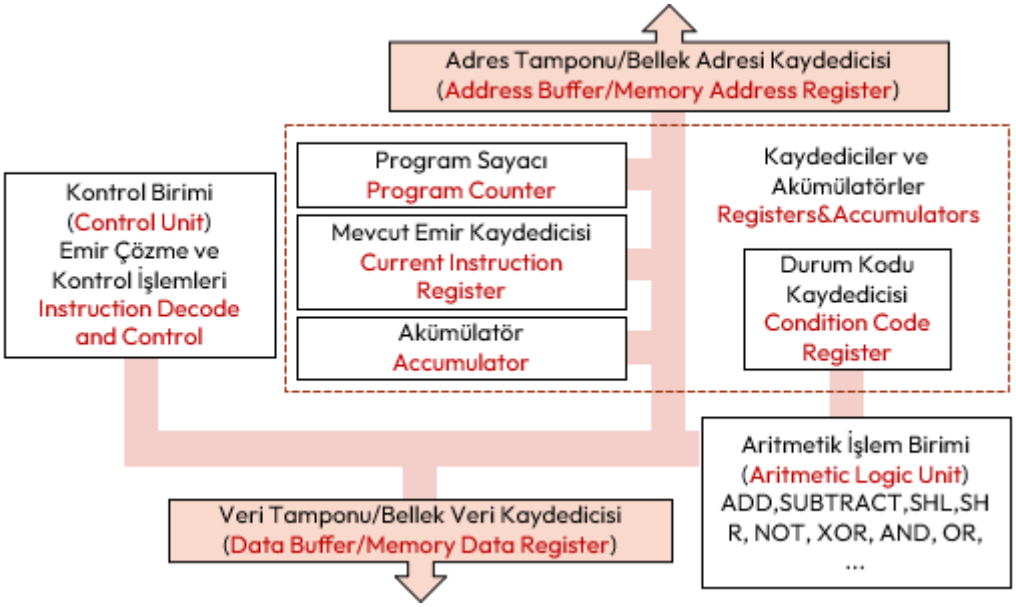


Şekil 1.2: Frekansı 1 olan Kare Dalga İşareti

1.5 Emri Alma, Çözme ve İcra Çevrimi

Basit bir işlemcideki bileşenleri aşağıdaki şekilde açıklayabiliriz;

- **Kaydediciler** (register), işlemci içerisinde **çok hızlı e geçici adres veya veri saklayan** alanlardır.
- **Akümülatörler** (accumulator) ise bellekten alınan verileri saklamak veya işlenen sonuçları saklamak için en sık kullanılan kaydedicilerdir.
- **Program sayacı** PC (program counter), işlemcinin icra edeceği **emri** (instruction) takip için kullanılır. Getirilecek bir sonraki emrin bellek adresini içerir.
- **Mevcut Emir kaydedicisi** CIR (current instruction register) işlemcinin o an icra ettiği emirdir.
- **Koşul kodu kaydedicisi** (condition code register), herhangi bir işlemin durumunu gösteren farklı bayraklar (flag) içeren kaydedicidir.
- **Aritmetik İşlem Birimi** ALU (aritmetic logic unit), işlemcinin toplama çıkarma ve karşılaştırma yapan birimidir.
- **Kontrol Birimi** CU (control unit) Orkestra şefi gibidir, emirleri çözer ve yorumlar ve gerekli bileşenlere sinyal gönderir.
- **Veri Yolları** (bus,), verinin bileşenler arasında taşındığı birden çok elektrik hattıdır.



Şekil 1.3: Basit Bir İşlemcinin Yapısı

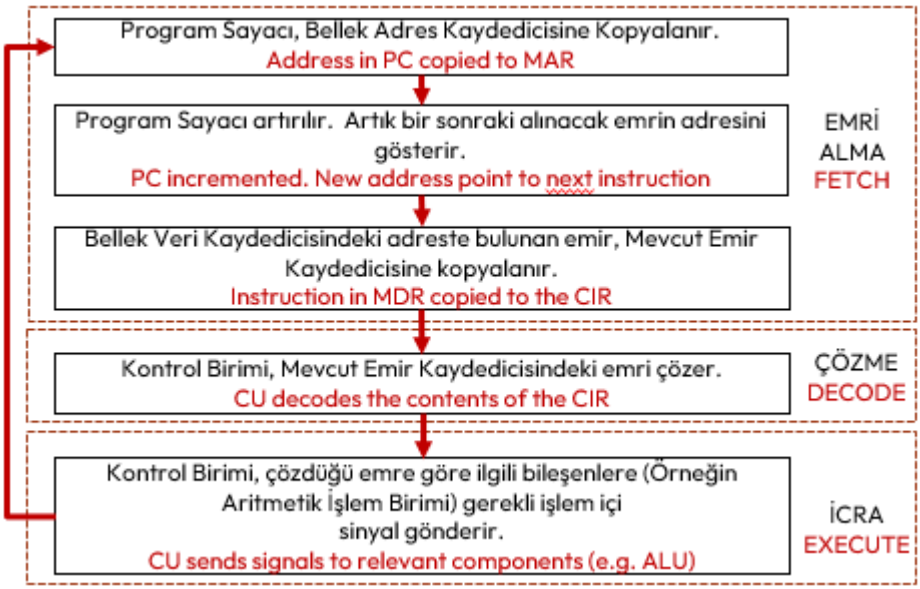
Bu işlemci tasarımı aynı zamanda **depolanmış program** (stored program) kavramı olarak adlandırılır ve **Von Neumann Mimarisi** olarak da bilinir. *John Von Neumann* tarafından 1946 ila 1949 yılları arasında tasarlanan ve Şekil 1.3'de verilen bu mimari günümüzde modern bilgisayarlarda hala kullanılmaktadır.

Basit bir bilgisayara elektrik verildiğinde, işlemci aşağıdaki üç adımlık **emir alma** (fetch), **çözme** (decode) ve **icra** (execute) çevrimini elektrik kesilene kadar sürekli tekrarlanır [2]. Bu çevrime **emir çevrimi** (instruction cycle) adı da verilir.

1. **Emri Alma** (fetch): Bu aşamada program sayacının (program counter) gösterdiği adresteki emir (instruction) bellekten okunur:
   (a) Program sayacı , bir sonraki emrin adresini tutar.
   (b) Bu adres **bellek adres kaydedicisine** (memory address registermemory address register) kop-

[2] https://www.youtube.com/watch?v=ByllwN8q2ss

yalanır.
   (c) Talimat bellekten çağrılır ve **bellek veri kaydedicisine** (memory data register), oradan da **mevcut emir kaydedicisine** (current instruction register) aktarılır.
   (d) Program Sayacı, bir sonraki emri işaret etmek için bir artırılır.
2. **Çözme** (decode): Kontrol birimi (control unit) tarafından emrin ne anlama geldiği ve buna bağlı nelerin yapılacağını (örneğin ulaşılacak bellek adresi değiştirilir) belirler:
   (a) Mevcut emir kodu (current instruction code), ne yapılacağını belirten kısım yani işlem kodu (opcode) ve hangi veriyle yapılacağını belirten işlenen (operand) kısımdan oluşur.
   (b) Kontrol birimi, varsa emrin işaret ettiği veriyle işlemin gerçekleşmesi için gerekli donanım parçalarını hazırlar.
3. **İcra** (execute): Çözülen emir koduna bağlı olarak işlemcinin bazı bileşenleri çalıştırılır:
   (a) Eğer bir matematiksel işlem veya mantıksal karşılaştırma gerekiyorsa, bu görev aritmetik işlem birimini ALU (Aritmetic Logic Unit) tarafından yapılır.İşlem sonuçları ortaya çıkan değerler ise akümülatörlere (accumulator) veya bellek veri kaydedicisi (memory data register) üzerinden belleğe yazılır.
   (b) Bazı durumlarda ise kaydedici (register) içerikleri değiştirilir.



Şekil 1.4: İşlemcinin Emri Alma, Çözme ve İcra Çevrimi

Burada çözülecek emirlerin (instruction) her biri bir ikili sayıya karşılık gelir. Bu sayılara **emir kodu** (instruction code), **makine kodu** (machine code) veya **işlem kodu** (opcode) olarak da bilinirler. İşlemciler tüm sayıları emir kodu olarak anlamaz. Yani sınırlı sayıda sayı emri tanırlar. Tablo 1.1'de verilen Motorola 6802 işlemcisi 72 farklı emri, Intel 8086 işlemcisi ise 116 farklı emri tanır. Çözülebilecek tüm emirlerin oluşturduğu kümeye **emir seti** (instruction set) adı verilir.

1.6 İşlemci Tasarlama Stratejileri

İşlemciler, aşağıda verilen üç strateji ile tasarlanırlar;

1. **İndirgenmiş Emir Setli İşlemciler** olan RISC (Reduced Instruction Set Computing) İşlemci: Az sayıda emir alan yüksek hızlı işlemci tasarımı. RISC işlemciler az sayıda emre sahip olduğundan emrin çözülmesi ve icrası da kısa zaman almaktadır. Bu da çalıştırılacak kodu büyütür.
2. **Karmaşık Emir Setli İşlemciler** olan CISC (Complex Instruction Set Computing) İşlemci: Çok fazla emri anlayan karmaşık nispeten yavaş işlemci tasarımı. CISC işlemciler, çok sayıda emir tanıyabildiğinden emrin çözülmesi ve icra edilmesi nispeten daha uzun sürer. Ancak yüksek düzey dillerin daha kolay makine koduna çevrilmesini kolaylaştırır.
3. **Özel İşlemci**: Bunlar grafik kartları ve yapay zekâ gibi belli amaçlar için tasarlanmış işlemcilerdir.

Genel olarak işlemcilerde emir çevrimi (instruction cycle) 1 saat periyodunda tamamlanmaz. RISC işlemciler için bu çevrim, birçok emir için 1 saat periyodunda tamamlanır. CISC işlemciler için ise birden fazla saat periyodu gerekir. Çünkü CISC işlemcilerin emir seti daha geniş olduğundan emrin çözülmesi daha fazla vakit alır. Bu nedenle RISC işlemciler CISC işlemcilere göreceli olarak daha hızlıdır.

Günümüzde ticari olarak satılan işlemcilerin çoğu ortak emir seti kullanmaya başlamışlardır;

- ◇ Masaüstü bilgisayarların birçoğu Intel ve AMD Marka işlemciler kullanır ve bunlar IA-16, IA-32, x86-16, x86-64, AMD64, ... emir setleri ve eklentilerini desteklemektedir.
- ◇ Bilinen birçok telefon işlemcisi Acorn Risc Machine-ARM tabanlı işlemciler kullanır. Bu işlemcilerde ARMv7, ARMv8, ... emir setleri ve eklentileri kullanılmaktadır.

1.7 Onaltılı Sayı Sisteminin Kullanılması

İşlemci ile bellek arasında verilerin elektrik yolları (bus) ile taşındığı anlatılmıştı. Veri taşıyan elektrik hattı sayısı işlemci ile bellek arasında taşınabilecek verinin sınırlarını da belirler. Bir elektrik hattının taşıyacağı veri bir bit ile (`0` veya `1`) gösterilebilir. İki elektrik hattının taşıyacağı veri ise iki bit ile (`00`, `01`, `10` ve `11`) olmak üzere 2^2 kadar farklı gösterilebilir. Benzer şekilde n adet elektrik hattının taşıyacağı veri 2^n farklı alternatifte veri olacaktır.

Bit sayısı arttıkça ikili sayının okunup yazılması da çok zor olacaktır. Günümüzde 64 bitlik hatta 1024 bitlik veri yoluna sahip bilgisayarlar bulunmaktadır. Bunun üstesinden gelmek için (hexadecimal) sayı sitemi kullanılır. Çünkü her 4 elektrik hattının verisi onaltılık sayı sisteminin bir rakamına karşılık gelir.

Onaltılı	İkili	Onlu	Onaltılı	İkili	Onlu
`0`	`0000`	1	`8`	`1000`	8
`1`	`0001`	2	`9`	`1001`	9
`2`	`0010`	3	`A`	`1010`	10
`3`	`0011`	4	`B`	`1011`	11
`4`	`0100`	5	`C`	`1100`	12
`5`	`0101`	6	`D`	`1101`	13
`6`	`0110`	7	`E`	`1110`	14
`7`	`0111`	8	`F`	`1111`	15

Tablo 1.2: Onaltılı Sayı Rakamları ve İkili ve Onlu Karşılıkları

> Kod içerisinde onaltılık sayıların rakamlarının diğer sayı sistemindeki rakamlarla karışmaması için sonuna `h` konularak `113Ah` şeklinde veya çoğunlukla başına `0x` konularak `0x1133` şeklinde kullanılır.

Örnek olarak ikili tabanda verilen 64 bitlik `0001 1010 1011 1101 0101 0100 1001 1010 0011 0010 0100 0110 1001 1101 1100 1111` sayısının onaltılı tabanda karşılığı oldukça kısa (16 hane) olarak `1ABD 549A 3246 9DCF` şeklindedir.

> Onaltılı sayının `A`,`B`,`C`,`D`,`E` ve `F` rakamları büyük harf olacağı gibi küçük harf de olabilir. Anlam olarak bir şey değişmez.

Bilgisayar dünyasında ikili sayıları onaltılı sayı sistemiyle ifade etmek çoğunlukla zorunluluktur. Zamanla standart hale gelmiştir.

1.8 Program ve Programlama

Belli bir işlemi gerçekleştirmek üzere, işlemciye verilen bir dizi emre (instruction), **bilgisayar programı** (computer program) denir . Bu bir dizi emrin, işlemcinin çalıştıracağı şekilde hazırlanması işlemine **programlama** (programming) denir. Programlama **emir kodları** (instruction code) ile yapıldığından bu programın dili, **makine dili** (machine language) olarak adlandırılır[3].

Her işlemci üreticisi kullandığı emir seti farklı olduğundan yani her bir emre farklı makine kodu verdiğinden programlama oldukça zorlu ve teknik bilgi gerektirmekteydi. Burada daha önce yazılmış bir programın ne yaptığını anlamak için her defasında bu listelere defalarca bakmak gerekiyordu. Programın okunurluğunu artırmak için her bir emre, hatırlatıcı **sembolik isimler** (mnemonic) verilmeye başlandı. Tablo1.2'de 6802 işlemcisinin birkaç sembolik ismi verilmiştir.

Bu sembolik isimlerle yazılan program, **montaj kodu** (assembly code) olarak adlandırılır. Montaj kodları bir metin dosyasına yazılarak bir dosyaya saklanır. Bu dosyalar montaj dosyaları olarak adlandırılır. Montaj dosyalarını makine diline çeviren programlar yazılmıştır. Bu programlar ilk **derleyicilerdir** (compiler). Bu derleyiciler, sembolik isimlerle yazılmış program kodlarını makine koduna çeviren programlardır.

Aşağıda x86 emir seti kullanan Linux işletim sisteminde örnek montaj kodu örneği verilmiştir; Örnekte, genel amaçlı `CL` kaydedicisine koyulan sayıya kadar 2 ve 3'e bölünebilen sayıların toplamını bulup yine genel

[3] ISO/IEC 2382:2015. ISO. 2020-09-03. [Software includes] all or part of the programs, procedures, rules, and associated documentation of an information processing system

Sembolik İsim (Mnemonic)	Emir Kodu (Instruction Code)	Açıklama
ABA	1B	ADD A to B → A
INCA	4C	INCREMENT A
INCB	5C	INCREMENT B
SBA	10	SUBTRACT B from A→ A
LDA	96	LOAD M to A, M: Memory
LDB	D6	LOAD M to B, M: Memory

Tablo 1.3: 6800 Emir Seti Örneği

amaçlı DX kaydedicisine konulmaktadır[4].

```
section  .text          ; programı oluşturan emir kodları buradan başlar
         global   _start
_start:
         mov   dx, 0    ; dx 0 olsun. Toplamı tutatcak.
         mov   cl, 99   ; 99 kez tekrarlanacak.
         sum:
         mov   ax, cx   ; cx, ax e kopyalanır. Bölme işlemi için.
         mov   bl, 3    ; bl 3 olsun.
         div   bl       ; ax/bl.
         cmp   ah, 0    ; Kalan olup olmadığı kontrol ediliyor.
         jne   cont     ; Kalan yoksa, 'cont' etiketine git.
         mov   bl, 2    ; Sonra bl 2 olsun.
         div   bl       ; ax/bl
         cmp   ah, 0    ; Kalan olup olmadığı kontrol ediliyor.
         jne   cont     ; Kalan yoksa, 'cont' etiketine git.
         add   dx, c    ; Değilse, cx deki sayı 2 ve 3'>e bölünür. Toplama ekle.
cont:
         loop  sum      ; Bir sonraki sayı için 'sum' etiketine dön.
exit:
         mov   eax, 1   ; Artık programdan çıkıyoruz.
         mov   ebx, 0   ; Return 0.
         int   80h      ; linux çekirdeğini çağır. (soft interrupt 80h!).
```

1.9 Genel Amaçlı Bilgisayarlar ve İşletim Sistemleri

Genel amaçlı yani herkesin kendi amacına göre kullanacağı bilgisayarlar, veri girişini sağlayan klavye, verileri görüntüleneceği monitör, işlenen verilerin elektrik kesildiğinde saklanacağı disk ve benzeri donanımlara sahip olmalıdır.

Genel amaçlı bilgisayarın kullanıcı isteklerine cevap verebilmesi için;

1. Çeşitli programlara sahiptirler. İşte bu programlar bütününe işletim sistemi (operating system) adı verilir. DOS, Windows, Unix, Linux, MacOs, ChromeOS, OS/2 bunların en çok bilinenleridir.
2. Elektrik verildiğinde klavye, monitör ve disk olup olmadığı kontrol eden **BIOS** (Basic Input Output System) programı koşturulur. BIOS gerekli kontrollerden sonra bilgisayara işletim sistemini yükler. İşletim sistemi yüklendikten sonra, işletim sistemi kullanıcının vereceği komutları bekler.
3. Kullanıcı işletim sitemine vereceği komutlarla yeni bir program derleyebilir, derlenmiş bir programı çalıştırabilir.
4. Kullanıcının vereceği komutlar grafik olmayan işletim sistemlerinde (DOS, Unix, Linux gibi) komut yazılarak verilir. Komut yazılan bu ekrana konsol, terminal ya da komut satırı adı verilir. Sunucu işletim sistemlerinde hala tercih edilmektedir.
5. Grafik ara yüzlerine sahip işletim sistemlerinde (Windows ve MacOs İşletim Sistemleri ile KDE, Mate, GNome, Cinnamon, LXQt, XFce ve Deepin gibi grafik ara yüzleri içeren Linux İşletim Sistemleri) fare tıklamasıyla çoğu komut verilebilmektedir.

İşletim sistemlerinin yaygınlaşması ile kullanıcının bilgisayarı donanım bilmeden yapabilmesi sağlanmıştır.

Günümüzdeki işletim sistemlerinin birkaçı hariç tamamında temel programlama dili olarak C dili kullanılmaktadır.

[4]https://github.com/armut/x86-Linux-Assembly-Examples

1.10 Düşün ve Çöz

- **Düşün**: Bir mühendis ile teknisyen arasındaki temel farkı, bir yazılım projesindeki görev dağılımı üzerinden örneklendiriniz.
- **Düşün**: CPU içindeki kaydedicilerin (register) hız ve kapasite açısından bellekten farkı nedir? Neden tüm verilerimizi kaydedicilerde tutmuyoruz?
- **Düşün**: *Von Neumann* mimarisinde veri ve komutların aynı yolu (bus) kullanmasının **darboğaz** (bottleneck) yaratma riskini tartışınız.
- **Çöz**: 32-bit veri yoluna ve 24-bit adres yoluna sahip bir işlemcinin adresleyebileceği maksimum bellek miktarını MB cinsinden hesaplayınız.
- **Çöz**: 16-bit adres yoluna sahip bir mikrodenetleyicinin adresleyebileceği maksimum bellek miktarını KB cinsinden hesaplayınız.
- **Çöz**: Bir bilgisayarın 8 GB RAM'i tam olarak adresleyebilmesi için adres yolunun (address bus) en az kaç bit olması gerektiğini bulunuz.

2. Bölüm

Temel Yazılım Kavramları

2.1 Düşük Düzey Programlama

1642 Yılında *Blaise Pascal* tarafından icat edilen Pascalline Hesap Makinesi, eldeli toplama, ödünç almalı çıkarma yapıyordu.

1671 Yılında *Gottfried Wilhelm Leibniz* tarafından icat edilen Leibniz Çarkı, toplama, ödünç almalı çıkarmanın yanında bölme, çarpma ve karekök işlemlerini yapıyordu.

1801 Yılında *Joseph Maria Jacquard* dokuma tezgahlarında desenleri oluşturmak için **delikli kartlar** (punch card) kullanmayı icat etmiştir. Sonradan bilgisayara veri girdisi sağlamak ve sonuçların çıktısını göstermek için kullanılacaktır.

1830 Yılında *Charles Babbage* fark makinesi olan analitik makineyi icat etmiştir. Buhar gücü kullanan bu makinenin mantıksal işlem birimi, veri depolama birimi, giriş çıkış üniteleri bulunuyordu. *Augusta Ada Byron*, Babbage ile beraber çalışmıştır. Byron, analitik makinenin bir dizi matematiksel işlemi gerçekleştirebildiğini fark etti ve bu makineyi Bernoulli sayılarını üretmek için kullandı. Bu sayıların hesaplanması için yazdığı algoritma tarihteki ilk bilgisayar programı olarak kabul edilir.

Augusta Ada Byron, **ilk programcı** olarak kabul edilir.

1854 Yılında *George Boole* tarafından ikili sayı sistemini geliştirmiş bugün bilgisayarlarımızın kullandığı **bool cebiri** üzerine çalışmalar yapmıştır.

1890 Yılında *Herman Hollerith* tarafından delikli kartlarla bilgilerin yüklenebildiği ve bu bilgiler üzerinde toplama işlemlerinin yapılabildiği bir elektro mekanik araç geliştirdi. ABD'nin 1890 nüfus sayımında başarılı biçimde kullanıldı. Hollerith başarılı olunca bir şirket kurdu ve daha sonra üç firma birleşerek 1924 yılında adını **IBM** olarak değiştirdi.

1941 Yılında *Konrad Zuze* Z3 isimli elektrik motorları ile çalıştırılan mekanik bir bilgisayar yaptı. Bu (Z1, Z2, Z3 ve Z4 serisi) program kontrollü ilk bilgisayardır.

1944 Yılında *Howard Aitken*, IBM ile iş birliği yaparak **MARK-I**'i yaptı. Bilgiler, delikli kartlarla veriliyor ve sonuçlar yine delikli kartlarla alınıyordu. Saniyede 5 işlem yapabiliyordu. Boyutları,18 m uzunluğunda ve 2,5 m yüksekliğinde idi. İnsan müdahalesi olmadan sürekli olarak, hazırlanan programı yürüten ilk bilgisayar idi ve tam olarak elektronik bir bilgisayar değildi.

1943-1945 Yılları arasında *Konrad Zuze*, ilk yüksek düzey programlama dili olan Plankalkül'ü, Z1 bilgisayarı için geliştirmiştir.

Emir kodlarını (instruction code) ve sembolik isimleri (mnemonic) içermeyen programlama dilleri, **yüksek düzey programlama dili** (high level programming language) olarak adlandırılır. Yüksek düzey programlama dilleri çoğunlukla İngilizce sözcükleri kullanır.

1946 Yılında *John Von Neumann* önderliğinde Pensilvanya Üniversitesinde **ENIAC** (Elektronik Sayısal Hesaplayıcı ve Doğrulayıcı) isimli sayısal elektronik bilgisayar tamamlandı. Askeri amaçla üretildi ve top mermilerinin menzillerini hesaplamak için kullanıldı. Neumann tarafından geliştirilmiş bu mimari (Stored-program computer and Universal Turing machine § Stored-program computer), günümüzdeki bilgisayarlarda hala kullanılmaktadır. 18,000 adet elektronik tüp kullanılan ENIAC; 150 kW gücünde idi ve 50 ton ağırlığıyla 167 m^2 yer kaplıyordu. Saniyede 5.000 toplama işlemi yapabiliyordu. MARK-I bilgisayarından bin kat daha hızlıydı.

Yukarıda anlatılan bilgisayarlar, bir önceki bölümde anlatılan emir kodu (instruction code) veya sembolik isimlerle (mnemonic) yazılan programlar ile programlanmaktadır. Emir kodu ve sembolik isimlerle yapılan ve bilgisayar mimarisini bilmeyi gerektiren programlamaya **düşük düzey programlama** (low level programming) olarak adlandırılır. Daha çok elektronik ve bilgisayar mühendisleri ya da sistemi tasarlayan matematikçiler bu programlama türünü kullanır.

Düşük düzey programlamanın yapılabilmesi için hem işlemcinin hem de hafıza ve işlemciye bağlanan diğer bileşenlerin nasıl çalıştığı ve tasarlandığını çok iyi bilinmesi gerekir.

2.2 Genel Amaçlı Yüksek Düzey Diller

Sembolik isimler veya makine kodu yerine bildiğimiz konuşma diline yakın talimatlarla metin olarak yazılan programlama dilleri, **yüksek düzey programlama dilleri** (high level programming language)olarak adlandırılır. Yani emir kodu (instruction code) ve sembolik isim (mnemonic) kullanmayan programlama dilleri yüksek düzey programlama dilleridir.

Yüksek düzey dillerde **talimatlar** (statement) olarak yazılan programlar yine kendine has derleyiciler tarafından makine diline çevrilir. Her talimat, genellikle birden fazla emirden (instruction) oluşur.

Yüksek seviyeli bir dilde talimatlar, yapılması gerekeni kısa ve net olarak belirten **mantıksal satırlardır** (logical sequence).

1954 Yılında **FORTRAN** (FORmula TRANslation), *John Backus* liderliğinde IBM 704 için geliştirilirdi. FORTRAN, `READ`, `WRITE`, `IF` gibi 32 farklı talimata (statement) sahiptir. İlk genel amaçlı, yüksek düzey bir programlama dilidir. Aşağıda üç kenarı teyp ünitesinden alınan bir üçgenin alanını hesaplayan FORTRAN-II programı verilmiştir [1]. Görüldüğü üzere talimatlar kısa öz ve anlaşılırdır.

```
C AREA OF A TRIANGLE - HERON'S FORMULA
C INPUT - CARD READER UNIT 5, INTEGER INPUT
C OUTPUT -
C INTEGER VARIABLES START WITH I,J,K,L,M OR N
READ(5,501) IA,IB,IC
501 FORMAT(3I5)
IF (IA) 701, 777, 701
701 IF (IB) 702, 777, 702
702 IF (IC) 703, 777, 703
777 STOP 1
703 S = (IA + IB + IC) / 2.0
AREA = SQRT( S * (S - IA) * (S - IB) * (S - IC) )
WRITE(6,801) IA,IB,IC,AREA
801 FORMAT(4H A= ,I5,5H  B= ,I5,5H  C= ,I5,8H  AREA= ,F10.2,
$13H SQUARE UNITS)
STOP
END
```

1956 Yılında ticari amaçlarla kullanılabilen ve seri halde üretimi yapılan ilk bilgisayar **UNIVAC-I** oldu. UNIVAC, giriş-çıkış birimleri manyetik bant idi ve bir yazıcıya sahipti. UNIVAC 1951-1959 arasında üretilen bilgisayarlarda vakum tüpleri kullanıldı. Bu tüpler bir ampul büyüklüğünde, çok fazla enerji harcamakta ve çok fazla ısı yaymakta idiler. Veri ve programlar manyetik teyp ve tambur gibi bilgi saklama araçlarıyla saklandı. Veriler ve programlar bilgisayara delgi kartları ile yükleniyordu.

1959 yılı sonrasında üretilen bilgisayarlarda transistörler (yaklaşık 10 bin adet) kullanıldı. Bu yıl ve sonrasında transistör içeren **ikinci kuşak bilgisayarlar** hayatımıza girmiştir.

2.3 Arapsaçı Kod

1955-1959 Yılları arasında **FLOW-MATIC B0** (Business Language Version 0) Dili, *Grace Hopper* tarafından UNIVAC-I için geliştirildi. İngilizceye benzeyen ilk yüksek düzey programlama dilidir. Aşağıda B0 programlama dilinde örnek bir program verilmiştir [2].

```
(0)  INPUT INVENTORY FILE-A PRICE FILE-B ; OUTPUT PRICED-INV FILE-C UNPRICED-INV
FILE-D ; HSP D .
(1)  COMPARE PRODUCT-NO (A) WITH PRODUCT-NO (B) ; IF GREATER GO TO OPERATION 10 ;
IF EQUAL GO TO OPERATION 5 ; OTHERWISE GO TO OPERATION 2 .
(2)  TRANSFER A TO D .
(3)  WRITE-ITEM D .
(4)  JUMP TO OPERATION 8 .
```

[1] https://github.com/scivision/fortran-II-examples/blob/main/herons_formula.f
[2] https://gist.github.com/marcgg/f9f2da31a973ba319809

```
(5)  TRANSFER A TO C .
(6)  MOVE UNIT-PRICE (B) TO UNIT-PRICE (C) .
(7)  WRITE-ITEM C .
(8)  READ-ITEM A ; IF END OF DATA GO TO OPERATION 14 .
(9)  JUMP TO OPERATION 1 .
(10)  READ-ITEM B ; IF END OF DATA GO TO OPERATION 12 .
(11)  JUMP TO OPERATION 1 .
(12)  SET OPERATION 9 TO GO TO OPERATION 2 .
(13)  JUMP TO OPERATION 2 .
(14)  TEST PRODUCT-NO (B) AGAINST ZZZZZZZZZZZZ ; IF EQUAL GO TO OPERATION 16 ;
OTHERWISE GO TO OPERATION 15 .
(15)  REWIND B .
(16)  CLOSE-OUT FILES C ; D .
(17)  STOP . (END)
```

Görüldüğü üzere bu tarihlere kadar yazılan programlarda bir sürü `GO TO` ya da `JUMP TO` talimatı (statement) bulunmaktadır. Bu durumda programın ne yaptığı konusunda fikir yürütmeye çalıştığımızda **okunaklılık oldukça azdır**. İşte okunaklılığın az olduğu bu program kodlarına **arapsaçı kod** (spaghetti code) adı verilir. Günümüzde ne yaptığı kolay anlaşılamayan kodlar için bu kavram çokça kullanılmaktadır.

İşin doğası gereği **makine diline çevrilecek tüm kaynak kodlar gözle okunmalıdır!**.

1958 Yılında **LISP** programlama dili, *John McCarthy* önderliğinde *Steve Russell* tarafından geliştirilmiş ikinci yüksek düzey programlama dilidir;

```
(format t "Adinizi Giriniz: ")
(setq name (read))
(princ "Merhaba ")
(write name)
```

1959 Yılında **COBOL** (Common Business-Oriented Language) programlama dili, CODASYL komitesi tarafından İngilizceye benzer ikinci yüksek düzey bir dil olarak geliştirilmiştir;

```
IDENTIFICATION DIVISION.
PROGRAM-ID. GREET.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 USER-NAME PIC A(30).

PROCEDURE DIVISION.
DISPLAY "Enter your name: ".
ACCEPT USER-NAME.
DISPLAY "Hello, " USER-NAME "!".
STOP RUN.
```

1958 Yılında daha sonra **ALGOL** (ALGOritmic Language) adını alan **IAL** (International Algebraic Language) programlama dili geliştirilmiştir. Bu dil daha sonra ortaya çıkan birçok dile önderlik etmiştir. Bu dil ile birlikte kod blokları (code block), iç içe fonksiyon (nested function), değişken kapsamı (scope) ve dizi (array) kavramları programcının hayatına girmiştir. ALGOL dili, barındırdığı özellikler sebebiyle, kendisinden sonra ortaya çıkan bütün programlama dilleri için bir esin kaynağı olmuştur;

```
BEGIN
FILE F (KIND=REMOTE);
EBCDIC ARRAY E [0:11];
REPLACE E BY "HELLO WORLD!";
WRITE (F, *, E);
END.
```

1958 Yılındaki FORTRAN diline diline çıktı üretmeyen fonksiyonlar için `SUBROUTINE`, değer üreten fonksiyonlar için `FUNCTION`, `CALL` ve `RETURN` özellikleri eklenmiş ve **FORTRAN-II** adını almıştır.

Benzer şekilde 1958 ile 1969 yılları arasında ALGOL diline çıktı üretmeyen fonksiyonlar için `PROCEDURE` ve `BEGIN-END` arasında kod bloğu özellikleri eklenmiştir.

1966 Yılındaki FORTRAN-IV diline `INTEGER`, `REAL`, `DOUBLE PRECISION`, `COMPLEX` ve `LOGICAL` **veri tipleri** (data type) ile Mantıksal `IF`, aritmetik (üç yollu) `IF` ve `DO` döngüsü talimatları (statement) eklenmiştir.

1964 yılında Dartmouth College'da John G. Kemeny ve Thomas E. Kurtz tarafından geliştirilen **BASIC** (Beginner's All-purpose Symbolic Instruction Code) öğrenilmesi ve kullanılması kolay olacak şekilde tasarlanmış bir programlama dilidir. Bilimsel olmayan alanlardaki öğrencilerin ve yeni başlayanların bilgisayar kullanabilmesini sağlamak amacıyla İngilizce benzeri komutlarla tasarlanmıştır. Karmaşık donanım detaylarını gizleyerek programcının mantığa odaklanmasına izin verir. Kodlar genellikle satır satır gerçek zamanlı olarak çalıştırılır, bu da hataların anında bulunmasını kolaylaştırır. 1980'li yılların başında kadar ev bilgisayarlarının (Apple II, Commodore 64, IBM PC gibi) varsayılan dili haline gelmiştir;

```
10 REM A simple program to greet the user
20 CLS ' Clears the screen (command might vary by implementation)
30 PRINT "What is your name? ";
40 INPUT NAME$ ' The '$' indicates a string variable
50 PRINT "Hello, "; NAME$; "!"
60 GOTO 30 ' Jumps back to line 30 to run the program again
70 END ' Terminates the program
```

2.4 Değişken

Değişkenler (variable) aslında cebirde kullandığımız değişkenlerdir. Cebir (algebra) işlemlerinde doğrudan problemde verilen sayıları değil, onları temsil eden değişkenleri kullanırız.

$$3x^2 + 2x + 10$$
$$c^2 = a^2 + b^2$$
$$c = 2\pi r$$

Yukarıdaki cebirsel ifadelerin ilkinde bir polinom, ikincisinde ise bir dik üçgenin kenar uzunlukları arasındaki ilişki verilmiştir. Bu ifadelerde x, a, b ve r **bağımsız değişken**, c ise **bağımlı değişkendir**. Örneklerdeki 3,2,10,2 ve π ise **sabit** (constant) sayılardır.

Değişkenler, bilgisayarlarda bir miktar bellek bölgesini işgal eder ve bu bellek bölgesinde çeşitli biçimlerde tutulurlar.

Tam sayılar, **ikili** (binary) sayı olarak bellekte tutulur. Tam sayıyı ifade eden ikili sayının en anlamlı biti, işaret (sign) biti olarak adlandırılır ve bu bitin değeri `1` ise sayı negatif, `0` ise sayı pozitiftir. Tablo 2.1'de en çok kullanılan tam sayılara ilişkin bellek miktarı ve sayı sınırları verilmiştir.

Bit Sayısı	Bellek Miktarı	Sayı Sınırları
8 Bit	1 bayt	-127 ile +128 arasında veya 0 ile 255 arasında
16 bit	2 bayt	-32.767 ile +32.768 arasında veya 0 ile 65.535 arasında
32 bit	4 bayt	-2.147.483.648 (-2^{31}) ile +2.147.483.647 ($2^{31}-1$) arasında veya 0 ile 4.294.967.295 ($2^{32}-1$) arasında
64 bit	8 bayt	-9.223.372.036.854.775.808 (-2^{63}) ile 9.223.372.036.854.775.807 ($2^{63}-1$) arasında veya 0 ile 18.446.744.073.709.551.615 ($2^{64}-1$) arasında

Tablo 2.1: Tam Sayılar ve Sayı Sınırları

Gerçek sayılar, kayan noktalı sayı (floating point number) olarak biçimlendirilir. Kayan noktalı sayı kavramı ilk olarak 1914 yılında Leonardo *Torres Quevedo* tarafından Charles Babage'ın analitik makinesine kullanılmak üzere yayınlanmıştır.

$$12345 = 1234,5 \times 10^1 = 123,45 \times 10^2 = 12,345 \times 10^3 = 1,2345 \times 10^4 = 0,12345 \times 10^5$$

Yukarıda ondalık tabanda verilen gerçek bir sayıya ilişkin kesir noktasının kaydırılması ile aynı sayı ifade edildiği bir örnek verilmiştir. Kesir noktası sola kaydırıldığında sayı, 10 ile çarpılmış, sağa kaydırıldığında ise sayı 10'a bölünmüş olur. Kayan noktalı sayının üs kısmı dışında kalan kısmı taban veya **kesir** (fraction) olarak adlandırılır. Tabanın kesir noktası sağında kalan hanelerine ise **mantis** (mantissa) adı verilir. Tablo 2.2'de en çok kullanılan kayan noktalı sayılara ilişkin bellek miktarı ve sayı sınırları verilmiştir[3].

1938 yılında *Konrad Zuse*, ilk ikili (binary) programlanabilir mekanik bilgisayar olan Z1'i yapmıştı. Bu bilgisayar, 7 bitlik işaretli üs, 17 bitlik anlamlı değer ve bir işaret biti içeren 24 bitlik ikili tabanda kayan noktalı sayı kullanmıştır.

[3] IEEE Computer Society (2019-07-22). IEEE Standard for Floating-Point Arithmetic. IEEE STD 754-2019. IEEE. pp. 1–84.

Bit Sayısı	Bellek Miktarı	Sayı Sınırları
32 bit	4 bayt	$1,2 \times 10^{-38}$ ile $3,4 \times 10^{+38}$ arasında **tek hassasiyetli** gerçek sayılar; en soldaki 1 bit işaret biti, sonraki 8 bit üs ve geri kalan 23 bit ise kesir olarak kullanılan ikilik tabanda sayılardır.
64 bit	8 bayt	$2,3 \times 10^{-308}$ ila $1,7 \times 10^{+308}$ arasında **çift hassasiyetli** gerçek sayılar; en soldaki 1 bit işaret biti, sonraki 11 bit üs ve geri kalan 52 bit ise kesir olarak kullanılan ikilik tabanda sayılardır.

Tablo 2.2: Kayan Noktalı Sayılar ve Sınırları

Tam sayılarda olduğu gibi **kayan noktalı sayılar da bilgisayarlarda ikili (binary) sayı tabanında tutulur**. Örneğin ondalık tabanda 50,25 sayısı ikili tabanda $1.1001001x2^5$ olarak ifade edilir.

Program yazılırken tanımlanacak değişkenlere ilişkin bellek ihtiyacı programcı tarafından belirlenir. Programcı değişkene tahsis edilecek bellek miktarına ve tutacağı içeriğe göre **veri tipi** (data type) belirler. Değişkenlerin veri tipine kısaca **değişken tipi** (variable type) adı da verilir. Veri tiplerine verilen adlar dilden dile değişir; Örneğin PASCAL dilinde 8 bitlik tam sayıya `bayt` denilirken, C dilinde `char` adı verilir. Programcı tarafından kullanılacağı amaca göre belirleyeceği veri tipinde istediği kadar değişken tanımlayabilir.

2.5 Fonksiyon

Matematikte **fonksiyon** (function), bir kümenin bir veya daha fazla kümenin elemanını tek bir kümenin elemanına eşleyen **ilişkidir** (relationship).

Girdi	İlişki	Çıktı
1	x2	2
2	x2	4
3	x2	6
4	x2	8
...		

Tablo 2.3: İki İle Çarpma Fonksiyonu

Girdi olan bağımsız değişkenler ile çıktı olan bağımlı değişken arasındaki bu ilişki, matematikte aşağıdaki şekilde **ifade** (expression) edilir.

$$f(x) = 2x$$

Burada f, fonksiyon anlamında x ise girdi anlamında olup **parametre** olarak adlandırılır. Böylece tabloda verilen fonksiyon tablosuna her girdiyi yazmak zorunda kalmayız. Bazen fonksiyona aşağıdaki örneği verilen şekilde bir ad verilmez;

$$y = 2x$$

Bazı durumlarda ise girdi daha detaylı tanımlanır;

$$x \in R, f(x) = 2x + 33$$

Bu fonksiyon, reel sayı kümesindeki her x elemanını, hesaplanan f(x) değerine eşleyen bir ifadedir. Yani x=1.0 argümanını f(1.0)=5.0 reel sayısına eşler. Kısaca fonksiyon 5.0 reel sayısını hesaplayıp **döndürür** (return).

$$g(a, b) = 2a + b$$

Bu fonksiyon ise reel sayı kümesindeki her a elemanı ile b elemanını, hesaplanan g(a,b) değerine eşleyen bir ifadedir. Yani a=1.0 ve b=1.0 argümanlarını g(1.0,1.0) fonksiyonu yine reel sayı kümesinden 3.0 reel sayısına eşler. Kısaca bu fonksiyon, 3.0 reel sayısını hesaplayıp döndürür (return).

Örneklerdeki x, a ve b parametre olarak adlandırılır. f(1.0) ifadesindeki 1.0 ise parametrenin o an andığı değeri belirtir ve **gerçek parametre** (actual parameter) ya da **argüman** (argument) olarak adlandırılır.

$$f(3.0) + g(3.0, 2.0) + f(2.0) + g(1.0, 3.0)$$

Fonksiyonlar ifade olarak tanımlandıktan sonra yukarıdaki gibi tekrar tekrar **çağrılabilirler** (call).

2.6 Yapısal Programlama

1970'li yıllara kadar her ne kadar çıktı üretmeyen fonksiyonlar için `SUBROUTINE`, birden fazla çağrı noktası olabilen `COROUTINE` ve `FUNCTION` kullanılıyor olsa da hala `GO TO` ve `JUMP TO` ifadeleri kullanılmakta ve kafa

karışıklığına devam etmekteydi.

> 1968 Yılında *Edsger W. Dijkstra* önderliğinde **`GO TO` ve `JUMP TO` ifadeleri zararlı olarak ilan edilmiştir.**

1970 Yılında *Niklaus Wirth* tarafından **Pascal** dili geliştirildi. Bu dil ilk geliştirilen yapısal programlama dilidir. **Yapısal programlamada** (structural programming) verileri taşıyan veri yapıları (değişkenler ve diziler) ile bunu işleyen kontrol yapıları (`if`, `for`, `do`, `repeat`) birbirinden ayrılmıştır. Pascal, kendi kendini çağıran **özyinelemeli** (recursive) fonksiyonlar ile değişkenlerin adreslerini tutan **göstericileri** (pointer) desteklemektedir[4]. Aşağıda iki sayıyı kullanıcıdan alan ve konsola toplamı yazdıran bir program örneği verilmiştir;

```
program ToplamaIslemi;
var
sayi1, sayi2, toplam: integer; { Değişken tanımlama bölümü }
begin { Ana fonksiyon başlangıcı }
        { Kullanıcıdan veri alma }
        writeln('Birinci sayiyi giriniz:');
        readln(sayi1);
        writeln('Ikinci sayiyi giriniz:');
        readln(sayi2);
        { Hesaplama }
        toplam := sayi1 + sayi2;
        { Sonucu ekrana yazdırma }
        writeln('Girdiginiz sayilarin toplami: ', toplam);
end. { Ana fonksiyon bitişi }
```

> Yapısal programlamada programlama, fonksiyonların birbirini çağırmasıyla yapılır. **Yapısal programlamada veri ile veriyi işleyen yapılar birbirinden ayrıldığından arapsaçı programlamanın aksine okunaklılık çak yüksektir.**

2.7 Yapısal Programlama Genel Çerçevesi

1. Programın başladığı yeri belirten bir **ana fonksiyon** (main function) tanımlanır. Kodlaması yapılacak işi birbirini çağıran fonksiyonlar olarak bu ana fonksiyonda yazarız.
2. Her fonksiyonda ilk önce **veri yapıları** (data structure) tanımlanır. Veri yapısı;
   (a) En basit veri yapısı `char`, `integer`, `float` ve `double` gibi **ilkel veri tiplerinden** (primitive data type) olabilen **değişken** (variable),
   (b) İlkel veri tiplerinden meydana gelen **dizi** (array),
   (c) İlkel veri tiplerinin birkaçından veya dizilerinden bir araya gelmesiyle oluşan **yapılar** (structure),
   (d) Ya da dinamik olarak yani çalışma anında (run-time) birbirine bağlanmış verilerden oluşan **bağlı liste** (linked list) olabilir.
3. Her fonksiyonda tanımlanan veri yapılarının ardından bu yapıları işleyecek **kontrol yapıları** (control structure) tanımlanır. Kontrol yapıları bir veya daha fazla veya iç içe `if`, `if-else`, `switch`, `while`, `do-while`, `for`, `continue`, `break`, `goto` ve `return` **talimatlarından** (statement) oluşur.

Yapısal programlamanın en çok uygulandığı Pascal dilinde ana fonksiyon nokta ile biten `BEGIN-END.` bloğudur. C dilinde ana fonksiyon, `main()` fonksiyonudur.

C Programlama Dili, *Dennis M. Ritchie* tarafından Bell Laboratuvarlarında UNIX işletim sistemini geliştirmek için geliştirilen genel amaçlı, üst düzey bir dildir. C, ilk olarak 1972'de DEC PDP-11 bilgisayarında çalıştırılmıştır. *Ken Thompson* tarafından geliştirilen B adlı daha eski bir dilden gelişmiştir. 1978 yılında *Brian Kernighan ve Dennis Ritchie*, günümüzde K&R standardı olarak bilinen C'nin ilk kamuya açık kılavuzunu hazırlamışlardır.

C programlama dili; Öğrenmesi kolaydır, **yapısal programlama dilidir**, hızlı ve verimli programlar üretir, assembly dili desteği ile **düşük düzey programlamayı** (low level programming) da destekler ve standart başlık dosyaları sayesinde çeşitli bilgisayar platformlarında derlenebilir. Bu özellikleri nedeniyle neredeyse tüm işletim sistemlerinin çekirdeği C dilinde yazılmıştır ve sistem dili olarak da adlandırılmaktadır. Yukarıda verilen Pascal yapısal programının C dilinde kodlanmış hali aşağıda verilmiştir;

```
#include <stdio.h>
int main(){
```

[4] Wirth, Niklaus (1976). Algorithms + Data Structures = Programs. Prentice-Hall. p. 126

```
        // Değişken Tanımlama
        int sayi1, sayi2, toplam;
        // Kullanıcılardan veri alma
        printf("Birinci Sayıyı Giriniz:");
        scanf("%d",&sayi1);
        printf("İkinci Sayıyı Giriniz:");
        scanf("%d",&sayi2);
        // Hesaplama
        toplam=sayi1+sayi2;
        //Sonucu ekrana yazdırma
        printf("Girdiğiniz Sayıların Toplamı:%d",toplam);
        return 0;
}
```

2.8 Nesne Yönelimli Programlama İhtiyacı

C++ programlama dili, C diline yapılan nesne yönelimli programlama OOP (object oriented programming) eklentileri yapılarak geliştirilmiş bir programlama dilidir. 1979 senesinde bir Danimarkalı bilgisayar bilim adamı olan *Bjarne Stroustrup*, sonradan C++ olarak bilinecek olan **C with Classes** üzerinde çalışmaya başladı ve 1985 yılında ilk sürümünü yayınladı.

Yapısal programlamada talimatlar (statements) art arda koda yazılarak programlama yapılır ve programların neler yaptığı bu talimatlar izlenerek anlaşılabilir. Talimatların zincirin halkaları gibi birbirinin peşi sıra yazılarak yapılan programlamaya ise emreden programlama (imperative programming) paradigması adı verilir. Bu paradigmaya sahip diller, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler. Emreden paradigmanın bir örneği olan yapısal programlamada zamanla aşağıdaki problemler ortaya çıkmıştır;

- Kod ne kadar büyürse, modüllere ayırma imkânı olmasına rağmen, fonksiyonlar arasındaki bağımlılık da o kadar artar. Bir fonksiyonun parametrelerindeki değişiklik, onun kullanıldığı tüm yerlerde değişiklik gerektirir. Değişim yönetimi (change management) çok zordur ve uzun zaman alır.
- Hata durumunda yapılacak işlemlere ilişkin kod ile iş sürecini gerçekleştiren kod iç içedir. Aynı fonksiyonun bazı durumlarda hata, bazı durumlarda ise değer döndürmesi mümkündür. Çoğu durumda hataların izini sürmek (error handling) işi zorlaşır.
- Yapısal programlamada, gösterici (pointer) kullanımında erişilmesi istenmeyen bellek bölgelerine erişilmesi halinde istenmeyen program davranışları ortaya çıkar. Kodun güvenli (safe code) olarak çalıştığının incelenmesi çok zordur.
- Sürekli olarak benzer projelerde aynı kodları tekrar yazmak (duplicate code) gereklidir.
- Emreden paradigmanın burada belirtilen sebepler başta olmak üzere çeşitli problemleri bulunmaktadır. Bu nedenle 1980'li yıllarda birçok yazılım projesi başarısız (fail) olmuştur.

Bir yazılım; Kullanıcı ihtiyaçlarının karşılanamaması, Öngörülen bütçenin aşılması ve Zamanında teslim edilememesi durumlarında başarısız olur. Yapısal programlamadaki bu sıkıntıları gören yazılımcıların imdadına Simula ve Smalltalk programlama dillindeki yaklaşım yetişmiştir. Bu diller oldukça yavaş olmasına rağmen getirdiği çözümler oldukça yenilikçiydi.

Smalltalk, 1972 yılında Xerox Park şirketinde *Alan Kay* önderliğinde *Dan Ingalls*, *Adele Goldberg*, *Ted Kaehler*, *Diana Merry*, *Scott Wallace* ve *Peter Deutsch* tarafından geliştirilmiş bir dildir. Smalltalk dilinde nesneler (object) temel yapıtaşlarıdır. Nesneler;

- Durum (state) ve davranışlara (behavior) sahiptir.
- Programlama, nesnelerin birbirlerine ileti göndermesiyle (message-passing) yapılır.
- Nesnelerin kendi ya da bir başka nesnenin davranış ve durumlarını öğrenebilme yeteneği yani yansıma (reflection) özelliği vardır.

Yapısal programlamanın içinde olduğu emreden paradigmanın (imprative programming) aksine, **nesne yönelimli programlama** OOP (object oriented programming paradigmasında progamlama, nesnelerin birbirlerine ileti göndermesi bakış açısıyla yapılır.)

2.9 Düşün ve Çöz

- **Düşün**: Arapsaçı kod (spaghetti code) kavramının yapısal programlama ile nasıl çözüldüğünü ve fonksiyonların bu süreçteki rolünü tartışınız.
- **Düşün**: Bir fonksiyon, 'ne yapılacağını' mı yoksa 'nasıl yapılacağını' mı tarif eder? Bu ayrımı gerçek bir

örnek üzerinden (örneğin bir sıralama algoritması) nesne yönelimli ve yapısal programlama perspektifinden karşılaştırın.

- ◇ **Düşün**: Assembly dilinde yazılan bir program, yüksek seviyeli dilde yazılana göre neden daha hızlı çalışabilir? Ancak neden artık sistem programlaması dışında tercih edilmemektedir?
- ◇ **Düşün**: Değişken (variable) kavramı ile değişmez (literal) arasındaki temel fark nedir? Bir programda değişken yerine yanlışlıkla değişmez kullanılırsa hangi tür hatalar ortaya çıkabilir?
- ◇ **Düşün**: Yapısal programlamada alt programlar (procedure) ile nesne yönelimli programlamadaki yöntemler (method) arasındaki kavramsal farkı açıklayın. İkisi aynı şey midir?
- ◇ **Düşün**: Simula ve Smalltalk dillerinin nesne yönelimli programlamaya katkısını araştırın. Bu dillerin C++'a etkisi neden 'devrimci' olarak nitelendirilir?
- ◇ **Düşün**: Algoritma tasarımında yalancı kod (pseudocode) kullanmanın, doğrudan kod yazmaya başlamaya göre hata payını nasıl azalttığını açıklayınız.
- ◇ **Düşün**: Modüler programlama yaklaşımı, büyük bir yazılım projesinde farklı mühendislerin aynı anda çalışmasını nasıl kolaylaştırır?
- ◇ **Çöz**: Bir öğrencinin vize ve final notlarını temsil eden iki değişken tanımlayınız. Bu verilerin işlenerek ortalama bilgisine dönüşme sürecini basit bir algoritma ile gösteriniz.
- ◇ **Çöz**: Bir sayının asal olup olmadığını kontrol eden bir algoritmanın **akış diyagramını** (flowchart) çiziniz.
- ◇ **Çöz**: Kullanıcının girdiği üç sayıdan en büyüğünü bulan algoritmanın adımlarını mantıksal bir sıra ile listeleyiniz.

3. Bölüm

Derleyiciler ve Çalıştırma Ortamı

3.1 Derleyici

Genel amaçlı bilgisayarların çok kullanılmasıyla birlikte metin editörleri de (Windows Not Defteri, Mac TextEdit, Notepad++, Epsilon, EMACS, nano, vim veya vi) işletim sistemlerinin bir parçası olmuştur. Metin editörlerinde yazılan programlar ise **derleyiciler** (compiler) tarafından makine koduna çevrilerek **icra edilebilir** (executable) dosya olarak kaydedilebilmektedir.

C++ dilinde programlama öğrenmeye başlamak için ilk adım, programı yazabileceğiniz bir metin editörü kullanmak ve yazdığınız programa ilişkin kaynak kodları `.cpp` uzantısı ile dosya olarak saklamaktır. İkinci adım ise **işletim sisteminizde** (operating system) çalışabilen bir icra edilebilir dosya oluşturan bir derleyici kurmaktır. Yani bilgisayarınızda iki yazılım aracına ihtiyacınız vardır; birincisi C++ derleyicisi, ikincisi ise basit bir metin editörüdür.

Derleyici için girdi, kaynak kodu dosyasıdır. Kaynak dosyasında yazılan kaynak kodu, içerisinde C++ **talimatları** (statement) olan, insan tarafından okunabilen metin dosyasıdır. Kaynak kodda verilen talimatlara göre işlemcinizin programı çalıştırabilmesi için, bunun makine diline **derlenmesi** (compile) gerekir.

> Derleme, bir başka programlama dilinden **sembolik isim** (mnemonic) ya da **makine koduna** (instruction code) dönüştürme işlemidir.
> Bunu yapan programlara **derleyici** (compiler) adı verilir.

3.2 C++ derleyicileri

Günümüzde birçok C++ derleyicisi mevcuttur. En çok kullanılanları aşağıda verilmiştir;

- **GNU Derleyici Koleksiyonu (GCC)**: Popüler bir açık kaynaklı C++ derleyicisidir[1]. Windows, macOS ve Linux dahil olmak üzere çok çeşitli platformlarda kullanılabilir. GCC, çok çeşitli özellikleri ve çeşitli C++ standartlarına desteğiyle bilinir.
- **Clang**: LLVM projesinin bir parçası olan açık kaynaklı bir C++ derleyicisidir[2]. Windows, macOS ve Linux dahil olmak üzere çok çeşitli platformlarda kullanılabilir. Clang, hızı ve optimizasyon yetenekleriyle bilinir.
- **Microsoft Visual C++**: Microsoft tarafından geliştirilen tescilli bir C++ derleyicisidir[3]. Yalnızca Windows için kullanılabilir. Visual C++, Microsoft Visual Studio geliştirme ortamıyla entegrasyonuyla bilinir.

Bilgisayarınızın 64 bitlik Windows olduğu varsayılarak `g++` derleyicisi aşağıdaki şekilde kurulur;

- MINGW sürümü ilgili web sayfasından indirilerek kurulur[4].
- Kurulumu tamamlandıktan sonra `PATH` değişkenine `g++` derleyicisinin klasörü eklenir.

Böylece kurulum tamamlanmış olur. **Komut istemi** (command prompt veya terminal) açıldığında `g++ -v` komutu yazılarak derleyicinin düzgün kurulup kurulmadığı kontrol edilebilir.

```
C:\Users\ILHANOZKAN>g++ -v
Using built-in specs.
COLLECT_GCC=g++
COLLECT_LTO_WRAPPER=D:/msys64/ucrt64/bin/../lib/gcc/x86_64-w64-mingw32/13.2.0/lto-wrapper.exe
Target: x86_64-w64-mingw32
Configured with: ../gcc-13.2.0/configure --prefix=/ucrt64 --with-local-prefix=/ucrt64/local
--build=x86_64-w64-mingw32 --host=x86_64-w64-mingw32 --target=x86_64-w64-mingw32
--with-native-system-header-dir=/ucrt64/include --libexecdir=/ucrt64/lib --enable-bootstrap
--enable-checking=release --with-arch=nocona --with-tune=generic
--enable-languages=c,lto,c++,fortran,ada,objc,obj-c++,jit --enable-shared --enable-static
```

[1] https://gcc.gnu.org/
[2] https://clang.llvm.org/
[3] https://visualstudio.microsoft.com/tr/vs/features/cplusplus/
[4] https://www.mingw-w64.org/downloads/

```
--enable-libatomic --enable-threads=posix --enable-graphite --enable-fully-dynamic-string
--enable-libstdcxx-filesystem-ts --enable-libstdcxx-time --disable-libstdcxx-pch
--enable-lto --enable-libgomp --disable-libssp --disable-multilib --disable-rpath
--disable-win32-registry --disable-nls --disable-werror --disable-symvers --with-libiconv
--with-system-zlib --with-gmp=/ucrt64 --with-mpfr=/ucrt64 --with-mpc=/ucrt64 --with-isl=/ucrt64
--with-pkgversion='Rev3, Built by MSYS2 project'
--with-bugurl=https://github.com/msys2/MINGW-packages/issues --with-gnu-as --with-gnu-ld
--disable-libstdcxx-debug --with-boot-ldflags=-static-libstdc++
--with-stage1-ldflags=-static-libstdc++
Thread model: posix
Supported LTO compression algorithms: zlib zstd
gcc version 13.2.0 (Rev3, Built by MSYS2 project)
```

Kaynak kodumuzu yazmak için bir metin editörü gereklidir. **Windows** işletim sisteminde `notepad`, **Linux** ve **macOs** işletim sistemleri için `nano` programları metin yazmak için idealdir. İlk kaynak kodumuzu oluşturmak için Windows komut isteminde `notepad hello.cpp`, Linux ve macOs işletim sistemlerinde ise `nano hello.cpp` komutu yazılır. Editör uygulaması açılır. Eğer böyle bir dosya yoksa metin editörü bu dosyayı oluşturmak için onay ister. Açılan editörü penceresine ilk C++ programı yazılır ve `hello.cpp` adlı dosyaya kaydedilir.

```
#include <iostream>
int main() {
 std::cout << "Hello World!";
 return 0;
}
```

Kaydedilen dosyayı komut isteminde `g++ hello.cpp -o hello.exe` komutu ile derlenir. Böylece hazırlanan kaynak kodumuz derleyici tarafından derlenir (compile). Böylece kaynak kodumuz icra edilebilir (executable) `hello.exe` dosyasına dönüştürülmüş olur. Derleme ve çalıştırma sürecini **Windows** komut isteminde yapmak için aşağıdaki komutlar yazılır;

```
C:\Users\ILHANOZKAN>notepad hello.cpp
C:\Users\ILHANOZKAN>g++ hello.cpp -o hello.exe
C:\Users\ILHANOZKAN>hello.exe
Hello, World!
C:\Users\ILHANOZKAN>
```

Benzer şekilde derleme ve çalıştırma sürecini **Linux** veya **macOs** komut isteminde yapmak için aşağıdaki komutlar yazılır;

```
ozkan@ILHANOZKAN:~$ nano hello.cpp
ozkan@ILHANOZKAN:~$ g++ hello.cpp -o hello.exe
ozkan@ILHANOZKAN:~$ ./hello.exe
Hello, World!
ozkan@ILHANOZKAN:~$
```

3.3 Entegre Geliştirme Ortamı

Şimdiye kadar anlatılan kod yazma (implementation), derleme (compile), icra (execute) ya da koşma (run) ve izleme (trace) gibi süreçlerin tamamını tek bir çatı altında yürütmemizi sağlayan programlar, entegre geliştirme ortamı kısaca **IDE** (Integrated Development Environment) olarak adlandırılır.

IDE'ler kodumuzu renklendirir (source code coloring veya syntax highlighting) ve kod yazarken daha az hata yapmamızı sağlarlar. Ancak kodu dosyaya yine sade metin olarak kaydederler. Ayrıca kod yazarken talimatları **doğru tamamlamak** (intellisense) için programcıya yardımcı da olurlar. Bunun gibi kolaylaştırıcı birçok özellikleri bulunur. Örneğin yukarıda yazdığımız düz metin IDE içinde aşağıdaki gibi renkli görünür;

```
#include <iostream>
int main() {
        // İlk C++ Programı
        std::cout << "Hello World!";
        return 0;
}
```

C++ programlama dili için en çok kullanılan entegre geliştirme ortamları aşağıda verilmiştir;

- **Code::Blocks**: Açık kaynaklı bir C/C++ geliştirme ortamı olup bir grafik kullanıcı ara yüzüne **GUI**

(Graphic User Interface) sahiptir. Windows, macOSX ve Linux işletim sitemleri üzerine kurulabilmektedir.

- **Visual Studio**: Microsoft tarafından geliştirilen, C++ dilinde yazılmış, güçlü, yüksek performanslı uygulamalar oluşturmak için kullanılabilen bir geliştirme ortamıdır. Visual Studio, **kod tamamlama** (intellisense), kullanıcı ara yüzü **UI** (user interface) hazırlama desteği ve **hata ayıklama** (debugger) ve birçok **eklentisi** (plug-in) olan muazzam özelliklere sahiptir.
- **Visual Studio Code**: Microsoft tarafından geliştirilen açık kaynaklı bir geliştirme ortamıdır. Windows, macOS ve Linux gibi işletim sistemlerinde çalışır. Git **sürüm kontrol** (version control) sistemleriyle çok iyi çalışır. Ayrıca, akıllı kod tamamlamanın dikkate değer özellikleriyle birlikte gelir.
- **CLion**: Jetbrains tarafından geliştirilmiştir ve C++ programcıları için en çok önerilen çapraz platformlu (CMake derleme sistemiyle entegre macOS, Linux ve Windows altında çalışır) geliştirme ortamıdır. Ayrıca, yerel (local) ve uzaktan (remote) desteğe sahip birkaç IDE'den biri olarak kabul edilir. Bu da yerel bir makinede kod yazmanıza ancak uzak sunucularda derlemenize olanak tanır. Kaynak kodlarımız yönetmemizi sağlayan **CVS** (Concurrent Versions System) ve **TFS** (Team Foundation Server) ile entegre edilebilir.
- **Eclipse**: Java'da yazılmış ve IBM tarafından geliştirilmiş ücretsiz ve açık kaynaklı bir geliştirme ortamıdır. Yaklaşık otuz programlama dilini desteklediği için geniş topluluk desteğiyle iyi bilinir. C++ için Eclipse IDE, kod tamamlama, otomatik kaydetme, derleme ve hata ayıklama desteği, uzak sistem gezgini, statik kod analizi, **profilleme** (profiling) ve **yeniden düzenleme** (refactoring) gibi beklenebilecek tüm özelliklere sahiptir. Ayrıca çeşitli harici eklentileri entegre ederek işlevselliğini genişletebilirsiniz. Windows, Linux ve macOS'ta çalışabilir.
- **NetBeans**: Apache Yazılım Vakfı – Oracle Corporation tarafından geliştirilen ücretsiz ve açık kaynaklı bir geliştirme ortamıdır. Öğrenciler veya başlangıç seviyesindeki C/C++ geliştiricileri için şiddetle tavsiye edilmesinin sebebi, Eclipse'e benzer şekilde daha iyi sürükle ve bırak işlevlerine sahip olmasıdır. Windows, Linux, MacOSX ve Solaris gibi birden fazla platformda çalışır.
- **Xcode**: MacOSX işletim sisteminde yazılım geliştirmek için kullanılan bir geliştirme ortamıdır.

3.4 Çevrimiçi Derleyiciler

Bütün bu entegre geliştirme ortamlarının yanında online olarak da derleme yapılan web uygulamaları da bulunmaktadır. Bunlardan birkaçı aşağıda verilmiştir;

- https://www.tutorialspoint.com/compile_cpp_online.php
- https://www.onlinegdb.com/online_c++_compiler
- https://www.programiz.com/cpp-programming/online-compiler/
- https://onecompiler.com/cpp

3.5 Derleme Zamanı

Derleyici programının derleme işlemine başlayıp bitirildiği sürece **derleme zamanı** (compile-time) denir. Bu süreç başarısızlıkla da sonuçlanabilir ve eğer derleme işleminde hata meydana gelirse programcı hata mesajları ile uyarılır. Bir derleyici program, kaynak dosyayı makine diline çevirme çabasında, kaynak dosyanın C++ dilinin **sözdizimi** (syntax) kurallarına uygunluğunu da denetler.

Kod yazımı sırasında dilin kurallarına uyulmamışsa, derleyici bu durumu bildiren bir **derleme zamanı hatası** (compile-time error) alınarak derleme işlemi sonlandırılacaktır;

```cpp
#include <iostream>
int main() {
        std::cout << "Hello World!";
        return
}
```

Örneğindeki gibi hatalı yazılmış bir kaynak kod aşağıdaki hatayı verecektir;

```
ozkan@ILHANOZKAN:~$ g++ hello.cpp -o hello.exe
hello.cpp: In function ‘int main()’:
hello.cpp:5:1: error: expected primary-expression before ‘}’ token
5 | }
| ^
hello.cpp:4:15: error: expected ‘;’ before ‘}’ token
4 |         return
|               ^
|               ;
5 | }
```

```
| ~
ozkan@ILHANOZKAN:~$
```

Kod yazımı sırasında dilin kurallarına uyulmuş ise bazı durumlarda **derleme zamanı uyarısı** (compile-time warning) alınabilir. Ancak derleme bu durumda tamamlanır.

Kaynak kod içerisindeki metinlerde Türkçe karakter kullanılması halinde **Windows** altında derlenen programı çalıştırdığınızda aynı metni göremeyebilirsiniz. Bunu düzeltmek için komut isteminde `Chcp 65001` komutu verilerek komut satırının UTF-16 karakter kümesi ile işlem yapmasını sağlayabiliriz.

3.6 Hatalar

Programcı tarafından kodlama yapılırken genellikle aşağıdaki üç tip hata yapılır;

1. **Derleme zamanı hataları** (compile-time error): Bu tip hatalar genelde kullanılan dilin yazım kurallarına (syntax) uyulmadığından, talimatların (statement) yanlış yazılmasından ya da programcının kod metninde uygun olmayan karakterlerin kullanılmasından kaynaklanır.
2. **Çalışma zamanı hataları** (run-time error): Kaynak kod, kurallara uygun olarak yazılmıştır ve herhangi bir yazım hatası bulunmaz. Bu tip hatalara en iyi örneklerden birisi sıfıra bölme hatasıdır. Taşma hataları da bu hatalardandır. Daha sonra değinilecektir.
3. **Mantıksal hatalar** (logical error): Programcının çözüm için gerekli adımların oluşturulmasında, çözüm yönteminin yanlış olmasından ya da yanlış işleçler (operator) kullanmasından kaynaklanır. Örneğin bir büyüktür (`>`) işleci yerine küçüktür (`<`) işleci kullanıldığında ne bir yazım hatası ne de bir çalışma zamanı hatası ortaya çıkar. Fakat program kendisinden istenilen davranışları yerine getirmez ve uygun çıktıları üretmez. Faktöriyel hesaplanırken `0!=1` yerine `0!=0` kabul etmek buna örnek verilebilir.

3.7 C++ Dilinin Kullanım Alanları

C dili, başlangıçta sistem geliştirme çalışmaları için, özellikle işletim sistemini oluşturan programlar için kullanıldı. Montaj (assembly) dilinde yazılan kod kadar hızlı çalışan kod ürettiği için sistem geliştirme dili olarak benimsendi. Bilgisayar mühendislerinin bilmesi gereken bu dilin birçok kullanım alanı mevcuttur. C++ ise;

- Çok kullanılan ve popüler programlama dillerinden biridir. C++ işletim sistemleri, gömülü sistemler ve Grafiksel Kullanıcı Ara yüzleri yapımında kullanılır.
- Nesne yönelimli programlamayı destekler ve Soyutlama (abstraction), Sarma (encapsulaton) ve Miras alma (inheritance) gibi tüm OOP kavramlarını uygular, bu da programlara net bir yapı kazandırır ve kodun yeniden kullanılmasına olanak tanır, bu da geliştirme maliyetlerini düşürür ve güvenlik sağlar.
- Söz dizimi C++ diline benzediği için programcıların Java ve C# dillerine geçişini kolaylaştırır. C++ dili ara dil (intermediate language) kullanmaz de kodu doğrudan işlemcinin emir kodlarına derler. Bu nedenle C#, Python ve Java'dan daha hızlıdır.
- C'nin de bir seçenek olduğu sınırlı kaynak ortamlarında kullanışlıdır. Ayrıca yürütme hızına ihtiyaç duyduğumuz veya donanıma yakın çalışmamız gereken yerlerde de kullanılır.
- C ile karşılaştırıldığında C++, daha zengin kütüphanelere sahiptir, nesne yönelimli programlamayı destekler, şablonlar, istisna işleme ve daha birçok özelliği barındırır.

3.8 Düşün ve Çöz

- **Düşün**: "Derleme Zamanı Hatası" ile "Çalışma Zamanı Hatası" arasındaki farkı açıklayınız. Hangisinin bir mühendis için tespit edilmesi daha zordur? Neden?
- **Düşün**: Yorumlayıcı (interpreter) diller ile Derleyici (compiler) diller arasındaki performans farkının temel sebebi donanım seviyesinde nasıl açıklanır?
- **Düşün**: Statik bağlama (static linking) ve Dinamik bağlama (dynamic linking) arasındaki farkları araştırıp, programın dosya boyutu üzerindeki etkisini tartışınız.
- **Çöz**: Yazdığınız bir C kodunun makine diline dönüşene kadar geçtiği aşamaları (Önişlemci →Derleyici →Bağlayıcı) sırasıyla listeleyiniz.
- **Çöz**: Bir C++ programının derleme aşamasında oluşan ara dosyaları (`.i`, `.s`, `.o`) ve bu dosyaların içeriklerini gösteren bir şema hazırlayınız.
- **Çöz**: Sadece bir metin editörü (notepad, nano vb.) ve bir derleyici (`g++` gibi) kullanarak "Merhaba Dünya" programını komut satırından derleyip çalıştırınız.
- **Düşün**: Derleme zamanı hatası (compile-time error), **bağlama zamanı hatası** (link-time error) ve çalışma zamanı hatası (run-time error) arasındaki farkı somut C++ kod örnekleriyle açıklayın.
- **Düşün**: Bir C++ programının kaynak koddan çalıştırılabilir dosyaya dönüşme sürecini adım adım açıklayın: önişlemci, derleyici, assembler ve bağlayıcı (linker) aşamalarını belirtin.
- **Düşün**: Çevrimiçi derleyiciler (Compiler Explorer, Godbolt vb.) ne gibi durumlarda kullanışlıdır? Yerel

bir IDE'ye göre avantajları ve sınırlılıkları nelerdir?

- ◇ **Düşün**: C++ dilinin kullanıldığı üç farklı gerçek dünya uygulaması seçin (örn. Oyun motoru, işletim sistemi, gömülü sistem). Her birinde C++'ın tercih edilmesinin teknik gerekçesini açıklayın.
- ◇ textbfDüşün: GCC ve Clang derleyicilerini karşılaştırın. Optimizasyon seviyeleri (`-O0`, `-O1`, `-O2`, `-O3`) oluşturulan makine kodunu nasıl etkiler?

4. Bölüm

C++ Programlama Diline Giriş

4.1 En Basit C++ Programı

C programlama dili, her programcının ihtiyacını görecek genel amaçlı, emreden paradigmalı, yapısal programlama dilidir. Kısa ve öz talimatlar birbiri ardına verilerek yapılan programlamaya emereden paradigma (imperative programming) adı verilir. Yapısal programlamanın çerçevesi 2.7 başlığı altında anlatılmıştır.

Yapısal programlamada, programın başladığı yeri belirten bir **ana fonksiyon** (main function) tanımlanır. C dilinde bu fonksiyon `main()` fonksiyonudur. C++ dili, C programlama diline, nesne yönelimli programlama mantığını kodlayabilmesi için **eklenti yapılarak geliştirilmiştir**. C++ dilinde de ana fonksiyon, yine `main()` fonksiyonudur. Yani içinde `main()` fonksiyonu yazılmamış bir C++ programı çalışmaz. C++ dilinde de talimatlar noktalı virgül karakteri ile bittiğinden **gelişigüzel** (free format) yazılır. Aşağıda en basit C++ programı verilmiştir.

```
1 int main()
2 {
3         return 0;
4 }
```

Kodu şu şekilde açıklayabiliriz;

- Birinci satırda, yapısal programlamanın ana fonksiyonu olan (main function) tanımlanmıştır. Başındaki `int`, fonksiyonun bir tam sayı (integer) geri döndüreceğini belirtir. Program çalışmaya ana fonksiyondan başlar ve bittiğinde işletim sistemine geri döner.
- C dilinde her fonksiyon ikinci satırdaki gibi süslü parantez karakteri ile başlayan ve dördüncü satırdaki gibi biten bir kod bloğuna (code block) sahiptir. Her fonksiyonda olan bu blok, fonksiyon bloğu (function block) olarak da adlandırılır.
- Ana fonksiyon içinde üçüncü satırda yazılan `return 0;` talimatı (statement) ana fonksiyondan sıfır geri döndürerek çıkıldığını söyleyen talimattır. Sıfır dışında bir değerin işletim sistemine gönderilmesi, programın hata ile sonlandığı olarak kabul edilir. Eğer programdan çıkışta işletim sistemine `11` göndermek istersek bu talimatı `return 11;` olarak yazmalıyız. Ana fonksiyonda varsayılan olarak, hiçbir hata ile karşılaşılmadığı için, işletim sistemine `0` gönderilir. Bu nedenle `return 0;` talimatın yazılması zorunlu değildir. Ancak başka bir değer gönderilecek ise yazılır.

4.2 Söz Dizim Kuralları

C++ dili için oluşturulacak kaynak kod programcı tarafından metin dosyasına yazılırken belli **söz dizim kurallarına** (syntax) uyar. Bunlar aşağıda başlıklar halinde verilmiştir.

§4.2.1 Kaynak Kodumuzu Oluşturacak Karakterler

1. İngiliz alfabesindeki büyük harfler:
   `A B C D E F G H I J K L M N O P Q R S T U V W X Y Z`
2. İngiliz Alfabesindeki küçük harfler:
   `a b c d e f g h i j k l m n o p q r s t u v w x y z`
3. Rakamlar:
   `0 1 2 3 4 5 6 7 8 9`
4. Özel Karakterler:
   `< > . , _ ( ) ; $ : % [ ] # ? ' & { } " ^ ! * / | - ~ +`
5. Metinde görünmeyen ancak metni biçimlendiren **beyaz boşluk** (white space) karakterleri;
   (a) Boşluk (space)
   (b) Yeni satır (new line)
   (c) Satır başı (carriage return)
   (d) Geri (backspace)
   (e) Sekme (tab)

§4.2.2 Talimatlar ve Anahtar Kelimeler

Yüksek seviyeli bir dilde yazılmış bir **talimat** (statement), işlemciye belirtilen bir eylemi gerçekleştirmesini söyleyen cümledir. Yüksek seviyeli bir dildeki tek bir talimat, birkaç emir kodunu (instruction code) temsil edebilir. Talimatlar, yapılması gerekeni kısa ve net olarak belirten mantıksal satırlardır (logical sequence,).

Anahtar kelimeler (keywords), programlamada kullanılan ve C++ derleyicisi tarafından özel anlama sahip, önceden tanımlanmış, **ayrılmış kelimeler** (reserved words) olup programı oluşturan **talimatlar** (statement) bu kelimerler kullanılarak yazılırlar[1]. Tablo 4.1'de anahtar kelimeler verilmiştir.

Anahtar kelimeler, küçük-büyük harf ayrımı yapıldığından **her zaman küçük harflerle yazılırlar**.

`alignas` (1)	`constexpr` (1)(3)	`inline`(1) (3)	`static`
`alignof` (1)	`constinit` (4)	`int`(1)	`static_assert` (1)
`and`	`const_cast`	`long`	`static_cast`
`and_eq`	`continue`	`mutable`(1)	`struct` (1)
`asm`	`co_await` (4)	`namespace`	`switch`
`atomic_cancel`	`co_return` (4)	`new`	`synchronized`
`atomic_commit`	`co_yield` (4)	`noexcept` (1)	`template`
`atomic_noexcept`	`decltype` (1)(2)	`not`	`this` (5)
`auto` (1)(3)(4)(5)	`default` (1)	`not_eq`	`thread_local` (1)
`bitand`	`delete` (1)	`nullptr` (1)	`throw` (3)(4)
`bitor`	`do`	`operator` (1)	true
`bool`	`double`	`or`	`try`
`break`	`dynamic_cast`	`or_eq`	`typedef`
`case`	`else`	`private` (4)	`typeid`
`catch`	`enum` (1)	`protected`	`typename` (3)(4)
`char`	`explicit`	`public`	`union`
`char8_t` (4)	`export` (1)(4)	`reflexpr`	`unsigned`
`char16_t` (1)	`extern` (1)	`register` (3)	`using` (1)(4)
`char32_t` (1)	`false`	`reinterpret_cast`	`virtual`
`class` (1)	`float`	`requires` (4)	`void`
`compl`	`for` (1)	`return`	`volatile`
`concept` (4)	`friend`	`short`	`wchar_t`
`const`	`goto`	`signed`	`while`
`consteval` (4)(5)	`if` (3)(5)	`sizeof` (1)	`xor`
			`xor_eq`

Tablo 4.1: Anahtar Kelimeler

Yüksek seviyeli bir dilde yazılmış bir talimat (statement), işlemciye belirtilen bir eylemi gerçekleştirmesini söyleyen cümledir. Yüksek seviyeli bir dildeki tek bir talimat, birkaç makine dili emir kodunu (instruction code) temsil edebilir. Talimatlar, yapılaması gerekeni kısa ve net olarak belirten mantıksal satırlardır (logical sequences).

Her talimat (statement), anahtar kelimeler içerecek şekilde yazılır ve **noktalı virgül karakteri ile biter**.

Talimatlar BASIC ve FORTRAN gibi dillerde genellikle satır sonunda biter. FORTRAN dilinde yedinci sütundan yazılmaya başlanır. BASIC dilinde ise her satırın başına satır numarası yazılır. Pascal ve C dillerinde böyle bir kural söz konusu değildir. Talimat herhangi bir yerde başlar ve noktalı virgül karakteri ile biler. Aradaki beyaz boşluklar ne kadar çok olursa olsun tek bir tane varmış gibi kabul edilir. Bu nedenle kod **gelişigüzel** (free format) yazılır.

Aşağıda gelişigüzel yazılan programların hepsi derlenebilir ve derleme sonucu aynı çalıştırılabilir dosyayı üretir. Beyaz boşlukları alınmış en kısa kod aşağıda verilmiştir:

```
int main() {}
```

Kod içerisinde beyaz boşluk karakterleri anahtar kelimeleri bölmediği sürece istenildiği kadar kullanılabilir:

```
int
    main  ( ) {
```

[1]https://en.cppreference.com/w/cpp/keyword (1)C++11, (2)C++14, (3)C++17, (4)C++20 ve (5)C++23 uyarlamasında değişti veya eklendi anlamındadır.

```
                return                  0;
        }
```

Fonksiyon parantezleri içine veya fonksiyon blokları içine istenildiği kadar beyaz boşluk konulabilir. Parantezler ve fonksiyon blokları hizalı olmayabilir:

```
int main (              )
    {
  }
```

Kodun güzel görünmesi şart değildir:

```
        int main(
        )
{
                return
                   0;
                                }
```

Ancak kodun bir başkası tarafından bakımı yapılacağı göz önüne alınarak okunaklı yazılması çok önemlidir. **En doğrusu programı aşağıdaki sitilde** yazmaktır;

- Fonksiyonların gövdesi ve bazı kontrol talimatları kod bloklarından oluşur. Kod blokları için süslü parantez kullanılır.
- Süslü parantez içindeki kodlar, okunabilirliği artırmak için genellikle **4 boşluk** veya bir **sekme** (tab) içeriden başlar. Kodu bu şekilde yazmaya **girintili** (indented) yazma adı verilir.
- Ayrıca değişkenler ve anahtar kelimeler arasında boşluk karakteri veya sekme karakteri kullanılmalıdır:

```
int main() {
        //Ana fonksiyon bloğuna ait girinti
        int sayac=0;
        if (sayac==0) { //iç blok
                //iç bloğa ait girinti...
                int sonuc=1;
                return sonuc;
        }
        if (sayac==1) // Blok başlangıcı böyle de yazılabilir
        { //iç blok
                //iç bloğa ait girinti...
                int sonuc=1;
                return sonuc;
        }
}
```

§4.2.3 Açıklamalar

Talimatlar (statement), yapılması gerekeni kısa ve net olarak belirten **mantıksal satırlardır** (logical sequence). Programcı kodu yazarken **açıklama** (comment) yapma gereği duyabilir. Bunu iki şekilde yapar;

1. Kod metninde bulunulan yerden sonra satırın sonuna kadar olan bir açıklama yapılmak istendiğinde, açıklama öncesinde iki adet bölü (`//`) karakteri kullanılır.
2. Eğer birden çok satırı içeren bir açıklama yapılacak ise açıklama (`/*`) ile (`*/`) karakterleri arasına alınır.

Aşağıda açıklamalar eklenmiş ana fonksiyon örneği yer almaktadır;

```
/*
Bu program, açıklama (comment) içeren basit bir ana fonksiyonu olan C++ programıdır.
İlhan ÖZKAN, Ocak 2025
*/
int main() // Ana Fonksiyon
{   //fonksiyon bloğu başlagıcı
        /*
        Ana Fonksiyon, programın başladığı yerdir. Ana fonksiyonu olmayan bir program çalışmaz.
        */
} //fonksiyon bloğu bitişi
```

İstediğimiz kadar açıklama yapabiliriz. Derleyici açısından bunun önemi yoktur. Çünkü derlemenin ilk aşamasında **bütün bu açıklamalar ve beyaz boşluk karakterleri metinden çıkartılır**.

4.3 Değişken Tanımlama

Her yapısal programlama dilinde olduğu gibi C dilinde de **veri yapıları** (data structure) ve **kontrol yapıları** (control structure) birbirinden ayrıdır. Dolayısıyla veriyi işleyecek kontrol yapısı kodlanmadan önce veriyi taşıyan veri yapıları tanımlanmalıdır.

Değişkenler (variable) veri yapılarının temel öğeleridir. Bu başlıkta anlatılan tam sayı ve kayan noktalı sayı değişkenleri, belleğin belli bir bölgesini işgal eder ve fiziki olarak bellek bitlerden oluştuğundan ikili sayı (binary) olarak veri tutarlar.

§4.3.1 Tam Sayı Veri Tipleri

C++ dilinde 2.4 başlığında anlatılan tam sayılar (integer), **ilkel veri tiplerinden** (primitive data type) biridir. Tam sayı veri tipleri için aşağıdaki anahtar kelimeler kullanılır;

Veri Tipi	Boyut (Byte/Bit)	Kullanım Amacı / Özellik	Mimari Notu
`char`	1 Bayt (8 Bit)	İşaretli tam sayı / Karakter	En küçük adreslenebilir birim
`short`	2 Bayt (16 Bit)	Küçük ölçekli tam sayılar	Bellek tasarrufu sağlar
`int`	2/4 Bayt (16/32 Bit)	Standart tam sayı	Sistem mimarisine göre değişir
`long`	4 Bayt (32 Bit)	Geniş tam sayı	En az 32 bit garanti edilir
`long long`	8 Bayt (64 Bit)	Çok geniş tam sayı	Modern 64-bit hesaplamalar
`wchar_t`	2/4 Bayt (16/32 Bit)	Çok dilli (wide) metinler	Uluslararası karakterler
`char16_t`	2 Bayt (16 Bit)	UTF-16 Karakterler	Taşınabilir çok dilli destek
`char32_t`	4 Bayt (32 Bit)	UTF-32 Karakterler	Tam Unicode karakter desteği
`bool`	1 Bayt (8 Bit)	Mantıksal (`true`/`false`)	Arka planda 1 ve 0
`std::size_t`	Mimari Bağımlı	Nesne boyutu / İndis	`sizeof()` işlecinin sonucudur

§4.3.2 Kayan Noktalı Sayı Veri Tipleri

C++ dilinde 2.4 başlığında anlatılan kesirli sayılar için kullanılan kayan noktalı sayı (floating point number), **ilkel veri tiplerinden** (primitive data type) biridir ve bu veri tipleri için aşağıdaki anahtar kelimeler kullanılır;

Veri Tipi	Boyut (Byte/Bit)	Hassasiyet (Precision)	Kullanım Alanı
`float`	4 Bayt (32 Bit)	Tek Hassasiyet (Single)	Grafik / Düşük hassasiyetli işler
`double`	8 Bayt (64 Bit)	Çift Hassasiyet (Double)	Bilimsel ve genel hesaplamalar

§4.3.3 Diğer Veri Tipleri

Tam sayılar 4.3.1 başlığında anlatıldığı üzere standart tiplerin (`int`, `long` vb.) boyutları platformdan platforma değişebilir. Profesyonel projelerde tam kontrol sağlamak için `<cstdint>` başlığındaki sabit genişlikli tipler kullanılmalıdır. Bunlar;

- ◇ `int8_t` / `uint8_t`: Her platformda geçerli tam 8 Bit Tam Sayı
- ◇ `int16_t` / `uint16_t`: Her platformda geçerli tam 16 Bit Tam Sayı
- ◇ `int32_t` / `uint32_t`: Her platformda geçerli tam 32 Bit Tam Sayı
- ◇ `int64_t` / `uint64_t`: Her platformda geçerli tam 64 Bit Tam Sayı

Bunların dışında ileride yeri geldikçe göreceğimiz tam sayı gösterici (`std::intptr_t`) ve işaretsiz tam sayı gösterici (`std::uintptr_t`) tipleri de vardır. Bu, düşük seviyeli sistem programlaması için çok önemlidir.

C++ programlama dilinde bir de **hiçbir değeri olmayan ve bellekte yer kaplamayan** `void` veri tipi vardır. Bu veri tipini hiçbir değer döndürmeyen fonksiyon tanımlamalarında ve dinamik bellek kullanımlarında ileride çokça göreceğiz.

§4.3.4 Değişken Tanımlama

Değişkenlerin veri tipine (data type) bağlı olarak benzersiz bir **kimlik** (identifier) verilerek **tanımlanması** (definition) gerekir.

```
/* Bu program değişken tanımlamak için oluşturulmuştur.*/
```

```
int main()  {
        char yas; /* veri tipi "char" ve kimliği "yas" olan değişken tanımlandı. -128 ila +127
        ↪  arasında değer alabilir. */
        short kat;  /* veri tipi "short" ve kimliği "kat" olan değişken tanımlandı. -32768 ila
        ↪  +32768 arasında değer alabilir. */
        float kilo; /* veri tipi "float" ve kimliği "kilo" olan değişken tanımlandı. */
}
```

Yukarıda tanımlaması yapılan `yas`, `kat` ve `kilo` değişkenleri ana fonksiyon bloğu içinde tanımlandığından, tanımlanan değişkenler sadece bu blok içinde geçerlidir!

Aynı blok içinde bir değişkene verilen kimlik, bir başka değişkene verilemez!

Bildirim (declaration) derleyiciye böyle bir değişken, fonksiyon veya nesne var demektir. Bunu ileride daha detaylı göreceğiz. Şimdilik tanımlama gibi kabul edebilirsiniz. **Tanımlama**, (definition) derlemenin bağlama (link) ile birlikte sorunsuz olabilecek şekilde eksiksiz yapılan bir işlemdir.

§4.3.5 Değişken Kimliklendirme Kuralları

C++ programlama dilinde **değişkenlerin kimliklendirilmesinde** (identifier definition) dört temel kuralar vardır. Bunlar;

1. **Kimliklendirmede anahtar kelimeler kullanılamaz!**
2. **Kimliklendirme asla rakamla başlayamaz!**
3. **Kimlikler en fazla 32 karakter olmalıdır!** Daha fazlası derleyici tarafından dikkate alınmaz!
4. **Kimliklendirmede ancak İngiliz alfabesindeki Büyük ve Küçük harfler, rakamlar ile alt çizgi karakteri kullanılabilir**.

Aşağıda kimliklendirme kurallarına uygun tanımlama yapılmıştır;

```
/* Bu program değişken tanımlama örneklerini içerir. */
int main() {
        /* Tam sayı değişkenler: */
        char yas;
        short kisa_tam_sayi;
        long uzun_tam_sayi;
        /* Gerçek Sayı Değişkenler:*/
        float kilo;
        double reel_sayi;
        /* Aynı veri tipinde birden fazla değişkene aralarına virgül koyarak kimlik verilebilir.
        ↪  */
        int kat1,kat2,kat3;
        float olcek1,olcek2;
}
```

Bütün bu kuralların yanında değişkenlere kimlik verilirken yazılan kodun okunaklılığını artırmak için **deve yazımı** (camel case) stili kimlik verilir. Bu stilde ilk sözcük tamamıyla küçük, izleyen sözcüklerin ise baş harfi büyük yazılır. Bu kural zorunlu değildir ama koda bakım yapacak sonraki programcıların işini kolaylaştırmak için ahlaki olarak tercih edilir;

```
/* Bu program camelCase değişken kimliklendirme örneklerini içerir.*/
int main() {
        int sayac;
        int ogrenciBoyu;
        int asansorunOlduguKat;
        float asansorAgirligi;
        int ogrenciYasi;
        char ilkHarf;
        char ogrenciAdi[50];
        char ogrenciSoyadi[50];
        float ortalamaNot;
}
```

Değişkenin kapsamı (variable scope); Değişkenin bildiriminin (variable declaration) yapılabileceği, değişkene erişilebileceği (access), değişkenin üzerinde çalışılabileceği program kodu olarak tanımlanır. Değiş-

kenin kapsamı, tanımlandığı kod bloğu (ve varsa bu blok içindeki iç bloklar) ile sınırlıdır.

§4.3.6 Veri Tipi Değiştiricileri

İşaretli bir tam sayı kullanmak yerine en anlamlı işaret bitini de sayıya dahil ederek tam sayıları **işaretsiz olarak** (unsigned) kullanabiliriz. Tam sayı veri tipleri olan `char`, `short`, `int` ve `long` veri tipleri (data type) aslında **işaretli** (signed) tam sayılardır. Yani önlerinde `signed` sıfatı olduğu varsayılır. Tam sayıları işaretsiz olarak kullanmak istememiz halinde `unsigned` sıfatını tam sayı veri tipinin önünde kullanırız.

```
/* Bu program işaretsiz tam sayı değişken tanımlamak için oluşturulmuştur.*/
int main()  {
        unsigned yas; /* veri tipi "unsigned char" ve kimliği "yas" değişkeni
        tanımlandı. yas değişkeni 0 ile 255 arasında bir değer alabilir. */
        unsigned short kat; /* veri tipi "unsigned short" ve kimliği "kat" olan değişken
         ↪  tanımlandı.kat değişkeni 0 ila 65535 arasında değer alabilen bir değişkendir. */
        unsigned long pozitifTamsayi; /* pozitifTamsayi değişkeni 0 ile 4.294.967.295 arasında
         ↪  değer alabilen bir değişkendir. */
        uint8_t ogrenciYasi; //işatersiz veri tipi kullanarak yaş tanımı.
}
```

Yukarıda 4.3 başlığında verilen temel veri tiplerini değiştiren `unsigned` gibi **veri tipi değiştiricileri** (data type modifier) vardır. Değiştiriciler için kullanılan anahtar sözcükler;

- Bir değişkenin değerinin ilk değer atamasından sonra değiştirilemeyeceğini `const` sıfatı belirtir.
- Bir veri tipinin hem pozitif hem de negatif değerleri tutabileceğini `signed` sıfatı belirtir. Örneğin 8 bitlik `char` veri tipinin en anlamlı olan 8. bitini işaret biti olarak kullanılmasını sağlar. Böylece -127 ile +128 arasındaki sayılar `signed char` olarak tanımlanabilir.
- Bir veri tipinin yalnızca negatif olmayan değerleri tutabileceğini `unsigned` sıfatı belirtir. Örneğin 8 bitlik `char` veri tipinin en anlamlı olan 8. bitini sayı olarak kullanılmasını sağlar. Böylece 0 ile 255 arasındaki sayılar `unsigned char` olarak tanımlanabilir.
- Tam sayı tipinin daha kısa bir sürümünü `short` belirtir. Yani 16 bit tam sayı tanımlamak için `short int` veri tipi kullanılabilir.
- Tam sayı tipinin daha uzun bir sürümünü `long` belirtir. Yani 32 bit tam sayı tanımlamak için `long int` veri tipi kullanılabilir.
- Bir değişkenin değerinin programcının hazırladığı mevcut kod dışında beklenmedik şekilde değişebileceğini ve derleyicinin bazı optimizasyonları yapmaması gerektiğini `volatile` sıfatı belirtir. Örnek olarak bir aygıt tarafından veya bir **kesme** (interrupt) tarafından bir değişken değiştirilebilir. Derleyiciye bir değişkenin değerinin kodu yazılmış program tarafından değil de başka şekillerde değiştirilebileceğini söyler.

```
/* Bu program işaretsiz tam sayı değişken tanımlamak için oluşturulmuştur.*/
int main() {
        unsigned char ogrenciYasi; /* veri tipi "unsigned char" ve kimliği "ogrenciYasi" olan
        değişken bildirimi. Bu değişken 0 ile 255 arasında bir değer alabilir. */
        unsigned short asansorunBulunduguKat;  /* veri tipi "unsigned short" ve kimliği
        "asansorunBulunduguKat" olan değişken bildirimi. Bu değişken, 0 ile 65.535 arasında
        değer alabilen bir değişkendir */
        unsigned long int pozitifTamsayi; /* pozitifTamsayi değişkeni 0 ile 4.294.967.295
         ↪  arasında değer alabilen bir değişkendir. */
        volatile int deviceStatus=0; /* Programcının yazacağı kod dışında bu değişken
         ↪  değiştirilebilir! */
}
```

Profesyonel yazılım mimarisi ve taşınabilir kod için, sabit bit sayısına sahip işaretsiz veri tiplerinin (`uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`) kullanılması şiddetle tavsiye edilir. Tam sayı veri tipleri 4.3.1 başlığında bu tiplere yer verilmiştir. Bu tipler, tüm platformlarda tam olarak doğru bit sayısını garanti eder.

4.4 Değişmezler

Programcı, kodlama yaparken bazı **sabitleri** (constant) ister istemez kullanır. Sabitler iki türlü tanımlanır. Bunların biri **değişmez** (literal), diğeri ise değeri değiştirilemeyen **sabit değişkenlerdir** (const variable). Derleme öncesinde de sonrasında da kaynak koddakinin kelimesi kelimesine aynısı olan **değişmezler** (literal) kullanılır. Değişkenlere verilen **ilk değerler** (initial value) ile **katsayılar** (coefficient) en çok kullanılan

değişmezlerdir.

> Değişkenler tanımlanırken sahip olacağı ilk değerleri değişmezler ile verme işlemine **başlatma** (initialize) adı verilir.

Aşağıda çeşitli değişkenleri başlatma örneği verilmiştir;

```
1 /* Bu program değişkenleri değişmez (literal) ile başlatma örneklerini içerir. */
2 int main() {
3         short i,j=0;
4         char c=65;
5         float f=2.5f;
6         char harf='A'; //char harf=65;
7         bool dogruMu=false;
8 }
```

Yukarıda verilen kod şu şekilde açıklanabilir;

- 3. Satırda; `short` veri tipindeki `i` değişkeni tanımlandı ve yine `short` veri tipindeki `j` kimlikli değişkene 0 tam sayı değişmezi (integer literal) ile başlatıldı.
- 4. Satırda; `char` veri tipindeki `c` kimlikli değişkende 65 tam sayı değişmezi ile başlatılmıştır.
- 5. Satırda; `float` veri tipindeki `f` kimlikli değişkene Türkçe yazılışı `2,5` olan kayan noktalı değişmezi (`f` harfi tek incelikli olduğunu ifade eder) ile başlatıldı. Kod İngilizce yazıldığından, kodun içerisindeki gerçek sayı değişmezleri için **ondalık ayracı nokta olarak yazılır**.
- 6. Satırda; `char` veri tipindeki `harf` kimlikli değişkene `'A'` karakter değişmezi (character literal) ile başlatıldı. **Karakter değişmezler tek tırnak içinde yazılır.** Klavyeden basılan her bir harf bellekte sayı olarak tutulur. `'A'` karakterinin kodu 65, `'B'` karakterinin kodu 66, ... Aşağıda detaylı açıklama verilmiştir.
- 7. Satırda; `bool` veri tipindeki `dogruMu` kimlikli değişkene `false` bool değişmezi (bool literal) ile başlatıldı.

Karakter kodları için genellikle ASCII (American Standard Code for Information Interchange) kümesi kullanılır ve 0 ile 127 arasındaki karakterleri içeren 7 bitlik bir karakter kümesidir. ANSI (American National Standards Institute) kümesi ise, 8 bitlik karakter kümeleri için kullanılır. Bu karakter kümeleri, değişmemiş ASCII karakter kümesini içerir. ANSI karakter kümesi, ek olarak, 128 ila 255 arasında ek karakterler içerir. Bunlar Batı Avrupa özel karakterleri ile Arapça, Yunanca veya Kiril karakterleri gibi karakterlerdir.

> Kodu `0` olan karakter ise **boş karakter** (NULL Character) olarak adlandırılır.

Aşağıda birçok veri tipi için değişmez (literal) örneği tablo olarak verilmiştir. Bunların birçoğu ilerleyen konularda anlatılacaktır. Bu tabloda değişmezler tanımlanırken her zaman büyük harf son ekleri (örneğin `L`

Kategori	Veri Tipi	Değişmez Örneği	Açıklama/Önek/Sonek
Tam Sayı	`int`	42	Varsayılan (onluk)
	`unsigned int`	`42u` veya `42U`	`u` veya `U` soneki
	`long`	`42l` veya `42L`	`l` veya `L` soneki
	`unsigned long`	`42ul` veya `42UL`	`ul` soneki
	`long long`	`42ll` veya `42LL`	`ll` veya `LL` (64-bit)
	İkili (binary)	`0b101010`	`0b` ön eki (C++14)
	Onaltılı (hexadecimal)	`0x2A`	`0x` ön eki
Kayan Noktalı	`double`	`3.1415`	Varsayılan (8 byte)
	`float`	`3.14f` veya `3.14F`	`f` veya `F`(4 byte)
	`long double`	`3.14l` veya `3.14L`	`l` veya `L` (12-16 byte)
Karakter	`char`	`'A'` veya `"ABC"`	Standart (UTF-8/ASCII)
	`wchar_t`	`L'A'` veya `L"ABC"`	`L` ön eki (Geniş Kartakter)
	`char8_t`	`u8"ABC"`	`u8` öneki (C++20)
	`char16_t`	`u'A'` veya `u"ABC"`	`u` ön eki (UTF-16)
	`char32_t`	`U'A'` veya `U"ABC"`	`U` ön eki (UTF-32)
Mantıksal	`bool`	`true`, `false`	Doğru veya Yanlış
Pointer	`std::nullptr_t`	`nullptr`	Boş gösterici (C++11)

Tablo 4.2: Tüm Veri Tipleri İçin Değişmez Örnekleri

veya LL) tercih edilir. Küçük l, bazı fontlarda bir rakamı ile karıştırılabildiği için modern yazım standartlarında büyük harf daha güvenlidir. Ayrıca ileride göreceğimiz, C++11 ile gelen **kullanıcı tanımlı değişmez** UDL (user defined literal) işleci (operator"") sayesinde 100.0_km gibi fiziksel birimleri koda ekleyebiliriz.

Aşağıdaki kod örneğinde programcı, sırasıyla 3, 3.14, 2.0 ve 3.14 olmak üzere dört farklı değişmez (literal) kullanılmıştır. Bu değişmezlerin her biri derleyici tarafından farklı olarak değerlendirilir ve çalışma anında (run-time) değiştirilemez.

```
/* Bu program, değişmez örneğidir. */
int main() {
        int yariCap=3;
        float daireninAlani=3.14*yariCap*yariCap;
        float daireninCevresi=2.0*3.14*yariCap;
}
```

Değişkenler tanımlanırken parantez içerişinde değişkeler verilerek de başlatılabilir. Aşağıda yeni nesil **başlatma** (initialize) örnekleri verilmiştir;

```
/* Bu program değişkenlere C++ tipi ilk değer verme ve değişmez örneklerini içerir. */
int main() {
        int tamsayi(61); /* "tamsayi"  kimlikli "int" veri tipindeki değişken tanımlandı.
        ve ilk değeri 61 değişmezi ile verildi */
        char enter(13), tab(8), buyukA('A'); /* "enter" kimlikli "char" veri tipindeki
        değişkene 13 değişmezi ile ilk değer verildi.  "tab" kimlikli "char" veri tipindeki
        değişkene 8 değişmezi ile ilk değer verildi. "buyukA" kimlikli "char" veri tipindeki
        değişkene 'A' karakter değişmezi ile ilk değer verildi. */
        double reel(2.5); /* "reel" kimlikli "double" veri tipindeki değişkene Türkçe 2,5
        değişmezi ile ilk değer verildi. Ondalık ayracı NOKTA olarak yazılır. */
        bool test(false); /* "test" kimlikli "bool" veri tipindeki değişkene
        false değişmezi ile ilk değer verildi. false yerine 0 true yerine 1 de yazılabilir.*/
}
```

4.5 Sabit Değişkenler

Değişmezlerin (literal) çokça kullanılması derleme sonrası üretilen icra edilebilir makine kodunu da büyütür. Bunun yerine çok kullanılan değişmezler bellekte bir kez yer kaplasın diye **const sıfatıyla** (const qualifier) **sabit değişkenler** (const variable) tanımlanabilir. Sabit değişkenler etik olarak büyük harfle kimliklendirilir. **Sabit değişkenlere de ilk değerlerle başlatılmalıdır (initialize)!**

```
/* Bu program, const sıfatı örneğidir. */
int main() {
        int yariCap=3;
        const float PI(3.14); // Sabit değişken tanımı!
        float daireninAlani=PI*yariCap*yariCap;
        float daireninCevresi=2.0*PI*yariCap;
}
```

Böylece aynı sabit PI sabit değişkeni birden çok yerde kullanılır ve her seferinde aynı bellek bölgesine erişildiğinden bellekten tasarruf edilmiş olunur. Tanımlama (definition) sırasında const sıfatı kullanılan değişkene, verilen ilk değer (initial value) değişmez (literal) olarak verilmelidir. **Daha sonra bu değişkene bir değer ataması olamaz!**

Bir değişkeni sabit tanımlamanın üstünlükleri aşağıda sıralanmıştır;

- **Gelişmiş Kod Okunabilirliği**: Bir değişkene const sıfatı vermek, diğer programcılara değerinin değiştirilmemesi gerektiğini belirtir; bu da kodun anlaşılmasını ve sürdürülmesini kolaylaştırır.
- **Gelişmiş Veri Tipi Güvenliği**: const kullanılması, değerlerin yanlışlıkla değiştirilmemesini sağlayabilir ve kodumuzda hata ve eksiklik olma olasılığını azaltabilir.
- **Gelişmiş Optimizasyon**: Derleyiciler, değerlerinin program yürütme sırasında değişmeyeceğini bildikleri için sabit değişkenlerini daha etkili bir şekilde optimize edebilirler. Bu, daha hızlı ve daha verimli kodla sonuçlanabilir.
- **Daha İyi Bellek Kullanımı**: Değişkenlere const sıfatı vermek, değerlerinin bir kopyasını oluşturmayı engeller; bu da bellek kullanımını azaltabilir ve performansı artırabilir.
- **Gelişmiş Uyumluluk**: Değişkenlere const sıfatı vermek, kodunuzu sabit değişkenleri kullanan diğer başlıklarla daha uyumlu hale getirebilir.
- **Gelişmiş Güvenilirlik**: const kullanarak, değerlerin beklenmedik şekilde değiştirilmemesini sağlayarak kodunuzu daha güvenilir hale getirebilir ve kodunuzdaki hata ve eksiklik riskini azaltabilirsiniz.

4.6 Sabit İfadeler

C++'ta sabit değişkenler gibi bir de **sabit ifadeler** (constant expression) vardır. Sabit ifadeler `constexpr` anahtar kelimesiyle tanımlanır. Bir değişkenin veya fonksiyonun değerinin **derleme zamanında** (compile-time) hesaplanması gerektiğini belirten bir anahtar kelimedir. C++11 ile gelmiş ve modern C++'ın en güçlü özelliklerinden biri olmuştur. Temel mantığı şudur: "Eğer bir hesaplamayı program çalışırken değil de derlenirken yapabiliyorsak, yapalım; Böylece program daha hızlı çalışır."

Sabit ifadelerin hesaplaması, derleme sırasında yapılıp sonuç doğrudan koda gömüldüğü için, çalışma anında (run-time) işlemci yorulmaz ve icra hızlanır. Değer derleme aşamasında belirlendiği için, hatalı bir durum varsa (örneğin sıfıra bölme) derleyici bunu size hemen söyler. C++'ta standart dizilerin boyutu sabit olmalıdır. `constexpr` bir değeri dizi boyutu olarak güvenle kullanabilirsiniz.

```
int x;
std::cin >> x;
const int sabit1 = x;      // DOĞRU: x çalışma anında belli olur.
// constexpr int sabit2 = x; // HATA: x derleme anında bilinmiyor!
constexpr int sabit3 = 10 * 5; // DOĞRU: Derleyici sonucu 50 olarak hesaplayıp koyar.
```

`const` ile `constexpr` anahtar kelimelerinin karşılaştırması verilmiştir;

Özellik	const	constexpr
Zamanlama	Çalışma zamanında (run-time) belirlenebilir.	Sadece derleme zamanında (compile-time) belirlenir.
Başlatma	Değişkenin değeri çalışma anında (run-time) (örnek olarak kullanıcı girişi ile) atanabilir.	Değer mutlaka derleme anında (compile-time) bilinen bir sabit olmalıdır.
Temel Amaç	Verinin değiştirilmesini engellemek (read-only).	Performans artışı ve derleme zamanı hesaplamaları.
Dizi Boyutu	Her zaman dizi boyutu olarak kullanılamaz (derleyiciye bağlı).	Güvenle dizi boyutu (array size) olarak kullanılabilir.
Fonksiyonlar	Fonksiyon dönüş değerleri her zaman sabit değildir.	Fonksiyonun sonucu derleme anında hesaplanabilir.

Tablo 4.3: C++ const ve constexpr Karşılaştırması

4.7 Ön İşlemci Makrolarıyla Değişmez Tanımı

Kod içerisinde değişmezleri (literal) tekrar tekrar yazmamanın bir yolu da `#define` **ön işlemci yönergesi** (preprocessor directive) ile **makro tanımlamaktır**. Ön işlemciler, kaynak kodları derlemeden önce derleyiciyi yönlendirerek kaynak kodlar üzerinde değişiklik yapmayı sağlar. Aşağıdaki tanımlanan `PI` makrosuyla derleyici, derlemeye geçmeden `PI` görülen yerleri `3.14` olarak değiştirmesi söylenir. Derleme bu ön işlemci işleminden sonra gerçekleşir. Makro kimlikleri sabit değişkenler gibi **büyük harfle kimliklendirilir**.

Ön işlemci makroları sonunda noktalı virgül karakteri olmaz!
Ancak, Talimatlar noktalı virgül karakteriyle biter!
Modern C++ da ön işlemci makroları sabit tanımlamak için Kullanılmaz!.

```
/* Bu program, #define ön işlemci yönergesi örneğidir. */
#define PI 3.14
/* Bu makro, derleme öncesinde PI görülen yerlere 3.14 yazılmasını sağlar. */
int main() {
        int yariCap(3);
        float daireninAlani=PI*yariCap*yariCap;
        float daireninCevresi=2.0*PI*yariCap;
}
```

Ön işlemci yönergesini yerine getiren derleyici aynı kodu aşağıdaki şekle çevirerek derleme işlemine geçer. Ayrıca ön işlemci yönergeleri yerine getirilmeden önce bütün açıklamalar ve **beyaz boşluklar** (white space) kaynak koddan kaldırılır. Bu durumda ön işlemci yönergeleri sonrasında bir üstte verilen kaynak kodumuz aşağıdaki hale döner;

```
int main(){int yariCap=3;float daireninAlani=3.14*yariCap*yariCap;float
↪ daireninCevresi=2.0*3.14*yariCap;}
```

`#define` ön işlemci yönergeleri ile çok tekrarlanan değişmezler için makro tanımlanabildiği gibi aşağıdaki

örnek verildiği şekilde kod tasnifi için de makro kullanılabilir;

```
/* Bu program, #define ön işlemci yönergesinin bir başka kullanım örneğidir. */
#define BEGIN int main() {
#define END }
#define PI 3.14
BEGIN
const char ILKHARF='A';
const float AVAGADROSAYISI=6.022e23;
float yarimRadyan=PI/2;
END
```

4.8 Yazdığımız Kod Nasıl İcra Edilir?

Derleyici tarafından işlemcinin anlayacağı makine diline çevrilen programımız aslında **yazdığımız talimatları (statement) sırasıyla çalıştırır.** C++ dilinde her talimat noktalı virgül ile bittiği 4.2.2 başlığında anlatılmıştı. Aşağıdaki örnek programı göz önüne alalım;

```
1 int main() {
2         float boy(1.80);
3         float kilo(100.0);
4         float bki;
5         float boyKare;
6         boyKare=boy*boy;
7         bki=kilo/boyKare;
8         boy=1.50;
9 }
```

Yapısal programlamada program, ana fonksiyondan başlar. Bu nedenle icra, `main()` fonksiyonu içindeki ilk talimatla başlar ve aşağıdaki sırayla icra edilir;

1. İcra edilecek ilk talimat, `float boy(1.80);` talimatıdır. (2. Satır): `boy` Değişkenine bellek tahsis edilerek değişkene `1.80` değeri atanır. Yani `boy` değişkeni `1.80` değeriyle **başlatılır** (initialize).
2. Sonra `float kilo(100.0)` talimatı icra edilir (3. Satır): `kilo` Değişkeni `100.0` ile başlatılır.
3. Daha sonra `float bki;` talimatı icra edilir (4. Satır): tanımlanan `bki` değişkenine sadece bellek tahsis edilir ve değeri belli değildir. Bazı derleyiciler değerleri sıfır olarak varsayar.
4. Daha sonra `float boyKare;` talimatı icra edilir (5. Satır): Tanımlanan `boyKare` değişkenine sadece bellek tahsis edilir ve değeri belli değildir.
5. Daha sonra `boyKare=boy*boy;` icra edilir (6. Satır): Önceden tanımlanmış `boyKare` değişkenine `boy` değişkeni kendisiyle çaarpılarak işlem sonucu atanır. `boyKare` değişkeni öncesinde tanımlanmamış olsaydı derleme hatası alacaktık.
6. Daha sonra `bki=kilo/boyKare;` talimatı icra edilir (7. Satır): `kilo` Değişkenini `boyKare` değişkenine böler ve çıkan sonucu `bki` değişkenine atar.
7. Daha sonra `boy=1.50` talimatı icra edilir (8. Satır): `boy` Değişkenine `1.50` atanarak değeri değiştirilir.

4.9 İşleçler

İşleçler (operator) matematikten bildiğimiz işleçlerdir.

$$y = 3 + 2$$

Yukarıda verilen cebirsel ifadede toplama işleci (`+`) iki tarafına bulunan argümanları toplar. Bu argümanlara **işlenen** (operand) adı verilir. Yüksek düzey dillerde de aritmetik işleçlerin yanında birçok işleç de bulunur.

> En çok kullanılan işleç, atama (`=`) işlecidir.
> **Atama işleci, sağdaki işleneni soldakine atar.**
> Bu nedenle kodlama yapılırken `2+2=x` veya `a+b=x` yazılamaz!

§4.9.1 Aritmetik İşleçler

C++ programlama dilinde, C dilinde olduğu gibi, aritmetik işleçlerin çalışma şekli **işlenenlerin** (operand) veri tipine göre değişir. Aritmetik işlemler ve atama işleci için kurallar;

1. **İşlenenlerden birinin bellekte kapladığı yer küçük ise ikisi aynı olacak şekilde bellekte daha fazla yer kaplayanına dönüştürülür** ve işlem sonra yapılır ve sonuç dönüştürülen veri tipinde üretilir. Örnek olarak `char` veri tipindeki değişken ile `long` veri tipindeki bir değişken toplanmaya çalışıldığı zaman

`char` veri tipindeki değişkenin değeri `long` veri tipine otomatik dönüştürülür. Sonuç `long` veri tipinde üretilir.

2. **İşlenenlerden biri tam sayı diğeri kayan noktalı sayı ise işlem yapılmadan önce işlenenler kayan noktalı sayıya dönüştürülür.** Örneğin `int` veri tipindeki değişken ile `float` veri tipindeki bir değişken toplanmaya çalışıldığı zaman `int` veri tipindeki değişkenin değeri `float` veri tipine otomatik dönüştürülür. Sonuç `float` veri tipinde üretilir.

İşleç	İşlenenler üzerinde yapılan işlem
+	Sol ve sağındaki yer alan işlenenleri toplar.
-	Soldaki işlenenden sağdakini çıkarır.
*	Sol ve sağındaki yer alan işlenenleri çarpar.
/	Soldaki işleneni sağdakine böler. Eğer işlenenlerin her iki de tam sayı ise sadece kalan kısmı atılır. Bu durumda sonuç yine tam sayı olarak üretilir.
%	Kalan (modulus) işleci, sadece tam sayıları işlenen olarak kabul eder. Soldaki işlenenin sağdakine bölümünden kalanı verir. Kalan yine tam sayıdır.

Tablo 4.4: Aritmetik İşleçler

Aritmetik işleçlere (aritmetic operator) ilişkin örneği aşağıdaki görebilirsiniz;

```
/* Bu program, aritmetik işleç örneğidir. */
int main() {
        int a = 9, b = 4, res;
        float fa=6.6, fb= 2.2, fres;
        res = a + b; // işlem sonucu res=13
        res = a - b; // işlem sonucu res=5
        res = a * b; // işlem sonucu res=36
        res = a / b; // işlem sonucu res=9/4=2 tam 1/4=2
        /* Bölme işlecinin işlenenleri tam sayı olduğunda bölme sonucundaki tam sayı olarak
         ↪ hesaplanır. Bölme işleminin tam kısmını içeren sonuç döndürlür. */
        res = a / 4.3 ;  /* res tam sayı olduğundan işlem sonucu; res=9/4.3 =9.0/4.3 =2.0930 =2
         ↪ tam 0.0930 =2 */
        res = a % b;   // işlem sonucu res=9%4= 2 tam 1/4=1
        fres= fa / fb; // işlem sonucu res=6.6/2.2=3.00
}
```

C++ programlama dilinde tam sayı bölme işlemi işlemcinin en basit yetenekleriyle çözülür. Bu nedenle diğer programlama dillerinden ayrılır. Aşağıda buna ilişkin örnek verilmiştir.

```
/* Bu program, aritmetik işleç BÖLME örneğidir. */
int main () {
        int a = 19 / 2 ; // a = 2*9+1 = 9
        int b = 18 / 2 ; // b = 9
        int c = 255 / 4; // c = 4*63+3 = 63
        int d = 45 / 4 ; // d = 4*11+1 = 11
        double aa = 19 / 2.0 ; // aa = 9.5
        double bb = 18.0 / 2 ; // bb = 9.0
        double cc = 255 / 2.0; // cc = 127.5
        double dd = 45.0 / 4 ; // dd = 11.25
        double ee = 45 / 4 ; /* ee = 11 çünkü bölme tam sayılar arasında yapılmıştır */
}
```

§4.9.2 Tekli İşleçler

Tekli işleçler (unary operator) sadece bir işlenen üzerinde işlem yapar. Tekli demek tek bir işlenen (operand) ile işlem yapması anlamındadır.

Aşağıda tekli işleçlerin kullanımına ilişkin örnek program verilmiştir;

```
/* Bu program, tekli işleç örneğidir. */
int main() {
    int a = 5, b = 5, res;
        // Tekli eksi (unary minus)
        res = -a;
        // Önce-artırım (Pre-increment)
        res = ++a + 2;
        /* a önce 6 olur, sonra 2 ile toplanır. res=8, a=6 */
```

İşleç	İşlenen üzerinde yapılan işlem
`-`	Kendisinden sonra gelen işlenenin işaretini değiştirir. İşlenen pozitif ise negatif yapar, negatif ise pozitif yapar.
`+`	Kendisinden sonra gelen işlenenin işaretini pozitif yapar.
`++`	Artırım (increment) işleci, işlenenin sağında veya solunda kullanılabilir.
`--`	Eksiltme (decrement) işleci, işlenenin sağında veya solunda kullanılabilir.
`!`	Mantıksal DEĞİL işleci işlenenden önce kullanılır. İşleneninin mantıksal durumunu tersine çevirmek için kullanılır.
`sizeof()`	Bu işleç, kendisinden sonra gelen işlenenin bellek boyutunu bayt sayısı olarak `size_t` tipinde döndürür. Ayrıca bir veri tipinin (data type) bellek miktarını argüman olarak vererek öğreniriz.
`&`	Adres işleci (address operator), kendisinden sonra gelen işlenenin bellek adresini döndürür.

Tablo 4.5: Tekli İşleçler

```
        // Sonra-artırım (Post-increment)
        res = 2 + b++;
        /* İşlem b=5 ile yapılır (res=7), sonra b 6 olur. */
        // Değerleri sıfırlayalım
        a = 5; b = 5;
        // Önce-azaltım (Pre-decrement)
        res = --a + 2;
        /* a önce 4 olur, sonra 2 eklenir. res=6, a=4 */
        // Sonra-azaltım (Post-decrement)
        res = 2 + b--;
        /* İşlem b=5 ile yapılır (res=7), sonra b 4 olur. */
        // Mantıksal DEĞİL (Logical NOT)
        //C Dilinden gelen
        a = 1; b = 0;
        res = !a; // res = 0 (false)
        res = !b; // res = 1 (true)
        //C++ yöntemiyle
        bool ba=true, bb=false, bres;
        bres = !ba; // res = false
        bres = !bb; // res = true
        // sizeof işleci
        res = sizeof a; //4
        res = sizeof(char); //1
}
```

§4.9.3 İlişkisel İşleçler

İlişkisel işleçler (relational operator), işlenenlerin arasındaki ilişkiyi verirler. C++ dilinde, doğru (true) ya da yanlış (false) olabilen **bool veri tipinde değeri** döndürürler.

İşleç	İşlenenler üzerinde yapılan işlem
`==`	Eşit mi? İşleci: İki işlenenin değeri birbirine eşit ise `true`, değilse `false` verir
`!=`	Farklı mı? İşleci: İki işlenenin değeri birbirinden farklı ise `true`, değilse `false` verir
`>`	Büyük mü? İşleci: soldaki işlenenin değeri soldakinden fazla ise `true`, değilse `false` verir
`<`	Küçük mü? İşleci: soldaki işlenenin değeri soldakinden az ise `true`, değilse `false` verir
`>=`	Büyük veya eşit mi? İşleci: soldaki işlenenin değeri soldakinden fazla ya da iki işlenen eşit ise `true`, değilse `false` verir
`<=`	Küçük veya eşit mi? İşleci: soldaki işlenenin değeri soldakinden az ya da iki işlenen eşit ise `true`, değilse `false` verir
`<=>`	C++20 ile hayatımıza girmiştir. Şekilden ötürü **uzay gemisi** (spaceship operator) işleci olarak adlandırılır. Bu operatör tek bir işlemle küçüktür, eşittir veya büyüktür durumunu döndürür. İleride anlatılacaktır.

Tablo 4.6: İlişkisel İşleçler

Aşağıda verilen ilişkisel işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
/* Bu program, ilişkisel işleç örneğidir. */
int main() {
    int a = 5, b = 6;
        bool res;
        res = a<b;  // işlem sonucu res=true
        res = a<=b; // işlem sonucu res=true
        res = a>b;  // işlem sonucu res=false
        res = a>=b; // işlem sonucu res=false
        res = a==b; // işlem sonucu res=false
        res = a!=b; // işlem sonucu res=true
}
```

§4.9.4 Bit Düzeyi İşleçler

Bit düzeyi işleçler (bitwise operator) özellikle ikilik tabandaki **tam sayılarda karşılıklı bitler üzerinde yapılan işlemlerdir**. Aslında bu işlemler, işlemci üzerinde çarpma, toplama, bölme ve çıkarma işlemleri için kullanılır. Aşağıda tam sayılar üzerinde yapılan bit düzeyi işlemler gösterilmiştir;

```
/* Bu program, bit düzeyi işleç örneğidir. */
int main() {
        char a = 00000110; // 6
        char b = 00000010; // 2
        char bitwise_and = a & b; // 00000010
        char bitwise_or = a | b;  // 00000110
        char bitwise_xor = a ^ b; // 00000100
        char bitwise_not = ~b;    // 00001101
        char left_shift = b << 2; // 00000100
        char right_shift = b >> 1;// 00000001
}
```

A ve B iki tam sayının karşılıklı bitleri olmak üzere bit düzeyi VE, VEYA, ÖZEL VEYA ve DEĞİL işlemlerinin doğruluk tabloları aşağıda verilmiştir;

A	B	A & B	A | B	A ^ B	~ A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Tablo 4.7: VE, VEYA, ÖZEL VEYA ve DEĞİL İşlemlerinin Doğruluk Tabloları

İşleç	İşlenenler üzerinde yapılan işlem
&	Bit düzeyi VE (AND): Tam sayının karşılıklı bitleri VE işlemine tabi tutulur.
|	Bit düzeyi VEYA (OR): Tam sayının karşılıklı bitleri VEYA işlemine tabi tutulur.
^	Bit düzeyi ÖZEL VEYA (XOR): Tam sayının karşılıklı bitleri ÖZEL VEYA işlemine tabi tutulur.
~	Bit düzeyi DEĞİL (NOT): Tekli işleç olup sonrasında gelecek sayının bitleri DEĞİL işlemine tabi tutulur.
<<	Bit düzeyi sola kaydırma: Tekli işleç olup sonrasında gelecek sayının bitlerini bir bit sola kaydırır. En anlamlı bit taşar. Taşan bu bit, işlemcinin durum kaydedicisinde taşma bayrağına taşınır.
>>	Bit düzeyi sağa kaydırma: Tekli işleç olup sonrasında gelecek sayının bitlerini bir bit sağa kaydırır. En anlamsız bit taşar. Taşan bu bit, işlemcinin durum kaydedicisinde taşma bayrağına taşınır.

Tablo 4.8: Bit Düzeyi İşleçler

§4.9.5 Mantıksal İşleçler

Mantıksal ifadeler, doğru (true) ve yanlış (false) olabilen ifadelerdir. C++ dilinde sıfırdan farklı bir tam sayı ifade doğru (`true`), sıfır olan tam sayı yanlış (`false`) olarak değerlendirilir. **Mantıksal işleçlere** (logical operator) işlenen olarak giren ifadeler öncelikle `bool` veri tipine dönüştürülür. İşlenenler A ve B olmak üzere, mantıksal işleçlerin doğruluk tablosu aşağıda verilmiştir;

Bu işleçlerin çalışma şekli doğruluk tablosundaki gibidir ancak derleyici optimizasyonu gereği, ifadenin

A	B	A && B (VE)	A | | B (VEYA)	!A (DEĞİL)
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

Tablo 4.9: Mantıksal İfadelerin Doğruluk Tablosu

bir kısmı işlenmediğinden, **şartlı olarak çalışırlar**.

İşleç	İşlenenler üzerinde yapılan işlem
&&	**Şartlı** Mantıksal VE: Eğer iki işlenen sıfırdan farklı ise 1 verir. **Soldaki işlenen yanlış (false) ise sağdakine bakılmaz.**
||	**Şartlı** Mantıksal VEYA: Eğer iki işlenenden biri sıfırdan farklı ise 1 verir. **Soldaki işlenen doğru (true) ise sağdakine bakılmaz.**
!	Mantıksal DEĞİL: İşlenenden önce kullanılır. Tekli işleçlerde verilmiştir. İşlenenin mantıksal durumunu tersine çevirmek için kullanılır. Eğer işlenen sıfırdan farklı ise 0, değilse 1 verir.

Tablo 4.10: Mantıksal İşleçler

Aşağıda verilen ilişkisel ve mantıksal işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
/* Bu program, mantıksal işleç örneğidir. */
int main() {
        int a = 5, b = 6;
        bool res;
        res = a&&b; // işlem sonucu res=true
        res = a||b; // işlem sonucu res=true
        res = !a;   // işlem sonucu res=false
}
```

§4.9.6 Atama işleçleri

Atama işleçleri (assignment operator) en çok kullanılan işleçlerdir. **Atama işleçleri her zaman sağdaki değeri sola atar!** Dolayısıyla soldaki işlenen, atama yapılacak uygun veri tipinde bir değişken (variable) olmak zorundadır. Aşağıda verilen atama işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

İşleç	İşlenenler üzerinde yapılan işlem
=	Sağdaki işleneni sola atar.
+=	Soldaki işleneni sağdaki işlenen üzerine toplar ve sonucu sola atar.
-=	Soldaki işlenenden sağdaki işleneni çıkarır ve sonucu sola atar.
*=	Soldaki işleneni sağdaki işlenenle çarpar ve sonucu sola atar.
/=	Soldaki işleneni sağdaki işlenene böler ve sonucu sola atar. Eğer işlenenler tam sayı ise kesirli kısım göz ardı edilir. Bu durumda sonuç tam sayıdır.
%=	Bu işleç tam sayılarla çalışır. Soldaki işleneni sağdaki işlenene böler ve kalanı sola atar.

Tablo 4.11: Atama İşleçleri

```
/* Bu program, atama işleçleri örneğidir. */
int main() {
        int a = 10; // a değişkenine 10 değişmezi = işleciyle atanıyor.
        a += 10; // a=a+10; ile eşdeğer: a değişkenine 10 ekleniyor.
        a -= 10; // a=a-10; ile eşdeğer: a değişkeninden 10 çıkarılıyor.
        a *= 10; // a=a*10; ile eşdeğer: a değişkeni 10 kat yapılıyor.
        a /= 10; // a=a/10; ile eşdeğer: a değişkeni 1/10 kat yapılıyor.
        a %= 10; // a=a%10; ile eşdeğer: a değişkeninin 10 a kalanı bulunup ona eşitleniyor.
}
```

§4.9.7 İşleç Öncelikleri

Cebirsel ifadelerde nasıl ki önce çarpma ve bölme sonra toplama işlemleri yapılıyor. C programlama dilinde yazılan ifadelerde de işleç öncelikleri (operator precedence) vardır. İfadelerde bir işlemi öncelikli hale getirmek için parantez içerisine alınır. En içteki parantez en öncelikli olarak gerçekleştirilir.

Başka işleçler de bulunmaktadır bunlar ilerleyen bölümlerde yeri geldikçe anlatılacaktır. Aşağıda işleç-

Öncelik	İşleç Grubu	İşleçler	Gruptaki Öncelik
1	Sonek (postfix)	`() ++ --`	→
2	Tekli (unary)	`+ - ! ++ -- sizeof()`	←
3	Çarpım	`* / %`	→
4	Toplam	`+ -`	→
5	Bit Düzeyi Kaydırma	`<< >>`	→
6	İlişkisel (relational)	`< <= > >=`	→
7	Eşitlik (equality)	`== !=`	→
8	Bit düzeyi VE	`&`	→
9	Bit düzeyi ÖZEL VEYA	`^`	→
10	Bit Düzeyi VEYA	`|`	→
11	Mantıksal VE	`&&`	→
12	Mantıksal VEYA	`||`	→
13	Atama (assignment)	`= += -= *= /= %= <<= >>= &= ^= |=`	←

Tablo 4.12: İşleç Öncelikleri

lerin önceliklerine ilişkin örnek programı lütfen inceleyiniz.

```
/* Bu program, işleç öncelikleri örneğidir. */
int main() {
        int a=10, b=5;
        a= a-b+2; // Öncelik Sırası +, -, = İşlem sonunda a=7
        a= a/b+2; // Öncelik Sırası /, +, = İşlem Sonunda a=7/5+2=3
        a= 2*10-20/2+3+(12-10); // Öncelik Sırası: (-) * / - + +
        /* İfadenin çözüm sırası:
           > 2*10-20/2+3+2
           > 20-20/2+3+2
           > 20-10+3+2
           > 10+3+2
           > 13+2
           > 15 */
}
```

4.10 Kayan Noktalı Sayı Hesapları

Kayan noktalı sayıların IEEE-754 standardında ikili tabanda tutulduğu 2.4 başlığında anlatılmıştı. Kodlarken onluk tabanda değerler verdiğimiz kayan noktalı sayılar, aslında arka planda ikilik tabanda tutulur. Bu durum yazdığımız kodlarda garip davranışların ortaya çıkmasına sebep olur. Buna örnek olarak; hemen hemen her programcının yaptığı ilk hata, aşağıdaki kodun amaçlandığı gibi çalışacağını varsaymaktır;

```
float total = 0.0f;
for(float a = 0.0; a != 2; a += 0.01f)
    total += a;
```

Acemi programcı bunun `0`, `0.01`, `0.02`, `0.03`, gibi artarak, `1.97`, `1.98`, `1.99` aralığındaki her bir sayıyı toplayacağını ve 199 sonucunu, yani matematiksel olarak doğru cevabı vereceğini varsayar. Bu kodda doğru yürümeyen iki şey olur: Birincisi yazıldığı haliyle program asla sonuçlanmaz. `a` asla `2`'ye eşit olmaz ve döngü asla sonlanmaz. İkincisi ise döngü mantığını `a<2` şartını kontrol edecek şekilde yeniden yazarsak, döngü sonlanır, ancak toplam `199`'dan farklı bir şey olur. IEEE-754 uyumlu işlemlerde, genellikle bunun yerine yaklaşık `201`'e ulaşır. Bunun olmasının nedeni, kayan noktalı sayılarının onluk tabanda kendilerine atanan değerlerin, ikilik tabanda yaklaşık değerlerini temsil etmesidir. Bu duruma aşağıdaki klasik örnek verilir;

```
#include <iostream>
int main() {
        double a = 0.1;
        double b = 0.2;
        double c = 0.3;
        if(a + b == c)
                std::cout<<"Bu satır İcra Edilmez!"; //IEEE754 standardında işlem yaparsak.
        else
                std::cout<<"Kayan Noktalı Sayılar İkili Sayı Sisteminde Tutulduğundan Yaklaşık
                ↪ Olarak Hesaplanır." ;
}
```

Programcı olarak gördüğümüz şey onluk tabanda yazılmış üç sayı olsa da derleyicinin (ve altta yatan donanımın) gördüğü şey ikili sayılardır.`0.1`, `0.2` ve `0.3`, ona mükemmel bölmeyi gerektirdiğinden (ki bu onluk sayı sisteminde oldukça kolaydır), ancak ikili sayı sisteminde imkansızdır (bu sayılar, onluk sayı sistemindeki `1/3` sayısı gibi `0.333333333333333...` şeklinde kesin olmayan biçimde depolanması gerektiğindendir);

```
int main()
{
        /*64 bitlik kayan noktalı sayılar, tam sayı kısmı dahil olmak üzere 53 basamaklı
         ↪  hassasiyete sahiptir. Aşağıda ikili olarak değişmezler tanımlanmıştır.*/
        double a = 0011111110111001100110011001100110011001100110011001100110011010;
        //onluk sistemdeki 0.1 ikili sistemde kusurlu olarak saklanır
        double b = 0011111110010011001100110011001100110011001100110011001100110011010;
        //onluk sistemdeki 0.2 ikili sistemde kusurlu olarak saklanır
        double c = 0011111110100110011001100110011001100110011001100110011001100110011;
        //onluk sistemdeki 0.3 ikili sistemde kusurlu olarak saklanır
        double a + b = 0011111110100110011001100110011001100110011001100110011001100110100;
        /*c değişkeni ile a+b nin onluk sistemde 0,3'e tam olarak eşit olmadığını unutmayın! */
}
```

4.11 Düşün ve Çöz

- **Düşün**: C dilindeki anahtar kelimelerin (keywords) niçin değişken ismi olarak kullanılamayacağını dilin söz dizimi kuralları açısından değerlendiriniz.
- **Çöz**: `a = 5`, `b = 10`, `c = 15` değerleri için `a + b * c / a - b % 3` ifadesinin sonucunu işlem öncelik sırasına göre adım adım hesaplayan bir program parçacığı yazınız.
- **Düşün**: `x = x + 1` ile `x++` ifadeleri arasında performans açısından (assembly seviyesinde) bir fark var mıdır? Araştırınız.
- **Çöz**: Bit düzeyi (bitwise) işleçleri kullanarak, bir sayının 2'nin tam kuvveti olup olmadığını bulan bir C ifadesi yazınız.
- **Düşün**: Mantıksal VE (`&&`) operatöründe kısa devre (short-circuit) değerlendirmesi nedir? İlk koşul yanlış olduğunda ikincisine bakılmaması neden önemlidir?
- **Çöz**: Virgül operatörünün (`a = (b=3, b+2)`) çalışma mantığını analiz ediniz ve a ile b'nin son değerlerini tahmin ediniz.

5. Bölüm

Giriş Çıkış İşlemleri

5.1 Modüler Programlama

C++ programlama dili, C programlama dili üzerine eklenti olarak geliştirildiğinden, yapısal olmasının yanında aynı zamanda modüler bir programlama dilidir. **Modüler programlamada** (modular programming) yazılan fonksiyonlar gruplar halinde başka kod dosyalarına konulur ve bu bileşenler gerektikçe projemize `#include` **ön işlemci yönergeleriyle** (preprocessor directive) dahil edilir.

Modüllere ayırmanın **soyutlama** (abstraction), **değişim yönetimi** (change management) ve **yeniden kullanım** (reusing) gibi üstünlükleri vardır. Modüllere ayrılan bir kodun bakımı da daha kolaydır. Ancak daha fazla fonksiyonun çağrılması, işlemciyi yorar. Çünkü her fonksiyon çağrısında işlemcinin mevcut durumu ve kaydediciler belleğe itilir ve fonksiyondan geri dönüldüğünde eski duruma dönmek içi bu veriler tekrar geri çekilir. İşlemciye yüklenen bu **çağırma yükü** (calling overhead) günümüz bilgisayarlarında oldukça ihmal edilebilir seviyededir. Ancak mili saniyeler bazında yapılması gereken zaman kritik işler gerçekleştirmek için bu durum göz önüne alınmalıdır.

C++ programlama dilinde en çok kullanacağımız modüllerden biri de standart giriş-çıkış yani **IO** (input-output) işlemlerinin tanımlı olduğu `<iostream>` başlık (header) dosyasıdır. Bu modülü kodumuza **dahil etmek** (include) için kaynak kodun başına `#include` ön işlemci yönergesini (preprocessor directive) aşağıdaki şekilde ekleriz.

```
#include <iostream>
```

Böylece **konsola** bir şey yazmak için `std::cout` nesnesi, klavyeden bir şey okumak ve bir değişkene atamak için ise `std::cin` nesnelerini kullanabilir hale geliriz. **Konsol** (console) kelimesi UNIX/Linux işletim sitemindeki terminal, Windows işletim sisteminde ise komut istemini (command prompt) ifade eder.

C++ programlama dilinde her işletim siteminde de çalışan standart birçok başlık dosyası bulunmaktadır. Ancak bunların dışında, işletim sistemine özel olan başlık dosyaları da bulunmaktadır;

Başlık Dosyası	Açıklama
`<iostream>`	Konsol (`std::cout`) ve klavye (`std::cin`) gibi **akış** (stream) nesneleri ile giriş ve çıkış işlemleri ve `std::endl` gibi bildirimleri içerir.
`<cmath>`	`sqrt()` , `log2()` , `pow()` gibi matematiksel işlemleri yapmak için kullanılan sınıfı içerir.
`<cstdlib>`	Sistemle ilgili `exit()` ve `rand()` gibi işlevleri içerir.
`<vector>`	Dinamik diziler olan vektör (std::vector) **konteyneri** (container) tanımlarını içerir.
`<string>`	Metin işleme için (`std::string`) şablon sınıf olup işlevler sağlar
`<iomanip>`	Giriş çıkış işlemlerinde değişkenlerdeki ondalık basamak sayısını sınırlamak için `set()` ve `setprecision()` fonksiyonlarına erişmek için kullanılır.
`<cerrno>`	`errno()`, `strerror()`, `perror()` gibi istisna işleme işlemlerini gerçekleştirmek için kullanılır.
`<time>`	`setdate()` ve `getdate()` gibi `date()` ve `time()` ile ilgili fonksiyonlarla işlem yapmak için kullanılır. Ayrıca sistem tarihini değiştirmek ve CPU zamanını almak için de kullanılır.

Tablo 5.1: En Çok Kullanılan Başlık Dosyaları

5.2 Klavyeden Veri Okuma ve Konsola Veri Yazma

C++ dilinde standart giriş-çıkış (input-output) için iki sınıf bulunur. İleride detayını vereceğimiz akışları, C dilindeki dosyalar olarak düşünebiliriz. Konsola veri yazmak için;

1. `std::cout` nesnesi, varsayılan olarak konsol ile ilişkilendirilen standart çıkış akışı (output stream) nesnesidir.
2. `std::cout` nesnesi gibi çıktı olarak kullanılan akışlara verileri göndermek için **ekleme işleci** (insertion operator) (`<<`) kullanılır. Konsolda imlecin (cursor) yeri veriyi yazdıktan sonraki konuma taşınır.

3. Çıktı olarak kullanılan konsolda imlece (cursor) satır başı için `std::endl` değişmezini kullanırız. Aslında bu klavyedeki yeni satır (new line) karakterine ilişkin değişmezdir. Bu değişmez, Windows işletim sisteminde `'\n'` karakteridir. UNIX/Linux işletim sistemlerinde ise `'\n'+'\r'` karakterleridir.

Aşağıdaki örneğe geçmeden önce metin değişmezlerin (string literal) çift tırnak arasında yazıldığını unutmayalım. Aslında metinler her biri `char` değişkeni dizileridir. İleride bu konu anlatılacaktır. Aşağıdaki örnekte yer alan `"Yaşınız:"` ve `"Ağırlığınız:"` metin değişmezlerdir.

```
#include <iostream>
int main() {
        int yas(35);
        float agirlik(100.0f);
        std::cout << "Yaşınız:" << yas << std::endl;
        std::cout << "Ağırlığınız:"<< agirlik << std::endl;
}
/*Program Çıktısı:
Yaşınız:35
Ağırlığınız:100
*/
```

Klavyeden bir şeyler okumak için;

1. `std::cin` nesnesi, varsayılan olarak klavye ile ilişkilendirilen standart giriş akışı (input stream) nesnesidir.
2. `std::cin` nesnesi gibi girdi olarak kullanılan akışlardan veri almak için **ayıklama işleci** (extraction operator) (`>>`) kullanılır.

```
#include <iostream>
int main() {
        int yas;
        float agirlik;
        std::cout << "Yaş Giriniz:";
        std::cin >> yas; //klavyeden girilen verilerden yas tam sayısı ayıklanıyor (extraction)
        std::cout << "Ağırlık Giriniz:";
        std::cin >> agirlik; //klavyeden girilen verilerden agirlik kayan noktalı sayısı
         ↪  ayıklanıyor (extraction)
        std::cout << "Girilen Yaş:"<< yas << " ve Ağırlık:" << agirlik << std::endl;
}
/* Program çalıştırıldığında:
Yaş Giriniz:12
Ağırlık Giriniz:75.6
Girilen Yaş:12 ve Ağırlık:75.6
*/
```

Koda dahil edilen `<iostream>` başlığı birçok nesne ve bildirim içerir. Bunlardan standart giriş nesnesi olan `std::cin` ve standart çıkış nesnesi olan `std::cout` nesnesi `std` adlı **isim uzayında** (namespace) yer alırlar. Kodlama yapılırken bu isim uzayını sürekli olarak `std::` şeklinde yazmamak için `using namespace std;` yönergesi eklenebilir. İki nokta üst üste karakteri olan **kapsam çözümleme işleci** (scope resolution operator) (`::`) bunun için kullanılır. İsim uzayları sonradan anlatılacaktır. Bu durumda aynı kod aşağıdaki gibi kısaltılabilir;

```
#include <iostream>
using namespace std;
int main() {
        int yas;
        float agirlik;
        cout << "Yaş Giriniz:";
        cin >> yas;
        cout << "Ağırlık Giriniz:";
        cin >> agirlik;
        cout << "Girilen Yaş:"<< yas << " ve Ağırlık:" << agirlik<<endl;
}
```

Örnek olarak yarıçapı gerçek sayı olarak klavyeden alınan bir çemberin ve çevresini hesaplayan program aşağıda verilmiştir.

```
#include <iostream>
#define PI 3.1415
```

```
using namespace std;
int main() {
        float cemberinCevresi;
        cout << "Çemberin Çevresini Giriniz:";
        cin >> cemberinCevresi;
        float cemberinYariCapi=cemberinCevresi/2/PI;
        cout << "Çevresini Girdiğiniz Çemberin Yarıçapı:" << cemberinYariCapi;
}
```

5.3 Giriş ve Çıkışı Biçimlendirme

C++ dilinde `<iomanip>` başlığı `std::cin` ve `std::cout` ile ileride göreceğimiz akış nesnelerinde girdi ve çıktıyı biçimlendirebiliriz. Bunun için **manipülatörler** (manipulator) kullanılır. Aşağıdaki örnekte çıktıya gönderilecek hane sayısı `std::setprecision()` ile belirleniyor;

```
#include <iostream>
#include <iomanip>
#include <cmath>
#include <limits>
int main() {
        const long double pi = std::acos(-1.L);
        std::cout << "default precision (6): " << pi << std::endl
                << "std::precision(10): " << std::setprecision(10) << pi << std::endl
                << "max precision: "
                << std::setprecision(std::numeric_limits<long double>::digits10 + 1)
                << pi << std::endl;
}
/*Program Çıktısı:
default precision (6): 3.14159
std::precision(10): 3.141592654
max precision: 3.141592653589793239
*/
```

Aşağıda `std::setw()` ile yazdırılacak genişlik ve `std::setfill()` ile doldurulacak karakter örneği verilmiştir;

```
#include <iostream>
#include <iomanip>
int main() {
        int val(10);
        std::cout << ">" << val << "<" << std::endl;
        std::cout << ">" << std::setw(10)  << val << "<" << std::endl;
        val=350;
        std::cout << "default fill: " << ">"<<  std::setw(10)  << val << "<" << std::endl;
        std::cout << "setfill('*'): " << ">"  <<  std::setfill('*') << std::setw(10) << val <<
         ↪  "<" << std::endl;
}
/* Program Çıktısı:
>10<
>        10<
default fill: >       350<
setfill('*'): >*******350<
*/
```

Giriş çıkış işlemlerinde `setiosflags(mask)` kullanıldığında akışın tüm biçim bayraklarını maskenin belirttiği şekilde dışarı veya içeri ayarlar. Tüm `std::ios_base::fmtflags` listesi aşağıda verilmiştir;

Ayrıca `basefield`, `dec|oct|hex|0` maskeleme işlemleri için, `adjustfield`, `left|right|in` hizalama için ve `scientific|fixed|(scientific|fixed)|0` maskeleme işlemleri için kullanışlıdır. Aşağıda biçimlendirmelere ilişkin bir kod örneği verilmiştir;

```
#include <iostream>
#include<iomanip>
int main() {
        int iTemp = 47;
        std::cout<< std::resetiosflags(std::ios_base::basefield);
```

Bayrak	Açıklama
dec	Tam sayıları onluk (decimal) tabanda gösterir.
oct	Tam sayıları sekizlik (octal) tabanda gösterir.
hex	Tam sayıları onaltılık (hexadecimal) tabanda gösterir.
left	Çıktıyı sola yaslar (sağa dolgu ekler).
right	Çıktıyı sağa yaslar (sola dolgu ekler).
internal	İşaret (+/-) ile sayı arasına dolgu karakteri ekler.
scientific	Kayan noktalı sayıları bilimsel notasyonda gösterir.
fixed	Kayan noktalı sayıları sabit ondalık formatta gösterir.
boolalpha	Mantıksal değerleri 0/1 yerine true/false olarak yazdırır.
showbase	Sayı tabanını belirten önekleri (0x, 0) ekler.
showpoint	Ondalık nokta karakterini her zaman gösterir.
showpos	Pozitif sayıların başına + işareti koyar.
skipws	Giriş sırasında baştaki boşlukları atlar.
unitbuf	Her çıktı işleminden sonra tamponu (buffer) temizler.
uppercase	Çıktıdaki harfleri büyük harfe dönüştürür.

Tablo 5.2: iostream Biçimlendirme Bayrakları

```
        std::cout<< std::setiosflags(std::ios_base::oct) << iTemp << std::endl; //57
        std::cout << std::resetiosflags(std::ios_base::basefield);
        std::cout << std::setiosflags(std::ios_base::hex) << iTemp << std::endl; //2f
        std::cout << std::setiosflags(std::ios_base::uppercase) << iTemp << std::endl; //2F
        std::cout << std::setfill('0') << std::setw(12);
        std::cout << std::resetiosflags(std::ios_base::uppercase);
        std::cout << std::setiosflags(std::ios_base::right) << iTemp << std::endl; //00000000002f
        std::cout << std::resetiosflags(std::ios_base::basefield|std::ios_base::adjustfield);
        std::cout << std::setfill('.') << std::setw(10);
        std::cout << std::setiosflags(std::ios_base::left) << iTemp << std::endl;
         ↪  //47........
        std::cout << std::resetiosflags(std::ios_base::adjustfield) << std::setfill('#');
        std::cout << std::setiosflags(std::ios_base::internal|std::ios_base::showpos);
        std::cout << std::setw(10) << iTemp << std::endl; //+#######47
        double dTemp = -1.2;
        double pi = 3.14159265359;
        std::cout << pi << " " << dTemp << std::endl; //+3.14159 -1.2
        std::cout << std::setiosflags(std::ios_base::showpoint) << dTemp << std::endl; //-1.20000
        std::cout << setiosflags(std::ios_base::scientific) << pi << std::endl; //+3.141593e+00
        std::cout << std::resetiosflags(std::ios_base::floatfield);
        std::cout << setiosflags(std::ios_base::fixed) << pi << std::endl; //+3.141593
        bool b = true;
        std::cout << std::setiosflags(std::ios_base::unitbuf|std::ios_base::boolalpha) << b;
         ↪  //true
}
```

5.4 Uluslar Araşı Karakterlerle Çalışma

C++'ta wchar_t kullanarak Türkçe karakterleri (ş, ğ, ü, ı, ö, ç) doğru şekilde işlemek ve ekrana yazdırmak biraz dikkat gerektirir. Bunun sebebi, standart std::cout akışının 8-bitlik karakterlere odaklanması, uluslararası karakterler yani (wide) karakterler için std::wcout akışı ile uygun yerel ayar (locale) desteğine ihtiyaç duymasıdır.

Windows altında çalışan bir Türkçe wchar_t örneği

```
#include <iostream>
#include <cwchar>    // wchar_t fonksiyonları için
#include <clocale>   // setlocale için
int main() {
        // 1. ADIM: Yerel ayarları sistemin varsayılanına (Türkçe) ayarla.
        // Bu satır olmadan Türkçe karakterler ekranda bozuk görünür.
        std::setlocale(LC_ALL, "Turkish");
        // 2. ADIM: Geniş karakterli string tanımlama (L öneki ile)
        wchar_t mesaj[] = L"C++ ile Türkçe karakterler: Şş, Ğğ, Üü, İı, Öö, Çç";
```

```
        // 3. ADIM: wcout (wide cout) kullanarak ekrana yazdırma
        std::wcout << L"Mesajınız: " << mesaj << std::endl;
        // Tek bir karakter kullanımı
        wchar_t harf = L'Ğ';
        std::wcout << L"Seçilen Harf: " << harf << std::endl;
}
```

Bu kodda ve dikkat etmeniz gereken kritik noktalar:

- **L Öneki**: `wchar_t` değişmezleri tanımlarken her zaman başına `L` koymalısınız (`L"Türkçe Metin"`). Bu, derleyiciye o metni geniş karakter dizisi olarak saklamasını söyler.
- **`std::wcout` Kullanımı**: Geniş karakterleri yazdırmak için `std::cout` değil, `std::wcout` kullanmalısınız. Aksi takdirde ekranda karakterlerin kendisini değil, bellekteki sayısal adreslerini görebilirsiniz.
- **`setlocale` Fonksiyonu**: C++ konsolu varsayılan olarak "C" modundadır. Yani temiz ASCII modundadır. `std::setlocale(LC_ALL, "Turkish");` komutu, konsolun Türkçe karakter setini (Windows'ta genellikle Code Page 1254) tanımasını sağlar.
- **Derleyici ve Terminal Uyumu**: Eğer bu kodu Windows'ta Visual Studio (MSVC) ile çalıştırıyorsanız, kaynak dosyanızın UTF-8 with BOM veya Windows-1254 olarak kaydedildiğinden emin olmalısınız.

Benzer uygulamayı UNIX/Linux altında aşağıdaki şekilde yazabiliriz;

```
#include <iostream>
#include <cwchar>
#include <clocale>
int main() {
        /* Ubuntu'da sistemin varsayılan UTF-8 yerel ayarını kullanmak en güvenlisidir. ""
         ↪ parametresi sistemin mevcut dil ayarlarını (genelde en_US.UTF-8 veya tr_TR.UTF-8)
         ↪ yükler. */
        if (!std::setlocale(LC_ALL, "")) {
                std::wcerr << L"Yerel ayar yüklenemedi!" << std::endl;
                return 1;
        }
        // Geniş karakterli string
        wchar_t mesaj[] = L"Ubuntu üzerinde Türkçe: Şş, Ğğ, Üü, İı, Öö, Çç";
        /* Ubuntu terminali (Gnome Terminal vb.) doğrudan UTF-8 desteklediği için  wcout düzgün
         ↪ yapılandırıldığında karakterleri basacaktır. */
        std::wcout << L"Mesaj: " << mesaj << std::endl;
        // Boyut farkını görmek için (Ubuntu'da 4 çıkacaktır)
        std::wcout << L"wchar_t boyutu: " << sizeof(wchar_t) << L" byte." << std::endl;
}
```

Modern C++ uygulamalarında `wchar_t` yerine, taşınabilirlik (portability) için `char16_t` (UTF-16) veya `char32_t` (UTF-32) tercih edilmelidir.
`wchar_t`'nin Boyutu, Windows işletim sisteminde 2 byte iken Linux'ta 4 byte'tır. Bu durum sistemler arası veri aktarımında hatalara yol açabilir. Buna ilişkin örnek ileride verilecektir.

5.5 Düşün ve Çöz

- **Düşün**: `std::cin` ile klavyeden veri okurken kullanıcı beklenen tipte değil yanlış bir karakter girerse ne olur? Bu durumu ele almak için hangi kontrol mekanizmaları kullanılabilir?
- **Çöz**: `std::cout` ile çıktıyı formatlamak için `std::setw()`, `std::setprecision()` ve `std::fixed` manipulatörlerini kullanın. Pi sayısını farklı ondalık hassasiyetlerde (2, 4, 6 basamak) ekrana yazdıran bir program tasarlayın.
- **Düşün**: Konsol çıktısında Türkçe karakterler (ğ, ş, ı, ö, ü, ç) neden bazen bozuk görünür? Bu sorunu çözmek için hangi adımlar atılabilir?
- **Düşün**: Modüler programlama nedir ve neden önemlidir? Büyük bir C++ projesini tek bir `.cpp` dosyasına yazmak yerine birden fazla dosyaya bölmenin avantajlarını somutlaştırın.
- **Düşün**: `std::cin >> x;` ile `std::getline(std::cin, s);` arasındaki temel fark nedir? Bir tam sayı okuyup ardından bir satır metin okumak istediğinizde neden sorun çıkabilir ve nasıl çözülür?

6. Bölüm

Kontrol İşlemleri

6.1 Ardışık İşlem ve Kontrol İşlemleri

Yapısal Programlama başlığı altında programın ana fonksiyondan başlayacağı ve programlamanın ise birbirini çağıran fonksiyonlarla yapıldığı anlatılmıştı. Yapısal programlamada, ana fonksiyon dahil tüm fonksiyonlarda ilk önce işlenecek verileri taşıyan veri yapıları tanımlanır ve ardından bu verileri işleyen kontrol yapılarına ilişkin talimatlar yazılır. C++ dili de C diline eklenti yapılarak geliştirildiği için yapısal programlama mantığını iyi öğrenmeliyiz. Kal dı ki nesne yönelimli programdaki (object oriented programming) **nesne yöntemleri de yapısal programdaki fonksiyonlar gibi kodlanırlar.**

Şu ana karar örneğini verdiğimiz kodlarda veri yapısı olarak sadece değişkenler kullanılmıştır. Sonrasında ise giriş çıkış işlemleri talimatlar içeren programlar yazılmıştır. Programın icrası ilk talimattan başlar ve sırasıyla program bitene kadar devam eder. İşte **programın icrasını değiştirmeyen** bu tür talimatlara **ardışık işlem** (sequential operation) adı verilir.

Ardışık işlemler aşağıdaki üç tür talimattan (statement) oluşur.

1. Değişkenlerin kimliklendirildiği değişken tanımlamaları (identifier definition):
   (a) `int yariCap=3;`
   (b) `const float PI=3.14;`
   (c) `float daireninAlani,daireninCevresi;`
2. **İfadelerden** (expression): **İfade**, sabit, değişken ve işleç (operator) içeren söz dizimleridir;
   (a) `daireninAlani=PI*yariCap*yariCap;`
   (b) `float daireninCevresi=2.0*PI*yariCap;`
3. Klavyeden veri okuma ve konsola veri yazma gibi giriş-çıkış işlemleri (input-output operation):
   (a) `std::cout << "Kapasite Oranını (0.00-1.00) Giriniz:";`
   (b) `std::cin >> kapasiteOrani;`

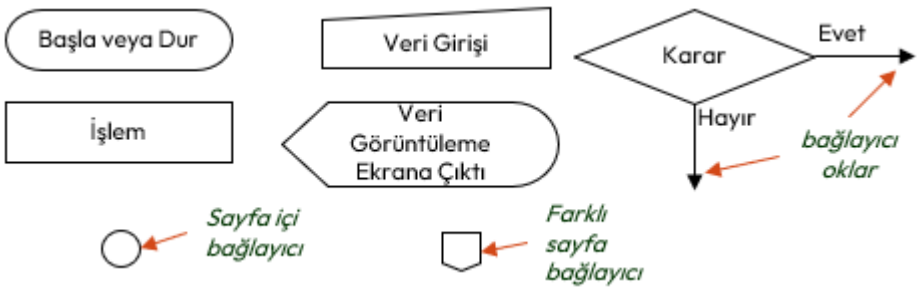
Kontrol işlemleri (control operation) ise programın icra sırasını değiştiren kontrol yapıları olup bu talimatlar (statement) olup üç türü vardır;

1. **Duruma göre seçimler** (conditional choice): `if`, `if-else` ve `switch` talimatları.
2. **İlişkisel döngü** (relational loop): `while`, `do-while` ve `for` talimatları.
3. **Dallanmalar** (jump): `continue`, `break`, `goto` ve `return` talimatları.

> Kontrol işlemlerinde; **kodlanan mantıksal satırlar (logical sequence) ile fiziksel olarak icra edilen satırlar (physical sequence) birbirinden farklıdır.**

6.2 Akış Diyagramları

Kontrol işlemi içeren programın icrasını anlamak için akış diyagramlarını da anlamak gerekir. Aşağıda akış diyagramlarındaki temel şekiller verilmiştir.

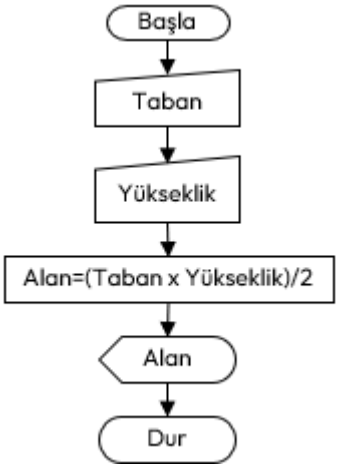


Şekil 6.1: Akış Diyagramları Genel Şekilleri

Akış diyagramları (flowchart), adımların grafik gösterimleridir. Algoritmaları ve programlama mantığını temsil etmek için bir araç olarak bilgisayar biliminden ortaya çıkmıştır. Ancak diğer tüm işlem türlerinde

kullanılmak üzere genişletilmiştir. Günümüzde akış diyagramları, bilgileri göstermede ve akıl yürütmeye yardımcı olmada son derece önemli bir rol oynamaktadır. Karmaşık süreçleri görselleştirmemize veya sorunların ve görevlerin yapısını açık hale getirmemize yardımcı olurlar. Bir akış şeması, uygulanacak bir süreci veya projeyi tanımlamak için de kullanılabilir.

Aşağıda taban ve yüksekliği klavyeden girilen bir dik üçgenin alanını hesaplamak için bir akış diyagramı örneği verilmiştir.



Şekil 6.2: Dik Üçgenin Alana Hesabını Gösteren Akış Diyagramı

Akış diyagramı çizilirken aşağıdaki kurallara uyulur;

- ◇ Akış şemalarında tek bir başlangıç simgesi olmalıdır
- ◇ Bitiş simgesi birden çok olabilir.
- ◇ Karar simgesinin haricindeki simgelere her zaman tek giriş ve tek çıkış yolu bulunur.
- ◇ Bağlaç simgesi sayfanın dolmasından ötürü parçalanan akış şemasının öğelerini birleştirmede kullanılır.
- ◇ Simgeler birbirleri ile tek yönlü okla bağlanırlar.
- ◇ Okların yönü algoritmanın mantıksal işlem akışını tanımlar.

6.3 Duruma Göre Seçimler

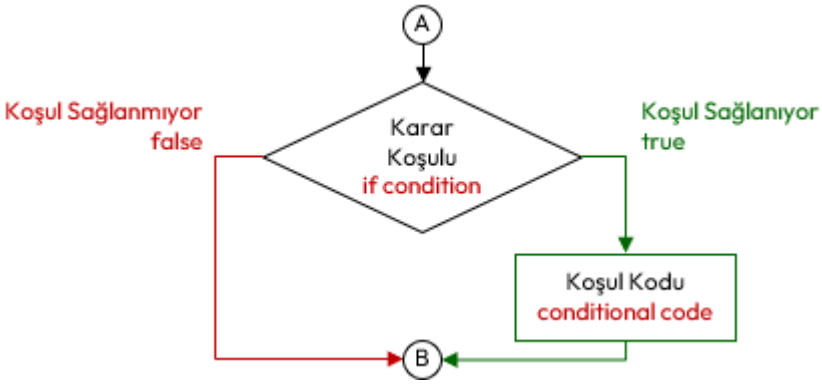
Duruma göre seçimler (conditional choice) programın akışını bir koşula göre değiştiren talimatlardır.

§6.3.1 If Talimatı

If talimatı, karar vermeye ilişkin bir talimat olup bir koşula bağlı olarak programın icrasını değiştirir. If talimatının sözde kodu aşağıda verilmiştir;

```
if (karar-koşulu)
    koşul-kodu;
```

Burada ilişkisel işleçler içeren **karar koşulu** (condition) ifadesi (expression) test edilir. Sonuç `true` ise **koşul kodu** (conditional code) icra edilir, değilse icra edilmez. Bu talimatın akış diyagramı da aşağıdaki gibidir;



Şekil 6.3: If Talimatı İcra Akışı

Aşağıdaki örnek programı inceleyelim;

```
1 #include <iostream>
```

```
2  int main() {
3          int yas(18);
4          char cinsiyet='K';
5          if (yas<30)
6                  std::cout << "Genç";
7          if (cinsiyet=='E')
8                  std::cout << "Erkek";
9  }
```

Örnek kodumuzu şu şekilde açıklayabiliriz;

1. 1. Satırda `std::cout` akışı için `<iostream>` başlığı koda dahil edilmiştir.
2. 2. Satırda programın başlayacağı ana fonksiyon tanımlanmıştır.
3. 3. ve 4. Satırda `yas` ve `cinsiyet` değişkenleri başlatılmıştır.
4. 5. Satırda bir `if` talimatı bulunmaktadır. **Karar koşulu**, `yas<30` ifadesiyle test edilecektir. Test sonucu `true` olduğundan 6. Satırdaki **koşul kodu** `std::cout << "Genç";` talimatı ile konsola `"Genç"` yazılır.
5. 7. Satırda da bir `if` talimatı bulunmaktadır. Karar koşulu, `cinsiyet=='E'` ifadesiyle test edilecektir. Test sonucu `false` olduğundan 8. Satırdaki koşul kodu talimatı **icra edilmez.**

If talimatında **koşul kodu**, her zaman yukarıdaki gibi **ardışık işlem** (sequential operation) olmaz. Bazen `if` gibi bir başka **kontrol işlemi** (control operation) de olabilir. Aşağıda **kademeli if** (compound if/cascade if) kullanımına ilişkin kod örneğinde ikinci `if`, birinci `if` talimatının koşul kodudur;

```
if (yas<30)
    if (yas<7) // Bu if, üsteki if talimatının koşul kodudur.
        std::cout << "Çoçuk";
```

Koşul kodu birden fazla talimattan oluşacak ise kod bloğu içine alınır. Aşağıda verilen kod örneğinde ilk `if` talimatında `yas<30` ile test edilen koşul doğrulandığında kod bloğunun içindeki tüm talimatlar icra edilir.

```
if (yas<30) { //koşul-kodu bloğu başlangıcı
        if (yas<7)
            std::cout << "Çocuk";
        if (yas<18)
            std::cout << "Genç";
        if (yas>60)
            std::cout << "Yaşlı";
} //koşul-kodu bloğu bitişi
```

Bazı durumlarda bir `if`, başka bir `if` bloğu içinde yani **iç içe** (nested if) olabilir. Buna ilişkin kod örneği de aşağıda verilmiştir.

```
if (cinsiyet=='E') { // dış if bloğu başlangıcı
        if (yas<7) { // iç if bloğu başlangıcı
            std::cout << "Erkek Çocuk ";
            std::cout << "Yada Erkek Bebek";
        } // iç if bloğu başlangıcı
        if (yas<18)
                std::cout << "Genç Delikanlı";
        if (yas>60)
                std::cout << "Yaşlı Adam";
} // dış if bloğu bitişi
```

If talimatında **koşul kodu**, her zaman tek bir test ifadesinden oluşmayabilir. Bahsedilen duruma ilişkin kod örneği de aşağıda verilmiştir.

```
if ((cinsiyet=='E') && (yas<18))  // hem Erkek, hem de genç
        std::cout << "Genç Delikanlı";
if ((cinsiyet=='K') && (yas>60)) // hem yaşlı, hem de kadın
        std::cout << "Yaşlı Kadın";
```

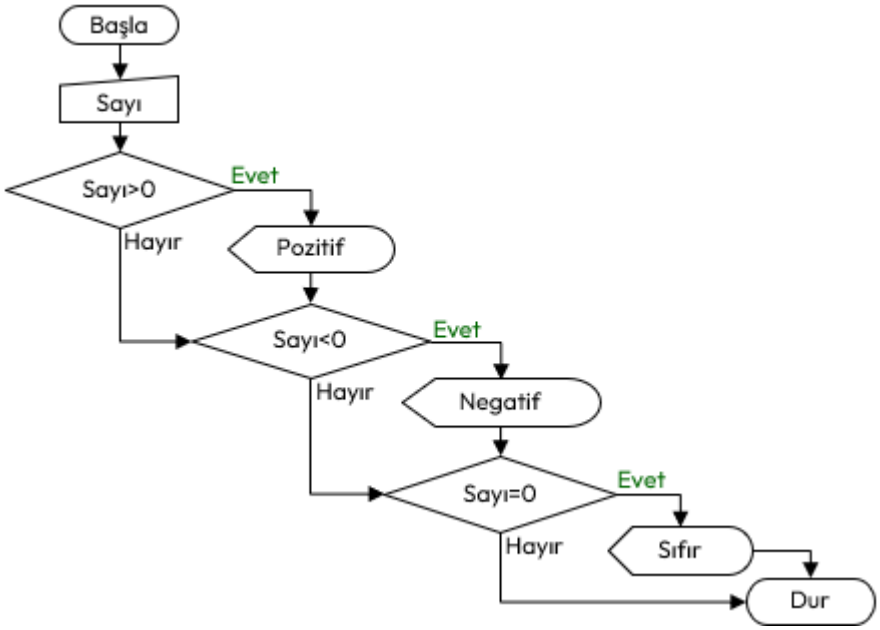
Ayrıca bazen programcı tarafından aşağıdaki gibi `if` talimatları yazabilir.

```
if (1) // veya if(true)
        std::cout << "Bu metin konsola her zaman yazılır.";
if (0) // veya if(false)
        std::cout << "Bu metin konsola hiçbir zaman yazılmaz!";
```

Örnek olarak klavyeden girilen bir sayının işaretini bulan bir C++ programı yazalım;

```
#include <iostream>
int main() {
        int sayi;
        std::cout << "Sayi Giriniz:";
        cin >> sayi;
        if (sayi>0) // sayının pozitif olup olmadığı test ediliyor.
                std::cout << "Pozitif"; // Burada sayının pozitif olduğu biliniyor.
        if (sayi<0) // sayının negatif olup olmadığı test ediliyor.
                std::cout << "Negatif"; // Burada sayının negatif olduğu biliniyor.
        if (sayi==0) // sayının sıfır olup olmadığı test ediliyor.
                std::cout << "Sıfır"; // Burada sayının sıfır olduğu biliniyor.
}
```

Bunun için oluşturacağımız akış diyagramı aşağıdaki şekilde olacaktır.



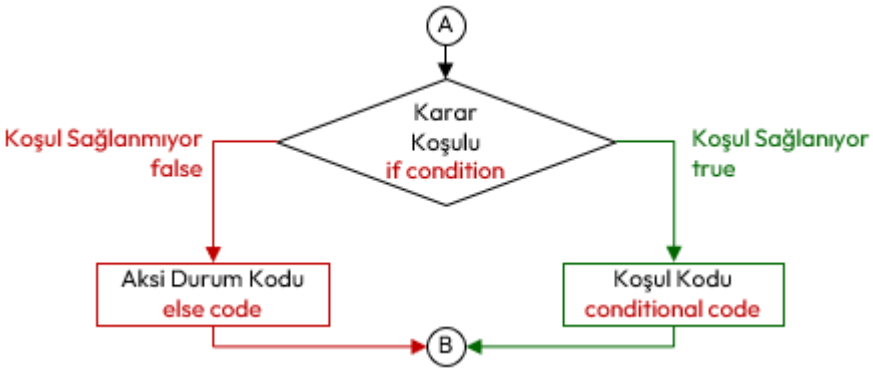
Şekil 6.4: Bir Sayının İşaretini Bulan Akış Diyagramı

§6.3.2 If-Else Talimatı

If-Else Talimatı, bir başka karar verme talimatı olup bir koşula bağlı olarak programın icrasını değiştirir. Sözde kodu aşağıda verilmiştir;

```
if (karar-koşulu)
    koşul-kodu;
else
    aksi-durum-kodu;
```

If-Else talimatında, **karar koşulu** (condition) ifadesi `true` ise **koşul kodu** (conditional code) icra edilir, karar koşulu `false` ise **aksi durum kodu** (else code) icra edilir.



Şekil 6.5: If-Else Talimatı İcra Akışı

Aşağıda verilen örnek programı inceleyelim;

```
using namespace std;
int main() {
        int yas(65);
        char cinsiyet('K');
        if (yas<50)
                std::cout << "Genç ";
        else
                std::cout << "Yaşlı ";
}
```

Örnek kodumuzu şu şekilde açıklayabiliriz;

1. 1. Satırda `std:cout` akışı için `<iostream>` başlığı koda dahil edilmiştir.
2. 2. satırda programın başlayacağı ana fonksiyon tanımlanmıştır.
3. 3. ve 4. Satırda `yas` ve `cinsiyet` değişkenleri başlatılmıştır.
4. 5. Satırda bir `if` talimatı bulunmaktadır. **Koşul kodu** `yas<50` ifadesiyle test edilecektir. Test sonucu `false` olduğundan 6. Satırdaki **koşul kodu** talimatı **icra edilmez.** Bunun yerine `else` sonrası 8. Satırdaki **aksi durum kodu** talimatı **icra edilir.** Yani `std::cout << "Yaşlı ";` talimatı ile konsola `"Yaşlı"` yazılır.

If talimatında olduğu gibi bu talimatta da **koşul kodu** ya da **aksi durum kodu** birden fazla talimattan oluşacak ise kod bloğu içine alınır. Buna ilişkin kod örneği aşağıda verilmiştir.

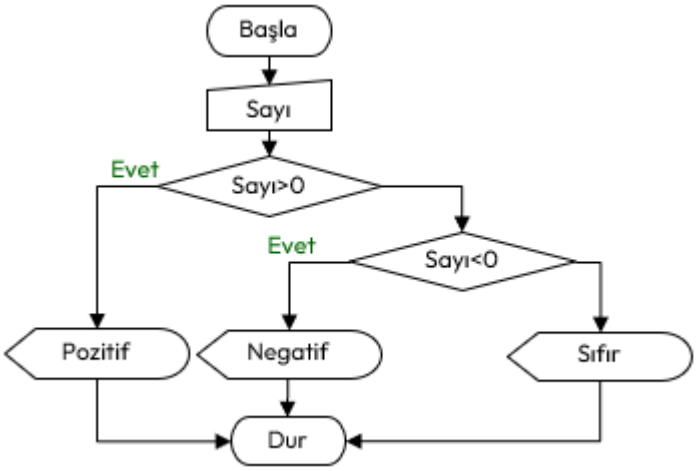
```
if (cinsiyet=='E') {
        if (yas<7)
                std::cout << "Erkek Çocuk ";
        if (yas<18)
                std::cout << "Genç Delikanlı ";
        if (yas>60)
                std::cout << "Yaşlı Adam ";
} else {
        if (yas<3)
                std::cout << "Kız Bebek ";
        if (yas<7)
                std::cout << "Kız Çocuk ";
        if (yas<18)
                std::cout << "Genç Kız ";
        if (yas>60)
                std::cout <<"Yaşlı Kadın ";
}
```

Bir sayının işaretini bulan C++ programının icra akışı olan Şekil 6.4 incelendiğinde ilk `if` talimatında yapılan test doğru çıkarsa geri kalan testlerin yapılmasına gerek kalmadığı anlaşılacaktır. Benzer şekilde ilk iki test geçilirse üçüncü if talimatındaki teste gerek yoktur. Bu durumu Şekil 6.6'de verilen akış diyagramında daha net görebiliriz. Bu durumda aynı program if-else talimatlarıyla aşağıdaki şekilde yeniden yazabiliriz;

```
#include <iostream>
int main() {
        int sayi;
        std::cout << "Sayi Giriniz:";
        cin >> sayi;
        if (sayi>0) { // sayının pozitif olup olmadığı test ediliyor.
                std::cout << "Pozitif"; // Burada sayının pozitif olduğu biliniyor.
        } else {  // Burada sayının pozitif olmadığı biliniyor.
                if (sayi<0) { // sayının negatif olup olmadığı test ediliyor.
                        std::cout << "Negatif"; // Burada sayının negatif olduğu biliniyor.
                } else  { // Burada sayının pozitif ve negatif olmadığı test biliniyor.
                        std::cout << "Sıfır"; // Burada sayının sıfır olduğu biliniyor.
                }
        }
}
```

Örnek kodumuzu şu şekilde açıklayabiliriz;

- Eğer `sayı` pozitif girilirse;
  - 6. Satırdaki `if` karar koşulunda `sayi>0` test edilip `true` çıkacak ve sadece 7. Satırdaki koşul kodu olan `std::cout << "Pozitif";` talimatı icra edilecektir. 8. Satırdan başlayan ve on üçüncü satırda



Şekil 6.6: Bir Sayının İşaretini Bulan If-Else Akış Diyagramı

biten aksi durum kodu icra edilmeyecektir.

- Eğer `sayı` negatif girilirse;
  - 6. Satırdaki `if` karar koşulunda `sayi>0` test edilip `false` çıkacak ve 8. Satırdan başlayan ve 14. Satırda biten aksi durum kodu icra edilecektir.
  - Bu blok içinde 9. Satırdaki `if` karar koşulu `sayi<0` test edilip `true` çıkacak ve 10. Satırdaki koşul kodu olan `std::cout << "Negatif";` talimatı icra edilecektir. 12. Satırlardaki aksi durum kodu icra edilmeyecektir.
- Eğer `sayı` sıfır girilirse;
  - 6. Satırdaki `if` karar koşulunda `sayi>0` test edilip `false` çıkacak ve 8. Satırdan başlayan ve 14. Satırda biten aksi durum kodu icra edilecektir.
  - Bu blok içinde 9. Satırdaki `if` karar koşulu `sayi<0` test edilip `false` çıkacak ve 10. Satırdaki koşul kodu talimatı icra edilemeyecektir. Onun yerine on 12. Satırlardaki aksi durum kodu olan `std::cout << "Sıfır";` icra edilecektir.

Ayrıca kod incelendiğinde her bir `if` ve `else` sonrasında tek talimat bulunmaktadır. Dolayısıyla blok kullanmaya gerek de yoktur. Bu durumda kodun nihai hali aşağıda verilmiştir.

```
#include <iostream>
int main() {
        int sayi;
        std::cout << "Sayi Giriniz:";
        cin >> sayi;
        if (sayi>0)
                std::cout << "Pozitif";
        else
                if (sayi<0)
                        std::cout << "Negatif";
                else
                        std::cout << "Sıfır";
}
```

§6.3.3 Sarkan Else

Aşağıdaki program örneği incelendiğinde `else`, hangi `if` talimatına aittir? Böyle bir kod programcı tarafından yazılabilir ve derleyici buna hiçbir sıkıntı çıkarmaz. Bu problem **sarkan else** (dangling else) problemi olarak bilinir.

```
if (cinsiyet=='E') if (yas<18) std::cout << "Delikanlı"; else std::cout << "Adam";
```

Bu durumda kodu aşağıdaki gibi okunaklı hale getirdiğimizde ilk `if` talimatının **koşul kodunun** ikinci if-else talimatı olduğu görülmektedir.

```
if (cinsiyet=='E')
        if (yas<18)
                std::cout << "Delikanlı";
        else
                std::cout << "Adam";
```

Kod bloğu kullanılmadan yazılan bu tip kodlarda `else` her zaman kendinden bir önceki `if` talimatına aittir.

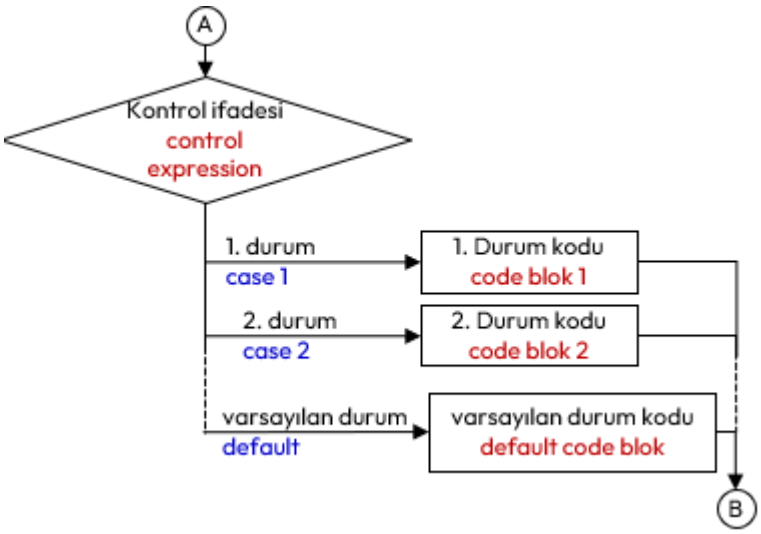
Sarkan else durumundaki mantıksal hataların önüne geçmek için
acemi kullanıcılar her zaman blok kullanmalıdır.

```
 if (cinsiyet=='E') {
        if (yas<18) {
                std::cout << "Delikanlı";
        } else {
                std::cout << "Adam";
        }
}
```

§6.3.4 Switch Talimatı

Switch talimatı, bir **kontrol ifadesi** (control expression) sonucunda birden fazla alternatif arasında seçim yapılmasını sağlar. Sözde kodu aşağıda verilmiştir;

```
switch (kontrolifadesi) { //switch bloğu başlangıcı
    case alternatif-1:
        alternatif-1-kodu;
        break; //break talimatı zorunlu değildir
    case alternatif-2:
        alternatif-2-kodu;
        break; //break talimatı zorunlu değildir.
    // ...
    default: // Bu etiket zorunlu değildir!
        varsayılan-alternatif-kodu;
} //switch bloğu bitişi
```



Şekil 6.7: Switch Talimatı İcra Akışı

Switch talimatına ilişkin kurallar;

1. **Kontrol ifadesi, sonucu tam sayı olan ifadedir** ve **bir kez değerlendirilir**.
2. **Kod bloğu olmayan** bir `switch` talimatı **yazılamaz!**
3. **Her bir alternatif**, bir **tam sayı değişmezi** (integer literal) izleyen ve iki nokta ile biten bir **etiket (label) olarak tanımlanır.**
4. **Alternatif**, hiçbir zaman **değişken veya ifade (expression) olamaz!**
5. `switch` **bloğu icra edilirken**, karşılaştığı ilk `break;` talimatında, `switch` bloğu **dışına çıkılır**. Bu talimat zorunlu değildir.
6. Tüm tam sayıları içerecek şekilde alternatiflerin tümü yazılmayabilir. Blok sonunda yer alacak şekilde **üzerinde yer alan alternatifler dışında kalan tüm alternatifler için** `default:` etiketli alternatif yazılabilir. Bu etiket zorunlu değildir.

Aşağıda örnek olarak menü seçimi için yazılmış bir program örneği verilmiştir;

```
1 #include <iostream>
```

```
2  int main() {
3          int menu;
4          std::cout << "Menü İçin Rakam Giriniz:";
5          std::cin >> menu;
6          switch(menu) {
7                  case 1:
8                          std::cout << "1 Numaralı Menüyü Seçtiniz.";
9                          std::cout << "Hamburger ve Ayran Hazırlanacak.";
10                         break;
11                 case 2:
12                         std::cout << "2 Numaralı Menüyü Seçtiniz.";
13                         std::cout << "Patates Kızartması ve Kola Hazırlanacak.";
14                         break;
15                 case 3:
16                 case 4:
17                         std::cout << "3 veya 4 Numaralı Menüyü Seçtiniz.";
18                         break;
19                 default:
20                         std::cout << "1,2, 3 veya 4 Dışında Menü Seçtiniz.";
21                         std::cout << "Böyle bir Menümüz Yok!.";
22         }
23 }
```

Örnek kodumuzu şu şekilde açıklayabiliriz;

- Eğer `menu`, `1` girilirse;
  - 6. Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu 1 olduğuna karar verilir.
  - Bu durumda 7. Satırdaki `case 1:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
    - İlk önce 8. Satırdaki `std::cout << "1 Numaralı Menüyü Seçtiniz.";` icra edilir ve bir sonraki talimata geçilir.
    - Daha sonra 9. Satırdaki `std::cout << "Hamburger ve Ayran Hazırlanacak.";` icra edilir ve bir sonraki talimata geçilir.
    - Son olarak 10. Satırdaki `break;` talimatı bizi `switch` bloğunun sonundan dışına çıkarır.
- Eğer `menu`, `2` girilirse;
  - 6. Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu 2 olduğuna karar verilir.
  - Bu durumda 11. Satırdaki `case 2:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
    - İlk önce 12. Satırdaki `std::cout << "2 Numaralı Menüyü Seçtiniz.";` icra edilir ve bir sonraki talimata geçilir.
    - Daha sonra 13. Satırdaki `std::cout << "Patates Kızartması ve Kola Hazırlanacak.";` icra edilir ve bir sonraki talimata geçilir.
    - 14. Satırdaki `break;` talimatı bizi `switch` bloğunun sonundan dışına çıkarır.
- Eğer `menu`, `3` girilirse;
  - 6. Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu 3 olduğuna karar verilir.
  - Bu durumda 15. Satırdaki `case 3:` etiketine gidilir ve bu alternatife ilişkin yazılmamıştır. Ancak ilk icra edilebilir koda kadar ilerlenir ve oradaki kodlar icra edilir;
    - İlk önce 17. Satırdaki `std::cout << "3 veya 4 Numaralı Menüyü Seçtiniz.";` icra edilir ve bir sonraki talimata geçilir.
    - Daha sonra 18. Satırdaki `break;` talimatı bizi `switch` bloğunun sonundan dışına çıkarır.
- Eğer `menu`, `4` girilirse;
  - 6. Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu 4 olduğuna karar verilir.
  - Bu durumda 16. Satırdaki `case 4:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
    - İlk önce 17. Satırdaki `std::cout << "3 veya 4 Numaralı Menüyü Seçtiniz.";` icra edilir ve bir sonraki talimata geçilir.
    - Daha sonra 18. Satırdaki `break;` talimatı bizi `switch` bloğunun sonundan dışına çıkarır.
- Eğer `menu`,`-1` , `0`, `12` ve `300` gibi yukarıdaki alternatiflerden farklı girilirse;
  - 6. Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu düzenlenememiş bir durum olduğuna karar verilir.
  - Bu durumda 19. Satırdaki `default:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;

- İlk önce 20. Satırdaki `std::cout << "1,2, 3 veya 4 Dışında Menü Seçtiniz.";` icra edilir ve bir sonraki talimata geçilir.
- Daha sonra 21. Satırdaki `std::cout << "Böyle bir Menümüz Yok!.";` talimatı bizi `switch` icra edilir. Artık `switch` bloğunun sonuna ulaşıldığından bloktan çıkılır.

Aşağıda klavyeden girilen bir sayının 4 rakamına bölünmesinde kalan üzerinden işlem yapan `switch` talimatı bir başka örnek program verilmiştir;

```
#include <iostream>
int main() {
        int sayi;
        std::cout << "Bir Sayı Giriniz:";
        std::cin >> sayi;
        switch(sayi%4) { //konrol ifadesi bir işlem içerir
                case 3:
                        std::cout << "Sayı 4 rakamına bölündüğünde kalan 3 dir.";
                        break;
                case 2:
                        std::cout << "Sayı 4 rakamına bölündüğünde kalan 2 dir.";
                        break;
                case 1:
                        std::cout << "Sayı 4 rakamına bölündüğünde kalan 1 dir.";
                        break;
                default: // Başka alternatif yoktur.
                        std::cout << "Sayı 4 Rakamına TAM Bölünür";
        }
}
```

§6.3.5 Üçlü Koşul İşleci

Daha önce İşleçler başlığı altında işlenenlere ek olarak, **ardışık işlemlerde** (sequential operation) if talimatlarına gerek kalmadan bir koşula göre **ifadeler** (expression) işlenecek ise **üçlü koşul işleci** (conditional tenary operator) kullanılır. Aşağıda üçlü işlecin iki sözdizimi verilmiştir;

```
koşul ? doğruifadesi : yanlışifadesi;
değişken = koşul ? doğruifadesi : yanlışifadesi;
```

Aşağıda oy kullanma durumu örneği bu işleçle işlenmiştir;

```
#include <iostream>
int main() {
        int yas;
        std::cout << "Yaşınız?: ";
        std::cin >> yas;
        (yas >= 18) ? std::cout << "Oy Kullanabilirsin." : std::cout << "Oy Kullanamazsın!";
}
```

Aşağıda başka kullanımlarına ilişkin farklı kod örneği verilmiştir;

```
#include <iostream>
int main() {
    int sayi,tek,cift;
        std::cout << "Bir Sayı Giriniz: ";
        std::cin >> sayi;
        tek= (sayi%2) ? 1 : 0;
        cift= tek ? 0 : 1;
        int sonuc1=tek ? sayi+30 : sayi+40;
        int sonuc2=cift ? sayi*30 : sayi*40;
}
```

6.4 İlişkisel Döngüler

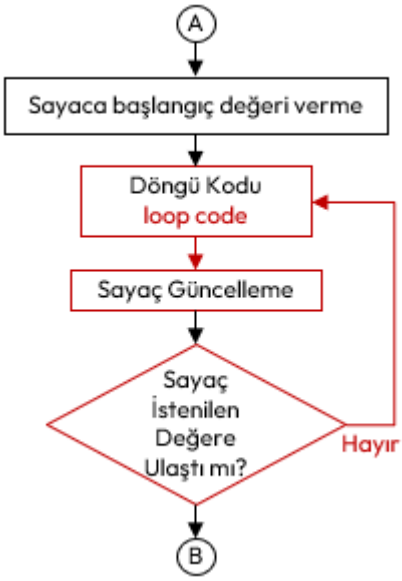
İşlemciler iyi bir sayıcıdırlar. CPU içindeki kaydediciler (registers) sayma işlevini ve birçok matematiksel işlemi yapmak için de kullanılır. **Birçok problemlerin çözümüne, belli adımların tekrarlanması ile gidilir.** Bu tip problemler şimdiye kadar olan yöntemlerle yapılırsa, tekrar tekrar aynı kodu yazmak zorunda kalırız. Konsola 10 defa ismimizi yazdıran bir program örneğini ele alalım.

```
#include <iostream>
int main() {
        std::cout << "ILHAN" << endl;  //1
        std::cout << "ILHAN" << endl;  //2
        std::cout << "ILHAN" << endl;  //3
        std::cout << "ILHAN" << endl;  //4
        std::cout << "ILHAN" << endl;  //5
        std::cout << "ILHAN" << endl;  //6
        std::cout << "ILHAN" << endl;  //7
        std::cout << "ILHAN" << endl;  //8
        std::cout << "ILHAN" << endl;  //9
        std::cout << "ILHAN" << endl;  //10
}
```

Program incelendiğinde aynı `std::cout` talimatının tekrar tekrar yazıldığını görürüz. **Kod tekrarı istenmeyen bir durumdur!**

§6.4.1 Sayaç Kontrollü Döngüler

Belirlenen bir sayaç değişkeninin istenilen bir değere ulaşıp ulaşmadığı kontrol ederek kodun tekrar tekrar icra edilmesi sağlanır. Yapısal programlama öncesinde tekrar edilecek kodun başına dönüş `goto` talimatıyla sağlanır. **Bu talimat icra sırasını etkileyen bir talimattır.** C++ dilinde Kodlama sırasında tekrar edilecek



Şekil 6.8: Sayaç Kontrollü Döngü İcra Akışı

kodun başına iki nokta ile biten bir **etiket** (label) konulur. Etiketlere kimlik verilirken yine 4.3.5 başlığındaki değişken kimliklendirme (identifier definition) kuralları uygulanır. Tekrar edilecek kodun sonunda ise `goto` talimatı etiketle birlikte kullanılır.

```
1 #include <iostream>
2 int main() {
3         int sayac(0); //Sayaca İlk Değer 0 Olarak Verildi
4         Etiket: //Tekrar Edilecek Kodun Başı ETİKETLENİR. Etilet kimliği "Etiket"
5                 std::cout << "ILHAN" << endl;
6                 sayac++; //Sayaç Güncelleme
7         if (sayac<10) //Sayaç Kontrolü
8                 goto Etiket; // İstenilen değere ulaşılmamış ise etikete git.
9 }
```

Örneği şu şekilde açıklayabiliriz;

- 3.Satırda tekrar edilecek koda girmeden sayacımıza ile değer verdik.
- 4.Satıra geri dönülecek kodun başını işaretleyen bir etiket konulmuştur.
- 5.Satırda tekrarlanacak talimat yazılmıştır. Birden fazla talimat da yazabiliriz.
- 6.Satırda sayacımızı güncelledik.

- ◇ 7.Satırda sayacın istenilen değere ulaşıp ulaşmadığını kontrol ettik. Eğer ulaşmamış ise 8.Satırdaki `goto Etiket;` talimatıyla 4.Satıra döndük.

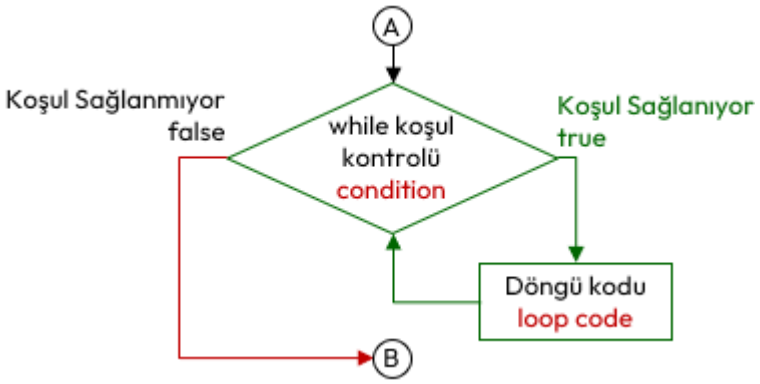
Sonuç olarak 5. ile 8. Satır arasında dört talimat içeren **mantıksal satır** (logical sequence) olmasına karşın 40 adet **fiziksel satır** (physical sequence) icra edilmiştir. C++ programlama dili, donanıma yakın bir dil olduğundan `goto` talimatı desteklenir. Ancak 1968 Yılında *Edsger W. Dijkstra* tarafından `goto` ifadelerinin zararlı olduğunu ilan ettiği unutulmamalıdır.

> Yukarıda verilen döngü kodu örneğindeki gibi arapsaçı kodlamadaki `goto` talimatı yerine, **yapısal programlamada** (structural programming) ilişkisel döngü (relational loop) talimatları olan `while`, `do-while` ve `for` talimatları geliştirilmiştir.
> **C++ dilinde `goto` talimatının kullanımı önerilmez!**

§6.4.2 While Talimatı

While döngüsünde **koşul** (condition), `true` olduğu sürece **döngü kodu** (loop code) icra edilir. Tekrarlanacak kod, tek bir talimat olabileceği gibi bir kod bloğu da olabilir. While döngüsünün sözde kodu aşağıda verilmiştir;

```
while (koşul)
    döngü-kodu;
```



Şekil 6.9: While Talimatı İcra Akışı

While talimatı kullanılarak 6.4.1 başlığında verilen döngü örneği, aşağıdaki şekilde yazılabilir;

```
#include <iostream>
int main() {
        int sayac(0); //Sayaca İlk Değer 0 Olarak Verildi
        while (sayac<10) // While koşul kontrolü
        {  // döngü bloğu başlangıcı
                std::cout << "ILHAN" << endl;
                sayac=sayac+1; //Sayaç Güncelleme
        }  // döngü bloğu bitişi
}
```

Sayaç güncelleme ifadesi, while **koşul** ifadesine eklenerek program daha da kısaltılabilir. Her koşul kontrolü sonrası sayaç artırılacaktır;

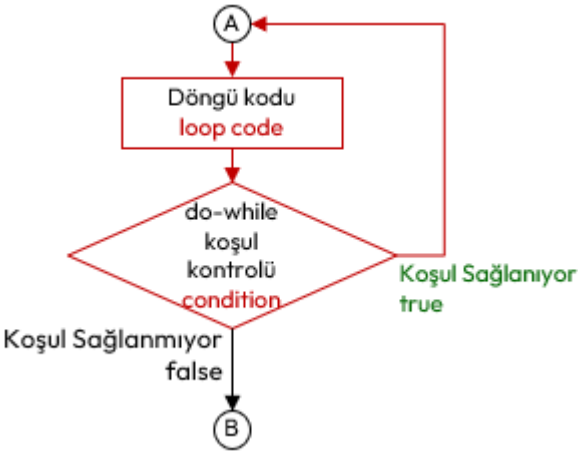
```
#include <iostream>
int main() {
        int sayac(0); //Sayaca İlk Değer 0 Olarak Verildi
        while (sayac++<10) // hem sayaç güncelleme hem de koşul kontrolü
                std::cout << "ILHAN" << endl; // tek talimat olduğundan blok kullanılmadı
}
```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu bir adet talimat içeren mantıksal satır (logical sequence) içermesine rağmen 10 adet fiziksel satır (physical sequence) icra edilmiştir.

§6.4.3 Do-While Talimatı

Do-While döngüsünde, `do` ile `while` saklı kelimeleri arasındaki **döngü kodu** (loop code), **koşul** (condition) `true` olduğu sürece tekrarlanarak icra edilir. Tekrarlanacak kod tek bir talimat olabileceği gibi bir kod bloğu da olabilir. Do-While talimatının sözde kodu aşağıda verilmiştir;

```
do   // "do" kelimesi goto talimatındaki etiket gibi davranır.
 döngü-kodu;
while (koşul);
```



Şekil 6.10: Do-While Talimatı İcra Akışı

Do-While talimatı kullanılarak 6.4.1 başlığında verilen döngü örneği, aşağıdaki şekilde yazılabilir;

```
#include <iostream>
int main() {
        int sayac(0); //Sayaca İlk Değer 0 Olarak Verildi
        do {  // döngü bloğu başlangıcı
                std::cout << "ILHAN" << endl;
                sayac=sayac+1; //Sayaç Güncelleme
        } // Döngü bloğu bitişi
        while (sayac<10); // Do-While koşul kontrolü
}
```

Sayaç güncelleme ifadesi, **koşul** ifadesine eklenerek program aşağıdaki şekilde daha da kısaltılabilir.

```
#include <iostream>
int main() {
        int sayac(0); //Sayaca İlk Değer 0 Olarak Verildi
        do
                std::cout << "ILHAN" << endl;
        while (sayac++<10); // Hem koşul kontrolü hem de sayaç güncelleme
}
```

Yukarıdaki son örnek incelendiğinde ibir adet talimat içeren mantıksal satır (logical sequence) içermesine rağmen 10 adet fiziksel satır (physical sequence) icra edilmiştir.

While döngüsünde koşul kontrolü en başta yapılır!
Do-While döngüsünde ise döngü bloğu en az bir kez çalıştırıldıktan sonra koşul kontrolü yapılır!

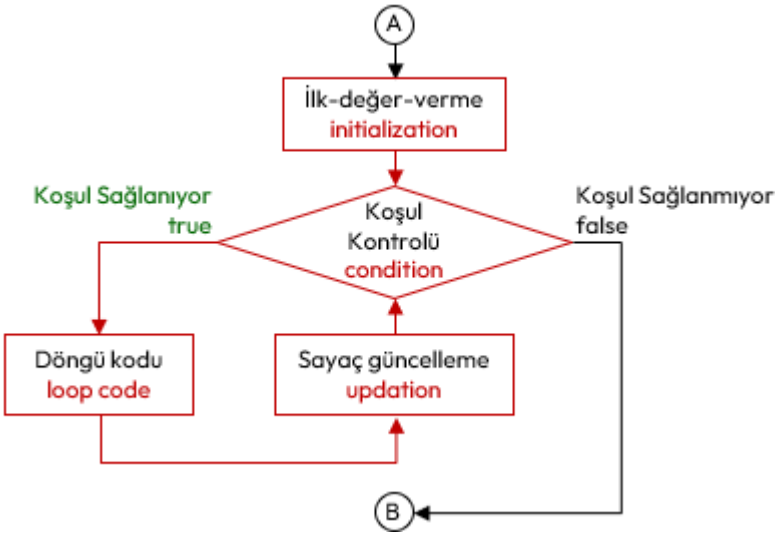
§6.4.4 For Talimatı

For döngüsü özellikle tekrar edilen işlemlerin sayısı belli olduğunda kullanılan bir döngü yapısıdır. Sayaç kontrollü döngüler için en iyi seçimdir. Sözde kodu aşağıda verilmiştir.

```
for (ilk-değer-verme; koşul-kontrolü; sayaç-güncelleme)
    döngü-kodu;
```

For talimatı aşağıdaki şekilde icra edilir;

1. **İlk değer verme ifadesi bir kez icra edilir!**
2. **Koşul kontrolü yapılır!** Eğer test sonucu `true` ise döngü kodu icrasına geçilir. Aksi halde döngüden çıkılır.
3. **Döngü kodu** icra edilir.
4. Döngü kodu icrası bitince **sayaç güncellemeleri yapılır**.
5. Tekrar ikinci adımdaki **koşul kontrolüne dönülür**.



Şekil 6.11: For Talimatı İcra Akışı

For talimatı kullanılarak 6.4.1 başlığında verilen döngü örneği, aşağıdaki şekilde yazılabilir.

```
#include <iostream>
int main() {
        for (int sayac=1; sayac<10; sayac++) { //1. mantıksal satır.
                std::cout << "ILHAN" << std::endl; //2. mantıksal satır
        }
}
```

Buradaki döngü mantıksal satır (logical sequence) olarak iki talimattan oluşmaktadır. Ancak fiziksel satır (physical sequence) olarak 10 satır icra edilmiştir.

Aşağıda konsola yazdığımız her bir satırın başına satır numarası koyan bir program örneği gösterilmektedir.

```
1  #include <iostream>
2  #include <iomanip> // setw(), setfill() için
3  int main() {
4          for (int sayac=1; sayac<11; sayac++)
5                  std::cout << std::setw(2) << std::setfill('0') << sayac << ". ILHAN" << std::endl;
6  }
7  /*Program Çıktısı:
8  01. ILHAN
9  02. ILHAN
10 03. ILHAN
11 04. ILHAN
12 05. ILHAN
13 06. ILHAN
14 07. ILHAN
15 08. ILHAN
16 09. ILHAN
17 10. ILHAN
18 */
```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu 4.Satırda belirtilen bir adet talimat içermektedir. Mantıksal olarak 1 satır içermesine rağmen 10 adet fiziksel satır icra edilmiştir.

For döngüsü, bir sayaç ile çalışılabileceği gibi **birden fazla sayaç ile de çalışılabilir**. Hem ilk değer verme hem de sayaç güncellemede birden fazla sayaca ilişkin işlem yapılabilir. Bunun için **virgül işleci** (comma operator) kullanılır.

```
#include <iostream>
#include <iomanip> // setw(), setfill() için
int main() {
        for (int sayac1=1, sayac2=30; sayac1<11; sayac1++,sayac2--)
```

```
        std::cout << std::setw(2) << std::setfill('0') << sayac1 << "-" << sayac2 << ". ILHAN" <<
        ↪ std::endl;
}
/*Program Çıktısı:
01-30. ILHAN
02-29. ILHAN
03-28. ILHAN
04-27. ILHAN
05-26. ILHAN
06-25. ILHAN
07-24. ILHAN
08-23. ILHAN
09-22. ILHAN
10-21. ILHAN
*/
```

Aşağıda klavyeden girilen iki sayı aralığında tek ve çift sayıların toplamını bulan ve ekrana yazan programı verilmiştir.

```
#include <iostream>
int main() {
        int sayac, sayi1,sayi2;
        int tekToplam=0,ciftToplam=0;
        std::cout << "İki Sayı Giriniz: ";
        std::cin >> sayi1 >> sayi2;
        if (sayi2<sayi1) { //sayi2, sayi1 den büyük olmalı.
                int temp=sayi1; //küçük ise yer değiştiriyoruz.
                sayi1=sayi2;
                sayi2=temp;
        }
        for (sayac=sayi1; sayac<=sayi2; sayac=sayac+1) {
                if (sayac%2==1) //sayac tek mi?
                tekToplam+=sayac;
                else
                ciftToplam+=sayac;
        }
        std::cout << "Tek Toplam:" << tekToplam << " Çift Toplam:" << ciftToplam;
}
/* Program Çıktısı:
İki Sayı Giriniz: 9 17
Tek Toplam: 65 Çift Toplam: 52
*/
```

Bir ve kendisinden başka tam böleni olmayan sayıya **asal sayı** denir. Klavyeden girilen pozitif bir tam sayının asal olup olmadığını ekrana yazdıran C++ programı aşağıdaki şekilde kodlanabilir;

```
#include <iostream>
int main() {
        int sayi,asal=1;
        std::cout << "Bir Sayı Giriniz:";
        std::cin >> sayi;
        for (int sayac=sayi-1; (sayac>1)&&(asal==1); sayac--)
                if (sayi%sayac==0)
                        asal=0;
        if(asal)
                std::cout << "Girilen " << sayi << " sayısı asaldır.";
        else
                std::cout << "Girilen " << sayi << " sayısı asal değildir.";
}
```

Klavyeden girilen pozitif bir tam sayının **asal çarpanlarını** ekrana yazdıran C++ programı aşağıdaki şekilde kodlanabilir;

```
#include <iostream>
int main() {
        int sayi;
```

```
        std::cout << "Bir Sayı Giriniz:";
        std::cin >> sayi;
        for (int sayac=sayi; sayac>0; sayac--)
        if (sayi%sayac==0)
        std::cout << "Asal çarpan:" << sayac <<std::endl;
}
/* Program Çıktısı:
Bir Sayı Giriniz:12
Asal çarpan:12
Asal çarpan:6
Asal çarpan:4
Asal çarpan:3
Asal çarpan:2
Asal çarpan:1
*/
```

§6.4.5 İç İçe Döngü Kodu

Döngüler içinde yer alan döngü kodu bir başka döngüyü içerebilir. Örneğin ekrana satır ve sütun içeren bir şekil oluşturmak istediğimizde **iç içe döngü** (nested loop) kullanırız. Aşağıda buna ilişkin bir örnek verilmiştir. İlk `for` döngüsünün tekrarlanan kodu ikinci bir `for` döngüsüdür.

```
#include <iostream>
int main() {
        int satir,sutun;
        for (satir=0; satir<5; satir++) {
                for (sutun=0; sutun<4; sutun++)
                std::cout << "x";
                std::cout << std::endl;
        }
}
/* Program Çıktısı:
xxxx
xxxx
xxxx
xxxx
xxxx
*/
```

Bir başka örnek olarak elemanları, satır ve sütun çarpımları olan 4x5 boyutlarındaki matrisi ekrana yazdıran program aşağıda verilmiştir;

```
#include <iostream>
#include <iomanip>
int main() {
        int satir,sutun;
        //Başlığı Yazdıran Kısım
        std::cout << "   Sütun: ";
        for(sutun=0; sutun<3;sutun++)
        std::cout << std::setw(3) << sutun+1 << " ";
        std::cout << std::endl;

        //Matrisi Yazdıran Kısım
        for (satir=0; satir<5; satir++) { //Satır için
                std::cout << std::setw(2) << std::setfill('0')  << satir+1 << ".Satir: ";
                ↪ //İmleç, yazdırılan metnin sonunda olacaktır.
                for(sutun=0; sutun<3;sutun++) //Sütun için
                std::cout << std::setw(3) << std::setfill('0') <<sutun << " "; //İmleç,
                ↪ yazdırılan metnin sonunda olacaktır.
                std::cout << std::endl; //Satır işi bitince imleci yeni satıra geçir!
        }
}
/* Program Çıktısı:
   Sütun:   1   2   3
```

```
01.Satir: 000 001 002
02.Satir: 000 001 002
03.Satir: 000 001 002
04.Satir: 000 001 002
05.Satir: 000 001 002
*/
```

§6.4.6 Gözcü Kontrollü Döngüler

Döngüler her zaman bir sayaca bağlı çalıştırılmaz. Bir döngüden çıkış koşulu döngü kodu içerisinde üretilen bir değere bağlı ise bu değere **gözcü değeri** (sentinel value) adı verilir. Buradaki gözcü değeri beklenen bir değer dışındaki bir değerdir. Buna örnek olarak klavyeden `0` girilene kadar girilen rakamları toplayan bir program verilmiştir.

```
#include <iostream>
int main() {
        int sayi; /* okunan tam sayı */
        int toplam(0); /* girilen sayıların toplamı. başlangıçta 0*/
        do {
                std::cout << "Bir sayı giriniz (Bitirmek için 0): ";
                std::cin >> sayi;
                toplam+=sayi; // sayı 0 girilse bile toplam etkilenmez
        } while (sayi != 0); //sentinel value 0

        std::cout << "Girilen Sayıların Toplamı:" << toplam;
}
```

Aşağıda pozitif sayı girilmesini sağlayan bir başka kod örneği verilmiştir.

```
#include <iostream>
int main() {
        int sayi,pozitifSayi(0);
        do { /* Pozitif girilmesini zorluyoruz */
                std::cout << "Lütfen pozitif sayı giriniz: ";
                std::cin >> sayi;
                if (sayi<=0)
                st6d::cout << "HATA:Pozitif Sayı GİRMEDİNİZ!" << std::endl;
        } while (sayi <= 0);
        pozitifSayi=sayi;
        std::cout <<"Girilen pozitif tam sayı: " << pozitifSayi;
}
```

Aynı örnek bir başka şekilde aşağıdaki gibi kodlanabilir. Ancak burada 4. ve 5. satırlar ile 8. ve 9. satırların aynı olduğuna ve kod tekrarı yapıldığına dikkat ediniz!

```
1  #include <iostream>
2  int main() {
3          int sayi,pozitifSayi(0);
4          std::cout << "Lütfen pozitif sayı giriniz: ";
5          std::cin >> sayi;
6          while (sayi <= 0) {
7                  std::cout << "HATA:Negatif sayı giriniz!" << endl;
8                  std::cout << "Lütfen pozitif sayı giriniz: ";
9                  std::cin >> sayi;
10         }
11         pozitifSayi=sayi;
12         std::cout <<"Girilen pozitif tam sayı: " << pozitifSayi;
13 }
```

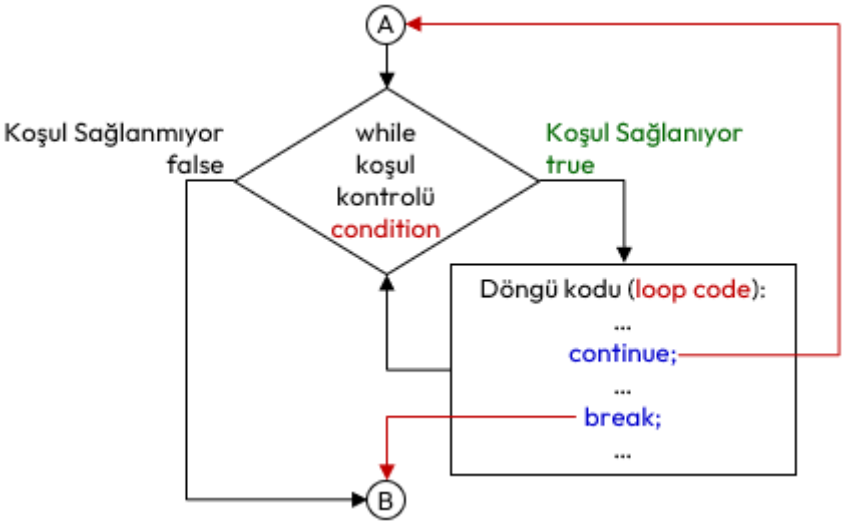
6.5 Dallanmalar

Dallanmalar (jump), bir döngünün yada fonksiyonun icrasını kesen ve bir başka talimata yönlendiren talimatlardır.

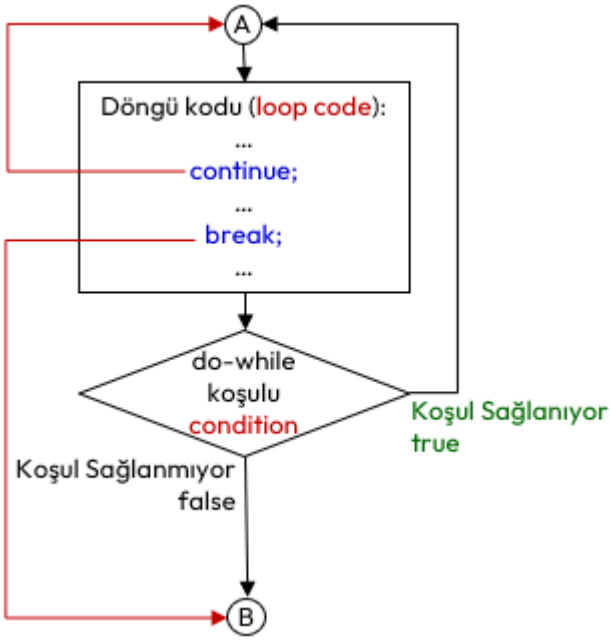
§6.5.1 Continue ve Break Talimatları

Continue talimatı , yalnızca geçerli **yinelemeyi** (iteration) sonlandırır ve sonraki yinelemelerle devam eder. Yani bu talimat, bir sonraki yinelemeyi yapmak için kullanılır. Dolayısıyla sadece `while`, `do-while` ve `for` **döngü kodları** (loop code) içinde kullanılabilir.

Break talimatı , döngüyü sonlandırır ve program icrası döngü sonrası ilk talimattan devam eder. Bunu daha önce `switch` talimatında görmüştük. Bu talimat, aynı şekilde `while`, `do-while` ve `for` döngüleri ile kullanılabilir. Bu talimatlarının döngü kodlarında kullanılması halinde icra akışı Şekil 6.12'de verilmiştir.



(a) While



(b) Do-While

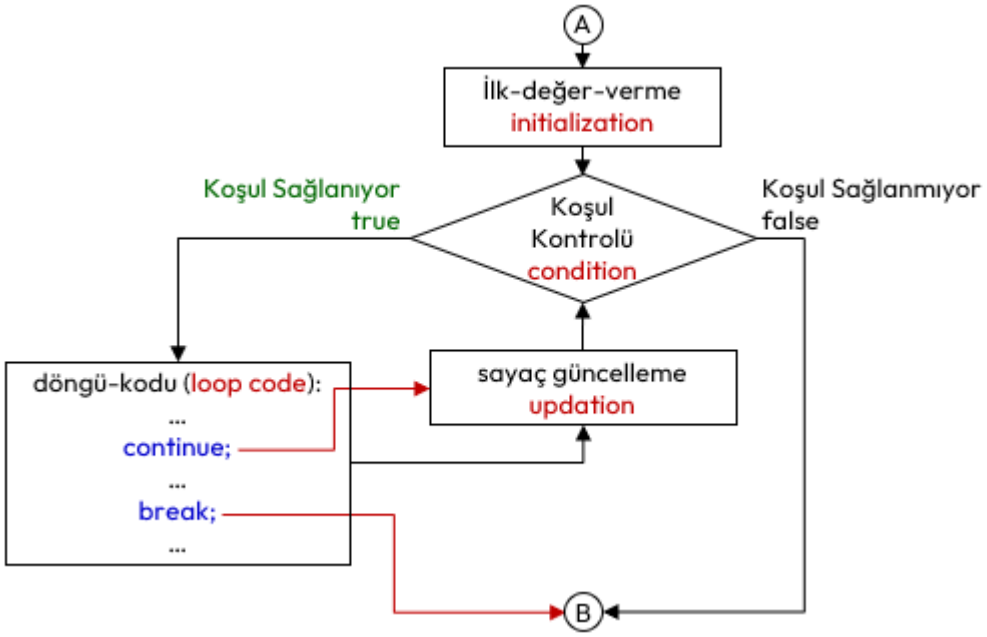
Şekil 6.12: While ve Do-While Döngülerinde Break ve Continue Talimatları İcra Akışı

Bu talimatlar döngü kodu içinde amacımıza uygun olarak birden çok kullanabiliriz. Aşağıda 0 ile 15 arasındaki sayıların beşe bölünenleri ve 10 dışındaki sayılar ekrana yazdıran bir program örneği verilmiştir.

```
#include <iostream>
int main() {
        int i;
        for (i=0; i<=15; i=i+1) {
                if (i<7) //i'nin değeri 7 den küçük ise
                        continue;  //bir sonraki yinelemeye (iteration) geç

                if (i==10) //i'nin değeri 10 ise
                        continue;  //bir sonraki yinelemeye (iteration) geç
```

```
                if (i%5==0) // i'nin değeri 5 in katı ise
                        continue;  //bir sonraki yinelemeye (iteration) geç
                std::cout << "i=" << i << std:endl;
        }
}
/*Program Çıktısı:
i = 7
i = 8
i = 9
i = 11
i = 12
i = 13
i = 14
*/
```

Yukarıdaki örnek incelendiğinde döngü bloğu üç adet talimat içeren mantıksal satır (logical sequence) içermesine rağmen 42 adet fiziksel satır (physical sequence) icra edilmiştir. Bazı `continue;` talimatları `std::cout` talimatının icrasını engellemiştir.



Şekil 6.13: For Döngüsünde Break ve Continue Talimatları İcra Akışı

Aşağıda pozitif sayı girilmesini zorlayan **sonsuz döngü** (infinitive loop) içeren program verilmiştir. Programda döngü, pozitif sayı girildiğinde `break;` talimatı ile kırılmaktadır.

```
#include <iostream>
int main() {
        int sayi, pozitifSayi(0);
        while (1) /* Sonsuz döngü */
        {
                std::cout << "Lütfen pozitif sayı giriniz: ";
                std::cin >> sayi;
                if (sayi<=0)
                        std::cout << "HATA:Pozitif Sayı GİRMEDİNİZ!" << std::endl;
                else
                        break; //döngüden çık
        }
        pozitifSayi=sayi;
        std::cout << "Girilen pozitif tam sayı: " << pozitifSayi;
}
```

Bu talimatları dikkatli kullanmalıyız. Bazı durumlarda sonsuz döngüye girme durumu tüm döngü talimatlarında yaşanabilir. `for` Döngüsünde bu durum farklıdır. `continue;` talimatı sonrasında bir sonraki yineleme için önce sayaç güncelleme ifadeleri icra edilir ve sonra koşul testi yapılır.

§6.5.2 Return Talimatı

Yapısal programlamada **ana fonksiyonun** (main function) sonunda çokça kullandığımız bu talimat, bir fonksiyonun üreteceği ya da belirlenen değeri geri döndürür. Kullanıldığı yerden fonksiyon bloğu dışına çıkılır. Aşağıda üç defa negatif sayı girilmesi halinde programı sonlandıran bir program verilmiştir.

```
#include <iostream>
int main() {
        int sayi;
        int pozitifSayi=0;
        int sayac=0; //kaç defa pozitif olmayan sayı girildi.
        do { // Pozitif girilmesini zorluyoruz
                std::cout << "Lütfen pozitif sayı giriniz: ";
                std::cin >> sayi;
                if (sayi<=0) {
                        std::cout <<"HATA: Pozitif Sayı GİRMEDİNİZ!" << std::endl;
                        sayac++;
                        if (sayac==3) {
                                std::cout << "Üç defa negatif sayı girdiniz. ";
                                std::cout << "Programdan Çıkılıyor..." << std::endl;
                                return 1; // ana fonksiyondan çıkılarak işletim sistemine 1
                                 ↪ gönderiliyor.
                        }
                }
        } while (sayi <= 0);
        pozitifSayi=sayi;
        std::cout << "Girilen pozitif tam sayı: " << pozitifSayi;
        return 0;
}
/*Program Çıktısı:
Lütfen pozitif sayı giriniz: -1
HATA: Pozitif Sayı GİRMEDİNİZ!
Lütfen pozitif sayı giriniz: -2
HATA: Pozitif Sayı GİRMEDİNİZ!
Lütfen pozitif sayı giriniz: -3
HATA: Pozitif Sayı GİRMEDİNİZ!
Üç defa negatif sayı girdiniz. Programdan Çıkılıyor.
*/
```

§6.5.3 Goto Talimatı

Tanımlı bir etikete program akışını yönlendiren `goto` talimatıdır. 6.4.1 başlığında örneği verilmiştir.

6.6 Düşün ve Çöz

- **Düşün**: Sarkan else (dangling else) problemi nedir? Bu problemi önlemek için neden her zaman süslü parantez kullanılması önerilir?
- **Düşün**: While ve do-while döngüleri arasındaki farkı, bir kullanıcıdan şifre isteme senaryosu üzerinden karşılaştırınız.
- **Düşün**: Switch talimatında `break;` kullanılmadığında oluşan başarısız olma (fall-through) durumunu bir hata değil, bir özellik olarak nasıl kullanabiliriz?
- **Çöz**: Kullanıcıdan sürekli sayı alan ve `0` girildiğinde o ana kadar girilen en büyük sayıyı ve sayıların ortalamasını ekrana basan bir gözcü kontrollü döngü (sentinel value) tasarımı yapınız.
- **Çöz**: İç içe döngüler kullanarak ekrana 1'den 10'a kadar olan sayıların çarpım tablosunu düzgün bir formatta yazdırınız.
- **Çöz**: Kullanıcı negatif bir sayı girene kadar sayı alan ve girilen tek sayıların toplamını, çift sayıların ise adedini ekrana basan programı yazınız.

7. Bölüm
Fonksiyonlar

7.1 Fonksiyon Nedir?

Yapısal Programlama mantığında çözülecek problem genel olarak fonksiyonların birbirini çağırmasıyla yapıldığı 2.7 başlığında anlatılmıştı. Yani kodlaması yapılacak işi birbirini çağıran fonksiyonlar yazarak ana fonksiyondan bu fonksiyonları çağırarak yaparız. Ayrıca yazılacak programda birbiriyle ilgili fonksiyonlar bir araya toplanarak ayrı bir dosyada tutulur. Bu durum 5.1 başlığı altında anlatılmıştı.

Bu bölüme kadar gördüğümüz kodlarda çözümü oluşturan tüm tasarım parçaları **ana fonksiyon** (main function) içerisine çözülmüştür. Bir bilgisayar programı, genel olarak, tek başına tüm görevleri yerine getiren tek bir main fonksiyonundan oluşmaz. Bu durumda ana problemin büyüklüğüne göre ana fonksiyon bloğumuz uzar, okunabilirliği azalır, müdahale etmek zorlaşır. Bu tip büyük programları kendi içerisinde, her biri özel bir işi çözmek için tasarlanmış parçacıklarından oluşturmak daha mantıklı olacaktır. Yani **böl ve yönet** (divide and conquer) tekniği uygulanır. Çünkü büyük bir problemin toptan çözülmesi, problemin parçalar halinde çözülmesinden her zaman daha yorucu ve zordur. Yazılım geliştirmede problemi büyük ana parçalara ayırırız. Daha sonra bu büyük parçaları daha küçük parçalara ayırarak çözeriz. Buna **yukarıdan aşağıya** (top down) yaklaşım adı verilir.

C dilindeki gibi C++ dilinde de fonksiyonlar;

- Fonksiyonlar, belirli bir görevi gerçekleştiren bir kod bloğudur.
- Parametreli alabilir, girdilerle ya da parametresiz bir şeyler yapar ve ardından cevabı verebilir. Bilgiyi fonksiyona argümanlar ile aktarırız.
- Her fonksiyonun bir kimliği vardır.
- Bir fonksiyon program içerisinde istenildiği kadar farklı yerlerden ulaşılarak çalıştırılabilir ya da çağrılabilir. Bir başka deyişle tekrar kullanılabilir.
- Bir fonksiyon, çağıran koda bir değer döndürebilir.
- Bir fonksiyon, bir başka fonksiyondan çağrılabilir.

C++ dilinde iki tip fonksiyon bulunmaktadır; başlık dosyalarında bize sunulan **hazır fonksiyonlar** ve programcının tanımlayacağı **kullanıcı tanımlı fonksiyonlar** (user defined function). Hazır fonksiyonlar, programcı için hazırlanmış ve her işletim sistemi için derlenmiş ve test edilmiş modüllerde (başlık dosyalarında) yer alır. Yani bir modül içindeki öğeler tek bir amaç etrafında birlikte çalışacak şekilde belirlenir. Bu belirleme işleminde **yüksek uyum** (high cohesion) ya da tutarlılık aranır. Yazılım modülleri arasındaki karşılıklı **düşük bağlaşımın** (low coupling) olması istenir. Yani modüller birbirine en az bağımlılık ile hazırlanmalıdır.

7.2 Hazır Fonksiyonlar

C++ geliştiricileri, programcıların kullanmaları için, önceden yazılmış olan hazır fonksiyonlar hazırlamışlardır. Hazır fonksiyonlar teknik olarak C++ dilinin parçası değillerdir. Yalnızca standart hale getirilmişlerdir. Programcı, kullanmak istediği hazır fonksiyon hangi modülde ise o modüle ilişkin **başlık** (header) dosyasını `#include` **ön işlemci yönergesi** (preprocessor directive) ile kendi projesine dahil eder. Aslında bu başlık

Kütüphane	Örnek Fonksiyon	Açıklama
`<cmath>`	`sqrt(x)`	x sayısının karekökünü hesaplar.
`<cstdlib>`	`rand(x)`	0 ile x arasında rastgele bir sayı üretir.
`<algorithm>`	`sort(b, e)`	Belirtilen aralığı sıralar.
`<string>`	`getline()`	Boşluk içeren metin satırını akıştan okur.
`<cctype>`	`isdigit(c)`	Karakterin rakam olup olmadığını denetler.

Tablo 7.1: Sık Kullanılan Standart C++ Fonksiyonlarından Bazıları

dosyalarında;

- Hazır fonksiyonların sadece **prototipleri** (prototype), başlık (header) dosyalarında bulunur. Prototip derleyiciye, parametreleri ve geri dönüş değeri şu olan bir fonksiyon var demenin yoludur.
- Fonksiyonların kendileri yani derlenmiş halleri her işletim sistemi için ayrı derlenmiş olan kütüphane (library) dosyaları içerisinde bulunur.
- Derleme işlemi sırasında **bağlayıcı** (linker) program tarafından bu fonksiyonlar icra edilebilir dosyaya

(executable file) dahil edilir.

§7.2.1 cmath Başlık Dosyası

Matematik işlemleri gerçekleştirmek için standart hale getirilmiş bir başlık dosyasıdır. Tablo 7.2'de fonksiyon listesi verilmiştir.

Kategori	Fonksiyon	Açıklama
Üstel ve Log.	`pow(x, y)`	x^y değerini hesaplar.
	`exp(x)`	e^x değerini hesaplar.
	`sqrt(x)`	x sayısının karekökünü hesaplar.
	`log10(x)`	x sayısının 10 tabanında logaritmasını ($\log x$) hesaplar.
	`log(x)`	x sayısının doğal logaritmasını ($\ln x$) hesaplar.
Yuvarlama	`ceil(x)`	x'i kendisinden büyük veya eşit en küçük tam sayıya yuvarlar.
	`floor(x)`	x'i kendisinden küçük veya eşit en büyük tam sayıya yuvarlar.
	`round(x)`	x'i en yakın tam sayıya yuvarlar.
Trigonometrik	`sin(x)`	Radyan cinsinden verilen açının sinüsünü hesaplar.
	`cos(x)`	Radyan cinsinden verilen açının kosinüsünü hesaplar.
	`tan(x)`	Radyan cinsinden verilen açının tanjantını hesaplar.
Mutlak Değer	`abs(x)`	Tam sayıların mutlak değerini döndürür.
	`fabs(x)`	Ondalıklı sayıların mutlak değerini döndürür.
Diğer	`fmod(x, y)`	x/y işleminin kalanını (ondalıklı) döndürür.
	`hypot(x, y)`	$\sqrt{x^2+y^2}$ (hipotenüs) değerini hesaplar.

Tablo 7.2: <cmath> Kütüphanesi Temel Fonksiyonları

Örnek olarak kenarları verilen bir üçgenin alanını hesaplayan bir program aşağıda verilmiştir.

```
#include <iostream>
#include <cmath>
int main() {
        int kenar1,kenar2,kenar3;
        double alan,kenarlarToplamininYarisi;
        std::cout << "Üçgenin Kenarlarını Giriniz:";
        std::cin >> kenar1 >> kenar2 >> kenar3;
        if ((kenar1+kenar2 <= kenar3) ||
                (kenar2+kenar3 <= kenar1) ||
                (kenar1+kenar3 <= kenar2) ) {
                        std::cout << "Böyle bir üçgen olamaz." << endl;
                        return 1;
        }
        kenarlarToplamininYarisi=(kenar1+kenar2+kenar3)/2.0;
        alan=sqrt( kenarlarToplamininYarisi*
                (kenarlarToplamininYarisi-kenar1)*
                (kenarlarToplamininYarisi-kenar2)*
                (kenarlarToplamininYarisi-kenar3) );
        std::cout << "Üçgenin alanı: " << alan ;
}
```

§7.2.2 cstdlib Başlık Dosyası

Bu başlıkta;

- Alfa sayısal metinleri ilkel veri tiplerine veya bunun tersini yapan tür dönüşüm fonksiyonları (`atoi()`, `itoa()`, `atof()`, `strtod()`, ...),
- **Çalışma anında** (run-time) bellek tahsisi ve tahsisli belleğin serbest bırakılmasını sağlayan hafıza yerleşim fonksiyonları (`malloc()`, `free()` ...)
- Rastgele sayı üretme fonksiyonları (`rand()`, `srand()`) bulunur.

Şimdilik rastgele sayı üretme üzerinde durulacaktır. Aşağıda bir zar için rastgele sayı üreten bir program verilmiştir.

```
#include <iostream>
#include <cstdlib>
int main() {
        int rastgeleZar;
        rastgeleZar=1 + rand() %6; /* std::rand() fonksiyonunun üreteceği sayıyı kalan işleci ile
         ↪ 6 sayısına         böleriz ki 0 ile 5 arasında bir sayı elde edelim. Buna da 1
         ↪ eklersek zarın rakamları ortaya çıkar. */
        std::cout << "Atılan Zar:" << rastgeleZar;
}
```

Fonksiyon	Açıklama
`int rand();`	`0` ile `RAND_MAX` yani `32767` arasında rastgele bir sayı üretir.
`int srand(unsigned int);`	`rand()` fonksiyonu belli bir başlangıç değerinden itibaren bir dizi matematiksel işlem sonucu rastgele bir değer üretir. `srand()` fonksiyonu, `rand()` fonksiyonuna başlangıç değerini farklı vermek için kullanılır.

Tablo 7.3: Stdlib.h Başlık Dosyasının Rastgele Sayı Fonksiyonları

Fakat programın her çalışmasında `rand()` aynı başlangıç değerini kullandığından **aynı rastgele değerleri üretir**. Bu nedenle aynı zar rakamı gelmiş olur. Bunu değiştirmek için `srand()` fonksiyonu kullanılır.

```
#include <iostream>
#include <cstdlib>
int main() {
        unsigned randBaslangicDegeri;
        std::cout << "std::rand() için başlangıç değeri olabilecek bir sayı giriniz:";
        std::cin >> randBaslangicDegeri;
        srand(randBaslangicDegeri); /* std::rand() fonksiyonunun üreteceği sayı her zaman aynı
         ↪ başlangıç değeriyle başladığından her program çalıştığında üterilecek ilk rastgele
         ↪ sayı ve sonrasındaki sayılar hep olur. Bu hesaplamanın bir başka başlangıçla
         ↪ başlaması üterilecek ilk rastgele sayı ve sonraski üretilecekleri değiştirir. Bunu
         ↪ değiştirmenin yolu std:srand() fonksiyonunu kullanmaktır. */
        int rastgeleZar;
        rastgeleZar=1 + rand() %6;
        std::cout << "Atılan Zar:" << rastgeleZar;
}
```

Rastgele sayı her seferinde kullanıcıdan alınan sayıya göre hesaplandığından her seferinde farklı bir zar rakamı üretilmiş olacaktır. **Ancak başlangıç değerinin her seferinde kullanıcıdan alınması bir başka problemdir**. Bunun üstesinden gelmek için bilgisayarın saatini sayı olarak veren `<ctime>` başlık dosyasındaki `time()` fonksiyonu `nullptr` parametresiyle kullanılabilir.

```
#include <iostream>
#include <cstdlib>
#include <ctime>
int main() {
        unsigned randBaslangicDegeri=time(nullptr);
        /* std:srand() fonksiyonu ile std::rand() fonksiyonu ilk değer verilir. Verilecek ilk
         ↪ değer std::time() fonksiyonuna nullptr verilerek sistem zamanından gelen değer
         ↪ kullanılır.Böylece her çalıştırmada farklı rastgele sayı üretilmesi sağlanacaktır. */
        srand(randBaslangicDegeri); // srand(time(nullptr)); olarak da yazılabilir.
        int rastgeleZar;
        rastgeleZar=1 + rand() %6;
        std::cout << "Atılan Zar:" << rastgeleZar;
}
```

7.3 Kullanıcı Tanımlı Fonksiyonlar

Programcılar kendi fonksiyonlarını oluşturarak kodlarını daha kullanışlı hale getirirler. Bu tür fonksiyonlara **kullanıcı tanımlı fonksiyonlar** (user defined function) denilir.

Şimdiye kadar yazdığımız kodlarda ana fonksiyonumuz olan `main()` hep `sqrt()`, `rand()` gibi fonksiyonları çağırmıştır. Bir fonksiyonu çalıştıran koda, **çağıran program** adı verilir. Hazır fonksiyonların işlendiği 7.2 başlığındaki örneklerde `main()` ana fonksiyonumuz hep çağıran programdır.

Kullanıcı tanımlı fonksiyonu oluşturmak ve kullanmak için bu dört unsuru bilmemiz gerekir;

1. **Fonksiyon Tanımı** (function definition): Bir fonksiyonun girdileri, yaptığı işlem ve döndüreceği değeri içerecek şekilde **başlık ve gövdesinin kodlanmasından oluşur**. Fonksiyon tanımı, fonksiyonu derleme yapılabilmesi için eksiksiz bir şekilde tamamlar. **Başlık**, fonksiyonun döndüreceği veri tipi, fonksiyon kimliği ve aç kapa parantez içindeki parametre listesinden oluşur. **Gövde** ise süslü parantezler ve içindeki fonksiyonun taptığı işi içeren kodlardır.
2. **Fonksiyon Bildirimi** (function declaration): Derleyiciye, programın ilerleyen kısımlarında (veya belki başka bir dosyada) verilen kimlikte ve verilen parametrelere sahip bir fonksiyonun var olduğunu söyler. Derleyici bu sayede, fonksiyonun gövdesini görmese bile onun nasıl çağrılacağını, dönüş tipi ve argümanlarını bilir. Bildirimin amacı, derleyiciye "Böyle bir fonksiyon var " bilgisini vermektir. Bildirim yapılıp, tanımı yapılmayan bir bir fonksiyona sahip program derlenmeye çalışıldığında, bağlama zamanında (link-time) hata alırız.
3. **Fonksiyon Prototipi** (function prototype): Aslında fonksiyon bildirimi ile aynı şeydir. Ancak prototip terimi, genellikle fonksiyonun kodun en tepesinde, `main()` fonksiyonundan önce yapılan bildirimini ifade etmek için kullanılır.
4. **Fonksiyon Çağırma**(function call/invoke function): Çağıran program içinde, bildirimi daha önce yapılmış bir fonksiyon, kimliğini ve ardından parantez içindeki argüman listesini yazarak çağırabiliriz.

§7.3.1 Fonksiyon Tanımı Nasıl Yapılır?

Fonksiyon tanımı (function definition), fonksiyonun işleyip **döndüreceği veri tipini**, **kimliği**, işleyeceği bağımsız değişkenleri temsil eden **parametre listesini**, içerecek şekilde **başlığının** ve **gövdesinin** kodlanmasından oluşur. Aşağıda bir fonksiyon tanımının sözde kodu verilmiştir;

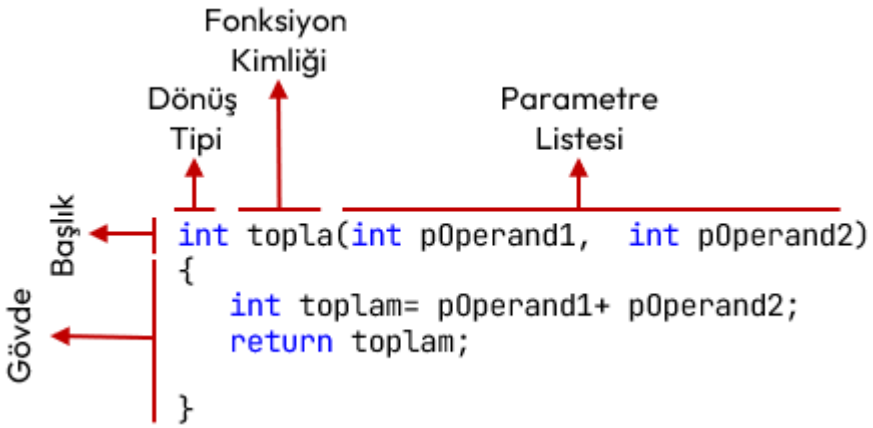
```
geri-dönüş-tipi fonksiyon-kimliği(parametre-listesi) //BAŞLIK
{ //GÖVDE BAŞLANGICI
 //...
 // Algoritma
 //...
 return geri-dönüş-değeri;
 //...
} //GÖVDE BİTİŞİ
```

Sözde kodu şu şekilde açıklayabiliriz;

- **Fonksiyonun kimliği benzersizdir**. Aynı zamanda **fonksiyon adı** (function name) olarak adlandırılır. Fonksiyon kimliklendirmede, 4.3.5 başlığındaki değişken kimliklendirme kuralları geçerlidir.
- Fonksiyon kimliğinden sonra **parametreleri barındıracak açıp kapatılan parantez** olmalıdır. Bu durum değişken tanımından fonksiyon tanımlarını ayırmak içindir.
- Bağımsız değişkenler, parametre olarak adlandırılır;
  - **Parametre listesi**, bir fonksiyonun parametrelerinin veri tipini, sırasını ve sayısını belirtir.
  - Parametreler isteğe bağlıdır, yani bir fonksiyon hiçbir parametre içermeyebilir.
  - **Ber bir parametreye kimlik verilmelidir**. Çünkü fonksiyon gövdesinde bu değişkenler işlenir.
  - Parametreler birbirinden **virgül işleci** (comma operator) ile ayrılır.
- Parametreler fonksiyon **gövdesinde** yapılacak işlere temel oluşturur;
  - Gövde, fonksiyonun ne yaptığını tanımlayan ve süslü parantezler arasında yer alan talimatları içerir.
  - Gövdede, **yapısal programlama yapıldığından**, ilk önce olacaksa değişkenler tanımlanmalı daha sonra parametrelerle birlikte bu değişkenleri işleyecek talimatlar yazılmalıdır.
  - Gövdede, fonksiyonda istenilen sonuca ulaşmak için kullanacağı adımlara karar verilir yani **algoritması belirlenir**.
- İşlenen parametrelerden ve gövde içerisinde tanımlanan değişkenlerden bir **geri dönüş değeri** hesaplanabilir. Değer hesaplanmış ise `return geri-dönüş-değeri;` talimatıyla fonksiyonun icrası sonlandırılır. textttreturn Talimatı çoğunlukla fonksiyonundaki son talimat olarak görünür ama fonksiyon gövdesinde herhangi bir yerden geri dönülebilir.
- `return` Talimatıyla geri döndürülen değerin tipi, **fonksiyonun geri dönüş tipidir**.
- Bazı durumlarda fonksiyon, geri değer döndürmeyebilir;
  - Bu durunda fonksiyonun **geri dönüş tipi yoktur** ve bu nedenle geri dönüş tipi`void` olarak belirtilir.
  - Geri dönüş tipi `void` olan fonksiyonun icrası `return;` talimatıyla sonlandırılır.
  - Eğer `return;` talimatı hiç yazılmaz ise süslü parantez kapatılıncaya kadar fonksiyon icra edilir

ve fonksiyondan geri dönülür.

> Bir de fonksiyonun çağrıldığı yerde, **fonksiyonun döndürdüğü değerin tipi ile eşleşen bir değişkene dönüş değeri atanabilir**. Zorunlu değildir.
> Ayrıca C++ dilinde **bir fonksiyon gövdesinde bir başka fonksiyon tanımlanamaz!**



Şekil 7.1: Fonksiyon Tanımı Örneği

Şekil 7.1'de; iki parametresini toplayıp, toplamı geri döndüren bir `topla` fonksiyonunun tanımlaması verilmiştir. Aşağıdaki şekilde açıklayabiliriz;

- Fonksiyonun kimliği `topla` olarak belirlenmiştir;
- `topla` Fonksiyonunun parametre listesinde iki adet `int` veri tipinde parametre (`pOperand1`, `pOperand2`) tanımlanmıştır.
- `topla` Fonksiyonu gövdesinde yapısal programlama yapıldığından ilk önce `toplam` yerel değişkeni tanımlandı. Daha sonra parametrelerin toplamı `toplam` değişkenine atandı.
- Son olarak `return toplam;` talimatı ile hesaplan değer geri döndürülmüştür.

Aşağıda bu `topla` fonksiyonu kullanan bir program örneği verilmiştir;

```
#include <iostream>

/* 1. Fonksiyon Tanımlama (Function Definition): */
int topla(int pOperand1,int pOperand2) //Fonksiyon başlığı
{ //Fonksiyon Gövdesi başlangıcı
        int toplam=pOperand1+ pOperand2;
        return toplam; // doğrudan "return pOperand1+ pOperand2;" da yazılabilirdi.
} //Fonksiyon Gövdesi bitişi: Sonunda noktalı virgül OLMAZ!
/* Bu noktadan sonra "topla" fonksiyonu her yerden çağrılabilir! */

/* 2. ÇAĞRI ORTAMI: main fonksiyonu içinde bu fonksiyon çağrılacak: */
int main () {
        int sonuc;

        /* 3. topla fonksiyonu 1 ve 2 argümanlarıyla çağrılıyor ama geri döndürdüğü değer
         ↪ kullanılmıyor. */
        topla(1,2);
        /* 4. topla fonksiyonu 2 ve 3 argümanlarıyla çağrılıyor. Geri döndürdüğü değer std::cout
         ↪ içine argüman olarak giriyor. */
        std::cout << "1. toplam:" <<  topla(2,3) << std::endl;

        /* 5. topla fonksiyonu 3 ve 4 argümanlarıyla çağrılıyor. Geri döndürdüğü değer sonuc
         ↪ değişkenine atanıyor.*/
        sonuc= topla(3,4);
        std::cout << "2. toplam:" << sonuc << std::endl;

        /* 6. topla fonksiyonu iki kez çağrılıyor. Her iki çağrıdaki geri dönüş değeri gecici bir
         ↪ yerde saklanıp toplamı sonuc değişkenine atanıyor. */
        sonuc= topla(1,2)+topla(3,4);
```

```
        std::cout << "3. toplam:" << sonuc << std::endl;

        /* 6. topla fonksiyonu üç kez çağrılıyor. İlk çağrıdaki pOperand1 parametresine
         ↪  topla(2,3) fonksiyonunun sonucu argüman olarak geçiriliyor. */
        sonuc= topla( topla(1,2) ,3)+topla(4,5);
        std::cout << "4. toplam: " << sonuc << std::endl;
}

/* Program Çıktısı:
1. toplam:5
2. toplam:7
3. toplam:10
4. toplam: 15
*/
```

Örnek program, aslında 2.7 başlığında bahsedilen yapısal programlamanın çerçevesidir. Ana program içinde problemin çözümü, çeşitli fonksiyonların birbirini çağırması ile yapılmıştır. Fonksiyon içinde işlenecek veriye ilişkin değişken tanımlanmış ve bu veriler ile veri yapılarını işleyecek kontrol işlemleri (bizim örneğimizde kontrol işlemi yok) birbirinden ayrılmıştır. **Okunaklılık çok yüksek bir kod elde edilmiştir.**

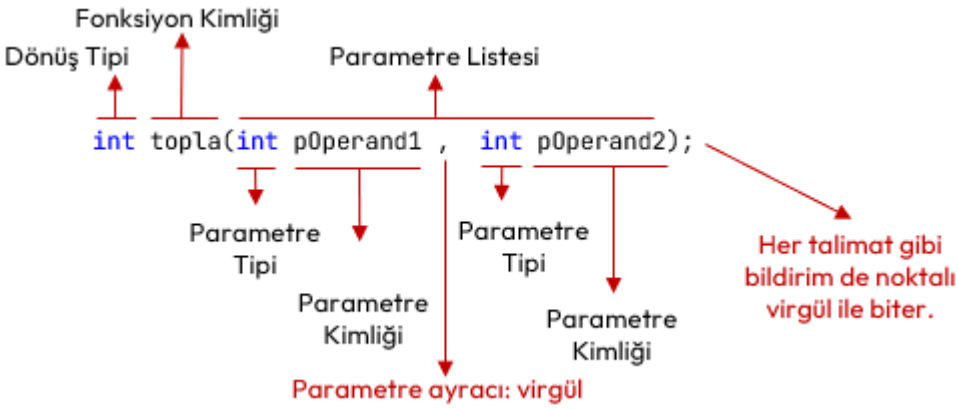
§7.3.2 Fonksiyon Bildirimi Nasıl Yapılır?

Fonksiyon bildirimi (function declaration), derleyiciye, programın ilerleyen kısımlarında (belki başka bir dosyada) belirtilen kimlikte ve verilen parametrelere sahip bir fonksiyonun var olduğunu söyler. Aşağıda bir fonksiyon bildiriminin sözde kodu verilmiştir;

```
geri-dönüş-tipi fonksiyon-kimliği(parametre-listesi);
```

Fonksiyon bildirimi, fonksiyon tanımındaki **başlık kısmı gibidir**;

- Bildirimin, parametre listesinde, **parametre kimlikleri zorunlu değildir.**
- Bildirim, talimatlar gibi **noktalı virgül ile biter**.
- Bildirimi yapılan fonksiyonun, fonksiyon tanımı (function definition) bir yerde mutlaka yapılmalıdır!
- Bildirimi yapılmış fonksiyonun tanımında aynı kimlik kullanılıdır.



Şekil 7.2: Fonksiyon Bildirimi Örneği

Şekil 7.2'de iki adet `int` veritipinde parametresi olan ve `int` veri tipinde geri değer döndüren `topla` kimlikli bir fonksiyon bildirimi örneği verilmiştir. Bildirim örneğini aşağıdaki şekilde açıklayabiliriz;

- Fonksiyonun kimliği `topla` olarak belirlenmiştir;
- `topla` Fonksiyonunun parametre listesinde iki adet `int` veri tipinde parametre (`pOperand1`, `pOperand2`) tanımlanmıştır. Parametre kimlikleri verilmeyebilirdi. Yani `int topla(int,int);` aynı bildirimdir.
- Bildirimde `topla` fonksiyonunun geri dönüş veri tipi, `int` olarak belirlenmiştir.

Fonksiyon tanımının yapıldığı 7.3.1 başlığındaki `topla` örneğimizi önce bildirim yapılarak aşağıdaki şekilde yazılabilir;

```
#include <iostream>

/* 1. Fonksiyon Bildirimi (Function Declaration): */
int topla(int, int);
/* Bu noktadan sonra bu bildirim ile topla fonksiyonu çağrılabilir ancak tanımı sonradan
 ↪  yapılmalıdır! */
```

```
/* 2. ÇAĞRI ORTAMI: main fonksiyonu içinde bu fonksiyon çağrılacak: */
int main () {
        int sonuc;

        /* 3. topla fonksiyonu 1 ve 2 argümanlarıyla çağrılıyor ama geri döndürdüğü değer
         ↪ kullanılmıyor. */
        topla(1,2);
        /* 4. topla fonksiyonu 2 ve 3 argümanlarıyla çağrılıyor. Geri döndürdüğü değer std::cout
         ↪ içine argüman olarak giriyor. */
        std::cout << "1. toplam:" <<  topla(2,3) << std::endl;

        /* 5. topla fonksiyonu 3 ve 4 argümanlarıyla çağrılıyor. Geri döndürdüğü değer sonuc
         ↪ değişkenine atanıyor.*/
        sonuc= topla(3,4);
        std::cout << "2. toplam:" << sonuc << std::endl;

        /* 6. topla fonksiyonu iki kez çağrılıyor. Her iki çağrıdaki geri dönüş değeri gecici bir
         ↪ yerde saklanıp toplamı sonuc değişkenine atanıyor. */
        sonuc= topla(1,2)+topla(3,4);
        std::cout << "3. toplam:" << sonuc << std::endl;

        /* 6. topla fonksiyonu üç kez çağrılıyor. İlk çağrıdaki pOperand1 parametresine
         ↪ topla(2,3) fonksiyonunun sonucu argüman olarak geçiriliyor. */
        sonuc= topla( topla(1,2) ,3)+topla(4,5);
        std::cout << "4. toplam: " << sonuc << std::endl;
}

/* 7. Fonksiyon Tanımlama (Function Definition): */
int topla(int pOperand1,int pOperand2) //BAŞLIK
{ //GÖVDE Blok Başlangıcı
        return pOperand1+ pOperand2;
} //GÖVDE Blok Bitişi
```

Fonksiyon tanımının yapıldığı 7.3.1 başlığındaki gibi **hiç bildirim yapılmadan da fonksiyonlar tanımlanabilir**. Ancak fonksiyon tanımlandığı yerden sonra bildirimi yapılmış sayılıp çağrılabilir!

Aşağıdaki fonksiyon bildirimi içeren bir başka programı inceleyelim. Bu program derleyici tarafından derlenir ancak **bağlama** (link) aşamasında bağlayıcı program (linker) hata verir. Çünkü **fonksiyon tanımlamaları** (function definition) yapılmamıştır!

```
/* 1. Fonksion Bildirimleri (Function Declaration): */
/* "ikiSayiyiTopla" kimlikli bir fonksiyon olduğu ve iki tam sayı parametre aldığı ve ayrıca bir
 ↪ tam sayı geri döndürdüğü derleyiciye bildiriliyor. */
int ikiSayiyiTopla(int,int);

/* "sayiyiKonsolaYaz" kimlikli bir fonksiyon olduğu ve bu fonksiyonun bir tam sayı parametre
 ↪ aldığı ve geri değer döndürmediği derleyiciye bildiriliyor. */
void sayiyiKonsolaYaz(int);

/* "konsoldanTamsayiOku" kimlikli bir fonksiyon olduğu ve bu fonksiyonun bir parametre almadığı
 ↪ ve bir tam sayı geri döndürdüğü derleyiciye bildiriliyor. */
int konsoldanTamsayiOku();

/* 2. ÇAĞRI ORTAMI: main fonksiyonu içinde bu fonksiyonlar çağrılacak: */
int main() {
        int sayi;
        sayi=konsoldanTamsayiOku();
        sayiyiKonsolaYaz(13); //sayiyiKonsolaYaz fonksiyonu 13 argümanı ile çağrıldı
        sayiyiKonsolaYaz(sayi); //sayiyiKonsolaYaz fonksiyonu sayi argümanı ile çağrıldı
        int toplam= ikiSayiyiTopla(sayi,20);  //ikiSayiyiTopla fonksiyonu sayi ve 20 argümanları
         ↪ ile çağrıldı
```

```
        //...
}
```

Program ana fonksiyondan başladığından her programcı ana fonksiyona odaklanır. Bu nedenle **ilk önce fonksiyon bildirimleri yapılıp fonksiyon tanımlarının ana fonksiyon sonrasına bırakılması etik bir davranıştır**.

Aşağıda pozitif sayı girilmesini garanti ettikten sonra sayının 10 sayısına bölünebilirliğini bulan program verilmiştir.

```
#include <iostream>
/* 1. Fonksion Bildirimleri (Function Declaration): */
int pozitifSayiOku(); // parametre almayan bir fonksiyon bildirimi.
int onaBolunurMu(int);
/* 2. ÇAĞRI ORTAMI: main fonksiyonu içinde bu fonksiyonlar çağrılacak: */
int main () {
        int birPozitifSayi=pozitifSayiOku();
        if (onaBolunurMu(birPozitifSayi))
                std::cout << "Girilen Pozitif Sayı 10 ile bölünebilir.";
        else
                std::cout << "Girilen Pozitif Sayı 10 ile tam bölünmez.";
}
/* 3. Fonksiyon Tanımlamaları (Function Definition): */
int pozitifSayiOku() {
        int okunan;
        do {
                std::cout << "Lütfen pozitif sayı giriniz: ";
                std::cin >> okunan;
                if (okunan<=0)
                        std::cout << "HATA:Pozitif Sayı GİRMEDİNİZ!" << std::endl;
        } while (okunan <= 0);
        return okunan;
}
int onaBolunurMu(int p){
        return (p%10)==0;
}
```

Bunların dışında hiçbir değer geri döndürmeyen fonksiyonlar da tanımlayabiliriz. Bunlara diğer dillerde **prosedür** (procedure) adı da verilir. Bu durumda değeri olmayan veri tipi olan `void` anahtar kelimesi kullanılır. Bu tür fonksiyonların da geri dönüş talimatı yalnızca `return;` şekilde yazılır.

```
#include <iostream>
/* 1. Fonksion Bildirimleri (Function Declaration): */
void printMenu(); // geri değer döndürmeyen fonksiyon bildirimi.
/* 2. ÇAĞRI ORTAMI: main fonksiyonu içinde bu fonksiyonlar çağrılacak: */
int main() {
        int secim;
        do {
                printMenu();
                std::cin >> secim;
                switch(secim) {
                case 1:
                        std::cout << "Verileri Yazdım..." << std::endl;
                        break;
                case 2:
                        std::cout << "Verileri Okudum..." << std::endl;
                        break;
                case 0:
                        std::cout << "Programdan Çıkılıyor..." << std::endl;
                        break;
                default:
                        std::cout << "Tekrar Seçim Yapınız!" << std::endl;
                }
```

```
        } while (secim!=0);
}
/* 3. Fonksiyon Tanımlamaları (Function Definition): */
void printMenu() {
        std::cout << std::endl;
        std::cout << "1-Verileri Yaz." << std::endl;
        std::cout << "2-Verileri Oku." << std::endl;
        std::cout << "0-ÇIKIŞ." << std::endl;
        return;
}
```

Tek Tanım Kuralı:

- Bildirimi yapılan bir fonksiyon, bir başlık (header) dosyasında olabilir ve bu **başlık dosyası birden çok kaynak kodumuza dahil edilebilir**.
- Bildirimi yapılan bir fonksiyon, derleyici tarafından **dahil edileceği kaynak dosyaların tamamında yalnızca bir kez tanımlanabilir (one definition rule)**. Kısaca birden çok dosya için derleme söz konusu olduğunda birden çok bildirim olabilir ama bir adet fonksiyon tanımı olmalıdır.
- İleride **başlık koruması** (header guard) kavramının detayı verilecektir.

7.4 Çağrı Kuralı

Çağrı kuralı (calling convention), bir fonksiyon çağrısıyla karşılaşıldığında argümanların fonksiyona hangi sıraya geçirileceğini belirtir. İki olasılık vardır; Birincisi argümanlar soldan başlanarak sağa doğru fonksiyona geçirilir. İkincisi ise C/C++ dilinde kullanılır ve **argümanlar sağdan başlanarak sola doğru fonksiyona geçirilir**.

Buna örnek olarak aşağıdaki verilen kod örneğindeki durumu inceleyelim;

```
#include <iostream>
void xyzYaz(int x, int y, int z) {
        std::cout << x << " " << y << " "<< z << std::endl;
}
int main() {
        int a = 1;
        xyzYaz(a, ++a, a++);
}
```

Bu kodun çıktısı `1 2 3` olarak beklenir ancak çağrı kuralından ötürü çıktı `3 3 1` olur. **Bunun nedeni C/C++ dilinin çağrı kuralının sağdan sola olmasıdır.** Bunu şöyle açıklayabiliriz;

1. Fonksiyona sağdan ilk argüman olarak `a` değişkeninin değeri 1 geçirilir.
2. Ardından `a++` ifadesi ile `a` değişkeni 2'ye yükseltilir.
3. Sonra `++a` ifadesi ile `a` değişkeni 3'e yükseltilir.
4. Fonksiyona ikinci argüman olarak 3 geçirilir.
5. Fonksiyona son olarak `a` değişkeninin son değeri olan 3 geçirilir.

7.5 Depolama Sınıfları

Depolama sınıfları (storage classes) bir değişkenin ömrünü (lifetime), görünürlüğünü (visibility), bellek konumunu (memory location) ve başlangıç değerini (initial value) belirlemek için kullanılır. Bu özellikler temel olarak bir programın çalışma süresi boyunca belirli bir değişkenin varlığını izlememize yardımcı olan **kapsam** (scope) görünürlüğü ve yaşam süresini içerir.

Yapısal programlama ve fonksiyon kavramının bilgisayarlarda kullanılmasıyla birlikte işlemcilerde de değişiklikler olmuştur. Şekil 1.3'de anlatılan işlemciye yeni kaydediciler de eklenmiştir;

- Veriler ile icra edilen kod farklı bellek bölgesinde bulunur. Bu nedenle verileri işaret eden **veri adresleri kaydedicisi** (data memory register) işlemciye eklenmiştir.
- **Yığın bellek kaydedicisi** (stack register), fonksiyon çağrıları sırasında mevcut işlemci durumu ve yerel değişkenler gibi bazı verileri depolamak için kullanılan belleği gösteren (stack pointer) adresi tutan kaydedici işlemciye eklenmiştir.
- **Bayrak kaydedicisi** (flag register), işlemcinin durumu veya sıfır (zero) bayrağı, taşıma (carry) bayrağı, işaret (sign) bayrağı, taşma (overflow) bayrağı, eşlik (parity) bayrağı, yardımcı taşıma (auxiliary carry) bayrağı ve kesme etkinleştirme (interrupt enable) bayrağı gibi çeşitli işlemlerin sonucunu tutmak için kullanılan özel bir kaydedici işlemciye eklenmiştir.

- ◇ **Genel amaçla kullanılan kaydediciler** (general purpose registers) işlemciye eklenmiştir.

Eklenen bu kaydediciler ile derlenen her bir program, dört bellek bölgesinden (segment) oluşur;

1. **Kod bellek**(code segment) ya da kaynak koda ithafla **metin bellek** (text segment) icra edilecek emirleri içeren bellektir.
2. Programın işleyeceği verileri tutan **veri belleği** (data segment).
3. Yığın bellek (stack segment), fonksiyon çağrıları sırasında kullanılan bellektir. Bu bellek, son giren veri ilk çıkar LIFO (last in first out) mantığıyla çalışır. Çünkü;
   - ◇ **Çağrıdan** önce mevcut işlemci durumu ve yerel değişkenler gibi bazı verileri depolamak ve
   - ◇ Fonksiyondan **dönülünce** işlemciyi ve çağrı ortamını eski hale getirmek için kullanılır.
4. Programın icrası sırasında dinamik olarak eklenip çıkarılan veriler için hurdalık gibi kullanılan **öbek bellek** (heap segment).

Depolama sınıfları, tanımlanan değişkenlerin hangi bellek bölgesinde (segment) yer alacağını belirler. Dört tür depolama sınıfı vardır. Aşağıda başlıklar halinde verilmiştir.

§7.5.1 auto Depolama Sınıfı

Otomatik depolama sınıfı (auto storage class), bir fonksiyon veya blok içinde kimliklendirilmiş tüm yerel değişkenler için varsayılan (default) depolama sınıfıdır.

> C++ dilinde bu depolama sınıfı için değişkenlerde `auto` sıfatı C++11'e kadar kullanılmıştır. C++11 ve sonrasında `auto` anahtar kelimesi ileride anlatılacak **tip çıkarımında** kullanılmaya başlanmıştır. Kısaca `auto` anahtar kelimesi **depolama sınıfı amacıyla kullanılmaz**!

Otomatik değişkenlere; **Yalnızca tanımlı olduğu blok içinden erişilebilir**. Yani değişkenin kapsamı, içinde bulunduğu blok ile sınırlıdır. Bu değişkenler, yığın bellek bölgesinde (stack segment) tutulur.

```
#include <iostream>
int main() {
        int a = 32;
        int b=20;
        /* a ve b değişkenleri bu fonksiyon bloğu ve iç bloklarda geçerlidir. Yani faaliyet
         ↪  alanları (scope) bu fonksiyon bloğu ile sınırlıdır.*/
        if (a==32) {
                int b=10; // Ana fonksiyon bloğunda tanımlı b değişkenine artık ulaşılamaz.
                int c=50; /* c değişkeni if bloğunda ve tanımlanabilecek iç bloklarda
                 ↪  geçerlidir.*/
                a=16;// Bu blokta a da geçerlidir.
                std::cout << "a:" << a << " b:" << b << " c:" << c << std::endl;
        }
        /*if bloğu dışında sadece ana fonksiyon bloğunda tanımlı değişkenlere ulaşılabilir. */
        std::cout << "a:" << a << " b:" << b;
}
/*Program Çıktısı:
a:16, b:10 c:50
a:16, b:20
*/
```

§7.5.2 static Depolama Sınıfı

Statik depolama sınıfı (static storage class) yaygın olarak kullanılan statik değişkenleri (static variable) saklamak için kullanılır. Bu değişkenler, `static` sıfatı (static specifier) ile tanımlanır. **Statik** olarak tanımlanan değişken, program başladığında veri bellek bölümünde oluşturulup program bitene kadar aynı bellek bölgesini işgal eden değişkendir;

- ◇ Statik değişkenlere **Yalnızca tanımlı olduğu blok içinden erişilebilir**. Yani değişkenin kapsamı, içinde bulunduğu blok ile sınırlıdır.
- ◇ Statik değişken kodun yazılı olduğu dosyanın başında tanımlı ise, bu değişkene dosya içerisinde her yerden erişilebilir.
- ◇ Statik değişkenler **kapsam (scope) dışına çıktıktan sonra bile değerlerini koruma özelliğine sahiptir**.
- ◇ Statik değişkenlere veri bellek bölgesinde (data segment) yer tahsis edilir. Derleme sırasında bu bellek hep sıfırla doldurulduğundan ilk değerler (initial value) hep sıfırdır.

```
#include <iostream>
void func() {
```

```
        for (int sayac = 1; sayac < 10; sayac++) {
                static int statikOlan = 5; //statik değişken tanımı. İlk değer vermeseydik değeri
                ↪  0 olurdu.
                int statikOlmayan = 10;
                statikOlan++;
                statikOlmayan++;
                std::cout << "sayaç: " << sayac << ", staticOlan: " << statikOlan << std::endl;
                std::cout << "sayaç: " << sayac << ", statikOlmayan: " << statikOlmayan <<
                ↪  std::endl;
        }
        //Burada statikOlan değişkene erişilemez.
}
int main() {
        func();
}
/* Program çalıştırıldığında:
Sayaç: 1, staticOlan: 6
sayaç: 1, statikOlmayan: 11
sayaç: 2, staticOlan: 7
sayaç: 2, statikOlmayan: 11
sayaç: 3, staticOlan: 8
sayaç: 3, statikOlmayan: 11
sayaç: 4, staticOlan: 9
sayaç: 4, statikOlmayan: 11
*/
```

§7.5.3 extern Depolama Sınıfı

Harici depolama sınıfı (extern storage class) bize basitçe değişkenin kullanıldığı blokta değil başka bir yerde tanımlandığını söyler;

- Harici tanımlanan değişkenlere değerler tanımlandığı yerde değil farklı bir yerde atanır ve üzerinde değişiklik yapılabilir.
- Bir değişkene `extern` sıfatı (extern specifier) ile tanımlandığında evrensel (global) değişken olur. Böyle tanımlanan değişkenlere programın her yerinden erişilebilir.
- Harici tanımlanan değişkenler de veri bellekte (data segment) yer alır. Derleme sırasında bu bellek hep sıfırla doldurulduğundan ilk değerler hep sıfırdır.

Bir harici değişkene `extern` sıfatı verildiğinde **harici değişken** (extern variable) adını alır ve bir başka dosyada tanımlanmış olduğu kabul edilir. Buna örnek olarak aşağıdaki iki farklı kaynak kod dosyası gereklidir. Birinci kaynak kod dosyası (`xtrn.cpp`) aşağıda verilmiştir.

```
//EXTERN.CPP
int externDegisken; //Diğer dosyalarda kullanılacak değişken tanımlandı.
void funcExtern() //Diğer dosyalarda kullanılacak fonksiyon tanımlandı.
{
        externDegisken++;
}
```

Bu dosyadaki değişken ve fonksiyonları kullanan bir başka kaynak kod `main.cpp` aşağıda verilmiştir;

```
//MAIN.CPP
#include <iostream>
extern void funcExtern(); // harici fonksiyon. Burada bildirim yapıldı. Başka dosyada tanımlandı.
int main() {
        extern int externDegisken; // harici değişken. Burada bildirim yapıldı. Tanımı başka
        ↪  dosyada! Aslında aynı değişkene evrensel olarak erişebiliyoruz!
        funcExtern();
        std::cout << "extern: " << externDegisken << std::endl;
        externDegisken = 2;
        std::cout << "extern: " << externDegisken << std::endl;
}
```

Bu iki dosyayı birlikte derlemek için `g++ xtrn.cpp main.cpp -o main.exe` komutu ile derleyiciye iki farklı dosya parametre olarak verilir.

§7.5.4 register Depolama Sınıfı

Kaydedici depolama sınıfı (register storage class), otomatik değişkenlerle aynı işlevselliğe sahiptir. Tek fark, derleyicinin, eğer boş bir işlemci kaydedicisi mevcutsa değişken olarak o kaydedicinin kullanılmasıdır. Genel amaçlı kaydediciler bu amaçla kullanılır. Bu da bu değişkenlerin, bellekte saklanan değişkenlerden çok daha hızlı olmasını sağlar. Bu değişkenler `register` sıfatı (register specifier) ile tanımlanırlar;

- Eğer boş bir CPU kaydedicisi mevcut değilse, değişken yalnızca bellekte saklanır.
- Sadece bloklar içinde tanımlanan yerel (local) değişkenler `register` ile tanımlanır. Evrensel (global) değişkenler bu anahtar sözcük ile kullanılamaz.

Aşağıda kaydedici bu depolama sınıfına ilişkin örnek verilmiştir. Buradaki sayma ve toplama işlemleri uygun olan işlemci kaydedicilerinde yapılır.

```
#include <iostream>
int main() {
        register int k, sum; //kaydedici değişkeni
        for(k = 1, sum = 0; k < 1000; sum += k, k++);  //döngü bloğu olmayan for talimatı örneği
                std::cout << sum << std::endl;
}
/*Program Çıktısı:
499500
*/
```

C++ standardı gereği, tek bir değişkenle birden fazla depolama sınıfının kullanılmasını yasaktır!
Aynı değişkene hem `static` hem `extern` sıfatı kullanılamaz.

Depolama sınıfları aşağıdaki tabloda özetlenmiştir; Tablodaki **çöp** (garbage) kavramı tahsis edilen bellek içeriği o anda ne ise değişkenin o değeri alması anlamındadır.

Depolama sıfatı	Bellek Bölgesi	Başlangıç Değeri	Değişken Kapsamı (scope)	Değişken Ömrü (lifetime)
`auto`	Yığın bellek (stack segment)	Çöp	Bulunduğu blok	Bulunduğu blok süresince
`static`	Veri bellek (data segment)	Sıfır	Bulunduğu Blok veya Bulunduğu Dosya	Program süresi
`extern`	Veri bellek (data segment)	Sıfır	Evrensel (global)	Program süresi
`register`	İşlemci Kaydedicileri	Çöp	Bulunduğu blok	Bulunduğu blok süresince

Tablo 7.4: Depolama Sınıfları

7.6 Evrensel ve Yerel Değişken

C++ bir değişkene her yerden erişilmek isteniyorsa, bu değişkenlere **evrensel değişken** (global variable) denir. Bu değişkenler herhangi bir kod bloğu içinde tanımlanmazlar. Genel olarak kaynak kodun başında tanımlanırlar. Sonra kodun her yerinde kullanılabilirler. Eğer kodumuz birden çok kaynak kod dosyasından oluşacak ise diğer tüm dosyalarda bu değişken `extern` sıfatı ile tanımlanır.

Blok içinde tanımlanmış tüm değişkenler, o blok içinden erişilebilen **yerel değişkenlerdir** (local variable). Yerel değişkenler otomatik depolama sınıfında (auto storage class) yer alır ve hepsi yığın belleği (stack segment) işgal eder.

```
1  #include <iostream>
2  int evrenselDegisken=10; /* evrenselDeğişken bu noktadan sonra her yerde kullanılabilir. */
3  void fonksiyon();
4  int main() {
5          int yerelDegisken=20; /* Bu yerel değişken sadece main bloğu içinde kullanılabilir.*/
6          std::cout << "evrensel:" << evrenselDegisken << ", yerel: "
7          << yerelDegisken << std::endl;
8          evrenselDegisken++; //evrensel değişken değiştiriliyor.
9          yerelDegisken--; //yerel değişken değiştiriliyor.
10         std::cout << "evrensel:" << evrenselDegisken << ", yerel:"
11         << yerelDegisken << std::endl;
```

```
12          fonksiyon();
13  }
14  void fonksiyon() {
15          int fonksiyonYerelDegiskeni=30;
16          evrenselDegisken++;
17          std::cout << "evrensel:"<< evrenselDegisken << ", fonsiyonYerel:"
18          << fonksiyonYerelDegiskeni << std::endl;
19          if (fonksiyonYerelDegiskeni==30) {
20                  int evrenselDegisken=100; //Artık evrensel değişkene bu blokta ulaşılamaz.
21                  std::cout << "evrensel:"<< evrenselDegisken << ", fonsiyonYerel:"
22                  << fonksiyonYerelDegiskeni << std::endl;
23          }
24  }
25  /* Program Çıktısı:
26  evrensel:10, yerel:20
27  evrensel:11, yerel:19
28  evrensel:12, fonsiyonYerel:30
29  evrensel:100, fonsiyonYerel:30
30  */
```

Burada fonksiyon içinde `if` bloğunda 18. satırda yer alan değişken her ne kadar `evrenselDegişken` olarak kimliklendirilse de yerel değişkendir ve sadece bu `if` bloğu içinde geçerlidir. Bu yerel değişken 2. satırdaki değişkeni gölgelemiştir. Yani tanımlamada **mantıksal hata** (logical error) yapılmıştır.

7.7 Değerle Çağırma

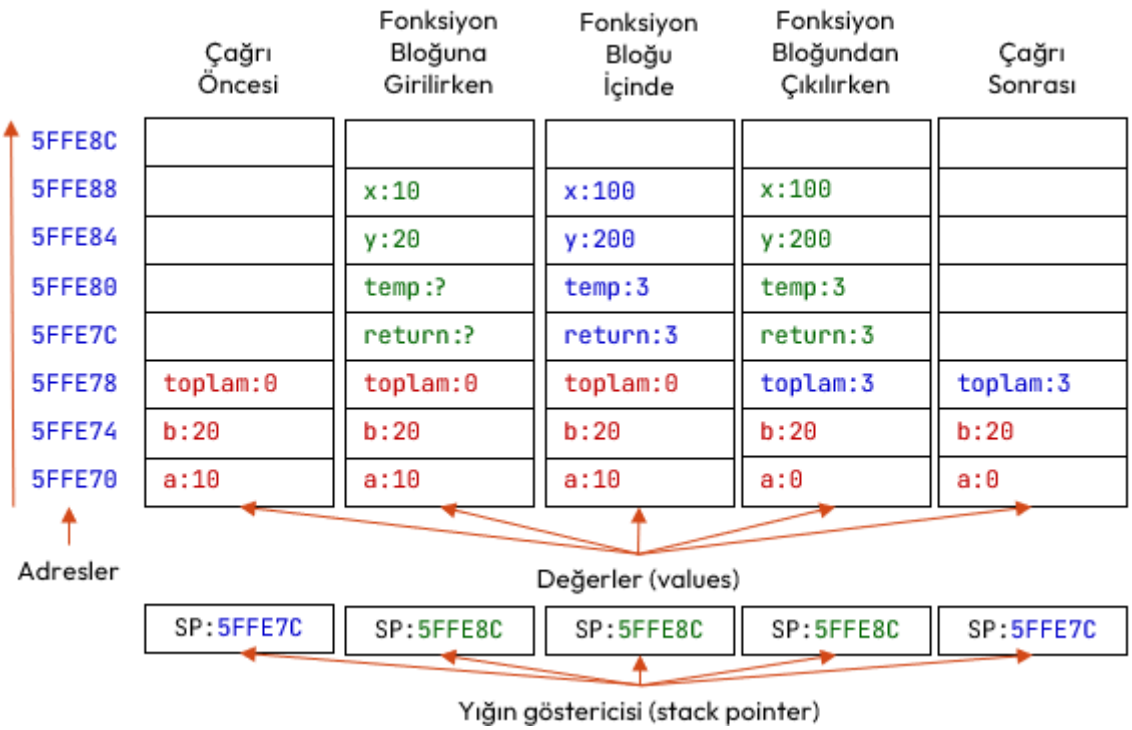
Yerel değişkenlerin yığın bellekte tutulduğu bir üst başlıkta anlatılmıştı. Fonksiyon blokları içindeki değişkenler de yerel değişkenlerdir. Bir fonksiyon çağrıldığında değişkenlerin bellekteki durumu ve değişimi aşağıdaki programdan anlaşılabilir;

```
#include <iostream>
int topla(int, int); /* İki tam sayıyı toplayan fonksiyon bildirimi/prototipi */
int main () {
        /* main() fonksiyonu içinde geçerli yerel (local) değişkenler*/
        int a = 10;
        int b = 20;
        int toplam = 0;
        std::cout << "çağrı öncesi a:" << a << ", b:" << b << ", toplam:" << toplam << std::endl;
        toplam = topla(a, b);
        std::cout << "çağrı sonrası a:"<< a << ", b:" <<b << ", toplam:" << toplam << std::endl;
}
int topla(int x, int y) { /* iki tam sayıyı toplayan fonksiyon tanımı*/
        /* Topla fonksiyonu yerel değişkenler */
        int temp= x+y;
        x=100; y=200;
        /*
        Burada değiştirilenler parametre değişkenleridir.
        main() içindeki yerel değişkenler değildir.
        */
        std::cout << "çağrı içinde x:" << x << ", y:" << y  << std::endl;
        return temp;
}
/* Program çalıştırıldığında:
çağrı öncesi a:10, b:20 toplam:0
çağrı içinde x:100, y:200
çağrı sonrası a:10, b:20 toplam:30
*/
```

Yukarıda örneği verilen programda `topla` fonksiyonunun çağırma sürecindeki yığın bellek bölgesindeki (stack segment) değişimi Şekil 7.3'de gösterilmektedir.

Yukarıdaki kod şu şekilde açıklanabilir;

- İcra sırası `topla(a,b)` fonksiyonuna geldiğinde fonksiyon çağrılmadan önce `main()` fonksiyonu yerel değişkenleri olan `a`, `b` ve `toplam` değişkenleri yığın belleğe (stack segment) itilmiştir (push).



Şekil 7.3: Fonksiyon Çağırma Sürecinde Yığın Bellek

- `topla(a,b)` fonksiyonu çağrıldığı anda `a` ve `b` **değişkenlerinin değerleri**, `x` ve `y` **parametrelerine kopyalanarak fonksiyona geçirilir**. Değerler parametrelere kopyalandığından buna **değerle çağırma** (call by value) adı verilir.
- `topla(a,b)` fonksiyonu icra edilirken fonksiyon yerelindeki `temp` ve parametre değişkenleri `x`, ve `y` istenildiği gibi değiştirilebilir. Geri dönüş değeri de belirlenir.
- `topla(a,b)` fonksiyonu içine bir başka fonksiyon da çağrılabilirdi. Böyle bir durumda `x`, `y` ve `temp` değerleri yığın belleğe itilir ve çağrılan fonksiyon icra edilirdi. Böylece iç içe çağrılan her fonksiyonda yığın bellek büyüyeceği görülmektedir. Program derlendiğinde yığın bellek (stack segment) için sınırlı bir bellek bölgesi ayrılır. İç içe çok fonksiyon çağrıldığında bu bellek sınırı aşılabilir. Bu durumda program **yığın taşması** (stack overflow) hatası verecektir.
- `toplam=topla(a,b)` fonksiyon bloğundan çıkılırken geri dönüş değeri çağrı ortamındaki `toplam` değişkenine kopyalanır ve fonksiyon için yığın belleğe itilen değerler geri çekilir (pop).

7.8 Özyinelemeli Fonksiyonlar

Özyineleme (recursion), bir fonksiyonun kendi çağrısını yapma tekniğidir. Bu teknik, karmaşık sorunları çözülmesi daha kolay basit sorunlara ayırmanın bir yolunu sağlar. Özyineleme, yineleme (iteration), döngüler (loop) veya tekrar yerine geçebilecek çok güçlü bir tekniktir. Özyinelemeli algoritmalarda, tekrarlar fonksiyonun kendi kendisini kopyalayarak çağırması ile elde edilir. Bu kopyalar işlerini bitirdikçe kaybolur.

Bir problemi özyineleme ile çözmek için problem iki ana parçaya ayrılır;

1. Cevabı kesin olarak bildiğimiz temel durum (base case)
2. Cevabı bilinmeyen ancak cevabı yine problemin kendisi kullanılarak bulunabilecek durum.

Faktöriyel örneğini inceleyelim; Matematikte, negatif olmayan bir tam sayı olan n sayısının faktöriyeli, $n!$ ile gösterilir ve n sayısına eşit ve küçük olan tüm pozitif tam sayıların çarpımıdır.

$$n! = n \times (n-1) \times (n-2) \times \cdot \times 3 \times 2 \times 1$$

Sıfır sayısı da pozitif bir sayıdır ama sonucun sıfır çıkmaması için **sıfır sayısının faktöriyeli 1 kabul edilir**. Bu aslında faktöriyel problemi için **temel durumdur** (base case) ve **boş ürün** (emty product) kuralı olarak bilinir. Aşağıda faktöriyel için girilen sayıdan sonra tüm sayıların çarpımıyla hesaplayan bir fonksiyon yazılmıştır.

```
#include <iostream>
unsigned int faktoriyel(unsigned int n) { //Fonksiyon tanımı yapıldı. Bildirim yapılmadı.
        int sonuc=1,indis;
```

```
        for (indis=n;indis>0;indis--)
                sonuc=sonuc*indis;
        return sonuc;
}
int main() {
        unsigned int sayi,sonuc; //Pozitif tam sayı tanımı yapılmış değişkenler.
        std::cout << "Pozitif Bir Sayi Giriniz:";
        std::cin >> sayi;
        sonuc=faktoriyel(sayi);
        std::cout << sayi << "!=" << sonuc;
}
```

Aslında negatif olmayan bir tam sayı olan sayının faktöriyeli, kendisi ile kendisinden bir küçük olan sayının faktörüyeli ile çarpımı şeklinde yazılabilir. Yani problemin tanımı yine kendisini içerir:

$$n! = n \times (n-1)!$$

Bu durumda faktöriyel fonksiyonu aşağıdaki şekilde **özyinelemeli** (recursive) tanımlanabilir;

$$n! = \begin{cases} 1 & \text{eğer } n = 0 \\ n \cdot (n-1)! & \text{eğer } n > 0 \end{cases}$$

Aynı faktöriyel fonksiyonu **parametreli olarak** aşağıdaki şekilde yine özyinelemeli olarak tanımlanabilir;

$$f(n) = \begin{cases} 1 & \text{eğer } n = 0 \\ n \cdot f(n-1) & \text{eğer } n > 0 \end{cases}$$

Buna örnek olarak aşağıdaki hesabı verebiliriz;

$$5! = 5 \times 4! = 5 \times 4 \times 3! = 5 \times 4 \times 3 \times 2! = 5 \times 4 \times 3 \times 2 \times 1! == 5 \times 4 \times 3 \times 2 \times 1 \times 0! = 120$$

Özyinelemeli (recursive) olarak ise faktöriyel fonksiyonu aşağıdaki şekilde kodlanabilir;

```
#include <iostream>
unsigned int faktoriyel(unsigned int n)
{
        if (n==0)
                return 1;
        else
                return n*faktoriyel(n-1);
}
int main() {
        unsigned int sayi,sonuc; //Pozitif tam sayı tanımı yapılmış değişkenler.
        std::cout << "Pozitif Bir Sayi Giriniz:";
        std::cin >> sayi;
        sonuc=faktoriyel(sayi);
        std::cout << sayi << "!=" << sonuc;
}
```

Bir başka örnek olarak, girilen taban ve üs ile kuvvet hesaplama fonksiyonu verilebilir. Bu fonksiyonun tanımı aşağıdaki şekilde yazılabilir;

$$f(taban, us) = \begin{cases} 0 & \text{eğer } taban = 0 \\ 1 & \text{eğer } us = 0 \\ taban \cdot f(taban, us-1) & \text{eğer } taban > 0 \text{ ve } us > 0 \end{cases}$$

Buna ilişkin kod aşağıda verilmiştir;

```
#include <iostream>
int Kuvvet(int taban, int us) {
        if (taban == 0)
                return 0; //Taban 0 ise 0 dön
        if (us == 0)
                return 1;  // Üs 0 ise 1 dön
        return taban* Kuvvet(taban, us - 1);
}
int main(){
        int taban,us;
        std::cout << "Tabanı Giriniz:";
```

```
        std::cin >> taban;
        std::cout << "Üssü Giriniz:";
        std::cin >> us;
        std::cout << taban <<" uzeri "<< us <<" = " << Kuvvet(taban,us);
}
/*Program Çıktısı:
Tabanı Giriniz:3
Üssü Giriniz:7
3 uzeri 7 = 2187
*/
```

7.9 Sabit İfadelerin Fonksiyonlarda Kullanımı

Sabit ifadeler (const expression), 4.6 başlığında işlenmişti. C++11 ile gelmiş ve modern C++'ın en güçlü özelliklerinden biri olan bu ifadeler `constexpr` anahtar kelimesini kullanır. Temel mantığı şudur: "Eğer bir hesaplamayı program çalışırken değil de derlenirken yapabiliyorsak, yapalım; böylece program daha hızlı çalışır. Aşağıda fonksiyonlardaki kullanımına örnek verilmiştir.

```
constexpr int kareAl(int n) {
        return n * n;
}
int main() {
        // Derleyici buraya doğrudan 25 yazar, fonksiyonu çağırmakla vakit kaybetmez.
        int dizi[kareAl(5)];
}
```

7.10 Satır İçi Fonksiyonlar

Fonksiyon Çağırma Süreci başlığında anlatıldığı üzere fonksiyonlar çağrılırken sürekli yığın belleğe (stack segment) it-çek (push-pop) yapılır. Bazı durumlarda bu işlem performans gereği istenmez. Bu durumda fonksiyonlar, **satır içi fonksiyon** (inline function) olarak tanımlanır. 2017 C sürümü ile gelen bu özellik, fonksiyona `inline` sıfatı (inline specifier) eklenerek kullanılır.

```
#include <iostream>
inline int topla (int op1, int op2) {
        int toplam= op1+op2;
        return toplam;
}
int main() {
        int x=10, y=15, sonuc;
        sonuc = topla(x,y);
        std::cout << x << "+" << y << "=" << sonuc;
}
```

Bu kod aşağıdaki şekle çevrilmiş gibi derlenir;

```
#include <iostream>
int main() {
        int x=10, y=15, sonuc;
        {
                int toplam= op1+op2;
                sonuc = toplam;
        }
        std::cout << x << "+" << y << "=" << sonuc;
}
```

Bir fonksiyon **aşağıda belirtilen durumlarda satır içi fonksiyon olarak derlenmez;**

1. Bir döngü içeriyorsa!
2. Statik değişkenler içeriyorsa!
3. Özyinelemeli ise!
4. Dönüş türü `void` haricinde olup `return` ifadesi işlev gövdesinde mevcut değilse!
5. `switch` veya `goto` talimatı içeriyorsa!

Bu tip fonksiyon tanımlamadaki amacımız, **fonksiyon çağırma yükünün azaltılması için işlemci kaydedi-**

cilerinin daha çok kullanılmasıdır. Bu da icra hızını yükseltir.

7.11 Varsayılan Argümanlar

Varsayılan argüman (default argument), bir fonksiyon bildiriminde bir parametre için sağlanan ve çağıran fonksiyon bu parametreler için bir değer sağlamazsa derleyici tarafından otomatik olarak atanan bir değerdir. Bu parametreye çağrı sırasında bir değer geçirilirse, varsayılan değer dikkate alınmaz.

```
#include <iostream>
void yaz(int a = 10) { //a parametresi için varsayılan argüman 10 dur.
        std::cout << a << std::endl;
}
int main() {
        yaz(); // 10
        yaz(200); //200
}
```

Varsayılan argüman fonksiyon bildiriminde (function declaration) farklı, fonksiyon tanımında (function definition) farklı verilemez!

```
#include <iostream>
void yaz(int a = 10); //a parametresi için varsayılan argüman 10 dur.
int main() {
        yaz(); // 10
        yaz(200); //200
}
void yaz(int a = 20); // Derleme hatası olur. 10 olarak verilmelidir.
{
        std::cout << a << std::endl;
}
```

Birden fazla parametreye varsayılan argüman atanacak ise sağdan sola doğru hepsine atanmalıdır;

```
void yaz(int x, int y = 20); // Geçerli Bildirim.
void yaz(int x = 10, int y); /* Derleme hatası olur. GEÇERSİZ Bildirim: sağdaki y parametresine
↪ varsayılan argüman atanmadı */
```

7.12 Aşırı Yükleme

C++ dilinde **aşırı yükleme** (overloading); fonksiyonların farklı parametrelerle yeniden tanımlanmasıdır. **Aşırı yüklemede fonksiyonun kimliği ve geri dönüş tipi değişmez!** Farklı argümanlar ile fonksiyon gövdesi yeniden tanımlanır. Aşağıda bir fonksiyonun aşırı yüklemesine bir örnek verilmiştir.

```
#include <iostream>
void yaz(int a, int b) {  //İlk tanımlanan yaz fonksiyonu
        std::cout << "toplam = " << (a + b);
}
void yaz(double a, double b) {  // Aşırı yüklenmiş yaz fonksiyonu
        std::cout << std::endl << "toplam = " << (a + b);
}
int main() {
        yaz(1, 2);
        yaz(2.5, 5.5);
}
```

Aşırı yüklemede argümanların farklı tanımlandığının derleyiciye bildirilmesi gerekir. Aksi durumda hata alınır;

```
int topla(int);
int topla(int x);  //Hata!
int topla(int y);  //Hata!
int main(){
        topla(1);
}
int topla(int i) {
        return i+10;
}
int topla(int x) { //Hata: redefinition of 'int topla(int)'
```

```
        return x+20;
}
int topla(int y) { //Hata redefinition of 'int topla(int)'
        return y+20;
}
```

Fonksiyonlarda ön tanımlı argümanlar kullanıldığında aşırı yükleme tanımlanmasına çok dikkat edilmelidir;

```
int topla(int a = 10, int b = 20) {
        return a+b;
}
int topla(int a = 10, int b = 0) { // HATA: fonksiyon imzası (signature) aynı!
        // redefinition of 'int topla(int, int)'
        return a+b;
}
int topla(int a = 10) { // HATA: üstteki fonksiyon da çağrıyı karşılar
        // call of overloaded 'topla(int)' is ambiguous
        return a+1;
}
int main()
{
        topla(1,2)
        topla(1);
}
```

7.13 Düşün ve Çöz

- **Düşün**: `static` bir yerel değişken ile `auto` bir yerel değişkenin bellek ömrü ve kapsamı arasındaki farkları karşılaştırınız.
- **Düşün**: Fonksiyonlara parametre aktarırken değerle çağırma (call by value) yönteminin yığın bellek kullanımına etkisini açıklayınız.
- **Düşün**: Özyinelemeli fonksiyonlarda temel durum (base case) unutulursa bellek seviyesinde ne gerçekleşir? Yığın taşması (stack overflow) nedir?
- **Çöz**: Bir tam sayının basamak değerleri toplamını özyinelemeli olarak hesaplayan fonksiyonu yazınız.
- **Çöz**: `extern` anahtar kelimesini kullanarak, iki farklı kaynak dosyasında (`.cpp`) ortak kullanılan bir evrensel değişken örneği kurgulayınız.
- **Düşün**: Değerle geçirme (pass by value) ile referansla geçirme (pass by reference) arasındaki farkı bellek diyagramı çizerek açıklayın. Büyük bir nesneyi fonksiyona geçirirken hangisi tercih edilmeli ve neden?
- **Düşün**: Özyinelemeli (recursive) bir fonksiyon ile aynı işi yapan döngüsel (iterative) çözümü karşılaştırın. Fibonacci sayı dizisini her iki yöntemle hesaplayın ve yığın (stack) kullanımı açısından değerlendirin.
- **Düşün**: Satır içi fonksiyon (inline function) kullanmanın avantajları nelerdir? Derleyici her zaman `inline` isteğini yerine getirir mi? Hangi durumlarda `inline` kullanımı anlamsız olur?
- **Düşün**: Depolama sınıfları (`auto`, `static`, `extern`, `register`) arasındaki farkları bir tablo halinde özetleyin. `static` bir yerel değişken ile global değişken arasındaki fark nedir?
- **Düşün**: Kaynak dosyada aynı fonksiyon adını birden fazla tanımlamak ne tür bir hataya yol açar? Bu hatayı `extern` ve başlık dosyaları kullanarak nasıl çözersiniz?

8. Bölüm
Diziler

8.1 Dizi Nedir

Şu ana kadar en basit **veri yapıları** (data structure) olan değişkenlerle karşılaştık. Programlamada kullanılan bir başka veri yapısı da **dizilerdir** (array). Matematikte diziler; bir sıralı elemanlardan oluşan bir listedir. Sıralı elemanların sayısına **dizinin uzunluğu** (array size) denir. Doğal olarak eleman sayısı tam sayıdır (integer). Elemanların sırasını tam sayı (integer) olan **indis** (index) belirtir. Kümelerin (set) aksine, aynı elemanlar dizide farklı konumlarda bulunabilir. Elemanlara terim adı da verilir. Örnekler;

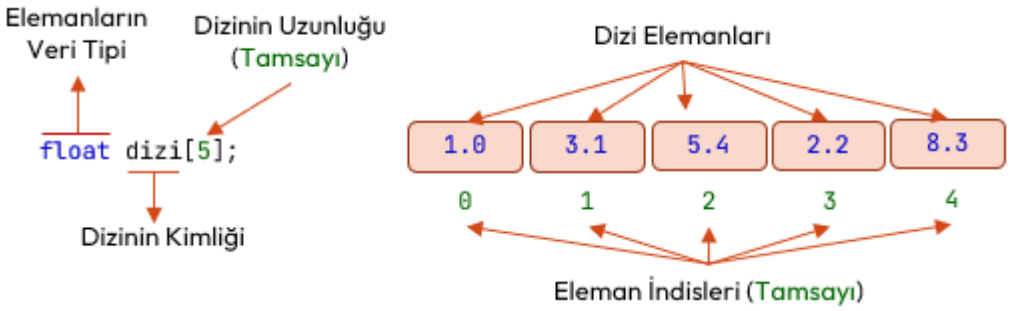
- Örnek olarak elemanları 1'den 10'a kadar tam sayıların karesinden oluşan bir dizi, {1,4,9,...,100} olarak gösterilir.
- Bir başka örnek olarak {K,İ,T,A,P} dizisi verilebilir. Bu dizi ile {P,A,T,İ,K} dizisi birbirinden farklıdır. Aynı elemanlardan oluşmasına rağmen elemanların sıraları yani indisleri farklıdır.
- {1,3,5,1,2,1,7} dizisinde birden farklı sırada yani indiste aynı 1 elemanı vardır. Bu dizinin geçersiz olduğu anlamına gelmez.

Matematikte sonsuz elemanı olan sonsuz diziler tanımlanabilir. Ama programlamada hep sonlu elemanlı diziler kullanılır. Dizinin eleman sayısı, dizinin **uzunluğudur** (size) ve dizinin uzunluğu dizi tanımlanırken belirlenir!

8.2 Tek boyutlu Diziler

C++ dilinde sonlu elemanlı diziler Şekil 8.1'deki gibi tanımlanır;

- Diziler, **aynı veri tipindeki öğeleri barındırır** ve bu öğeler **bitişik bellek bölgesini paylaşırlar**.
- Dizi elemanları, ilkel veri tipleri (`char`,`short`, `int`, `long`, `float`, `double`) veya daha sonra göreceğimiz kullanıcı tanımlı tip olan yapı `struct` veya gösterici (pointer) olabilir.
- Aynı değer birkaç farklı yerde dizi içinde bulunabilir!
- Dizinin veri tipi, **dizideki elemanlarının veri tipidir**. Bir başka deyişle **elemanların veri tipi dizinin veri tipiyle aynı olmalıdır**.
- Dizinin uzunluğu (size), **tam sayıdır.** Dizi **kimliklendirmesi sırasında belirlenir**! Derleyici buna göre bellekte yer ayırır. Uzunluk, sonradan değiştirilemez!
- Dizi uzunluğu, **sabit olmayan bir değişkenle** belirlenemez!
- Elemanların indisi (index) sıra belirttiğinden **her zaman tam sayıdır**.
- İlk elemanın indisi `0`, son elemanın indisi dizi uzunluğundan (size) bir küçük olan tam sayıdır. Bunların dışında bir indis girildiğinde **sınır dışı** (out of bounds) hatası yaşanır.



Şekil 8.1: Tek Boyutlu Dizi Tanımlama

Örnek olarak beş tam sayı elemanı olan bir dizi aşağıdaki şekilde tanımlanır;

```
int tamsayiDizisi[5];
```

Benzer şekilde beş gerçek sayı elemanı olan bir dizi ise aşağıdaki şekilde tanımlanır;

```
float reelsayiDizisi[5];
```

Dizinin elemanlarına tanımlama sırasında **başlangıç değeri** (initial value) süslü parantezler arasında verilebilir yani dizi bazı değerlerle **başlatılabilir** (initialize);

```
int tamsayiDizisi1[5]= {1, 2, 3, 4, 5}; //Elemanları 1,2,3,4,5 olacak şekilde başlatıldı.
int tamsayiDizisi2[5]= {0}; //Elemanların hepsi 0 olacak şekilde başlatılıyor.
```

Dizinin elemanlarına hepsine başlangıç değeri verilmeyebilir;

```
float a[5] = {1.0, 2.0, [4] = 12.0}; // 1.Eleman 1.0, 2.Elaman 2.0 ve 5.eleman 12.0
```

Dizinin elemanlarına ayrı ayrı erişilip değerleri değiştirilebilir ya da yeni değer ataması yapılabilir;

```
int a[5]={0}; // a dizisi, tüm elemanları 0 olacak şekilde başlatılıyor
a[0]=1; //Birinci elemana 1 değeri atandı
a[1]++; //İkinci elemanın değeri artırıldı
a[2]--; //Üçüncü elemanın değeri azaltıldı
a[4]=12; //Beşinci elemana 12 değeri atandı.
```

Diziler belleği verimli kullanan ve tek bir değişken ile elemanlara erişim sağlayan bir çözüm sunar. Bir dizideki elemanlar, bellekte bitişik konumda yer aldığından herhangi bir öğeye kolaylıkla erişebiliriz. Dizilerin başlıca üstünlükleri şunlardır:

- İndisleri kullanarak dizi öğelere rastgele ve hızlı erişim sağlanır. Her öğenin bir indisi olduğundan doğrudan erişilebilir ve değiştirilebilir.
- Birden fazla öğeden oluşan tek bir dizi oluşturduğundan daha az kod satırı yazılır.
- Daha az kod satırı yazılarak sıralama yapılabilir.

Dizi kullanmanın zayıf yönleri ise;

- Kimliklendirme sırasında dizi uzunluğuna bir tam sayı ile karar verildiğinden belli sayıda elemanın üzerinde işlem yapılır.
- Dizi dinamik değildir araya elaman ekleme veya çıkarma yapılamaz.

Örnek olarak klavyeden girilen 5 adet tam sayıyı, giriş sırasının tersinden ekrana yazan C++ programını verebiliriz;

```
#include <iostream>
int main() {
        int dizi[5]; /* 5 elemanı tam sayı olan bir dizi tanımlandı*/
        int indis; /* dizi elemanlarını gezecek ve indis olarak kullanılacak tam sayı bir
        ↪ değişken tanımlandı */
        for (indis =0; indis <5; indis ++){
                std::cout << indis << ". sayiyi girin:";
                std::cin >> dizi[ indis ]; /* indis ile belirtilen elemana klavyeden okunan değer
                ↪ atanıyor */
        }
        std::cout << "Tersten Sayilar:" << std::endl;
        for (indis=4; indis >=0; indis --)
                std::cout << "dizi[" << indis << "]=" << dizi[ indis ] << std::endl;
}
/* Program Çıktısı:
0. sayiyi girin:1
1. sayiyi girin:3
2. sayiyi girin:6
3. sayiyi girin:8
4. sayiyi girin:3
Tersten Sayilar:
dizi[4]=3
dizi[3]=8
dizi[2]=6
dizi[1]=3
dizi[0]=1
*/
```

Bir başka örnek; Klavyeden girilen 10 adet sınav notuna göre, ortalamanın üstünde olan notları ekrana yazan C++ programı;

```
#include <iostream>
#define BOYUT 10
int main() {
        unsigned notlar[BOYUT]; /* Her terimi pozitif olan 10 elemanlı dizi */
        float ortalama, toplam=0;
```

```
        int indis; //indis için tam sayı değişken
        for (indis=0; indis<BOYUT; indis++){
                std::cout << indis << ". notu girin:";
                std::cin >> notlar[indis];
                toplam+=notlar[indis]; // Her eleman girildiğinde toplanıyor
        }
        ortalama=toplam/BOYUT;
        std::cout << "Ortalamanın (" << ortalama << ") Üzerindeki Notlar:"<< std::endl;
        for (indis=0; indis<BOYUT; indis++)
                if (notlar[indis]>ortalama)
                        std::cout << "notlar [" << indis << "]=" << notlar[indis] << std::endl;
}
/* Program Çıktısı:
1. notu girin:85
2. notu girin:45
3. notu girin:60
4. notu girin:75
5. notu girin:40
6. notu girin:25
7. notu girin:35
8. notu girin:60
9. notu girin:50
Ortalamanın (57.5) Üzerindeki Notlar:
notlar [0]=100
notlar [1]=85
notlar [3]=60
notlar [4]=75
notlar [8]=60
*/
```

Bir başka örnek olarak 10 adet satış miktarlarının, ortalamaya olan uzaklıkları yani sapma (deviation) ve karesi alınmış sapmalar yani varyans (variance) ile standart sapmayı yazdıran aşağıdaki C++ programı verilebilir. Ortalamaya olan uzaklıklar yani sapma, verilerin ortalamaya ne kadar yakın olduğunu gösteren dağılımdır. Sapmaların toplamı sıfır olabileceğinden karelerinin toplamı yani varyans hesaplanır. Varyansın eleman sayısına bağlı karekökü de standart sapmayı verir. Standart sapmanın büyük olması verilerin ortalamadan daha uzak yayıldıklarını; küçük bir standart sapma ise verilerin ortalama etrafında daha yakın gruplaştıklarını gösterir.

```
#include <iostream>
#include <iomanip>  // setw() ve setprecision için
#include <cmath>
#define BOYUT 10
int main() {
        float satislar[BOYUT]={100.0,850.0,500.0,600.0,750.0,650.0,450.0,800.0,900.0,110.0};
        float ortalama, toplam=0, varyansToplam=0, standartSapma;
        int indis;
        for (indis=0; indis<BOYUT; indis++)
        toplam+= satislar[indis];
        ortalama=toplam/BOYUT;
        std::cout << "Ortalama:" << ortalama << std::endl;
        for (indis=0; indis<BOYUT; indis++) {
                float sapma= satislar[indis]-ortalama;
                varyansToplam+=sapma*sapma;
                std::cout << "Sapma: " << std::setw(5) << sapma << " Varyans: " <<  std::setw(8)
                 ↪  << sapma*sapma << std::endl; /* setw(5): izleyen değişken konsola yazılırken
                 ↪  5 karakter genişliğinde yazılsın. setw(8): izleyen değişken konsola
                 ↪  yazılırken 8 karakter genişliğinde yazılsın */
        }
        standartSapma=sqrt(varyansToplam/BOYUT);
        std::cout << "Standart Sapma:"<< std::setprecision(8) << standartSapma; /*
         ↪  setprecision(8): izleyen reel sayı değişkeni konsola yazılırken 9 karakter
         ↪  genişliğinde yazılsın. */
}
```

```
/* Program Çıktısı:
Ortalama:571
Sapma:  -471 Varyans:   221841
Sapma:   279 Varyans:    77841
Sapma:   -71 Varyans:     5041
Sapma:    29 Varyans:      841
Sapma:   179 Varyans:    32041
Sapma:    79 Varyans:     6241
Sapma:  -121 Varyans:    14641
Sapma:   229 Varyans:    52441
Sapma:   329 Varyans:   108241
Sapma:  -461 Varyans:   212521
Standart Sapma:270.49768
*/
```

Bir başka örnek olarak 5 elemanlı diziye girilen elemanlardan **tekil** (unique) olanlarını bulan program verilebilir.

```
#include <iostream>
#define BOYUT 5
int main() {
        int dizi[BOYUT];
        int indis;
        for (indis=0; indis<BOYUT; indis++) {
                std::cout << indis << ". Sayıyı Girin:";
                std::cin >> dizi[indis];
        }
        std::cout << "Dizi içinde tekil (unique) olan rakamlar:"<< std::endl;
        for (indis=0; indis<BOYUT; indis++) {
                int indis2, sayac = 0;  // Her elemanda sayacı sıfırla
                for (indis2 = 0; indis2 < BOYUT; indis2++)
                if (indis != indis2) //elemanın kendisini kontrol etmiyoruz
                if (dizi[indis] == dizi[indis2])
                sayac++;
                if (sayac == 0)
                std::cout << "dizi["<< indis << "]:" << dizi[indis] << " tekil olarak dizide
                 ↪  bulundu."<< std::endl;
        }
}
/* Program Çıktısı:
0. Sayıyı Girin:23
1. Sayıyı Girin:12
2. Sayıyı Girin:23
3. Sayıyı Girin:14
4. Sayıyı Girin:14
Dizi içinde tekil (unique) olan rakamlar:
dizi[1]:12 tekil olarak dizide bulundu.
*/
```

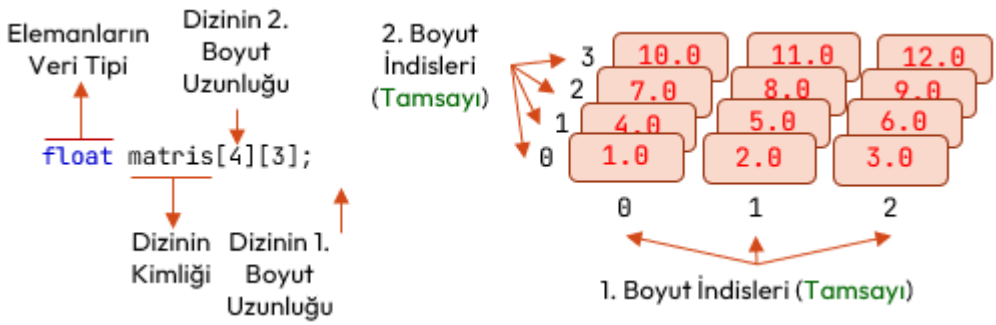
Bir başka örnek olarak 5 elemanlı diziye girilen en büyük elemanına en küçük elemanını ekleyen program verilebilir;

```
#include <iostream>
#include <iomanip> // setw()
#define BOYUT 5
int main() {
        int dizi[BOYUT]={10,12,3,4,6};
        int enBuyuk, enKucuk, enKucukIndex, enBuyukIndex, i ;
        std::cout << "ÖNCE:" << std::endl; //Dizinin İlk hali konsola yazdırılıyor.
        for (i = 0; i < BOYUT; i++)
        std::cout << std::setw(4) << dizi[i];
        std::cout << std::endl;
        enKucuk=dizi[0]; enBuyuk=dizi[0]; /* İlk elemanlar hem en küçük
        hem de en büyük olsun. */
```

```
        enKucukIndex=0,enBuyukIndex=0; /* En küçük ve en büyük elemanların
        indisi 0 olsun. */
        for (i=0; i<BOYUT; i++){ /* En küçük ve en büyük eleman ile
                indislerini bulan kısım. */
                if (dizi[i]<enKucuk) {
                        enKucuk=dizi[i];
                        enKucukIndex=i;
                }
                if (dizi[i]>enBuyuk) {
                        enBuyuk=dizi[i];
                        enBuyukIndex=i;
                }
        }
        std::cout << "En Büyük Eleman: dizi[" << enBuyukIndex << "]:"
        << dizi[enBuyukIndex] << std::endl;
        std::cout << "En Küçük Eleman: dizi["<< enKucukIndex << "]: "
        << dizi[enKucukIndex] << std::endl;
        dizi[enBuyukIndex]+=dizi[enKucukIndex]; //En büyük elemana en küçüğünü ekleme
        std::cout << "SONRA:" << std::endl; //Dizinin son hali konsola yazdırılıyor.
        for (i = 0; i < BOYUT; i++)
        std::cout << std::setw(4) << dizi[i];
}
/* Program Çıktısı:
ÖNCE:
10  12   3   4   6
En Büyük Eleman: dizi[1]:12
En Küçük Eleman: dizi[2]: 3
SONRA:
10  15   3   4   6
*/
```

8.3 İki Boyutlu Diziler

Şu ana kadar gördüğümüz tek boyutlu dizilerdi. **İki boyutlu diziler** (two-dimentional array) matematikten de bildiğimiz üzere matris (matrix) olarak adlandırılır. İki boyutlu dediğimizde en-boy ve satır-sütun gibi kavramlar akla gelmelidir. Matris işlemleri gibi bazı problemlerde; bir dizinin her bir elemanının da dizi olması istenir. Bu tür iki boyutlu dizilerde en içteki dizinin boyutu kimliklendirmede sağda yer alır.



Şekil 8.2: İki Boyutlu Dizi Tanımlama

Örneği verilen iki boyutlu dizinin ikinci boyutundan bakacak olursak her biri ayrı bir tek boyutlu dizidir. Bellekte ikici boyutu oluşturan bu diziler de sırasıyla birbirine bitişik (contigious) olarak yer alır. Aslında `float dizi[3][4];` ile `float dizi[12];` aynı bellek yerleşimine sahiptir. Aşağıda iki boyutlu matris tanımlamalarına örnek verilmiştir;

```
float matris[2][3];  //Her bir elemanı 3 elemanlı bir dizi olan 2 satır üç sütunlu bir matris
//Aşağıda 2x3 matrisi başlatma:
float matris2[2][3]= {
        {1.0,2.0,3.0}, //Birinci Satır: 3 Elemanlı bir dizi ile başlatılıyor
        {2.0,4.0,6.0}  //İkinci Satır: 3 Elemanlı bir başka dizi ile başlatılıyor.
};
int karematris[2][2];
//2x2 matrise ilk değer verme:
```

```
int karematris2[2][2]= {
        {1,2}, //Birinci Satır: 2 elemanlı bir dizi
        {5,6} //Birinci Satır: 2 elemanlı bir dizi
};
int karematris3[3][3];
```

Örnek olarak üç satır ve dört sütundan oluşmuş 3x4'lük matris elemanlarını klavyeden girip, tablo halinde ekrana yazdıran programı verebiliriz;

```
#include <iostream>
#include <iomanip> // setw()
#define SATIR 3
#define SUTUN 4
int main() {
        int matris[SATIR][SUTUN];
        int i,j;
        /*
        for (j=0; j<SUTUN; j++)  //Birinci Satır Okuyalım
        std::cin >> matris[0][j];
        for (j=0; j<SUTUN; j++)  //İkinci Satır Okuyalım
        std::cin >> matris[1][j];
        for (j=0; j<SUTUN; j++)  //Üçüncü Satır Okuyalım
        std::cin >> matris[2][j];
        //Bunun yerine iç içe iki for tanımlanır;
        */
        for (i=0; i<SATIR; i++) //İkinci boyut için i indisi
        for (j=0; j<SUTUN; j++)  //Birinci boyut için j indisi
        std::cin >> matris[i][j];
        std::cout <<"TABLO:"<< std::endl;
        for (i=0; i<SATIR; i++) {  //İkinci Boyut İçin
                for (j=0; j<SUTUN; j++)  //Birinci Boyut İçin
                std::cout << std::setw(4) << matris[i][j];
                std::cout << std::endl;
        }
}
/* Program Çıktısı:
1 3 4 5 8 7 6 5 4 4 5 6 8 9 0 2
TABLO:
1   3   4   5
8   7   6   5
4   4   5   6
*/
```

Bir başka örnek olarak elemanları klavyeden girilen 3x2'lik matrisin satır ve sütun toplamlarını ekrana yazan programı verilebilir;

```
#include <iostream>
#define SATIR 3
#define SUTUN 2
int main() {
        int matris[SATIR][SUTUN]; //Matris Tanımı
        int i,j;  //i sutun için, j satır için tanımlandı
        for (i=0; i<SATIR; i++) {  //İkinci Boyut (SATIR) İçin
                std::cout << i << ". Satırı Girin:";
                for (j=0; j<SUTUN; j++)  //Birinci Boyut (SUTUN) İçin
                        std::cin >> matris[i][j];
        }
        int satirToplamlar[SATIR]={0}; //Bütün elemanları 0 olan dizi
        for (i=0; i<SATIR; i++) {  // İkinci Boyut İçin
                for (j=0; j<SUTUN; j++) // Birinci Boyut İçin
                        satirToplamlar[i]+=matris[i][j];
                        /* İçdeki for bitince tüm satırdaki elemanlar toplanmış oldu */
        } /* Dışdaki for ile de her bir satır için toplam tekrarlanıyor */
        std::cout << "Satır Toplamları:" << std::endl;
```

```
        for (i=0; i<SATIR; i++)
                std::cout << i << ". Satır Toplamı:" <<satirToplamlar[i] << std::endl;
        int sutunToplamlar[SUTUN]={0}; //Bütün elemanları 0 olan dizi
        for (j=0; j<SUTUN; j++) // Birinci Boyut İçin
                for (i=0; i<SATIR; i++) {  // İkinci Boyut İçin
                        sutunToplamlar[j]+=matris[i][j];
                        /* İçdeki for bitince tüm sutundaki elemanlar toplanmış oldu*/
                }/* Dışdaki for ile de her bir sutun için toplam tekrarlanıyor */
        std::cout << "Sütun Toplamları:" << std::endl;
        for (j=0; j<SUTUN; j++)
                std::cout << j << ". Sütun Toplamı:" << sutunToplamlar[j] << std::endl;
}
/* Program Çıktısı:
0. Satırı Girin:2 8
1. Satırı Girin:13 17
2. Satırı Girin:16 34
Satır Toplamları:
0. Satır Toplamı:10
1. Satır Toplamı:30
2. Satır Toplamı:50
Sütun Toplamları:
0. Sütun Toplamı:31
1. Sütun Toplamı:59
*/
```

8.4 Çok Boyutlu Diziler

Şu ana kadar gördüğümüz tek boyutlu diziler veya iki boyutlu diziler olan matrislerdir. Çok boyutlu dizilere örnek olarak En-Boy-Yükseklik içeren 3 boyutlu diziler verilebilir. Üç boyutlu dizilerde dizinin her bir elemanı bir matristir. Çok boyutlu dizilerde de en içteki dizinin boyutu kimliklendirmede sağda yer alır. Aşağıdaki örnekte; çeşitli boyutlarda diziler tanımlanmıştır;

```
int ucbuyutludizi1[3][2][2]; // ucbuyutludizi1: elemanları 3 adet 2x2 matris olan üç boyutlu bir
↪ dizi
int ucbuyutludizi1[2][3][3]; // ucbuyutludizi2: elemanları 2 adet 3x3 matris olan üç boyutlu bir
↪ dizi
int ucbuyutludizi1[4][3][2]; // ucbuyutludizi3: elemanları 4 adet 3x2 matris olan üç boyutlu bir
↪ dizi
int dortbuyutludizi1[5][2][3][2]; /* dortbuyutludizi1: elemanları 5 adet olan ve her bir elemanı
↪ 2 adet 3x2 matris olan dört boyutlu bir dizi */
```

Aşağıda üç boyutlu dizilerde bir ilk değer verme kod örneği verilmiştir;

```
#define DERINLIK 3
#define SATIR 3
#define SUTUN 3
int ucBoyutluDizi[DERINLIK][SATIR][SUTUN]={
        { {1,2,3},{4,5,6},{7,8,9} },
        { {11,12,13},{14,15,16},{17,18,19} },
        { {21,22,23},{24,25,26},{27,28,29} }
};
```

Çok boyutlu dizi örneği olarak şöyle bir örnek verilebilir; 3 farklı ders alan 10 öğrenci, her bir dersten 4 değişik not almaktadır. Bu notların ağırlıkları sırasıyla %10, %30, %20 ve %40'tır. Notlar rastgele 0 ile 100 arasında verilecektir. Her bir sınavın ortalaması ile her bir öğrencinin ağırlıklı not ortalamalarını bulan C++ programı aşağıdaki gibidir;

```
#include <iostream>
#include <vector>
#include <random> // Modern rastgele sayı üretimi için
#include <iomanip> // Çıktı formatlama (F2, F1 gibi) için
int main() {
        // Sabit değerlerin tanımlanması
        const int DERS_SAYISI = 3;
        const int SINAV_SAYISI = 4;
```

```
        const int OGRENCI_SAYISI = 10;
        // C++ Çok Boyutlu Dizi Tanımı
        int notlar[DERS_SAYISI][SINAV_SAYISI][OGRENCI_SAYISI];
        int agirlik[] = { 10, 30, 20, 40 };
        // Modern C++ Rastgele Sayı Üreticisi
        std::random_device rd;
        std::mt19937 gen(rd());
        std::uniform_int_distribution<> dis(0, 100);
        // 1. Notları Rastgele Doldurma
        for (int i = 0; i < DERS_SAYISI; i++) {
                for (int j = 0; j < SINAV_SAYISI; j++) {
                        for (int k = 0; k < OGRENCI_SAYISI; k++) {
                                notlar[i][j][k] = dis(gen);
                        }
                }
        }
        // 2. Sınav Ortalamalarını Hesaplama
        std::cout << "Sinav Ortalamalari:\n";
        for (int i = 0; i < DERS_SAYISI; i++) {
                std::cout << i << ". Ders:\n";
                for (int j = 0; j < SINAV_SAYISI; j++) {
                        std::cout << " " << j << ". Sinav: ";
                        int notToplam = 0;
                        for (int k = 0; k < OGRENCI_SAYISI; k++) {
                                notToplam += notlar[i][j][k];
                        }
                        float ortalama = static_cast<float>(notToplam) / OGRENCI_SAYISI;
                        std::cout << std::fixed << std::setprecision(2) << ortalama << " ";
                }
                std::cout << "\n";
        }
        // 3. Her bir Öğrencinin Ağırlıklı Ortalaması
        std::cout << "\nHer bir Ogrencinin Agirlikli Ortalamasi:\n";
        for (int k = 0; k < OGRENCI_SAYISI; k++) {
                std::cout << k << ". Ogrenci: ";
                for (int i = 0; i < DERS_SAYISI; i++) {
                        std::cout << " " << i << ". Ders: ";
                        float agirlikliNot = 0.0f;
                        for (int j = 0; j < SINAV_SAYISI; j++) {
                                agirlikliNot += (notlar[i][j][k] * agirlik[j]) / 100.0f;
                        }
                        std::cout << std::fixed << std::setprecision(1) << agirlikliNot << " ";
                }
                std::cout << "\n";
        }
}
/*Program Çıktısı:
Sinav Ortalamalari:
0. Ders:
0. Sinav: 51.00  1. Sinav: 46.30  2. Sinav: 47.50  3. Sinav: 35.30
1. Ders:
0. Sinav: 61.10  1. Sinav: 60.00  2. Sinav: 74.30  3. Sinav: 40.80
2. Ders:
0. Sinav: 42.10  1. Sinav: 31.60  2. Sinav: 56.90  3. Sinav: 43.70

Her bir Ogrencinin Agirlikli Ortalamasi:
0. Ogrenci:  0. Ders: 68.2  1. Ders: 42.7  2. Ders: 66.4
1. Ogrenci:  0. Ders: 40.2  1. Ders: 51.3  2. Ders: 18.0
2. Ogrenci:  0. Ders: 34.0  1. Ders: 68.8  2. Ders: 55.5
3. Ogrenci:  0. Ders: 29.8  1. Ders: 72.6  2. Ders: 48.2
4. Ogrenci:  0. Ders: 42.3  1. Ders: 75.5  2. Ders: 42.2
5. Ogrenci:  0. Ders: 35.9  1. Ders: 30.6  2. Ders: 35.8
```

```
6. Ogrenci:  0. Ders: 7.7  1. Ders: 46.1  2. Ders: 49.6
7. Ogrenci:  0. Ders: 47.9  1. Ders: 36.0  2. Ders: 29.7
8. Ogrenci:  0. Ders: 72.3  1. Ders: 59.0  2. Ders: 51.4
9. Ogrenci:  0. Ders: 47.8  1. Ders: 70.3  2. Ders: 28.7
*/
```

8.5 Parametre Olarak Diziler

Parametre olarak gönderilen dizinin boyutu kadar köşeli parantez açılır ve kapatılır, ilk köşeli parantez içerisine eleman sayısını ifade eden değer yazılmaz, fakat diğerlerine eleman sayıları verilmek zorundadır. Bu durumda tek boyutlu diziler parametre olacak ise dizinin boyutu yazılmayabilir. Aşağıda dizileri parametre olarak alan fonksiyon bildirim örnekleri bulunmaktadır;

```
void diziIsleyenFonksiyon1(int tekBoyutluDizi[],int boyut);
void diziIsleyenFonksiyon2(int pDizi[][2],int ikinciBoyut,int birinciBoyut);
void diziIsleyenFonksiyon3(int pDizi[][3][2],int ucuncuBoyut,int ikinciBoyut,int birinciBoyut);

// Bu fonksiyonları aşağıdaki şekilde çağırabiliriz;
unsigned notlar[10]; //10 öğrenci notu barındırır.
unsigned dersNotlari[2][10]; //2 dersi alan 10 öğrenci için notlar;
unsigned bolumDersNotlari[4][2][10]; /* 4 bölümde 2 dersi alan 10 öğrenci için notlar;*/
//...
diziIsleyenFonksiyon1(notlar,10);
diziIsleyenFonksiyon2(dersNotlari,2,10);
diziIsleyenFonksiyon3(bolumDersNotlari,4,2,10);
//...
```

Aşağıda bir diziyi okuyan, ekrana yazan ve eleman ortalamalarını hesaplayan fonksiyonlara sahip bir program verilmiştir;

```
#include <iostream>
#include <iomanip> // setw()
void diziOku(int pDizi[],int pUzunluk);
void diziYaz(int pDizi[],int pUzunluk);
float diziOrtalama(int pDizi[],int pUzunluk);
int main() {
        int dizi[5];
        float ortalama;
        diziOku(dizi,5); /* fonsiyon geri dönüş değeri olmadığından bir değişkene atanmıyor! */
        diziYaz(dizi,5); /* fonsiyon geri dönüş değeri olmadığından bir değişkene atanmıyor! */
        ortalama=diziOrtalama(dizi,5); /* fonksiyon geri dönüş değeri bir değişkene atanıyor.
         ↪  Atandığı değişkenin tipi ile fonksiyonun geri dönüş tipi aynı olmalı! */
        std::cout << "Dizi Ortalaması:" <<ortalama ;
        return 0;
}
void diziOku(int pDizi[],int pUzunluk){
        int sayac;
        std::cout << pUzunluk << " Elemanlı Dizi Okunacaktır:";
        for (sayac=0; sayac < pUzunluk; sayac++)
                std::cin >> pDizi[sayac];
}
void diziYaz(int pDizi[],int pUzunluk){
        int sayac;
        std::cout << pUzunluk << " Elemanlı Dizi:" << std::endl;
        for (sayac=0; sayac < pUzunluk; sayac++)
                std::cout << std::setw(5) << pDizi[sayac] ;
        std::cout << std::endl;
}
float diziOrtalama(int pDizi[],int pUzunluk){
        int sayac;
        float toplam=0;
        for (sayac=0; sayac < pUzunluk; sayac++)
                toplam+=pDizi[sayac];
        return toplam/pUzunluk;
```

```
}
/* Program Çıktısı:
5 Elemanlı Dizi Okunacaktır:10 20 30 40 50
5 Elemanlı Dizi:
10   20   30   40   50
Dizi Ortalaması:30
*/
```

8.6 Aralık Tabanlı For Döngüsü

Yalnızca bir dizideki öğeler arasında döngü oluşturmak için kullanılan döngü, **her bir eleman için döngü** (foreach loop) yada **aralık tabanlı döngü** (range based looprange based loop) olarak da bilinir. Bu döngü ileride göreceğimiz Konteyner Şablonları başlığındaki veri yapılarıyla da kullanılır. Aşağıdaki gibi yazılır;

```
for (veritipi dizi-elemanını-temsil-eden-değişken : dizi-kimliği)
    döngü-kodu;
```

Buna örnek olarak aşağıda bir dizi programı örneği verilebilir;

```
#include <iostream>
#include <iomanip> // setw()
int main() {
        int dizi[5]={2,4,6,8,10};
        std::cout << "Dizi:" << std::endl;
        for (int eleman: dizi)
                std::cout << std::setw(5) << eleman ;
        std::cout << std::endl;
        float toplam=0;
        for (int eleman: dizi)
                toplam+=eleman;
        std::cout << "Dizi Ortalaması:" <<toplam /(sizeof (dizi)/sizeof (int));
}
/* Program Çıktısı:
Dizi:
2    4    6    8   10
Dizi Ortalaması:6
*/
```

8.7 Düşün ve Çöz

- **Düşün**: Çok boyutlu dizilerin (matrislerin) bellekte aslında tek boyutlu ve ardışık olarak tutulmasının, dizi elemanlarına erişim hızına etkisini tartışınız.
- **Düşün**: C++ dilinde dizi sınırlarının (out of bounds) kontrol edilmemesinin, bellek güvenliği ve ara bellek taşması (buffer overflow) saldırıları açısından riskini tartışınız.
- **Çöz**: 10 elemanlı rastgele sayılardan oluşan bir diziyi kabarcık sıralama (bubble sort) algoritması ile küçükten büyüğe sıralayan fonksiyonu yazınız.
- **Çöz**: 20 elemanlı bir dizideki elemanların frekansını (hangi sayıdan kaç tane olduğunu) bulan ve ekrana listeleyen programı yazınız.
- **Çöz**: Kullanıcıdan alınan iki adet 3x3'lük matrisin toplamını bulan ve sonucu ekrana matris formunda basan fonksiyonu kodlayınız.
- **Çöz**: İki boyutlu bir diziyi bellekte satır öncelikli (row-major) düzende saklandığını düşünerek, `int matris[3][4];` dizisinin `matris[1][2]` elemanının bellek adresini hesaplayın (başlangıç adresi 1000, her int 4 bayt).
- **Düşün**: Aralık tabanlı for (range-based for) döngüsünü klasik indis tabanlı döngüyle karşılaştırın. Aralık tabanlı for döngüsünün avantajları ve sınırlılıkları nelerdir? Hangi durumda klasik döngü zorunlu hale gelir?
- **Düşün**: Bir diziyi fonksiyona parametre olarak geçirdiğinizde C++'da ne olur? Fonksiyon içinde dizinin boyutunu `sizeof` ile öğrenebilir misiniz? Bu **dizi bozunması** (array decay) sorununu nasıl çözersiniz?
- **Çöz**: Çok boyutlu dizi ile işaretçiler dizisi arasındaki farkı açıklayın. `int mat[3][4];` ile `int* mat[3];` ifadelerinin bellekte nasıl farklı yerleştiğini gösterin.

9. Bölüm

Göstericiler ve Referanslar

9.1 Gösterici nedir? Niçin İhtiyaç Duyarız?

Şu ana kadar gördüğümüz `char`, `int`, `long`, `float`, `double` tipli değişkenler ve bunlara ait diziler, değer tipler (value type) olarak adlandırılır. Çünkü kimliklendirilen değişken, değeri doğrudan ifade eder. Değer tiplerin yanı sıra değişkenlerin adreslerini tutan değişkenlere de ihtiyaç duyarız. İşte adres tutan bu değişkenlere **gösterici tipler** (pointer type) adını veririz. Göstericileri işaret ettikleri veri tipine (data type) göre tanımlarız. Bu nedenle göstericiler, **türetilmiş tipler**(derived type) olarak adlandırılır.

Gösterici tipler, gösterdiği yerdeki verileri değiştirmek için de kullanılır. **Gösterici tipler işaret ettiği verinin tipine göre tanımlanırlar. Bütün gösterici tipler bellekte aynı miktarda yer kaplar. Çünkü hepsi adres tutar**. Bütün bu tanımlamaların ışığında göstericileri, vatandaşların ikametlerini gösteren adres numaraları olarak düşünebiliriz.

Gösterici tiplere ihtiyaç duyulmasının sebebi hız ve esnekliktir. Genel olarak birkaç madde ile sıralayacak olursak;

- Belleğin istenilen bölgesine erişim sağlamak için göstericiler kullanılır. Günümüz modern dillerinde **referans tipler** (referans type) kullanılır. Göstericilerden farklı olarak referans tipler, gösterilen adreste bir verinin/nesnenin bulunduğu garanti eder. Göstericiler ise her bellek bölgesine erişebilir.
- Bazen çalışma anında (run-time) yeni bellek bölgelerine ihtiyaç duyarız bu durumda göstericiler gereklidir. **Dinamik veri yapıları** (dynamic data structure) göstericiler yardımıyla oluşturulup yönetilir.
- Fonksiyonlara parametre geçirilirken argüman olarak kullanılan yerel değişkenlerin kopyaları kullanılır. Bu durum Şekil 7.3'de açıklanmıştı. Fonksiyondan geri dönüldüğünde ise yerel değişkenler eski haline yığın bellekten geri çekilerek çevrilir. Fonksiyonun yerel değişken ya da argümanlardan birini değiştirmesi istendiğinde fonksiyona argüman olarak değişkenin adresi verilir. Böylece fonksiyona parametre olarak geçirilen yerel değişkenler göstericiler üzerinden fonksiyon içinde değiştirilebilir hale gelir.

Göstericilere olan ihtiyacı bir başka örnekle de açıklayabiliriz; Bir ormandaki ağaçları boylarına göre sıralamaya kalkarsak her birini yer değiştirme yöntemiyle sıralamak oldukça külfetli ve zaman alan bir işlemdir. Halbuki ağaçlara numara vererek ve bu numaraların karşısına uzunluklarını yazacağımız bir liste oluşturulduğunda sadece numaraları sıralamak oldukça kolay ve zahmetsiz bir işlemdir. İşte buradaki numaraları karşılık gelen değişkenler göstericilerdir.

9.2 Gösterici Tanımlama

Göstericiler, işaret ettiği verinin tipine göre tanımlanırlar. Gösterici tanımlanırken veri tipi izleyen **başvuru kaldırma işleci** (dereference operator) (`*`) kullanılır. Başvuru kaldırma deyimi, gösterici tarafından tutulan bellek adresinde saklanan değere erişmeyi ifade eder.

```
//GÖSTERİCİ TANIMLAMA:
veritipi *gostericiKimligi1;
veritipi* gostericiKimligi2;

//AYNI VERİ TİPİNDE TEK SATIRDA BİRDEN ÇOK GÖSTERİCİ TANIMLAMA:
veritipi* birinciGösterici, ikinci gösterici;

//AYNI VERİ TİPİNDE TEK SATIRDA HEM DEĞİŞKEN HEM DE GÖSTERİCİ TANIMLAMA:
veritipi değişkenKimliği, *göstericiKimliği3;
```

Göstericinin işaret ettiği değişkene değer atama aşağıdaki şekilde yapılır. Burada göstericinin veri tipi ile atanan değer aynı veri tipinde olmalıdır;

```
gostericikimligi = &degiskenkimliği;
```

Bir değişkenin adresinin **adres işleci** ile elde edilebileceği 4.9.2 başlığında anlatılmıştı. Aşağıda göstericilere ilişkin kod örnekleri verilmiştir;

```
char *ptrToChar;  /*göstericinin işaret ettiği adreste "char" tipinde veri olan "ptrToChar"
 ↪  kimlikli göstericisi tanımlandı.*/

int i=10;
int *ptrToInt=&i; /*göstericinin işaret ettiği adreste "int" tipinde veri olan "ptrToInt"
 ↪  kimlikli göstericisi tanımlandı. Ancak göstericinin işeret etteği adres olarak i değişkeninin
 ↪  adresi atanıyor.*/

*ptrToInt =20; /* "ptrToInt" göstericisinin işaret ettiği adresteki tam sayı değeri 20 yaptık.
 ↪  "ptrToInt" göstericisi, i değişkeninin adresini işaret ettiği i değişkeninin değeri de 20
 ↪  olmuştur */
```

Veri tiplerinin işlendiği 4.3 başlığında, **hiçbir değeri olmayan ve bellekte yer kaplamayan** `void` veri tipi anlatılmıştı. Veri tipi `void` olan `void*` göstericisine **jenerik gösterici** (generic pointer) adı verilir.

Bazı durumlarda göstericilerin veya gösterdiği değerlerin sabit olması gerekebilir. Bunun için üç durum ortaya çıkabilir;

1. **İşaret edilen değerin sabit olması durumu**: Bu durumda gösterici tanımlaması aşağıdaki gibi yapılır;
```
int main() {
        int i=10;
        const int *ptrToi=&i; // değerin sabit olması durumunda gösterici tanımı
        *ptrToi=20; /* HATA: error: assignment of read-only location '*ptrToi'
        pi göstericisinin işaret ettiği adresteki tam sayı değeri 20 yapılamaz. */
}
```
2. **Göstericinin işaret ettiği adresin sabit olması durumu**: Kısaca göstericinin sabit olması durumunda tanımlama aşağıdaki gibi yapılır;
```
int main() {
        int i=10;
        int *const ptrToi=&i; // göstericinin sabit olması durumunda gösterici tanımı
        /* Bu tür tanımlamada bir adres ilk değer (initialize) olarak mutlaka
         ↪  verilmelidir. */
        int j=30;
        *ptrToi=20; // pi göstericisinin işeret ettiği adresteki tam sayı değeri 20 olur.
        ptrToi =&j; /*HATA:error: assignment of read-only variable 'ptrToi'
        pi göstericisinin işeret ettiği adres değiştirilemez!*/
}
```
3. **Hem göstericinin hem de değerin sabit olması durumu**:
```
int main() {
        int j=10, k=100;
        const int * const ptr=&j; /* Hem gösterici, hem de gösterilen veri sabit */
        /* Bu tür tanımlamada da bir adres ilk değer (initialize) olarak mutlaka
         ↪  verilmelidir. */
        *ptr=20; /* HATA: error: assignment of read-only location '*(const int *)ptr'*/
        ptr=&k; /*HATA: error: assignment of read-only variable 'ptr' */
}
```

Gösterici tipe atamaya uygun kesin bir adresiniz yoksa, `nullptr` **boş göstericisini** (null pointer) atamak her zaman iyi bir uygulamadır. İlk değeri olmayan gösterici tanımlamak yerine, ilk değeri `nullptr` olan göstericiler tanımlanmak doğru bir yöntemdir. Aşağıdaki gibi kullanılır;

```
veritipi *gostericikimligi = nullptr; //tanımlarken
gostericikimligi = nullptr; //tanımlı bir göstericiye nullptr ataması
```

Aşağıda boş gösterici kullanımına ilişkin örnek bir uygulama verilmiştir;

```
#include <iostream>
int main(){
        int *ptr = nullptr;
        std::cout << "ptr göstericisinin işaret ettiği adres: " <<  ptr << std::endl;
        int agirlik=80;
        ptr=&agirlik;
        std::cout << "ptr göstericisinin işaret ettiği adres: " << ptr << " ve değeri: " << *ptr
         ↪  <<std::endl;
}
```

```
/* Program Çıktısı:
ptr göstericisinin işaret ettiği adres: 0
ptr göstericisinin işaret ettiği adres: 0x7ffda3348764 ve değeri: 80
*/
```

Bir göstericinin işaret ettiği yerde bir başka gösterici olabilir. Buna **çifte gösterici** (double pointer) adı verilir. Aşağıda buna ilişkin bir örnek verilmiştir;

```
#include <iostream>
int main(){
        int var = 10;
        int *intPtr = &var;
        int **ptrToPtr = &intPtr;  //double pointer
        std::cout << "var:" << var << " &var:"<< &var<< std::endl;
        std::cout << "inttptr:"<< intPtr <<" *inttptr:" << &intPtr <<std::endl;
        std::cout << "var:"<< var <<" *intPtr:" << *intPtr << std::endl;
        std::cout << "ptrToPtr:" << ptrToPtr <<" &ptrtptr:" << &ptrToPtr << std::endl;
        std::cout << "&intPtr:" << &intPtr << " *ptrToPtr:" << *ptrToPtr << std::endl;
        std::cout << "var:" << var <<" *intPtr:" << *intPtr <<" **ptrToPtr:" << **ptrToPtr <<
         ↪  std::endl;
}
/*Program Çıktısı:
var:10 &var:0x7ffc7a8533ac
inttptr:0x7ffc7a8533ac *inttptr:0x7ffc7a8533a0
var:10 *intPtr:10
ptrToPtr:0x7ffc7a8533a0 &ptrtptr:0x7ffc7a853398
&intPtr:0x7ffc7a8533a0 *ptrToPtr:0x7ffc7a8533ac
var:10 *intPtr:10 **ptrToPtr:10
*/
```

Bir tam sayı bellekte 4 bayt yer işgal eder. Bu baytların bellekte nasıl yerleştiğini gösteren program verilmiştir.

```
#include <iostream>
#include <iomanip> // std::hex, std::setw ve std::setfill için
int main() {
        int i = 0x10203040;
        unsigned char* p = reinterpret_cast<unsigned char*>(&i); // C++'da daha gösterici
         ↪  dönüşümleri için güvenli olan static_cast veya reinterpret_cast tercih edilir. Burada
         ↪  i tam sayısının adresinde "unsigned char" işaret eden bir gösterici tanımladık.
        std::cout << "Sayı  :" << std::setw(10) << std::dec << i << "-" << std::hex << i <<
         ↪  std::endl;
        // Baytları yazdırma (1. bayttan 4. bayta)
        for (int n = 0; n < 4; ++n) {
                std::cout << n + 1 << ".bayt:" << std::setw(10) << std::dec
                        << static_cast<int>(*(p + n)) // "unsigned char" sayısal değer için int'e
                         ↪  cast edilir
                        << "-" << std::hex << std::setw(8 - (n * 2)) << std::setfill(' ')
                        << static_cast<int>(*(p + n)) << std::endl;
        }
}
/*Program Çıktısı:
Sayı  : 270544960-10203040
1.bayt:        64-      40
2.bayt:        48-    30
3.bayt:        32-  20
4.bayt:        16-10
*/
```

9.3 Dinamik Değişken Kullanımı

Şu ana kadar nesnelerin imal edildiği bellek bölgesi, yığın bellek bölgesidir (stack segment). Değişkenleri çalışma anında (run-time), öbek bellekte (heap segment) de imal edebiliriz. Değişkenleri dinamik başlatma aşağıdaki şekilde yapılır;

```
//Bellekte tek bir değişken için bellek tahsisi:
veri-tipi* göstericisi = new veri-tipi;
//Bellekte tek bir değişken için bellek tahsisi:
//Ancak veri tipinde bir değişmez ile başlatma yapılmak istenirse:
veri-tipi* göstericisi = new veri-tipi(ilk-değer);
//boyut kadar elemanı olan dizi değişkeni için bellek tahsisi:
veri-tipi* dizi-göstericisi = new veri-tipi[boyut];
```

Dinamik olarak **new işleci** (new operator) ile imal edilecek değişkene öbek bellekte (heap segment) yer tahsis edilir (memory allocation). Ardından tahsis edilen belleği işaret eden gösterici geri döner. Diziler için dizinin ilk elemanını işaret eden gösterici geri döner. Göstericin işaret ettiği adresteki veriye **başvuru kaldırma işleci** (dereference operator) (->) ile erişilir.

Dinamik olarak imal edilmiş nesne kullanıldıktan sonra **delete işleci** (delete operator) ile bellekten kaldırılabilir (deallocate memory). Aşağıda buna ilişkin bir örnek verilmiştir;

```
#include <iostream>
int main() {
        // 1. DİNAMİK TEKİL DEĞİŞKEN (Geleneksel Yöntem)
        double* dPtr = new double; // tek değişkene bellek tahsis ediliyor.
        *dPtr = 3.14;
        std::cout << "Double Deger: " << *dPtr << " | Adres: " << dPtr << std::endl;
        delete dPtr; // tek değişken için tahsis edilen bellek serbest bırakılıyor.
        double* dPtr2 = new double(3.1415); // tek değişkene bellek tahsis ediliyor.
        std::cout << "Double Deger: " << *dPtr << " | Adres: " << dPtr << std::endl;
        delete dPtr2; // tek değişken için tahsis edilen bellek serbest bırakılıyor.
        // 2. DİNAMİK DİZİ (Geleneksel Yöntem)
        int boyut = 5;
        double* diziPtr = new double[boyut]; // 5 elemanlık yer açıldı
        diziPtr[0] = 10.5;
        diziPtr[1] = 20.7;
        diziPtr[2] = 30.9;
        //Atanmayan dizi elemanlarına dikkat ediniz!
        std::cout << "--- Dinamik Dizi Elemanlari ---" << std::endl;
        for (int i = 0; i < boyut; i++) {
                std::cout << i << ". eleman: " << diziPtr[i] << " | Adres: " << &diziPtr[i] <<
                 ↪  std::endl;
        }
        delete[] diziPtr; // KRİTİK: Dizi serbest bırakılırken mutlaka köşeli parantez
         ↪  kullanılmalı!
        // 3. MODERN YAKLAŞIM: Akıllı Göstericiler (Smart Pointer - unique_ptr)
        // 'delete' yazmanıza gerek kalmaz, kapsam bitince kendi silinir. İleride Anlatılacaktır.
        std::unique_ptr<double[]> akilliDizi = std::make_unique<double[]>(2);
        akilliDizi[0] = 100.1;
        akilliDizi[1] = 200.2;
        std::cout << "Akilli Isaretci ile Dizi: " << akilliDizi[0] << ", " << akilliDizi[1] <<
         ↪  std::endl;
}
/*
Double Deger: 3.14 | Adres: 0x21ad92b0
Double Deger: 3.1415 | Adres: 0x21ad92b0
--- Dinamik Dizi Elemanlari ---
0. eleman: 10.5 | Adres: 0x21ad96e0
1. eleman: 20.7 | Adres: 0x21ad96e8
2. eleman: 30.9 | Adres: 0x21ad96f0
3. eleman: 0 | Adres: 0x21ad96f8
4. eleman: 0 | Adres: 0x21ad9700
Akilli Isaretci ile Dizi: 100.1, 200.2
*/
```

9.4 Gösterici Aritmetiği

Göstericilerde toplama ve çıkarma tam sayılardan farklı şekilde çalışır. **Bir gösterici artırıldığında veya azaltıldığında, işaret ettiği adres, işaret ettiği veri tipinin bellek boyutu kadar artırılır veya azaltılır.** Bunun

nedeni bilgisayarların tasarımından ötürü işlemciye bağlı belleklerde her bir bayt ayrı adreslenmesidir.

Bir `char*` göstericisi, işaret ettiği yerdeki `char` verisinden bir sonraki `char` verisini göstermesi istendiğinde **gösterici adresi 1 artar**. Çünkü `char` veri tipi bellekte 1 bayt yer kaplar. Aşağıdaki program çıktısındaki adres değişimlerini inceleyiniz;

```
#include <iostream>
const int BOYUT = 5;
void printDizi(const char* d) {
        std::cout << "DİZİ:{";
                for (int i = 0; i < BOYUT; i++)
                std::cout << "'" << d[i] << "'" << (i < BOYUT - 1 ? "," : "");
                std::cout << "}" << std::endl;
}
int main() {
        char dizi[BOYUT] = {'I', 'L', 'H', 'A', 'N'};
        // Dizinin ilk elemanını işaret eden gösterici
        char* ptrToChar = &dizi[0];
        // C++'da char* adresini yazdırmak için void*'e cast etmek şarttır
        std::cout << "İşaret Edilen Adres: " << static_cast<void*>(ptrToChar) << std::endl;
        *ptrToChar = 'X';
        printDizi(dizi);
        // Göstericiyi bir sonraki bayta (karaktere) ilerlet
        ptrToChar++;
        std::cout << "İşaret Edilen Adres: " << static_cast<void*>(ptrToChar) << std::endl;
        *ptrToChar = 'Y';
        printDizi(dizi);
        // Göstericiyi 3 adım ileri taşı (L'den N'ye)
        ptrToChar += 3;
        std::cout << "İşaret Edilen Adres: " << static_cast<void*>(ptrToChar) << std::endl;
        *ptrToChar = 'Z';
        printDizi(dizi);
}
/*Program Çıktısı:
İşaret Edilen Adres: 0x7ffe0bfe4523
DİZİ:{'X','L','H','A','N'}
İşaret Edilen Adres: 0x7ffe0bfe4524
DİZİ:{'X','Y','H','A','N'}
İşaret Edilen Adres: 0x7ffe0bfe4527
DİZİ:{'X','Y','H','A','Z'}
*/
```

Benzer şekilde `int*` göstericisi, işaret ettiği yerdeki `int` verisinden bir sonraki `int` verisini göstermesi istendiğinde **gösterici adresi 4 artar**. Çünkü `int` veri tipi bellekte 4 bayt yer kaplar. Aşağıdaki program çıktısındaki adres değişimlerini inceleyiniz;

```
#include <iostream>
const int BOYUT = 5;
void printDizi(const int* d) {
        std::cout << "DİZİ:{";
                for (int i = 0; i < BOYUT; i++)
                std::cout << "'" << d[i] << "'" << (i < BOYUT - 1 ? "," : "");
                std::cout << "}" << std::endl;
}
int main() {
        int dizi[BOYUT] = {10, 20, 30, 40, 50};
        // Dizinin ilk elemanını işaret eden gösterici
        int* ptrToInt = &dizi[0];
        // C++'da int* adresini yazdırmak için void*'e cast etmek şarttır
        std::cout << "İşaret Edilen Adres: " << static_cast<void*>(ptrToInt) << std::endl;
        *ptrToInt = -1;
        printDizi(dizi);
        // Göstericiyi bir sonraki bayta (karaktere) ilerlet
        ptrToInt++;
```

```
        std::cout << "İşaret Edilen Adres: " << static_cast<void*>(ptrToInt) << std::endl;
        *ptrToInt = -2;
        printDizi(dizi);
        // Göstericiyi 3 adım ileri taşı
        ptrToInt += 3;
        std::cout << "İşaret Edilen Adres: " << static_cast<void*>(ptrToInt) << std::endl;
        *ptrToInt = -3;
        printDizi(dizi);
}
/*Program Çıktısı:
İşaret Edilen Adres: 0x7ffd26121c90
DİZİ:{'-1','20','30','40','50'}
İşaret Edilen Adres: 0x7ffd26121c94
DİZİ:{'-1','-2','30','40','50'}
İşaret Edilen Adres: 0x7ffd26121ca0
DİZİ:{'-1','-2','30','40','-3'}
*/
```

Bunun gibi;

- `short*` göstericisi, işaret ettiği yerdeki `short` verisinden bir sonraki `short` verisini göstermesi istendiğinde gösterici adresi 2 artar. Çünkü `short` veri tipi bellekte 2 bayt yer kaplar.
- `long*` göstericisi, işaret ettiği yerdeki `long` verisinden bir sonraki `long` verisini göstermesi istendiğinde gösterici adresi 4 artar. Çünkü `long` veri tipi bellekte 4 bayt yer kaplar;
- `float*` göstericisi, işaret ettiği yerdeki `float` verisinden bir sonraki `float` verisini göstermesi istendiğinde gösterici adresi 4 artar. Çünkü `float` veri tipi bellekte 4 bayt yer kaplar.
- `double*` göstericisi, işaret ettiği yerdeki `double` verisinden bir sonraki `double` verisini göstermesi istendiğinde gösterici adresi 8 artar. Çünkü `double` veri tipi bellekte 8 bayt yer kaplar;

Ayrıca göstericinin artırılması yanında eksiltilmesi de mümkündür. Bu durumda `-`, `-=` ve `--` işleçleri kullanılır.

- Görüldüğü üzere **göstericilerin üzerinde işleçler (operator) işlem yaparken göstericinin işaret ettiği verinin tipine göre farklı işlem yaparlar.**
- Gösterici aritmetiğinde (`++`, `--`, `+`, `-`, `+=` ve `-=`) ile (`<`, `>` ve `==`) çalışır. **Diğer işleçler çalışmaz.**

9.5 Gösterici Dizileri

Göstericiler de dizi olarak tanımlanabilir. Aşağıda birbirinden bağımsız tanımlanmış yerel değişken adreslerinden bir gösterici dizisi tanımlanmıştır. Bu dizi üzerinden değişkenler daha kolay işlenebilir;

```
#include <iostream>
#include <vector>
int main() {
        int i = 10, j = 20;
        float f = 1.0; // Kodda doğrudan kullanılmıyor ama yerini koruduk
        int k = 30, l = 40;
        // C++'da ham dizi yerine std::array de kullanılabilir,
        // ancak mantığı korumak için gösterici dizisi (pointer array) tanımlıyoruz:
        int* ptr[4];
        // Göstericilere adres atamaları
        ptr[0] = &i; // i'nin adresi
        ptr[1] = &j; // j'nin adresi
        ptr[2] = &l; // l'nin adresi
        ptr[3] = &k; // k'nin adresi
        // ptr[2] (l değişkeni) üzerinden değer atama
        *ptr[2] = -1;
        // Değerlere ulaşma ve yazdırma
        for (int indis = 0; indis < 4; indis++) {
                std::cout << "ptr[" << indis << "] ile gösterilen değer: " << *ptr[indis] <<
                ↪  std::endl;
        }
}
```

```
/* Program Çıktısı:
ptr[0] ile gösterilen değer: 10
ptr[1] ile gösterilen değer: 20
ptr[2] ile gösterilen değer: -1
ptr[3] ile gösterilen değer: 30
*/
```

9.6 Referanslar

Referanslar (reference), bellekte halihazırda var olan bir değişkenin **takma adı** (alias) gibidir. Bir değişken gibi kendisine yeni bir bellek alanı tahsis edilmez, sadece mevcut bir değişkene ilişkin bellek bölgesine ikinci bir isim verir. Referansın adresi ile bağlandığı değişkenin adresi tamamen aynıdır. Bir değişken, bildiriminde kimlik önüne (&) konularak referans olarak bildirilebilir.

```
veri-tipi varolan-bir-değişken;
veri-tipi &referans-kimliği = varolan-bir-değişken;
veri-tipi& referans-kimliği = varolan-bir-değişken;
```

Aşağıda buna ilişkin örnek bir program verilmiştir;

```
#include <iostream>
using namespace std;
int main(){
        int x = 10;
        int &ref = x; // ref ile var olan x değişkenine referans tanımlanmıştır.
        ref = 20; // x değişkeninin yeni değeri 20 olmuştur.
        cout << "x = " << x << endl; // x = 20
        x = 30; // x değişkeninin yeni değeri 30 olmuştur
        cout << "ref = " << ref << endl; // ref = 30
}
```

Özellikle fonksiyonlara büyük veriler (örneğin binlerce elemanlı bir dizi veya büyük bir nesne) gönderirken kopyalama maliyetinden kurtulmak için kullanılır. Değerle gönderirseniz her seferinde kopyası alınır bu da icra süresini uzatır yani yavaştır, referansla gönderirseniz orijinal veri üzerinden işlem yapılır bu da icra süresini çok kısaltır yani çok hızlıdır.

Referanslar ile göstericileri karşılaştırdığımızda;

- Göstericiler herhangi bir zamanda boş gösterici (NULL Pointer) olabildiği gibi başka bir nesneyi işaret edilebilir. Bir referans tanımlandığı anda ise mevcut bir değişken ile ilk değer verilmelidir. Göstericiler tanımlandıktan sonra herhangi bir zamanda değeri değişebilir ya da gösterdiği değişken değiştirilebilir.
- Hiçbir değişkeni işaret etmeyen boş referansınız (NULL Reference) olamaz! Her zaman bir referansın meşru bir değişkene bağlı olduğunu varsayabilmelisiniz.
- Bir referans imal edilen bir nesneyi işaret ediyorsa, başka bir nesneyi göstermek üzere referansı değiştirilemez!

Göstericiler yerine referanslar kullanmak kodumuzun okunmasını ve bakımını daha kolay hale getirilebilir. Bir C++ fonksiyonu, bir gösterici döndürdüğü gibi bir referansı da döndürebilir. Bir fonksiyon bir referans döndürdüğünde, dönüş değerine örtük bir gösterici döndürür. Böylece atama ifadesinin sol tarafında bir fonksiyon kullanılabilir hale gelir.

```
#include <iostream>
#include <ctime>
float dizi[5] = {10.0, 20.0, 30.0, 40.0, 50.0};
float& indistekiDeger( int indis ) { //dizinin bir elemanının referansını döndüren bir fonksiyon
        return dizi[indis];
}
int main () {
        std::cout << "Değiştirmeden Önce Dizi" << std::endl;
        std::cout << "{";
        for ( int i = 0; i < 5; i++ )
                std::cout << dizi[i] << ",";
        std::cout << "}" << std::endl;
        //Referanslar üzerinden değerler değiştiriliyor...
```

```
        indistekiDeger(1) = 100.0;
        indistekiDeger(3) = 200.0;
        std::cout << "Değiştirildikten Sonra Dizi" << std::endl;
        std::cout << "{";
        for ( int i = 0; i < 5; i++ )
                std::cout << dizi[i] << ",";
        std::cout << "}" << std::endl;
}
/* Program Çıktısı:
Değiştirmeden Önce Dizi
{10,20,30,40,50,}
Değiştirildikten Sonra Dizi
{10,100,30,200,50,}
*/
```

Bir fonksiyondan bir referans döndürürken, referans alınan nesnenin kapsam (scope) dışına çıkmamasına dikkat edilmesi gerekir;

```
int& fonksiyon() {
        int q;
        // return q; /* Derleme hatası olur. Çünkü fonksiyon dışına çıkıldığında bu değişken
        ↪  bellekte yığın bellekte yoktur.*/
        static int x;
        return x; /* x değişkeni tüm program süresince hayatta olduğundan bu kod güvenlidir. */
}
```

9.7 Referansla Çağırma

Fonksiyonlara parametre geçirilirken argüman olarak kullanılan yerel değişkenlerin kopyaları kullanılır. Bu durum Şekil 7.3'de açıklanmıştı. Fonksiyondan geri dönüldüğünde ise yerel değişkenler eski haline yığın bellekten geri çekilerek çevrilir. Buna ilişkin örnek aşağıda verilmiştir;

```
#include <iostream>
// Fonksiyon Bildirimi/Prototipi
void degistir(int, int);
int main() {
        int a = 10, b = 20;
        degistir(a, b);
        // Çağrı sonrası yerel değişkenler yığın bellekten çekildiği için değerler değişmez
        std::cout << "1- a:" << a << ", b:" << b << std::endl; // Çıktı: a:10, b:20
}
void degistir(int pX, int pY) {
        int z = pX;
        pX = pY;
        pY = z;
}
```

Fonksiyona geçirilen yerel değişken ya da argümanlardan birinin değiştirilmesi istendiğinde bu fonksiyona argüman olarak değişkenin adresi verilir. Böylece fonksiyona parametre olarak geçirilen yerel değişkenler dolaylı olarak göstericiler üzerinden fonksiyon içinde değiştirilebilir hale gelir. Fonksiyona geçirilecek argümanlar gösterici olarak tanımlanır. Çağrı ortamına geri dönüldüğünde yerel değişkenler de değişmiş olur. Bu işleme **göstericililer üzerinden referansla çağırma** (call by reference over pointer) adı verilir. Çünkü fonksiyona değişken adresleri değer olarak geçirilmiştir. Buna ilişkin örnek aşağıda verilmiştir;

```
#include <iostream>
// Fonksiyon prototipi: int* (gösterici) kabul eder
void degistir(int* pX, int* pY);
int main() {
        int a = 10, b = 20;
        // Değişkenlerin adreslerini (&) gönderiyoruz
        degistir(&a, &b);
        // Çıktı: a:20, b:10
        std::cout << "2- a:" << a << ", b:" << b << std::endl;
}
void degistir(int* pX, int* pY) {
```

```
        int z = *pX; // pX'in işaret ettiği adresteki değeri al
        *pX = *pY;   // pY'nin değerini pX'in adresine yaz
        *pY = z;     // Yedeklenen değeri pY'nin adresine yaz
}
```

Göstericiler üzerinden referansla çağırma yerine bir önceki 9.6 başlığındaki referanslar üzerinden fonksiyonu çağırmak daha güvenlidir. Buna referansla çağırma (call by reference) bu durumda yukarıdaki örnek daha güvenli olarak aşağıda verilmiştir;

```
#include <iostream>
// Fonksiyon prototipi: int& (referans) kabul eder
void degistir(int& pX, int& pY);
int main() {
        int a = 10, b = 20;
        // Adres (&) veya yıldız (*) operatörüne gerek yok! Normal değişken gibi gönderiyoruz.
        degistir(a, b);
        // Çıktı: a:20, b:10
        std::cout << "3- Referans ile a:" << a << ", b:" << b << std::endl;
}
void degistir(int& pX, int& pY) {
        int z = pX; // pX artık main'deki 'a'nın ta kendisidir
        pX = pY;    // pY'den gelen değer 'a'ya yazılır
        pY = z;     // z'deki değer 'b'ye yazılır
}
```

9.8 Göstericilerin Dizileri İşaret Etmesi

Çoğu durumda, bir dizi yardımıyla gerçekleştirdiğimiz işleri gösterici ile de gerçekleştirilebiliriz. Dizi değişkeni, dizinin ilk öğesinin adresi işaret eder. Kısaca dizi değişkenleri gösterici gibi davranırlar.

```
#include <iostream>
int main() {
        int dizi[5] = {10, 20, 30, 40, 50};
        int* ptr = dizi; // Dizinin ilk elemanının adresini tutar
        std::cout << "Dizi elemanlarına gösterici ile erişim: " << std::endl;
        for (int indis = 0; indis < 5; ++indis)
                // *(ptr + indis) ifadesi pointer aritmetiği kullanarak değere ulaşır
                std::cout << "dizi[" << indis << "]: " << *(ptr + indis) << std::endl;
}
/* Program Çıktısı:
Dizi elemanlarına gösterici ile erişim:
dizi[0]: 10
dizi[1]: 20
dizi[2]: 30
dizi[3]: 40
dizi[4]: 50
*/
```

Fonksiyonlara dizileri işaret eden göstericileri parametre olarak verebiliriz. Dizi değişkeni, dizinin ilk öğesinin adresi işaret ettiğinden dolayı d**izi değişkenleri (array) parametre olarak kullanılırken adres işleci (address operator) kullanılmaz.** Bu durumda parametre olarak dizi elemanının veri tipinde bir göstericiyi parametre olarak tanımlarız.

```
float notlar[10];
float notlarOrtalamasi(float*); //prototype
//...
float ort=notlarOrtalamasi(notlar); // yada notlarOrtalamasi(&notlar[0]);
//...
float matris[4][3];
float defetminant(float**,int,int); //prototype
//...
float det=defetminant(matris,4,3);
//...
```

Aşağıda öğrencilerin notlarına ilişkin işlemler yapılan bir program verilmiştir.

```
#include <iostream>
#include <iomanip>
// C++'ta sabitler için constexpr veya const önerilir
constexpr int OGRENCISAYISI = 10;
float notlarOrtalamasi(const float*); // Ortalamada dizi değiştirilmediği için const eklendi
void notlariArtir(float*);
int main() {
        float notlar[OGRENCISAYISI] = { 55.0, 60.0, 70.0, 35.0, 30.0, 50.0, 65.0, 90.0, 95.0,
        ↪  100.0 };
        float ortalama = notlarOrtalamasi(notlar);
        std::cout << "Notlar Ortalaması: " << std::fixed << std::setprecision(1) << ortalama <<
        ↪  std::endl;
        notlariArtir(notlar); // Dizi içeriği fonksiyon içinde değiştiriliyor
        for (int indis = 0; indis < OGRENCISAYISI; ++indis)
                std::cout << "Not[" << indis << "]=" << notlar[indis] << std::endl;
}
float notlarOrtalamasi(const float* pNotlar) {
        float top = 0.0f;
        for (int indis = 0; indis < OGRENCISAYISI; ++indis)
                top += pNotlar[indis];
        return top / OGRENCISAYISI;
}
void notlariArtir(float* pNotlar) {
        for (int indis = 0; indis < OGRENCISAYISI; ++indis)
                if (pNotlar[indis] <= 90.0f)
                        pNotlar[indis] += 10.0f;
}
/* Programın Çıktısı:
Notlar Ortalaması: 65.0
Not[0]=65.0
Not[1]=70.0
Not[2]=80.0
Not[3]=45.0
Not[4]=40.0
Not[5]=60.0
Not[6]=75.0
Not[7]=100.0
Not[8]=95.0
Not[9]=100.0
*/
```

9.9 Fonksiyonlardan Bir Diziyi Geri Döndürme

C dilinde bir fonksiyonun geri dönüş değeri olarak tüm bir dizi verilemez! Ancak, bir dizinin göstericisini fonksiyondan geri dönüş değeri olarak döndürebilirsiniz. **Burada dikkat edilmesi gereken fonksiyondan çıkılınca dizinin bellekten kaldırılmamasıdır.** Çünkü yerel değişkenler yığın bellekte tutulduğundan fonksiyondan çıkılınca bellekten silinirler. Aşağıda bir tam sayı dizi göstericisini döndüren bir fonksiyon örnekleri verilmiştir;

```
int* tekBoyutluDiziDondur() {
 /*... */
}
int** ikiBoyutluDiziDondur() {
 /*... */
}
```

Yerel değişkenlerin yığın bellekte (stack segment) tutulduğu ve fonksiyondan geri dönünce fonksiyon yerelindeki değişkenlerin kaybolduğu 7.5.1 başlığında ve Şekil 7.3'de anlatılmıştı. Bu nedenle bir fonksiyon içinde tanımlanan bir dizinin göstericisi geri döndürülecek ise veri bellekte (data segment) yer almasını sağlayacak `static` tanımlamasıdır.

```
#include <iostream>
#include <iomanip> // setw için
#include <cstdlib> // rand ve srand için
```

```
#include <ctime>   // time için
constexpr int BOYUT = 10;
// Fonksiyon int* (gösterici) döndürüyor
int* rastgeleOgrenciNotlari() {
        // static dizi: Fonksiyon bitse bile bellekte kalmaya devam eder.
        static int notlar[BOYUT];
        for (int i = 0; i < BOYUT; ++i)
                notlar[i] = std::rand() % 101;
        return notlar; // Dizinin ilk elemanının adresini döndürür
}
void notlariYaz(const int* pNotlar, int pUzunluk) {
        std::cout << "Öğrenci Notları:" << std::endl;
        for (int i = 0; i < pUzunluk; ++i)
                std::cout << std::setw(4) << pNotlar[i];
        std::cout << std::endl;
}
int main() {
        // Rastgele sayı üretecini kur
        std::srand(static_cast<unsigned int>(std::time(nullptr)));
        // Fonksiyondan gelen adresi bir işaretçiye alıyoruz
        int* notlar = rastgeleOgrenciNotlari();
        notlariYaz(notlar, BOYUT);
}
/*Program Çıktısı:
Öğrenci Notları:
40  80  43  79  36  67  75  48  50  17
*/
```

C++ dilinde `static` dizi döndürmek tehlikeli olabilir (thread-safe değildir ve bellek yönetimini zorlaştırır). İleride göreceğimiz dinamik dizileri kullanarak (`std::vector`) nesneyi güvenli bir şekilde fonksiyondan dışarı taşıyabiliriz.

9.10 Fonksiyon Göstericisi

Geri çağırma (callback) fonksiyonları, özellikle **olay güdümlü programlamada** (event-driven programming) çok kullanışlıdır. Bir olay tetiklendiğinde, bu olaylara yanıt olarak ona eşlenen bir geri çağırma fonksiyonu çalıştırılır. Genellikle grafik ara yüzlü işletim sistemlerinde kullanıcı ara yüzünde bir düğmeye tıklanması gibi bir olay, önceden tanımlanmış bir dizi işlemi başlatabilir. Bu durum, geri çağırma işlevleriyle gerçekleştirilir.

Geri çağırma fonksiyonu, temel olarak, başka bir koda argüman olarak geçirilen ve belirli bir zamanda argümanı geri çağırması beklenen bir icra edilebilir koddur. Geri çağırma mekanizması fonksiyon göstericisine bağlıdır. **Fonksiyon göstericisi** (function pointer), bir fonksiyonun bellek adresini depolayan bir değişkendir. Aşağıda buna ilişkin basit bir örnek bulunmaktadır;

```
#include <stdio.h>
void merhaba() {
        printf("Merhaba!\n");
}
void callback(void (*ptr)()) {
        printf("Geri çağırma fonksiyonu çağrılacak!\n");
        (*ptr)();   // Geri çağırma fonksiyonu çağrılıyor
}
main() {
        void (*ptr)() = merhaba;
        callback(ptr);
}
```

9.11 Dinamik Hafıza Yönetimi

C++ dilinde bir fonksiyon bloğu içerisinde tanımlanan tüm değişkenler yığın bellek (stack segment) üzerinde yer kaplayacaktır. Program çalıştığında bir değişkene belleği çalışma anında (run-time) tahsis etmek için **öbek bellek** (heap segment) kullanılır. Buna **dinamik bellek tahsisi** adı verilir. Dinamik bellek tahsisi için `new` işlecini (new operator) kullanırız. Bu işleç, dinamik dizi için öbek bellekte yer tahsis edilip tahsis edilen bellek

bölgesinin ilk baytının adresini göstericiye atar. Tahsis edilen bellek bölgesini serbest bırakmak için ise `delete` işleci (delete operator) kullanılır. Aşağıda bir `float` değişkene dinamik olarak yer tahsisi örneği verilmiştir;

```
#include <iostream>
int main(){
        float* ptrFloat=nullptr;
        ptrFloat=new float; // bir float değişkene heap segment üzerinde yer ayrılır ve adresi
         ↪  döndürülür.
        if (ptrFloat!=nullptr) {
                *ptrFloat=4.5;
                std::cout << *ptrFloat << std::endl;
        } else
                std::cout << "Bellek tahsisi yapılamadı!" << std::endl;
        delete ptrFloat; // tahsis edilen bellek bölgesi serbest bırakılır.
        ptrFloat=nullptr; /*Kod devam edecek ise bu talimat yazılmalıdır. */
}
```

C++ dilinde diziler de dinamik olarak göstericiler yardımıyla oluşturulur. Dinamik bellek tahsisi, çalışma zamanı sırasında bellek tahsis etmemize olanak tanır. Bir dizideki dinamik tahsis, özellikle bir dizinin boyutu derleme zamanında bilinmediğinde ve çalışma zamanı sırasında belirtilmesi gerektiğinde faydalıdır. Bir diziyi dinamik olarak tahsis etmek için;

- Tahsis edilen dizinin ilk elemanının adresini depolayacak bir gösterici tanımlanır.
- Sonra, belirli bir veri tipindeki bir diziyi barındıracak dizi için `new` işleci ile bellek alanını tahsis edilir.
- Bu tahsisi yaparken, kaç öğe içerebileceğini belirten dizinin boyutunu belirtilir. Bu belirtilen boyut, tahsis edilecek bellek miktarını belirler.

Dizi uzunluğu tam sayı olacak şekilde dinamik dizi aşağıdaki gibi tanımlanır;

```
veritipi* diziGöztericiKimliği= new veritipi[diziUzunluğu];
```

Aşağıda çalışma anında (run-time) dinamik olarak oluşturulan bir dizi örneği verilmiştir;

```
#include <iostream>
int main() {
        // Çalışma Anında Oluştutulacak Dizi Boyutu Soruluyor
        std::cout << "Dizinin Boyutunu Girin: ";
        int size;
        std::cin >> size;
        int* arr = new int[size]; // Diziye dinamik bellek (heap) tahsis ediliyor.
        if (!arr) { // if (arr!=nullptr) de yazılabilir
                for (int i = 0; i < size; i++) // Bir döngü ile dizi elemanlarına değer atanıyor
                        arr[i] = i * 2;
                std::cout << "Dinamik Dizi Elemanları: " << std::endl;
                for (int i = 0; i < size; i++)
                        std::cout << arr[i] << " ";
                std::cout << std::endl;
        }
        delete[] arr; // dizi bellekten kaldırılıyor. Tahsis edilen bellek serbest bırakılıyor.
}
/* Program Çıktısı:
Dizinin Boyutunu Girin: 5
Dinamik Dizi Elemanları:
0 2 4 6 8
*/
```

> C++'ta `new` ile ayrılan her bellek alanı, programcı tarafından mutlaka `delete` kullanarak elle serbest bırakılmalıdır. `delete` talimatı kullanılmazsa, program kapansa bile o bellek alanı işletim sistemi temizleyene kadar tahsis edilmiş kalabilir.

Çok boyutlu dizilere de dinamik olarak bellek tahsisi yapılabilir;

```
double** pvalue  = nullptr;   // Göstericiye ilk değer olarak nullptr veriliyor
pvalue  = new double [3][4];  // 3x4 buyutlu martis için yer tahsisi yapılıyor.
//...
delete[] pvalue; // tahsis edilen bellek serbest bırakılıyor
```

C++'ta dinamik bellek yönetimi yaparken, manuel `new` ve `delete` kullanımı yerine Modern C++ (C++11/14/17/20) tekniklerini kullanmak, kodunuzu hem daha güvenli hale getirir hem de **bellek sızıntılarını** (memory leak) tamamen önler. İleride anlatılacaktır.

9.12 Metinler ve Göstericiler

C dilinden miras kalan ve C++'ta da hâlâ geçerli olan eski yöntem **metinler** (string), özünde birer **karakter dizisidir**. Modern `std::string` sınıfının aksine, bu yaklaşım çok daha düşük seviyelidir ve bellek yönetimi tamamen yazılımcının sorumluluğundadır.

§9.12.1 Karakter Dizisi

Eski yöntemde bir metin (string), `char` tipindeki verilerin yan yana dizildiği bir dizi (array) içinde saklanır. Ancak, bilgisayarın bu dizinin nerede bittiğini bilmesi gerekir. İşte bu noktada **boş karakter** (NULL Character) `'\0'` devreye girer. Boş karakter, değeri 0 olan özel bir karakterdir. Metnin bittiğini işaret eder. C stili tüm fonksiyonlar metni bu karaktere kadar okur. Bu karakter, metnin gerçek uzunluğuna +1 bayt eklenmesini gerektirir.

```
//Karakter Dizisi olarak dizgi tanımı:
char metin1[]={'C','L','a','n','g','\0'};
// metin1 dizgisi 6 karakterden oluşan dizi ile başlatılmıştır.
char metin2[]={'C','L','a','n','g', 0 };
// metin2 dizgisi 6 karakterden oluşan dizi ile başlatılmıştır.
char metin3[50]={'C','L','a','n','g', 0 };
// metin3 dizgileri 6 karakterden oluşmasına rağmen bellekte 30 bayt yer kaplar.
```

Metinleri oluşturan karakter dizilerine ilk değer vermek için çift tırnak karakteri kullanılır. Çift tırnak arasında metni oluşturan harflerin yazılmasıyla karakter dizisi (metin) değişmezleri (string literal) yazılır. Metni ifade eden karakter dizilerini uzun uzun dizi şeklinde başlatmak yerine metinler çift tırnak karakteri kullanılır;

```
char metin1[] = "CLang";
//Kısaca  6 karakterden oluşan dizgi tanımı
char metin2[40] = "Clang"
// metin3 dizgileri 6 karakterden oluşmasına rağmen bellekte 40 bayt yer kaplar.
```

Şimdiye kadar anlatılanlar ışığında dizgi tanımlama örneği aşağıda verilmiştir;

```
#include <cstdio> // C-stili I/O için C++'ta kullanılır
#include <iostream>
int main() {
        // YÖNTEM A: Boyut belirterek ve dizgi literal ile atama
        char dizi1[10] = "Merhaba";
        // "Merhaba" 7 karakterdir. '\0' için en az 8 bayt gerekir.
        // Kalan 2 bayt otomatik olarak '\0' ile doldurulur.
        // YÖNTEM B: Boyutu derleyiciye bırakma
        char dizi2[] = "Dunya";
        // Derleyici boyutu otomatik olarak 6 (5 harf + '\0') olarak ayarlar.
        // YÖNTEM C: Karakter karakter atama (Tavsiye edilmez, hata risklidir)
        char dizi3[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
        // DİKKAT: '\0' unutulursa, fonksiyonlar bellekte rastgele okumaya devam eder!
        std::cout << "Dizi 1: " << dizi1 << std::endl;
        std::cout << "Dizi 2: " << dizi2 << std::endl;
        std::cout << "Dizi 3: " << dizi3 << std::endl;
}
```

§9.12.2 Gösterici ile Metinlerin İlişkisi

C++ dilinde bir dizinin adı, aslında o dizinin ilk elemanının bellek adresini gösteren **sabit bir göstericidir** (constant pointer). Çift tırnak içindeki metin değişmezleri (string literal) de bellekte bir yerde saklanır ve program o adresi kullanır. Bu durumda gösterici karakter dizisinin ilk elemanını gösterir;

```
char* metin3="CLang";
```

Buna ilişkin örnek program aşağıda verilmiştir;

```
#include <iostream>
```

```
int main() {
        char ad[] = "Ahmet"; // Değiştirilebilir bir karakter dizisi
        const char* ptr = "Ahmet"; // Bellekteki sabit bir metne tanımlanan gösterici
        // Adresin kendisini yazdırma
        std::cout << "Dizinin bellek adresi: " << (void*)ad << std::endl;
        std::cout << "Isaretcinin bellek adresi: " << (void*)ptr << std::endl;
        // İşaretçi aritmetiği
        std::cout << "ptr[0]: " << ptr[0] << std::endl; // 'A'
        std::cout << "ptr[1]: " << *(ptr + 1) << std::endl; // 'h'
        // DİKKAT: Göstericinin gösterdiği sabit metin değiştirilemez!
        // ptr[0] = 'B'; // HATA! Program çalışma anında çöker.
}
/*Program Çıktısı:
Dizinin bellek adresi: 0x7fff9f4397b2
Isaretcinin bellek adresi: 0x402004
ptr[0]: A
ptr[1]: h
*/
```

§9.12.3 Metinlerle İşlemler

C-stili karakter dizilerinde standart operatörler (`=`, `+`, `==`) metin işlemleri için doğrudan çalışmaz. Bu işlemler için `<cstring>` başlık dosyasındaki fonksiyonlar kullanılır.

Atama işleci (= operator)dizinin içeriğini kopyalamaz, sadece adresi kopyalamaya çalışır (veya hata verir). İçeriği kopyalamak için `strcpy()` fonksiyonu kullanılır. **Bu kopyalamada hedef dizinin, kaynak diziyi ve boş karakteri alacak kadar büyük olduğundan emin olmalısınız.**

```
#include <iostream>
#include <cstring>
int main() {
        char kaynak[] = "Merhaba";
        char hedef[20]; // Yeterince büyük bir alan ayırdık
        // hedef = kaynak; // HATA! Dizilere doğrudan atama yapılamaz.
        strcpy(hedef, kaynak); // kaynak'ın içeriğini hedef'e kopyalar.
        std::cout << "Hedef: " << hedef << std::endl;
        std::cout << "Boyut: " << strlen(hedef) << " karakter" << std::endl;
}
```

Metin birleştirme (string concatenate) için toplama işleci (+ operator) yerine `strcat()` fonksiyonu kullanılır. Bu fonksiyon, bir metnin sonuna başka bir metni eklemek için kullanılır. Hedef dizinin, mevcut metin + eklenecek metin + boş karakteri alacak kadar büyük olduğundan emin olmalısınız. **Bu kontrol yapılmazsa tampon taşması (buffer overflow) hatası oluşur ve bu çok ciddi bir güvenlik açığıdır.**

```
#include <iostream>
#include <cstring>
int main() {
        char ad[30] = "Ahmet"; // Başlangıçta boş yer bıraktık
        char soyad[] = " Yilmaz";
        strcat(ad, soyad); // ad = ad + soyad gibi düşünün
        std::cout << "Tam Ad: " << ad << std::endl; //Tam Ad: Ahmet Yilmaz
}
```

Metinleri karşılaştırma (string compare) için `strcmp()` fonksiyonu kullanılır. eşit mi işleci (== operator) metinlerin içeriğini değil, bellekteki adreslerinin aynı olup olmadığını karşılaştırır. İçerik karşılaştırması için `strcmp()` kullanılır. Dönüş değeri 0 ise iki metin birbirine eşittir. Dönüş değeri sıfırdan farklı ise metinler farklıdır (alfabetik sıraya göre farkın yönünü gösterir). Üstelik sadece ASCII (American Standard Code for Information Interchange) kodları üzerinden karşılaştırma yapar.

```
#include <iostream>
#include <cstring>
int main() {
        char s1[] = "elma";
        char s2[] = "elma";
        char s3[] = "armut";
        // HATALI KULLANIM:
```

```
        if (s1 == s2) { // Bu muhtemelen 'yanlis' döner, çünkü adresler farklıdır.
                std::cout << "[==] Metinler esit (HATALI MANTIK)" << std::endl;
        }
        // DOGRU KULLANIM:
        if (strcmp(s1, s2) == 0)
        std::cout << "[strcmp] s1 ve s2 esit" << std::endl; //[strcmp] s1 ve s2 esit
        if (strcmp(s1, s3) != 0)
        std::cout << "[strcmp] s1 ve s3 farkli" << std::endl; //[strcmp] s1 ve s3 farkli
}
```

§9.12.4 Metinler İçin Tavsiye Edilen Yöntem

Gördüğünüz gibi eski yöntem çok fazla dikkat gerektirir ve bellek taşması hatalarına çok açıktır. Bu yüzden modern C++ kodlarında, bu düşük seviyeli detaylarla uğraşmamak ve daha güvenli, okunabilir kod yazmak için her zaman `std::string` sınıfını kullanmalısınız. İleride anlatılacaktır. C-stili metinleri sadece eski C kütüphaneleriyle uyumluluk gibi zorunlu durumlarda tercih etmelisiniz. Aşağıda modern kullanıma yönelik örnek kullanım verilmiştir.

```
#include <iostream>
#include <string> // string sınıfı için gerekli
int main() {
        std::string metin1;
        std::cout << "metin1:" << metin1 << std::endl; //metin1:
        metin1="İlhan";
        std::cout << "metin1:" << metin1 << std::endl; //metin1:İlhan
        std::string metin2=metin1;
        std::cout << "metin2:" << metin2 << std::endl; //metin2:İlhan
        metin2="İlhan";
        if (metin1==metin2)
                std::cout << "metin1 ile metin2 eşittir." << std::endl; //metin1 ile metin2
                 ↪  eşittir.
        metin2="Özkan";
        std::string metin3=metin1+" "+metin2;
        std::cout << "metin3:" << metin3 << std::endl; metin3:İlhan Özkan
}
```

9.13 Düşün ve Çöz

- **Düşün**: "Bir gösterici aslında sadece bir tam sayıdır (adres)" ifadesi doğru mudur? Gösterici tipi (`int*`, `char*` vb.) gösterici aritmetiğini nasıl etkiler?
- **Düşün**: `const int *p;` ile `int * const p;` tanımları arasındaki farkı bellek ve erişim yetkisi açısından açıklayınız.
- **Düşün**: Göstericiler ile diziler arasındaki adres-offset ilişkisini, dizi ismiyle gösterici aritmetiği yaparak açıklayınız.
- **Çöz**: Bir fonksiyon yazınız; bu fonksiyon iki tam sayının adresini parametre olarak alsın ve bu sayıların değerlerini bellekte yer değiştirsin.
- **Çöz**: Bir karakter dizgisini (string) gösterici aritmetiği kullanarak tersten ekrana yazdıran bir fonksiyon yazınız.
- **Çöz**: Bir fonksiyona parametre olarak başka bir fonksiyonun adresini gönderen (function pointer) basit bir matematiksel işlem seçici (topla/çıkar) tasarlayın.

10. Bölüm

Yapılar, Birlikler ve İsim Uzayları

10.1 Yapılar

Yapısal programlamaya adını veren yapı (structure), türetilmiş (derived) veya kullanıcı tanımlı (user defined) bir veri tipidir. Farklı tiplerdeki elemanları bir arada gruplandıran özel bir veri tipi tanımlamak için `struct` anahtar kelimesini kullanırız. Yapı bir veya birden fazla ilkel veri tipinin (`char`, `int`, `long`, `float`, `double`, ...) ve bu tiplere ait dizilerin bir araya gelmesiyle oluşturulan yeni veri tipleridir.

Diziler, aynı tipte elemanlardan oluşmasına rağmen, yapılar farklı dipte elemanların bir araya gelmesiyle oluşabilir. Aşağıda sözde kodu verilmiştir;

```
struct yapı-kimliği {
 veri-tipi yapı-elemanı-kimliği1;
 veri-tipi yapı-elemanı-kimliği1;
 ...
};
```

Örnek olarak `adı`, `soyadı`, `numarası`, `yaşı`, `cinsiyeti` gibi farklı öğeler ile bir `Ogrenci` yapısı tanımlanabilir;

```
struct Ogrenci { // C++'ta tür isimlerini büyük harfle başlatmak yaygındır
        std::string adi;
        std::string soyadi;
        int yas;
        int cinsiyet; // 1: Erkek, 2: Kadın
};
```

Yapı tanımlandıktan sonra aşağıdaki sözde kodda verildiği gibi yeni değişkenler tanımlanabilir;

```
yapı-kimliği değişken-kimliği[,değişken-kimliği2][,değişken-kimliği3];
```

Yapının elemanlarına tanımlama sırasında **başlangıç değeri** (initial value) verilebilir yani yapı değişkeni bazı değerlerle **başlatılabilir** (initialize). Bunun için diziler gibi süslü parantez kullanılır;

```
Ogrenci ogrenci1; // Başlatılmamış
Ogrenci ogrenci2={"Deniz", "AK", 35, 2};
Ogrenci ogrenci3={"Damla", "YILDIZ", 28, 2};
```

Bir yapının içinde bir başka yapı da tanımlanabilir. Yani iç içe yapılar (nested structs) tanımlanabilir.

10.2 Yapı Elemanlarına Erişim

Tanımlanan yapı değişkenleri üzerinden her bir alamana **nokta işleci** (dot operator) erişilir. Bu işlecin önceliği parantez ile aynıdır. Atama işleci (=) bir yapıyı doğrudan kopyalamak için kullanılabilir. Ayrıca, bir yapının üyesinin değerini başka birine atamak için atama işlecini de kullanabiliriz.

```
#include <iostream>
#include <string> // string sınıfı için gerekli
struct Ogrenci { // C++'ta tür isimlerini büyük harfle başlatmak yaygındır
        std::string adi;
        std::string soyadi;
        int yas;
        int cinsiyet; // 1: Erkek, 2: Kadın
};
int main() {
        // ogrenci1 yapısı başlatılmadığından değerleri aşağıdaki gibi güncelleniyor:
        // C++ std::string ile doğrudan atama yapabiliriz, strcpy'ye gerek yok.
        Ogrenci ogrenci1;
        ogrenci1.adi = "Ilhan";
        ogrenci1.soyadi = "OZKAN";
```

```
        ogrenci1.yas = 50;
        ogrenci1.cinsiyet = 1;
        std::cout << "Ogrenci 1 Bilgileri:" << std::endl;
        std::cout << "Adi: " << ogrenci1.adi << std::endl;  // nokta işleci ile adi alanına
         ↪ erişildi.
        std::cout << "Soyadi: " << ogrenci1.soyadi << std::endl; // nokta işleci ile soyadi
         ↪ alanına erişildi.
        std::cout << "Yasi: " << ogrenci1.yas << std::endl; // nokta işleci ile yas alanına
         ↪ erişildi.
        std::cout << "Cinsiyeti: " << ogrenci1.cinsiyet << std::endl; // nokta işleci ile
         ↪ cinsiyet alanına erişildi.
        std::cout << "-------------------" << std::endl;
        // yapı bazı değerlerle başlatıldı. 'struct' anahtar kelimesi C++'ta zorunlu değil.
        Ogrenci ogrenci2 = {"Deniz", "AK", 35, 2};
        std::cout << "Ogrenci 2 Bilgileri:" << std::endl;
        std::cout << "Adi: " << ogrenci2.adi << std::endl;
        std::cout << "Soyadi: " << ogrenci2.soyadi << std::endl;
        std::cout << "Yasi: " << ogrenci2.yas << std::endl;
        std::cout << "Cinsiyeti: " << ogrenci2.cinsiyet << std::endl;
        std::cout << "-------------------" << std::endl;
        // yapı kopyalama (sığ kopyalama)
        Ogrenci ogrenci3 = ogrenci2;
        std::cout << "Ogrenci 3 Bilgileri (Ogrenci 2'den kopyalandi):" << std::endl;
        std::cout << "Adi: " << ogrenci3.adi << std::endl;
        std::cout << "Soyadi: " << ogrenci3.soyadi << std::endl;
        std::cout << "Yasi: " << ogrenci3.yas << std::endl;
        std::cout << "Cinsiyeti: " << ogrenci3.cinsiyet << std::endl;
}
```

10.3 Yapı Göstericileri

Yapı göstericileri, tıpkı diğer değişkenlere gösterici tanımladığımız gibi tanımlayabiliriz. Gösterici üzerinden yapı değişkenlerine **dolaylı işleç** (indirection operator) yani (->) ile erişilir. Bu nokta işleci ile aynı önceliklidir.

Yapılar ve göstericileri; veri tabanları, dosya yönetim uygulamaları ve ağaç ve bağlı listeler gibi karmaşık veri yapılarını işlemek gibi farklı uygulamalarda kullanılır.

```
#include <iostream> // std::cout için
#include <string>   // std::string için
// C++'ta struct isimlerini Büyük Harfle başlatmak yaygındır
struct Ogrenci {
        std::string adi;
        std::string soyadi;
        int yas;
        int cinsiyet; // 1: Erkek, 2: Kadın
};
int main() {
        Ogrenci ogrenci1;
        // Alanlar std::string tanımlandığından strcpy() yerine doğrudan atama yapabiliriz. Çok
         ↪ daha güvenli.
        ogrenci1.adi = "Ilhan";
        ogrenci1.soyadi = "OZKAN";
        ogrenci1.yas = 50;
        ogrenci1.cinsiyet = 1;
        // İşaretçi tanımı. Yine 'struct' kelimesi yok.
        Ogrenci* ogrenciGosterici;
        ogrenciGosterici = &ogrenci1;
        // nokta işleci yerine '->' işleci kullanılır.
        std::cout << "Adi: " << ogrenciGosterici->adi << std::endl
        << "Soyad: " << ogrenciGosterici->soyadi << std::endl
        << "Yasi: " << ogrenciGosterici->yas << std::endl
        << "Cinsiyeti: " << ogrenciGosterici->cinsiyet << std::endl;
```

```
}
```

10.4 Yapılar ve Fonksiyonlar

Sıradan dizilere benzer şekilde bir yapı dizisi tanımlayabiliriz. Ayrıca yapı değişkenini bir fonksiyona parametre olarak gönderebilir ve bir fonksiyondan bir yapı döndürebilirsiniz;

```
#include <iostream>
#include <string> // std::string sınıfı için gerekli
struct Ogrenci {
        std::string adi;
        std::string soyadi;
        int yas;
        int cinsiyet; // int yerine ileride göreceğimiz doğrudan enum tipi kullanmak daha güvenli
};
Ogrenci yeniOgrenci();
void ogrenciYaz(Ogrenci);
int main() {
        Ogrenci ogrenciler[30]; // 30 elemanlı Öğrenci dizisi
        Ogrenci o = yeniOgrenci();
        ogrenciYaz(o);
}
Ogrenci yeniOgrenci() {
        // std::string ile doğrudan atama yapabiliriz. ERKEK enum değeri tam sayı gibi
        ↪ davranabilir.
        Ogrenci yeni = {"Ilhan", "Ozkan", 50, 2};
        return yeni;
}
void ogrenciYaz(Ogrenci pOgrenci) {
        std::cout << "Ogrenci Bilgileri:" << std::endl;
        std::cout << "Adi: " << pOgrenci.adi << std::endl;
        std::cout << "Soyadi: " << pOgrenci.soyadi << std::endl;
        std::cout << "Yasi: " << pOgrenci.yas << std::endl;
        std::cout << "Cinsiyeti: " << pOgrenci.cinsiyet << " (0:Belirtilmemis, 1:Kadin, 2:Erkek)"
        ↪ << std::endl;
        std::cout << "-------------------" << std::endl;
}
```

Yapı elemanları değer tipler olduğundan, yapılara ait referansları parametre olarak kullanmak, olabilecek değişiklikleri aktarmak için üstünlük sağlar. Gösterici kullanımı tehlikelidir;

```
#include <iostream>
#include <string> // std::string sınıfı için gerekli
struct Ogrenci {
        std::string adi;
        std::string soyadi;
        int yas;
        int cinsiyet;
};
Ogrenci yeniOgrenci();
void ogrenciOku(Ogrenci& pOgrenci);
void ogrenciYaz(const Ogrenci& pOgrenci);
int main() {
        Ogrenci o = yeniOgrenci();
        ogrenciYaz(o);
        ogrenciOku(o);
        ogrenciYaz(o);
}
Ogrenci yeniOgrenci() {
        Ogrenci yeni = {"Ilhan", "Ozkan", 50, 1};
        return yeni;
}
void ogrenciOku(Ogrenci& pOgrenci) {
        std::cout << "Adi Giriniz: ";
```

```
        std::cin >> pOgrenci.adi; //referans ile de üyerlere nokta işleci ile erişilir.
        std::cout << "Soyadi Giriniz: ";
        std::cin >> pOgrenci.soyadi;
        std::cout << "Yas Giriniz: ";
        std::cin >> pOgrenci.yas;
        std::cout << "Cinsiyet Giriniz (0:Belirtilmemis, 1:Erkek, 2:Kadin): ";
        std::cin >> pOgrenci.cinsiyet;
}
// 'const Ogrenci*' yapının içeriğinin değiştirilmesini engeller (güvenli).
void ogrenciYaz(const Ogrenci& pOgrenci) {
        std::cout << "-------------------" << std::endl;
        std::cout << "Ogrenci Bilgileri:" << std::endl;
        std::cout << "Adi: " << pOgrenci.adi << std::endl;
        std::cout << "Soyadi: " << pOgrenci.soyadi << std::endl;
        std::cout << "Yasi: " << pOgrenci.yas << std::endl;
        std::cout << "Cinsiyeti: " << pOgrenci.cinsiyet << std::endl;
        std::cout << "-------------------" << std::endl;
}
```

10.5 İç İçe Yapılar

İç içe yapılara (nested structs) ait `Ogrenci` yapımıza doğum tarihini içerecek yapı eklenen örnek, aşağıda verilmiştir;

```
#include <iostream>
#include <string> // std::string için gerekli
// C++'ta yapı isimlerini Büyük Harfle başlatmak yaygındır (Ogrenci)
struct Ogrenci {
        std::string adi;
        std::string soyadi;
        int yas;
        int cinsiyet; // 1:erkek, 2:Kadın, 0:Belirtmiyor
        // İç yapıyı isimlendirmek ve bir üye olarak tanımlamak modern C++'ta daha temiz ve
         ↪ güvenlidir.
        struct Tarih {
                int gun;
                int ay;
                int yil;
        } dogumTarihi;
};
Ogrenci yeniOgrenci();
void ogrenciYaz(Ogrenci);
int main() {
        Ogrenci o = yeniOgrenci();
        ogrenciYaz(o);
}
Ogrenci yeniOgrenci() {
        Ogrenci yeni = {"Ilhan", "Ozkan", 50, 1, {1, 1, 1970}};
        return yeni;
}
void ogrenciYaz(Ogrenci pOgrenci) {
        std::cout << pOgrenci.adi << std::endl;
        std::cout << pOgrenci.soyadi << std::endl;
        std::cout << "Yas:" << pOgrenci.yas << std::endl;
        std::cout << "Cinsiyet:" << pOgrenci.cinsiyet << std::endl;
        // İsimlendirilmiş iç yapı kullandığımız için erişim 'dogumTarihi' üyesi üzerinden
         ↪ yapılır.
        std::cout << "Dogum Tarihi:" << pOgrenci.dogumTarihi.gun << "-" <<
         ↪ pOgrenci.dogumTarihi.ay << "-" << pOgrenci.dogumTarihi.yil << std::endl;
}
```

10.6 Öz Referanslı Yapılar

Öz referanslı yapı (self-referetial struct), öğelerinden bir veya daha fazlası kendi tipindeki göstericilerden oluşan bir yapıdır. Öz referanslı yapılar, bağlantılı listeler ve ağaçlar gibi karmaşık olan **dinamik veri yapıları** (dynamic data structure) içinde yaygın olarak kullanılırlar.

```
#include <iostream> // std::cout için
#include <string>   // std::string için
struct Ogrenci {
        std::string adi;
        std::string soyadi;
        int yas;
        Ogrenci* sonrakiOgrenci; // Kendi veri tipinden bir yapı göstericisi
};
// 'const Ogrenci*' yapının içeriğinin değiştirilmesini engeller (güvenli).
void ogrencileriYaz(const Ogrenci*);
int main() {
        // std::string ve nullptr kullanımı ile temiz bir başlatma
        // İlk düğüm: Ilhan
        Ogrenci ilk = {"Ilhan", "OZKAN", 50, nullptr};
        // İkinci düğüm: Ayse
        Ogrenci sonraki = {"Ayse", "YILMAZ", 40, nullptr};
        // Bağlantı: Ilhan'dan Ayse'ye
        ilk.sonrakiOgrenci = &sonraki;
        // Üçüncü düğüm: Tahsin
        Ogrenci birsonraki = {"Tahsin", "BULUT", 35, nullptr};
        // Bağlantı: Ayse'den Tahsin'e
        sonraki.sonrakiOgrenci = &birsonraki;
        // Listenin başını yazdır fonksiyonuna gönderiyoruz
        ogrencileriYaz(&ilk);
}
// 'const Ogrenci*' yapının içeriğinin değiştirilmesini engeller (güvenli).
void ogrencileriYaz(const Ogrenci* pIlk) {
        int sayac = 0;
        while (pIlk != nullptr) {
                std::cout << "OGRENCI " << ++sayac << ": "
                << "Adi: " << pIlk->adi << " "
                << "Soyad: " << pIlk->soyadi << " "
                << "Yas: " << pIlk->yas << std::endl;

                // Bir sonraki düğüme geçiş
                pIlk = pIlk->sonrakiOgrenci;
        }
}
```

10.7 Yapı Dolgusu

C++ dilinde **yapı dolgusu** (structure padding), işlemci (CPU) mimarisi ile belirlenir. Dolgu işlemi ile üyelerinin bellekte doğal olarak hizalanması için belirli sayıda boş bayt eklenir. Bunun nedeni 16, 32 veya 64 bitlik bir bilgisayarda işlemcinin tek seferde bellekten okuyacağı bayt sayısının farklı okumasından kaynaklanmaktadır.

```
#include <iostream>
struct Yapi1 {
        char a;
        char b;
        int c;
};
struct Yapi2 {
        char d;
        int e;
        char f;
};
int main() {
```

```
        std::cout << "char bellek miktari: " << sizeof(char) << std::endl;
        // char bellek miktarı: 1
        std::cout << "int bellek miktari: " << sizeof(int) << std::endl;
        // int bellek miktarı: 4
        std::cout << "------------------------" << std::endl;
        // Teorik beklenen boyut
        std::cout << "Yapilardaki 2 adet char ve 1 adet int olmasi gereken bellek miktari : "
        << (2 * sizeof(char) + sizeof(int)) << std::endl;
        // yapilar için olmasi gereken bellek miktarı: 6
        // Gerçek bellek boyutları (bellek hizalaması nedeniyle farklılık gösterir)
        std::cout << "Yapi1 icin bellekte ayrilan miktar: " << sizeof(Yapi1) << std::endl;
        // Yapi1 için bellekte ayrılan miktar: 8 (1bayt a + 1bayt b + 2bayt dolgu + 4bayt c)
        std::cout << "Yapi2 icin bellekte ayrilan miktar: " << sizeof(Yapi2) << std::endl;
        // Yapi2 için bellekte ayrılan miktar: 12 (1bayt d + 3bayt dolgu + 4bayt e + 1bayt f +
        ↪  3bayt dolgu)
}
```

Yukarıda verilen örnekte aynı bellek miktarına sahip elemanlar için yapının bütününe bakıldığında farklı miktarda bellek ayrıldığı anlaşılmaktadır. Dolgu (padding) işlemi ile üyelerinin bellekte doğal olarak hizalanması için belirli sayıda boş bayt eklenir. **Kod içerisindeki tanımlanacak tüm yapılar için bunu engellemenin yolu** aşağıdaki ön işlemci yönergesini (preprocessor directive) kaynak koda eklemektir;

```
#pragma pack(1)
```

Eğer yalnızca belirlenen bir yapı için bunun yapılması isteniyorsa yapı tanımına `__attribute__((packed))` özelliği eklenir. Bu komutlar C dilinde olduğu gibi C++'ta da aynı şekilde çalışır, ancak bunlar standart C++'ın bir parçası değildir ve derleyiciden derleyiciye (GCC, Clang, MSVC) farklılık gösterebilir. Aşağıdaki kod GCC ve Clang tabanlı derleyicilerde sorunsuz çalışacaktır. Yapı dolgusuna ilişkin program örneği aşağıda verilmiştir;

```
#include <iostream>
#pragma pack(1) // Tüm yapılarda dolgu baytlarını devre dışı bırakır (Derleyiciye özgü)
struct Yapi1 {
        char a;
        char b;
        int c;
};
struct __attribute__((packed)) Yapi2 {
        char a;
        int b;
        char c;
};
int main() {
        std::cout << "char bellek miktari: " << sizeof(char) << std::endl;
        // char bellek miktarı: 1
        std::cout << "int bellek miktari: " << sizeof(int) << std::endl;
        // int bellek miktarı: 4
        std::cout << "------------------------" << std::endl;
        // Teorik beklenen boyut (paketlenmiş yapılar için)
        std::cout << "Yapilardaki 2 adet char ve 1 adet int olmasi gereken bellek miktari: "
        << (2 * sizeof(char) + sizeof(int)) << std::endl;
        // yapılar için olması gereken bellek miktarı: 6
        // Gerçek bellek boyutları (paketlenmiş oldukları için 6 olmalıdır)
        std::cout << "Yapi1 icin bellekte ayrilan miktar: " << sizeof(Yapi1) << std::endl;
        // Yapi1 için bellekte ayrılan miktar: 6 (paketlenmiş)
        std::cout << "Yapi2 icin bellekte ayrilan miktar: " << sizeof(Yapi2) << std::endl;
        // Yapi2 için bellekte ayrılan miktar: 6 (paketlenmiş)
}
```

10.8 Yapılarda Bit Alanları

Bit alanları (bit field), boyutu azaltmak için C ve C++ dili yapılarını (struct) sıkıca paketler. Yapı üyeleri için bit sayısı verilir ve derleyici bit düzeyinde alanları uyarlar. Bu nedenle bit alan üyesinin adresini alınamaz! `sizeof()` kullanımına da izin verilmez!

```
struct Date
{
        unsigned int Year : 13; // 2^13 = 8192, Yıl değerini tutmak için yeterlidir
        unsigned int Month: 4;  // 2^4 = 16, Ay değerini tutmak için yeterlidir.
        unsigned int Day:   5;  // 2^5 = 32, Gün değerini tutmak için yeterlidir
};
```

Tüm yapı sadece 22 bit kullanıyor ve normal derleyici ayarlarıyla bu yapının boyutu 4 bayt olur. Kullanımı oldukça basittir. Sadece değişkeni bildirimi yapılır ve sıradan bir yapı gibi kullanılır;

```
#include <iostream>
struct Date
{
        unsigned int Year : 13; // 2^13 = 8192, Yıl değerini tutmak için yeterlidir
        unsigned int Month: 4;  // 2^4 = 16, Ay değerini tutmak için yeterlidir.
        unsigned int Day:   5;  // 2^5 = 32, Gün değerini tutmak için yeterlidir
};
int main() {
        Date d;
        d.Year = 2025;
        d.Month = 2;
        d.Day =  28;
        std::cout << "Year:" << d.Year << std::endl //2025
        << "Month:" << d.Month << std::endl //2
        <<  "Day:" << d.Day << std::endl; //28
        std::cout << sizeof d << std::endl; // 4
        //std::cout << sizeof d.Year << std::endl; HATA Olur!
}
```

10.9 Birlikler

Birlikler (union), yapılar (struct) gibi tanımlanır. Aralarındaki fark **yapı elemanlarının her birine ayrı bellek bölgesi ayrılırken, birlik üyelerinin her biri aynı bellek bölgesini paylaşırlar.** Birlik tanımına ilişkin sözde kod aşağıda verilmiştir;

```
union yapı-kimliği {
 veri-tipi yapı-elemanı-kimliği1;
 veri-tipi yapı-elemanı-kimliği1;
 ...
};
```

Birliğin bellek boyutu, elemanlarından en fazla bellek kaplayanı kadardır.

Aşağıda üyelerinin aynı bellek bölgesini paylaştığını gösteren program örneği bulunmaktadır;

```
#include <iostream>
#include <iomanip>
union Birlik { // C++'ta union isimlerini Büyük Harfle başlatmak yaygındır
        char baytlar[4];
        int tamsayi;
        char bayt;
        short int kisatamsayi;
};
int main() {
        Birlik b;
        // Boyut kontrolleri: birlik üyeleri kaç bayt?
        std::cout << "sizeof Birlik.baytlar: " << sizeof(b.baytlar) << std::endl;
        std::cout << "sizeof Birlik.bayt: " << sizeof(b.bayt) << std::endl;
        std::cout << "sizeof Birlik.tamsayi: " << sizeof(b.tamsayi) << std::endl;
        std::cout << "sizeof Birlik.kisatamsayi: " << sizeof(b.kisatamsayi) << std::endl;
        std::cout << "sizeof Birlik: " << sizeof(Birlik) << std::endl;
        b.tamsayi = 0x1B2C3D4F; // b'in tam sayı üyesi değişiyor. Bakalım diğer üyeler ne oldu?
        std::cout << "------------------------" << std::endl;
        std::cout << "b.tamsayi: 0x" << std::hex << std::setw(8) << std::setfill('0') <<
        ↪  b.tamsayi << std::dec << std::endl;
```

```
        // char türlerini hex olarak yazdırmak için (int) cast'i gereklidir, aksi takdirde
         ↪ karakter olarak yazdırılırlar.
        std::cout << "b.baytlar: " << std::hex << std::setw(2) << std::setfill('0') <<
         ↪ (int)(unsigned char)b.baytlar[0] << "-" << std::hex << std::setw(2) <<
         ↪ std::setfill('0') << (int)(unsigned char)b.baytlar[1] << "-" << std::hex <<
         ↪ std::setw(2) << std::setfill('0') << (int)(unsigned char)b.baytlar[2] << "-" <<
         ↪ std::hex << std::setw(2) << std::setfill('0') << (int)(unsigned char)b.baytlar[3] <<
         ↪ std::dec << std::endl;
        std::cout << "b.bayt: 0x" << std::hex << std::setw(2) << std::setfill('0') <<
         ↪ (int)(unsigned char)b.bayt << std::dec << std::endl;
        std::cout << "b.kisatamsayi: 0x" << std::hex << std::setw(4) << std::setfill('0') <<
         ↪ b.kisatamsayi << std::dec << std::endl;
        std::cout << "------------------------" << std::endl;
        // Üyelerden biri değiştirildiğinde diğer tüm üyelerin içeriği değişir!
        b.bayt = 0x11;
        std::cout << "Değiştikten Sonra: b.tamsayi: 0x" << std::hex << std::setw(8) <<
         ↪ std::setfill('0') << b.tamsayi << std::dec << std::endl;
}
/*Program Çıktısı:
sizeof Birlik.baytlar: 4
sizeof Birlik.bayt: 1
sizeof Birlik.tamsayi: 4
sizeof Birlik.kisatamsayi: 2
sizeof Birlik: 4
------------------------
b.tamsayi: 0x1b2c3d4f
b.baytlar: 4f-3d-2c-1b
b.bayt: 0x4f
b.kisatamsayi: 0x3d4f
------------------------
Değiştikten Sonra: b.tamsayi: 0x1b2c3d11
*/
```

10.10 İsim Uzayları

İsim uzayları (namespace), değişkenleri (variable), sabitler (constexpr), fonksiyonları (function), yapıları (struct), birlikleri (union) ve ileride göreceğimiz sınıfları (class) bir isim altında toplayabileceğimiz bir tanımlamadır. Birden fazla kütüphane kullanırken isim çakışmalarını önlemek için kullanılır. Böylece aynı kimlikli sınıf, değişken ve fonksiyon birden fazla isim uzayında bulunabilir.

Buna bir örnek verelim. `fonk()` kimlikli bir fonksiyonu olan bir kod yazıyor olabilirsiniz ve aynı fonksiyon aynı kimlikle başka bir başlıkta (header) da mevcut olabilir. Bu durumda derleyicinin, kodunuz içinde hangi `fonk()` fonksiyona atıfta bulunduğunuzu bilmesinin bir yolu yoktur. İsim uzayları, bu zorluğun üstesinden gelmek için tasarlanmıştır ve farklı kütüphanelerde bulunan aynı kimliğe sahip benzer fonksiyonları, sınıfları, değişkenleri vb. ayırt etmek için ek bilgi olarak kullanılır. İsim uzaylarına en iyi örnek, C++ standart kütüphanesidir (`std`). Bu nedenle, C++ programı yazarken her seferinde `std::` yazmamak için kodun başında `using namespace std;` yönergesini ekleriz;

§10.10.1 İsim Uzayı Tanımlama

Bir isim uzayının tanımı, aşağıdaki gibi `namespace` anahtar sözcüğüyle tanımlanır;

```
namespace  isim-uzayı-kimliği
{
 // kod;
 // -değişken tanımlama (int a,b;)
 // -fonksiyon tanımlama (void topla();)
 // -yapı tanımlama (struct Yapi{ /* ... ?/ };)
 // -birlik tanımlama (union Yapi{ /* ... ?/ };)
 // -sınıf tanımlama (class Kisi{ /* ... ?/ };)
}
```

Burada isim uzayına ilişkin bloğun noktalı virgül ile bitmediği gözden kaçırılmamalıdır. Çünkü **isim uzayına ait bloğun görevi tanımlamaları gruplamaktır.** İsim uzayının içindeki bir öğeye erişmek için **kap-**

sam çözümleme işleci (scope resolution operator) kullanılır. Ayrıca `using namespace` yönergesini (directive) kullanarak isim uzaylarını ile kapsam çözümleme işlecinin önek olarak eklenmesini de önleyebilirsiniz.

```
#include <iostream>
using namespace std; /* std:: isim uzayını kullanmak için yönerge: Bu yönerge ile "std::cout"
 ↪ gibi talimatlar sadece "cout" şeklinde kullanılabilir. */
namespace birinci // birinci kimlikli isim uzayı
{
        int tamsayi=100; // Birinci isim uzayındaki tamsayi değişkeni
        void fonksiyon() {
                cout << "Birinci isim uzayındaki fonksiyon." << endl;
        }
}
namespace ikinci // ikinci kimlikli isim uzayı
{
        int tamsayi=200; // ikinci isim uzayındaki tamsayi değişkeni
        void fonksiyon() {
                cout << "İkinci isim uzayındaki fonksiyon." << endl;
        }
}
using namespace birinci;  /* birinci:: isim uzayını kullanmak için yönerge: Bu yönerge ile
 ↪ "birinci::tamsayi" gibi talimatlar sadece "tamsayi" şeklinde kullanılabilir. */
int main () {
        fonksiyon(); // birinci isim uzayındaki fonksiyon çağrılır.
        cout << tamsayi << endl; // birinci isim uzayındaki tamsayi kullanılır.
        ikinci::fonksiyon(); // kapsam çözümleme işleci ile ikinci isim uzayındaki fonksiyon
         ↪ çağrılır.
        cout << ikinci::tamsayi << endl; // kapsam çözümleme işleci ile ikinci isim uzayındaki
         ↪ tamsayi kullanılır.
}
/* Programın Çıktısı:
Birinci isim uzayındaki fonksiyon.
100
İkinci isim uzayındaki fonksiyon.
200
*/
```

İsim uzayları, aşağıdaki şekilde iç içe (nested) de yerleştirilebilir;

```
#include <iostream>
using namespace std; /* std:: isim uzayını kullanmak için yönerge: Bu yönerge ile "std::cout"
 ↪ gibi talimatlar sadece "cout" şeklinde kullanılabilir. */

namespace birinci // birinci kimlikli isim uzayı
{
        int tamsayi=100; // Birinci isim uzayındaki tamsayi değişkeni
        void fonksiyon() {
                cout << "Birinci isim uzayındaki fonksiyon." << endl;
        }
        namespace ikinci // ikinci kimlikli iç isim uzayı
        {
                int tamsayi=200; // ikinci isim uzayındaki tamsayi değişkeni
                void fonksiyon() {
                        cout << "İkinci isim uzayındaki fonksiyon." << endl;
                }
        }
}
using namespace birinci::ikinci; /* birinci::ikinci:: iç isim uzayını kullanmak için yönerge */
int main () {
        birinci::fonksiyon(); // birinci isim uzayındaki fonksiyon çağrılır.
        cout << birinci::tamsayi << endl;
        fonksiyon(); // birinci isim uzayı içindeki ikinci isim uzayındaki fonksiyon çağrılır.
        cout << tamsayi << endl;
}
```

```
/* Program çalıştığında:
Birinci isim uzayındaki fonksiyon.
100
İkinci isim uzayındaki fonksiyon.
200
*/
```

§10.10.2 Bağımlı Argüman Arama

Açık bir isim uzayı yönergesi olmadan bir fonksiyonu çağırırken, derleyici o işlevin prototipinden biri de o isim uzayında ise, o isim uzayı içindeki bir işlevi çağırmayı seçebilir. Buna **bağımlı argüman arama** ADL (argument dependent lookup) denir;

```
namespace Test
{
        int func(int i);
        struct Yapi {
                //...
        };
        int func2(const Yapi &data);
}
int main() {
        //...
        func(5); /*Hata alınır. Hangi İsim Uzayından olduğu belli değil. Çünkü "int func(int);"
         ↪ şeklide prototipi olan bir sürü fonksiyon vardır. */

        Test::Yapi data;
        func2(data); /* Hata alınmaz. Kullandığı argümana göre isim uzayı belli. Çünkü "int
         ↪ func(const Yapi &);" şeklide prototipi olan tek bir fonksiyon vardır. */
}
```

> En uygun kullanım, isim uzayı adı ile kapsam çözümleme işlecini birlikte kullanmaktır.

§10.10.3 İsim Uzaylarını Genişletme

İsim alanlarının kullanışlı bir özelliği de onları genişletebilmemizdir. Daha sonradan başka bir dosyada bile isim uzayına yeni üyeler ekleyebilirsiniz. Buna örnek olarak `<iostream>` ve `<string>` başlık dosyaları verilebilir. Birinci kaynak dosyamız aşağıdaki gibi olabilir;

```
//TEST1.CPP Dosyası:
namespace Test
{
        void fonk1() {}
}
//arada başka kodlar yer alabilir
namespace Test
{
        void fonk2() {}
}
```

İkinci dosyamız da aşağıdaki gibi olabilir;

```
//TEST2.CPP Dosyası:
        namespace Test
        {
                void fonk3() {}
        }
        //arada başka kodlar yer alabilir
        namespace Test
        {
                void fonk4() {}
        }
```

§10.10.4 using Anahtar Kelimesinin İsim Uzaylarında Kullanılması

Bir using yönergesi, namespace anahtar sözcüğüyle birleştirildiğinde o isim uzayını belirtmeden içindeki kullanırsınız.

```
namespace Test
{
        void func() {}
        void func2() {}
}
int main( {
        //...
        Test::func2(); //kapsam çözümleme işleci kullanarak istediğimiz üyeyi kullanabiliriz
        //...
        using namespace Test; /* Test isim uzayını bu yönerge ile projemize dahil ediyoruz. Artık
         ↪ kapsam çözümleme işlecini kullanmamıza gerek kalmaz. */
        func();
        func2();
        //...
}
```

Tüm isim uzayı yerine, isim uzayındaki seçili üyeleri projeye dahil etmek de mümkündür;

```
namespace Test
{
        void func() {}
        void func2() {}
}
int main( {
        //...
        Test::func2(); //kapsam çözümleme işleci kullanarak istediğimiz üyeyi kullanabiliriz
        //...
        using namespace Test::func; /* Test isim uzayının içindeki func fonksiyonunu bu yönerge
         ↪ ile projemize dahil ediyoruz. Artık bu fonksiyon için kapsam çözümleme işlecini
         ↪ kullanmamıza gerek kalmaz. */
        func(); //Başarılı bir şekilde projeye dahil edilmiştir.
        func2(); //Hata: Bu fonksiyon projeye dahil edilmemiştir.
        //...
}
```

Özellikle başlık (header) dosyalarında using namespace kullanmak çoğu durumda kötü görülür. Bu yapılırsa, isim uzayı dahil edilmiş başlığı içeren her dosya projeye aktarılır. Bu da çatışmalara yol açabilir. Birinci başlık dosyamız söyle olsun;

```
\\BIRINCI.H Başllık Dosyası:
namespace AAA
{
        class C;
}
```

İkinci başlık dosyamız da söyle olsun;

```
\\IKINCI.H Başllık Dosyası:
namespace BBB
{
        class C;
}
```

Birinci başlşık dosyasını kullanan bir başka başlık dosyamız aşağıdaki gibi olsun;

```
//UCUNCU.h Baslik Dosyasi:
using namespace AAA;
```

Son olarak bu başlık dosyalarını kullanan ana dosyamız şöyle olsun;

```
//MAIN.CPP Dosyasi:
#include "IKINCI.h"
#include "UCUNCU.h"
using namespace AAA;
```

```
int main() {
        //...
        C c; // Hata: BBB::C mi? AAA::C mi?.
        //...
}
```

10.11 Düşün ve Çöz

- **Düşün**: Yapı Dolgusu kavramını açıklayınız. İşlemcinin veriye erişim hızı ile bellek tasarrufu arasındaki denge bu noktada nasıl kurulur?
- **Düşün**: Birlik (union), neden aynı anda sadece bir üyenin değerini tutabilir? Bellek paylaşımı mantığını bir şema ile açıklayınız.
- **Düşün**: Bit alanları (bit field) kullanımı, düşük seviyeli donanım kontrolünde (register) neden vazgeçilmezdir?
- **Çöz**: Bir öğrencinin adını, numarasını ve üç dersinin notunu tutan bir `struct` tanımlayınız. Bu yapıdan bir dizi oluşturup sınıf ortalamasını hesaplayınız.
- **Çöz**: İç içe yapılar (nested structs) kullanarak bir "Şirket" yapısı ve bu yapının içinde "Departman" ve "Çalışan" bilgilerini tutan bir model oluşturunuz.
- **Çöz**: Bir `union` kullanarak, 4 baytlık bir `int` sayının bellekteki her bir baytına karakter göstericisiyle erişen bir program yazınız.
- **Çöz** İsim çakışması (name collision) sorunu büyük projelerde neden kritik bir probleme dönüşür? İsim uzaylarının (namespace) bu sorunu nasıl çözdüğünü somut bir örnekle gösterin.
- **Düşün**: `using namespace std;` direktifinin kullanımının neden büyük projelerde sorunlara yol açabileceğini açıklayın. Alternatif olarak ne kullanılması önerilir?
- **Düşün**: Bağımlı argüman arama ADL (argument dependent lookup) nedir? `operator<<` gibi işleçlerin neden doğru ad uzayında bulunabildiğini ADL mekanizmasıyla açıklayın.

11. Bölüm

Nesne Yönelimli Programlama

11.1 Nesne Yönelimli Programlama Paradigması

Yapısal programlamada talimatlar (statement) art arda koda yazılarak programlama yapılır ve programların neler yaptığı bu talimatlar izlenerek anlaşılabilir. Talimatların zincirin halkaları gibi birbirinin peşi sıra yazılarak yapılan programlamaya **emreden programlama** (imperative programming) paradigması adı verilir. Bu paradigmaya sahip diller, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler.

Emreden programlamanın bir örneği olan yapısal programlamada talimatlar yine art arda koda yazılarak programlama yapılır. 1980'li yıllara kadar yazılım ihtiyaçları yapısal programlama ile çözülmüştür. Kodlar büyüdükçe sorunlar artmış ve 2.8 başlığı altında anlatılan problemler ortaya çıkmıştır. Bu yıllarda Simula ve Smalltalk yaklaşımı yazılımcıların imdadına yetişmiştir. Bu dillerde **nesneler** (object) programın temel yapıtaşlarıdır. Nesneler; **durum** (state) ve **davranışlara** (behavior) sahiptir. Programlama, nesnelerin birbirlerine **ileti göndermesiyle** (message-passing) yapılır.

1979 senesinde bir Danimarkalı bilgisayar bilim adamı olan *Bjarne Stroustrup*, sonradan C++ olarak bilinecek olan sınıflarla C (C with classes) üzerinde çalışmaya başladı ve 1985 yılında ilk sürümünü yayınladı. C++ Programlama Dili;

- C Dilinden türetilmiştir.
- Düşük düzey programa yapılabilir
- İcra hızı yüksektir.
- C dilinden daha fazla kütüphaneye sahiptir.
- Nesne yönelimli programlamayı destekler.
- Diğer dillere göre daha etkin hafıza yönetimi sağlar
- **Standart Şablon Kütüphanesi** STL (Standart Template Library) ile jenerik (generic) programlamayı destekler.
- Birçok kavramı bir anda özümsemek gerektiğinden öğrenme eğrisi diktir.

Emreden paradigmanın aksine nesnelerin birbirlerine ileti göndermesi bakış açısıyla yapılan programlamaya **nesne yönelimli programlama** (object oriented programming) paradigması adı verilir.

11.2 Sınıf ve Nesne Nedir? Nasıl Tanımlanır?

Gerçek dünyamızda nesneleri sınıflamaya tabi tutarız. Örneğin bir birine benzeyen at, eşek ve zebra gibi hayvanları daha çocukluktan itibaren ayırt edebilir. İşte kafamızda oluşan bu mekanizmaya **sınıflayıcı** (classifier) adı verilir. Bu mekanizma ile sınıflanmış her bir türün gerçek dünyadaki her birine ise **örnek** (instance) adı verilir.

Nasıl ki dünyamızda evler bir mimari plan (blueprint) üzerinden inşa ediliyor, yazılımda kullanacağımız **nesneler** (object) de bir plan üzerinden inşa edilir. Nesnelere ilişkin verilerin tutulduğu **durumların** (state) ve nesnelerin göstereceği **davranışlarının** (behavior) tanımlandığı bu plana **sınıf** (class) adı verilir. Nesneler (object) yukarıda anlatılan gerçek dünya **örneklerine** (instance) karşılık gelir. Sınıflar ise **sınıflayıcıya** (classifier) karşılık gelir.

Bir mimari plandan birçok ev yapılabileceği gibi, **bir sınıftan da birçok nesne imal edilebilir.**

Gerçek dünyadaki nesneleri modelleyen **kullanıcı tanımlı sınıflar** (user defined class), aşağıdaki şekilde tanımlanır;

```
class sınıf-kimliği {
    veri-tipi alan-değişkeni-kimliği; // field variable: alan değişkeni
    veri-tipi davranış-kimliği();  // method: yöntem
} [örnek-değişkeni-1][, örnek-değişkeni-2];
```

İmal edilecek nesnelerin durumlar (state), sınıftaki alanlarla (field) tanımlanır. Nesnelerin göstereceği davranışlar da sınıf içinde fonksiyon gibi tanımlanır. **Aynı davranış, farklı yöntemlerle yerine getirilebile-**

ceğinden, davranışları tanımlayan bu fonksiyonlara **yöntem** (method) adı verilir. İşin doğası gereği davranışlar durumlardan etkilenirler.

> Özet olarak sınıflar, kodumuzda **imal edilecek nesnelere ilişkin ortak davranış ve durumları tasnif eden bir şablon bileşenidir**.

Aşağıda ortak davranış ve özelliklerin tanımlandığı bir `Ogrenci` sınıfı örneği verilmiştir;

```
1  class Ogrenci {
2  public:
3          unsigned yas; // alan(field)
4          char cinsiyet; // alan(field)
5          void yasiniSoyle() { // yöntem (method)
6                  std::cout << "Yaşım:" << yas << std::endl;
7          }
8          void cinsiyetSoyle() { // yöntem (method)
9                  if (cinsiyet=='E' || cinsiyet=='e')
10                         std::cout << "Cinsiyetim: ERKEK" << std::endl;
11                 else if (cinsiyet=='K' || cinsiyet=='k')
12                         std::cout << "Cinsiyetim: KADIN" << std::endl;
13                 else
14                         std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
15         }
16 } ilhan, methet;
```

Bu tanımlamayı şu şekilde açıklayabiliriz;

- 1. Satırda `class` anahtar kelimesi ile `Ogrenci` kimlikli bir sınıf tanımlanacağı belirtilmiştir. Sınıfın durum ve davranışları tarif etmek için blok açılmıştır. Kod bloğu olmadan sınıf tanımlanmaz!
- 2. Satırda `public:` etiketinden sonra tanımlanacak davranış ve özelliklerinin yer alacağı kısım başlayacağı belirtilmiştir. Bu kısımda sınıftan imal edilecek nesnelerin umuma açık (public) durum (state) ve davranışlarıdır (behavior). Bir sonraki başlıkta anlatılmıştır.
- 3. Satırda tanımlanan `yas` kimlikli alan (field), imal edilecek nesnelerin yaşına ait durum (state) verilerini tutar.
- 4. Satırda tanımlanan `cinsiyet` kimlikli alan (field), imal edilecek nesnelerin cinsiyetine ait durum (state) verilerini tutar.
- 5. Satırda başlayan ve 7. satırda biten `yasiniSoyle()` yöntemi (method), imal edilecek nesnelerin yaşını söyleme davranışını (behavior) tanımlıyor. Davranışlar 2.7 başlığında anlatılan yapısal programlamadaki fonksiyonlar gibi tanımlanır.
- 8. Satırda başlayan ve 15. satırda biten `cinsiyetSoyle()` yöntemi(method), imal edilecek nesnenelerin cinsiyetini söyleme davranışını(behavior) tanımlıyor.
- 16. Satırda tanımlanan sınıfa ait açılan blok kapatılıyor ve `Ogrenci` sınıfından `ilhan` ve `mehmet` nesneleri imal edildi. Son olarak tanımlama tamamlandığından noktalı virgül karakteri ile talimat tamamlanıyor. Nesne imal edilmese bile kullanılacaktı.

Tanımlanmış bir sınıftan değişken tanımlar gibi `sınıf-kimliği örnek-değişkeni;` şeklinde yeni nesneler imal edilebilir. İmal edilen nesnenin durum ve davranışlarına **nokta işleci** (dot operator) ile erişilir.

```
#include <iostream>
class Ogrenci {
        public:
        unsigned yas; // alan(field)
        char cinsiyet; // alan(field)
        void yasiniSoyle() { // yöntem (method)
                std::cout << "Yaşım:" << yas << std::endl;
        }
        void cinsiyetSoyle() { // yöntem (method)
                if (cinsiyet=='E' || cinsiyet=='e')
                std::cout << "Cinsiyetim: ERKEK" << std::endl;
                else if (cinsiyet=='K' || cinsiyet=='k')
                std::cout << "Cinsiyetim: KADIN" << std::endl;
                else
                std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
        }
```

```
} ilhan, methet;
int main() {
        std::cout << "ilhan nesnesi durum ve davranışları:" << std::endl;
        ilhan.yas=50; // Sınıf tanımı ile birlikte tanımlanmış "ilhan" nesnesinin "yas" özelliği
         ↪ değiştirildi.
        ilhan.yasiniSoyle(); /* "ilhan" nesnesine yasiniSoyle() iletisi gönderildi (message
         ↪ passing). */
        ilhan.cinsiyet='E';
        ilhan.cinsiyetSoyle();  /* "ilhan" nesnesine cinsiyetSoyle() iletisi gönderildi (message
         ↪ passing).*/

        Ogrenci damla; // Ogrenci sınıfından "damla" nesnesi imal edildi
        std::cout << "damla nesnesi durum ve davranışları:" << std::endl;
        damla.yas=30; // "damla" nesnesinin "yas" özelliği değiştirildi.
        damla.cinsiyet='K';
        damla.yasiniSoyle(); /* "damla" nesnesine yasiniSoyle() iletisi gönderildi (message
         ↪ passing). */
        damla.cinsiyetSoyle();  /* "damla" nesnesine cinsiyetSoyle() iletisi gönderildi (message
         ↪ passing). */
}
/* Program çalıştırıldığında:
ilhan nesnesi durum ve davranışları:
Yaşım:50
Cinsiyetim: ERKEK
damla nesnesi durum ve davranışları:
Yaşım:30
Cinsiyetim: KADIN
*/
```

Burada imal edilen nesnelere ilişkin durumlarının, davranışları etkilediği gözden kaçırılmamalıdır. İmal edilen `ilhan` nesnesinin `cinsiyetiniSoyle()` davranışı ve `damla` nesnesinin `cinsiyetiniSoyle()` davranışı birbirinden farklıdır. Çünkü `cinsiyet` durumu, her iki nesnede farklıdır.

> Nesne durumlarına ilişkin verileri tutacak değişkenler, sınıf tanımındaki alanlarla (field) tanımlanır. **Bu alanlar, her nesne için tahsis edilen bellek bölgesinde nesneye özel değişkenler haline gelir**.

Yukarıda verilen `Ogrenci` sınıfının yöntemlerini mutlaka sınıf içinde tanımlamak gerekmez. Kapsam çözümleme işleci (scope resolution operator) kullanılarak sınıf dışında da tanımlanabilir:

```
#include <iostream>
class Ogrenci {
        public:
        unsigned yas; // alan(field)
        char cinsiyet; // alan(field)
        void yasiniSoyle(); //Yöntem sınıf dışında tanımlanmalı!
        void cinsiyetSoyle(); //Yöntem sınıf dışında tanımlanmalı!
};
void Ogrenci::yasiniSoyle() {
        std::cout << "Yaşım:" << yas << std::endl;
}
void Ogrenci::cinsiyetSoyle() { // yöntem (method)
        if (cinsiyet=='E' || cinsiyet=='e')
                std::cout << "Cinsiyetim: ERKEK" << std::endl;
        else if (cinsiyet=='K' || cinsiyet=='k')
                std::cout << "Cinsiyetim: KADIN" << std::endl;
        else
                std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
}
int main() {
        Ogrenci ilhan;
        ilhan.yas=45;
        ilhan.cinsiyet='e';
        ilhan.yasiniSoyle();
```

```
        ilhan.cinsiyetSoyle();
}
/*Program Çıktısı:
Yaşım:45
Cinsiyetim: ERKEK
*/
```

11.3 Erişim Değiştiricileri

Sınıftan imal edilen nesnelerin üyelerine başka nesne ve programların nasıl erişeceğini **erişim değiştiricileri** (access modifier) ile belirlenir. Görünürlük (visibility) sıfatları olarak da adlandırılan bu değiştiriciler, sınıf üyelerinin sınıf içinden ve dışından erişimi belirleyen sıfatlardır.

- ◇ `public:` İmal edilecek nesnenin diğer nesneler tarafından ulaşılabilen yani **umuma açık** (public) davranış ve özellikleri sınıf içinde bu belirleyici altında toplanır.
- ◇ `private:` İmal edilen nesnelerin **mahrem/şahsi/kişisel** davranış ve özellikleri sınıf içinde bu belirleyici altında toplanır. Varsayılan (default) erişim değiştiricidir. Bir sınıfta hiçbir erişim değiştirici yoksa tanımlanan durum ve davranışlar **mahrem** (private) olarak kabul edilir.
- ◇ `protected:` Bir sınıftan imal edilen nesneler ile bu sınıftan miras alan sınıflardan imal edilecek nesnelerin ulaşabileceği **korumalı** (protected) davranış ve özellikleri bu belirleyici altında toplanır. Bu değiştirici ileride incelenecektir.

Aşağıda erişim belirleyicilerin uygulandığı basit bir `Kisi` sınıfı örnek verilmiştir. Bu sınıftan imal edilen nesnelerin `yas` durumu diğer tüm nesneler tarafından erişilip değiştirilebilir, ancak `sir` durumu ise hiçbir nesne tarafından erişilemez. `maas` özelliğine ise `Kisi` sınıfı ve bu sınıftan türemiş sınıflardan imal edilen nesneler tarafından bu özelliğe erişilebilir. İleride göreceğiz.

```
class Kisi {
public: /* imal edilecek nesnelerin umuma açık (public) davranış ve özellikleri:*/
        unsigned yas; /* "yas"  kimlikli alan(field), ogrencinin yaşına ait  durumlara(state)
         ↪  ilişkin verileri tutar. İmal edilecek nesnelerda umuma açık bir özelliktir. */
private:/* imal edilecek nesnelerin mahrem (private) davranış ve özellikleri:*/
        int sir; /* "sir"  kimlikli alan(field), ogrencinin sırrına ait durumlara(state) ilişkin
         ↪  verileri tutar. İmal edilecek nesnelerde mahrem bir özelliktir. diğer
         ↪  nesneler/programlar tarafından bu özelliğe erişilemez. */
protected: /* bu sınıftan ve türetilmiş sınıflardan imal edilecek nesnelerin korumalı (protected)
 ↪  davranış ve özellikler: */
        float maas;/* "maas"  kimlikli alan(field), ogrencinin maaşına ait durumlara(state)
         ↪  ilişkin verileri tutar. imal edilecek nesnelerde mahrem bir özelliktir. Ogrenci
         ↪  sınıfından türemiş sınıflar (ileride göreceğiz) dışında diğer nesneler/programlar
         ↪  tarafından bu özelliğe erişilemez.*/

};  //Hiç nesne tanımlanmamış! Noktalı virgül ile bitmiş!
int main() {
        Kisi ilhan;  //Kisi sınıfından "ilhan" nesnesi imal edildi
        ilhan.yas=50; // "ilhan" nesnesinin "yas" özelliği değiştirildi.
        //ilhan.sir=-1; //HATA: Ana program, ilhan nesnesinin sir özelliğine ulaşamaz.
        //ilhan.maas=10000.0; //HATA: Ana program, ilhan nesnesinin maas özelliğine ulaşamaz.
}
```

11.4 Nesne Yapıcısı

Bir sınıftan birçok nesne de imal edilebileceği anlatılmıştı. İmal edilen her nesne (object) sınıfın bir örneğidir (instance) ve bu nesnelerin varsayılan (default) olarak belirlenen durumlarla (state) imal edilmesi gerekiyorsa **nesne yapıcısı** (constructor) kullanılır. Yapıcıların görevi bir sınıftan nesneleri, varsayılan durumlara sahip olarak imal etmektir. **Nesnelere ait veriler, alanlarda (field) tutulurlar ve nesnenin durumları (state), nesnenin davranışını (behavior) etkiler!** Bu nedenle daha nesneler imal edilirken durumlarının belirlenen değerde olması istenir.

Yapıcıların kimliği, sınıf kimliğiyle aynıdır. Yapıcılar, yöntemlerin aksine geri değer döndürmezler. Bu nedenle bir yapıcı içerisinde `return` talimatı da kullanılmaz.

Aşağıda verilen örnekte `Ogrenci` sınıfından imal edilecek her nesnenin yaşı `6` ve cinsiyeti `'E'` olarak üretilecektir. Hiçbir parametresi olamayan bu yapıcıya **varsayılan yapıcı** (default constructor) adı verilir.

```
#include <iostream>
class Ogrenci {
public: /* Umuma açık (public) davranış ve özellikler: */
        Ogrenci() { // Ogrenci sınıfı yapıcısı: varsayılan yapıcı
                // Yapıcı içerisinde, imal edilecek nesnelerine ilk değer verilir.
                yas=6;
                cinsiyet='E';
                // Yapıcı içerisinde return talimatı bulunmaz!
        }
        unsigned yas; /* "yas"  kimlikli alan(field) */
        char cinsiyet; /* "cinsiyet"  kimlikli alan(field),*/
        void yasiniSoyle() { /* yasiniSoyle() yöntemi(method) */
                std::cout << "Yaşım:" << yas << std::endl;
        }
        void cinsiyetSoyle() { /* cinsiyetSoyle() yöntemi(method)*/
                if (cinsiyet=='E' || cinsiyet=='e')
                        std::cout << "Cinsiyetim: ERKEK" << std::endl;
                else if (cinsiyet=='K' || cinsiyet=='k')
                        std::cout << "Cinsiyetim: KADIN" << std::endl;
                else
                        std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
        }
};
int main() {
        Ogrenci ilhan; /* Ogrenci sınıfından "ilhan" nesnesi; Ogrenci() varsayılan yapıcısı
         ↪ kullanılarak yası 6 ve cinsiyeti E olarak imal edildi. */
        ilhan.yasiniSoyle(); /* "ilhan" nesnesine yasiniSoyle() iletisi gönderildi (message
         ↪ passing). */
        ilhan.cinsiyetSoyle();  /* "ilhan" nesnesine cinsiyetSoyle() iletisi gönderildi (message
         ↪ passing).*/
        std::cout << "----" << std::endl;
        ilhan.yas=50; // "ilhan" nesnesinin "yas" özelliği değiştirildi.
        ilhan.yasiniSoyle(); /* "ilhan" nesnesine yasiniSoyle() iletisi gönderildi (message
         ↪ passing). */
}
/*Program Çıktısı:
Yaşım:6
Cinsiyetim: ERKEK
----
Yaşım:50
*/
```

Nesne imal edilirken belirlen durumlarda nesne imal etmek istenebilir. Bu durumda **parametreli yapıcılar** (parameterized constructor) kullanılır;

```
#include <iostream>
class Ogrenci {
public:  /* Umuma açık (public) davranış ve özellikler: */
        Ogrenci() { // Ogrenci sınıfı varsayılan yapıcısı (default constructor)
                yas=6;
                cinsiyet='E';
        }
        Ogrenci(int pYas, char pCinsiyet): yas(pYas), cinsiyet(pCinsiyet) { // Ogrenci sınıfı
         ↪ parametreli yapıcısı (parameterized constructor)
        }
        /*
        // Parametreli yapıcı, alan değişkenlerine blok içinde ilk değer verilecek şekilde
         ↪ tanımlanabilirdi:
        Ogrenci(int pYas, char pCinsiyet) {
                yas=pYas;
                cinsiyet=pCinsiyet;
```

```
        }
        */
        unsigned yas; /* "yas"  kimlikli alan(field) */
        char cinsiyet; /* "cinsiyet"  kimlikli alan(field) */
        void yasiniSoyle() { /* yasiniSoyle() yöntemi(method) */
                std::cout << "Yaşım:" << yas << std::endl;
        }
        void cinsiyetSoyle() { /* cinsiyetSoyle() yöntemi(method) */
                if (cinsiyet=='E' || cinsiyet=='e')
                        std::cout << "Cinsiyetim: ERKEK" << std::endl;
                else if (cinsiyet=='K' || cinsiyet=='k')
                        std::cout << "Cinsiyetim: KADIN" << std::endl;
                else
                        std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
        }
};
int main() {
        Ogrenci ilhan; /* Ogrenci sınıfından "ilhan" nesnesi; Ogrenci() varsayılan yapıcısı
         ↪ kullanılarak yası 6 ve cinsiyeti E olarak imal edildi. */
        ilhan.yasiniSoyle();
        ilhan.cinsiyetSoyle();
        std::cout << "----" << std::endl;
        Ogrenci fatma(45,'K'); /* Ogrenci sınıfından "fatma" nesnesi; Ogrenci(int pYas, char
         ↪ pCinsiyet) parametreli yapıcısı kullanılarak yası 6 ve cinsiyeti E olarak imal
         ↪ edildi. */
        fatma.yasiniSoyle();
        fatma.cinsiyetSoyle();
}
/*Program Çıktısı:
Yaşım:6
Cinsiyetim: ERKEK
----
Yaşım:45
Cinsiyetim: KADIN
*/
```

Bazı durumlarda bir yapıcı bir başka yapıcıyı çağırabilir. Bu durumda **devredilen yapıcı** (delegating constructor) kullanılır.

```
#include <iostream>
class Ogrenci {
public:  /* Umuma açık (public) davranış ve özellikler: */
        Ogrenci() { // Ogrenci sınıfı varsayılan yapıcısı (default constructor)
                yas=6;
                cinsiyet='E';
        }
        Ogrenci(int pYas) { // Ogrenci sınıfı parametreli yapıcısı (parameterized constructor)
                yas=pYas;
        }
        Ogrenci(int pYas, char pCinsiyet): Ogrenci(pYas) { // Devredilen yapıcı (delegated
         ↪ constructor)
                cinsiyet=pCinsiyet;
        }
        unsigned yas; /* "yas"  kimlikli alan(field) */
        char cinsiyet; /* "cinsiyet"  kimlikli alan(field) */
        void yasiniSoyle() { /* yasiniSoyle() yöntemi(method) */
                std::cout << "Yaşım:" << yas << std::endl;
        }
        void cinsiyetSoyle() { /* cinsiyetSoyle() yöntemi(method) */
                if (cinsiyet=='E' || cinsiyet=='e')
                        std::cout << "Cinsiyetim: ERKEK" << std::endl;
                else if (cinsiyet=='K' || cinsiyet=='k')
                        std::cout << "Cinsiyetim: KADIN" << std::endl;
```

```
                else
                        std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
        }
};
int main() {
        Ogrenci ilhan; /* Ogrenci sınıfından "ilhan" nesnesi; Ogrenci() varsayılan yapıcısı
         ↪ kullanılarak yası 6 ve cinsiyeti E olarak imal edildi. */
        ilhan.yasiniSoyle();
        ilhan.cinsiyetSoyle();
        std::cout << "----" << std::endl;
        Ogrenci ayse(30); /* Ogrenci sınıfından "ayse" nesnesi; Ogrenci(int pYas)
         ↪ yetkilendirilmiş yapıcı kullanılarak yası 6 olarak imal edildi. */
        ayse.yasiniSoyle();
        ayse.cinsiyetSoyle();
        std::cout << "----" << std::endl;
        Ogrenci fatma(45,'K'); /* Ogrenci sınıfından "fatma" nesnesi; Ogrenci(int pYas, char
         ↪ pCinsiyet) parametreli yapıcısı kullanılarak yası 6 ve cinsiyeti E olarak imal
         ↪ edildi. */
        fatma.yasiniSoyle();
        fatma.cinsiyetSoyle();
}
/*Program Çıktısı:
Yaşım:6
Cinsiyetim: ERKEK
----
Yaşım:30
Cinsiyetim: SÖYLEMEK İSTEMİYORUM!
----
Yaşım:45
Cinsiyetim: KADIN
*/
```

11.5 Dinamik Nesne İmalatı

Değişkenleri çalışma anında üretip dinamik olarak kullanma 9.3 başlığında anlatılmıştı . Nesneleri de çalışma anında da yapıcılar (constructor) kullanarak imal ederiz ve buna **dinamik başlatma** (dynamic initialization) adı verilir. Dinamik başlatma aşağıdaki şekilde yapılır;

```
SınıfKimliği* imal-edilen-nesne-göstericisi = new SınıfKimliği(yapıcı-argümanları);
```

- Dinamik olarak **new işleci** (new operator) ile imal edilecek nesneye öbek bellekte (heap segment) yer tahsis edilir (memory allocation). Ardından nesnenin yapıcısı ile veri tutan alanlarına (field) ilk değer değerler için mantık çalıştırılır ve nesne göstericisi geri döner.
- Nesne göstericileri üzerinden sınıf üyelerine **dolaylı işleç** (indirection operator) (`->`) ile erişilir. Bu işleç, dinamik olmayan nesneler için kullanılan nokta işleci (dot operator) ile aynı önceliklidir.
- Dinamik olarak imal edilmiş nesne kullanıldıktan sonra **delete işleci** (delete operator) ile bellekten kaldırılabilir (deallocate memory).

Çeşitli yapıcılara sahip, dinamik olarak nesne imal edilen bir kod örneği aşağıda verilmiştir;

```
#include <iostream>
class Karmasik {
        public:
        float gercek, sanal;
        // 1. Varsayılan Yapıcı: Üye ilklendirme listesi kullanımı daha performanslıdır.
        Karmasik() : gercek(0.0f), sanal(0.0f) {
                std::cout << "Varsayilan (default) yapici cagrildi" << std::endl;
        }
        // 2. Parametreli Yapıcı
        Karmasik(float pGercek, float pSanal) : gercek(pGercek), sanal(pSanal) {
                std::cout << "Iki Parametreli yapici cagrildi" << std::endl;
        }
        // 3. Devredilen Yapıcı (Delegating Constructor)
```

```
        Karmasik(float pGercek) : Karmasik(pGercek, 0.0f) {
                std::cout << "Tek parametreli yapici cagrildi. (Iki parametreliye devredildi)" <<
                 ↪  std::endl;
        }
        // 'const' ekleyerek metodun nesne üzerinde değişiklik yapmayacağını garanti ediyoruz.
        void yaz() const {
                // Kodundaki +"i" kısmındaki fazla '+' işaretini kaldırdım.
                std::cout << gercek << " + " << sanal << "i";
        }
};
int main() {
        std::cout << "--- Nesne 1 ---" << std::endl;
        Karmasik* k1 = new Karmasik(); /* Dinamik olarak bir Karmasik nesnesi imal edildi.
         ↪  Nesneye yer tahsis edildikten sonra varsayılan yapıcı Karmasik() ile başlatma mantığı
         ↪  çağrıldı */
        k1->yaz();
        std::cout << "\n--- Nesne 2 ---" << std::endl;
        Karmasik* k2 = new Karmasik(2.0f, 3.0f);  /* Yine Dinamik olarak bir Karmasik nesnesi
         ↪  imal edildi. Nesneye yer tahsis edildikten sonra Karmasik(float pGercek, float
         ↪  pSanal) yapıcısı ile başlatma mantığı çağrıldı */
        k2->yaz();
        std::cout << "\n--- Nesne 3 ---" << std::endl;
        Karmasik* k3 = new Karmasik(4.0f); /* Bir başka Dinamik olarak bir Karmasik nesnesi imal
         ↪  edildi. Nesneye yer tahsis edildikten sonra Karmasik(float pGercek) yapıcısı ile
         ↪  başlatma mantığı çağrıldı */
        k3->yaz();
        // Bellek temizliği
        delete k1;
        delete k2;
        delete k3;
        // Modern C++'ta new/delete yerine genellikle stack (yığın) nesneleri tercih edilir.
        // Bunun yerine daha güvenli olan ileride anlatacağımız Akıllı göstericiler (smart
         ↪  pointers) kullanılmalıdır.
}
/* Program Çıkısı:
--- Nesne 1 ---
Varsayilan (default) yapici cagrildi
0 + 0i
--- Nesne 2 ---
Iki Parametreli yapici cagrildi
2 + 3i
--- Nesne 3 ---
Iki Parametreli yapici cagrildi
Tek parametreli yapici cagrildi. (Iki parametreliye devredildi)
4 + 0i
*/
```

11.6 this Göstericisi

C++'da her nesne, `this` adı verilen önemli bir gösterici aracılığıyla kendi adresine erişebilir. `this` Göstericisi (this pointer), tüm üye yöntemler için örtük bir değişkendir. Aşağıda karmaşık sayılar örneği verilmiştir;

```
#include <iostream>
class Karmasik {
        float gercek, sanal;
        public:
        Karmasik(float pGercek=0.0, float pSanal=0.0) {
                this->gercek=pGercek;
                this->sanal=pSanal;
        }
        void yaz() { //yaz yöntemi
                std::cout<< this->gercek << "+" << this-> sanal << +"i" << std::endl;
        }
```

```
};
int main(void) {
        Karmasik c1(1.0,2.0);
        c1.yaz();
        Karmasik c2(3.0,5.0);
        c2.yaz();
}
```

11.7 Soyut, Somut ve Bilgi Gizleme

Soyut (abstract) kavramı kelime itibariyle detay içermeyen, özet anlamına gelir. Soyutlama (abstraction), bir nesnenin sadece "ne yaptığını" ortaya koyup, "nasıl yaptığını" gizleme sürecidir. Kullanıcıya sadece gerekli olan temel özellikleri sunar, karmaşık detayları arka planda tutar. Örnek olarak bir `Araba` sınıfı düşünün. Kullanıcı bu sınıftan imal edilmiş bir nesneden `gazaBas()` davranışını göstermesi için ileti gönderebilir. Motorun içindeki yanma odalarında neler olduğu (detay), kullanıcının bilmesi gereken bir şey değildir (soyutlama).

Somut (concrete) ise en ince detayına kadar bilinen, elle tutulur anlamına gelir. **Somutlama** (concretization), soyut olarak tanımlanmış bir yapının gerçek hayattaki karşılığının oluşturulması veya ileride göreceğimiz bir arayüzün (interface) içinin doldurulmasıdır. Yani kodlayarak uygulamanın hayata geçirilmesidir (implementation).

Bilgi gizleme (information hiding), bir nesnenin iç durumuna (verilerine) dışarıdan doğrudan erişilmesini engellemek ve bu verileri koruma altına almaktır. Genellikle ileride göreceğimiz kapsülleme (encapsulation) ile yapılır.

Bir sınıfta **soyutlama** (abstraction) iki türlü yapılır; Birincisi **veri soyutlaması** (data abstraction), nesnenin durumları üzerinde yapılan soyutlamadır. İkincisi ise **kontrol soyutlaması** (control abstraction), nesnenin davranışları üzerinde yapılan soyutlamadır.

11.8 Kapsülleme

Yukarıda verilen `Ogrenci` örneğinden devam edecek olursak imal edilecek nesnenin `yas` durumuna bir başka nesne veya program, `-1` gibi bir değer atayabilir. Bu durumda `yasiniSoyle()` davranışında istenmeyen bir çıktı verilmesine sebep olur. Bunu gidermek için veri soyutlaması (data abstraction) yapılması gerekir. Bunun için `yas` alanına bir değer koymayı ve `yas` alanından bir değer almayı kontrol altına almalıyız. Benzer durum `cinsiyet` alanı için de geçerlidir.

`Ogrenci` sınıfında **veri soyutlamasını** yapmak için `yas` ve `cinsiyet` alanlarını **mahrem** (private) olarak belirleyip bu alanlara değer atama ve değer okuma işlevlerini yerine getirecek `get` ve `set` yöntemlerini umuma açık (public) olarak tanımlamalıyız;

```
#include <iostream>
class Ogrenci {
private: // mahrem (private) davranış ve özellikler:
        unsigned yas; /* "yas"  kimlikli alan(field) */
        char cinsiyet; /* "cinsiyet"  kimlikli alan(field) */

public: // umuma açık (public) davranış ve özellikler:
        Ogrenci() { // Ogrenci yapıcısı
                yas=6;
                cinsiyet='E';
        }
        //VERi SOYUTLAMASI İÇİN YÖNTEMLER:
        void setYas(unsigned yeniYas) {
                if (yeniYas <5)
                std::cout << "Öğrenci 5 Yaşından Küçük Olamaz." << std::endl;
                else
                yas=yeniYas;
        }
        unsigned getYas() {
                return yas;
        }
        void setCinsiyet(char yeniCinsiyet) {
                if (yeniCinsiyet == 'E' ||
                yeniCinsiyet == 'E' ||
                yeniCinsiyet == 'K' ||
```

```
                yeniCinsiyet == 'k' ||
                yeniCinsiyet == 'B' ||
                yeniCinsiyet == 'b')
                cinsiyet=yeniCinsiyet;
                else
                std::cout << "Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz." << std::endl;
        }
        char getCinsiyet() {
                return cinsiyet;
        }
        //DAVRANIŞLAR:
        void yasiniSoyle() { /* yasiniSoyle() yöntemi(method) */
                std::cout << "Yaşım:" << yas << std::endl;
        }
        void cinsiyetSoyle() { /* cinsiyetSoyle() yöntemi(method) */
                if (cinsiyet=='E' || cinsiyet=='e')
                        std::cout << "Cinsiyetim: ERKEK" << std::endl;
                else if (cinsiyet=='K' || cinsiyet=='k')
                        std::cout << "Cinsiyetim: KADIN" << std::endl;
                else
                        std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
        }
};
int main() {
        Ogrenci ilhan; /* Ogrenci sınıfından imal edilen "ilhan" nesnesi*/
        ilhan.yasiniSoyle(); /* "ilhan" nesnesine yasiniSoyle() iletisi gönderildi (message
         ↪ passing). */
        ilhan.cinsiyetSoyle();  /* "ilhan" nesnesine cinsiyetSoyle() iletisi gönderildi (message
         ↪ passing).*/

        //ilhan.yas=12; // Bu talimat çalışmaz. Çünkü yaz alanı private

        ilhan.setYas(0); // "ilhan" nesnesinin "yas" özelliği değiştirildi.
        ilhan.yasiniSoyle(); /* "ilhan" nesnesine yasiniSoyle() iletisi gönderildi (message
         ↪ passing). */
        ilhan.setCinsiyet('H');
        ilhan.cinsiyetSoyle();
}
/*Program Çıktısı:
Yaşım:6
Cinsiyetim: ERKEK
Öğrenci 5 Yaşından Küçük Olamaz.
Yaşım:6
Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz.
Cinsiyetim: ERKEK
*/
```

İşte **bir durumun (state) veri soyutlaması yapılmış haline özellik (property) adı veririz.** Çünkü artık duruma dışarıdan doğrudan müdahale edilemez. Nesnenin durumu üzerinde kontrol hakları vardır.

Benzer şekilde davranışlar (behavior) da nesne dışına kontrollü olarak açılabilir. Bu durumda da **kontrol soyutlaması** (control abstraction) yapılır. Aşağıda faktöriyel hesaplayan bir hesap makinesinin ekranı yenileme mahrem davranışı buna örnek verilebilir;

```
#include <iostream>
class HesapMakinesi {
private: // mahrem (private) davranış ve özellikler:
        int hesaplanan;
        void ekraniYenile() {
                std::cout << "Hesaplanan:" << hesaplanan << std::endl;
        }
public: // umuma açık (public) davranış ve özellikler:
```

```
        HesapMakinesi() { //Yapıcı
                hesaplanan=0;
                ekraniYenile();
        }
        void setHesapMakinesi(int pSayi) {
                hesaplanan=pSayi;
                ekraniYenile();
        }
        int getHesapMakinesi() {
                return hesaplanan;
        }
        void makineyiSifirla() {
                hesaplanan=0;
                ekraniYenile();
        }
        void FaktoriyelHesapla() {
                unsigned temp=1;
                for (int sayac = hesaplanan; sayac > 0; sayac--)
                temp =  temp*sayac;
                hesaplanan=temp;
                ekraniYenile();
        }
};
int main ()
{
        HesapMakinesi hesaplayici; // hesaplayici nesnesi imal edildi
        hesaplayici.setHesapMakinesi(4);
        hesaplayici.FaktoriyelHesapla();
        hesaplayici.setHesapMakinesi(6);
        hesaplayici.FaktoriyelHesapla();
        hesaplayici.makineyiSifirla();
}
/*Program Çıktısı:
Hesaplanan:0
Hesaplanan:4
Hesaplanan:24
Hesaplanan:6
Hesaplanan:720
Hesaplanan:0
*/
```

Bir sınıf üzerinde veri soyutlaması (data abstraction) yapılarak veriler üzerine kılıf (capsule) çekilir, hem de kontrol soyutlaması (control abstraction) ile davranışların üzerine bir kılıf (capsule) çekilir. **Bir sınıf üzerinde her iki soyutlamanın yapılmasına kapsülleme (encapsulation) adı verilir.**

Soyutlamaların gerekli olup olmadığına programcı karar verir. Kapsülleme işlemi örneklerde görüldüğü üzere erişim değiştiricilerle (access modifier) yapılır. Böylece imal edilen nesnelerin verileri ve davranışları güvenli bir şekilde dış dünyaya açılmış, istenmeyen detaylar dış dünyadan gizlenir (information hiding). Böylece, sınıf ve nesne gibi bileşenlerin içeriğini bilmeden soyut (abstract) olarak kullanmak mümkün olmaktadır. Programcı, tasarladığı sınıfın soyut olarak kullanılacağını düşünerek sınıfları tasarlamalıdır.

Kapsülleme (encapsulation) tekniğini uygulamak, **kullanımı kolay** sınıf (class), bileşen (component), kütüphane (library) ve çerçeve yapılar (framework) elde etmemizi sağlar;

- n adet durum/veri (state/data) ve n adet davranış (behavior) kapsülleme işlemine tabi tutulursa sınıf,
- n adet sınıf kapsülleme işlemine tabi tutulursa bileşen,
- n adet bileşen kapsülleme işlemine tabi tutulursa kütüphane,
- n adet kütüphane kapsülleme işlemine tabi tutulursa çerçeve yapılar oluşur.

Sonuç olarak kapsülleme, projemizde proje süresini ve maliyetini kısaltma üstünlüğünü verir.

11.9 Kalıtım

Öğrenci, öğretmen ve müdürün olduğu bir sistemi örnek alarak ele alalım. Bunlardan her birinden nesne imal etmek için her birine ayrı ayrı sınıf tanımlamak gerekir. Öğrenci için bir sınıf;

```
class Ogrenci {
private:
        unsigned yas;
        char cinsiyet;
        unsigned ogrenciNo;
public:
        Ogrenci() {
                yas=18u;
                cinsiyet='E';
                ogrenciNo=0u;
        }
        void setYas(unsigned yeniYas) {
                if (yeniYas <18)
                        std::cout << "Yaş 18 den Küçük Olamaz." << std::endl;
                else
                        yas=yeniYas;
        }
        unsigned getYas() {
                return yas;
        }
        void setCinsiyet(char yeniCinsiyet) {
                if (yeniCinsiyet == 'E' || yeniCinsiyet == 'E' || yeniCinsiyet == 'K' ||
                 ↪  yeniCinsiyet == 'k' || yeniCinsiyet == 'B' || yeniCinsiyet == 'b')
                        cinsiyet=yeniCinsiyet;
                else
                        std::cout << "Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz." <<
                         ↪  std::endl;
        }
        char getCinsiyet() {
                return cinsiyet;
        }
        void yasiniSoyle() {
                std::cout << "Yaşım:" << yas << std::endl;
        }
        void cinsiyetSoyle() {
                if (cinsiyet=='E' || cinsiyet=='e')
                        std::cout << "Cinsiyetim: ERKEK" << std::endl;
                else if (cinsiyet=='K' || cinsiyet=='k')
                        std::cout << "Cinsiyetim: KADIN" << std::endl;
                else
                        std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
        }
        void dersCalis() {
                std::cout << "Ders Çalışıyorum..." << std::endl;
        }
};
```

Öğretmen için ayrı bir sınıf;

```
class Ogretmen {
private:
        unsigned yas;
        char cinsiyet;
        unsigned verdigiDersKodu;
public:
        Ogretmen() {
                yas=18u;
                cinsiyet='E';
                verdigiDersKodu=0u;
```

```
        }
        void setYas(unsigned yeniYas) {
                if (yeniYas <18)
                        std::cout << "Yaş 18 den Küçük Olamaz." << std::endl;
                else
                        yas=yeniYas;
        }
        unsigned getYas() {
                return yas;
        }
        void setCinsiyet(char yeniCinsiyet) {
                if (yeniCinsiyet == 'E' || yeniCinsiyet == 'E' || yeniCinsiyet == 'K' ||
                 ↪  yeniCinsiyet == 'k' || yeniCinsiyet == 'B' || yeniCinsiyet == 'b')
                        cinsiyet=yeniCinsiyet;
                else
                        std::cout << "Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz." <<
                         ↪  std::endl;
        }
        char getCinsiyet() {
                return cinsiyet;
        }
        void yasiniSoyle() {
                std::cout << "Yaşım:" << yas << std::endl;
        }
        void cinsiyetSoyle() {
                if (cinsiyet=='E' || cinsiyet=='e')
                        std::cout << "Cinsiyetim: ERKEK" << std::endl;
                else if (cinsiyet=='K' || cinsiyet=='k')
                        std::cout << "Cinsiyetim: KADIN" << std::endl;
                else
                        std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
        }
        void dersVer() {
                std::cout << "Ders Veriyorum..." << std::endl;
        }
};
```

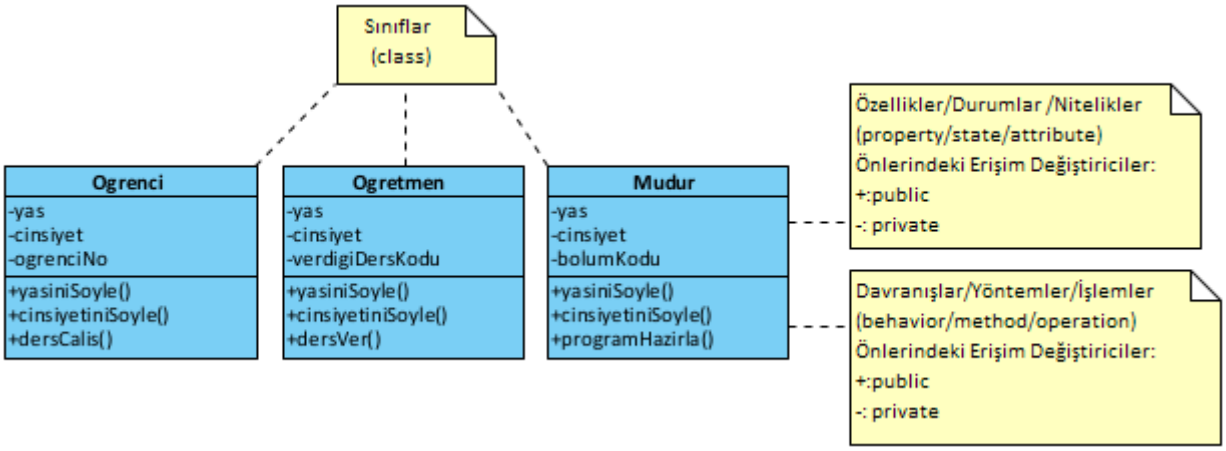
Müdür için ayrı bir sınıf;

```
class Mudur {
private:
        unsigned yas;
        char cinsiyet;
        unsigned muduruOlduguBolumKodu;
public:
        Mudur() {
                yas=18u;
                cinsiyet='E';
                muduruOlduguBolumKodu=0u;
        }
        void setYas(unsigned yeniYas) {
                if (yeniYas <18)
                        std::cout << "Yaş 18 den Küçük Olamaz." << std::endl;
                else
                        yas=yeniYas;
        }
        unsigned getYas() {
                return yas;
        }
        void setCinsiyet(char yeniCinsiyet) {
                if (yeniCinsiyet == 'E' || yeniCinsiyet == 'E' || yeniCinsiyet == 'K' ||
                 ↪  yeniCinsiyet == 'k' || yeniCinsiyet == 'B' || yeniCinsiyet == 'b')
                        cinsiyet=yeniCinsiyet;
```

```
                else
                        std::cout << "Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz." <<
                        ↪  std::endl;
        }
        char getCinsiyet() {
                return cinsiyet;
        }
        void yasiniSoyle() {
                std::cout << "Yaşım:" << yas << std::endl;
        }
        void cinsiyetSoyle() {
                if (cinsiyet=='E' || cinsiyet=='e')
                        std::cout << "Cinsiyetim: ERKEK" << std::endl;
                else if (cinsiyet=='K' || cinsiyet=='k')
                        std::cout << "Cinsiyetim: KADIN" << std::endl;
                else
                        std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
        }
        void dersProgramiHazırla() {
                std::cout << "Ders Programı Hazırlıyorum..." << std::endl;
        }
};
```

Bu sınıfların her birinde `yas` ve `cinsiyet` gibi ortak özellikler (property) veya her sınıfta aynı kodu içerecek şekilde `yasiniSoyle()` ve `cinsiyetSoyle()` gibi ortak davranışlar (behavior) bulunur. Her sınıf için bunlar tekrar tekrar tanımlanırlar. Bunun gibi ortak özellikler ve davranışlar tek bir sınıf altında toplanabilir. Böylece proje zamanı kısalır ve daha yönetilebilir bir kod elde edilir.

Yazılım analizi yapılırken takım içerisinde iletişimi kolaylaştırmak için aşağıdaki **sınıf diyagramları** (class diagram) kullanılır. Bunlar UML (Unified Modelling Language) dilinin bir parçasıdır. Sonraki bölümde anlatılacaktır. Aşağıda yukarıda sınıfları kodlanan okul projesinin sınıf diyagramları verilmiştir;



Şekil 11.1: Okul Projesi Sınıfları UML Diyagramı

Ortak özellik ve davranışların bir sınıfta toplanarak bu sınıftan miras alınmasına **kalıtım** (inheritance) adı verilir. Yukarıdaki örnekte ortak özellik ve davranışlar `Kisi` sınıfına toplanarak aşağıdaki şekilde kod yeniden düzenlenebilir.

```
class Kisi {
private:
        unsigned yas;
        char cinsiyet;
public:
        Kisi() {
                yas=18u;
                cinsiyet='E';
        }
        void setYas(unsigned yeniYas) {
                if (yeniYas <18)
```

```
                std::cout << "Yaş 18 den Küçük Olamaz." << std::endl;
                else
                yas=yeniYas;
        }
        unsigned getYas() {
                return yas;
        }
        void setCinsiyet(char yeniCinsiyet) {
                if (yeniCinsiyet == 'E' || yeniCinsiyet == 'E' || yeniCinsiyet == 'K' ||
                ↪  yeniCinsiyet == 'k' || yeniCinsiyet == 'B' || yeniCinsiyet == 'b')
                cinsiyet=yeniCinsiyet;
                else
                std::cout << "Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz." << std::endl;
        }
        char getCinsiyet() {
                return cinsiyet;
        }
        void yasiniSoyle() {
                cout << "Yaşım:" << yas << endl;
        }
        void cinsiyetSoyle() {
                if (cinsiyet=='E' || cinsiyet=='e')
                std::cout << "Cinsiyetim: ERKEK" << std::endl;
                else if (cinsiyet=='K' || cinsiyet=='k')
                std::cout << "Cinsiyetim: KADIN" << std::endl;
                else
                std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
        }
};
```

Yukarıda verilen ve `Kişi` olarak tanımlanan **genel sınıf** (general class), **taban sınıf** (base class) veya **ebeveyn sınıf** (parent class) olarak adlandırılır. Daha sonra tanımlanacak `Ogrenci`, `Ogretmen` ve `Mudur` sınıflarının ortak özellik ve davranışlarını barındırır.

Aşağıda `Kisi` taban sınıfından türemiş (derived) `Ogrenci`, `Ogretmen` ve `Mudur` sınıfları verilmiştir. Bu sınıflar genel sınıf olan Kişi sınıfının yanında kendine özel özellik ve davranışlara da sahiptir. **Özel sınıf** (private class), **türemiş sınıf** (derived class) veya **çocuk sınıf** (child class) olarak adlandırılan bu sınıflar, mirasçı olup, kendilerinden imal edilmiş nesneler; hem taban sınıfın özellik ve davranışlarını, hem de kendi özellik ve davranışlarını sergilerler.

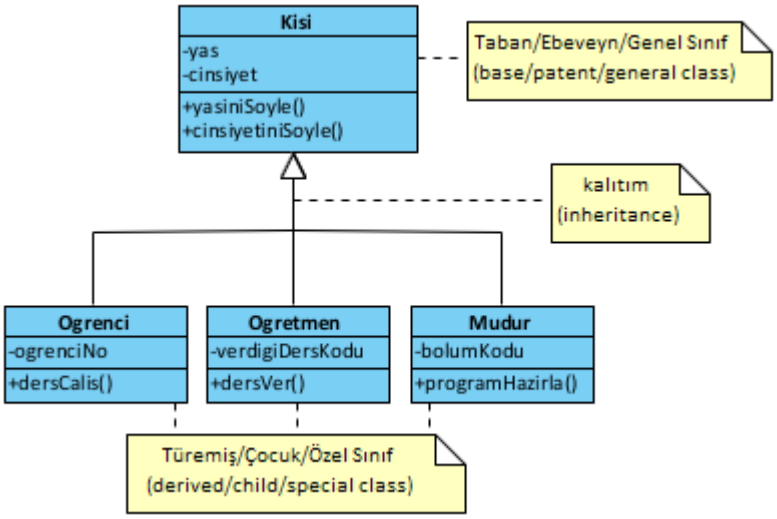
```
class Ogrenci: public Kisi { //Kisi sınıfından türemiş Ogrenci sınıf
private:
        unsigned ogrenciNo;
public:
        Ogrenci() {
                ogrenciNo=0u;
        }
        void dersCalis() {
                std::cout << "Ders Çalışıyorum..." << std::endl;
        }
};
class Ogretmen:public Kisi { //Kisi sınıfından türemiş Ogretmen sınıf
private:
        unsigned verdigiDersKodu;
public:
        Ogretmen() {
                verdigiDersKodu=0u;
        }
        void dersVer() {
                std::cout << "Ders Veriyorum..." << std::endl;
        }
};
class Mudur:public Kisi { //Kisi sınıfından türemiş Mudur sınıf
private:
```

```
        unsigned muduruOlduguBolumKodu;
public:
        Mudur() {
                muduruOlduguBolumKodu=0u;
        }
        void dersProgramiHazırla() {
                std::cout << "Ders Programı Hazırlıyorum..." << std::endl;
        }
};
```

Programlamanın tamamlandığı ana fonksiyonda bu üç sınıftan da nesne imal edilmiş ve çeşitli davranışlar göstermesi için kendilerine ileti gönderilmiştir (message-passing).

```
#include <iostream>
//Sınıflar Buraya gelecek...
int main() {
        Ogrenci ilhan;
        Ogretmen mehmet;
        Mudur abdullah;

        ilhan.yasiniSoyle();
        ilhan.cinsiyetSoyle();
        ilhan.dersCalis();

        mehmet.yasiniSoyle();
        mehmet.cinsiyetSoyle();
        mehmet.dersVer();

        abdullah.yasiniSoyle();
        abdullah.cinsiyetSoyle();
        abdullah.dersProgramiHazırla();
}
```

Kalıtım uygulanmış haliyle sınıf diyagramları aşağıda verilmiştir;



Şekil 11.2: Kalıtım Uygulanmış Okul Projesi Sınıfları UML Diyagramı

Örneğimizden gidecek olursak, `Kisi` sınıfında yapılacak bir özellik veya davranış eklemesi anında bu sınıftan türemiş sınıflarda yapılmış olur. Türemiş sınıflarda bununla ilgili değişiklik gerekmez.

Değişim yönetimini kolayca yapma üstünlüğünü veren kalıtımın (inheritance) iki önemli özelliği vardır;

1. **Paylaşılan bir ortak kod** (shared code) vardır.
2. Paylaşılan **kodda yapılan değişiklik anında alt sınıflara yansır** (snap change).

Türemiş sınıflar, taban sınıftan miras alırken üç farklı şekilde alabilirler;

1. **Umuma açık (public) kalıtım**: Taban sınıfın umuma açık üyelerini türetilmiş sınıfta umuma açık hale getirir ve taban sınıfın korumalı (protected) üyeleri de türetilmiş sınıfta korumalı olarak kalır.
2. **Mahrem (private) kalıtım**: Taban sınıfın umuma açık ve korumalı üyelerini türetilmiş sınıfta mahrem hale getirir.
3. **Korumalı (protected) kalıtım**: Taban sınıfın umuma açık ve korumalı üyelerini türetilmiş sınıfta korumalı hale getirir.

Aşağıda bu durumu açıklayan sınıf örneği verilmiştir;

```
class Base { //taban sınıf
public:
        /* Burada tanımlanan üyeler umuma açıktır. Yani bu üyelere sınıf dışı ve içinden
         ↪  erişilebilir. */
        int x;
protected:
        /* burada tanımlı üyelere türeyen sınıftan türemiş sınıflardan erişilir. Bir başka
         ↪  deyişle miras alan sınıflar tarafından burada tanımlanan üyelere erişilir. */
        int y;
private:
        /* burada tanımlı üyelere yalnızca bu sınıftan imal edilen nesneler erişir. Bir başka
         ↪  deyişle yalnızca bu sınıf erişebilir.*/
        int z;
};

class PublicDerived: public Base { //public inheritance
        // Bu sınıfta x alanı(field), public olmuştur.
        // Bu sınıfta y alanı(field), protected olmuştur.
        // Bu sınıfta z alanı(field) ERİŞİLEMEZ olmuştur. -> Public-Derived
};
class PrivateDerived: private Base { //private inheritance
        // Bu sınıfta x alanı(field), private olmuştur.
        // Bu sınıfta y alanı(field), private olmuştur.
        // Bu sınıfta z alanı(field) ERİŞİLEMEZ olmuştur. -> Private-Derived
};
class ProtectedDerived: protected Base { //protected inheritance
        // Bu sınıfta x alanı(field), protected olmuştur.
        // Bu sınıfta y alanı(field), protected olmuştur.
        // Bu sınıfta z alanı(field) ERİŞİLEMEZ olmuştur. -> Protected-Derived
};
```

Kalıtımda üyelere erişim, özet olarak aşağıdaki tabloda verilmiştir; Korumalı (protected) olarak taban sınıftan

Üyelere Erişim	public	protected	private
Aynı Sınıf İçinde	var	var	var
Türemiş Sınıfta	var	var	yok
Sınıf Dışından	var	yok	yok

Tablo 11.1: Kalıtımda Sınıf Üyelerine Erişim

alınan durum ve davranışlara ait bu durumu gösteren bir örnek aşağıda verilmiştir;

```
#include <iostream>
class Kisi {
protected: //Burada tanımlana alan ve davranışlara türemiş sınıflardan da erişilebilir.
        unsigned yas;
        char cinsiyet;
public:
        Kisi() {
                yas=18;
                cinsiyet='E';
        }
        void setYas(unsigned yeniYas) {
                if (yeniYas <18)
                std::cout << "Yaş 18 den Küçük Olamaz." << std::endl;
                else
                yas=yeniYas;
```

```
        }
        unsigned getYas() {
                return yas;
        }
        void setCinsiyet(char yeniCinsiyet) {
                if (yeniCinsiyet == 'E' || yeniCinsiyet == 'E' || yeniCinsiyet == 'K' ||
                ↪  yeniCinsiyet == 'k' || yeniCinsiyet == 'B' || yeniCinsiyet == 'b')
                cinsiyet=yeniCinsiyet;
                else
                std::cout << "Cinsiyet E,e,K,k,B,b Dışında Bir Değer Olamaz." << std::endl;
        }
        char getCinsiyet() {
                return cinsiyet;
        }
        void yasiniSoyle() {
                std::cout << "Yaşım:" << yas << std::endl;
        }
        void cinsiyetSoyle() {
                if (cinsiyet=='E' || cinsiyet=='e')
                std::cout << "Cinsiyetim: ERKEK" << std::endl;
                else if (cinsiyet=='K' || cinsiyet=='k')
                std::cout << "Cinsiyetim: KADIN" << std::endl;
                else
                std::cout << "Cinsiyetim: SÖYLEMEK İSTEMİYORUM!" << std::endl;
        }
};
class Ogrenci: public Kisi {
private:
        unsigned ogrenciNo;
public:
        Ogrenci() {
                yas=35u;  // Artık yas alanına bu türemiş sınıftan da erişilebiliyor.
                cinsiyet='K'; // Artık cinsiyet alanına bu türemiş sınıftan da erişilebiliyor.
                ogrenciNo=0u;
        }
        void dersCalis() {
                std::cout << "Ders Çalışıyorum..." << std::endl;
        }
};
int main() {
        Ogrenci ayse;
        ayse.yasiniSoyle();
        ayse.cinsiyetSoyle();
        ayse.dersCalis();
}
```

C++ dilinde bir türemiş sınıfın birden fazla ebeveyn sınıftan miras alması istenmez. Mantık olarak bir ev, birden fazla mimari plandan hazırlanmayacağından, **çoklu kalıtım** (multiple inheritance) sınıf tasarımında kullanılmaz!

Ancak C++ dilinde 11.19 başlığında inceleyeceğimiz **ara yüzler** (interface) de sınıf olarak kodlandığından sanki çoklu kalıtım varmış gibi algılanmaktadır.

Saf nesne yönelimli diller olan Java ve .Net dillerinde ara yüzler `interface` anahtar kelimesiyle tanımlandığından **birden çok sınıftan miras alma engellenmiştir**.

Türetilmiş bir sınıf, aşağıdaki istisnalar dışında tüm taban sınıf yöntemlerini miras alır;

- Taban sınıfın yapıcıları, **yıkıcıları** (destructor) ve **kopya yapıcıları** (copy constructor).
- Taban sınıfın aşırı yüklenmiş işleçleri.
- Taban sınıfın **arkadaş** (friend) fonksiyonları.

Bu kavramlar ilerleyen bölümlerde anlatılacaktır.

11.10 Yöntemlerin Aşırı Yüklenmesi

Fonksiyonlar için 7.12 başlığında anlatılan aşırı yükleme, sınıflardaki **yöntemlere** (method) de uygulanabilir. Yani farklı parametrelerle yöntem yeniden tanımlanabilir. Aşırı yüklemede yöntemin kimliği ve geri dönüş tipi değişmez. Farklı argümanlar ile yöntemin gövdesi yeniden tanımlanır. Yani aynı davranışın, farklı yöntemlerle yeniden tanımlanmasıdır.

```
#include <iostream>
class Karmasik {
public:
        float gercek, sanal;
        Karmasik() { // varsayılan yapıcı (default constructor)
                gercek = 0.0;
                sanal = 0.0;
        }
        void yaz() { //yaz yöntemi
                std::cout<< gercek << "+" << sanal << +"i" << std::endl;
        }
        void yaz(char pKarakter) { // aşırı yüklenmiş yaz yöntemi
                //aşırı yüklenmiş yaz yöntemi
                std::cout << pKarakter << gercek << "+" << sanal << +"i" <<pKarakter << std::endl;
        }
        void yaz(char pIlkKarakter, char pSonKarakter) { //aşırı yüklenmiş bir başka yaz yöntemi
                std::cout << pIlkKarakter << gercek << "+" << sanal << "i" << pSonKarakter <<
                 ↪  std::endl;
        }
};
int main(void) {
        Karmasik c1;
        c1.gercek=1;
        c1.sanal=2;
        c1.yaz();               // 1+2i
        c1.yaz('@');            // @1+2i@
        c1.yaz('[', ']');       // [1+2i]
}
```

Hali hazırda tanımlı olan **işleçlere** (operator) de bizim tanımladığımız sınıflarda aşırı yükleme yapılabilir. Ya da istenmeyen işleçler `=delete` belirleyicisi (=delete specifier) ile sınıfla işlem yapması kaldırılabilir. Bu işleç aynı zamanda kalıtımdan davranılan davranışları ve yapıcıları kaldırmak için de kullanılabilir.

Aşağıdaki örnekte toplama işlecine **aşırı yükleme** (operator overloading) yapılmış ve çıkarma işleci **kaldırılmıştır**. Ayrıca nesneleri atama işleciyle kopyalayan **kopya yapıcı** (copy constructor) da yazılmıştır.

```
#include <iostream>
class Karmasik {
        public:
        float gercek, sanal;
        Karmasik() { // varsayılan yapıcı (default constructor)
                gercek = 0.0f;
                sanal = 0.0f;
        }
        Karmasik(float pGercek, float pSanal): gercek(pGercek),sanal(pSanal) {
        }
        Karmasik(float pGercek): gercek(pGercek) {
                sanal = 0.0f;
        }
        void yaz() { //yaz yöntemi
                std::cout<< gercek << "+" << sanal << +"i" << std::endl;
        }
        Karmasik operator+(const Karmasik& pKarmasik) { /* + işleci aşırı yükleniyor
         ↪  (overloading): iki karmaşık sayı toplanıyor */
                std::cout << "+ işleci karmaşık sayı parametresiyle çalışacak." << std::endl;
                Karmasik toplam;
                toplam.gercek = gercek + pKarmasik.gercek;
                toplam.sanal = sanal + pKarmasik.sanal;
```

```
                return toplam;
        }
        Karmasik operator+(const float pGercek) { /* + işleci aşırı yükleniyor (overloading):
         ↪  karmaşık sayı ile gerçek sayı toplanıyor */
                std::cout << "+ işleci gerçek sayı parametresiyle çalışacak." << std::endl;
                Karmasik toplam;
                toplam.gercek = gercek + pGercek;
                toplam.sanal = sanal;
                return toplam;
        }
        Karmasik(const Karmasik& pKarmasik) { /* Aşırı yüklenmiş kopya yapıcı (copy constructor):
         ↪  atama (=) için tanımlanmalıdır. */
                std::cout << "Kopya Yapıcı Çalışıtı." << std::endl;
                gercek= pKarmasik.gercek;
                sanal=pKarmasik.sanal;
        }
        void operator-(const Karmasik &) = delete; // Çıkarma işlemi engellenmiş */
};
int main(void) {
        Karmasik karmasik1;
        karmasik1.yaz();
        Karmasik karmasik2(1.0f,1.0f);
        karmasik2.yaz();
        Karmasik karmasik3=karmasik2;
        karmasik3.yaz();
        Karmasik karmasik4=karmasik2+karmasik3;
        karmasik4.yaz();
        Karmasik karmasik5=karmasik2+11.0f;
        karmasik5.yaz();
}
/*Program Çıktısı:
0+0i
1+1i
Kopya Yapıcı Çalışıtı.
1+1i
+ işleci karmaşık sayı parametresiyle çalışacak.
2+2i
+ işleci gerçek sayı parametresiyle çalışacak.
12+1i
*/
```

11.11 Yapıcıların Aşırı Yüklenmesi

Yapıcıların görevi kısaca imal edilecek nesnelerin durumlarına (state) ilk değer vermektir. Durumlara ilişkin veriler de alanlarda (field) tutulurlar. Nesneleri de dışarıdan verilen parametrelerle farklı durumlarda imal etmek için aşırı yüklenmiş yapıcıları kullanabiliriz. Yani farklı parametrelerle yapıcıları yeniden tanımlayabiliriz.

Yapıcıların aşırı yüklenmesinin çeşitli üstünlükleri vardır;

- Nesne imal etmede esneklik: Aynı sınıftan farklı durumlara sahip nesneler imal edilebilir.
- Gelişmiş kod bakımı ile daha temiz ve okunabilir kod oluşur.
- Nesne imal etme mantığı diğer nesnelerden/programlardan gizlenir.
- Kopya yapıcılar ile nesne klonlama basitleşir.

§11.11.1 Varsayılan Yapıcı

Hiç parametresi olmayan yapıcıya **varsayılan yapıcı** (default constructor) adı verilir.

```
#include <iostream>
class Karmasik {
public:
        float gercek, sanal;
        Karmasik() { /* varsayılan yapıcı (default constructor):
                Gerçek ve sanal kısım 0 olarak nesneler imal edilir. */
```

```
                gercek = 0.0;
                sanal = 0.0;
        }
        void yaz() { //yaz yöntemi
                std::cout<< gercek << "+" << sanal << +"i" << std::endl;
        }
};
int main(void) {
        Karmasik c1; // varsayılan yapıcı ile imal edilen c1 nesnesi
        c1.yaz(); // 0+0i
}
```

§11.11.2 Parametreli Yapıcılar

Varsayılan yapıcı dışındaki aşırı yüklenmiş yapıcılar ise **parametreli yapıcı** (parameterized constructor) adını alır.

```
#include <iostream>
class Karmasik {
public:
        float gercek, sanal;

        Karmasik() { /* Varsayılan Yapıcı (default constructor): Gerçek ve sanal kısım 0 olarak
        ↪  nesneler imal edilir. */
                gercek = 0.0;
                sanal = 0.0;
        }
        Karmasik(float pGercek, float pSanal): gercek(pGercek), sanal(pSanal) { /* Aşırı
        ↪  yüklenmiş parametreli yapıcı (parameterized constructor)*/
        }
        /*
        //Yukarıdaki yapıcı bu şekilde de yazılabilir;
        Karmasik(float pGercek, float pSanal) {
                gercek = pGercek;
                sanal = pSanal;
        }
        */
        Karmasik(float pGercek): gercek(pGercek), sanal(0.0f){ /* Aşırı yüklenmiş bir başka
        ↪  parametreli yapıcı (parameterized constructor) */
        }
        /*
        //Yukarıdaki yapıcı bu şekilde de yazılabilir;
        Karmasik(float pGercek) {
                gercek=pGercek;
                sanal=0.0f;
        }
        */
        void yaz() { //yaz yöntemi
                std::cout<< gercek << "+" << sanal << +"i" << std::endl;
        }
};
int main(void) {
        Karmasik c1; // varsayılan yapıcı ile imal edilen c1 nesnesi
        c1.yaz(); // 0+0i
        Karmasik c2(2.0,1.0); // aşırı yüklenmiş yapıcı ile imal edilen c2 nesnesi
        c2.yaz(); // 2+1i
        Karmasik c3(12.0);
        c3.yaz(); //12+0i
}
```

§11.11.3 Devredilen Yapıcı

Üstte örneği verilen ikinci yapıcı, **devredilen yapıcı** (delegating constructor) ile aşağıdaki şekilde de tanımlanabilir;

```
#include <iostream>
class Karmasik {
public:
        float gercek, sanal;
        Karmasik() { /* varsayılan (default constructor): Gerçek ve sanal kısım 0 olarak nesneler
         ↪ imal edilir. */
                gercek = 0.0f;
                sanal = 0.0f;
        }
        Karmasik(float pGercek, float pSanal): gercek(pGercek), sanal(pSanal) {
        }
        Karmasik(float pGercek): Karmasik (pGercek, 0.0f) { //Devredilen Yapıcı (delagating
         ↪ constructor)
        }
        void yaz() { //yaz yöntemi
                std::cout<< gercek << "+" << sanal << +"i" << std::endl;
        }
};
int main(void) {
        Karmasik c1; // varsayılan yapıcı ile imal edilen c1 nesnesi
        c1.yaz(); // 0+0i
        Karmasik c2(2.0,1.0); // aşırı yüklenmiş yapıcı ile imal edilen c2 nesnesi
        c2.yaz(); // 2+1i
        Karmasik c3(12.0);
        c3.yaz(); //12+0i
}
```

§11.11.4 Kopya Yapıcı

Aynı sınıftan bir nesneyi bir başka imal edilmiş nesnenin durumlarına sahip olarak imal edebiliriz. Bir nesneyi bir yapıcıya argüman olarak geçirmek ve yeni nesneye argümanın durumlarını vermek gerekir. Bu yapıcıya **kopya yapıcı** (copy constructor) adı verilir;

```
#include <iostream>
class Karmasik {
public:
        float gercek, sanal;
        Karmasik() { /* varsayılan yapıcı (default constructor)*/
                gercek = 0.0;
                sanal = 0.0;
        }
        Karmasik(float pGercek, float pSanal): gercek(pGercek), sanal(pSanal) { /* parametreli
         ↪ yapıcı (parameterized constructor)*/
        }
        Karmasik(Karmasik& pKarmasik) { /* Kopya yapıcı (copy constructor): Parametre olarak bir
         ↪ başka karmaşık sayı veriliyor ve Gerçek ve sanal durumları parametreden gelen
         ↪ karmaşık sayı ile aynı olacak şekilde nesneler imal edilir. */
                gercek= pKarmasik.gercek;
                sanal=pKarmasik.sanal;
        }
        void yaz() { //yaz yöntemi
                std::cout<< gercek << "+" << sanal << +"i" << std::endl;
        }
};
int main(void) {
        Karmasik c1; // öntanımlı yapıcı ile imal edilen c1 nesnesi
        c1.yaz();    // 0+0i
        Karmasik c2(2.0,1.0); // parametreli yapıcı ile imal edilen c2 nesnesi
        c2.yaz();    // 2+1i
        Karmasik c3(c2); // kopya yapıcı ile imal edilen c3 nesnesi
        c3.yaz();    // 2+1i
}
```

§11.11.5 Atama İşlecinin Aşırı Yüklenmesi

Kopya yapıcı gibi çalışan ancak mevcut iki nesne üzerinde işlem yapan atama işlecine ilişkin aşırı yüklemeye bazen **eşitlik yapıcısı** (assignment constructor) adı verilir. **Aslında bu tam bir yapıcı değildir** mevcut nesnenin durumunu bir başka nesnenin durumuna eşler.

```
#include <iostream>
class Karmasik {
public:
        float gercek, sanal;
        Karmasik() { /* öntanımlı yapıcı (default constructor)*/
                gercek = 0.0;
                sanal = 0.0;
        }
        Karmasik(float pGercek, float pSanal): gercek(pGercek),sanal(pSanal) {
        }
        Karmasik(Karmasik& pKarmasik) { /* Kopya yapıcı (copy constructor)*/
                this->gercek= pKarmasik.gercek;
                this->sanal=pKarmasik.sanal;
        }
        Karmasik& operator=(const Karmasik& pKarmasik) { /* atama işleci aşırı yüklemesi */
                this->gercek= pKarmasik.gercek;
                this->sanal=pKarmasik.sanal;
                return *this;
        }
        void yaz() { //yaz yöntemi
                std::cout<< gercek << "+" << sanal << +"i" << std::endl;
        }
};
int main(void) {
        Karmasik c1; // öntanımlı yapıcı ile imal edilen c1 nesnesi
        c1.yaz();    // 0+0i
        Karmasik c2(2.0,1.0); // parametreli yapıcı ile imal edilen c2 nesnesi
        c2.yaz();    // 2+1i
        Karmasik c3(c2); // kopya yapıcı ile imal edilen c3 nesnesi
        c3.yaz();    // 2+1i
        c3=c1;       // atama işleci aşırı yüklemesi
        c3.yaz();    // 0+0i
}
```

§11.11.6 Varsayılan Yapıcının Değiştirilmesi

Yukarıda açıklanan **varsayılan yapıcı** (default constructor), `=default` belirleyicisi (=default specifier) ile değiştirilebilir. Bu belirleyici aşırı yüklenmiş fonksiyonlarda da kullanılabilir.

C++ standartlarında, `float` gibi temel tipleri parametre alan bir yapıcıyı doğrudan `= default;` şeklinde tanımlayamazsınız! Derleyici bu parametreleri nesne üyelerine nasıl eşleyeceğini bilemez. Ancak üyeleri sınıf içinde ilk olarak hangisinin seçileceğini belirleyebilirsiniz. Parametresiz yapıcıyı `default` yaparak aynı etkiyi çok daha temiz bir şekilde elde edebilirsiniz.

Aşağıda bu belirleyiciye ilişkin örnek verilmiştir;

```
#include <iostream>
class Karmasik {
        public:
        // Üyeleri burada başlatıyoruz (C++11 ve sonrası için en iyi pratik)
        float gercek = 0.0f;
        float sanal = 0.0f;
        // Öntanımlı yapıcıyı 'default' yaparak yukarıdaki değerleri kullanmasını sağlıyoruz
        Karmasik() = default;
        // Parametreli yapıcı: İlklendirme listesi ile en verimli yol
        Karmasik(float pGercek, float pSanal) : gercek(pGercek), sanal(pSanal) {}
        // Kopya yapıcıyı da derleyiciye bırakmak (default) en performanslısıdır
        Karmasik(const Karmasik&) = default;
        // Atama operatörünü de derleyiciye bırakabiliriz
        Karmasik& operator=(const Karmasik&) = default;
```

```
        void yaz() const {
                std::cout << gercek << " + " << sanal << "i" << std::endl;
        }
};
int main() {
        Karmasik k1;
        k1.yaz(); //0 + 0i
        k1.gercek=3;
        k1.sanal=5;
        Karmasik k2=k1;
        k2.yaz(); //3 + 5i
}
```

Kalıtım (inheritance) durumunda ise taban sınıfın varsayılan yapıcısı çağrılır. Bu duruma aşağıdaki örnek verilebilir;

```
#include <iostream>
class Kisi {
private:
        unsigned yas;
        char cinsiyet;
public:
        Kisi() {// Taban Sınıf Varsayılan Yapıcısı
                yas = 18;
                cinsiyet = 'E';
                std::cout << "-> Kisi varsayilan yapicisi calisti (Yas: 18, Cinsiyet: E)" <<
                ↪  std::endl;
        }
        // Bilgi veren metodlar
        void yasiniSoyle() { std::cout << "Yasim: " << yas << std::endl; }
        void cinsiyetSoyle() {
                std::cout << "Cinsiyetim: " << (cinsiyet == 'E' ? "ERKEK" : "KADIN") << std::endl;
        }
};
class Ogrenci : public Kisi {
private:
        unsigned ogrenciNo;
public:
        Ogrenci() { // Türetilmiş Sınıf Varsayılan Yapıcısı
                ogrenciNo = 0;
                // Burada Kisi() gizli olarak (implicit) zaten cagirildi.
                std::cout << "-> Ogrenci varsayilan yapicisi calisti (No: 0)" << std::endl;
        }
        void dersCalis() {
                std::cout << "Ogrenci ders calisiyor..." << std::endl;
        }
};

int main() {
        std::cout << "Ogrenci nesnesi olusturuluyor..." << std::endl;
        // Ogrenci nesnesi olustugu an silsile baslar:
        // 1. Kisi() cagrilir
        // 2. Ogrenci() cagrilir
        Ogrenci ilhan;
        ilhan.yasiniSoyle();    // Kisi'den miras alindi
        ilhan.cinsiyetSoyle();  // Kisi'den miras alindi
        ilhan.dersCalis();      // Ogrenci'ye ozgu
}
```

§11.11.7 Taşıma Yapıcısı

Bir kopya yapıcı (copy constructor), mevcut bir nesnenin durumunu başka bir nesneye aktararak yeni bir örnek oluşturmak için kullanılır. Bu işlem için genellikle aşağıdaki söz dizimi tercih edilir:

```
        // Kopya Yapıcı
```

```
        Karmasik(const Karmasik &diger) {
                this->gercek = diger.gercek;
                this->sanal = diger.sanal;
        }
```

Burada `Karmasik` sınıfı, aynı türden başka bir nesneye sabit (const) bir referans alarak üyeleri elle kopyalar. Eğer özel bir kopyalama mantığı gerekmiyorsa, tüm üyelerin otomatik kopyalanması için `Karmasik(const Karmasik&) = default;` ifadesi de kullanılabilir.

Öte yandan, modern C++ ile gelen taşıma yapıcısı (move constructor), bir nesneyi kopyalamak yerine durumunu (veya kaynaklarını) yeni nesneye taşımak için kullanılır. Bu işlemde **lvalue** referansı yerine `rvalue` referansı (rvalue reference) (`&&`) kullanılır. Taşıma semantiği, özellikle dinamik bellek yönetimi içeren sınıflarda performans açısından "sahipliğin aktarılmasına" veya "durumun devralınmasına" izin verir.

```
        // Taşıma Yapıcısı
        Karmasik(Karmasik &&diger) noexcept {
                this->gercek = diger.gercek;
                this->sanal = diger.sanal;
                // Kaynak nesnenin durumunu sıfırlıyoruz
                diger.gercek = 0.0f;
                diger.sanal = 0.0f;
        }
```

Burada, durumu taşınan (kaynak) nesnenin değerlerinin sıfırlandığına dikkat edilmelidir. Taşıma işlemi, orijinal örnekten **durum çalmaya** (pilfering) olanak tanır. Orijinal örneğin bu işlemden sonraki hali, programın tutarlılığı açısından kritik bir öneme sahiptir. Eğer değerleri sıfırlamasaydık, taşıma sonrası her iki nesne de aynı veriye sahip olurdu ki bu, özellikle gösterici (pointer) içeren yapılarda bellek hatalarına yol açabilirdi.

Aşağıdaki kullanımda bu fark açıkça görülmektedir:

```
        Karmasik sayi1;
        sayi1.gercek = 5.5f;
        sayi1.sanal = 2.0f;
        // sayi1'in durumu sayi2'ye taşınır
        Karmasik sayi2(std::move(sayi1));
        std::cout << "Sayi 1: " << sayi1.gercek << " + " << sayi1.sanal << "i" << std::endl; // 0
         ↪  + 0i
        std::cout << "Sayi 2: " << sayi2.gercek << " + " << sayi2.sanal << "i" << std::endl; //
         ↪  5.5 + 2i
```

Böylece, bellek üzerinde yeni bir kopyalama maliyetine katlanmadan, eski bir nesnenin sahip olduğu veriyi yeni bir nesneye verimli bir şekilde aktarmış oluruz.

§11.11.8 Yapıcılarda Varsayılan Argümanlar

Fonksiyonlarda 7.11 başlığında işlenen **varsayılan argümanlar** (default argument) yapıcılarda da kullanılabilir. Karmaşık sayılar örneği aşağıda verilmiştir;

```
#include <iostream>
class Karmasik {
        float gercek, sanal;  //bu üyeler varsayılan olarak private
public:
        Karmasik(float pGercek=0.0f, float pSanal=0.0f): gercek(pGercek),sanal(pSanal) { }
        // Varsayılan argümanlara yapıcı tanımı:
        void yaz() { //yaz yöntemi
                std::cout<< gercek << "+" << sanal << +"i" << std::endl;
        }
        Karmasik operator+(const Karmasik& pKarmasik) {
                Karmasik toplam;
                toplam.gercek = gercek + pKarmasik.gercek;
                toplam.sanal = sanal + pKarmasik.sanal;
                return toplam;
        }
        Karmasik operator+(const float pGercek) {
                Karmasik toplam;
                toplam.gercek = gercek + pGercek;
                toplam.sanal = sanal;
                return toplam;
```

```
        }
        Karmasik(const Karmasik& pKarmasik) { // copy constructor
                gercek= pKarmasik.gercek;
                sanal=pKarmasik.sanal;
        }
};
int main(void) {
        Karmasik c1; // Varsayılan argümanlara Karmasik(0.0f,0.0f) yapıcısı kullanılarak imal
         ↪ edildi.
        c1.yaz();
        Karmasik c2(1.0,1.0); // Karmasik(1.0f,1.0f) yapıcısı ile imal edildi.
        c2.yaz();
        Karmasik c3(1.0); // Varsayılan argümanlara Karmasik(1.0f,0.0f) yapıcısı kullanılarak
         ↪ imal edildi.
        c3.yaz();
        Karmasik c4=c2+c3; // c4 kopya yapıcısı ile imal edildi.
        c4.yaz();
        c4=c2+c3; // c4 kopya yapıcısı ile imal edildi.
        c4.yaz();
}
```

11.12 using Anahtar Kelimesi ile Yapıcıların Miras Alınması

`using` anahtar kelimesi, 10.10.4 başlığındaki kullanımı yanında, C++11/14 ile `using` anahtar kelimesi, türetilmiş bir sınıfın (derived class), temel sınıfın (base class) tüm yapıcıları tek satırda miras almasını sağlar.

```
class Temel {
        public:
        Temel(int x) { /*...*/ }
        Temel(double y, int z) { /*...*/ }
};
class Turemis : public Temel {
        public:
        using Temel::Temel; // Temel sınıfın TÜM yapıcılarını içeri aktarır!
};
int main() {
        // Artık Turemis sınıfı için int veya double alan yapıcıları tek tek yazmaya gerek yok.
        Turemis t(10); //türemiş sınıfta using kullanıldığı için bu yapılabilir!
}
```

11.13 Nesne Yıkıcı

Yıkıcı (destructor), bir nesne yok edileceği zaman otomatik olarak çağrılan bir yöntemdir. Bir sınıf içinde yalnızca bir yıkıcı tanımlanabilir. Yıkıcı, bir nesne yok edilmeden önce çağrılacak son yöntemdir. Aşağıdaki gibi tanımlanır.

```
~sınıf-kimliği() {
 // ...
}
```

Yıkıcılar, daha çok kaynak kullanan nesnelerin yok edilmesi öncesinde çağrılmak üzere kodlanır. Çalışma mantığına ilişkin örnek verilmiştir;

```
#include <iostream>

static int nesneSayaci=0;

class Karmasik {
        float gercek, sanal;
        public:
        Karmasik(float pGercek=0.0f, float pSanal=0.0f): gercek(pGercek),sanal(pSanal) {
                nesneSayaci++;
                std::cout << nesneSayaci << " nesne imal edildi" << std::endl;
        }
        void yaz() { //yaz yöntemi
```

```
                std::cout<< gercek << "+" << sanal << +"i" << std::endl;
        }
        Karmasik operator+(const Karmasik& pKarmasik) {
                Karmasik toplam;
                toplam.gercek = gercek + pKarmasik.gercek;
                toplam.sanal = sanal + pKarmasik.sanal;
                return toplam;
        }
        Karmasik(const Karmasik& pKarmasik) {
                gercek= pKarmasik.gercek;
                sanal=pKarmasik.sanal;
        }
        ~Karmasik() {
                nesneSayaci--;
                std::cout << nesneSayaci << " nesne yok edildi" << std::endl;
        }
};
int main(void) {
        Karmasik c1;
        c1.yaz();
        Karmasik c2(3.0,5.0);
        c2.yaz();
        Karmasik c3=c1+c2;
        c3.yaz();
}
/*Program Çıktısı:
1 nesne imal edildi
0+0i
2 nesne imal edildi
3+5i
3 nesne imal edildi
3+5i
2 nesne yok edildi
1 nesne yok edildi
0 nesne yok edildi
*/
```

11.14 friend Anahtar Kelimesi

Bir sınıfın **arkadaş** (friend) yöntemi, o sınıfın bloğu dışında tanımlanır ancak sınıfın tüm mahrem (private) ve korumalı (protected) üyelerine erişim hakkına sahiptir. Arkadaş yöntemlerin bildirimleri/prototipleri sınıf içerisinde yer almasına rağmen, arkadaşlar üye fonksiyon değildir.

```
#include <iostream>
class Karmasik {
        float gercek, sanal;  //private members
        public:
        Karmasik(float pGercek=0.0, float pSanal=0.0) {
                this->gercek=pGercek;
                this->sanal=pSanal;
        }
        friend void yaz(Karmasik pKarmasik);
};
void yaz(Karmasik pKarmasik) {
        std::cout << pKarmasik.gercek << "+" <<pKarmasik.sanal <<"i" << std::endl;
}
int main(void) {
        Karmasik c1(1.0,2.0);
        yaz(c1); // 1+2i
        Karmasik c2(3.0,5.0);
        yaz(c2); // 3+5i
}
```

Buna, bir sınıfla bir akışının (stream) ilişkilendirilmesi örnek verilebilir. Nesne verilerinin akıştan çıkarıl-

masında **ayıklama işleci** (extraction operator) (>>) kullanılabilir. Benzer şekilde nesne verilerinin akışlara göndermek için **ekleme işleci** (insertion operator) (<<) kullanılabilir. Karmaşık sayı sınıfı için aşağıdaki gibi bir örnek verilebilir;

```
#include <iostream>
class Karmasik {
        float gercek, sanal; // Varsayılan olarak private
public:
        // Parametreli yapıcı (Modern ilklendirme listesi kullanımı)
        Karmasik(float pGercek = 0.0f, float pSanal = 0.0f)
        : gercek(pGercek), sanal(pSanal) {}
        // Çıkış Akışı (Output Stream) İşleci
        friend std::ostream& operator<<(std::ostream& output, const Karmasik& pKarmasik) {
                output << pKarmasik.gercek << "+" << pKarmasik.sanal << "i";
                return output;
        }
        // Giriş Akışı (Input Stream) İşleci
        friend std::istream& operator>>(std::istream& input, Karmasik& pKarmasik) {
                // Not: Giriş operatöründe nesne const olamaz çünkü değerlerini değiştireceğiz.
                input >> pKarmasik.gercek >> pKarmasik.sanal;
                return input;
        }
};
int main(void) {
        Karmasik c1(1.0f, 2.0f);
        std::cout << "c1 nesnesi: " << c1 << std::endl; // 1+2i
        Karmasik c2;
        std::cout << "Lutfen gercek ve sanal kismi giriniz (orn: 3.5 4.2): ";
        std::cin >> c2; // Kullanıcıdan veri alma
        std::cout << "Girilen c2 nesnesi: " << c2 << std::endl;
}
```

11.15 Statik Sınıf Üyeleri

C++ Dilinde **statik üye yöntemi** (static member methodstatic member method), imal edilecek herhangi bir nesneye değil, sınıfın kendisine ait olan özel bir fonksiyon türüdür. Bu yöntemleri tanımlamak için statik anahtar kelimesi kullanılır. Sınıfın bir örneği olacak nesne imal edilmesine gerek kalmadan, sadece sınıf kimliğini kullanarak doğrudan çağrılabilirler. Bu tür yöntemlere **yardımcı yöntem** (utility method) adı verilir. Bu yöntemler program çalışmaya başladığında var olur ve program bitinceye kadar erişilebilirler. Ayrıca yalnızca sınıfın statik üyeleri veya diğer statik fonksiyonlarıyla çalışabilirler. `static` bir üyeye sınıf içerisinde veya yapısında ilk değer veremeyiz! Bu nedenle sınıf dışında veri tipi belirterek başlatmalıyız!

```
#include <iostream>
class YardimciSinif {
        public:
        static int genelSayac;
        static void programciAdi() {
                std::cout << "Programcı Adı: Ilhan OZKAN" << std::endl;
                std::cout << genelSayac << std::endl;
        }
};
/* sınıf üyesini sınıf sışında veri tipi belirterek başlatma */
int YardimciSinif::genelSayac=100;
int main(void) {
        YardimciSinif::programciAdi(); /* static üye yöntemi çağırma */
        YardimciSinif::genelSayac--; /* static üye alana erişim */
        YardimciSinif::programciAdi();
}
```

Statik yöntemlere benzer şekilde statik alanlar da tanımlanabilir. Statik olan bu veriler veri bellekte (data segment) saklanırlar ve bu veri üyeleri de statik yöntemler gibi kullanılır. Statik üyeler genellikle bir sınıfın tüm örnekleri arasında paylaşılması gereken paylaşılan kaynakları veya sayaçları yönetmek için kullanılır. İleride anlatılacak olan **tasarım desenlerinden** (design pattern) **tekil nesne deseni** (singleton pattern) statik üyelere örnek olarak verilebilir. Bu desende sınıftan yalnızca bir nesne/örnek imal edilen ve bu tekil nes-

neye evrensel erişim sağlayan bir tasarım desenidir. Burada statik üyeler, sınıfın tek bir paylaşılan örneğinin bakımına izin verdikleri için bu deseni uygulamak için mükemmeldir.

11.16 const Sınıf Üyeleri

Sınıfların `const` yöntemleri (const method), veri üyelerinin yani durumları (state) tutan alanları (field) değiştirme izni verilmeyen fonksiyonlardır. Bir üye fonksiyonunu `const` yapmak için, `const` anahtar kelimesi fonksiyon prototipine ve ayrıca fonksiyon tanımındaki başlığına eklenir.

Üye yöntemler gibi bir sınıfın nesneleri de `const` olarak bildirilebilir. `const` olarak bildirilen bir nesne değiştirilemez ve bu nedenle, bu fonksiyonlar nesneyi değiştirmemeyi garantilediğinden, yalnızca `const` üye yöntemlerini çağırabilir.

Bir `const` nesnesi, nesne bildirimine `const` anahtar sözcüğünü ön ek olarak ekleyerek tanımlanabilir. Bu nesneler **değiştirilemez** (immutable) olarak adlandırılır. `const` nesnelerinin veri üyesini değiştirmeye yönelik herhangi bir girişim, derleme zamanı hatasıyla (compile-time error) sonuçlanır.

```
#include <iostream>
class Sinif {
        private:
        int alan;
        public:
        Sinif() {
                alan=-1;
        }
        int getAlan() const {
                return alan;
        }
        void setAlan(int pAlan) { // Bu yöntem const tanımlanırsa hata alınır!
                alan=pAlan;  // çünkü alan değişkenini değişitir!
        }
};
int main() {
        Sinif* nesne=new Sinif();
        std::cout<<"nesne.alan:" << nesne->getAlan() << std::endl;
        nesne->setAlan(12);
        std::cout<<"nesne.alan:" << nesne->getAlan() << std::endl;
        delete nesne;
        const Sinif* constNesne=new Sinif();
        //constNesne->setAlan(9); // Hata: const nesne (immutable object) değiştirilemez!
        delete constNesne;
}
```

11.17 mutable Depolama Sınıfı

Bazen, `const` ile sabit olarak tanımlanmış nesne veya yapının (struct) bir veya daha fazla veri üyesini değiştirme gereksinimi doğabilir. İşte bu durumda bu üyeler `mutable` anahtar kelimesiyle tanımlanır.

```
#include <iostream>
class Kisi {
        public:
        unsigned long tcno;
        mutable int okulno;
        Kisi(){
                tcno = 43233445505UL;
                okulno = -1;
        }
};
int main(){
        const Kisi ogrenci1; // ogrenci1 nesnesi const olarak tanımlandı
        ogrenci1.okulno = 2000; // sabit olan nesnenin okulno değiştiriliyor.
        std:: cout << ogrenci1.okulno << std::endl;
        //ogrenci1.tcno = -1; //HATA: sabit nesnenin alanı değiştirilemez.
        /* assignment of member 'Kisi::tcno' in read-only object*/
}
```

11.18 Geçersiz Kılma

Geçersiz kılma (overriding), nesne yönelimli programlamada taban sınıfta önceden tanımlanmış bir yöntemi türemiş bir sınıfta yeniden tanımlamasına olanak tanıyan bir kavramıdır. Yani taban sınıfın davranışının türemiş sınıfta değiştirilmesidir. C++ dili iki tür fonksiyon geçersiz kılmayı destekler; Birincisi derleme zamanı geçersiz kılma, diğeri ise çalışma zamanı geçersiz kılmadır.

Derleme zamanında geçersiz kılma aşağıdaki şekilde yapılır;

```
class Taban-Sınıf-Kimliği {
erişim-belirleyici:
 // geçersiz kılınacak yöntem:
 veri-tipi yöntem-kimliği() {
 }
};

class Türemiş-Sınıf-Kimliği: erişim-belirleyici Taban-Sınıf-Kimliği {
erişim-belirleyici:
 // geçersiz kılan yöntem:
 veri-tipi yöntem-kimliği() {
 }
};
```

Her iki sınıf ta da yöntemin kimliği aynıdır. Taban sınıftan imal edilen nesne taban sınıfın yöntemini, türemiş sınıftan imal edilen nesne ise türemiş sınıfın yöntemini icra eder.

```
#include <iostream>
class Baba {
        public:
        void yap() {
                std::cout << "Baba şu şekilde yapar ..." << std::endl;
        }
};
class Cocuk : public Baba {
        public:
        void yap() {
                std::cout << "Çocuk şu şekilde yapar..." << std::endl;
        }
};
int main() {
        Baba baba;
        baba.yap(); // Baba şu şekilde yapar ...
        Cocuk cocuk;
        cocuk.yap(); // Çocuk şu şekilde yapar...
}
```

Çalışma zamanında geçersiz kılma aşağıdaki şekilde yapılır;

```
class Taban-Sınıf-Kimliği {
erişim-belirleyici:
 // geçersiz kılınacak yöntem:
 virtual veri-tipi yöntem-kimliği() {
 }
};

class Türemiş-Sınıf-Kimliği: erişim-belirleyici Taban-Sınıf-Kimliği {
erişim-belirleyici:
 // geçersiz kılan yöntem:
 veri-tipi yöntem-kimliği() override {
 }
};
```

Her iki sınıf ta da yöntemin kimliği aynıdır. Geçersiz kılınacak yöntem, taban sınıfın yöntemini türemiş sınıfta değiştirilebileceğini belirtmek için `virtual` sıfatı () ile tanımlanır. Türemiş sınıfta ise geçersiz kılacağı yöntemi `override` belirleyicisi () ile yeniden tanımlar.

```
#include <iostream>
```

```
class Baba {
        public:
        virtual void yap() {
                std::cout << "Baba şu şekilde yapar ..." << std::endl;
        }
};
class Cocuk : public Baba {
        public:
        void yap() override {
                std::cout << "Çocuk şu şekilde yapar..." << std::endl;
        }
};
int main() {
        Cocuk cocuk;
        cocuk.yap(); // Çocuk şu şekilde yapar...
        Baba* babaPtr=&cocuk;
        babaPtr->yap(); // Çocuk şu şekilde yapar...
        /*
        Baba& babaPtr=cocuk;
        babaPtr.yap(); // Çocuk şu şekilde yapar...
        */
}
```

11.19 Ara Yüzler

Ara yüzler (interface) nesnelerin sahip oldukları rollerdir (role). Roller bir grup davranışı (behavior) barındırır. C++ dilinde roller, sınıf olarak kodlanır. Ancak saf nesne yönelimli dillerde ara yüzler, `interface` saklı kelimesiyle tanımlanırlar.

Örnek olarak bir öğretmen; Evde baba rolünde olup babanın sahip olduğu çocuklara bakma, yemek yapma gibi davranışları gösterir. İşte öğretmen rolündedir ve öğrenmenin sahip olduğu; öğretme, sınav yapma, değerlendirme, karne hazırlama, ders notu hazırlama gibi davranışları vardır. Bu öğretmen müzisyen rolünde ise gitar çalma, akort yapma gibi davranışları da vardır. İşte bunların hepsi ayrı bir roldür ve her bir rolde o role ilişkin davranışları sergiler.

> Java, C# gibi saf nesne yönelimli dillerin aksine C++ Dilinde, **çoklu kalıtım** (multiple inheritance) desteklenir. Bir sınıfın ara yüzleri miras almasına **arayüz gerçekleştirme** (interface realization) yada **arayüz uygulaması** (interface imlementation) adı verilir.

Ara yüzler, soyut (abstract) sınıflarda tanımlanır ve veri soyutlaması (data abstraction) yapılmaz. Bir sınıfın soyut olabilmesi için en az bir **saf sanal yönteme** (pure virtual method) sahip olmalıdır. Sanal sınıflardan nesne imal edilemez! Sanal sınıflardaki sanal yöntemler türeyen sınıfta yeniden tanımlanır ve mutlaka geçersiz kılınır (override).

```
#include <iostream>
#include <string> // Metinler Dizgiler başlığında detaylandırılacaktır.
class IBaba { //Baba Rolündeki davranışlar:
public:
        virtual void cocugaBak() = 0; // saf sanal yöntem-pure virtual method
        virtual void hanimaYardimEt() = 0; // saf sanal yöntem-pure virtual method
        /* Bu sınıfta en az bir saf sanal yöntem olduğundan bu sınıf soyut sınmıf olur. Bu
         ↪ nedenle bu sınıftan nesne imal edilemez! */
};
class IMuzisyen { //Müzisyen Rolündeki Davranışlar:
public:
        virtual void gitarCal() = 0; // saf sanal yöntem-pure virtual method
        virtual void piyanoCal() = 0; // saf sanal yöntem-pure virtual method
        /* Bu sınıfta en az bir saf sanal yöntem olduğundan bu sınıf soyut sınmıf olur. Bu
         ↪ nedenle bu sınıftan nesne imal edilemez! */
};
class IOgretmen { //Öğretmen Davranışları:
public:
        virtual void ogret() = 0; // saf sanal yöntem-pure virtual method
```

```
        virtual void dersNotuHazirla() = 0; // saf sanal yöntem-pure virtual method
        /* Bu sınıfta en az bir saf sanal yöntem olduğundan bu sınıf soyut sınmıf olur. Bu
         ↪ nedenle bu sınıftan nesne imal edilemez! */
};
class Kisi:  public IBaba, public IMuzisyen, public IOgretmen {
        /* (interface realization/implementation): Gerçekleştirilen IBaba, IMuzisyen ve IOgretmen
         ↪ arayüzlerinde saf sanal yöntemler olduğundan bu Kişi sınıfında hepsi geçersiz
         ↪ kılınmalıdır (override). */
public:
        std::string adi;
        std::string soyadi;
        void adiniSoyle() {
                std::cout << adi << " " << soyadi << std::endl;
        }
        // IBaba aracılığıyla miras alınan davranışlar:
        void cocugaBak() override {
                std::cout << "Çocuğa Bakıyorum..." << std::endl;
        }
        void hanimaYardimEt() override {
                std::cout << "Hanıma Yardım Ediyorum..." << std::endl;
        }
        // IMuzisyen aracılığıyla miras alınan davranışlar:
        void gitarCal() override {
                std::cout << "Gitar Çalıyorum..." << std::endl;
        }
        void piyanoCal() override {
                std::cout << "Piyano Çalıyorum..." << std::endl;
        }
        // IOgretmen aracılığıyla miras alınan davranışlar:
        void ogret() override {
                std::cout << "Öğretiyorum..." << std::endl;
        }
        void dersNotuHazirla() override {
                std::cout << "Ders notu hazırlıyorum..." << std::endl;
        }
};
int main() {
        Kisi ilhan;
        ilhan.adi = "İlhan";
        ilhan.soyadi = "ÖZKAN";

        //imal edilen nesne tüm arayüzdeki davranışları gösterebilir:
        ilhan.adiniSoyle(); // İlhan ÖZKAN
        ilhan.ogret(); // Öğretiyorum...
        ilhan.gitarCal(); // Gitar Çalıyorum...
        ilhan.hanimaYardimEt(); // Hanıma Yardım Ediyorum...

        //imal edilen nesne sadece bir roldeki davranışları da gösterebilir:
        IBaba& baba=ilhan;
        baba.cocugaBak(); // Çocuğa Bakıyorum...
        baba.hanimaYardimEt(); // Hanıma Yardım Ediyorum...

        IMuzisyen& muzisyen = ilhan;
        muzisyen.gitarCal(); // Gitar Çalıyorum...
        muzisyen.piyanoCal(); // Piyano Çalıyorum...

        IOgretmen& ogretmen = ilhan;
        ogretmen.dersNotuHazirla(); // Ders notu hazırlıyorum...
        ogretmen.ogret(); // Öğretiyorum...
}
```

11.20 Çok Biçimlilik

Çok biçimlilik (polymorphism), birden çok nesnenin olduğu bir ortamda, **verilen aynı iletiye (message) karşı, nesnelerin farklı davranış (behavior) göstermeleridir** (same message-different behavior). Çok biçimliliğin kodlamada uygulanabilmesi (implementation) için üç şey gereklidir;

1. **Kalıtım** (inheritance): Ortak davranışın tanımlandığı taban sınıf ve bu davranışın değiştiği türeyen sınıflar.
2. **Sanal Yöntem** (virtual method): Nesnelere aynı iletiyi verebilmek için ortak davranışı taban sınıfta tanımlayan sanal yönteme sahip olmalıdır.
3. **Geçersiz Kılma** (override): Türemiş sınıfta devralınan davranışı kendine uyarlayan ve taban sınıftaki sanal yöntemi geçersiz kılan yeni bir yöntem.

```
#include <iostream>
#include <string>
class SoyutKisi { //Soyut Kisi Sınıfı
public:
        /* Sanal Yıkıcı (Virtual Destructor): Eğer temel sınıfın yıkıcısı sanal olmazsa, delete
         ↪ kisi[i] dendiğinde türemiş sınıfların (Ogrenci, Mudur vb.) özel alanları
         ↪ temizlenmeyebilir.*/
        virtual ~SoyutKisi() = default;
        virtual void raporYaz() = 0;
        /* saf sanal yöntem: Bu yöntem, Kişi sınıfını SOYUT yapar. Ayrıca;        raporYaz():
         ↪ Türeyen sınıflardan imal edilecek nesnelere aynı mesajı vermek için tanımlanan ortak
         ↪ davranıştır. */
        std::string getAdi() {
                return adi;
        }
        void setAdi(std::string pAdi) {
                if (pAdi!="") this->adi=pAdi;
        }
        SoyutKisi(std::string pAdi): adi(pAdi) {
        }
        private:
        std::string adi;
};
class Ogrenci: public SoyutKisi {  //Ortak davranış SoyutKisi sınıfından devralınıyor.
public:
        void raporYaz() override { //devralınan yöntemler geçersiz kılınıyor (override)
                std::cout << getAdi() <<":Öğrenci Ödevilerine İlişkin Rapor Yazıyor..." <<
                 ↪ std::endl;
        }
        Ogrenci(std::string pOgrenciAdi):SoyutKisi(pOgrenciAdi){
        }
};
class Ogretmen: public SoyutKisi {  //Ortak davranış SoyutKisi sınıfından devralınıyor.
public:
        void raporYaz() override { //devralınan yöntemler geçersiz kılınıyor (override)
                std::cout << getAdi() <<":Ogretmen Verdiği Derslere İlişkin Rapor Yazıyor..."
                << std::endl;
        }
        Ogretmen(std::string pOgretmenAdi):SoyutKisi(pOgretmenAdi){
        }
};
class Mudur: public SoyutKisi {  //Ortak davranış SoyutKisi sınıfından devralınıyor.
public:
        void raporYaz() override { //devralınan yöntemler geçersiz kılınıyor (override)
                std::cout << getAdi() <<":Müdür Akademik Takvime İlişkin Rapor Yazıyor..."
                << std::endl;
        }
        Mudur(std::string pMudurAdi):SoyutKisi(pMudurAdi){
        }
};
```

```
int main() {
        /* Modern C++'ta çıplak diziler yerine std::vector ve smart pointer kullanılır.
         ↪  std::unique_ptr sayesinde 'delete' yazmanıza gerek kalmaz, bellek sızıntısı önlenir.
         ↪  İleride anlatılacaktır. */
        SoyutKisi* kisi[3];
        kisi[0]=new Ogrenci("İlhan ÖZKAN");
        kisi[1]=new Ogretmen("Hasan YILMAZ");
        kisi[2]=new Mudur("Recep AY");
        for (int i=0; i<3; i++)
                kisi[i]->raporYaz();
        /*
        Her kişiye aynı "raporYaz()" mesajı veriliyor->
        Karşılığında farklı davranışlar sergileniyor.
        */
        delete kisi[0];
        delete kisi[1];
        delete kisi[2];
}
/* Program Çıktısı:
İlhan ÖZKAN:Öğrenci Ödevilerine İlişkin Rapor Yazıyor...
Hasan YILMAZ:Ogretmen Verdiği Derslere İlişkin Rapor Yazıyor...
Recep AY:Müdür Akademik Takvime İlişkin Rapor Yazıyor...
*/
```

Çok biçimlilik bize çalışma anında (run-time) üstünlük sağlar. Bu nedenle nesneler de çalışma anında imal edilmiştir. Program çalıştırıldığında her `kisi` nesnesine aynı ileti verilmiş ama bu nesneler farklı davranış göstermiştir. Yazılımcı, nesnenin tipine bakmaksızın, nesnelerden aynı davranışı göstermesini ister. Burada gösterilecek davranışın, taban sınıfın davranışı mı? olduğu veya türeyen sınıfın davranışı mı? olduğuna çalışma anında (run-time) karar verilir. İşte bu karar verme işlemine **geç bağlama** (late binding) veya **dinamik bağlama** (dynamic binding) adı verilir. Esnekliğin istenmediği bazı durumlarda ise karar verme işlemi derleme anında (compile-time) yapılır. Buna ise **statik bağlama** (static binding) adı verilir. İkisi arasındaki seçim programcı tarafından yapılır.

Yazılım yapılacak sistemin başlangıç koşullarına bağlı tasarım kararlarının tümü bilinemez. Ayrıca **sistemin genişlemesi için gerekli tüm kodlamanın bitirilmesi beklenemez**. İşte bunu gerçekleştiren **dinamik bağlama**, yazılım mimarisinin daha esnek (mevcut bileşenin sistemde yeniden yapılandırmasının kolayca yapılması) ve genişleyebilir (yeni bileşenlerin kolayca sisteme eklenmesi) olmasını sağlar.

11.21 Sınıf Çeşitleri

Soyut Sınıf (abstract class): Bu sınıfların bir örneği (instance) olamaz. Yani bu sınıftan nesne imal edilemez. Bu sınıfın en az bir soyut yöntemi olur. Soyut yöntemler, **saf sanal yöntemlerdir** (pure viryual method). Soyut sınıflar hangi davranışın gösterileceğini belirler ama bu davranışların nasıl gösterileceğini anlatmaz. Bu nedenle soyut yöntem için gövde tanımlanmaz.

```
#include <iostream>
#include <string>
class SoyutKisi { //Soyut Kisi Sınıfı
public:
        virtual void raporYaz() = 0;  // Bu saf sanal yöntem sınıfı soyut yapar.
        std::string getAdi() {
                return adi;
        }
        void setAdi(std::string pAdi) {
                if (pAdi!="") this->adi=pAdi;
        }
        SoyutKisi(std::string pAdi): adi(pAdi) {}
private:
        std::string adi;
};
int main() {
        // SoyutKisi soyutkisi; //HATA: Soyut sınıftan nene imal edilemez!
}
```

Somut sınıf (concrete class): Bu sınıflardan nesne imal edilebilir yani örneği (instance) olabilir. Bu sınıflarda

davranış ve durumlar eksiksiz tanımlıdır. Bir sınıf tanımlarken soyut bir sınıfa genel davranış ve durumları toplamak ve ondan miras alarak somut sınıfları tanımlamak her zaman iyi bir seçimdir.

```
#include <iostream>
#include <string>
class SoyutKisi { //Soyut Kisi Sınıfı
public:
        virtual void raporYaz() = 0;  // Bu saf sanal yöntem sınıfı soyut yapar.
        std::string getAdi() {
                return adi;
        }
        void setAdi(std::string pAdi) {
                if (pAdi!="") this->adi=pAdi;
        }
        SoyutKisi(std::string pAdi): adi(pAdi) {}
private:
        std::string adi;
};
class SomutKisi: public SoyutKisi { //Ortak davranışlar SoyutKisi sınıfından devralınıyor.
public:
        void raporYaz() override {
                std::cout << getAdi() <<": Rapor Yazıyor..." << std::endl;
        }
        // Bu sınıfta sanal yöntem olmadığından nesne imal edilebilir.
        SomutKisi (std::string pAdi):SoyutKisi(pAdi){}
};
int main() {
        SomutKisi ilhan("Ilhan");
        ilhan.raporYaz(); // "Ilhan: Rapor Yazıyor..."
}
```

Taban sınıf (base class): Bu sınıf, miras alınan sınıf veya genel özellik ve davranışların toplandığı yani genelleştirmenin yapıldığı **genel sınıf** (general class) veya **ebeveyn sınıftır** (parent class).

Türemiş sınıf (derived class): Bu sınıf, miras alan sınıf veya davranış ve durumlar hakkında uzmanlaşmış yani **uzman sınıf** (special class) veya **çocuk sınıftır** (child class). Türemiş ya da miras almış sınıf.

Kök sınıf (root class): Bu sınıf, babası olmayan ya da yetim sınıf olup aynı zamanda baba sınıftır.

Yaprak sınıf (leaf class): Bu sınıf, kendisinden henüz miras alan bir sınıf olmayan bir sınıftır. Kısaca türememiş sınıftır.

Tekil sınıf (singleton class): Bu sınıftan yalnızca bir nesne imal edilebilir. Yani yalnızca bir örneği vardır. Bunun olabilmesi için mahrem (private) yapıcısı olması gerekir. İleride incelenecektir.

Yardımcı sınıf (utility class): Bu sınıf, üyeleri statik olan sınıflardır. Ayrıca evrensel yapılandırma ayarlarını (configuration settings) veya değişmezleri (literal) depolamak için kullanılabilir. Bir kaynak havuzunu (önbellek, veri tabanı bağlantı havuzu, vb.) yönetmek ve örnekler arasında paylaşılan bir günlük sistemi uygulamak için yararlıdır. Bunların dışında, izleme yöntemi (trace method) çağrıları için de kullanılabilir.

Kısır sınıf (sealed class): Bu sınıf, kendisinden türeyen bir sınıf tanımlayamayan bir sınıftır. Kısaca türeyemeyen bir sınıftır. C++ diline 2011 yılında bunu yapabilmek için `final` anahtar kelimesi eklenmiştir.

```
#include <iostream>
class Dikdortgen final { //Somut Dikdirtgen Sınıfı
public:
        virtual float alanHesapla() {
                return en*boy;
        };
        Dikdortgen(float pEn,float pBoy): en(pEn),boy(pBoy) {}
private:
        float en, boy;
};
class Kare: public Dikdortgen {  //HATA: Dikdörtgen sınıfı final ile tanımlanmış
public:
        Kare(float pEn) : Dikdortgen(pEn, pEn) {}
};
```

```
int main() {
        Dikdortgen dikdortgen(4.0,5.0);
        std::cout << "Dikdörgenin alanı:" << dikdortgen.alanHesapla() << std::endl;
}
```

final anahtar kelimesi bir sınıfta sadece yöntemler için de kullanılabilir. Bu durumda türeyen sınıfta bu yöntem geçersiz kılınamaz (override);

```
#include <iostream>
class Dikdortgen { //Somut Dikdirtgen Sınıfı
public:
        virtual float alanHesapla() {
                return en*boy;
        };
        virtual float kisaKenar() final {
                return (en>boy)?boy:en;
        }
        Dikdortgen(float pEn,float pBoy): en(pEn),boy(pBoy) {}
private:
        float en, boy;
};
class Kare: public Dikdortgen {
public:
        /*
        float kisaKenar() overide { //HATA: Dikdörtgen sınıfında kisaKenar yöntemi final ile
         ↪  tanımlanmış
                //...
                return 0.0f;
        }
        */
        Kare(float pEn) : Dikdortgen(pEn, pEn) {
        }
};
int main() {
        Dikdortgen dikdortgen(4.0,5.0);
        std::cout << "Dikdörgenin kısa kenarı:" << dikdortgen.kisaKenar() << std::endl;
}
```

11.22 Numaralandırma Sınıfı

Özel bir sınıf olan **numaralandırma Sınıfı** (enumaration class), C++ dilinde **kapsamlı numaralandırma** (scopped enumaration) olarak da adlandırılır. Bir grup tam sayıdan oluşan değişmezden (integral literal) oluşan numaralandırılmış kullanıcı tanımlı bir veri tipidir. Bir bütünün parçalarını tam sayılar olarak ifade eden değişmezlere kullanıcı tanımlı adlar atamak istediğinizde kullanışlıdır.

```
enum class numaralandırmakimliği {
    adlandırılmış-tam-sayı1 [=DEĞER1],
    adlandırılmış-tam-sayı2 [=DEĞER2],
    ...
};
```

Sözde kodda DEĞER1, DEĞER2 olarak belirtilenler aslında tam sayı değişmezleridir . Yani tam sayı değişmezlerdir (integer literal). Bu değişmezler büyük harfle yazılırlar ve belirtilmedikçe ilkinin değeri 0, ikincisinin değeri 1, ... şeklinde ilerler.

```
enum class Durum {
        OK = 0,     // 0
        HATA = 1,   // 1
        UYARI = 2   // 2
};

enum class Cinsiyet {
        BELIRTILMEMIS, // 0
        ERKEK,   // 1
        KADIN    // 2
```

```
};
```

Bu numaralandırma sınıfı öğelerine bir kod içerisinde yine **kapsam çözümleme işleci** (scope resolution operator) (`::`) ile erişilir. İstenirse tam sayılardan oluşan numaralandırma `int` yerine başka bir tam sayı veri tipinde (`char`, `unsigned`, `short`, ...) de oluşturulabilir. Bu durumda numaralandırma sınıfı aşağıdaki şekilde tanımlanır.

```
enum class numaralandırmakimliği: ferklı-bir-tamsayı-veritipi {
    adlandırılmış-tam-sayı1 [=DEĞER1],
    adlandırılmış-tam-sayı2 [=DEĞER2],
    ...
};
```

Cinsiyet örneğimizi aşağıdaki şekilde de yazabiliriz;

```
enum class cinsiyetKodu:char {
        BELIRSIZ = 'B',
        ERKEK = 'E',
        KADIN = 'K'
};
```

Farklı bir tiple numaralandırma sınıfı oluşturulmuş ise kullanılırken `static_cast` işleci ile tip dönüşümü yapılmalıdır. Tip dönüşümleri ileride anlatılacaktır. Aşağıda buna ilişkin bir örnek verilmiştir.

```
#include <iostream>
#include <string>
enum class Cinsiyet : char {
        BELIRSIZ = 'B',
        ERKEK = 'E',
        KADIN = 'K'
};
struct Ogrenci { // C++ dilinde struct ile class arasında çok bir fark bulunmaz!
        unsigned int yas;
        Cinsiyet cinsiyet;
        float kilo;
        unsigned int boy;

        void bilgileriYazdir(const std::string& etiket) const {
                std::cout << etiket << ":" << std::endl
                        << "Yaşı: " << yas
                        << ", Cinsiyeti: " << static_cast<char>(cinsiyet) // Enum değerini char
                         ↪ olarak yazdırır
                        << ", Kilosu: " << kilo
                        << ", Boyu: " << boy << std::endl;
        }
};
int main() {
        // Nesne oluşturma ve atama
        Ogrenci ogrenci1;
        ogrenci1.yas = 19;
        ogrenci1.cinsiyet = Cinsiyet::ERKEK;
        ogrenci1.kilo = 75.5f;
        ogrenci1.boy = 180u;
        ogrenci1.bilgileriYazdir("Öğrenci 1");

        Ogrenci ogrenci2 = {25, Cinsiyet::KADIN, 55.0f, 165u};
        ogrenci2.bilgileriYazdir("Öğrenci 2");
}
/*Program Çıktısı:
Öğrenci 1:
Yaşı: 19, Cinsiyeti: E, Kilosu: 75.5, Boyu: 180
Öğrenci 2:
Yaşı: 25, Cinsiyeti: K, Kilosu: 55, Boyu: 165
*/
```

11.23 Düşün ve Çöz

- **Düşün**: Aşırı yükleme (overloading) ve varsayılan argümanlar (default arguments) ne zaman kullanılmalıdır? İkisini aynı anda kullanmak belirsizliğe (ambiguity) neden olabilir mi? Örnek verin.
- **Düşün**: Kapsülleme (encapsulation), kalıtım (inheritance) ve çok biçimlilik (polymorphism) ilkelerini gerçek dünyadan birer örnekle açıklayın. Bu üç ilke birbiriyle nasıl ilişkilidir?
- **Çöz**: Yapıcı (constructor) aşırı yüklemesinin kullanıldığı bir 'Dikdörtgen' sınıfı tasarlayın. Parametresiz yapıcı, kenar uzunlukları alan yapıcı ve kopya yapıcı tanımlayın. Nesne yıkıcıyı (destructor) ne zaman yazmak zorunlu hale gelir?
- **Düşün**: Sanal yöntem (virtual method) ile sanal olmayan yöntem arasındaki farkı veri tablosu (vtable) mekanizması üzerinden açıklayın. Çalışma zamanı çok biçimliliği (runtime polymorphism) bu mekanizma olmadan mümkün olur muydu?
- **Düşün**: Soyut sınıf (abstract class) ile arayüz (interface) kavramını karşılaştırın. C++'da arayüz nasıl simüle edilir? Saf sanal fonksiyon (`= 0`) bildirmenin pratik etkisi nedir?
- **Çöz**: `this` göstericisinin amacını açıklayın. Bir yöntem içinde `return *this;` talimatını döndürmenin ne işe yaradığını, zincirleme yöntem çağrısı (method chaining) örneğiyle gösterin.
- **Düşün**: Statik sınıf üyeleri `static` neden "sınıfa ait" olarak tanımlanır?

12. Bölüm

Tip Dönüşümleri ve Lamda İfadeleri

12.1 Tip Dönüşümü Nedir?

Tip dönüşümü, bir veri tipini anlamını kaybetmeyecek şekilde başka uyumlu tipe dönüştürmek anlamına gelir.

```
1 #include <iostream>
2 int main() {
3         int i = 10;
4         char c = 'A';
5         std::cout << c << " karakterinin ASCII kodu:" << (int)c << std::endl; // A karakterinin
          ↪ ASCII kodu:65
6         int toplam = i + c; /* c karakteri "char" "veri tipinden "int" veritipine dönüştürüldü ve
          ↪ i değişkeni ile toplandı ardından atama işleci ile int veri tipindeki toplam
          ↪ değişkenine atandı */
7         std::cout << "Toplam:" << toplam; //75
8 }
```

Yukarıda verilen kodda iki tane tip dönüşümü yapılmıştır;

1. 5. Satırda konsola karakter kodunu yazdırmak için (`(int)c`) ifadesi ile **kasıtlı tip dönüşümü** (explicit type casting) yapılmıştır.
2. 7. Satırda ise `i + c` ifadesinde `int` veri tipinde `i` değişkenini `char` veri tipinde `c` değişkeniyle toplamak için `c` değişkeni bellekte daha fazla yer kaplayan `int` veri tipine **üstü kapalı tip dönüşümü** (imlpicit type casting) yapılır. Daha sonra toplama işlemi yapılır.

12.2 Üstü Kapalı Tip Dönüşümleri

C++ dilinde **üstü kapalı veri tipi dönüşümü** (implicit type casting), **zorlama** (coercioncoercion), **standart tip dönüşümü** (standart type casting) veya **otomatik tip dönüşümü** (automatic type conversion) olarak da bilinir. Bu tip dönüşümünde programcının açık talimatlar vermesine gerek kalmadan derleyici bir veri tipini otomatik olarak başka bir uyumlu veri tipine dönüştürür. Şu durumlarda gerçekleşir;

- ◇ Dönüşüm farklı veri tiplerinin değerleri üzerinde gerçekleştirilir.
- ◇ Farklı bir veri tipi bekleyen bir fonksiyona bir argüman geçirmeniz halinde gerçekleşir.
- ◇ Bir veri tipinin değerini başka bir veri tipinin değişkenine atama durumunda gerçekleşir.

> Üstü kapalı tip dönüşümde bellekte az yer kaplayan veri tipinden tanımlanmış değişkenler, bellekte çok yer kaplayan değişkenlere atanmaya çalışıldığında otomatik olarak çok yer kaplayanın veri tipine dönüştürülür (widening casting).
>
> `bool`→`char`→`short int`→`int`→`unsigned int`→`long`→`unsigned`→`long long`→`float`→`double`→`long double`

```
int main() {
        char c;   //1 bayt
        int i;    //4 bayt
        long l;   //8 bayt
        float f;  //4 bayt
        double d; //8 bayt
        //...
        c=65; // 65 int değişmezi char veritipine dönüştürürlür.
        i=c;  //int veri tipine atanmadan önce; char, int veritipine dönüştürülür.
        l=i;  //long veri tipine atanmadan önce; int, long veritipine dönüştürülür.
        f=12.50f; // 12.50 float değişmez olduğundan tip dönüşümü yapılmaz.
        d=f; //double veri tipine atanmadan önce; float, double veritipine dönüştürülür.
        //...
}
```

Ayrıca Üstü kapalı tip dönüşümde bellekte çok yer kaplayan veri tipinden tanımlanmış değişkenler, bellekte az yer kaplayan değişkenlere atanmaya çalışıldığında, tip dönüşümü otomatik olarak yapılır ancak **veri kaybı söz konusu olur** (narrowing casting).

```
int main() {
        char c;
        int i;
        float f;
        i=4095; //i=0x00000FFF -> 0x00,0x00,0x0F,0xFF olarak 4 bayt
        c=i; /* c=0xFF -> c değişkenine int veritipinden char veritipine en anlamsız baytı
         ↪  atanır.*/
        f=12.50f;
        /* Kayan noktalı bir sayıdan tam sayıya atama yapılıyor ise tam kısmı tam sayıya çevrilir
         ↪  ve sonrasında atama yapılır. */
        i=f; //i=12
        //...
}
```

12.3 Bilinçli Tip Dönüşümü

Bazen belli işlemleri daha kontrollü yapmak amacıyla programcı yapılan işlemde işlenen veri tipini bir başka ver tipine kasıtlı olarak değiştirir. Bu tür dönüşümüne **bilinçli tür dönüşümü** (explicit type casting) denir.

C dilinde **tekli tip dönüştürme işleci** (unary cast operator) ile yapılır.Diğer tekli işleçler gibi ifadenin (expression) önünde kullanılır ve parantez içerisinde dönüştürülmek istenen veri tipi yazılarak `int j=(int) 12.8/4;` şeklinde kodlanır. Bu dönüşümler C dilinden gelen bilinçli dönüştürmedir.

§12.3.1 Derleme Zamanı Kasıtlı Tip Dönüşümü

Modern C++'ta daha güvenli ve ne yapıldığı daha net gösteren `static_cast` işleci (static_cast operator) tercih edilir. Bu da derleme zamanında (compile-time) hataların yakalanmasına izin verir. Yani derleme zamanı tip dönüşümleri için kullanılır. Aşağıda buna ilişkin bir örnek program verilmiştir;

```
#include <iostream>
int main() {
        float x = 9.9f; // 'f' soneki değişmezin bir float olduğunu netleştirir
        int y = x + 3.3; /* Otomatik (Implicit) dönüşüm: 9.9 + 3.3 = 13.2, int'e atanırken 13
         ↪  olur. */

        int z = static_cast<int>(x) + 3.3; /* C++ Tarzı Kasıtlı (Explicit) dönüşüm: static_cast
         ↪  kullanımı: Önce x tam sayıya (9) çekilir, sonra 3.3 eklenir (12.3), int'e atanırken
         ↪  12 olur. */

    double d = 1.2;
        int s = static_cast<int>(d + 1); /* C++ Tarzı Kasıtlı (Explicit) dönüşüm: static_cast
         ↪  kullanımı: Önce d tam sayıya (1) çekilir, sonra 1 tamsayısı eklenir (2), int'e
         ↪  atanırken 2 olur. */

        std::cout << "x: " << x << std::endl;  // Çıktı: 9.9
        std::cout << "y: " << y << std::endl;  // Çıktı: 13
        std::cout << "z: " << z << std::endl;  // Çıktı: 12
        std::cout << "s: " << s << std::endl;  // Çıktı: 2
}
```

§12.3.2 Dinamik Dönüşüm

Kasıtlı tip dönüşümünde **dinamik dönüşüm** (dynamic cast), çok biçimlilik (polymorphism) ve kalıtımda (inheritance) çalışma zamanı (run-time) tip dönüşümü için kullanılır. Alt sınıfı üst sınıf veri tipine dönüştürmek için `dynamic_cast` işleci (dynamic_cast operator) kullanılır.

```
#include <iostream>
class Taban {
        public:
```

```
        virtual ~Taban() = default;
        virtual void fonksiyon() {
                std::cout << "Taban sinif fonksiyonu..." << std::endl;
        }
};
class Turemis : public Taban {
        public:
        // 'override' anahtar kelimesi hatayı önler ve okunabilirliği artırır
        void fonksiyon() override {
                std::cout << "Turemis sinif fonksiyonu geçersiz kılınmış!" << std::endl;
        }
};
int main() {
        // 1. Nesne oluşturma
        Taban* b = new Turemis();
        // 2. dynamic_cast: Taban işaretçisini Türemiş işaretçisine güvenli şekilde çevirir
        Turemis* d = dynamic_cast<Turemis*>(b);
        if (d != nullptr) { // C++11 sonrası nullptr kullanımı standarttır
                std::cout << "Donusturme basarili! d nesnesi uzerinden cagri yapiliyor:" <<
                 ↪  std::endl;
                d->fonksiyon(); // Artık Türemiş sınıfın fonksiyonu çalışır
        } else {
                std::cout << "Donusturme basarisiz!" << std::endl;
        }
        // 3. Bellek temizliği (Modern C++'ta bunu delete ile yapmak zorundayız)
        delete b;
}
```

§12.3.3 Sabit Dönüşüm

Kasıtlı tip dönüşümünde **sabit dönüşüm** (const cast), `const` veya `volatile` niteleyicilerini kaldırır veya ekler. Bunun için `const_cast` işleci (const_cast operator) kullanılır.

```
#include <iostream>
class Kisi {
        private:
        int yas;
        public:
        Kisi(int pYas) : yas(pYas) {}
        // ÖNEMLİ: Bir fonksiyonun amacı veriyi değiştirmekse, o fonksiyon 'const' olmamalıdır.
        // Ancak eğitim amaçlı const_cast kullanacaksanız:
        void yasDegistir() const {
                // 'this' üzerindeki const özelliğini kaldırarak yas alanına erişim sağlar
                const_cast<Kisi*>(this)->yas = 30;
        }
        // Getter fonksiyonları veriyi değiştirmediği için 'const' olmalıdır
        int getYas() const {
                return yas;
        }
};
// Fonksiyon parametresi değiştirilmeyecekse const int* tercih edilir, ancak burada toplama yapıp
 ↪  döndüğü için int* kalabilir.
int fonksiyon(int* ptr) {
        return (*ptr + 100);
}
int main() {
        // 1. Nesne işlemleri
        Kisi ilhan(50);
        std::cout << "Eski Yas: " << ilhan.getYas() << std::endl;
        ilhan.yasDegistir();
        std::cout << "Yeni Yas: " << ilhan.getYas() << std::endl;
        // 2. Sabit (const) bir değişken üzerinde const_cast kullanımı
        const int val = 500;
```

```
        const int* ptr = &val;
        // Bir const int* değişkenini, int* bekleyen bir fonksiyona göndermek için:
        int* ptr1 = const_cast<int*>(ptr);
        std::cout << "Fonksiyon Sonucu: " << fonksiyon(ptr1) << std::endl;
}
/*Program Çıktısı:
Eski Yas: 50
Yeni Yas: 30
Fonksiyon Sonucu: 600
*/
```

§12.3.4 Yeniden Yorumlama Dönüşümü

Kasıtlı tip dönüşümünde **yeniden yorumlama dönüşümü** (reinterpret cast), dönüştürme öncesi ve sonrası veri tipleri farklı olsa bile, bir veri tipindeki göstericiyi başka bir veri tipindeki göstericiye dönüştürmek için kullanılır. Gösterici tipinin ve göstericinin işaret ettiği verinin aynı olup olmadığı kontrol edilmez. Kısaca bu operatör, bellekteki ham bitleri (raw bits) bir tipten tamamen farklı bir tipe zorla yorumlamak için kullanılır. Bunun için `reinterpret_cast` işleci (reinterpret_cast operator) kullanılır.

```
#include <iostream>
int main() {
        // 1. Bellekte bir tam sayı (int) için yer aç ve 65 değerini ata.
        // 65 sayısı ASCII tablosunda 'A' harfine karşılık gelir.
        int* p = new int(65);
        // 2. reinterpret_cast: int* adresini char* olarak yorumla.
        // Bellekteki verinin bitlerine dokunmaz, sadece bakış açısını değiştirir.
        char* ch = reinterpret_cast<char*>(p);
        std::cout << "Integer Degeri (*p): " << *p << std::endl; // Integer Degeri (*p): 65
        std::cout << "Karakter Degeri (*ch): " << *ch << std::endl; // Karakter Degeri (*ch): A
        std::cout << "Integer Adresi (p): " << p << std::endl; // Integer Adresi (p): 0x2166a2b0
        // 3. char* yazdırırken adres görmek istiyorsak void*'a çevirmeliyiz.
        // Aksi halde cout bunu bir metin (string) sanıp yanındaki bellekleri de okur.
        std::cout << "Karakter Adresi (ch): " << static_cast<void*>(ch) << std::endl; //Karakter
        ↪  Adresi (ch): 0x2166a2b0
        // Bellek temizliği (new kullanıldığı için şart)
        delete p;
}
```

§12.3.5 Kasıtlı Tip Dönüşümü Örneği

Bir tam sayının hangi 4 bayt bellek alanından oluştuğu 4.3 başlığında anlatılmıştı. Aşağıda, tanımlanan bir tam sayının hangi baytlardan oluştuğunu gösteren bilinçli tip dönüşümlerini içeren program örneği verilmiştir;

```
#include <iostream>
#include <iomanip> // hex, dec ve setfill için
int main() {
        // 0x10203040 sayısı belleğe bayt bayt yerleşir.
        int i = 0x10203040;
        // int* adresini char* olarak yorumluyoruz (Explicit Cast)
        char* p = reinterpret_cast<char*>(&i);
        // Genel bilgi
        std::cout << "Sayi (Onluk): " << std::dec << i << std::endl;
        std::cout << "Sayi (Onaltilik): 0x" << std::hex << i << std::endl;
        std::cout << "--------------------------" << std::endl;
        // Baytları tek tek inceleyelim
        // static_cast<int> kullanıyoruz çünkü char doğrudan cout'a giderse
        // ASCII karşılığını basmaya çalışır, biz sayısal değerini istiyoruz.
        for (int n = 0; n < 4; ++n) {
                std::cout << n + 1 << ". bayt (Adres: " << static_cast<void*>(p + n) << "): "
                << "Onluk: " << std::dec << static_cast<int>(*(p + n)) << " \t"
                << "Onaltilik: 0x" << std::hex << static_cast<int>(*(p + n)) << std::endl;
        }
}
/*Program Çıktısı:
```

```
Sayi (Onluk): 270544960
Sayi (Onaltilik): 0x10203040
--------------------------
1. bayt (Adres: 0x7ffc89aaf71c): Onluk: 64         Onaltilik: 0x40
2. bayt (Adres: 0x7ffc89aaf71d): Onluk: 48         Onaltilik: 0x30
3. bayt (Adres: 0x7ffc89aaf71e): Onluk: 32         Onaltilik: 0x20
4. bayt (Adres: 0x7ffc89aaf71f): Onluk: 16         Onaltilik: 0x10
*/
```

12.4 Tip Dönüşümünün Üstünlük ve Zayıflıkları

Tip dönüşümünün üstün yönleri;

- **İşlemlerde Esneklik**: Farklı veri tiplerini içeren işlemleri gerçekleştirmede esneklik sağlar. Programcının verileri bir tipten diğerine açıkça dönüştürmesini sağlayarak karışık tip aritmetiğini ve diğer işlemleri kolaylaştırır.
- **Uyumluluk**: Farklı veri tipleri arasında uyumluluğun sağlanmasına yardımcı olur. Programcının verileri belirli bir bağlamda kullanmadan önce uyumlu bir tipe dönüştürmesini sağlayarak veri uyumsuzluğu hatalarını önler.
- **Hassasiyet Kontrolü**: Hassasiyet kontrolünün kritik olduğu durumlarda, programcının özellikle sayısal işlemlerde veri tipleri arasında dönüşüm yaparak istenen hassasiyeti açıkça belirtmesini sağlar.
- **Açıklık**: Tip dönüşümü, programcının veri tipini değiştirme niyetini belirterek kodu daha açık hale getirir. Bu, kod okunabilirliğini artırabilir ve işlenen veri tipiyle ilgili kafa karışıklığını azaltabilir.

Tip dönüşümünün zayıf yönleri;

- **Hassasiyet Kaybı**: Tip dönüştürmenin en büyük dezavantajlarından biri hassasiyet kaybı potansiyelidir. Örneğin, kayan noktalı bir sayıyı tam sayıya dönüştürürken kesirli kısım kesilir ve bu da bilgi kaybına yol açar.
- **Çalışma Zamanı Yükü**: Bilinçli tip dönüştürme genellikle program yürütme sırasında dönüştürmenin gerçekleştirilmesi gerektiğinden çalışma zamanı yüküne neden olur. Bu ek işlem, özellikle tip dönüştürmenin sık olduğu durumlarda performansı etkileyebilir.
- **Derleyici Uyarıları ve Hataları**: Yanlış veya güvenli olmayan tip dönüştürme, derleyici uyarılarına veya hatalarına yol açabilir. Örneğin, uyumsuz tipler arasında dönüştürme yapmaya çalışmak veya geçersiz dönüştürme sözdizimi kullanmak, hata ayıklaması zor olabilecek sorunlara yol açabilir.
- **Tanımsız Davranış Potansiyeli**: Bazı durumlarda, tip dönüştürme, özellikle uyumsuz tipler arasında dönüştürme yaparken veya dönüştürülen değer hedef tipin aralığının dışında olduğunda tanımsız davranışa yol açabilir. Bu, programda öngörülemez durumlara neden olabilir.
- **Kod Bakım Zorlukları**: Bilinçli tip dönüşümüne kodun bakımı ve anlaşılması, özellikle karmaşık veya iç içe ifadelerde gerçekleştirildiğinde, daha zor hale gelebilir. Bu, artan kod karmaşıklığına ve azalan okunabilirliğe yol açabilir.
- **Taşınabilirlik Endişeleri**: Bilinçli tip dönüşümüne farklı platformlar veya derleyiciler arasında daha az taşınabilir olabilir. C'de tip dönüşümünün davranışı değişebilir ve belirli veri tipi boyutları hakkındaki varsayımlar tüm ortamlarda geçerli olmayabilir.

12.5 Tip Çıkarımı

Tip çıkarımı (type inference), bir ifadenin veri tipinin otomatik olarak belirlenmesi anlamına gelir. C++ dilinin 2011 sürümünden sonra, `auto` ve `decltype` gibi anahtar sözcükler bu amaçla dile dahil edilmiştir. Böylece bir programcının tür çıkarımını derleyicinin kendisine bırakmasına olanak sağlıyor. Tür çıkarımı yetenekleriyle, derleyicinin zaten bildiği şeyleri yazmak gerekmez. Tüm tipler yalnızca derleyici aşamasında belirlendiği için, derleme için gereken süre biraz artar ancak programın çalışma süresini etkilemez.

C++ dilinde `auto` anahtar sözcüğü (auto keyword), bildirilen değişkenin tipinin ilk değer verme işleminden otomatik olarak çıkarır. Fonksiyonlar söz konusu olduğunda, dönüş tipleri `auto` ise çalışma zamanında (run-time) bu dönüş tipi belirlenir.

```
#include <iostream>
#include <typeinfo> // typeid kullanımı için gerekli
int main() {
        // auto kullanımı: Derleyici sağdaki değere bakarak tipi belirler.
        // auto a; // HATA! İlk değer atanmadığı için tip belirlenemez.
        auto x = 4;           // int
        auto y = 3.37;        // double (varsayılan)
        auto z = 3.37f;       // float ('f' soneki nedeniyle)
```

```
        auto c = 'a';          // char
        auto ptr = &x;         // int* (pointer to int)
        auto pptr = &ptr;      // int** (pointer to a pointer)
        // typeid().name() çıktıları derleyiciye (GCC, Clang, MSVC) göre değişir.
        // i = int, d = double, f = float, c = char, Pi = pointer to int vb.
        std::cout << "Degisken Tipleri:" << std::endl;
        std::cout << "x    : " << typeid(x).name()    << " (Değer: " << x << ")" << std::endl;
        std::cout << "y    : " << typeid(y).name()    << " (Değer: " << y << ")" << std::endl;
        std::cout << "z    : " << typeid(z).name()    << " (Değer: " << z << ")" << std::endl;
        std::cout << "c    : " << typeid(c).name()    << " (Değer: " << c << ")" << std::endl;
        std::cout << "ptr  : " << typeid(ptr).name()  << " (Adres: " << ptr << ")" << std::endl;
        std::cout << "pptr : " << typeid(pptr).name() << " (Pointer Adresi: " << pptr << ")" <<
        ↪  std::endl;
}
/*Program Çıktısı:
Degisken Tipleri:
x    : i (Değer: 4)
y    : d (Değer: 3.37)
z    : f (Değer: 3.37)
c    : c (Değer: a)
ptr  : Pi (Adres: 0x7ffe2d5453f4)
pptr : PPi (Pointer Adresi: 0x7ffe2d5453e8)
*/
```

C++ dilinde `auto` anahtar sözcüğü belirli bir tipe sahip bir değişken bildirimine olanak tanırken, `decltype` ifadeden (expression) tip çıkarmanıza olanak tanır; dolayısıyla `decltype`, geçirilen ifadenin tipini değerlendiren bir **tip işlecidir** (decltype operator). Kısaca `auto` bir değerden tip çıkarırken, `decltype` **bir ifadeden veya fonksiyondan tip çıkarır**.

```
#include <iostream>
#include <typeinfo> // typeid ve name() metodu için gerekli
// Örnek fonksiyonlar
int fun1() { return 10; }
char fun2() { return 'g'; }
int main() {
        // 1. Fonksiyon dönüş tiplerinden değişken tanımlama
        // fun1() çağrılmaz, sadece dönüş tipine (int) bakılır.
        decltype(fun1()) x = 0;
        // fun2() dönüş tipine (char) bakılır.
        decltype(fun2()) y = 'a';
        std::cout << "x degiskeninin tipi: " << typeid(x).name() << " (int)" << std::endl;
        std::cout << "y degiskeninin tipi: " << typeid(y).name() << " (char)" << std::endl;
        // 2. Mevcut bir değişkenden tip kopyalama
        char a = 65; // 'A'
        // b'nin tipi a ile aynı (char) olur.
        // Dikkat: a + 5 işleminin sonucu int olsa bile decltype(a) zorlayıcıdır.
        decltype(a) b = a + 5;
        std::cout << "b degiskeninin tipi: " << typeid(b).name() << " (char)" << std::endl;
        std::cout << "b degiskeninin degeri: " << b << " (ASCII 70)" << std::endl;
}
/*Program Çıktısı:
x degiskeninin tipi: i (int)
y degiskeninin tipi: c (char)
b degiskeninin tipi: c (char)
b degiskeninin degeri: F (ASCII 70)
*/
```

12.6 Lamda İfadeleri

Bir **lamda ifadesi** (lambda expression), basit fonksiyon nesneleri oluşturmak için özlü bir yol sağlar. **Lamda** adı, *Alonzo Church* tarafından 1930'larda mantık ve hesaplanabilme hakkındaki soruları araştırmak için icat edilen matematiksel bir form olan lamda hesabından gelir. Lamda hesabı, fonksiyonel bir programlama dili olan LISP'in temelini oluşturmuştur.

C++14 ile hayatımıza giren lamda ifadesi genellikle çağrılabilir bir nesne alan fonksiyonlara argüman olarak kullanılır. Lamda ifadesi, gövdesi tanımlı bir fonksiyon oluşturmaktan daha basit olabilir. Lamda ifadeleri fonksiyon nesnelerini **satır içi** (inline) tanımlamaya izin verirler.

Söz dizimi aşağıda verilen lamda ifadesi tipik olarak dört bölümden oluşur;

```
[yakalama-listesi] (parametreler) -> donus_tipi { fonksiyon-govdesi }
```

1. **[] Yakalama Listesi** (capture clause): Lambdanın içinde olduğu bloktaki yerel değişkenlere nasıl erişeceğini belirler. Hiçbir yerel değişkene erişilmeyecek ise boş bırakılabilir.
2. **() Parametre Listesi** (parameters): Normal fonksiyonlardaki gibi argüman listesidir. Parametre yoksa boş bırakılabilir.
3. **-> Dönüş tipi** (return type): Dönüş tipidir. Çoğu zaman derleyici bunu tip çıkarımı (type inference) yaparak otomatik olarak belirler, bu yüzden yazılması zorunlu değildir.
4. **{} Gövde** (body): Fonksiyonun yapacağı işlerin yazıldığı bloktur. Yapılacak bir şey yoksa boş bırakılabilir.

§12.6.1 Yakalama Listesi

Varsayılan olarak köşeli parantez içindeki `[]` yakalama listesi, içinde olduğu kapsamın değişkenlerine bir lamda tarafından erişilemez. Bir değişkeni yakalamak, onu lamda içinde, bir kopya veya bir referans olarak erişilebilir hale getirir. Yakalanan değişkenler lamdanın bir parçası haline gelir; fonksiyon argümanlarının aksine, lamda çağrılırken parametre gibi geçirilmeleri gerekmez. Yakalama listesi, lambdanın çevresindeki değişkenleri "içeriye" nasıl alacağını belirler.

1. `[x]` (**Değere Göre Belirli Değişkeni Yakalama**): Sadece `x` değişkenini kopyalayarak içeri alır. Lamda içinde `x` değişkeninin kopyasıyla çalışılır, dışarıdaki orijinal `x` değişmez.
2. `[=]` (**Değere Göre Her Şeyi Yakalama**): Kapsamdaki (scope) tüm yerel değişkenleri kopyalayarak içeri alır. Güvenlidir çünkü orijinal veriler korunur, ancak çok sayıda büyük değişkenlerin kopyalama maliyeti oluşabilir.
3. `[&]` (**Referans ile Her Şeyi Yakalama**): Kapsamdaki tüm yerel değişkenlere referans (adres) yoluyla erişir. Lamda içinde yapılan değişiklikler dışarıdaki orijinal değişkeni de değiştirir. Çok hızlıdır (kopyalama yapmaz) ama değişkenin ömrü biterse hata riski taşır.

```cpp
#include <iostream>
int main() {
        int i = 10;
        //auto lamda1 = []() { return a*9; }; // Hata: 'a' yakalanmadığı için erişilemez
        auto lamda2 = [i]() { return i*9; }; // 'a' değerinden yakalanır
        std::cout << "lamda2:" << lamda2() << std::endl; //lamda2:90
        auto lamda3 = [&i]() { return ++i; }; // 'a' referansından yakalanır
        std::cout << "lamda3:" << lamda3() << std::endl; //lamda3:11
        int a=10,b=20;
        // [a] -> Sadece a'yı kopyalar
        auto lamdaA = [a]() {
                // a = 30; // HATA! Kopyalanan değer varsayılan olarak değiştirilemez (read-only).
                std::cout << "Sadece a: " << a << std::endl;
        };
        // [=] -> a ve b'yi kopyalar
        auto lamdaHepsi = [=]() {
                // a = 30; // HATA! Kopyalanan değer varsayılan olarak değiştirilemez (read-only).
                std::cout << "Kopyalanan a+b: " << a + b << std::endl;
        };
        // [&] -> a ve b'ye referansla erişir
        auto lamdaReferansHepsi = [&]() {
                a = 100; // Dışarıdaki 'a' değişkenini doğrudan değiştirir!
        };
        lamdaReferansHepsi();
        std::cout << "Orijinal a artik: " << a << std::endl; // Çıktı: 100
        lamdaHepsi(); // Çıktı: 30 (ReferansHepsi'den önce a=10, b=20 idi.)
}
```

§12.6.2 Parametre Listesi

Lamda fonksiyonlarında parantezler `()` parametre listesidir ve normal fonksiyonlardaki ile hemen hemen aynıdır. Lamda hiçbir argüman almazsa, bu parantezler atlanabilir (tabiki lamdayı değiştirilebilir olarak bildir-

meniz gerekmediği sürece).

```
#include <iostream>
struct Sinif {
        void yontem() const {
                std::cout << "Nesne::yontem() fonksiyonu calisti!" << std::endl;
        }
};
int main() {
        int a = 100;
        Sinif nesne; // Lambdalar içinde kullanılacak nesne
        // 1. Lambda Tanımlama: [capture](parameters){body}
        // nesne nesnesini içeriye 'değer' olarak (kopyalayarak) alır.
        auto call_func1 = [nesne]() {
                nesne.yontem();
        };
        // 2. Parametre parantezleri boşsa opsiyoneldir (C++11 ve sonrası)
        auto call_func2 = [nesne] {
                nesne.yontem();
        };
        // 3. Değişkeni yakalayan bir lambda örneği (call_b için)
        auto lambda_b = [a]() {
                return a + 50;
        };

        int call_b = lambda_b(); // Lambda fonksiyonunu çağırıyoruz
        // Fonksiyonları çağıralım
        call_func1();
        call_func2();

        std::cout << "call_b degiskeninin degeri (a + 50): " << call_b << std::endl;
}
/*Program Çıktısı:
Nesne::yontem() fonksiyonu calisti!
Nesne::yontem() fonksiyonu calisti!
call_b degiskeninin degeri (a + 50): 150
*/
```

Parametre listesi gerçek tipler yerine yer tutucu tipi olan `auto` tipini kullanabilir. Bunu yaparak, bu argüman bir fonksiyon şablonunun şablon (template) parametresi gibi davranır. Aşağıdaki lamdalar, genel kodda bir vektörü (vector) sıralamak istediğinizde eşdeğerdir. Şablon ve dinamik dizi olan vektörü kavramını iletide anlatacağız.

```
#include <iostream>
#include <vector>
#include <algorithm> // std::sort için gerekli
int main() {

        std::vector<int> sayilar = {45, 12, 89, 33, 7};
        std::vector<std::string> kelimeler = {"C++", "Python", "Java", "Assembly"};
        // 1. C++11 Tarzı Lambda: Tip açıkça belirtilmelidir (Örn: int)
        auto sort_cpp11 = [](const int& lhs, const int& rhs) {
                return lhs < rhs;
        };
        // 2. C++14 Tarzı (Generic) Lambda: 'auto' sayesinde her tipi kabul eder
        auto sort_cpp14 = [](const auto& lhs, const auto& rhs) {
                return lhs < rhs;
        };
        // --- Uygulama ---
        // Sayıları C++11 lambdası ile sıralayalım
        std::sort(sayilar.begin(), sayilar.end(), sort_cpp11);
        // Kelimeleri C++14 generic lambdası ile sıralayalım (Aynı lambda int için de çalışırdı!)
        std::sort(kelimeler.begin(), kelimeler.end(), sort_cpp14);
        // Sonuçları Yazdıralım
```

```
        std::cout << "Siralanmis Sayilar: ";
        for(int n : sayilar)
                std::cout << n << " ";
        std::cout << std::endl <<"Siralanmis Kelimeler: ";
        for(const auto& s : kelimeler)
                std::cout << s << " ";
}
/*Program Çıktısı:
Siralanmis Sayilar: 7 12 33 45 89
Siralanmis Kelimeler: Assembly C++ Java Python
*/
```

§12.6.3 Gövde

Lamda ifadelerinde süslü parantez içinde yer alan gövde, normal fonksiyonlardaki ile aynı olan gövdedir. Bir lamda ifadesinin sonuç nesnesi için, diğer fonksiyon nesnelerinde olduğu gibi `return` talimatı kullanılır.

```
#include <iostream>
int main() {
        int multiplier = 5;
        // [&multiplier] -> multiplier değişkenine bir referans (adres) tutar.
        auto timesDynamic = [&multiplier](int a) {
                return a * multiplier;
        };
        std::cout << "Ilk sonuc: " << timesDynamic(2) << std::endl; // 2 * 5 = 10
        multiplier = 15; // Orijinal değişkeni güncelliyoruz
        // Lambda referans tuttuğu için güncel değeri görür.
        std::cout << "Guncel sonuc: " << timesDynamic(2) << std::endl; // 2 * 15 = 30
}
```

Bazı durumlarda varsayılan olarak, bir lamda ifadesinin dönüş tipi türetilir. Aşağıdaki durumda dönüş tipi `bool` olur.

```
        auto f=[](){ return true; };
```

Aşağıdaki söz dizimini kullanarak dönüş tipi belirtebilirsiniz:

```
        auto f=[]() -> bool { return true; };
```

Dönüş tipi belirlenirken şu durumlar göz önüne alınmalıdır;

```
#include <iostream>
int main() {
        auto d1 = [](int value) {
                return value > 10; // Return bool
        };
        auto d2 = [](int value) {
                if (value < 10) {
                        return 1; // Hata: lamda dönüş tipini belirleyemedi int?
                } else {
                        //return 1.5; // Hata: lamda dönüş tipini belirleyemedi double?
                }
        };
        auto d3 = [](int value) -> double {// Dönüş tipi 'double'
                if (value < 10) {
                        return 1;
                } else {
                        return 1.5;
                }
        };
}
```

§12.6.4 Lamda Örnekleri

Lamdadaki değerle yakalanan nesneler varsayılan olarak değişmez nesnelerdir (immutable object). Bunun nedeni, oluşturulan kapatma nesnesinin varsayılan olarak const olmasıdır.

```
        auto func = [c = 0](){++c; std::cout << c;}; /* ++c lamda'nın durumunu değiştirmeye
        ↪  çalıştığı için derlenemez. */
```

Değiştirilmeye, `mutable` anahtar sözcüğü kullanılarak izin verilebilir;

```
auto func1 = [c = 0]() mutable {++c; std::cout << c;};
auto func2 = [c = 0]() mutable -> int {++c; std::cout << c; return c;};
```

Aşağıda ana fonksiyonun lamda olarak düzenlenmiş haline bir örnek verilmiştir;

```
#include <iostream>
#include <iomanip> // setw() için gerekli
// C++11 Trailing Return Type kullanımı (auto ... -> int)
auto main() -> int
{
        // Sabitleri 'const' veya 'constexpr' ile tanımlamak iyi bir pratiktir
        int constexpr satir = 3;
        int constexpr sutun = 4;

        // Matris tanımı
        int const matris[satir][sutun] =
        {
                { 1,  2,  3,  4},
                { 5,  6,  7,  8},
                { 9, 10, 11, 12}
        };

        // Matrisi ekrana yazdıran döngüler
        for (int y = 0; y < satir; ++y) {
                for (int x = 0; x < sutun; ++x) {
                        std::cout << std::setw(4) << matris[y][x];
                }
                std::cout << std::endl;
        }
}
```

Bir başka örnek olarak klavyeden eleman sayısı girilen bir diziyi sıralama verilebilir;

```
#include <iostream>
#include <algorithm> // std::sort
#include <vector>    // Modern dinamik dizi için
// Trailing return type kullanımı (Okunabilirlik için güzel bir tercih)
auto int_from_cin() -> int {
        int x;
        if (!(std::cin >> x)) return 0; // Hatalı girişe karşı basit bir koruma
        return x;
}
auto main() -> int {
        std::cout << "Girilen Tam Sayilari Siralama..." << std::endl;
        std::cout << "Kac adet sayi gireceksiniz: ";
        int const n = int_from_cin();
        // MODERN YAKLAŞIM: 'new int[n]' yerine 'std::vector' kullanımı.
        // Vector, kapsam dışına çıktığında belleği OTOMATİK temizler (delete[] gerektirmez).
        std::vector<int> a(n);
        for (int i = 0; i < n; ++i) {
                std::cout << i + 1 << ". elamani girin: ";
                a[i] = int_from_cin();
        }
        // std::sort artık vector'ün begin ve end iterator'ları ile çalışır
        std::sort(a.begin(), a.end());
        std::cout << "Siralanmis Dizi: ";
        for (int x : a) { // Modern 'range-based' for döngüsü
                std::cout << x << ' ';
        }
        std::cout << std::endl;
}
/*Program Çıktısı:
Girilen Tam Sayilari Siralama...
```

```
Kac adet sayi gireceksiniz: 5
1. elamani girin: 100
2. elamani girin: 10
3. elamani girin: 30
4. elamani girin: 20
5. elamani girin: 90
Siralanmis Dizi: 10 20 30 90 100
*/
```

12.7 explicit Sıfatı

C++ dilinde `explicit` sıfatı (explicit specifier), tiplerin üstü kapalı olarak dönüştürülmemesi için yapıcıları (constructor) nitelendirmek için kullanılır. Aşağıdaki kodu göz önüne alalım;

```
1  #include <iostream>
2  class Karmasik {
3  private:
4          double gercek;
5          double sanal;
6  public:
7          // Default parametreli yapıcı (Aynı zamanda Converting Constructor)
8          Karmasik(double r = 0.0, double i = 0.0) : gercek(r), sanal(i) {}
9          // İYİLEŞTİRME 1: Parametreyi 'const reference' (&) olarak almak kopyalamayı önler.
10         // İYİLEŞTİRME 2: Fonksiyonun sonuna 'const' eklemek, bu fonksiyonun nesneyi
           ↪ değiştirmediğini garanti eder.
11         bool operator==(const Karmasik& c) const {
12                 return (gercek == c.gercek && sanal == c.sanal);
13         }
14 };
15 int main() {
16         Karmasik k(3.0, 0.0);
17         // std::cout ve std::endl kullanımı (namespace belirtilmediği için)
18         if (k == 3.0)
19                 std::cout << "Ayni" << std::endl; // Çıktı bu olur.
20         else
21                 std::cout << "Farkli" << std::endl;
22 }
```

Yukarıdaki örneği 12. satırındaki kontrol ifadesindeki `&&` işlecinin solundaki ifade doğru ise sağdakine bakılmaz. Bu nedenle program çıktısı `Aynı` olacaktır. Bunu önlemek için `explicit` anahtar kelimesi **yapıcı önünde kullanılır**.

```
        explicit Karmasik(double r = 0.0, double i = 0.0) : gercek(r), sanal(i) { }
```

Bu durumda program derlenince `no match for 'operator==' (operand types are 'Karmasik' and 'double')` derleme hatası alınır. Hala `double` değerlerini `Karmasik` tipe dönüştürebiliriz, ancak bu **durumda açıkça tip dönüşümü yapmamız gerekir**;

```
        if (k == (Karmasik) 3.0)
                std::cout << "Aynı";
        else
                std::cout << "Farklı";
```

12.8 Takma İsimler

C Dilinde halihazırda mevcut veri tiplerinin adını yeniden tanımlamak ya da **takma isim** (typedef) verilebilir;

```
typedef mevcutisim takmaisim;
```

Aşağıda takma isim verilmiş yeni veri tipleriyle yazılmış bir kod örneği verilmiştir;

```
#include <iostream>
typedef char karakter;
typedef int tamsayi;
typedef long long buyukolceklitamsayi;
typedef unsigned int pozitiftamsayi;
int main() {
```

```
        karakter k='A';
        tamsayi t=12;
        buyukolceklitamsayi bot=0x0FFFFFFF;
        pozitiftamsayi pt=1;
        std::cout << k << "," << t << "," << bot << "," << pt << std::endl;
}
```

Bunun dışında fonksiyonlara da takma isim verilebilir;

```
typedef int Fonk(int); /* Fonk bir fonksiyona verilen bir takma isimdir; Fonk, int tipinde
↪ parametre alan ve int geri döndüren her fonksiyon olabilir */
int faktoriyel(int n) { //faktoriyel fonksiyonu da Fonk tipindedir.
        return (n==1)? 1: n*faktoriyel(n-1);
}
int onKat(int i) {
        return i*10;
}
int main() {
        Fonk *fn; // fn bir Fonk tipinde olan fonksiyonlara olan göstericidir.
        fn = &faktoriyel; // faktoriyel fonksiyonunu göster
        fn(3); // 3! Hesaplanır
        fn = &onKat; // onKat fonksiyonunu göster
        fn(4); // 4*10 hesaplanır
}
```

Modern C++ standartlarında(C++11/14/17), `typedef` **anahtar kelimesi kullanılmaz**. Bunun yerine ileride göreceğimiz `using` anahtar kelimesi kullanılır.

12.9 using Anahtar Kelimesinin Takma İsim Olarak Kullanılması

`using` Anahtar kelimesi, 10.10.4 ve 11.12 başlıklarındaki kullanımı yanında, **takma ad** (type aliasing) olarak çok uzun veri tiplerini kısaca kullanımını sağlar;

```
#include <iostream>
#include <vector> //std::vector için
namespace alanadi{
        namespace icalanadi{
                class Sinif {
                        private:
                        int durum;
                        public:
                        Sinif(int pDurum):durum(pDurum) {}
                        int getDurum() {return this->durum; }
                };
        }
}
// typedef yerine 'using' kullanımı (Modern C++ Yaklaşımı)
using karakter = char;
using tamsayi = int;
using buyukolceklitamsayi = long long;
using pozitiftamsayi = unsigned int;
using KisacaSinif = alanadi::icalanadi::Sinif;
int main() {
        KisacaSinif nesne(100);
        std::cout  << nesne.getDurum();
        karakter k = 'A';
        tamsayi t = 12;
        buyukolceklitamsayi bot = 0x0FFFFFFF;
        pozitiftamsayi pt = 1;
        std::cout << "k: " << static_cast<int>(k) << " veya " << k << std::endl;
        std::cout << "t: " << t << std::endl;
        // Onaltılık (hex) gösterimi istersek std::hex kullanabiliriz
        std::cout << "bot (desimal): " << bot << std::endl;
```

```
    std::cout << "bot (hex): 0x" << std::hex << std::uppercase << bot << std::dec <<
    ↪  std::endl;
    std::cout << "pt: " << pt << std::endl;
    using StringList = std::vector<std::string>;
    StringList isimler; // std::vector<std::string> ile StringList aynıdır.
}
```

12.10 Düşün ve Çöz

- **Çöz**: `static_cast`, `dynamic_cast`, `const_cast` ve `reinterpret_cast` işleçlerini karşılaştırın. Her birinin güvenli ve uygun kullanım senaryolarını örneğiyle belirtin.
- **Düşün**: Daraltıcı (narrowing) dönüşüm neden tehlikeli olabilir? `int` veri tipinden `char` veri tipine dönüşümde veri kaybının yaşanacağı bir örnek gösterin ve modern C++ bunu nasıl önlemeye çalışır?
- **Düşün**: Lamda ifadelerinin söz dizimini açıklayın. Yakalama listesi `[=]`, `[&]` ve `[x]` arasındaki farkı bir örnekle gösterin. Kapatma (closure) kavramı nedir?
- **Düşün**: `auto` anahtar kelimesinin tip çıkarımını (type inference) nasıl gerçekleştirdiğini açıklayın. `auto` kullanımının kodun okunabilirliğini artırdığı ve azalttığı durumları örnekleyin.
- **Düşün**: `explicit` sıfatı olmayan ve olan yapıcı fonksiyonlar arasındaki farkı gösterin. Örtülü tip dönüşümü ne zaman istenmeyen sonuçlara yol açar?

13. Bölüm

Sınıflar Arası İlişkiler ve SOLID İlkeleri

13.1 Unified Modeling Language

UML(Unified Modeling Language), nesneye dayalı yazılım geliştirme ile birlikte gelişen bir modelleme aracıdır. Bu nedenle 11 başlığında açıklanan nesne yönelimli birçok kavram, izleyen sayfalarda daha da pekişecektir. Adından da anlaşılacağı üzere nesneye dayalı problem çözmenin kalbi, model oluşturmaktır.

Model, gerçek dünyadaki karmaşık problemin esaslarını soyut olarak önümüze koyar. Buradan hareketle UML dilini, basit bir dil, gösterim ya da söz dizimi gibi algılamalıyız. UML dilinin, yazılımı nasıl geliştireceğimizi kesin olarak belirtmediği göz önüne alınmalıdır.

UML, bir grafik modelleme dili olup yazılım sistemlerini oluşturan ana elemanları (Ki bunlar İngilizce "artifact" olarak adlandırılıp ve el yapımı anlamına gelmektedir.) çeşitli şekillerden oluşacak şekilde tanımlamak için oluşturulmuş bir kurallar bütünüdür. Örnek verecek olursak, bir mimari projeye ilişkin maket ne ise yazılım için UML de aynı şeyi ifade eder. Yani UML, söz konusu yazılımın neyi yapacağını anlatan çeşitli şekillerin anlamlı bir şekilde bir araya getirilmesini sağlayan bir standarttır.

Yazılım mühendisliğiyle (software engineering) birlikte 1989 ve 1994 yılları arasında elliden fazla modelleme dili kullanılmaya başlanmıştır. Bu nedenle bu dönem yöntem savaşlarının yaşandığı bir dönem olarak adlandırılır. Bu yöntemlerin çoğu kendi söz dizimini oluşturmuş olmakla birlikte kullandıkları dil elemanları açısından birbirleriyle benzerlik göstermektedirler. Doksanlı yılların ortalarında üç yöntem, daha güçlü ve yaygın hale gelmiştir. Bu yöntemlerin her biri, diğerlerinin içerdiği elemanları da içeriyordu ancak her birinin diğerine karşı çeşitli güçlü özellikleri bulunuyordu. Bu yöntemler:

- Booch yönteminin tasarım (design) ve kodlama (implementation) yönleri oldukça iyi idi. *Grady Booch*, Ada diliyle oldukça çalışmıştı ve nesne yönelimli tekniklerin Ada diline uygulanması konusunda yaptığı çalışmalarla bu konuda önemli bir oyuncu olmuştur. Booch yöntemi güçlü olmasına rağmen model, birçok bulut şeklinden oluşmasından dolayı gösterim şekli oldukça sevimsizdi.
- OMT(Object Modeling Technique) tekniği, *Jim Rumbaugh* tarafından geliştirilmiş olup, analiz ve veri merkezli bilgi sistemlerinin modellenmesinde oldukça iyi bir yöntemdir.
- OOSE (Object Oriented Software Engineering) modeli **use-case** olarak bilinen bir model olup *Ivar Jacobson* tarafından geliştirilmiştir. Use-case modeli, yazılımı yapılacak sistemin davranışlarını anlamaya yönelik gelişmiş güçlü bir tekniktir.

1994 yılında *Jim Rumbaugh* ve General Electric firmasından ayrılan *Grady Booch* birlikte Rational şirketini kurarak bir araya geldiler. Bu birlikteliğin amacı, kendi yöntemlerini bir araya getirecek yeni ve tek bir yöntem olan "Unified Method" oluşturmak idi.

1995 yılında *Ivar Jacobson* da Rational şirketine katıldı. *Ivar Jacobson*'ın use-case'ler konusundaki fikirleriyle birlikte Rational şirketi, bugün UML olarak adlandırılan, yeni bir "Unified Method" geliştirdi. Üç kişiden oluşan bu grup "Three Amigos" olarak bilinir.

Yöntem savaşlarının ardından güçlü yazılım üreticileri bir araya gelerek OMG (Object Management Group) adında bir şirketler birliği kurdular[1]. Bu şirketler birliği 1997 yılında UML'e sahip çıkarak herkes tarafından kullanılan bir dil olmasını ve bir standart haline gelmesini sağlamıştır.

UML sadece, gerçek dünyadaki süreçlerin modelini ortaya koyan bir gösterim (notation) sistemidir. Süreçlerin standartlaştırılmasına yönelik bir dil değildir. UML ile ortaya çıkan modeller, süreçleri soyut (abstract) olarak tanımlayacaktır, ancak süreçlerin doğruluğu hakkında bir bilgi vermeyecektir. Yani, A şirketi için çalışan bir süreç, B şirketine uygulandığında felaketle sonuçlanabilir. Model, yukarıda da anlatıldığı üzere çözülecek probleme ilişkin bir soyutlamadır (abstraction). Etki alanı (domain) ise problemin yaşandığı gerçek dünyayı ifade eder.

Model, birbirlerine ileti (message) gönderen nesnelerden (object) oluşur. Bu modelde nesneler canlı (alive) olarak düşünülmelidir. Nesneler bir şeyler bilir (know) ve bildikleriyle bir şeyler yaparlar (do). Modelde nesneler, nesne tanımında verilen özellik (property) ve alanları (field) gösteren niteliklere (attribute) sahiptirler. Nesnelerin gösterdikleri davranış (behavior) ya da geliştirdiği yöntemler (method) ise işlem (operation) olarak adlandırılır. Nesnenin özelliklerine ait o anki değerler ise nesnenin durumunu (state) belirler. UML, sekiz tür

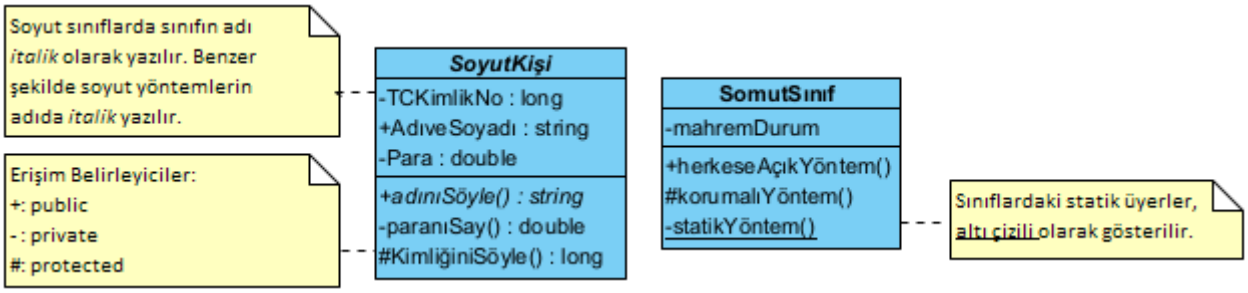
[1] OMG (Object Management Group), www.omg.org

diyagrama sahiptir ve bu kitapta **sınıf diyagramlarına** (class diagram) yer verilecektir[2].

13.2 Sınıf Diyagramları

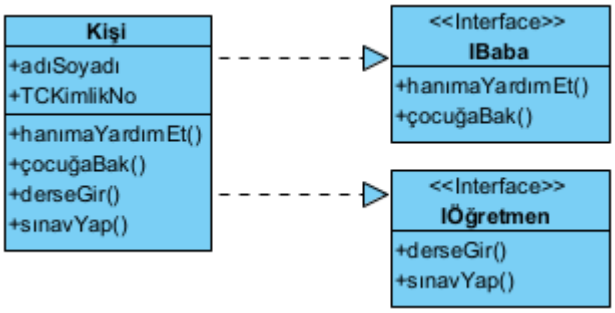
Sınıf diyagramları (class diagram), yazılımı geliştirilecek olan sisteme ait sınıfları (class) ve bu sınıflar arasındaki ilişkiyi (relationship) gösterir. Sınıf diyagramları durağandır (static). Yani sadece sınıfların birbiriyle etkileşimini (what interacts) gösterir, bu etkileşimler sonucunda ne olduğunu (what happens) göstermez.

Sınıf diyagramlarında sınıflar, bir dikdörtgen ile temsil edilir. Bu dikdörtgen üç parçaya ayrılır. En üstteki birincisine sınıf ismi yazılır. Ortadaki ikincisinde nitelikler (attribute), en alttaki ve sonuncusunda ise ve işlemler (operation) yer alır.



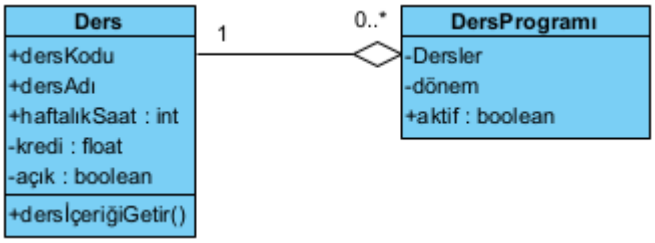
Şekil 13.1: Örnek Bir Sınıf Diyagramı

Sınıf isminin italik yazılması, sınıfın soyut (abstract class) sınıf olduğunu gösterir. Benzer şekilde işlemlerin yer aldığı kısımda, yöntemlerin italik yazılması halinde, ilgili yöntemin soyut yöntem (abstract method) olduğunu gösterir. Sınıflardaki erişim belirleyicilerin(access modifier), özel karakterlerle birbirinden ayrılır. Burada (-) karakteri mahrem (private), (+) karakteri umuma açık (public), (#) karakteri ise korumalı (protected) olduğunu anlatmaktadır. Statik (static) üyeler altı çizili olarak gösterilir. **Ara yüz gerçekleştirme** (in-



Şekil 13.2: Sınıf Diyagramlarında Arayüz Gerçekleştirme

terface realization) işlemi, ucunda üçgen olan kesikli çizgi ile gösterilir. Üçgenin sivri köşesi gerçekleştirilen ara yüzü gösterir. Çemberler, ara yüzleri (interface) göstermenin bir başka yoludur. Bu durumda daha kısa ve etkili anlatıma sahip diyagramlara sahip oluruz. Bu tip basitleştirilmiş diyagramlar **lolipop diyagramı** olarak da adlandırılır. Çokluk (multiplicity), ilişkide bir sınıfın örnek (instance) sayısını gösterir. İlişkiyi gösteren



Şekil 13.3: Sınıf Diyagramlarında Sınırlama ve Çokluk

çizginin sonunda bir numara olarak gösterilir. Bu numara, mümkün olabilecek örnek sayısıdır. Çokluk bir sonraki başlıkta verilen, **iş birliği** (association), **bileşim** (composition) ve **bütünleşmeye** (aggregation) uygulanır. Çokluk örnekleri:

- `0..1`: Sıfır ya da 1 örnek.

[2] www.togethersoft.com/services/practical_guides/umlonlinecourse/index.html

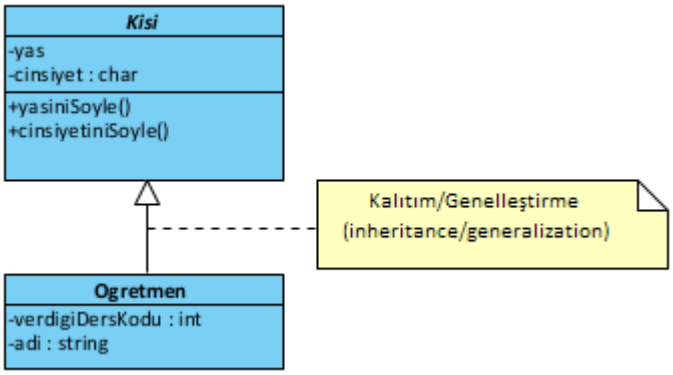
- `0..` veya `*`: Örnek sayısı sınırsızdır.
- `1`: Yalnızca 1 örnek
- `1..`: En az bir örnek

13.3 Sınıflar Arası İlişkiler

Nesneleri imal ettiğimiz sınıflar arasında ya ilişki yoktur ya da aşağıda sıralanan altı çeşit ilişki vardır.

§13.3.1 Genelleştirme İlişkisi

Genelleştirme (generalization), taban sınıfla (base class) ile türemiş sınıf (derived class) arasındaki **kalıtım** (inheritance) ilişkisidir. Örneği 11.9 başlığında verilmiştir. Aşağıda `Kisi` sınıfı ile bu sınıftan miras alan `Ogretmen` sınıfının UML diyagramı verilmiştir.

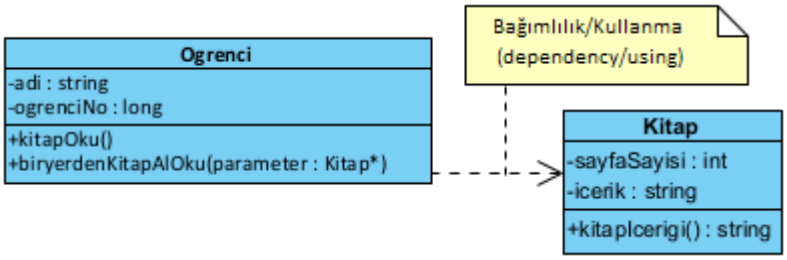


Şekil 13.4: Genelleştirme İlişkisi Örneği

§13.3.2 Bağımlılık İlişkisi

Bağımlılık (dependency) ya da bir başka deyişle **kullanma** (using), bir sınıf ile diğer arasındaki geçici ilişkidir. Burada kullanan sınıf istemci (client) diğer sınıf ise sağlayıcı (supplier) olarak adlandırılır. İki türlü bağımlılık vardır;

1. İstemci sınıf, sağlayıcıyı sınıfı **parametre olarak** kullanır (dependency as a parameter).
2. İstemci sınıf, sağlayıcı sınıftan **örnekleme** yapar (dependency as an instantiation).



Şekil 13.5: Bağımlılık İlişkisi Örneği

Aşağıda `Ogrenci`-`Kitap` ilişkisini gösteren bir bağımlılık örneği verilmiştir.

```
#include <iostream>
#include <string>
class Kitap {
public:
        std::string kitapIcerigi() {
                return icerik;
        }
        Kitap(int pSayfaSayisi, std::string pIcerik) : sayfaSayisi(pSayfaSayisi), icerik(pIcerik)
        ↪ {}
private:
        std::string icerik;
        int sayfaSayisi;
};
class Ogrenci {
public:
```

```
        void kitapOku() {  //Dependency as Instantiation
                // Yöntem içinde oluşturma (Sıkı bağlılık örneği)
                Kitap* kitap = new Kitap(150, "C++ ile Nesne Yonelimli Programlama Notlari");
                std::cout << "kitap imal edilip kullaniliyor..." << std::endl;
                std::string notlar = kitap->kitapIcerigi();
                delete kitap; // Yöntem bitmeden temizlik yapılıyor
        }
        void biryerdenKitapAlOku(Kitap* pKitap) { //Dependency as a Parameter
                // Dışarıdan enjekte edilme (Esnek bağımlılık örneği)
                if(pKitap) {
                        std::cout << "kitap disardan alinip kullaniliyor..." << std::endl;
                        std::string okunacak = pKitap->kitapIcerigi();
                }
        };
        Ogrenci(std::string pAdi, int pNo) : adi(pAdi), no(pNo) {}
private:
        std::string adi;
        int no;
};
int main() {
        Ogrenci* ogrenci = new Ogrenci("Ali", 12000);
        ogrenci->kitapOku();
        Kitap* kitap = new Kitap(350, "Kurumsal yazilim Gelistirme");
        ogrenci->biryerdenKitapAlOku(kitap);
        delete kitap;
        delete ogrenci;
}
```

Bu program, nesne yönelimli programlamada sınıflar arasındaki **bağımlılık** (dependency) ilişkisini ve bu bağımlılığın iki farklı şekilde nasıl kurulabileceğini gösteren çok temiz bir örnektir. Programda `Ogrenci` sınıfı, işini yapabilmek (kitap okumak) için `Kitap` sınıfına ihtiyaç duyar. Örnek Program, bir nesnenin başka bir nesneye olan bağımlılığını iki ana senaryoda ele almıştır;

1. **Örnekleme Olarak Bağımlılık (Dependency as Instantiation)**: `kitapOku()` yöntemi içinde, `Kitap` nesnesi metodun içinde `new` ile oluşturuluyor.
   - **Özellik**: `Ogrenci` nesnesi, hangi kitabın okunacağına kendisi karar verir ve kitabın yaşam döngüsünü (oluşturma ve silme) kendisi yönetir.
   - **Risk**: Bu durum **sıkı bağlılık** (tight coupling) yaratır. Eğer `Kitap` sınıfının yapısı (yapıcı parametreleri gibi) değişirse, `Ogrenci` sınıfının içindeki bu yöntemi da değiştirmek zorunda kalırız.
2. **Parametre Olarak Bağımlılık (Dependency as a Parameter)**: `biryerdenKitapAlOku(Kitap* pKitap)` yönteminde, kitap nesnesi dışarıdan bir parametre olarak geliyor.
   - **Özellik**: Bu, modern yazılım mimarilerinde **bağımlılık enjeksiyonu** (dependency injection) kavramının temelidir. `Ogrenci` nesnesi, kendisine hangi kitap verilirse onu okur.
   - **Avantaj**: Daha esnektir. Öğrenciye farklı kitap nesneleri (roman, ders kitabı vb.) dışarıdan kolayca gönderilebilir.

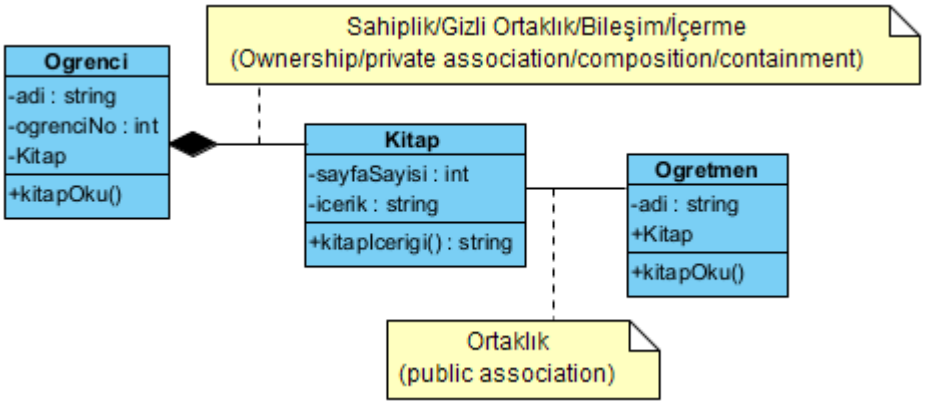
§13.3.3 Sahiplik ve Ortaklık İlişkisi

Sahiplik (ownership) ve **ortaklık** (public association) ilişkileri, iki farklı sınıf arasındaki sürekli olan bütün-parça (whole-part) ilişkileridir. Bunlar;

1. Sahiplik ilişkisinde, bir sınıf diğer sınıftan nesne örnekler (imal eder), kullanır ve öldürür. İlişkinin sürekli olabilmesi için kullanılan sınıftan bir alan değişkeni (field) mahrem (`private`) olarak tanımlanır. Bu nedenle **içerme** (containment), **gizli ortaklık** (private association) veya **bileşim** (composition) olarak da adlandırılır.
2. Ortaklık ilişkisinde, bir sınıf diğer sınıftan nesne örnekler (imal eder), kullanır ve öldürür. Ancak bu nesnenin dışarıdan değiştirilmesine izin verir. İlişkinin geçici olabilmesi için kullanılan sınıftan bir alan değişkeni (field) umuma açık (`public`) olarak tanımlanır.

Aşağıda `Kitap`, `Ogrenci` ve `Ogretmen` nesnelerinin olduğu bir ortaklık örneği verilmiştir. `Ogrenci`, `Kitap` nesnesine sahip iken `Ogretmen` nesnesi `Kitap` ile ortaklık ilişkisindedir;

```
#include <iostream>
#include <string>
class Kitap {
public:
```



Şekil 13.6: Sahiplik ve Ortaklık İlişkisi Örneği

```
        std::string kitapIcerigi() {
                return icerik;
        }
        Kitap(int pSayfaSayisi, std::string pIcerik): sayfaSayisi(pSayfaSayisi), icerik(pIcerik)
         ↪  { }
private:
        std::string icerik;
        int sayfaSayisi;
};
class Ogrenci { // private association to kitap
private:
        std::string adi;
        int no;
        Kitap* ptrKitap;
public:
        Ogrenci(std::string pAdi,int pNo): adi(pAdi),no(pNo) {
                //SAHİPLİK İLİŞKİSİ: kitap nesnesi ile ogrenci nesnesi birlikte imal ediliyor
                ptrKitap=new Kitap(150,"C ile Yapısal Programlama");
        }
        void kitapOku() {
                std::cout << "Öğrenci ile birlikte imal edilen kitap kullanılıyor..." <<
                 ↪  std::endl;
                std::cout << "Okunan Kitap:" << ptrKitap->kitapIcerigi() << std::endl;
                //...
        }
        // YIKICI: Bellek sızıntısını önler. Çünkü sınıf içinde Kitap* ptrKitap; talimatı var.
        ~Ogrenci() { delete ptrKitap; }
};
class Ogretmen { // Kitapsız öğretmen olmaz! Ama kitabını değiştirebilir.
private:
        std::string adi;
        Kitap* ptrKitap;  /* ORTAKLIK İLİŞKİSİ: public association to Kitap, setKitap() ve
         ↪  getKitap() yöntemleri ile Özellik haline getirilip umuma açık hal getiriliyor! */
public:
        Ogretmen(std::string pAdi): adi(pAdi) {
                //kitap nesnesinin referansı ile ogretmen nesnesi birlikte imal ediliyor:
                ptrKitap=new Kitap(200,"C++ ile Nesne Yönelimli Programlama");
        }
        // Kitap* üyesi get ve set yöntemleriye public hale getiriliyor.
        void setKitap(Kitap* pKitap) {
                if (ptrKitap!=nullptr) delete ptrKitap;
                ptrKitap=pKitap;
        }
        Kitap* getKitap() {
                return ptrKitap;
        }
```

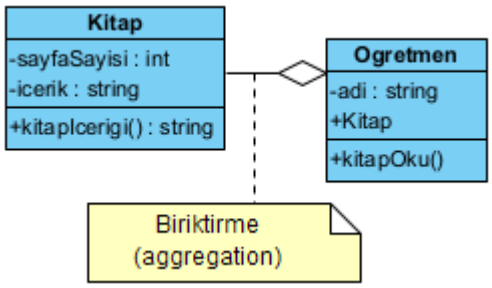
```
        void kitapOku() {
                std::cout << "Öğretmen ile birlikte imal edilen kitap kullanılıyor..." <<
                 ↪  std::endl;
                std::cout << "Okunan Kitap:" << ptrKitap->kitapIcerigi() << std::endl;
                //...
        }
};
int main() {
        Ogrenci* ogrenci=new Ogrenci("Ali",12000);
        ogrenci->kitapOku();
        Ogretmen* ogretmen=new Ogretmen("Ilhan");
        ogretmen->kitapOku();
        Kitap* ogretmeninOkuduguKitap=ogretmen->getKitap();
        std::cout << "Orgetmenin Okuduğu Kitaba Main İçinden Ulaşılabiliyor;" << std::endl
        << "Okunan Kitap:" << ogretmeninOkuduguKitap->kitapIcerigi() << std::endl;
        Kitap* baskaKitap=new Kitap(250,"Kurumsal Yazılım Geliştirme");
        ogretmen->setKitap(baskaKitap);
        ogretmen->kitapOku();
        delete ogrenci;
        delete ogretmen;
        delete baskaKitap;
}
```

Örnek program, nesne yönelimli programlamada nesneler arasındaki sahiplik (ownershipownership) ve birliktelik (association) ilişkilerini, özellikle de mahrem (private) ve umuma açık (public) erişim düzeylerinin bu ilişkileri nasıl kısıtladığını veya esnettiğini gösterir. Programdaki temel kavramları ve teknik detayları;

1. **Sahiplik** (ownership/ composition/ private association): `Ogrenci`-`Kitap` Sınıfları
   - **Durum**: `Ogrenci` nesnesi yaratıldığında `Kitap` nesnesi de onun içinde otomatik olarak `new` ile oluşturuluyor.
   - **Gizlilik**: `ptrKitap` üyesi `private` olduğu ve dışarıya açan bir `get` metodu bulunmadığı için, `main()` fonksiyonu içinden öğrencinin kitabına erişilemez veya bu kitap değiştirilemez.
   - **Yaşam Döngüsü**: Kitap, öğrenciye bağımlıdır.
2. **Ortaklık** (public association): `Ogretmen`-`Kitap` Sınıfları
   - **Durum**: Öğretmen de bir kitapla doğar, ancak `getKitap()` ve `setKitap()` yöntemleri sayesinde bu özellik (property) dış dünyaya açılmıştır.
   - **Esneklik**: `main()` fonksiyonu içinde öğretmenin okuduğu kitaba ulaşabildik (`getKitap()`) ve hatta kitabını başka bir kitap nesnesiyle değiştirebildik (`setKitap()`).
   - **Bağımsızlık**: Öğretmen nesnesi hala hayattayken, ona dışarıdan bambaşka bir kitap nesnesi **enjekte edilebilir**.

§13.3.4 Biriktirme İlişkisi

Biriktirme (aggregation) ilişkisi, iki farklı sınıf arasındaki sürekli olan bütün-parça (whole-part) ilişkisidir. Ancak **sahiplik** (ownership) ve **ortaklık** (public association) ilişkilerinden farklı olarak burada nesnelerin yaşam döngüleri birbirinden ayrılmıştır.



Şekil 13.7: Biriktirme İlişkisi Örneği

```
#include <iostream>
#include <string>
class Kitap {
public:
        std::string kitapIcerigi() { return icerik; }
```

```
        Kitap(int pSayfaSayisi, std::string pIcerik) : sayfaSayisi(pSayfaSayisi), icerik(pIcerik)
         ↪ {}
private:
        std::string icerik;
        int sayfaSayisi;
};
class Ogretmen {
private:
        std::string adi;
        Kitap* ptrKitap; /* setKitap() ve getKitap() yöntemleri ile Özellik haline getirilip
         ↪ umuma açık hal getiriliyor! */
public:
    /* BİRİKTİRME İLİŞKİSİ: Yapıcı yöntemde varolan bir kitabın adresi alınarak Ogretmen imal
     ↪ ediliyor. Kitap olmadan asla imal edilemez! */
        Ogretmen(std::string pAdi, Kitap* ppKitap) : adi(pAdi), ptrKitap(ppKitap) {}
        void setKitap(Kitap* pKitap) { ptrKitap = pKitap; }
        Kitap* getKitap() { return ptrKitap; }
        void kitapOku() {
                if (ptrKitap) { // Güvenlik kontrolü
                        std::cout << adi << " su kitabi okuyor: " << ptrKitap->kitapIcerigi() <<
                         ↪ std::endl;
                }
        }
};
int main() {
        // 1. Parça (Kitap) önce oluşturulur
        Kitap kitap1(150, "C++ ile Nesne Yonelimli Programlama");
        Kitap kitap2(200, "Veri Yapilari ve Algoritmalar");
        // 2. Bütün (Ogretmen) parçaya bağlanarak oluşturulur
        Ogretmen ilhan("Ilhan", &kitap1);
        ilhan.kitapOku();
        // 3. Parça çalışma zamanında değiştirilebilir
        ilhan.setKitap(&kitap2);
        std::cout << "Ogretmen kitabini degistirdi." << std::endl;
        ilhan.kitapOku();
}
```

Programdaki biriktirme (aggregation) ilişkisi `Ogretmen` ve `Kitap` arasında zayıf bir sahiplik ilişkisidir;

- **Bağımsız Yaşam Döngüsü**: `Kitap` nesnesi, `Ogretmen` nesnesinden önce `main()` içinde oluşturulur. Burada `Öğretmen` emekli olsa (nesne silinse) bile kitap nesnesi `main()` içinde var olmaya devam eder.
- **Zorunlu Varlık**: `Ogretmen` yapıcısı olan `Ogretmen(..., Kitap* ppKitap)` bir kitap adresi beklediği için, sistemde bir kitap olmadan öğretmen nesnesi imal edilemez.
- **Esneklik**: `setKitap()` yöntemi sayesinde `Ogretmen` elindeki kitabı bırakıp başka bir kitabı alabilir.

13.4 Sınıf Tasarlama İlkeleri

Nesne yönelimli programlama (object oriented programming) kodumuzda değişim yönetimi (change management), gösterici kullanımı yerine referansları kullanarak güvenli kod (safe code) aynı kodları tekrar yazmak (duplicate code) yerine kodların yeniden kullanımı (reusing) sağlayan aşağıdaki özellikleri kullanırız;

- Kod küçüldüğü için kalıtım (inheritance),
- Veri ve kontrol soyutlaması (abstraction) yaparak daha güvenli bileşen tasarladığımız için sarmalama (encapsulation),
- Yazılım mimarisinin daha esnek ve genişleyebilir olması için çok biçimlilik (polymorphism).

Ayrıca yazılım geliştirmede;

- Zayıf bağlaşımın (loose couple) sağlanması için yazılımın bir noktasında olabilecek değişikliğin, diğer kısımlarda değişiklik gerektirmeyecek şekilde tasarlanması gerekir. Yani yapılan değişiklik, çorap söküğü gibi yazılımın diğer kısımlarını değiştirmemelidir.
- Aynı çağrışım kümesine ilişkin bütün bileşenler bir arada geliştirilmeli diğer kümelerle karıştırılmamalıdır (seperation of concern). Yani bileşenler arasında yüksek uyum (high cohesion) olmalıdır.

İşte yeniden kullanım, zayıf bağlaşım ve yüksek uyum sağlamak için programcı, nesne yönelimli programlama özellikleriyle birlikte temel ilkelere (principle) uyar. Programcı kodu tasarlarken, yazarken ve yeniden

düzenlerken bu ilkeleri aklında tutarsa; Kod çok daha temiz, genişletilebilir ve test edilebilir olur. İlkelerin İngilizce baş harflerinin alınarak **SOLID ilkeleri** (SOLID principles) olarak isimlendirilir.

§13.4.1 Tek Sorumluluk İlkesi

Bu ilke ile SRP (Single Responsibility Principle) sınıflar, tasarlanırken birden fazla işten sorumlu tutulmamalıdır. Mümkün olduğunca bir işle ilgili olarak tasarlanmalıdır. Bir sınıfın tek bir şey yapması gerektiğini ve dolayısıyla değişmesi için tek bir nedeninin olması gerektiğini belirtir.

```
// Kötü Örnek: Birden fazla sorumluluk
class Employee {
        void calculatePay() { /* ... */ }
        void saveToDatabase() { /* ... */ }
        void generateReport() { /* ... */ }
};

// İyi Örnek: Her sınıf tek sorumluluk
class Employee {
        // Sadece employee verisi
};
class PayrollCalculator {
        void calculate(const Employee& emp) { /* ... */ }
};
class EmployeeRepository {
        void save(const Employee& emp) { /* ... */ }
};
class ReportGenerator {
        void generate(const Employee& emp) { /* ... */ }
};
```

§13.4.2 Açık-Kapalı İlkesi

Bu ilke ile OCP (Open Closed Principle) sınıflar, **değişikliklere kapalı** (closed for modification) **eklentilere açık** (open for extension) şekilde tasarlanmalıdır. Bu şekilde değişikliklere açık genişleyebilir modüllere sahip oluruz. Değişikliklere kapalı olmak, mevcut bir sınıfın kodunu değiştirmemek anlamına gelirken, eklentilere açık olma yeni işlevler eklemek anlamına gelir.

```
// Eklentiler için açık, değişiklik için kapalı
class Shape {
public:
        virtual ~Shape() = default;
        virtual double area() const = 0;
};
class Circle : public Shape {
private:
        double radius_;
public:
        double area() const override {
                return M_PI * radius_ * radius_;
        }
};
class AreaCalculator {
public:
        //std::vector ve std::unique_ptr için şablonlar başlığını inceleyiniz
        double totalArea(const std::vector<std::unique_ptr<Shape>>& shapes) {
                double total = 0;
                for (const auto& shape : shapes) {
                        total += shape->area();
                }
                return total;
        }
};
```

§13.4.3 Liskov İkame İlkesi

Bu ilkede LSP (Liskov Substitution Principle) alt sınıflar türediği sınıfların yerine konulabilmelidir. Bu prensipten ilk olarak *Barbar Liskov* tarafından bahsedilmiştir. Alt sınıflar, türediği sınıf davranışlarıyla kullanılabilmelidir. Burada amaç, alt sınıfların daha çok değişebileceği göz önüne alınarak değişimlerden an az şekilde etkilenmektir.

```
// Alt sınıflar üst sınıf yerine kullanılabilmeli
class Bird {
public:
        virtual ~Bird() = default;
        virtual void eat() { /* ... */ }
};
class FlyingBird : public Bird {
public:
        virtual void fly() { /* ... */ }
};
class Sparrow : public FlyingBird {
        void fly() override { /* Can fly */ }
};
class Penguin : public Bird {
        // Uçamaz, FlyingBird'den türemez
};
```

Türemiş sınıfından bir nesneyi, Taban sınıfından bir nesne bekleyen herhangi bir yönteme parametre olarak geçirdiğimizde yöntemin herhangi bir tuhaf çıktı vermemesi gerektiği anlamına gelir. Bu beklenen davranıştır, çünkü kalıtım kullandığımızda alt sınıfın, üst sınıfın sahip olduğu her şeyi miras aldığını varsayarız. Alt sınıf davranışı genişletir ancak asla daraltmaz. Dolayısıyla bir sınıf bu ilkeye uymadığında, tespit edilmesi zor bazı kötü hatalara yol açar.

§13.4.4 Ara Yüzleri Ayırma İlkesi

Bu ilke ile ISP (Interface Segregation Principle) genel amaçlı ara yüzler yerine istemciye bağlı ara yüzler tasarlanmalıdır. Yani roller, birbirinden ayrı olacak şekilde tanımlanmalıdır. Birden fazla rol tek bir rol altında olacak şekilde düşünülmemelidir. Bu nedenle buna rollerin ayrılması prensibi RSP (Role Segregation Principle) olarak da adlandırılır.

```
// Küçük, işe özel (spesific) interface'ler
class IPrintable {
public:
        virtual void print() = 0;
};
class IScannable {
public:
        virtual void scan() = 0;
};
class IFaxable {
public:
        virtual void fax() = 0;
};
class SimplePrinter : public IPrintable {
        void print() override { /* ... */ }
};
class MultiFunctionPrinter : public IPrintable, public IScannable, public IFaxable {
        void print() override { /* ... */ }
        void scan() override { /* ... */ }
        void fax() override { /* ... */ }
};
```

§13.4.5 Bağımlılıkları Ters Çevirme İlkesi

Bu ilke DIP (Dependency Inversion Principle), sınıflarımızın somut sınıflara ve fonksiyonlara değil, ara yüzlere veya soyut sınıflara bağlı olması gerektiğini belirtir. Nesne yönelimli programlamanın en önemli ilkesidir. Aslında bu ilke ile açık-kapalı ilkesi doğrudan birbiriyle ilişkilidir.

```
// Soyutlamalara bağımlı ol, somut sınıflara değil
class IDatabase {
```

```
public:
        virtual ~IDatabase() = default;
        virtual void save(const std::string& data) = 0;
};
class MySQLDatabase : public IDatabase {
        void save(const std::string& data) override {
                // MySQL implementation
        }
};
class PostgreSQLDatabase : public IDatabase {
        void save(const std::string& data) override {
                // PostgreSQL implementation
        }
};
class UserService {
private:
        // std::unique_ptr için şablonlar başlığını inceleyiniz
        std::unique_ptr<IDatabase> database_;
public:
        UserService(std::unique_ptr<IDatabase> db):database_(std::move(db)) {}
        void saveUser(const std::string& user) {
                database_->save(user);
        }
};
```

13.5 Düşün ve Çöz

- **Düşün**: UML sınıf diyagramında birliktelik (composition) ile biriktirme (aggregation) arasındaki farkı somut bir örnekle gösterin. 'Araba–Motor' ilişkisi ile 'Üniversite–Öğrenci' ilişkisi bu iki kavramdan hangisine karşılık gelir?
- **Çöz**: SOLID ilkelerinden 'Tek Sorumluluk İlkesi'ni ihlal eden bir sınıf örneği yazın, ardından bu sınıfı ilkeye uygun hale getirin. Refactoring öncesi ve sonrası kodları karşılaştırın.
- **Düşün**: Açık-Kapalı İlkesi'ne göre bir sınıf 'genişlemeye açık, değişikliğe kapalı' olmalıdır. Bir ödeme sistemi örneği üzerinden bu ilkeyi uygulamak için kalıtım veya kompozisyon kullanarak nasıl tasarım yaparsınız?
- **Çöz**: Liskov İkame İlkesi'ni ihlal eden klasik 'Kare–Dikdörtgen' örneğini açıklayın. Bu ihlalin programın davranışını nasıl bozduğunu kodla gösterin ve çözümünü önerin.
- **Çöz**: Bağımlılıkları Ters Çevirme İlkesi neden yüksek seviyeli modüllerin düşük seviyeli modüllere bağımlı olmaması gerektiğini söyler? Bağımlılık enjeksiyonu (dependency injection) bu ilkenin pratikte nasıl uygulandığını örnekleyin.
- **Düşün**: Sıkı bağlaşım (tight coupling) ile gevşek bağlaşım (loose coupling) arasındaki farkı UML diyagramı çizerek gösterin. Gevşek bağlaşım sağlamak için hangi tasarım teknikleri kullanılabilir?

14. Bölüm
İstisna Yönetimi

14.1 Yapısal Programlamada Hata Yönetimi

Yapısal programlamada sorun olan, hata ayıklama kodu ile işlevsel kodun iç içe olması durumudur. Nesne yönelimli programlamanın getirdiği yenilikle bu sorun tarihe karışmıştır. Yapısal programlamada hatalı bir duruma yol açmamak ya da hatalı bir durumla karşılaşıldığında programdan çıkılır. Aşağıda bir C program örneği verilmiştir;

```
#include <stdio.h>
int faktoriyel(int sayi) {
        if (sayi<0) return -1;
        if(sayi <= 1) return 1;
        return sayi * faktoriyel(sayi - 1);
}
int main() {
        int okunan,deger,hata;
        do {
                printf("Bir Sayı Giriniz:");
                scanf("%d",&okunan);
                deger=faktoriyel(okunan);
                printf("%d nin faktöriyeli: %d\n", okunan, deger);
                if (deger==-1) {
                        hata=100;
                        break; // exit(100);
                }

        } while (okunan!=0);
        return hata;
}
/* Programın Çıktısı:
Bir Sayı Giriniz:-1
-1 nin faktöriyeli: -1
*/
```

Bu program örneğinde `faktoriyel` fonksiyonu hata durumunda `-1` geri döndürmektedir. Ancak gerçek hayatta bu fonksiyonun böyle bir değer döndürmesi mümkün değildir.

14.2 İstisna Yönetimi

Nesne Yönelimli Programlamada ise **istisna** (exception), bir programın yürütülmesi sırasında ortaya çıkan beklenmeyen bir sorundur. İstisna, çalışma sırasında (run-time) oluşur. Bu tür beklenmeyen durumlarda imal edilen istisna nesneleri (exception object) vardır. İstisna iki türlü ortaya çıkar;

1. **Eşzamanlı** (synchronous): Giriş verilerindeki bir hata nedeniyle bir şeylerin ters gitmesi veya programın çalıştığı mevcut veri türünü işleme durumunda oluşan istisnalar (örneğin bir sayıyı sıfıra bölmek).
2. **Zaman Uyumsuz** (asynchronous): Disk arızası, klavye kesintileri vb. gibi yazılan programın kontrolü dışındaki istisnalar.

İstisnai durumları yönetmek için `throw`, `try` ve `catch` anahtar kelimeleri kullanılır. İstisnaları işlemek için sözde kod aşağıda verilmiştir;

```
try {
 /*
 ...
 İstisnai Durumun Ortaya Çıkabileceği Kod...
 ...
 */
```

```
  throw SomeExceptionType("İstisnai Durum Açıklaması");
  /* throw an exception: istisnai bir duruma düşme */
 } catch( ExceptionName1 e1 )  {
  /* catch bloğu try bloğundaki istisnayı yakalamak ve gerekli işlemi yapmak için kullanılır. */
 }
```

Bu sözde kod şu şekilde açıklanabilir;

- **try talimatı**: İçerisinde istisnai bir duruma düşülebileceğini tahmin ettiğimiz bir kod bloğunu temsil eder. Bu bloğu bir veya daha fazla `catch` bloğu takip eder.
- **catch talimatı**: `try` bloğunda bir istisnai duruma düşüldüğünde yürütülecek bir kod bloğunu temsil eder. İstisnayı işlemek için kullanılan kod, `catch` bloğunun içine yazılır.
- **İstisnai duruma düşme**: Bir istisnai duruma `throw` anahtar sözcüğü kullanılarak da düşülebilir. Bu aynı zamanda istisna nesneleri havuzuna **bir istisna fırlatmadır** (throw an exception).
- **İstisna sonrası süreç**: Bir program bir `throw` ifadesiyle karşılaştığında hemen içinde bulunduğu `try` bloğu dışına çıkılır ve ilgili istisnayı işlemek için eşleşen bir `catch` bloğu bulmaya başlar.
- Bir `try` bloğu içinde birden fazla `throw` ifadesi bulunabilir.

Bu yeni bakış açısıyla 14.1 başlığındaki yapısal programlama faktöriyel örneği aşağıdaki gibi yazabiliriz;

```
#include <iostream>
// Faktöriyel fonksiyonu: Hata durumunda istisna havuzuna istisna nesnesi fırlatır
int faktoriyel(int sayi) {
        if (sayi < 0) {
                throw ("Negatif sayilarin faktoriyeli hesaplanamaz!");
                // Metin olarak hata nesnesi oluşturduk.
        }
        if (sayi <= 1) return 1;
        return sayi * faktoriyel(sayi - 1);
}
int main() {
        int okunan;

        do {
                std::cout << "Bir Sayi Giriniz (Cikis icin 0): ";
                std::cin >> okunan;
                if (okunan == 0) break;
                try { // Hata riski olan kodu try bloğu içine alıyoruz
                        int sonuc = faktoriyel(okunan);
                        std::cout << okunan << " sayisinin faktoriyeli: " << sonuc << std::endl;
                }
                catch (const char* e) { // Metin olarak imal edilmiş hata nesnesini yakalıyoruz.
                        std::cerr << "HATA: " << e << std::endl;
                        return -1;
                }

        } while (okunan != 0);
}
/*Program Çıktısı:
Bir Sayi Giriniz (Cikis icin 0): -5
HATA: Negatif sayilarin faktoriyeli hesaplanamaz!
*/
```

C++ dilinde istisna işleme yapısal programdakinden farklıdır. Şöyle ki;

- Hata işleme kodu, fonksiyonel koddan yani işin gereği yazılan koddan ayrılır.
- Fonksiyonlar ve yöntemler yalnızca seçtikleri istisnaları işleyebilir. Bir fonksiyon yada yöntemde birden çok istisnai duruma düşülebilir. Ancak bunlardan bazılarını `catch` bloğunda işlemeyi seçebiliriz. Geri kalanlarını yöntemi çağıran yere bırakabiliriz. Böylece istisnayı kimin işleyeceğini programcı seçebilir.
- Hem temel tiplerden (`int`, `float`, char, ...) hem de nesneler (`std::string`, ...) istisna nesnesi imal edilebilir. Böylece hata türlerinin gruplanması yapılabilir.

14.3 Standart İstisna Tipleri

Aşağıda sıfıra bölme hatasını işleyen bir program örneği verilmiştir. Örnekte standart istisna tiplerinden biri olan `runtime_error` istisnası imal edilmiştir. Bu tip istisna tipleri için projemize `<stdexcept>` başlığı dahil

edilmelidir.

```
#include <iostream>
#include <stdexcept> // runtime_error için gerekli
int main() {
        std::cout << "Burasi her zaman icra edilir." << std::endl;
        try {
                // try BLOĞU: İstisna oluşma ihtimali olan kod bloğu
                int bolunen = 10;
                int bolen = 0;
                int sonuc;
                if (bolen == 0) {
                        // std:: ön eki eklendi
                        throw std::runtime_error("Sifira Bolmeye Izin Verilmez!");

                        // throw satırından sonraki kodlar asla çalışmaz
                        std::cout << "Burasi hicbir zaman icra edilmez." << std::endl;
                }
                sonuc = bolunen / bolen;
                std::cout << "Bolme Sonucu: " << sonuc << std::endl;
        } catch (const std::exception& e) {
                // catch BLOĞU: Hata yakalandığında buraya atlanır
                std::cout << "Istisna Yakalandi: " << e.what() << std::endl;
        }
        std::cout << "Burasi da her zaman icra edilir." << std::endl;
}
/*Program Çıktısı:
Burasi her zaman icra edilir.
Istisna Yakalandi: Sifira Bolmeye Izin Verilmez!
Burasi da her zaman icra edilir.
*/
```

Yukarıdaki örnekte `try` bloğunda izin verilmeyen bölme işleminde istisnai bir durum fark ediliyor ve istisna nesnesi imal ediliyor ve **`throw` ile fırlatılıyor** (throw an exception). `catch` bloğunda ise (istisna nesneleri havuzundan) **istisna nesnesi yakalanarak (catch an exception)** işleniyor. Örnekte standart tipte istisna nesnesi kullanılmıştır.Bu tiplerin hepsi `std::exception` sınıfından türetilmiştir. Bunlar;

- `std::runtime_error`: Çalışma anında ortaya çekebilecek genel hatalar,
- `std::logic_error`: Büyüktür yenine küçüktür yazılması durumlarında ortaya çıkabilecek hatalar,
- `std::invalid_argument`: Bir fonksiyon veya yönteme istenmeyan bir argümanın gelmesi hataları,
- `std::range_error`: Dizi yada koleksiyon sınırlarının dışına taşılması hataları,
- `std::out_of_range`: Dizi yada koleksiyonlarda indisin sınırlar dışına çıkması hatası,
- `std::overflow_error`: Dizi yada koleksiyonlarda indisin üst sınırın dışına çıkması hatası,
- `std::underflow_error`: Dizi yada koleksiyonlarda indisin alt sınırın dışına çıkması hatası.

Bir `try` bloğunda birden fazla istisna nesnesi (istisna havuzuna) fırlatılabilir. Ama tüm `catch` talimatlarıyla bu istisnalar yakalanmayabilir. Ayrıca tüm istisna nesnelerin tipleri de bilinmeyebilir. Bu durumda varsayılan `catch` kullanılır Aşağıdaki buna ilişkin örnek verilmiştir..

```
1  #include <iostream>
2  int main() {
3          try {
4                  // DİKKAT: throw ile ilk istisna nesnesi oluştuğunda program hemen uygun 'catch'
                   ↪ bloğuna atlar.
5                  // Alttaki throw satırlarına asla ulaşılamaz.
6
7                  throw 10; /* Bu satır çalışır ve 13. satırdaki 'int' catch bloğuna gider. */
8
9                  // Aşağıdaki satırlar ölü koddur (unreachable code):
10                 throw "Hata";
11                 throw 'A';
12         }
13         catch (int excpInt) {
14                 std::cout << "Tam Sayi Istisnasi Yakalandi: " << excpInt << std::endl;
15         }
```

```
        catch (const char* excpStr) {
                // Modern C++'ta string literalleri 'const char*' olarak yakalanmalıdır.
                std::cout << "Metin Istisnasi Yakalandi: " << excpStr << std::endl;
        }
        catch (char excpChar) {
                std::cout << "Karakter Istisnasi Yakalandi: " << excpChar << std::endl;
        }
        catch (...) {
                // Default catch: Yukarıdakilere uymayan her şey buraya düşer.
                std::cout << "Diger Catchlerde islenmeyen bir istisna burada islendi." <<
                ↪  std::endl;
        }
}
/* Programın Çıktısı:
Tam Sayi Istisnasi Yakalandi: 10
*/
```

Aşağıda `Kisi` sınıfta `yas` özelliğine ilişkin istisna içeren bir kod örneği verilmiştir;

```
#include <iostream>
#include <stdexcept>
#include <string>
class Kisi {
private:
        int yas;
public:
        Kisi() : yas(15) {} // Yapıcı (Constructor)
        int getYas() const { // Nesneyi değiştirmediği için 'const' eklemek iyi bir pratiktir
                return yas;
        }
        void setYas(int pYas) {
                if (pYas < 0)
                throw std::range_error("Gecersiz Yas Degeri: <0!");
                else if (pYas > 120)
                throw std::range_error("Cok Buyuk Yas Degeri: >120!");
                yas = pYas;
        }
};
int main() {
        Kisi ali;
        try {
                // 1. Geçerli bir değer atama
                ali.setYas(12);
                std::cout << "Guncel Yas: " << ali.getYas() << std::endl;
                // 2. İstisna yaratacak değer (Hata fırlatıldığı an try bloğu terk edilir)
                ali.setYas(-1);
                // --- BU SATIRDAN SONRASI İCRA EDİLMEZ ---
                std::cout << "Bu satira asla ulasilamayacak: " << ali.getYas() << std::endl;
                ali.setYas(130);
                std::cout << ali.getYas() << std::endl;
        }
        catch (const std::exception& istisna) {
                // Yakalanan istisna nesnesinin içeriğini yazdırıyoruz
                std::cout << "Istisnai Bir Durum Yakalandi!" << std::endl;
                std::cout << "Durum Aciklamasi: " << istisna.what() << std::endl;
        }
}
/*Program Çıktısı:
Guncel Yas: 12
Istisnai Bir Durum Yakalandi!
Durum Aciklamasi: Gecersiz Yas Degeri: <0!
*/
```

14.4 std::exception ve what() Yöntemi

C++'ta tüm standart hata sınıfları (örneğin `std::out_of_range`, `std::bad_alloc`) `std::exception` sınıfından türetilmiştir. Bu sınıfın içinde `what()` adında sanal bir yöntem bulunur. Bu yöntemin görevi, **hatayı açıklayan bir karakter dizisi** `(const char*)` döndürmektir. Bu yöntem `catch` bloğu içinde hata nesnesi üzerinden çağrılır.

```
#include <iostream>
#include <vector>
#include <stdexcept> // Standart istisnalar için
int main() {
        std::vector<int> sayilar = {10, 20, 30};
        try {
                // Vektörün sınırları dışında bir elemana erişmeye çalışıyoruz (indis 5 yok)
                // .at() fonksiyonu hata oluşunca std::out_of_range fırlatır.
                std::cout << sayilar.at(5) << std::endl;
        }
        catch (const std::exception& e) {
                // 'e' nesnesi üzerinden what() çağrılır
                std::cout << "Hata Yakalandi: " << e.what() << std::endl;
        }
}
/*
Program Çıktısı:
Hata Yakalandi: vector::_M_range_check: __n (which is 5) >= this->size() (which is 3)
*/
```

`std::exception` ve `what()` kullanmalıyız! Çünkü;

- **Hiyerarşi**: Tüm hataları `(const std::exception& e)` referansı ile tek bir `catch` bloğunda yakalayabilirsiniz. Arka planda çok biçimlilik çalışır.
- **Bilgi Paylaşımı**: Hatanın nedenini kodun her yerinde manuel cout yapmak yerine, hata nesnesinin içinde taşırsınız.
- **Standartlara Uyumluluk**: Büyük kütüphaneler (ileride anlatacağımız STL gibi) bu yapıyı kullanır. Sizin kodunuzun da bu yapıya uyması, kodun profesyonelliğini artırır.

> İyi bir istisna yönetimi için `catch (std::exception e)` yerine `catch (const std::exception& e)` kullanın! Bu, nesnenin kopyalanmasını önler ve **nesne dilimleme** (object slicing) problemini engeller. Ayrıca her zaman önce en özel hata sınıflarını (örneğin MatematikHatasi) yakalanmalı ve en son ise genel `std::exception` sınıfını yakalanmalıdır.

14.5 noexcept Anahtar Kelimesi

C++'ta noexcept anahtar kelimesi, hem bir sıfat/belirleyici (noexcept specifier) hem de bir işleç (noexcept operator) olarak görev yapan, C++11 ve sonrası performans optimizasyonunun temel taşlarından biridir.

§14.5.1 noexcept İşleci

noexcept işleci (noexcept operator), bir ifadenin (expression) istisna (exception) fırlatıp fırlatmayacağını **derleme zamanında** (compile-time) kontrol eder. Sonuç olarak `bool` bir değer döndürür:

1. `true`: Eğer ifade istisna fırlatmayacağı garanti edilmişse.
2. `false`: Eğer ifade istisna fırlatabilirse.

Aşağıda buna ilişkin bir örnek verilmiştir;

```
#include <iostream>
#include <stdexcept>
void fonksiyon() { throw std::runtime_error("oops"); }
void bosfonksiyon() {} // Belirteç kullanılmadığı için potansiyel tehlike!
struct yapi {};
int main() {
        // 1. fonksiyon() içinde throw var, derleyici bunun fırlatabileceğini biliyor.
        std::cout << noexcept(fonksiyon()) << std::endl;      // Çıktı: 0
        // 2. bosfonksiyon() boş olsa bile 'noexcept' olarak işaretlenmediği için
        // derleyici her ihtimale karşı istisna fırlatabilir kabul eder.
```

```
        std::cout << noexcept(bosfonksiyon()) << std::endl;   // Çıktı: 0
        // 3. Temel aritmetik işlemler istisna fırlatmaz.
        std::cout << noexcept(1 + 1) << std::endl;            // Çıktı: 1
        // 4. Basit bir struct'ın varsayılan yapıcısı (constructor) istisna fırlatmaz.
        std::cout << noexcept(yapi()) << std::endl;           // Çıktı: 1
}
```

§14.5.2 noexcept Belirleyicisi

`noexcept` Belirleyicisi (noexcept specifier), bir fonksiyonun istisna fırlatmayacağını derleyiciye söz vererek bildirmek için fonksiyon imzasında kullanılır. Eğer bu sözü verirseniz, `noexcept` işleci bu fonksiyon için `true` döndürür.

```
#include <iostream>
void guvenliFonksiyon() noexcept {
        // Bu fonksiyonun hata fırlatmayacağı garanti edildi.
}
int main() {
        std::cout << noexcept(guvenliFonksiyon()); // Çıktı: 1
}
```

14.6 Kullanıcı Tanımlı İstisna Tipleri

Her zaman standart istisna nesneleri kullanmak zorunda değiliz. Bu durumda `<exception>` başlığını projemize dahil ederek kendi istisna nesnemizi de tanımlayabiliriz. Aşağıdaki örnekte profesyonel yaklaşımdan biri sergilenmiştir. `std::exception` sınıfından kendi sınıfını türetilmiş ve `what()` fonksiyonunu kendi hata mesajı geçersiz kılınmaktadır (override).

```
#include <iostream>
#include <exception>
#include <string>
// Özel hata sınıfı: Maksimum yaş sınırı için
struct yasMaxException : public std::exception {
        // C++11 ve sonrasında 'throw()' yerine 'noexcept' kullanılır
        // what() fonksiyonunu geçersiz kılıyoruz (override)
        // 'noexcept' bu fonksiyonun kendisinin hata fırlatmayacağını garanti eder
        const char* what() const noexcept override {
                return "Cok Buyuk Yas Degeri: >120!";
        }
};
// Özel hata sınıfı: Minimum yaş sınırı için
struct yasMinException : public std::exception {
        // what() fonksiyonunu geçersiz kılıyoruz (override)
        // 'noexcept' bu fonksiyonun kendisinin hata fırlatmayacağını garanti eder
        const char* what() const noexcept override {
                return "Gecersiz Yas Degeri: <0!";
        }
};
class Kisi {
        private:
        int yas;
        public:
        Kisi() : yas(15) {}
        int getYas() const { return yas; }
        void setYas(int pYas) {
                if (pYas < 0)
                throw yasMinException(); // Özel hata nesnesi fırlatılıyor
                else if (pYas > 120)
                throw yasMaxException();
                yas = pYas;
        }
};
int main() {
        Kisi ali;
```

```
        try {
                ali.setYas(12);
                std::cout << ali.getYas() << std::endl;
                // Hata burada fırlatılacak
                ali.setYas(-1);
                // Bu satırlar icra edilmez
                std::cout << ali.getYas() << std::endl;
                ali.setYas(130);
        }
        catch (const std::exception& istisna) {
                // Polimorfizm sayesinde her iki özel hata da burada yakalanır
                std::cout << "Istisnai Bir Durum Yakalandi!" << std::endl;
                std::cout << "Durum Aciklamasi: " << istisna.what() << std::endl;
        }
}
/*Program Çıktısı:
12
Istisnai Bir Durum Yakalandi!
Durum Aciklamasi: Gecersiz Yas Degeri: <0!
*/
```

14.7 Yığın Çözülmesi

Yığın Çözülmesi (stack unwinding), C++'ın bellek yönetimi ve hata güvenliği (exception safety) konusundaki en "kahraman" özelliklerinden biridir. İstisnalar fırlatıldığında arka planda gerçekleşen bu süreci anlamak, profesyonel bir C++ geliştiricisi olmak için şarttır.

Bir fonksiyon içinde `throw` ile hata fırlatıldığında, çalışma zamanında (run-time) bu hatayı yakalayacak uygun bir `catch` bloğu aramaya başlar. Bu arama sırasında, hatanın fırlatıldığı noktadan `catch` bloğuna kadar olan tüm yerel nesnelerin yıkıcıları (destructor) otomatik olarak çağrılır. İşte bu temizlik sürecine **yığın çözülmesi** (stack unwinding) denir.

Süreç şöyle işler;

1. **Hata Fırlatılır**: Bir fonksiyonda `throw` satırı çalışır.
2. **Geriye Doğru Tarama**: Program o anki fonksiyondan çıkar ve çağıran (caller) fonksiyona geri döner.
3. **Otomatik Temizlik**: Çıkılan her fonksiyondaki yerel (stack üzerindeki) nesnelerin yıkıcıları (`~ClassName`) sistem tarafından sırayla çağrılır.
4. **Catch Bloğu Bulunur**: Uygun bir `catch` bloğuna ulaşılana kadar bu süreç "yukarı doğru" devam eder. Eğer main fonksiyonuna kadar uygun bir blok bulunamazsa, `std::terminate` çağrılır ve program çöker.

Bunu aşağıdaki örneği inceleyerek anlayabiliriz;

```
#include <iostream>
#include <string>
class Kaynak {
        std::string ad;
        public:
        Kaynak(std::string n) : ad(n) { std::cout << ad << " acildi.\n"; }
        ~Kaynak() { std::cout << ad << " otomatik olarak KAPATILDI (Yikici calisti).\n"; }
};
void fonksiyonB() {
        Kaynak b_kaynagi("FonksiyonB Kaynagi");
        std::cout << "FonksiyonB icinde hata firlatiliyor...\n";
        throw std::runtime_error("Kritik Hata!"); // Hata burada firlatilir
        std::cout << "Bu satir asla calismaz.\n";
}
void fonksiyonA() {
        Kaynak a_kaynagi("FonksiyonA Kaynagi");
        fonksiyonB();
        std::cout << "FonksiyonA devam ediyor...\n"; // Buraya asla gelinmez
}
int main() {
        try {
```

```
                fonksiyonA();
        }
        catch (const std::exception& e) {
                std::cout << "\nCatch Blogu Yakaladi: " << e.what() << "\n\n";
        }
        return 0;
}
/*Program Çıktısı:
FonksiyonA Kaynagi acildi.
FonksiyonB Kaynagi acildi.
FonksiyonB icinde hata firlatiliyor...
FonksiyonB Kaynagi otomatik olarak KAPATILDI (Yikici calisti).
FonksiyonA Kaynagi otomatik olarak KAPATILDI (Yikici calisti).

Catch Blogu Yakaladi: Kritik Hata!
*/
```

Yığın çözülmesi, C++'taki RAII (Resource Acquisition Is Initialization) prensibinin temelidir ve 15.7.1 başlığında incelenecektir. Eğer bu özellik olmasaydı:

- Açık kalan dosyalar kapanmazdı.
- Kilitlenmiş (mutex) veriler kilitli kalırdı (deadlock).
- Bellek sızıntıları (memory leaks) kaçınılmaz olurdu.

Yığın çözülmesi sırasında bir nesnenin yıkıcısı (`~ClassName`) çalışırken, o yıkıcının içinden **yeni bir hata fırlatılmamalıdır!**
Eğer bir istisna işlenirken (yığın çözülmesi sürerken) ikinci bir istisna fırlatılırsa, C++ çalışma zamanı ne yapacağını bilemez ve programı anında sonlandırır (`std::terminate`).
Bu nedenle **yıkıcılar her zaman güvenli olmalı ve hataları kendi içinde yutmalıdır (noexcept)**.

İstisnalar sadece bir hata bildirme yöntemi değildir; aynı zamanda C++'ın yığın belleğindeki nesneleri güvenli bir şekilde imha etme garantisidir. `throw` ile `catch` arasındaki mesafe ne kadar uzak olursa olsun, aradaki tüm yerel nesneler usulünce yok edilir.

14.8 Düşün ve Çöz

- **Düşün**: Geleneksel hata kodu döndürme yöntemi ile C++ istisna mekanizması arasındaki farkı karşılaştırın. İstisnalar her durumda daha iyi bir çözüm müdür? Gömülü sistemlerde istisnalar neden sorunlu olabilir?
- **Çöz**: `try-catch-throw` mekanizmasını kullanarak bir bölme işleminin sıfıra bölme hatasını zarif biçimde ele alan bir program yazın. Farklı istisna tiplerini `std::exception` ve `std::runtime_error` birbiriyle karşılaştırın.
- **Düşün**: İstisna güvenliği (exception safety) kavramını açıklayın. Temel garanti, güçlü garanti ve hata atmama garantisi (nothrow guarantee) arasındaki farklar nelerdir? Her biri için örnek verin.
- **Çöz**: Özel bir istisna sınıfı (`class DivisionByZeroException`) tasarlayın. Bu sınıfı `std::exception`'dan türeterek `what()` yöntemini geçersiz kılın (override) ve kullanımını bir örnekle gösterin.
- **Düşün**: `noexcept` sıfatının anlamı nedir? Derleyici optimizasyonları açısından `noexcept` neden önemlidir? Hangi tür fonksiyonlar `noexcept` olarak işaretlenmelidir?

15. Bölüm
Şablonlar

15.1 Şablon Nedir?

Şablonlar (template), veri tiplerine bağlı kalmasan çalışan fonksiyonlar, sınıflar, algoritmalar yazmak için jenerik programlama (generic programming) ile oluşturulan yapılardır.

Jenerik programlama, kodun belirli bir veri tipine (`int`, `float`, `string` vb.) bağlı kalmadan, veri tipinden bağımsız bir şekilde yazılmasına olanak tanıyan bir yazılım geliştirme yaklaşımıdır. En temel amacı, aynı mantığa sahip işlemleri farklı veri tipleri için tekrar tekrar yazmak yerine, bir kez **şablon** (template) olarak hazırlayıp her tipte güvenle kullanabilmektir. Yani;

- Bir kez yazılan bir algoritma (örneğin sıralama veya arama), hem sayılar hem de metinler için çalışabilir. Böylece aynı kodu yeniden kullanılabiliriz (reusing).
- Kod derleme zamanında (compile-time) kontrol edilir. Hatalar program çalışmadan yakalanır. Böylece tip güvenliği (type safety) sağlanmış olur.
- Çalışma zamanında (run-time) bir tip kontrolü yapılmaz; derleyici her tip için özel bir kod üretir, bu yüzden çok hızlıdır. Böylece performans yakalamış oluruz.

Jenerik programlamada bir fonksiyon veya sınıf yazarken, tip kısmına gerçek bir isim (örneğin `int`) yerine bir yer tutucu (placeholder) koyarsınız. Genellikle bu yer tutucu için `T` harfi kullanılır.

Örnek olarak iki değeri karşılaştırıp büyüğünü bulan bir jenerik fonksiyon düşünelim. Jenerik olmayan kodda her veri tipi için bir fonksiyon yazmayı gerektirir;

```
intMax(int a, int b) {
//...
}
doubleMax(double a, double b) {
//...
}
```

Jeneric olan kodda ise tek bir şablon fonksiyon yazarsınız.

```
template <typename T> T Max(T a, T b) {
 return (a>=b)?a:b;
}
```

Derleyici, siz içine ne koyarsanız (int veya double) ona uygun gerçek kodu arka planda sizin için oluşturur. Eğer jenerik programlama olmasaydı, her yeni veri tipi için aynı fonksiyonları baştan yazmak zorunda kalırdık. Bu da hem çok fazla **kod kalabalığına** (code bloat) hem de hata yapma riskinin artmasına neden olurdu.

15.2 Foksiyon Şablonları

Jenerik programlamanın temel taşı olan fonksiyon şablonları (function templates), aynı mantığı yeniden yazmadan bir fonksiyonun farklı veri tipleri üzerinde çalışmasını sağlayan bir fonksiyon planı tanımlar. Bu plan aşağıdaki şekilde tanımlanır;

```
template <typename değişken-kimliği> foksiyon-bildirimi;
```

Aşağıdaki kodda, iki değerin yerini değiştiren (swap) bir fonksiyonu hem `int` hem `double` hem de `string` için tek bir şablonla nasıl çalıştırdığımızı görebilirsiniz.

```
#include <iostream>
#include <string>
// 'typename T' ifadesi, T'nin herhangi bir veri tipi olabileceğini belirtir.
template <typename T>
void DegerDegistir(T &a, T &b) {
        T gecici = a;
        a = b;
        b = gecici;
```

```
}
int main() {
        // 1. Tam Sayılarla Kullanım
        int x = 10, y = 20;
        DegerDegistir(x, y); // Derleyici T'yi 'int' olarak anlar.
        std::cout << "Tam Sayilar: x=" << x << ", y=" << y << std::endl;

        // 2. Ondalıklı Sayılarla Kullanım
        double d1 = 1.5, d2 = 9.9;
        DegerDegistir(d1, d2); // Derleyici T'yi 'double' olarak anlar.
        std::cout << "Ondalikli: d1=" << d1 << ", d2=" << d2 << std::endl;

        // 3. Metinlerle Kullanım
        std::string s1 = "Dunya", s2 = "Merhaba";
        DegerDegistir(s1, s2); // Derleyici T'yi 'string' olarak anlar.
        std::cout << "Metinler: s1=" << s1 << ", s2=" << s2 << std::endl;
}
/*Program Çıktısı:
Tam Sayilar: x=20, y=10
Ondalikli: d1=9.9, d2=1.5
Metinler: s1=Merhaba, s2=Dunya
*/
```

15.3 Değişken Sayıda Argüman Alabilen Fonksiyon Şablonları

Değişken sayıda argüman alabilen fonksiyon şablonları (variadic function template), herhangi bir sayıda (sıfır veya daha fazla) argüman alabilen sınıf veya fonksiyon şablonlarıdır. C++ dilinde şablonlar, yalnızca bildirim sırasında belirtilmesi gereken sabit sayıda parametreye sahip olabilir. Ancak *Douglas Gregor* ve *Jaakko Järvi* tarafından ortaya koyulan **değişken sayıda argüman alabilen şablonlar** (variadic template) bu sorunun üstesinden gelmeye yardımcı olur.

C++11 ile hayatımıza giren ve bir fonksiyona sınırsız sayıda ve farklı türde argüman göndermemize olanak tanıyan oldukça güçlü bir jenerik programlama özelliğidir. Değişken sayıda argüman alabilen şablonlar (variadic şablonlar) bir **parametre paketi** (parameter pack) kullanarak her şeyi kabul edebilir. Temel söz diziminde üç nokta (`...`) karakterleri bulunur;

```
template <typename... Args>
void fonksiyon(Args... args) {
 // Args: Şablon parametre paketi (tipler)
 // args: Fonksiyon parametre paketi (değerler)
}
```

Bu söz diziminde; `typename... Args` demek buraya sıfır veya daha fazla tip gelebilir demektir. `Args... args` ise bu tiplerden oluşan paketlenmiş değerleri al demektir.

Değişken sayıda argüman alabilen şablonlar (variadic şablonlar), 7.8 başlığında anlatılan özyinelemeli mantıkla çalışır. Paketin içindeki ilk elemanı alırız, geri kalan paketi tekrar aynı fonksiyona göndeririz. Paket boşaldığında ise durmak için bir **sonlandırıcı fonksiyon** (base case) yazarız. Örnek olarak argüman sayısı belli olmayan bir yazdır fonksiyonunu örnek verebiliriz;

```
#include <iostream>
// 1. Sonlandırıcı (Base Case): Paket boşaldığında bu çalışır.
void Yazdir() {
        std::cout << std::endl;
}
// 2. Variadic Şablon:
template <typename T, typename... Args>
void Yazdir(T ilk, Args... geriKalan) {
        std::cout << ilk << " "; // İlk elemanı yazdır

        // Paketi "açarak" (unpacking) geri kalanını tekrar gönderiyoruz
        Yazdir(geriKalan...);
}
int main() {
        Yazdir(1, 2.5, "Merhaba", 'A', 42);
```

```
        // Adım 1: 1 yazdırılır, (2.5, "Merhaba", 'A', 42) gönderilir.
        // Adım 2: 2.5 yazdırılır, ("Merhaba", 'A', 42) gönderilir.
        // ... en son boş paket Yazdir() fonksiyonuna düşer.
}
/*Program Çıktısı:
1 2.5 Merhaba A 42
*/
```

Argüman paketinin içinde kaç argümanın olduğunu **`sizeof...` işlecini** (sizeof... operator) kullanarak öğrenebiliriz. Ayrıca C++17 öncesinde yukarıdaki gibi özyinelemeli (recursive) yazmak zorundaydık. C++17 ile gelen **katlama ifadeleri** (fold expressions) sayesinde artık döngüye veya **sonlandırıcı fonksiyona** (base case) gerek kalmadan paketi tek satırda açabiliriz:

```
#include <iostream>
template<typename... Args>
void YeniYazdir(Args... args) {
        std::cout << "Gelen argüman sayisi: " << sizeof...(args) << std::endl;
        // Tüm argümanların arasına << koyarak tek seferde yazdır (Fold Expression)
        (std::cout << ... << args) << std::endl;
}
int main() {
        YeniYazdir(1, 2.5, "Merhaba", 'A', 42);
        // Adım 1: 1 yazdırılır, (2.5, "Merhaba", 'A', 42) gönderilir.
        // Adım 2: 2.5 yazdırılır, ("Merhaba", 'A', 42) gönderilir.
        // ... en son boş paket Yazdir() fonksiyonuna düşer.
}
/*Program Çıktısı:
Gelen argüman sayisi: 5
12.5MerhabaA42
*/
```

15.4 Katlama İfadeleri

Katlama ifadeleri (fold expressions), C++17 standardı ile gelen ve değişken sayıda parametre alan şablonlar ile çalışmayı inanılmaz derecede kolaylaştıran bir özelliktir. C++11 ve C++14'te, bir parametre paketindeki (parameter pack) tüm elemanları toplamak veya çarpmak için özyinelemeli (recursive) fonksiyonlar yazmak zorundaydık. Katlama ifadeleri sayesinde artık tek bir satırda bu işlemi yapabiliyoruz.

Parametre Paketini katlamaya şu örnek verilebilir; Örneğin, elinizde `1, 2, 3, 4` gibi bir paket varsa, `+` işleciyle bunu `(((1 + 2) + 3) + 4)` haline getirir. Dört Farklı Katlama Türü vardır. Aşağıdaki tabloda `args` parametre paketini, `op` ise `+` ve `*` gibi ikili (iki işleneni olan) işlecleri temsil eder;

Katlama Türü	Yazım Şekli	Açılımı (Örnek)
Unary Right Fold	`(args op ...)`	`(a1 op (a2 op (a3 op a4)))`
Unary Left Fold	`(... op args)`	`(((a1 op a2) op a3) op a4)`
Binary Right Fold	`(args op ... op init)`	`(a1 op (a2 op (a3 op (a4 op init))))`
Binary Left Fold	`(init op ... op args)`	`((((init op a1) op a2) op a3) op a4)`

Tablo 15.1: Katlama Türleri

Katlama ifadeleri şunlar için kullanmalıyız;

- **Kod Kısalığı**: Özyinelemeli fonksiyon şablonları yazmanıza ve "durma koşulu" (base case) belirlemenize gerek kalmaz.
- **Derleme Hızı**: Derleyici bu ifadeleri doğrudan açtığı için özyinelemeli şablonlara göre daha hızlı derlenir.
- **Okunabilirlik**: Niyetinizi (tüm paketi bir operatörle birleştirmek) koda çok daha net yansıtır.

> Neredeyse tüm ikili operatörler desteklenir: `+ - * / % ^ & | = < > << >> == != <= >= && || , .* ->*`

Değişken sayıda sayıyı toplama **Unary Left Fold** katlama ile eskiden sayfalarca süren bu işlem artık tek satırdır;

```
#include <iostream>
template<typename... Args>
auto toplam(Args... args) {
```

```
        return (... + args); // Unary Left Fold
}
int main() {
        std::cout << toplam(1, 2, 3, 4, 5); // Çıktı: 15
        return 0;
}
```

Hepsini ekrana yazdırmak için virgül işleci (comma operator) katlama ifadeleri ile kullanıldığında, her bir argüman için bir işlem yapılmasını sağlar:

```
#include <iostream>
template<typename... Args>
void hepsiniYazdir(Args... args) {
        (std::cout << ... << args) << std::endl; // Binary Left Fold (std::cout başlangıç değeri)
}
int main() {
        hepsiniYazdir("C++ ", 17, " Fold ", "Expressions ", "Harika!");
}
```

Aşağıdaki örnekte, gönderilen tüm argümanların `true` olup olmadığını kontrol eden bir fonksiyon örneği vardır;

```
#include <iostream>
#include <vector>
// Hepsi doğru mu? (Logical AND Fold)
template<typename... Args>
bool hepsiDogruMu(Args... args) {
        return (... && args); // Unary Left Fold: (a1 && a2 && a3 ...)
}
// En az biri doğru mu? (Logical OR Fold)
template<typename... Args>
bool enAzBiriDogruMu(Args... args) {
        return (... || args); // Unary Left Fold: (a1 || a2 || a3 ...)
}
int main() {
        bool sonuc1 = hepsiDogruMu(true, 1, (5 > 2), true);
        std::cout << "Hepsi dogru mu? " << std::boolalpha << sonuc1 << std::endl; // true
        bool sonuc2 = hepsiDogruMu(true, 0, true);
        std::cout << "Hepsi dogru mu? " << sonuc2 << std::endl; // false (cunku 0, false kabul
        ↪ edilir)
        bool sonuc3 = enAzBiriDogruMu(false, 0, (10 < 2), true);
        std::cout << "En az biri dogru mu? " << sonuc3 << std::endl; // true
}
```

Katlama için iki durum meydana gelebilir;

1. **Kısa Devre Yapma** (short-circuiting): Katlama ifadeleri, mantıksal operatörlerin doğal davranışını korur. Örneğin (`... && args`) ifadesinde, paket içindeki ilk eleman `false` ise, derleyici diğer elemanlara bakmaz ve sonucu anında false döner. Bu performans için kritiktir.
2. **Boş Parametre Paketi Durumu**: Eğer fonksiyona hiç parametre göndermezseniz (`hepsiDogruMu()`), C++ standartlarına göre mantıksal katlamlar varsayılan değerler döner: `&&` (AND) için boş paket `true` döner. `||` (OR) için boş paket `false` döner.
3. `,` (virgül) için boş paket Geçersizdir (`void`).

15.5 Sınıf Şablonları

Sınıf şablonları (class templates),fonksiyon şablonlarında olduğu gibi, bir sınıfı belirli bir veri tipine bağlı kalmadan, tipten bağımsız bir kalıp olarak tanımlamanızı sağlar.Bir sınıf şablonu yazdığınızda, aslında bir sınıf değil, o sınıfın nasıl oluşturulacağına dair bir reçete yazmış olursunuz. Temel söz dizimi aşağıda verilmiştir;

```
template <typename T>
class SablonSinif {
private:
 T icerik; // T tipinde bir veri saklar
public:
 SablonSinif(T pIcerik) : icerik(pIcerik) {}
```

```
 T getIcerik() const {
  return icerik;
 }
 void setIcerik(T pYeniIcerik) {
  icerik = pYeniIcerik;
 }
};
```

Bir sınıf şablonundan nesne üretirken, hangi veri tipini şablona parametre vereceğinizi küçük ve büyük karakterleri (< >) içinde belirtirsiniz.

```
#include <iostream>

template <typename T>
class SablonSinif {
        private:
        T icerik; // T tipinde bir veri saklar
        public:
        SablonSinif(T pIcerik) : icerik(pIcerik) {}
        T getIcerik() const {
                return icerik;
        }
        void setIcerik(T pYeniIcerik) {
                icerik = pYeniIcerik;
        }
};
int main() {
        // T yerine 'int' geçiyoruz.
        SablonSinif<int> tamSayiSinifi(42);
        // T yerine 'string' geçiyoruz.
        SablonSinif<std::string> metinSinifi("Merhaba C++");

        std::cout << tamSayiSinifi.getIcerik() << std::endl;
        std::cout << metinSinifi.getIcerik() << std::endl;
}
/*Program Çıktısı:
42
Merhaba C++
*/
```

Eğer bir şablon sınıfın metodunu sınıfın dışında tanımlamak isterseniz, her seferinde şablon ifadesini tekrar etmeniz gerekir:

```
#include <iostream>

template <typename T>
class SablonSinif {
        private:
        T icerik; // T tipinde bir veri saklar
        public:
        SablonSinif(T pIcerik) : icerik(pIcerik) {}
        T getIcerik() const;
        void setIcerik(T pYeniIcerik);
};
template <typename T>
T SablonSinif<T>::getIcerik() const {
        return icerik;
}
template <typename T>
void SablonSinif<T>::setIcerik(T pYeniIcerik) {
        icerik = pYeniIcerik;
}
int main() {
        // T yerine 'int' geçiyoruz
```

```
        SablonSinif<int> tamSayiSinifi(42);
        // T yerine 'string' geçiyoruz
        SablonSinif<std::string> metinSinifi("Merhaba C++");

        std::cout << tamSayiSinifi.getIcerik() << std::endl;
        std::cout << metinSinifi.getIcerik() << std::endl;
}
/*Program Çıktısı:
42
Merhaba C++
*/
```

Sınıf şablonunda sadece bir veri tipi değil, birden fazla farklı veri tipini de aynı anda kullanabilirsiniz. Örnek olarak `Cift<T1, T2>` şablonu aşağıda verilmiştir:

```
#include <iostream>
#include <string>
template <typename T1, typename T2>
class Cift {
public:
        T1 birincil;
        T2 ikincil;
        // Yapıcı (Constructor)
        Cift(T1 a, T2 b) : birincil(a), ikincil(b) {}
        void ciftYaz() {
                // T1 ve T2 tiptir, değişkenler ise 'birincil' ve 'ikincil'dir.
                std::cout << birincil << ": " << ikincil << std::endl;
        }
};
int main() {
        // Örnek 1: Yaş bilgisi (string ve int)
        Cift<std::string, int> yasVerisi("Yasim", 45);
        yasVerisi.ciftYaz();
        // Örnek 2: Cinsiyet bilgisi (string ve string)
        Cift<std::string, std::string> cinsiyetVerisi("Cinsiyetim", "ERKEK");
        cinsiyetVerisi.ciftYaz();
}
/*Program Çıktısı:
Yasim: 45
Cinsiyetim: ERKEK
*/
```

15.6 Faydalı Şablonlar

C++ dünyasındaki `<utility>` başlığı "isviçre çakısı" rolüne sahiptir. Bu başlık büyük veri yapıları sunmaz; bunun yerine algoritmaların ve sınıfların birbiriyle konuşmasını sağlayan, performansı artıran küçük ama hayati araçlar sunar. Bu araçlar tek başlarına birer dev olmasalar da, taşıma semantiği (move semantics) ve genel programlama (generic programming) gibi modern C++ kavramlarının temelini oluştururlar.

§15.6.1 std::pair

En eski ve en temel bileşenlerden biridir. İki farklı veri tipini tek bir birim olarak paketlemenizi sağlar. Genellikle bir fonksiyondan iki değer döndürmek gerektiğinde tercih edilir.

```
        #include <iostream>
        #include <utility>
        #include <string>
        int main() {
                // İkilinin oluşturulması
                std::pair<int, std::string> ogrenci(101, "Ali Yilmaz");
                // Elemanlara erişim (.first ve .second)
                std::cout << "ID: " << ogrenci.first << " Isim: " << ogrenci.second << std::endl;
                // std::make_pair ile tip belirtmeden oluşturma
                auto urun = std::make_pair("Defter", 25.50);
        }
```

§15.6.2 std::move

C++11 ile gelen en devrimsel araçlardan biridir. Bir nesnenin içeriğini kopyalamak yerine, o içeriği başka bir nesneye "taşımanızı" sağlar. Özellikle büyük vektörler veya metinlerle çalışırken performansı dramatik şekilde artırır. `std::move()` aslında bir şeyi taşımaz; sadece nesneyi "taşınabilir" (rvalue) olarak işaretler. Asıl taşıma işlemini sınıfın taşıma yapıcısı (move constructor) metodu yapar.

```
#include <vector>
#include <string>
#include <utility>
std::string metin = "Bu cok buyuk bir veri kumesidir...";
std::vector<std::string> liste;
// Kopyalama yapmak yerine veriyi listeye taşıyoruz:
liste.push_back(std::move(metin));
// Bu noktadan sonra 'metin' değişkeni boştur veya belirsiz bir durumdadır.
```

§15.6.3 std::swap

İki değişkenin değerlerini yer değiştirmek için kullanılır. `<utility>` içindeki `std::swap()`, eğer nesne destekliyorsa otomatik olarak taşıma semantiğini kullanır, bu da onu son derece hızlı kılar.

```
int x = 10, y = 20;
std::swap(x, y); // x=20, y=10 olur.
```

§15.6.4 std::forward

Şablon (template) yazarken, bir fonksiyonun aldığı parametreleri başka bir fonksiyona "olduğu gibi" (lvalue ise lvalue, rvalue ise rvalue olarak) aktarmak için kullanılır. Özellikle jenerik programlama (generic programming) yaparken vazgeçilmezdir.

```
template <typename T>
void araciFonksiyon(T&& arg) {
        // arg'ı gercekIslem fonksiyonuna geldiği haliyle aktarır
        gercekIslem(std::forward<T>(arg));
}
```

§15.6.5 Değişmez Değerler ve Diğerleri

C++17 ile gelen `std::as_const`, bir değişkeni geçici olarak salt okunur (const) hale getirir. Özellikle yanlışlıkla veri değiştirilmesini önlemek için fonksiyonlara parametre gönderirken kullanılır.

C++14 ile birlikte gelen `std::exchange`, bir değişkenin değerini yenisiyle değiştirirken, eski değerini döndürür. Özellikle **yıkıcı** (destructor) veya **taşıma atama işleci** (move assignment) yazarken çok kullanışlıdır.

```
int eski_deger = std::exchange(guncel_deger, 0); // guncel_deger'i 0 yapar, eskiyi
↪ döndürür.
```

15.7 Akıllı Göstericiler

§15.7.1 Kaynak Edinimi İlk Değer Atamasıdır! Prensibi

Modern C++'ın bel kemiği sayılabilecek en önemli konulardan birine geldik. RAII (Resource Acquisition Is Initialization), yani "Kaynak Edinimi İlk Değer Atamasıdır" prensibi, C++'ı diğer dillerden ayıran ve bellek yönetimini güvenli kılan temel felsefedir.

RAII, bir kaynağın (bellek, dosya göstericisi, ağ bağlantısı, mutex kilidi vb.) ömrünü bir nesnenin ömrüne bağlama tekniğidir. Bu prensip sayesinde C++'ta **çöp toplayıcı** (garbage collector) mekanizmasına ihtiyaç duyulmadan bellek sızıntıları önlenir.

RAII'nin temel işleyişi yapıcı (constructor) ve yıkıcıların (destructor) Gücüne dayanır. İki ana adıma dayanır:

Yapıcıda, kaynak (örneğin new ile ayrılan bellek veya açılan bir dosya) nesne oluşturulduğu anda edinilir. Yıkıcıda ise nesne kapsam (scope) dışına çıktığı anda kaynak otomatik olarak serbest bırakılır.

Geleneksel programlamada en büyük risk, kaynağı serbest bırakmayı unutmaktır;

```
void riskliFonksiyon() {
        FILE* dosya = fopen("data.txt", "r");
        // ... işlemler yapılırken bir hata oluştu ve 'return' edildi ...
```

```
        if (hataOlustu) return; // Dosya kapatılmadı! (Sızıntı)

        fclose(dosya);
}
```

RAII Çözümümü, **kendi kendini temizleyen sınıf yazmaktır**. Aynı kodu RAII ile aynı işlemi güvenli hale getirelim:

```
#include <iostream>
#include <fstream>
#include <string>
class DosyaYonetici {
        private:
        std::ifstream dosya;
        public:
        // Kaynak yapıcıda edinilir (Acquisition is Initialization)
        DosyaYonetici(std::string dosyaAdi) {
                dosya.open(dosyaAdi);
                std::cout << "Dosya acildi.\n";
        }
        // Kaynak yıkıcıda serbest bırakılır
        ~DosyaYonetici() {
                if (dosya.is_open()) {
                        dosya.close();
                        std::cout << "Dosya otomatik kapatildi.\n";
                }
        }
        void oku() { /* ... okuma islemleri ... */ }
};
void güvenliFonksiyon() {
        DosyaYonetici yönetici("test.txt");
        yönetici.oku();
        // Fonksiyon burada bitse de, hata fırlatsa da 'yönetici' nesnesi
        // kapsam dışı kalacağı için yıkıcısı çalışır ve dosya kapanır.
}
```

C++'ta bir fonksiyon bittiğinde, o fonksiyon içindeki yerel nesnelerin yıkıcıları garantili olarak çağrılır. RAII bu garantiyi kullanarak kaynakları temizler.

Modern C++'ta kendi RAII sınıflarınızı yazabileceğiniz gibi, Standart Kütüphane (STL) zaten birçok hazır RAII aracı sunar:

- **Akıllı göstericiler** (`std::unique_ptr`, `std::shared_ptr`): Dinamik belleği (heap) RAII prensibiyle yönetir. `delete` yazmanıza gerek kalmaz.
- **Konteynerler** (`std::vector`, `std::string`): İçlerindeki dizileri otomatik büyütür ve yok ederler.
- **Kilit Yönetimi** (`std::lock_guard`): Multi-thread programlamada bir mutex'i kilitler ve kapsam bitince kilidi otomatik açar.

C++ dünyasında **akıllı göstericiler** (samart pointers), bellek yönetimini otomatik olarak yapan ve **bellek sızıntısı** (memory leak) gibi korkulu rüyaları ortadan kaldıran modern bir sarmalayıcı (wrapper) şablonlardır.

Geleneksel C++ dilinde `new` ile tahsis edilen bir belleği `delete` ile silmeyi unutmak programın şişmesine neden olurken; Akıllı göstericiler, nesnenin ömrü dolduğunda belleği kendi kendine serbest bırakır. 9.Göstericiler başlığında işlenen geleneksel göstericilerde şu üç temel risk vardır:

- Bellek Sızıntısı: `delete` satırı unutulursa bellek geri verilmez.
- Erken Silme: Bellek silindikten sonra hala o adrese erişmeye çalışan sarkan gösericiler (dangling pointer) programı çökertir.
- Çift Silme : Aynı adresi yanlışlıkla iki kez serbest bırakmaya (double free) çalışmak.

Göstericiler `<memory>` başlık dosyasında yer alır. Kullanmak için projemize dahil etmemiz gerekir. Üç ana akıllı gösterici bulunur ve aşağıda başlıklar halinde verilmiştir.

§15.7.2 std::unique_ptr ile Tekil Sahiplik

`std::unique_ptr` ile **Akıllı gösterici nesnesinin sadece tek bir sahibi olabilir**. Kopyalanamaz, sadece `std::move` fonksiyonu ile taşınabilir. Akıllı gösterici nesnenin ömrü tek bir kapsam (scope) içindeyse veya sahiplik net bir şekilde devredilecekse kullanılır. Bildiğimiz göstericiler kadar hızlıdır, ek yük getirmez.

```
#include <iostream>
#include <memory> // Akıllı göstericiler için
int main() {
        std::unique_ptr<int> ptr1 = std::make_unique<int>(10);
        // std::unique_ptr<int> p2 = p1; // HATA! Kopyalanamaz.
        std::unique_ptr<int> ptr2 = std::move(ptr1); // p1 artık boş, p2 sahibi.
}
```

§15.7.3 std::shared_ptr ile Paylaşılan Sahiplik

`std::shared_ptr` ile bir gösterici nesnenin birden fazla sahibi olabilir. Arka planda bir **referans sayacı** (reference counter) tutar. Yeni bir `shared_ptr` nesneye bağlandığında sayaç artar. Bir işaretçi yok olduğunda sayaç azalır. Sayaç sıfır olduğunda bellek temizlenir. Sayaç yönetimi nedeniyle çok küçük bir çalışma zamanı yükü vardır.

```
#include <iostream>
#include <memory> // Akıllı göstericiler için
int main() {
        auto s1 = std::make_shared<int>(100);
        {
                auto s2 = s1; // Sayaç 2 oldu.
                auto s3 = s1; // Sayaç 3 oldu.
        } // s2,s3 kapsam dışı, sayaç 1'e düştü.
        // main bitince sayaç 0 olur ve bellek silinir.
}
```

§15.7.4 std::weak_ptr ile Zayıf Takipçi

`std::shared_ptr` tarafından yönetilen bir nesneye erişim sağlar ancak sayacı artırmaz. **Döngüsel bağımlılık** (circular dependency) sorununu çözmek için kullanılır. İki nesne birbirini `shared_ptr` ile tutarsa sayaç asla sıfırlanmaz; `weak_ptr` bu düğümü çözer. Aşağıdaki senaryoda bir `Anne` ve bir `Cocuk` nesnesi var. Anne çocuğunu, çocuk da annesini `shared_ptr` ile işaret ediyor. Döngüsel bağımlılık oluşmuştur.

```
#include <iostream>
#include <memory>
struct Cocuk; // Forward declaration: Cocuk diye bir tanımlama olacak!
struct Anne {
        std::shared_ptr<Cocuk> cocukPtr;
        ~Anne() { std::cout << "Anne yok edildi.\n"; }
};
struct Cocuk {
        std::shared_ptr<Anne> annePtr;
        ~Cocuk() { std::cout << "Cocuk yok edildi.\n"; }
};
void hataOlustur() {
        auto anne = std::make_shared<Anne>();
        auto cocuk = std::make_shared<Cocuk>();

        anne->cocukPtr = cocuk; // Anne çocuğu sahipleniyor (Sayaç: 1)
        cocuk->annePtr = anne; // Çocuk anneyi sahipleniyor (Sayaç: 1)
} // Fonksiyon biterken sayaçlar 0'a inmeliydi ama birbirlerini tuttukları için 1'de kalırlar!
 ↪ Bellek sızıntısı olur.
```

`hataOlustur()` fonksiyonu bittiğinde ekranda hiçbir "yok edildi" yazısı görmezsiniz ancak bellek sızıntısı olmuştur. `weak_ptr`, bir nesneyi "gözlemler" ama onun sahibi olmaz (referans sayacını artırmaz). Döngüyü kırmak için çocuk nesnesinin annesini `weak_ptr` ile tutması gerekir.

```
#include <iostream>
#include <memory>
struct Cocuk;
struct Anne {
        std::shared_ptr<Cocuk> cocukPtr;
        ~Anne() { std::cout << "Anne yok edildi.\n"; }
};
struct Cocuk {
        std::weak_ptr<Anne> annePtr; // shared_ptr yerine weak_ptr kullandık!
```

```
        ~Cocuk() { std::cout << "Cocuk yok edildi.\n"; }
        void anneminAdiniDeSoyle() {
                // weak_ptr doğrudan kullanılamaz, önce shared_ptr'a dönüştürülmeli (lock)
                if (auto sPtr = annePtr.lock()) {
                        std::cout << "Annem hala yasiyor.\n";
                } else {
                        std::cout << "Annem bellekten silinmis.\n";
                }
        }
};
int main() {
        {
                auto anne = std::make_shared<Anne>();
                auto cocuk = std::make_shared<Cocuk>();

                anne->cocukPtr = cocuk;
                cocuk->annePtr = anne; // weak_ptr olduğu için Anne'nin sayacı artmaz!
        } // Kapsam biterken Anne silinir, Anne silindiği için Cocuk da silinir.
        std::cout << "Program sonu.\n";
}
/*Program Çıktısı:
Anne yok edildi.
Cocuk yok edildi.
Program sonu.
*/
```

Örnek kodlarda akıllı gösterici imal etmekte kullanılan `make_` fonksiyonlarının kullanımı çok önemlidir. Akıllı gösterici oluştururken doğrudan `new` kullanmak yerine `std::make_unique` ve `std::make_shared` kullanılması önerilir.
Nesne oluşturulurken istisna (exception) oluşursa bellek sızıntısını önler. `make_shared`, nesne ve sayaç bloğu için tek bir bellek tahsisi yaparak daha hızlı çalışır.

§15.7.5 Akıllı Gösterici Örnekleri

9.3 Başlığında dinamik değişken kullanımını anlatan eski tip kullanım örneğini aşağıdaki gibi yenileyebiliriz;

```
#include <iostream>
#include <memory> // std::unique_ptr ve std::make_unique için şart
int main() {
        // 1. TEKİL DEĞİŞKEN (Modern Yöntem)
        // make_unique hem belleği tahsis eder hem de değeri içine koyar.
        std::unique_ptr<double> dPtr = std::make_unique<double>(3.14);
        std::cout << "Double Deger: " << *dPtr << " | Adres: " << dPtr.get() << std::endl;

        // 2. TEKİL DEĞİŞKEN (Farklı İlklendirme)
        auto dPtr2 = std::make_unique<double>(3.1415);
        std::cout << "Double Deger 2: " << *dPtr2 << " | Adres: " << dPtr2.get() << std::endl;
        // NOT: Burada 'delete dPtr' yazmıyoruz! Kapsam (main) bitince otomatik silinirler.
        // 3. DİNAMİK DİZİ (Modern Yöntem)
        int boyut = 5;
        // unique_ptr dizi olduğunu <double[]> ifadesinden anlar.
        std::unique_ptr<double[]> diziPtr = std::make_unique<double[]>(boyut);

        diziPtr[0] = 10.5;
        diziPtr[1] = 20.7;
        diziPtr[2] = 30.9;
        // Atanmayan 3. ve 4. indisler varsayılan olarak 0.0 ile doldurulur (make_unique
         ↪ özelliği).
        std::cout << "\n--- Akilli Dizi Elemanlari ---" << std::endl;
        for (int i = 0; i < boyut; i++) {
```

```
                std::cout << i << ". eleman: " << diziPtr[i] << " | Adres: " << &diziPtr[i] <<
                 ↪  std::endl;
        }
        // KRİTİK: delete[] yazmanıza gerek yok. unique_ptr dizi olduğunu bildiği için
        // arka planda doğru silme işlemini (delete[]) kendisi yapar.
}
/*Program Çıktısı:
Double Deger: 3.14 | Adres: 0x2712e2b0
Double Deger 2: 3.1415 | Adres: 0x2712e6e0

--- Akilli Dizi Elemanlari ---
0. eleman: 10.5 | Adres: 0x2712e700
1. eleman: 20.7 | Adres: 0x2712e708
2. eleman: 30.9 | Adres: 0x2712e710
3. eleman: 0 | Adres: 0x2712e718
4. eleman: 0 | Adres: 0x2712e720
*/
```

Benzer şekilde 11.5 Başlığında dinamik nesne imalatını anlatan eski tip kullanım örneğini aşağıdaki gibi yenileyebiliriz;

```
#include <iostream>
#include <memory> // std::unique_ptr ve std::make_unique için gerekli
class Karmasik {
        public:
        float gercek, sanal;
        // 1. Varsayılan Yapıcı
        Karmasik() : gercek(0.0f), sanal(0.0f) {
                std::cout << "Varsayilan yapici cagrildi" << std::endl;
        }
        // 2. Parametreli Yapıcı
        Karmasik(float pGercek, float pSanal) : gercek(pGercek), sanal(pSanal) {
                std::cout << "Iki Parametreli yapici cagrildi" << std::endl;
        }
        // 3. Devredilen Yapıcı (Delegating Constructor)
        Karmasik(float pGercek) : Karmasik(pGercek, 0.0f) {
                std::cout << "Tek parametreli yapici cagrildi (Iki parametreliye devredildi)" <<
                 ↪  std::endl;
        }
        // Yıkıcı (Destructor): Belleğin ne zaman silindiğini görmek için ekledik
        ~Karmasik() {
                std::cout << "Nesne bellekten siliniyor: " << gercek << " + " << sanal << "i" <<
                 ↪  std::endl;
        }
        void yaz() const {
                std::cout << gercek << " + " << sanal << "i" << std::endl;
        }
};
int main() {
        std::cout << "--- Nesne 1 (Default) ---" << std::endl;
        // std::make_unique parametresiz çağrıldığında varsayılan yapıcıyı kullanır.
        auto k1 = std::make_unique<Karmasik>();
        k1->yaz();
        std::cout << "\n--- Nesne 2 (Parametreli) ---" << std::endl;
        // make_unique içine gönderilen argümanlar uygun yapıcıya iletilir.
        auto k2 = std::make_unique<Karmasik>(2.0f, 3.0f);
        k2->yaz();
        std::cout << "\n--- Nesne 3 (Tek Parametreli) ---" << std::endl;
        auto k3 = std::make_unique<Karmasik>(4.0f);
        k3->yaz();
        std::cout << "\n--- Program Sonu ---" << std::endl;
        // NOT: Burada 'delete' yazmıyoruz!  Program (veya kapsam) sona erdiğinde akıllı
         ↪  işaretçiler otomatik olarak yıkıcıyı (~Karmasik) çağırır.
```

```
}
/*Program Çıktısı:
--- Nesne 1 (Default) ---
Varsayilan yapici cagrildi
0 + 0i

--- Nesne 2 (Parametreli) ---
Iki Parametreli yapici cagrildi
2 + 3i

--- Nesne 3 (Tek Parametreli) ---
Iki Parametreli yapici cagrildi
Tek parametreli yapici cagrildi (Iki parametreliye devredildi)
4 + 0i

--- Program Sonu ---
Nesne bellekten siliniyor: 4 + 0i
Nesne bellekten siliniyor: 2 + 3i
Nesne bellekten siliniyor: 0 + 0i
*/
```

Akıllı göstericiler ile yapılan güvenli kod örneklerine tasarım desenleri başlığında bolca örnek bulabilirsiniz.

15.8 Standart Şablon Kütüphanesi

Standart şablon kütüphanesi STL(Standart Template Library), C++ dilinin en güçlü kütüphanesidir. Jenerik programlama (generic programming) ile birçok algoritma programcıya bu kütüphane ile sunulur. Şablon (template) kavramını anlamışsanız bu kütüphaneyi iyi kavrarsınız.

Standart şablon kütüphanesi, vektörler, listeler, kuyruklar ve yığınlar gibi popüler ve yaygın olarak kullanılan güçlü bir şablon sınıfları kümesidir. Bu kütüphane algoritmaları ve veri yapılarını içeren şablonlarla genel amaçlı sınıflar ve fonksiyonlar sağlar. Bu şablonlar başlıkta incelebilir;

1. **Konteynerler**: Belirli bir türdeki nesne kümeleri yani koleksiyonlarını yönetmek için kullanılan veri yapıları şablonudur. `deque`, `list`, `vector`, `map` gibi çeşitli farklı konteyner türleri vardır. Konteynerler, tuttukları verilerden bağımsız olarak hazırlanmış şablon şeklinde dinamik veri yapılarıdır (dynamic data structure).
2. **Algoritmalar**: Konteynerler üzerinde etki eder. Konteynerlerin içeriklerinin başlatılmasını, sıralanmasını, aranmasını ve dönüştürülmesini gerçekleştireceğiniz araçları sağlarlar.
3. **Yineleyiciler**: Koleksiyonlarının elemanları arasında gezinmek için kullanılır. Gezinme, koleksiyonların kapsayıcılarında veya kapsayıcıların alt kümeleri olabilir.
4. **Fonksiyon Nesneleri**: Bir fonksiyon sanki bir fonksiyonmuş gibi çağrılabilen bir nesnedir. Normal fonksiyonlar gibi çağrılabilir. Bu yetenek, **fonksiyon çağrı işleci** (function call operator) `()` yani fonksiyon parantezlerinin aşırı yüklenmesiyle elde edilir. Bu kelimelerin kısaltılmış hali olan **functor** olarak adlandırılırlar.

15.9 Konteyner Şablonları

Konteyner şablonları bize nesne kümelerini depolamak ve yönetmek için kullanılan veri yapılarıdır. Dinamik veri yapıları yapısal programlama sürecinde çokça geliştirilen ve artık standart hale gelen koleksiyonlardır. Üç başlıkta incelenebilirler;

1. **Ardışık konteynerler** (sequential container), elemanları belirli bir doğrusal düzende depolarlar. Elemanları ve sırasını ve düzenlemesini yönetmek için işlevlere doğrudan erişim sağlarlar;
   - `array` (Dizi): Bunlar sabit boyutlu eleman koleksiyonlarıdır.
   - `vector` (Vektör): Gerektiğinde boyutunu değiştirebilen dinamik bir dizidir.
   - `deque` (İki Uçlu Kuyruk): Her iki uçtan hızlı ekleme ve silmeye izin veren çift uçlu kuyruk.
   - `list` (Liste): Çift yönlü yinelemeye izin veren çift bağlantılı liste.
   - `forward_list` (İleri Liste): Verimli ekleme ve silmeye izin veren ancak yalnızca bir yönde gezinmeye izin veren tek bağlantılı listedir.
   - `string` (Dizgi): Diğer ardışık konteynerlere benzeyen dinamik bir karakter dizisidir.
2. **İlişkili konteynerler** (associated container), elemanları anahtarlara (key) göre hızlı bir şekilde geri almaya izin veren sıralı bir düzende depolar. Bu konteynerler, her bir elemanın benzersiz bir anahtarla ilişkilendirildiği bir anahtar-değer (key-value) çifti yapısında çalışır. Bu anahtar, karşılık gelen değere hızlı erişim için kullanılır;

- ◇ `set` (Küme): Belirli bir düzende sıralanmış benzersiz elemanların bir koleksiyonudur.
- ◇ `map` (Harita): Anahtarların benzersiz olduğu anahtar-değer çiftlerinin bir koleksiyonudur.
- ◇ `multiset` (Çoklu Küme): Bir kümeye benzer, ancak yinelenen elemanların izin verir.
- ◇ `multimap` (Çoklu Harita): Bir haritaya benzer, ancak yinelenen anahtarlara izin verir.

3. **Sıralanmamış ilişkisel konteynerler** (unordered associated container), elemanları sırasız bir şekilde depolar ve anahtarlara (key) dayalı olarak verimli erişim, ekleme ve silmeye olanak tanır. Elemanların sıralı bir düzenini korumak yerine verileri düzenlemek için karma (hash) kullanırlar;
   - ◇ `unordered_set` (Sıralanmamış Küme): Belirli bir sırası olmayan, benzersiz elemanlardan oluşan bir koleksiyondur.
   - ◇ `unordered_map` (Sıralanmamış Harita): Belirli bir sırası olmayan, anahtar-değer çiftlerinden oluşan bir koleksiyondur.
   - ◇ `unordered_multiset` (Sıralanmamış Çoklu Küme): Belirli bir sırası olmayan, yinelenen elemanlara izin verir.
   - ◇ `unordered_multimap` (Sıralanmamış Çoklu Harita): Belirli bir sırası olmayan, yinelenen anahtarlara izin verir.

Konteyner adaptörleri (container adapter), mevcut konteynerler için farklı bir ara yüz sağlar. Temel konteynerlerin davranışlarını belirli kullanım durumlarına uyacak şekilde adapte ederler;

- ◇ `stack` (Yığın): Son Giren İlk Çıkar LIFO (last in first out) ilkesini izleyen bir veri yapısıdır.
- ◇ `queue` (Kuyruk): İlk Giren İlk Çıkar FIFO (first in first out) ilkesini izleyen bir veri yapısıdır.
- ◇ `priority_queue` (Öncelikli Kuyruk): Elemanların önceliğe göre kaldırıldığı özel bir kuyruk türüdür.

§15.9.1 std::array

Dizileri, 8 başlığından biliyoruz. Ancak bu şablon her türlü veri tipiyle oluşturulabilir. Bu sınıf şablonunun iki parametresi vardır. `class T`, Dizi elemanlarının veri tipini belirtir. `std::size_t N` ise dizide kaç eleman olduğunu belirtir.

Diziyi Başlatma

Dizinin sayılarla ifade edilen (scalar) veri tiplerini barındırması halinde aşağıdaki şekilde ilk değer verilebilir ya da başlatılabilir);

```
//1. Dizilere ilk değer verme yöntemini kullanarak
std::array<int, 3> a { 0, 1, 2 };
// buna eşdeğer ifade:
std::array<int, 3> a = { 0, 1, 2 };
//2.Kopya yapıcı (copy constructor) kullanarak
std::array<int, 3> a { 0, 1, 2 };
std::array<int, 3> a2(a);
// buna eşdeğer ifade
std::array<int, 3> a2 = a;
//3.Tasıma yapıcısı (move constructor) kullanarak;
std::array<int, 3> a = std::array<int, 3>{ 0, 1, 2 };
```

Dizinin sayılarla ifade edilmeyen (non scalar) veri tiplerini barındırması halinde aşağıdaki şekilde ilk değer verilebilir ya da başlatılabilir);

```
struct A { int values[3]; };
//4.Tırnaklı parantez ile ilk değer verilebilir
std::array<A, 2> a{ 0, 1, 2, 3, 4, 5 };
// buna eşdeğer ifade:
std::array<A, 2> a = { 0, 1, 2, 3, 4, 5 };
//5.Tırnaklı parantezleri alt elemanlarda kullanarak;
std::array<A, 2> a{ A{ 0, 1, 2 }, A{ 3, 4, 5 } };
// buna eşdeğer ifade:
std::array<A, 2> a = { A{ 0, 1, 2 }, A{ 3, 4, 5 } };
//6.Tamamıyla tırnaklı parantez kullanarak;
std::array<A, 2> a{{ { 0, 1, 2 }, { 3, 4, 5 } }};
// buna eşdeğer ifade:
std::array<A, 2> a = {{ { 0, 1, 2 }, { 3, 4, 5 } }};
//7.Kopya yapıcı (copy constructor) kullanarak;
std::array<A, 2> a{ 1, 2, 3 };
std::array<A, 2> a2(a);
// buna eşdeğer ifade:
std::array<A, 2> a2 = a;
```

```
//8.Taşıma yapıcısı (move constructor) kullanarak;
std::array<A, 2> a = std::array<A, 2>{ 0, 1, 2, 3, 4, 5 };
```

Dizi Elemanlarına Erişim

1. `at(pos)` fonksiyonu ile: `pos` konumundaki elemana sınır denetimiyle bir referans döndürür. `pos`, konteynerin aralığında değilse, `std::out_of_range` türünde bir istisna atılır.

```
#include <array>
int main()
{
        std::array<int, 3> arr;
        arr.at(0) = 3; //0. Elemana 2 atandı
        arr.at(1) = 4; //1. Elemana 2 atandı
        arr.at(2) = 6; //2. Elemana 2 atandı
        int a = arr.at(0); // a değişkenine 0. Dizi elemanı atandı
        int b = arr.at(1); // a değişkenine 1. Dizi elemanı atandı
        int c = arr.at(2); // a değişkenine 2. Dizi elemanı atandı
}
```

2. `operator[pos]` işleci ile: pos konumundaki elemana sınır denetimi olmadan bir başvuru döndürür. `pos`, konteynerin aralığında değilse, bir çalışma zamanı hatası (segmentation violation) oluşabilir. Bu yöntem, klasik dizilere eşdeğer eleman erişimi sağlar ve bu nedenle `at(pos)`'tan daha verimlidir.

```
#include <array>
int main()
{
        std::array<int, 3> arr;
        arr[0] = 3;
        arr[1] = 4;
        arr[2] = 6;
        int a = arr[0];
        int b = arr[1];
        int c = arr[2];
}
```

3. `std::get<pos>` ile: `std::array` sınıfının üyesi olmayan bu fonksiyon, sınır denetimi olmaksızın derleme zamanı sabit konumu `pos`'taki elemana bir referans döndürür. `pos`, konteynerin aralığında değilse, bir çalışma zamanı hatası (segmentation violation) oluşabilir.

```
#include <array>
int main()
{
        std::array<int, 3> arr;
        std::get<0>(arr) = 3;
        std::get<1>(arr) = 4;
        std::get<2>(arr) = 6;
        int a = std::get<0>(arr);
        int b = std::get<1>(arr);
        int c = std::get<2>(arr);
}
```

4. `front()` fonksiyonu ile: Konteynerdeki ilk elemana bir referans döndürür. Boş bir konteynerde `front()`'u çağırmak tanımsızdır.

```
#include <array>
int main()
{
        std::array<int, 3> arr{ 2, 4, 6 };
        int a = arr.front();
}
```

5. `back()` fonksiyonu ile: Konteynerdeki son elemana bir referans döndürür. Boş bir konteynerde `back()`'u çağırmak tanımsızdır.

```
#include <array>
int main()
{
        std::array<int, 3> arr{ 2, 4, 6 };
```

```
        int a = arr.back();
}
```

6. `data()` fonksiyonu ile: Eleman depolaması olarak hizmet eden altta yatan diziye gösterici döndürür.

```
#include <iostream>
#include <array>
void yaz(const int* p, std::size_t size){
        for (std::size_t i = 0; i < size; ++i)
        std::cout << p[i] << ' ';
}
int main (){
        std::array<int, 3> a { 0, 1, 2 };
        yaz(a.data(), 3);
}
```

Dizi Elemanları Üzerinde Yineleme

```
#include <iostream>
#include <array>
int main() {
        std::array<int, 3> arr = { 1, 2, 3 };
        for (auto i : arr)
        std::cout << i << std::endl;
}
```

Dizi Boyutunu Kontrol Etme

```
#include <iostream>
#include <array>
int main() {
        std::array<int, 3> arr = { 1, 2, 3 };
        std::cout << arr.size() << std::endl;
}
```

Dizi Elemanlarını Tek Seferde Değiştirme

```
#include <iostream>
#include <array>
int main() {
        std::array<int, 3> arr = { 1, 2, 3 };
        arr.fill(100);
}
```

§15.9.2 std::vector

Bir vektör, otomatik olarak işlenen depolamaya sahip dinamik bir dizidir. Bir vektördeki elemanlara, bir dizideki elemanlar kadar verimli bir şekilde erişilebilir; avantajı, vektörlerin boyut olarak dinamik olarak değişebilmesidir. Depolama açısından vektör verileri (genellikle) dinamik olarak tahsis edilmiş belleğe yerleştirilir, bu nedenle bazı küçük ek yükler gerektirir; tersine `std::array`, beyan edilen boyuta göre otomatik depolama kullanır ve bu nedenle herhangi bir ek yüke sahip değildir.

Vertörler `std:array` gibi başlatılabilir.

Vektör Elemanlarına Erişim

Vektör elemanlarına `[]` işleci ve `at()` fonksiyonu ile erişebiliriz.

```
#include <iostream>
#include <vector>
int main() {
        std::vector<int> v{ 1, 2, 3 };
        // [] işleci:
        int a = v[1];
        v[1] = 4;
        // at() fonksiyonu:
        int b = v.at(2);
        v.at(2) = 5;
        //int c = v.at(3); // Hata 4. Eleman Yok!
```

```
        for (std::size_t i = 0; i < v.size(); ++i) {
                v[i] = 1;
        }
}
```

Ayrıca `front()` ve `back()` fonksiyonları ile de vektör elemanlarına erişilebilir;

```
std::vector<int> v{ 4, 5, 6 };
int a = v.front();
v.front() = 3;
int b = v.back(); // v.back() eşdeğerdir: v[v.size() - 1]
v.back() = 7;
```

Vektöre Eleman Ekleme ve Çıkarma

Vektöre eleman ekleme `push_back()`, vektörden eleman çıkarma `pop_back()` ve boş olduğunu kontrol etme `empty()` fonksiyonları ile yapılabilir;

```
#include <iostream>
#include <vector>
int main ()
{
        std::vector<int> v;
        int toplam (0);
        for (int i=1;i<=10;i++) v.push_back(i); //vektör oluştutulup elemanlar ekleniyor
        while (!v.empty()) //vektörün boş olup olmadığı kontrol edilip ilerleniyor
        {
                toplam += v.back();
                v.pop_back(); //son elemanı vektörden çıkar
        }
        std::cout << "toplam: " << toplam << std::endl;
}
```

Vektör Verisi

Vektörde `data()` yöntemi, `std::vector`'un elemanlarını dahili olarak depolamak için kullandığı ham belleğe bir gösterici döndürür. Bu gösterici, genellikle vektör verilerini 8 başlığındaki C tarzı bir dizi bekleyen eski koda geçirirken kullanılır.

```
std::vector<int> v{ 1, 2, 3, 4 };
int* p = v.data();
*p = 4; // v[0]=4
++p;
*p = 3; // v[1]=3
p[1] = 2;
*(p + 2) = 1;
```

Aşağıda bir diziye benzeyen ancak büyümesi durumunda kendi depolama gereksinimlerini otomatik olarak karşılayan vektör konteynerini kullanan program örneği verilmiştir;

```
#include <iostream>
#include <vector>
int main() {
        std::vector<double> vec; // int değerler tutacak std::vector konteyneri
        int i;
        std::cout << "std::vector Uzunluğu = " << vec.size() << std::endl;
        for(i = 0; i < 5; i++) { // 5 değer vektöre koyuluyor.
                vec.push_back(i*1.5);
        }
        std::cout << "Vektörün Son Uzunluğu = " << vec.size() << std::endl;
        for(i = 0; i < 5; i++) {// vektör elemanlarına erişiliyor
                std::cout << "vec[" << i << "] = " << vec[i] << ",";
        }
        std::cout << std::endl;
        // yineleyiciler üzerinden vektör elemanlarına erişim:
        std::vector<double>::iterator v = vec.begin();
        while( v != vec.end()) {
                std::cout  << *v << ",";
```

```
                v++;
        }
        std::cout << std::endl;
}
/*Program Çıktısı:
std::vector Uzunluğu = 0
Vektörün Son Uzunluğu = 5
vec[0] = 0,vec[1] = 1.5,vec[2] = 3,vec[3] = 4.5,vec[4] = 6,
0,1.5,3,4.5,6,
*/
```

§15.9.3 std::map, std::multimap ve std::pair

Haritalar, benzersiz anahtarlara sahip **anahtar-değer** (key-value) çiftleri (pair) içeren sıralı bir ilişkisel konteynerlerdir. Anahtarlar, `compare()` karşılaştırma işlevi kullanılarak sıralanır. Arama, kaldırma ve ekleme işlemleri olan search(), `insert()`, `delete()` için logaritmik karmaşıklığa sahiptir.

- `std::map` veya `std::multimap`'ten herhangi birini kullanmak için `<map>` başlık dosyası projeye dahil edilmelidir.
- `std::map` ve `std::multimap` her ikisi de elemanlarının anahtarların artan sırasına göre sıralanmış halde tutar.
- `std::multimap` durumunda, aynı anahtarın değerleri için sıralama yapılmaz.
- `std::map` ve `std::multimap` arasındaki temel fark, `std::map`'in aynı anahtar için yinelenen değerlere izin vermemesidir.

Çiftleri Oluşturma

Çiftler aşağıdaki gibi oluşturulup karşılaştırılabilir;

```
std::pair<int, int> p1 = std::make_pair(1, 2);
std::pair<int, int> p2 = std::make_pair(2, 2);
if (p1 == p2)
        std::cout << "eşit";
else
        std::cout << "eşit değil!"
```

Çiftleri Sıralama

Çiftler bir konteyner içindeyken de küçük işleci ile sıralanabilir;

```
#include <iostream>
#include <utility>
#include <vector>
#include <algorithm>
#include <string>
int main() {
        std::vector<std::pair<int, std::string>> v = { {2, "Ali"}, {2, "Mustafa"}, {1, "İlhan"} };
        std::sort(v.begin(), v.end());
        for(const auto& p: v)
                std::cout << "(" << p.first << "," << p.second << ") ";
}
/*Program Çıktısı:
(1,İlhan) (2,Ali) (2,Mustafa)
*/
```

Haritalarda Yineleme

Haritalarda elemanlar anahtar değer ikilisidir ve aşağıdaki şekilde yineleyiciler ile erişilir;

```
#include <iostream>
#include <map>
int main() {
        std::map < int, std::string > siralama {
                std::make_pair(2, "ahmet"),
                std::make_pair(1, "ali") };
        siralama[1]="mustafa";
        std::cout << siralama.at(2) << std::endl; // ahmet
        auto it = siralama.begin();
```

```
        std::cout << it->first << " : " << it->second << std::endl; // "1 : mustafa"
        it++;
        std::cout << it->first << " : " << it->second << std::endl; // "2 : ahmet"
        std::map < std::string , int> notlar {
                std::make_pair("ahmet",100),
                std::make_pair("ali",90),
                std::make_pair("mustafa",85)
        };
        notlar["mustafa"]=95;
        std::cout << notlar.at("ali") << std::endl; // 90
        auto it2 = notlar.rbegin(); //tersten
        std::cout << it2->first << " : " << it2->second << std::endl; // "mustafa: 95"
        it2++;
        std::cout << it2->first << " : " << it2->second << std::endl; // "ali: 90"
}
```

std::multimap Örneği

```
#include <iostream>
#include <map>
int main() {
        std::multimap < int, std::string > mmp {
                std::make_pair(2, "ahmet"),
                std::make_pair(1, "ali"),
                std::make_pair(2, "mustafa")

        };
        auto it = mmp.begin();
        std::cout << it->first << " : " << it->second << std::endl; //"1 : ali"
        it++;
        std::cout << it->first << " : " << it->second << std::endl; //"2 : ahmet"
        it++;
        std::cout << it->first << " : " << it->second << std::endl; //"2 : mustafa"
        std::map < int, std::string > mp {
                std::make_pair(2, "ahmet"),
                std::make_pair(1, "ali"),
                std::make_pair(2, "mustafa")
                // Aynı anathardan var!
        };
        auto it2 = mp.rbegin(); //tersten
        std::cout << it2->first << " : " << it2->second << std::endl; //"2 : ahmet"
        it2++;
        std::cout << it2->first << " : " << it2->second << std::endl; //"1 : ali"
}
```

Haritaya Çift Ekleme

Haritaya anahtar değerler aşağıdaki şekilde eklenebilir;

```
std::map< std::string, size_t > meyvesayisi;
meyvesayisi.insert({"üzüm", 20});
meyvesayisi.insert(make_pair("portakal", 30));
meyvesayisi.insert(pair<std::string, size_t>("muz", 40));
meyvesayisi.insert(map<std::string, size_t>::value_type("çilek", 50));
```

`insert()` fonksiyonu bir yineleyici ve bir `bool` değerden oluşan bir çift döndürür. Döndürülen değer `true` ise haritaya çift eklenmiştir. Değilse eklenmeye çalışılan çift zaten haritada vardır.

```
auto success = meyvesayisi.insert({"üzüm", 20});
if (!success.second) { // zaten üzüm var
        success.first->second += 20; // üzüm müktarı değiştirlebilir
}
meyvesayisi["karpuz"]=10; //haridata olmayan karpuz eklenmişir.
meyvesayisi.insert({{"şeftali", 1}, {"kayısı", 1}, {"limon", 1}, {"mango", 7}});
```

Konteynerlere başlatma listeleriyle de eleman eklenebilir;

```
std::map< std::string, size_t > meyvelistesi{ {"zeytin", 0}, {"elma", 0}, {"kavun", 0}};
meyvesayisi.insert(meyvelistesi.begin(), meyvelistesi.end());
```

Haritaya eklenecek çiftler yukarıdaki örneklerde olduğu gibi `make_pair()` ve `emplace()` yöntemleriyle hazırlanabilir;

```
meyvesayisi.emplace("Harnup", 200);
```

Haritalarda bir anahtarın ilk oluşumunun yineleyicisini almak için `find()` fonksiyonu kullanılabilir. Anahtar mevcut değilse `end()` döndürür;

```
std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} };
auto it = mmp.find(6);
if(it!=mmp.end())
        std::cout << it->first << ", " << it->second << std::endl; // 6, 5
else
        std::cout << "Aranan Anahtar Bulunamadı!" << std::endl;
it = mmp.find(66);
if(it!=mmp.end())
        std::cout << it->first << ", " << it->second << std::endl;
else
        std::cout << "Aranan anahtar bulunamadı!" << std::endl;
```

Haritada Çift Arama

Haritalarda bir anahtarın mevcut olduğunu bulmanın bir başka yolu da `count()` fonksiyonunu kullanmaktır. Bu fonksiyon, belirli bir anahtarla ilişkili değer sayısını sayar;

```
std::map< int , int > mp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} };
if(mp.count(3) > 0)
        std::cout << "Anahtar bulundu!" << std::endl;
else
        std::cout << "Anahtar Bulunamadı!" << std::endl;
```

Haritaları Başlatma

Haritaları tanımlama ve ilk değer verme;

```
std::multimap < int, std::string > mmp { std::make_pair(2, "C Lang"),
        std::make_pair(1, "CPP Lang"),
        std::make_pair(2, "Java Lang") };
std::map < int, std::string > mp { std::make_pair(2, "C lang"),
        std::make_pair(1, "CPP Lang"),
        std::make_pair(2, "Java Lang") // Eklenmeyecek!
};
// Yineleyici kullanarak;
std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {6, 8}, {3, 4}, {6, 7} };
auto it = mmp.begin();
std::advance(it,3); //baştan üçüncü çifte git {6, 5}
std::map< int, int > mp(it, mmp.end()); // {6, 5}, {8, 9}, {6, 8}, {3, 4}, {6, 7}
//std::pair dizisi kullanarak;
std::pair< int, int > arr[10];
arr[0] = {1, 3};
arr[1] = {1, 5};
arr[2] = {2, 5};
arr[3] = {0, 1};
std::map< int, int > mp(arr,arr+4); //{0 , 1}, {1, 3}, {2, 5}
//std::pair vektörü kullanarak
std::vector< std::pair<int, int> > v{ {1, 5}, {5, 1}, {3, 6}, {3, 2} };
std::multimap< int, int > mp(v.begin(), v.end()); // {1, 5}, {3, 6}, {3, 2}, {5, 1}
```

Haritaların boş olup olmamasına bağlı olarak `true` veya `false` döndüren bir üye `empty()` fonksiyonuna sahiptir. Ayrıca sahip olunan eleman sayısı da `size()` üye fonksiyonuna da sahiptir.

```
std::map<std::string , int> rank {{"facebook.com", 1} ,{"google.com", 2}, {"youtube.com", 3}};
if(!rank.empty())
        std::cout << "Hatitadadaki eleman sayısı: " << rank.size() << std::endl;
else
        std::cout << "Harita Boş!" << std::endl;
```

Karma Harita ve Düzenlenmemiş Harita

Bir de **karma harita** (hash map) vardır. **Düzenlenmemiş harita** (unordered map) olarak da adlandırılan bu haritalar, normal bir haritaya benzer şekilde anahtar-değer çiftlerini depolar. Ancak elemanları anahtara göre sıralamaz. Bunun yerine, anahtar için bir **karma** (hash) değer, ihtiyaç duyulan anahtar-değer çiftlerine hızlı bir şekilde erişmek için kullanılır;

```
#include <string>
#include <unordered_map>
std::unordered_map<std::string, size_t> meyveSayisi;
```

Haritadan Çift Silme

Haritadaki elemanlar aşağıdaki şekilde silinebilir;

```
std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} };
mmp.clear(); //hepsini sil
auto it = mmp.begin();
std::advance(it,3); // baştan üçüncü elemana git {6, 5}
mmp.erase(it); // {1, 2}, {3, 4}, {8, 9}, {3, 4}, {6, 7}
```

Belli anahtarlara sahip elemanlar `erase()` üye fonksiyonu ile silinebilir;

```
std::multimap< int , int > mmp{ {1, 2}, {3, 4}, {6, 5}, {8, 9}, {3, 4}, {6, 7} };
mmp.erase(6); // {1, 2}, {3, 4}, {3, 4}, {8, 9}
```

§15.9.4 std::list ve std::forwardlist

Konteynerin herhangi bir yerinden öğelerin hızlı bir şekilde eklenmesini ve kaldırılmasını istiyorsak ileri liste (`std::forward_list`) iyi bir seçimdir. Hızlı rastgele erişim desteklenmez. Bu konteyner, `std::list` ile karşılaştırıldığında, çift yönlü yineleme gerekmediğinde daha fazla alan tasarrufu sağlar.

```
#include <forward_list>
#include <list>
#include <string>
#include <iostream>
template<typename T> std::ostream& operator<<(std::ostream& s, const std::forward_list<T>& v) {
        s << "Forward List";
        s.put('[');
        char comma[3] = {'\0', ' ', '\0'};
        for (const auto& e : v) {
                s << comma << e;
                comma[0] = ',';
        }
        return s << ']';
}
template<typename T> std::ostream& operator<<(std::ostream& s, const std::list<T>& v) {
        s << "List";
        s.put('[');
        char comma[3] = {'\0', ' ', '\0'};
        for (const auto& e : v) {
                s << comma << e;
                comma[0] = ',';
        }
        return s << ']';
}
int main() {
        std::forward_list<std::string> words1 {"İlhan", "Ahmet", "Mehmet", "Mustafa",
         ↪  "Abdullah"};
        std::cout << "İsimler1: " << words1 << '\n';
        std::list<std::string> words2 {"İlhan", "Ahmet", "Mehmet", "Mustafa", "Abdullah"};
        std::cout << "İsimler2: " << words2 << '\n';
}
/* Program Çıktısı:
İsimler1: Forward List[İlhan, Ahmet, Mehmet, Mustafa, Abdullah]
İsimler2: List[İlhan, Ahmet, Mehmet, Mustafa, Abdullah]
*/
```

§15.9.5 std::set ve std::multiset

Küme (`std::set`), elemanları sıralanmış ve benzersiz olan bir tür konteynerdir. Kümede aynı eleman iki defa bulunmaz! Ancak `std::multiset` birden fazla eleman aynı elemana sahip olabilir.

```
#include <iostream>
#include <set>
#include <stdlib.h>
struct custom_compare final
{
        bool operator() (const std::string& left, const std::string& right) const
        {
                int nLeft = atoi(left.c_str());
                int nRight = atoi(right.c_str());
                return nLeft < nRight;
        }
};
int main () {
        std::set<std::string> sut({"1", "2", "5", "23", "6", "290"});
        std::cout << "Ön tanımlı sıralama std::set<std::string> :" << std::endl;
        for (auto &&data: sut)
                std::cout << data << ", ";
        std::set<std::string, custom_compare> sut_custom({"1", "2", "5", "23", "6",
        ↪  "290"},custom_compare{});
        std::cout << std::endl << "Özel Sıralama :" << std::endl;
        for (auto &&data : sut_custom)
                std::cout << data << ", ";
        auto compare_via_lamda = [](auto &&lhs, auto &&rhs){ return lhs > rhs; };
        using set_via_lamda = std::set<std::string, decltype(compare_via_lamda)>;
        set_via_lamda sut_reverse_via_lamda({"1", "2", "5", "23", "6", "290"},compare_via_lamda);
        std::cout << std::endl << "Lamda Sıralaması :" << std::endl;
        for (auto &&data : sut_reverse_via_lamda)
                std::cout << data << ", ";
        return 0;
}
/*g++ main.cpp -std=C++14 ile derlenen Program Çıktısı:
Ön tanımlı sıralama std::set<std::string> :
1, 2, 23, 290, 5, 6,
Özel Sıralama :
1, 2, 5, 6, 23, 290,
Lamda Sıralaması :
6, 5, 290, 23, 2, 1,
*/
```

§15.9.6 std::optional

Değeri mevcut olabilen veya olmayabilen bir veri tipi temsil etmek için kullanılan `std:optional` olarak tanımlanan değişkenler isteğe bağlı/belki tipleri olarak da bilinir.

C++17'den önce, `nullptr` değerine sahip göstericilere sahip olmak genellikle bir değerin yokluğunu temsil ediyordu. Bu, dinamik olarak tahsis edilmiş ve zaten göstericiler tarafından yönetilen büyük nesneler için iyi bir çözümdür. Ancak, bu çözüm nadiren göstericiler tarafından dinamik olarak tahsis edilen veya yönetilen `int` gibi küçük veya ilkel türler için iyi çalışmaz. Çünkü bu veri tipinde tanımlanan bir değişken her zaman değeri vardır. `std::optional` bu yaygın soruna uygulanabilir bir çözüm sağlar.

```
#include <iostream>
#include <optional>
#include <string>
struct Hayvan {
        std::string adi;
};
struct Person {
        std::string adi;
        std::optional<Hayvan> evcilhayvan;
};
```

```
int main() {
        Person person;
        person.adi = "Ali";
        if (person.evcilhayvan) {
                std::cout << person.adi << " Adlı Kişinin " << person.evcilhayvan->adi
                << "Evcil Hayvanı Var" << std::endl;
        }
        else {
                std::cout << person.adi << " adlı kişinin evcil hayvanı yok!" << std::endl;
        }
}
/* g++ -std=c++17 main.cpp olarak derlenir. Program Çıktısı:
Ali adlı kişinin evcil hayvanı yok!
*/
```

Fonksiyondan geri dönüş değeri olarak da `std::optional` kullanılabilir;

```
std::optional<float> divide(float a, float b) {
        if (b!=0.f) return a/b;
        return {};
}
```

std:optional olarak tanımlı bir değeri işlemek için `value_or()` kullanılır;

```
void print_name( std::ostream& os, std::optional<std::string> const& name ) {
        std::cout "Name is: " << name.value_or("<name missing>") << '\n';
}
```

§15.9.7 std::function ve std::bind

Herhangi bir fonksiyona kılıf çekmek için kullanılır. C tipi fonksiyon göstericisi kullanmak yerine C++ dilinde `std::function` kullanılır.

```
#include <iostream>
#include <functional>
std::function<void(int , const std::string&)> myFuncObj;
void theFunc(int i, const std::string& s) {
        std::cout << s << ": " << i << std::endl;
}
int main(int argc, char *argv[]) {
        myFuncObj = theFunc;
        myFuncObj(10, "hello world");
}
```

Argümanlarla bir fonksiyonu geri çağırmamız gereken bir durumda `std::bind` ile kullanılan `std::function` aşağıda gösterildiği gibi çok güçlü bir tasarım yapısı verir.

```
#include <iostream>
#include <functional>
class Ocak {
        public:
        std::function<void(int, const std::string&)> YumurtaKaynadi = nullptr;
        void YumurtaKaynat() {
                if (YumurtaKaynadi) {
                        YumurtaKaynadi(10, "Yumurta Pişti");
                }
        }
};
class Kisi {
        public:
        Kisi() {
                auto yumurtaPisinceHaberVer = std::bind(&Kisi::eventHandler, this,
                std::placeholders::_1,
                std::placeholders::_2);
                ocakNesnesi.YumurtaKaynadi = yumurtaPisinceHaberVer;
        }
        void eventHandler(int i, const std::string& s) {
```

```
                std::cout << i << " dakikada "<< s <<std::endl;
        }
        void OcaktaYumurtaKaynat() {
                ocakNesnesi.YumurtaKaynat();
        }
        Ocak ocakNesnesi;
};
int main(int argc, char *argv[]) {
        Kisi ali;
        ali.OcaktaYumurtaKaynat();
}
/*Program Çıktısı:
10 dakikada Yumurta Pişti
*/
```

Aşağıda `std:function` ile çeşitli fonksiyonların çağrılmasını gösteren bir program verilmiştir;

```
#include <iostream>
#include <functional> //std::function, std::placeholders
#include <vector>  //std:vector
double c_function(int x, float y, double z) {
        double res = x + y + z;
        std::cout << "c_function çağrıldı: " << x << "+" << y << "+" << z << "=" << res  <<
        ↪  std::endl;
        return res;
}
struct yapi
{
        double yapi_function(int x, float y, double z) {
                double res = x + y + z;
                std::cout << "yapi::yapi_function çağrıldı: " << x << "+" << y << "+" << z << "="
                ↪  << res << std::endl;
                return res;
        }
        double farkli_yapi_function(int x, double z, float y, long xx) {
                double res = x + y + z + xx;
                std::cout << "farkli_yapi_function çağrıldı: "  << x << "+" << y << "+" << z <<
                ↪  "+" << xx << "=" << res
                << std::endl;
                return res;
        }
        double operator()(int x, float y, double z) {
                double res = x + y + z;
                std::cout << "yapi::operator() çağrıldı: " << x << "+" << y << "+" << z << "=" <<
                ↪  res << std::endl;
                return res;
        }
};
int main(void) {
        using function_type = std::function<double(int, float, double)>;
        yapi ornekyapi;
        std::vector<function_type> bindings;
        function_type var1 = c_function;
        bindings.push_back(var1);
        function_type var2 = std::bind(&yapi::yapi_function, ornekyapi, std::placeholders::_1,
        ↪  std::placeholders::_2, std::placeholders::_3);
        bindings.push_back(var2);
        function_type var3 = std::bind(&yapi::farkli_yapi_function, ornekyapi,
        ↪  std::placeholders::_1, std::placeholders::_3, std::placeholders::_2, 11l);
        bindings.push_back(var3);
        function_type var4 = ornekyapi; //operator()
        bindings.push_back(var4);
        function_type var5 = [](int x, float y, double z) {
```

```
                double res = x + y + z;
                std::cout << "lamda çağrıldı: " << x << "+" << y << "+" << z << "=" << res <<
                ↪ std::endl;
                return res;
        };
        bindings.push_back(var5);
        std::cout << "Vektöre saklana fonksiyonlar test ediliyor: x = 1, y = 2, z = 3" <<
        ↪ std::endl;
        for (auto f : bindings)
                f(1, 2, 3);
}
/*Program Çıktısı:
Vektöre saklana fonksiyonlar test ediliyor: x = 1, y = 2, z = 3
c_function çağrıldı: 1+2+3=6
yapi::yapi_function çağrıldı: 1+2+3=6
farkli_yapi_function çağrıldı: 1+2+3+11=17
yapi::operator() çağrıldı: 1+2+3=6
lamda çağrıldı: 1+2+3=6
*/
```

§15.9.8 std::tupple

Farklı tiplerdeki değerler de dahil olmak üzere herhangi bir sayıda değeri tek bir dönüş nesnesine toplamak için `std::tuple` kullanılır.

```
        içerik...
std::tuple<int, int, int, int> foo(int a, int b) { // or auto (C++14)
        return std::make_tuple(a + b, a - b, a * b, a / b);
        /*dört farklı int değeri geri döndürülüyor */
}
```

C++ 2017 uyarlamasından sonra tırnaklı parantez olan başlatma listesi kullanılır;

```
std::tuple<int, int, int, int> foo(int a, int b) {
        return {a + b, a - b, a * b, a / b};
}
```

Döndürülen tuple'dan değerleri almak zahmetlidir ve `std::get` şablon fonksiyonunun kullanımını gerektirir:

```
auto mrvs = foo(5, 12);
auto add = std::get<0>(mrvs);
auto sub = std::get<1>(mrvs);
auto mul = std::get<2>(mrvs);
auto div = std::get<3>(mrvs);
```

Eğer tipler fonksiyon dönmeden önce bildirilebiliyorsa, o zaman `std::tie` bir tuple'ı mevcut değişkenlere açmak için kullanılabilir:

```
int add, sub, mul, div;
std::tie(add, sub, mul, div) = foo(5, 12);
```

`std::tie` ile döndürülen değerlerden birine ihtiyaç duyulmuyorsa `std::ignore` kullanılabilir:

```
std::tie(add, sub, std::ignore, div) = foo(5, 12);
```

Yapılandırılmış bağlamalar `std::tie`'dan kaçınmak için kullanılabilir:

```
auto [add, sub, mul, div] = foo(5,12);
```

§15.9.9 std::string

Metinlerin (string) bellekte son karakteri sıfır olan karakter dizileri olarak tutulurlar. Dizgiler, boş bir karakter (NULL Character) olan '\0' ile sonlanan karakter dizisi olarak saklandığı 9.12 başlığında anlatılmıştı.

Modern C++ dilinde, `std::string` şablonu metinler için kullanılır. C dilinden kalan karakter dizisi yaklaşımını kullanmak çok daha düşük seviyelidir ve bellek yönetimi tamamen yazılımcının sorumluluğundadır. C++ `std::string` sınıfı, `std` isim uzayının bir parçasıdır ve 1998'de standartlaştırılmıştır.

Metin Değişkenlerini Başlatma

Metinleri tutan değişkenler aşağıdaki şekilde tanımlanabilir;

```
std::string metin1 = "Merhaba";
std::string metin2("İlhan");
std::string metin3={'M','e','r','h','a','b','a'}; //Sonunda '\0' karakteri olamadan karakter
↪ dizisi
std::string metin4=metin2; //önceden tanımlı bir metinden
std::string metin5(metin3); //önceden tanımlı bir metinden yapıcı ile
```

Metni Parçalara Bölme

Dizgiler belirlenen bir ayraç metni ile belirteç (token) denilen küçük parçalara strtok() fonksiyonu ile ayrılabilir;

```
std::string str{ "Pijamalı hasta yağız şoföre çabucak güvendi." };
vector<std::string> tokens;
for (auto i = strtok(&str[0], " "); i != NULL; i = strtok(NULL, " "))
        tokens.push_back(i);
```

Bir `std::string` verilerine `const char*` göstericisi ile erişim sağlamak için `c_str()` üye yöntemi kullanılır;

```
//Göstericiler str nesnesinin altında yatan karakter dizisini gösterir;
std::string str("This is a string.");
const char* cstr = str.c_str(); // cstr göstericisi bu metni gösterir: "This is a string.\0"
const char* data = str.data();  // data göstericisi bu metni gösterir: "This is a string.\0"

//Göstericilerin str nesnesinden ömür boyu bağını çözmek için kopyasını oluşturun
std::string str("This is a string.");
std::unique_ptr<char []> cstr = std::make_unique<char[]>(str.size() + 1);
// Yukarıdaki satırın eşdeğeri: char* cstr_unsafe = new char[str.size() + 1];
std::copy(str.data(), str.data() + str.size(), cstr);
cstr[str.size()] = '\0'; // NULL karakter sonuna eklenir.
// delete[] cstr_unsafe;
std::cout << cstr.get();
```

Buradan da anlaşılacağı üzere `std::string` şablonu arka planında metinler karakter dizileri şeklinde tutulur. `std:string` altında yatan `const char*` üzerinden işlem yapmak yerine C++17 ile birlikte gelen sabit

std::string	**Karakter Dizisi**
`std::string`, akış (stream) olarak temsil edilebilen nesneleri tanımlar.	`'\0'` olarak kodlanan boş karakter (NULL Character), karakterlerden oluşan bir karakter dizisini sonlandırır.
`std::string` dizi bozulması meydana gelmez çünkü dizeler nesneler olarak temsil edilir.	Bir dizinin tip ve boyut kaybına dizinin bozulması (decay of array) denir. Bu durum genellikle diziyi değer veya gösterici ile fonksiyona geçirdiğimizde ortaya çıkar. Karakter dizisinde böyle bir durum ortaya çıkabilir.
Bir `std::string` sınıfı, metinleri işlemek için çok sayıda yöntem sağlar.	Karakter dizileri, metinleri işlemek için C++ dilinde yerleşik fonksiyonlar sunmaz.
`std::string` metinlerine bellek dinamik olarak tahsis edilir.	Karakter dizisine boyutu kadar veri bellekte (data segment) veya yığın bellekte (stack segment) yer statik olarak tahsis edilir.

Tablo 15.2: std::string ile Karakter Dizilerinin Karşılaştırması

metin şablonu olan `std::string_view` üzerinden işlemler yapılır. Değiştirilemeyen metin verileri gerektiren işlevler için üstünlük sağlar;

```
//1)Alt metin elde etmek için kopya oluşturarak;
std::string str = "Çoook uzun biiiir metiiiiiin";
//'string::substr' returns a new string (expensive if the string is long)
std::cout << str.substr(15, 10) << '\n';
//2)Hiçbir kopya oluşturmadan;
std::string_view view = str;
// string_view::substr returns a new string_view
std::cout << view.substr(15, 10) << '\n';
```

Geniş Karakterli Metinler

std::string, char veri tipindeki elemanlarla oluşturulmuştur. std::wstring ise uluslararası karakterleri temsil eden wchar_t tipindeki geniş karakterler (wide character) oluşturulmuştur. Bu karakterlerden oluşmuş metin değişmezleri için L ön eki kullanılır. Bunlar arasında çeviri yapılabilir;

```
#include <iostream>
#include <string>
#include <codecvt>
#include <locale>
int main() {
        std::locale::global(std::locale(""));
        std::wcout.imbue(std::locale());
        setlocale(LC_ALL, "Turkish");
        const wchar_t message_turkish[] = L"ıİüÜğĞiİşŞçÇöÖ"; //Türkçe Karakterler
        //std::wcout << message_turkish << std::endl;
        std::string input_str = "İlhan-this is a -string-";
        std::wstring input_wstr = L"İlhan-this is a -wide- string";
        // conversion
        std::wstring str_turned_to_wstr =
        std::wstring_convert<std::codecvt_utf8<wchar_t>>().from_bytes(input_str);
        std::wcout << str_turned_to_wstr << std::endl;
        std::string wstr_turned_to_str =
        std::wstring_convert<std::codecvt_utf8<wchar_t>>().to_bytes(input_wstr);
        std::cout << wstr_turned_to_str << std::endl;
        wstr_turned_to_str =
        std::wstring_convert<std::codecvt_utf8<wchar_t>>().to_bytes(message_turkish);
        std::cout << wstr_turned_to_str << std::endl;
}
```

Metni Oluşturan Karakter Dizisine Erişme

Dizgenin altında yatan karakter dizisine [] işleci ile at(), front() ve back() üye fonksiyonları ile erişilebilir;

```
std::string str("Hello world!");
char c1 = str[6]; // 'w'
char c2 = str.at(7); // 'o'
char c3 = str.front(); // 'H'
char c4 = str.back(); // '!'
```

Dizginin altında yatan karakter dizisi üzerinde yineleme yapılabilir;

```
std::string str = "Hello World!";
//1) Foreach
for (auto c : str)
        std::cout << c;
//2) Geleneksel
for (std::size_t i = 0; i < str.length(); ++i)
        std::cout << str[i];
```

Metni Diğer Veri Tiplerine Dönüştürme

Metinleri diğer veri tiplerine de dönüştürebiliriz;

```
std::string ten = "10";
int num1 = std::stoi(ten);
long num2 = std::stol(ten);
long long num3 = std::stoll(ten);
float num4 = std::stof(ten);
double num5 = std::stod(ten);
long double num6 = std::stold(ten);
```

Metinleri dönüştürürken sayı tabanları da kullanılabilir;

```
std::string ten = "10";
std::string ten_octal = "12";
std::string ten_hex = "0xA";
int num1 = std::stoi(ten, 0, 2); // Returns 2
```

```
int num2 = std::stoi(ten_octal, 0, 8); // Returns 10
long num3 = std::stol(ten_hex, 0, 16); // Returns 10
long num4 = std::stol(ten_hex); // Returns 0
long num5 = std::stol(ten_hex, 0, 0); // Returns 10 as it detects the leading 0x Metinler + ve +=
↪ işleçleri birleştirilebilir;
std::string hello = "Hello";
std::string world = "world";
std::string helloworld = hello + world; // "Helloworld"
hello += world; // "Helloworld"
```

Metnin Sonuna Yeni Metin Ekleme

Metinler birinin sonuna diğeri gelecek şekilde `append()` üye fonksiyonu ile birleştirilebilir;

```
std::string app = "test and ";
app.append("test"); // "test and test"
```

Metnin İçinde Karakter Arama

Bir karakteri veya başka bir dizeyi bulmak için `find()` kullanabilirsiniz. İlk eşleşmenin ilk karakterinin konumunu döndürür. Eşleşme bulunamazsa, `std::string::npos` döndürür. `find_first_of()` ile karakterlerin ilk oluşumu bulunur, `find_first_not_of()` ile karakterlerin ilk yokluğunu bulunur, `find_last_of()` ile karakterlerin son oluşumu bulunur, `find_last_not_of()` ile karakterlerin son yokluğu bulunur;

```
std::string str = "Hello world!";
auto it = str.find("world");
if (it != std::string::npos)
        std::cout << "Found at position: " << it << '\n';
        else
std::cout << "Not found!\n";
```

Metin İşleme Fonksiyonları

C++ dilinde metinleri kopyalamak ve birleştirmek için `strcpy()` ve `strcat()` fonksiyonları gibi metin işleme için kullanılan bazı yerleşik yöntemlere sahiptir. Bunlardan Tablo 15.3'de verilmiştir.

Bunların yanında metni tutan metin değişkeni kullanılarak metin karakterleri üzerinde işlem yapmak için yineleme (itertation) yöntemlerini kullanabiliriz. Yineleme Şablonları altında örneği verilmiştir.

Klavyeden Bir Satır Metin Okuma

Bir girdi nesnesinden metin okumak için en yaygın yol, C++ dininde akıştan veri **ayıklama işleci** (extraction operator) olan (`>>`) ile `std::cin` nesnesini kullanmaktır.

```
#include <iostream>
#include <string>
int main() {
        std::string metin;
        std::cout << "Bir metin Giriniz: ";
        std::cin >> metin;  //sadece ilk sözcük metin değişkenine atanır!
        std::cout<< "Girilen Metin: " << metin << std::endl;
}
/* Program Çalıştığında:
Bir metin Giriniz: Ilhan Ozkan
Girilen Metin: Ilhan
*/
```

Program incelendiğinde girilen metin iki sözcükten okunmasına rağmen ilk sözcük metin değişkenine atanmıştır. Eğer girilen tüm satırı bu metne aktarmak istersek `<string>` başlık dosyasındaki `getline()` fonksiyonunu kullanmalıyız;

```
#include <iostream>
#include <string>
int main() {
        std::string metin;
        std::cout << "Bir metin Giriniz: ";
        getline(std::cin, metin); // Satır sonu karakteri girilene kadarki metin değişkene
        ↪ atanır!
        std::cout<< "Girilen Metin: " << metin << std::endl;
```

Fonksiyonlar	Açıklama
length()	Metnin karakter uzunluğunu döndürür.
swap()	İki metni yer değiştirmek için kullanılır.
size()	Metnin boyutunu bulmak için kullanılır. Metne bellekte kaç karakter yer ayrılmış onu döner.
resize()	Metnin uzunluğunu belirtilen karakter sayısına kadar yeniden boyutlandırmak için kullanılır.
find()	Metnin içinde parametre olarak geçirilen bir başka metni arar.
push_back()	Parametre olarak geçirilen karakteri metnin sonuna eklemek kullanılır.
pop_back()	Metinden son karakteri çıkarmak için kullanılır.
clear()	Metnin tüm karakterlerini silmek için kullanılır.
strncmp()	Metin ile parametre olarak geçirilen bir başka metnin en fazla ilk num kadar karakterini karşılaştırır.
strcpy()	Kaynak metni hedef metin değişkenine kopyalar
strncpy()	strcpy() fonksiyonuna benzerdir, ancak en fazla n bayt kopyalanır.
strrchr()	Metindeki bir karakterin son bulunduğu yeri bulur.
strcat()	Kaynak metnin bir kopyasını hedef metnin sonuna ekler.
find()	Bir metnin içerisinde belirli bir alt metni aramak için kullanılır ve alt metnin ilk karakterinin konumunu döndürür.
replace()	first-last aralığındaki eski değere eşit olan her bir elemanı yeni değerle değiştirmek için kullanılır
substr()	Verilen bir metinden bir alt metin oluşturmak için kullanılır.
compare()	İki metni karşılaştırmak ve sonucu tam sayı biçiminde döndürmek için kullanılır.
erase()	Bu fonksiyon bir metnin belli bir kısmını silmek için kullanılır.
rfind()	Metnin son bulunduğu konumu bulmak için kullanılır.
capacity()	Bu fonksiyon, derleyici tarafından dizgiye tahsis edilen kapasiteyi döndürür.
shrink_to_fit()	Bu fonksiyon dizginin kapasitesini en az (minimum) olacak şekilde azaltır.

Tablo 15.3: Metin İşleme Yöntemleri

```
}
/* Program Çalıştığında:
Bir metin Giriniz: İlhan Özkan
Girilen Metin: İlhan Özkan
*/
```

Klavyeden Geniş Karakterli Metin Okuma

Uluslararası karakterlerle çalışılan aşağıdaki örnek verilebilir. Bu kodu derlemede hata almaya devam ediyorsak linux üzerinde gerekli dil paketlerini `sudo locale-gen tr_TR.UTF-8` ve `sudo update-locale` komutlarını art arda vererek kurmalıyız.

```
#include <iostream>
#include <string>
int main() {
        std::wstring message_turkish = L"ıIüÜğĞiİşŞçÇöÖ"; //Türkçe "Diğer Karakterler
        std::locale::global(std::locale(""));
        std::wcout.imbue(std::locale());
        setlocale(LC_ALL, "Turkish");
        std::wcout << message_turkish << std::endl;
        wchar_t message[100];
        std::wcout << L"Tükçe Bir Metin Giriniz:";
        std::wcin.getline(message,100);
        std::wcout << "Girilen metin:" << message <<std::endl;
        std::wstring message2;
        std::wcout << L"Tükçe Bir Başka Giriniz:";
        getline(std::wcin, message2);
        std::wcout << L"Girilen metin:" << message2;
```

}

15.10 Algoritma Şablonları

Standart Şablon Kütüphanesi'ndeki algoritmalar (algotrithm), konteynerler üzerinde işlem yapmak için özel olarak tasarlanmış büyük bir fonksiyon kümesidir. Bu fonksiyonlar, konteynerlerin iç yapılarını bilmeye gerek kalmadan konteynerler arasında geçiş yapmak için kullanılan yineleyiciler (iterator) kullanılarak uygulanır.

Sıra değiştirmeyen algoritmalar;

- `for_each`: Bir aralıktaki her bir elemanı bulmak için bir fonksiyonda kullanılır.
- `count`: Bir aralıktaki değerin oluşum sayısını sayar.
- `find`: Bir aralıktaki değerin ilk oluşumunu bulur.
- `find_if`: Bir yordamı karşılayan ilk elemanı bulur.
- `find_if_not`: Bir yordamı karşılamayan ilk elemanı bulur.
- `equal`: İki aralığın eşit olup olmadığını kontrol eder.
- `search`: Bir dizi içinde bir alt diziyi arar.

Sıra değiştiren algoritmalar;

- `copy`: Elemanları bir aralıktan diğerine kopyalar.
- `copy_if`: Bir yordamı karşılayan elemanları başka bir aralığa kopyalar.
- `copy_n`: Belirli sayıda elemanı bir aralıktan diğerine kopyalar.
- `move`: Elemanları bir aralıktan diğerine taşır.
- `transform`: Bir aralığa bir fonksiyon uygular ve sonucu depolar.
- `remove`: Belirli bir değere sahip elemanları bir aralıktan kaldırır.
- `remove_if`: Bir yordamı karşılayan elemanları kaldırır.
- `unique`: Ardışık yinelenen elemanları kaldırır.
- `reverse`: Bir aralıktaki elemanların sırasını tersine çevirir.
- `swap`: Elemanları değiştirir.

Sıralama algoritmaları;

- `sort`: Bir aralıktaki elemanları sıralar.
- `stable_sort`: Eşdeğer elemanların göreli sırasını koruyarak elemanları sıralar.
- `partial_sort`: Bir aralığın bir kısmını sıralar.
- `nth_element`: Aralığı, n'inci elemanın son konumunda olacak şekilde bölümlere ayırır.

Arama algoritmaları;

- `binary_search`: Sıralanmış bir aralıkta bir elemanın bulunup bulunmadığını kontrol eder.
- `lower_bound`: Sırayı korumak için bir elemanın eklenebileceği ilk konumu arar.
- `upper_bound`: Belirtilen değerden büyük olan ilk elemanın konumunu arar.
- `equal_range`: Eşit elemanların aralığını döndürür.

Öbek (heap) algoritmaları;

- `make_heap`: Bir aralıktan bir öbek oluşturur.
- `push_heap`: Bir öbeğe bir eleman ekler.
- `pop_heap`: Bir öbekten en büyük eleman kaldırır.
- `sort_heap`: Bir öbekteki elemanları sıralar.

Küme algoritmaları;

- `set_union`: İki kümenin birleşimini hesaplar.
- `set_intersection`: İki kümenin kesişimini hesaplar.
- `set_difference`: İki küme arasındaki farkı hesaplar.
- `set_symmetric_difference`: İki küme arasındaki simetrik farkı hesaplar.

Sayısal algoritmalar;

- `accumulate`: bir aralığın toplamını (veya diğer işlemleri) hesaplar.
- `inner_product`: İki aralığın iç çarpımını hesaplar.
- `adjacent_difference`: Bitişik elemanlar arasındaki farkları hesaplar.
- `partial_sum`: bir aralığın kısmi toplamlarını hesaplar.

Diğer algoritmalar;

- `generate`: Bir fonksiyon tarafından üretilen değerlerle bir aralığı doldurur.
- `shuffle`: Bir aralıktaki elemanları rastgele karıştırır.
- `partition`: Elemanları bir öngörüye göre iki gruba ayırır.

Bu algoritmalar genellikle <algorithm> başlığı altında toplanır ve yineleyiciler (iterator) aracılığıyla veri yapıları (vector, list, array) üzerinde işlem yapar. Aşağıda birkaçına örnek verilmiştir;

1. std::sort (Sıralama Algoritması) :En sık kullanılan şablonlardan biridir. Verilen aralıktaki elemanları (varsayılan olarak küçükten büyüğe) sıralar.

```
#include <iostream>
#include <vector>
#include <algorithm> // Algoritma şablonları için
int main() {
        std::vector<int> sayilar = {50, 10, 40, 20, 30};
        // Şablon her türlü karşılaştırılabilir tip için çalışır
        std::sort(sayilar.begin(), sayilar.end());
        for(int n : sayilar)
                std::cout << n << " "; // Çıktı: 10 20 30 40 50
}
```

2. std::find (Arama Algoritması): Bir veri kümesi içinde belirli bir değeri arar. Eğer değeri bulursa o elemana ait bir yineleyici, bulamazsa sonu (end()) döndürür.

```
#include <iostream>
#include <vector>
#include <algorithm>
int main() {
        std::vector<std::string> isimler = {"Ali", "Veli", "Ayse"};
        // "Veli" ismini arıyoruz
        auto it = std::find(isimler.begin(), isimler.end(), "Veli");
        if (it != isimler.end()) {
                std::cout << "Bulundu: " << *it << std::endl;
        } else {
                std::cout << "Bulunamadi!" << std::endl;
        }
}
```

3. std::for_each (Her Eleman İçin İşlem): Belirli bir aralıktaki her elemana, sizin belirlediğiniz bir fonksiyonu veya lamda ifadesini uygular.

```
#include <iostream>
#include <vector>
#include <algorithm>
int main() {
        std::vector<int> v = {1, 2, 3, 4, 5};
        // Her elemanın karesini ekrana yazdıran bir lambda şablonu
        std::for_each(v.begin(), v.end(), [](int &n) {
                n *= n;
                std::cout << n << " ";
        }); // Çıktı: 1 4 9 16 25
}
```

4. std::accumulate (Biriktirme/Toplam Algoritması): Bir aralıktaki tüm değerleri toplamak (veya başka bir işleme sokmak) için kullanılır. <numeric> başlığı altında bulunur.

```
#include <iostream>
#include <vector>
#include <numeric> // accumulate için gerekli
int main() {
        std::vector<double> fiyatlar = {10.5, 20.0, 5.25};
        // 0.0 başlangıç değeri üzerine tüm elemanları ekler
        double toplam = std::accumulate(fiyatlar.begin(), fiyatlar.end(), 0.0);
        std::cout << "Toplam Fiyat: " << toplam << std::endl; // 35.75
}
```

15.11 Yineleme Şablonları

Standart Şablon Kütüphanesi'ndeki yineleyiciler (iterator), bir konteyner içindeki elemanlara gösterici görevi gören ve verilere erişmek ve bunları düzenlemek için tekdüze bir ara yüz sağlayan nesnelerdir. Algoritmalar ve konteynerler arasında bir köprü görevi görürler;

- ◇ Giriş Yineleyicileri (input iterator): Elemanlara salt okunur erişime izin verirler.
- ◇ Çıkış Yineleyicileri (output iterator): Elemanlara salt yazılır erişime izin verirler.
- ◇ İleri Yineleyiciler (forward iterator): Artırılabilen okuma ve yazmaya izin verirler.
- ◇ Çift Yönlü Yineleyiciler (bidirectional iterators): Artırılabilir ve azaltılabilirler.
- ◇ Rastgele Erişim Yineleyicileri (random access iterator): Aritmetik işlemleri desteklerler ve elemanlara doğrudan erişebilirler.

Yineleme nesneleri, diziler gibi birden çok aynı nesneyi tutan veri yapılarında elemanlar üzerinde hareket etmeyi sağlayan nesnelerdir.

Fonksiyon	Açıklama
`begin()`	Bu fonksiyon, dizginin başlangıcına işaret eden bir yineleyici (`iterator`) nesne döndürür.
`end()`	Bu fonksiyon, dizginin sonunu işaret eden bir yineleyici nesne döndürür.
`rbegin()`	Bu fonksiyon, dizginin başlangıcına işaret eden ters yineleyici (`reverse_iterator`) nesne döndürür.
`rend ()`	Bu fonksiyon, dizginin sonunu işaret eden ters yineleyici nesne döndürür.
`cbegin ()`	Bu fonksiyon, dizginin başlangıcına işaret eden bir sabit yineleyici (`const_iterator`) döndürür.
`cend()`	Bu fonksiyon dizginin sonunu işaret eden bir sabit yineleyici (`const_iterator`) döndürür.
`crbegin()`	Bu fonksiyon dizginin sonunu işaret eden bir sabit bir ters yineleyici (`const_reverse_iterator`) döndürür.
`crend()`	Bu fonksiyon, dizginin başlangıcına işaret eden bir ters yineleyici (`const_reverse_iterator`) döndürür.

Tablo 15.4: Yineleme Fonksiyonları

Yineleme şablonlarına ilişkin örnek aşağıda verilmiştir;

```
#include <iostream>
#include <string>
int main() {
        std::string::iterator itr; //string yineleme nesnesi
        std::string::reverse_iterator rit; //string ters yineleme nesnesi
        std::string metin = "Merhana Ilhan.";
        itr = metin.begin();
        std::cout << "Yineleyicinin işaret ettiği karakter: " << *itr<< std::endl;
        itr = metin.end() - 1;
        std::cout << "Yineleyicinin işaret ettiği son metin karakteri: " << *itr << std::endl;
        rit = metin.rbegin();
        std::cout << "Ters yineleyicinin işaret ettiği karakter:: " << *rit << std::endl;
        rit = metin.rend() - 1;
        std::cout << "Ters yineleyicinin işaret ettiği on metin karakteri:: " << *rit <<
         ↪  std::endl;
}
/*Program çalıştığında:
Yineleyicinin işaret ettiği karakter: M
Yineleyicinin işaret ettiği son metin karakteri: .
Ters yineleyicinin işaret ettiği karakter:: .
Ters yineleyicinin işaret ettiği on metin karakteri:: M
*/
```

15.12 Fonksiyon Nesneleri

Fonksiyon nesnesi (function object), durum saklayabilen ve aşırı yüklenmiş fonksiyon işlecine (function operator) `()` sahip nesnelerdir. Bu nesnelere kısaltma olarak functor adı verilir. Bu işleci aşırı yüklemiş nesneler, bir fonksiyonmuş gibi çağrılabilirler ve bir fonksiyon veya fonksiyon göstericisiymiş gibi ele alınırlar.

Aşağıda bir dizinin elemanlarını 1 artıran bir dönüşüm programı verilmiştir;

```
#include <bits/stdc++.h> //transform()
int artirim(int x) { return (x+1); }
int main() {
```

```
        int dizi[] = {1, 2, 3, 4, 5};
        int adet = sizeof(dizi)/sizeof(dizi[0]);
        /* Aşağıdaki dönüşüm fonksiyonu ile dizi elemanları 1 artırılıyor */
        std::transform(dizi, dizi+adet, dizi, artirim);
        for (int i=0; i<adet; i++)
        std::cout << dizi[i] <<" ";
}
/* Program Çıktısı:
2 3 4 5 6
*/
```

Aşağıda bir fonksiyon nesnesi üzerinden aynı işlemi yapan bir program örneği verilmiştir;

```
#include <bits/stdc++.h> // transform
class artirim// Functor
{
        private:
        int sayi;
        public:
        artirim(int n) : sayi(n) { } //Yapıcı işlenecek parametreyi alır
        /* Aşağıda verilen işleç önyüklemesi ile sayi kadar artırım işlemi yapılacaktır. */
        int operator () (int pMiktar) const {
                return sayi + pMiktar;
        }
};
int main() {
        int dizi[] = {1, 2, 3, 4, 5};
        int adet = sizeof(dizi)/sizeof(dizi[0]);
        int miktar = 5;
        std::transform(dizi, dizi+adet, dizi, artirim(miktar));
        for (int i=0; i<adet; i++)
                std::cout << dizi[i] << " ";
}
/*Program Çıktısı:
6 7 8 9 10
*/
```

Aritmetik functor (aritmetic functor), aritmetik operatörler temel matematiksel işlemleri gerçekleştirmek için kullanılır. Toplama (`+`), Çıkarma (`-`), Çarpma (`*`), Bölme (`/`), Kalan (`%`), Negatif (`-`) işleç işlevlerini yerine getirirler.

Karşılaştırma fonksiyon nesneleri (comparision functors), özellikle konteynerlerde sıralama veya arama yapmak için değerleri karşılaştırmak amacıyla kullanılır. Küçüktür (`<`), Büyüktür (`>`), Küçüktür veya Eşittir (`<=`), Büyük veya Eşit (`>=`), Eşit (`=`), Eşit Değil (`!=`) işleç işlevlerini yerine getirirler.

Mantıksal fonksiyon nesneleri (logical functors), Boole mantığını içeren senaryolar için mantıksal işlemleri gerçekleştirir. Mantıksal VE (`&&`), Mantıksal VEYA (`||`) ve Mantıksal DEĞİL (`!`) işleç işlevlerini yerine getirirler.

Bit düzeyi fonksiyon nesneleri (bitwise functors), bit düzeyi işlemleri gerçekleştirir. Bit düzeyi VE (`&`), VEYA (`|`) ve ÖZEL VEYA (`^`) işleç işlevlerini yerine getirirler.

15.13 using Anahtar Kelimesinin Şablonlarda Kullanımı

`using` anahtar kelimesi, 10.10.4, 11.12 ve 12.9 başlıklarındaki kullanımı yanında `using` anahtar kelimesi, bir şablonun sadece bazı parametrelerini sabitleyip geri kalanını esnek bırakabilirsiniz;

```
template <typename T>
using NumerikSozluk = std::map<std::string, T>;

// Kullanımı:
NumerikSozluk<int> yaslar;    // std::map<string, int> yerine
NumerikSozluk<double> notlar; // std::map<string, double> yerine
```

C++14 ile gelen en büyük yeniliklerden biri değişken şablonlarıdır (variable templates). `using` doğrudan bir değişken şablonu olmasa da, tip dönüşümlerinde `using` ile tanımlanmış tiplerin bu yeni değişken şablonlarıyla kullanımı (özellikle `std::enable_if_t` gibi yapılarda) kodun sadeleşmesini sağlamıştır. C++11 uyarlamasında bir tipin `int` olup olmadığını kontrol etmek için uzunca şunu yazardık;

```
typename std::enable_if<std::is_integral<T>::value>::type
```

C++14 ve sonrasında `using` ve değişken şablonları sayesinde bu şuna dönüştü:

```
std::enable_if_t<std::is_integral_v<T>>
```

15.14 Düşün ve Çöz

- **Düşün**: C++'da dizi boyutunun derleme zamanında bilinmesi zorunluluğu neden vardır? `std::vector` ile sabit boyutlu dizi arasındaki temel farkı bellek yönetimi açısından açıklayın.
- **Düşün**: Fonksiyon şablonu (function template) ile sınıf şablonu (class template) arasındaki farkı açıklayın. `std::sort` gibi bir STL algoritmasının şablon mekanizmasına dayandığını gösterin.
- **Düşün**: Akıllı göstericiler (`std::unique_ptr`, `std::shared_ptr`, `std::weak_ptr`) ham göstericiler (raw pointer) göre neden tercih edilmelidir? Her birinin sahiplik (ownership) semantiğini açıklayın.
- **Düşün**: `std::vector` ile `std::array` arasındaki farkları bellek yönetimi, performans ve kullanım kolaylığı açısından karşılaştırın. Hangisi hangi durumda tercih edilmelidir?
- **Çöz**: `std::map` ile `std::unordered_map` arasındaki farkı arama karmaşıklığı (O(log n) vs O(1)) açısından karşılaştırın. Her birinin kullanım senaryosunu örnekleyin.
- **Çöz**: Değişken sayıda şablon parametresi alan (variadic template) bir fonksiyon yazın. `std::tuple` veri yapısının altında yatan şablon mekanizmasını açıklayın.
- **Düşün**: Fonksiyon nesneleri (function objects/functors) lamda ifadelerinden önce neden yaygın kullanılırdı? `std::function` ve `std::bind` ile bu yapıların modern C++'daki yerini açıklayın.
- **Çöz**: `std::string` ile C-tarzı karakter dizisi (`char[]`) arasındaki temel farkları karşılaştırın. Bir C-dizisini `std::string`'e dönüştürmenin güvenli yolunu gösterin.
- **Çöz**: `std::getline` neden `std::cin >>` yerine metin okumak için daha uygun bir seçimdir? Boşluk içeren bir cümleyi okuyup içindeki kelimeleri ayrıştıran bir program yazın.
- **Çöz**: `std::string`'in `find()`, `substr()`, `replace()` ve `erase()` fonksiyonlarını kullanarak bir metin içinde belirtilen bir kelimeyi bulup başka bir kelimeyle değiştiren bir fonksiyon yazın.
- **Çöz**: Dizgi dönüşüm fonksiyonlarını (`std::stoi()`, `std::stod`, `std::to_string()`) kullanarak kullanıcıdan alınan sayı metinlerini aritmetik işlemlerde kullanan bir hesap makinesi tasarlayın.
- **Düşün**: Karakter göstericileri (`const char*`) ile `std::string` arasındaki güvenlik farkını açıklayın. Tampon taşması (buffer overflow) saldırısının neden karakter dizileriyle daha kolay gerçekleştiğini belirtin.

16. Bölüm
Akışlar

16.1 Akış Nedir?

C++'ta **akış** (stream), verilerin bir kaynaktan (klavye, dosya, ağ) bir hedefe (ekran, dosya, bellek) doğru aktığı **soyut bir "kanal" veya "boru hattı"** olarak düşünülür. Akışlar, verinin fiziksel olarak **nerede saklandığıyla ilgilenmez**; sadece **verinin sıralı bir şekilde transfer edilmesini sağlar**.

16.2 Akış Hiyerarşisi ve Temel Sınıflar

C++ dilinde akış işlemleri `<iostream>` başlığı altında toplanan bir sınıf hiyerarşisine dayanır:

- `ios_base`: Tüm akış sınıfları için taban sınıftır. Biçimlendirme bayraklarını (`hex`, `dec`, `fixed` vb.) yönetir.
- `istream`: Giriş akışları için kullanılır (Örnek olarak klavyeden veri okuyan `std::cin`).
- `ostream`: Çıkış akışları için kullanılır (Örnek olarak konsola veri yazan `std::cout` ve standart hata çıktısı yazdığımız akış olan `std::cerr`).
- `iostream`: Hem giriş hem de çıkış yapabilen akışlardır.

16.3 Standart Akış Nesneleri

Her C++ programı başladığında dört temel akış nesnesi otomatik olarak oluşturulur ve sürekli açıktırlar:

Standart Akışlar	Akış Nesnesi	Sınıf	Açıklama
Standart Giriş	`std::cin` (`std::cinistream`)	`istream`	Standart giriş (Genellikle klavye): Geleneksel klavye girişini okumak için kullanılır.
Standart Çıktı	`std::cout` (`std::coutostream`)	`ostream`	Standart çıkış (Genellikle ekran). Tamponlanmıştır (buffered). Geleneksel olarak konsola çıktıyı yazmak için kullanılır.
Standart Hata	`std::cerr` (`std::cerrostream`)	`ostream`	Standart hata çıkışı. Tamponlanmamıştır. Hatalar anında basılır. Verileri bir ara belleğe kaydetmez. Bu, hataların hemen görüntülenmesi gerekir.
Standart İz Kaydı	`std::clog` (`std::clogostream`)	`ostream`	**İz kaydı** (log) yada bir başka deyişle **günlük** çıkışı. Tamponlanmıştır (buffered). Verileri bir ara belleğe kaydeder. Çünkü iz kayıtlarına hemen bakılması gerekmez.

Tablo 16.1: Standart Akış Nesneleri

16.4 Akış İşleçleri

Akışlar iki temel işleç (operator) ile yönetilir:

1. **Ekleme İşleci** (`<<`) (insertion operator): Veriyi akışa gönderir (Konsol veya dosyadan).
2. **Ayıklama İşleci** (`>>`) (extraction operator): Veriyi akıştan çeker (Klavye veya dosyadan). Ayıklama işleci varsayılan olarak beyaz boşlukları (white space) yani boşluk, tab, yeni satır gibi karakterleri atlar. Tüm satırı okumak için `std::getline()` fonksiyonu tercih edilmelidir.

16.5 Standart olmayan Akış Nesneleri

Yazdığımız programlarda sadece girdi ve çıktı işlemleri yaparsak, program çalıştığı sürece veriler var olur, program sonlandırıldığında o verileri tekrar kullanamayız. Bunun için elektrik kesildiğinde bellekteki veriler kaybolacağından harici hafıza ortamında (external memory) verileri saklamak gerekir.

Kısaca **dosya** (file), verilerin bir araya geldiği (collection of data) ikincil saklama ortamıdır (seconder storage). Bu ikincil ortam; Bir bilgisayar belleği (memory) olabileceği gibi, verilerin elektrikler kesildiğinde kaybolmayacağı sabit disk (hard disc), ya da bir başka ortama/bilgisayara veri gönderen (send) veya alan (receive) modem ve benzeri bir cihaz (device) olabilir.

Dosya akışları (file streams) ile çalışmak için `<fstream>` başlığı kullanılır. Akış mantığı burada da aynıdır,

sadece kaynak klavye değil diskteki bir dosyadır. Dosya ile işlem yapmak için üç akış sınıfı kullanılır; `ifstream` sınıfı dosyadan `veri okumak için`, `ofstream` sınıf dosyaya **veri yazmak için** ve `fstream` sınıfı ise dosyaya **hem veri yazmak, hem de dosyadan veri okumak için** kullanılır.

Standart olmayan akışlar sürekli açık olamadıkları için ile açıp (open) işimiz bittikten sonra kapatmamız (close) gerekir. Çünkü bu kaynakları kullanmanın maliyeti yüksektir.

Bir akışı **veri okumak veya veri yazmak için** açmak istersek `open()` yöntemi kullanılır;

```
void open (const char* filename, ios_base::openmode mode);
void open (const string& filename, ios_base::openmode mode); // C++11
```

Yönteminin ilk argümanı açılacak dosyanın adını ve konumunu belirtirken, ikinci argümanı ise dosyanın hangi modda açılacağını tanımlar. Bunlar;

İşlem Modu	Açıklama
`ios::in`	Dosya okumak için yani sadece veri girdisi (input) için kullanılır.
`ios::out`	Dosya yazmak için yani dosyaya sadece veri göndermek (output) için kullanılır.
`ios::binary`	İşlemler metin yerine ikili (binary) modda gerçekleştirilir.
`ios::ate`	Dosyanın sonunda (at the end) gidilerek hem okuma hem de yazma amacıyla dosya açılır.
`ios::app`	Gönderilen bütün veri, dosyanın sonuna (append) eklenir.
`ios::trunc`	Mevcut bir dosya varsa açmadan önce içi boşaltılır (truncate). .

Tablo 16.2: Dosya Açma Modları

Dosya açma modu; `ifstream` sınıfı için `ios_base::in`, `ofstream` sınıfı için `ios_base::out` ve `fstream` sınıfı için `ios_base::in|ios_base::out` **varsayılan argümandır**. Görüldüğü üzere dosya açma modlarından ikisini veya daha fazlasını bit düzeyi VEYA (`|`) işlemiyle birleştirebiliriz!

Akış nesnesi olarak açılan dosyaya aynı `std::cout` nesnesinde olduğu gibi akışa veri **ekleme işleci** (insertion operator) (`<<`) ile veri ekleyebilir veya `std::cin` nesnesinde oluğu gibi akıştan veri **ayıklama işleci** (extraction operator) (`>>`) ile veri okuyabilirsiniz. Aşağıda programın çalıştığı klasörde `veriler.txt` adlı bir metin dosyasına metin yazılıp okuma örneği verilmiştir.

```
#include <fstream> // ofstream, ifstream, open() ve close()
#include <string>
void dosyaOrnegi() {
        // Dosyaya yazma
        std::ofstream yazici("veriler.txt"); // mode=ios_base::out
        if (yazici.is_open()) {
                yazici << "Merhaba C++ Akislari!" << std::endl;
                yazici.close();
        }
        // Dosyadan okuma
        std::ifstream okuyucu("veriler.txt"); // mode=ios_base::in
        std::string satir;
        while (std::getline(okuyucu, satir)) { // Tüm satırı okumak için getline()
                std::cout << "Dosyadan okundu: " << satir << std::endl;
        }
        okuyucu.close();
}
```

Örneğin programın çalıştığı klasörde `cikti.txt` adlı bir dosyayı her seferinde içini boşaltarak sadece yazmak için akış nesnesi aşağıdaki gibi açılabilir.

```
std::ofstream outfile;
outfile.open("cikti.txt", ios::out | ios::trunc );
if(outfile.is_open()){
        os << "Merhaba İlhan!";
}
```

Benzer şekilde, aynı dosyayı hem okuma ve hem de yazma amacıyla aşağıdaki gibi açabilirsiniz;

```
std::fstream  dosya;
dosya.open("cikti.txt", ios::out | ios::in );
```

```
if (!dosya.is_open()) {
        throw CustomException(afile, "Dosya Açılamadı!");
}
```

Çıktı dosyası akışında ekleme işleci (<<) yerine, `write()` yöntemi de kullanılabilir:

```
if(afile.is_open()){
        char metin[] = "İlhan";
        os.write(metin, 3); // Metinden 3 karakter akışa yazılıyor
}
```

Programın çalıştığı klasörde `metin.txt` dosyasında aşağıdaki verilerin olduğunu varsayalım;

```
İlhan Özkan 25 4 6 1987
Mehmet Yıldırım 15 5 24 1976
```

Bu durumda dosyadan veri okuma aşağıdaki şekilde yapılabilir;

```
std::ifstream is("metin.txt");
std::string adi, soyadi;
int yas, dogumAyi, dogumGunu, dogumYili;
while (is >> adi >> soyadi >> yas >> dogumAyi >> dogumGunu >> dogumYili)
cout << adi << " " << soyadi;
```

Bir C++ programından çıkıldığında otomatik olarak tüm akışları temizler, ayrılmış belleği serbest bırakır ve açık tüm dosyaları kapatır. **Ancak bir programcının programı sonlandırmadan önce açık olan tüm dosyaları kapatması her zaman iyi bir uygulamadır.**

Aşağıda dosya açma ve kapatmaya bir başka örnek verilmiştir;

```
#include <fstream>
#include <iostream>
#include <string>
int main () {
        std::string adiSoyadi;
        int yas;
        char cinsiyet;
        std::cout << "Adınızı ve Soyadınızı Giriniz: ";
        std::getline(std::cin, adiSoyadi);
        std::cout << "Yaşınızı Giriniz: ";
        std::cin >> yas;
        std::cout << "cinsiyetinizi Giriniz (K-E): ";
        std::cin >> cinsiyet;

        //Verileri kalıcı olarak dosyaya yazalım:
        std::ofstream outfile;  // çıkış akışı nesnesi tanımlandı.
        outfile.open("output.txt"); //dosya açıldı
        outfile << "Bu dosya Yaş ve cinsiyet Bilgilerini İçerir" << std::endl;
        outfile << "Adı:" << adiSoyadi << std::endl;
        outfile << "Yaşı:" << yas << std::endl;
        outfile << "cinsiyeti:" << cinsiyet << std::endl;
        outfile.close(); // dosya kapandı

        //Yardığımız dosyadan verileri okuyalım:
        std::string satir;
        std::ifstream infile; // giriş akışı nesnesi tanımlandı.
        infile.open("output.txt"); //dosya açıldı
        std::getline(infile, satir);
        std::cout << "Dosyadan Okunan Satır:" << std::endl << satir << std::endl;
        std::getline(infile, satir);
        std::cout << "Dosyadan Okunan Bir Sonraki Satır:" << std::endl << satir << std::endl;
        infile.close(); // dosya kapandı
}
/*Program Çıktısı:
Adınızı Giriniz: İlhan Özkan
Yaşınızı Giriniz: 50
```

```
Cinsiyetinizi Giriniz (K-E): E
Dosyadan Okunan Satır:
Bu dosya Yaş ve Cinsiyet Bilgilerini İçerir
Dosyadan Okunan Bir Sonraki Satır:
Adı:İlhan Özkan
*/
```

Program çalıştığında oluşan `output.txt` metin dosyasının içeriği aşağıda verilmiştir;

```
Bu dosya Yaş ve Cinsiyet Bilgilerini İçerir
Adı:İlhan Özkan
Yaşı:50
Cinsiyeti:E
```

> Buraya kadar işlediğimiz akışlar konsola yazdığımız veya klavyeden okuduğumuz gibi metin olarak çalışır. **Yani oluşan dosyalar gözle okunabilir metinlerdir.**

16.6 Akış Biçimlendirme

Verinin nasıl görüneceğini kontrol etmek için `<iomanip>` başlığındaki **manipülatörler** (manipulator) kullanılır. Bu biçimlendirme aslında 5.3 başlığında detaylı verilmiştir. Aşağıda çok kullanılanları verilmiştir;

- `std::setw(n)` : Verinin çıktıya gönderilirken kaç karakter aralıkla biçimlendirileceğini yani genişliğini (width) belirler.
- `std::setprecision(n)` : Ondalıklı sayıların çıktıya gönderilirken kesir kısmındaki basamak sayısını yani duyarlılığını ayarlar.
- `std::fixed`/`std::scientific` : Kayan noktalı sayı biçimini belirler.
- `std::hex`, `std::dec`, `std::oct` : Çıktıya gönderilecek tam sayının sayı tabanını değiştirir.

16.7 Metin Akışları

Bazen bir metni (string) sanki bir giriş akışıymış gibi parçalamak veya farklı türdeki verileri bir metinde birleştirmek gerekebilir. Bunun içi bellekte oluşturulan **metin akışları** (string streams) kullanılır ve `<sstream>` başlık dosyasında yer alır. Metin akışlarına bellek akışı (memory stream) da denir.

```
#include <sstream>
std::stringstream ss;
ss << "Hiz:" << 120 << " km/s";
std::string rapor = ss.str(); // "Hiz:120 km/s"
```

16.8 Akış Durum Kontrolleri

Bir akışın sağlıklı çalışıp çalışmadığını kontrol etmek için bayraklar (flags) kullanılır: Akış durumlarını (stream states) kontrol eden yöntemler aşağıda verilmiştir;

- `good()`: Her şey yolunda.
```
int sayi;
std::cout << "Bir sayi girin: ";
if (std::cin >> sayi && std::cin.good()) {
        std::cout << "Basariyla okundu: " << sayi << std::endl;
}
```
- `fail()`: Tip uyumsuzluğu gibi bir hata oluştu (Örn: int beklerken harf girilmesi).
```
int yas;
std::cout << "Yasinizi girin: ";
std::cin >> yas;
if (std::cin.fail()) {
        std::cout << "Hata: Sayi yerine gecersiz bir karakter girdiniz!" << std::endl;
        std::cin.clear(); // Hata bayragini temizler
        std::cin.ignore(1000, '\n'); // Tampondaki hatali girisi atlar
}
```
- `eof()`: Dosyanın veya akışın sonuna gelindi (End of File).
```
std::ifstream dosya("veriler.txt");
char karakter;
while (dosya.get(karakter)) {
        // Okuma yapiliyor...
```

```
}
if (dosya.eof()) {
        std::cout << "Dosyanin sonuna başarıyla ulasildi." << std::endl;
}
```

◇ `bad()`: Ciddi bir sistem hatası (Örneğin dosyaya yazarken disk doldu).

```
std::ofstream cikis("yedek.db");
cikis << "Cok buyuk veri seti...";
if (cikis.bad()) {
        std::cerr << "Kritik Hata: Yazma islemi sirasinda sistem hatasi olustu!" <<
         ↪  std::endl;
}
```

16.9 İkili Akışlar

Çalışılan dosyalar, yukarıdaki gibi her zaman metin dosyası olamayabilir. Bir nesneyi bellekte olduğu gibi yani ikili olarak aynen dosyaya saklamak isteyebiliriz. Bu durumda akışları ikili dosya (binary file) yazıp okuyacak şekilde açıp işlem yapmalıyız. Bu durumda dosya açma modu olarak `ios::binary` kullanılmalıdır. Aşağıda ikili olarak oluşturulan bir dosya örneği verilmiştir.

```
#include <iostream>
#include <fstream>
struct Kisi {
        char adiSoyadi[16];
        int yas;
        char cinsiyet;
        float kilo;
};
int main() {
        Kisi kisi1={"Ilhan OZKAN",50,'E',100.0};
        Kisi kisi2={"Yagmur OZKAN",45,'K',60.0};
        int dizi[5]={1,2,3,4,5};
        std::ofstream os("kisi.bin", std::ios::binary);
        if (!os) {
                std::cerr << "Dosya Açılamadı!" << std::endl;
                return 1;
        }
        //Veriler baytlar halinde peşpeşe akışa yazılıyor.
        os.write(reinterpret_cast<char*>(&kisi1), sizeof(Kisi));
        os.write(reinterpret_cast<char*>(&kisi2), sizeof(Kisi));
        os.write(reinterpret_cast<char*>(&dizi), sizeof(dizi));
        os.close();
}
```

Programın çalışması oluşan `kisi.bin` dosyasına ilk önce iki adet kişi yapısı kaydedilmiştir. Sonrasında 5 tam sayıdan oluşan dizi kaydedilmiştir. Bellekte saklandığı şekliyle dosyaya kaydedildiğinden içeriği gözle okunamaz. İkili dosyaları açan görüntüleyiciler ile açılıp görülebilir. Bir ikili dosyayı onaltılık rakamlara çevirip metin olarak görüntülemek için Windows ortamında aşağıdaki komut verilebilir;

```
C:\Users\ILHANOZKAN>certutil -encodehex kisi.bin kisi.txt
Input Length = 77
Output Length = 367
CertUtil: -encodehex command completed successfully.
C:\Users\ILHANOZKAN>
```

Benzer işlem Linux ortamında `hexdump -C kisi.bin | less >>kisi.txt` komutuyla yapılabilir. Artık dönüştürülen dosyayı (`kisi.txt`) istediğimiz metin editöründe açıp okuyabiliriz.

```
0000 49 6c 68 61 6e 20 4f 5a  4b 41 4e 00 00 00 00 00   Ilhan OZKAN.....
0010 32 00 00 00 45 00 00 00  00 00 c3 88 42 59 61 67   2...E.......BYag
0020 6d 75 72 20 4f 5a 4b 41  4e 00 00 00 00 2d 00 00   mur OZKAN....-..
0030 00 4b 00 00 00 00 00 70  42 01 00 00 00 02 00 00   .K.....pB.......
0040 00 03 00 00 00 04 00 00  00 05 00 00 00             .............
```

Oluşturulan aynı dosyayı ikili olarak okuyan program aşağıdaki şekilde yazılabilir. Burada yazım sırasındaki nesneleri okumak çok önemlidir. Yani dosyaya yazan kişi neyi hangi sırada yazdığından okumayı da aynı programcı yapmalıdır;

```
#include <iostream>
#include <fstream>
struct Kisi {
        char adiSoyadi[16];
        int yas;
        char cinsiyet;
        float kilo;
};
int main() {
        Kisi kisi1, kisi2;
        int dizi[5];
        std::ifstream is("kisi.bin", std::ios::binary);
        if (!is) {
                std::cerr << "Dosya Açılamadı!" << std::endl;
                return 1;
        }
        is.read(reinterpret_cast<char*>(&kisi1), sizeof(Kisi));
        std::cout << "Okunan Kisi Yapısı:" << std::endl << "Kişi Adı:" << kisi1.adiSoyadi <<
         ↪  std::endl << "Kişi Yaşı:" << kisi1.yas << std::endl << "Kişi Cinsiyeti:" <<
         ↪  kisi1.cinsiyet << std::endl << "Kişi Kilo:" << kisi1.kilo << std::endl;
        is.read(reinterpret_cast<char*>(&kisi2), sizeof(Kisi));
        std::cout << "Okunan Kisi Yapısı:" << std::endl << "Kişi Adı:" << kisi2.adiSoyadi <<
         ↪  std::endl << "Kişi Yaşı:" << kisi2.yas << std::endl << "Kişi Cinsiyeti:" <<
         ↪  kisi2.cinsiyet << std::endl << "Kişi Kilo:" << kisi2.kilo << std::endl;
        is.read(reinterpret_cast<char*>(&dizi), sizeof(dizi));
        std::cout << "Okunan Dizi:" << dizi[0] << "," << dizi[1] << "," << dizi[2] << "," <<
         ↪  dizi[3] << "," << dizi[4] << std::endl;
        is.close();
}
/* Program Çalıştığında:
Kişi Kilo:100
Okunan Kisi Yapısı:
Kişi Adı:Yagmur OZKAN
Kişi Yaşı:45
Kişi Cinsiyeti:K
Kişi Kilo:60
Okunan Dizi:1,2,3,4,5
*/
```

Eğer sadece diziyi okumak isteseydik dosya konum göstericisini okuyacağımız yere konumlandırmak gerekir. Bu durumda okuyucu program aşağıdaki gibi olurdu:

```
#include <iostream>
#include <fstream>
struct Kisi {
        char adiSoyadi[16];
        int yas;
        char cinsiyet;
        float kilo;
};
int main() {
        Kisi kisi1, kisi2;
        int dizi[5];

        std::ifstream is("kisi.bin", std::ios::binary);
        if (!is) {
                std::cerr << "Dosya Açılamadı!" << std::endl;
                return 1;
        }
        is.seekg(2*sizeof(Kisi),std::ios::beg); // iki adet Kisi yapısı kadar ilerlendi!
        // dizi iki kişi kaydı sonrasında kaydedilmişti.
        is.read(reinterpret_cast<char*>(&dizi), sizeof(dizi));
```

```
        std::cout << "Okunan Dizi:" << dizi[0] << "," << dizi[1] << "," << dizi[2] << "," <<
        ↪  dizi[3] << "," << dizi[4] << std::endl;
        is.close();
}

/* Program Çalıştığında:
Okunan Dizi:1,2,3,4,5
*/
```

Hem `istream` hem de `ostream` sınıfındaki akışlarda ikili olarak açılan dosya için dosya konum göstericisini yeniden konumlandırmak için yöntemler sağlar. Bunlar; Okuma durumunda imlecin konumunu (indisini) öğrenmek için `seekg()` kullanılır. Yazma durumunda ise yeniden konumlandırma için `seekp()` yöntemi kullanılır. Bu yöntemlerde konumlandırmanın yönü, bir akışın **başlangıcına göre** konumlandırma `ios::beg` ile yapılır (varsayılan). Konumlandırma yönü, bir akıştaki geçerli konuma göre yapılması için `ios::cur` veya bir akışın sonuna göre konumlandırma yapılması için için `ios::end` bayrakları kullanılır;

```
#include <iostream>
#include <fstream>
int main() {
        // 1. Dosyayı hem okuma hem yazma modunda açıyoruz
        std::fstream dosya("ornek.txt", std::ios::out | std::ios::in | std::ios::trunc);
        if (!dosya) {
                std::cerr << "Dosya acilamadi!" << std::endl;
                return 1;
        }
        // Dosyaya ilk veriyi yazalım
        dosya << "Merhaba Dunya";
        // 2. "Dunya" kelimesini degistirmek istiyoruz.
        // "Merhaba " (8 karakter) sonrasına gitmemiz lazım.
        // seekp(8) -> Dosyanın başından itibaren 8. bayta git demektir.
        dosya.seekp(8); // Varsayılan olarak dosyanın başından: ios::beg
        // 3. Yeni veriyi tam o noktadan itibaren yazıyoruz
        dosya << "C++";
        // Sonucu kontrol etmek için imleci başa alıp okuyalım
        dosya.seekg(0); // Varsayılan olarak dosyanın başına: ios::beg
        std::string icerik;
        std::getline(dosya, icerik);
        std::cout << "Dosyanin son hali: " << icerik << std::endl;
        // Çıktı: Merhaba C++ya (C++ 3 harf olduğu için sadece 'Dun' kısmını ezdi)
        dosya.close();
}
```

Eğer bir dosyayı hem okuma hem yazma (`ios::in|ios::out`) modunda açtıysanız, `seekg()` ve `seekp()` imleçleri çoğu sistemde ortaktır. Yani birini hareket ettirdiğinizde diğeri de hareket eder. Bu yüzden okuma işleminden yazma işlemine geçerken imleç konumunu her zaman kontrol etmelisiniz.

`seekg()` yöntemi ile bir dosyanın yerine gidiyoruz ama "Şu an dosyanın tam olarak neresindeyim?" diye sormak istersen `tellg()` ve `tellp()` fonksiyonları kullanılır. Okuma imlecinin mevcut konumunu (indisini) `tellg()` döndürür. Yazma imlecinin mevcut konumunu ise `tellp()` döndürür. `tellg()` yöntemi, okuma imlecinin yerini söyler. En yaygın kullanım alanı, imleci sona gönderip konumunu alarak dosya boyutunu bulmaktır.

```
#include <iostream>
#include <fstream>
int main() {
        std::ifstream dosya("veriler.txt", std::ios::binary);
        if (!dosya) {
                std::cerr << "Dosya acilamadi!" << std::endl;
                return 1;
        }
        // 1. İmleci dosyanın en sonuna götür
        dosya.seekg(0, std::ios::end);
        // 2. tellg() ile şu anki konumu (yani toplam bayt sayısını) al
```

```
        std::streampos boyut = dosya.tellg();
        std::cout << "Dosyanin toplam boyutu: " << boyut << " bayt." << std::endl;
        dosya.close();
}
```

`tellp()` yöntemi, yazma imlecinin o an hangi baytta olduğunu söyler. Büyük bir dosyaya veri eklerken nerede kaldığımızı kaydetmek için idealdir.

```
#include <iostream>
#include <fstream>
int main() {
        std::ofstream cikis("kayit.txt");
        cikis << "Satir 1" << std::endl; // "Satir 1\n" yazıldı
        // Şu an kaçıncı bayttayız?
        std::streampos konum1 = cikis.tellp();
        std::cout << "Ilk yazimdan sonraki konum: " << konum1 << std::endl;
        cikis << "Ikinci Uzun Satir";
        std::streampos konum2 = cikis.tellp();
        std::cout << "Ikinci yazimdan sonraki konum: " << konum2 << std::endl;
        cikis.close();
}
```

Yukarıdaki örneklerde `int` yerine `std::streampos` kullandık. Bunun sebebi:

1. **Büyük Dosyalar**: standart bir `int`, 2 GB'den büyük dosyaların adresini tutamaz. `streampos` ise çok daha büyük dosya boyutlarını destekler.
2. **Platform Bağımsızlık**: Farklı işletim sistemlerinde dosya konumu işaretleme biçimleri değişebilir; `streampos` bu karmaşıklığı C++ standartlarında çözer.

16.10 std::cerr ve std:clog Nesnelerini Dosyaya Yönlendirme

Hataları izlemek için kullanılan `std::cerr` nesnesi genellikle konsola hata çıkışı vermek için kullanılsa da bir dosyaya da yönlendirilebilir. Bu, bir programda hataları günlüğe kaydetmeniz gerektiğinde yararlı olabilir.

```
#include <iostream>
#include <fstream>
int main() {
        std::ofstream errorLog("hata.txt");
        std::cerr.rdbuf(errorLog.rdbuf());
        std::cerr << "Hata mesajları bu dosyaya yazılacak!." << std::endl;
        errorLog.close();

        std::ofstream logLog("log.txt");
        std::clog.rdbuf(logLog.rdbuf());
        std::clog << "İz Kaydı (log) mesajları bu dosyaya yazılacak!." << std::endl;
        logLog.close();
}
```

Özet olarak neden akış kullanmalıyız?

- **Tip Güvenliği**: C dilindeki `printf()` fonksiyonundaki gibi biçim belirleyici karakterler (`%d`,`%f`) gerektirmez; veri tipinin kendisi anlar.
- **Genişletilebilirlik**: Kendi yazdığınız sınıflar için ayıklama (`<<`) ve ekleme (`>>`) işleçlerini aşırı yükleyerek (overloading), sınıflarınızı standart akışlarla uyumlu hale getirebilirsiniz.
- **Soyutlama**: Verinin diskten mi, internetten mi yoksa bellekten mi geldiğiyle ilgilenmeden aynı kod yapısını kullanmanıza olanak tanır.

16.11 Düşün ve Çöz

- **Düşün**: C++'da standart akışları (`std::cin`, `std::cout`, `std::cerr`, `std::clog`) karşılaştırın. `std::cerr` ile `std::clog` arasındaki tamponlama farkı ne anlama gelir ve hata ayıklamada önemi nedir?
- **Çöz**: Bir metin dosyasına öğrenci notlarını ('isim', 'numara', 'not') yazıp okuyan bir program yazın. `std::ofstream` ve `std::ifstream` kullanımını, dosya açma modlarını (`ios::app`, `ios::trunc`) açıklayın.
- **Çöz**: `std::stringstream` sınıfını kullanarak bir CSV satırını parçalara ayıran (parse eden) bir fonksiyon

yazın. Bu sınıfın metin dönüşümlerinde nasıl kullanışlı olduğunu örnekleyin.

◇ **Çöz**: İkili (binary) modda dosya okuma/yazma ile metin modundaki farkı açıklayın. Bir `struct` nesnesini ikili dosyaya yazıp geri okuyan bir program tasarlayın.

◇ **Düşün**: Dosya akışlarında hata kontrolü nasıl yapılır? `is_open()`, `good()`, `eof()`, `fail()` ve `bad()` yöntemlerini açıklayın. Sağlam bir dosya okuma döngüsü nasıl yazılmalıdır?

17. Bölüm

Tasarım Desenleri

17.1 Desen Nedir?

Nesne yönelimli programlamanın ortaya çıkışıyla beraber, daha önceden yazılmış hazır bileşenlerin (component) yeniden kullanımı (reusing) konusunda oldukça önemli ilerlemeler sağlamıştır. Bunun paralelinde tecrübelerin (experience) yeniden kullanımı konusunda da çeşitli çalışmalar yapılmış ve **desen** (pattern) kavramı ortaya çıkmıştır. Desenlerin aksine **anti-desen** (anti-pattern) ise başarısızlık tecrübelerini anlatır.

Yazılım geliştirme öncesi, geliştirme aşması ve sonrasında daha önce yaşanmış çeşitli problemlere karşı birçok kimse tarafından çeşitli çözümler getirilmiştir. **Desenler, daha önce geliştirme yapan programcıların yanlış yaparak doğru yapmayı öğrendikleri başarılı tecrübelerin anlatılması için bir araçtır**. Gerçekleştirilen bu çözümlerin, daha sonradan yaşanacak benzer nitelikli problemlerde kullanılması amacıyla desenler kullanılır. Desen kavramının temelleri Mimar *Christopher Alexander*'ın 1970 sonlarında başlattığı çalışmalara dayanmaktadır[1].

1987 yılında uluslararası "*Nesne Yönelimli Programlama, Sistemler, Diller ve Uygulamalar*" konferansına kadar desenlerle ilgili bir çalışma ortaya çıkmamıştır[2]. Bu tarihten sonra ise başta *Grady Booch*, *Richard Helm*, *Erich Gamma* ve *Kent Beck* olmak üzere pek çok bilim adamı desenlerle ilgili makale ve sunumlar yayınlamışlardır. 1994 yılında *Erich Gamma*, *Richard Helm*, *Ralph Johnson* ve *John Vlissides* tarafından yayınlanan "*Tasarım Desenleri: Tekrar kullanılabilir Nesne Yönelimli Yazılımın Temelleri*" kitabı, tasarım desenlerin yazılımda kullanılmasında dönüm noktası olmuştur[3]. Yazılımlarda kullanılan desenler yedi ana başlıkta toplanırlar. Bunlar;

1. Mimari desenler (architectural pattern)
2. Analiz desenleri (analysis pattern)
3. Tasarım desenleri (design pattern)
4. Kodlama desenleri (implementation pattern)
5. Test desenleri (test pattern)
6. Çözüm desenleri (solution pattern)
7. Veri desenleri (data pattern)

Bu desenlerin en önemlisi ve ilk gelişenlerden birisi, **tasarım desenleridir** (design pattern). Tasarım desenleri aslında, Nesne Yönelimli Tasarım Prensiplerini, yazılımlara uygulamanın bir başka ifadesidir. Bu nedenle çok önemlidir. Tasarım desenleri ise üç ana başlıkta toplanmaktadırlar.

1. Nesne imali ile ilgili desenler (creational patterns)
2. Yapısal desenler (structural patterns)
3. Davranışlarla ilgili desenler (behavioral patterns)

Desenlerin çok sık kullanıldığı programlama yöntemine günümüzde **desen yönelimli programlama** (pattern oriented programming) adı verilmektedir[4]. İzleyen sayfalarda verilen desen UML diyagramlarının birçoğu *Erich Gamma*'nın **Design Pattern CD** yayınından alınmıştır[5]. Bölümün devamında verilecek tasarın desenlerinin UML diyagramları visual-paradigm adresinden alınmıştır[6].

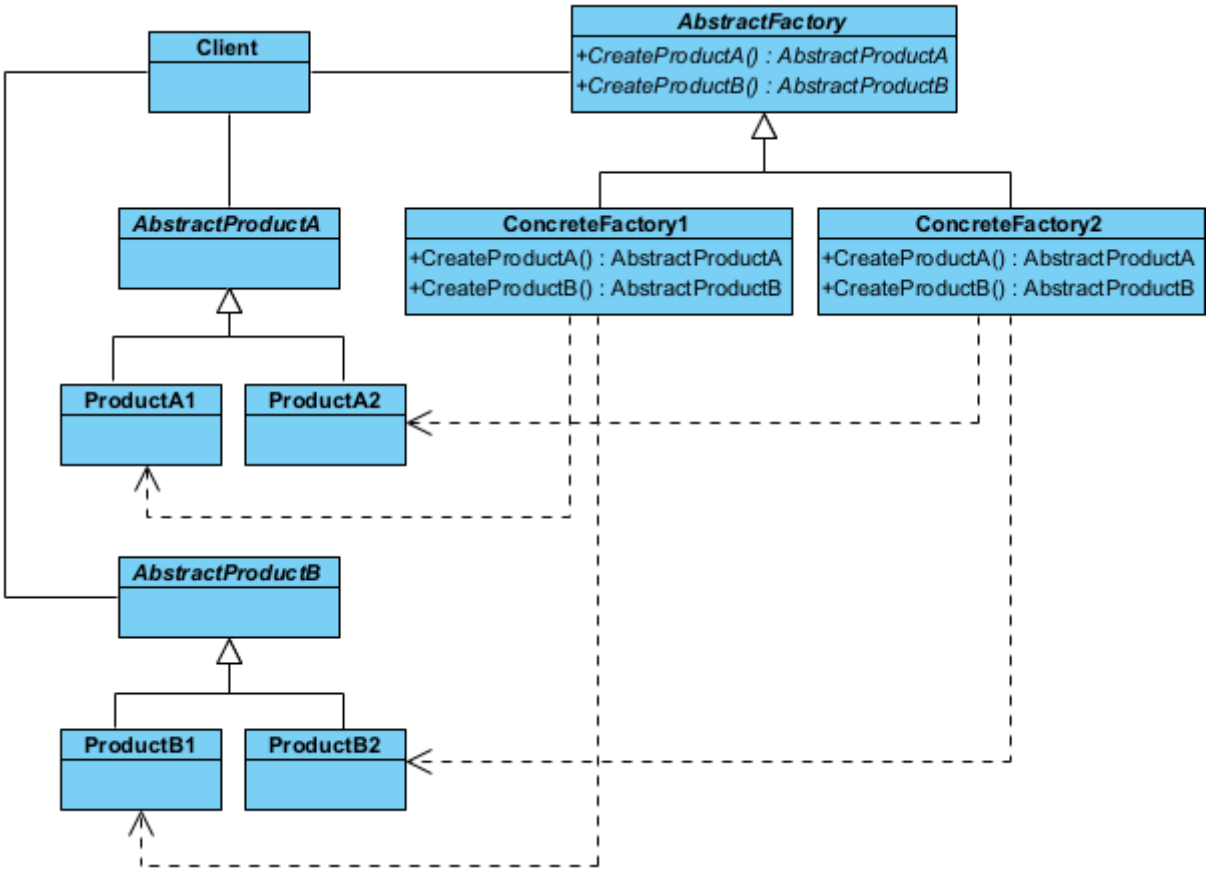
17.2 Nesne İmalatı ile İlgili Desenler

Nesne imalatı ile ilgili desenler (creational patterns), nesne (object) örnekleme ile ilgili ortaya çıkabilecek çeşitli problemlere çözüm getirmektedirler. Bu desenlerde nesneler dinamik olarak imal edilirler, çünkü değişim yönetiminde statik nesneler kullanılmazlar.

[1] http://gee.cs.oswego.edu/dl/ca/ca/ca.html
[2] OOPSLA, Object Oriented Programming, Systems, Languages, and Applications
[3] Design Patterns: Elements of Reusable Object-Oriented Software, ISBN 0-201-63361-2
[4] www.mathcs.sjsu.edu/faculty/pearce/patterns.html
[5] Design Pattern CD, Addison-Wesley,1998
[6] https://circle.visual-paradigm.com/category/uml-diagrams/gof-design/

§17.2.1 Soyut Fabrika

Soyut fabrika deseninde (abstract factory pattern) amaç, nesnelerin imal edilmesi ve kullanılması ile ilgili olan kısımlarının birbirinden ayrılmasıdır. Bu amaca, birbiriyle ilgili ya da birbirine bağımlı olabilecek nesnelerin imal edilmesinde, nesnelerin somut (concrete) sınıflarını değil soyut (abstract) sınıfları kullanılarak ulaşılır. Somut sınıflar istemciden izole edilir ve programcıya somut sınıfları daha kolay değiştirme için olanak sağlar.



Şekil 17.1: Soyut Fabrika Deseni UML Diyagramı

Bu tasarım deseninde, birbirleriyle ilişkili veya bağımlı nesne ailelerini, somut sınıflarını belirtmeden oluşturmak için kullanılan imalatla ilgili (creational) bir desendir. "Fabrikaların fabrikası" olarak da adlandırılır. Eğer sisteminizin farklı ürün aileleriyle (örneğin; Windows uyumlu buton/pencere ailesi ve Mac uyumlu buton/pencere ailesi) çalışması gerekiyorsa ve bu ailelerin birbirine karışmasını istemiyorsanız bu deseni kullanırsınız.

- Soyut Fabrika Arayüzü (`AbstractFactory`), ilgili tüm ürünleri oluşturacak yöntemlerin imzalarını (şablonunu) tanımlar. "Bir buton oluştur" ve "Bir pencere oluştur" gibi soyut görevleri belirler.
- Somut Fabrikalar (`ConcreteFactory`),belirli bir ürün ailesine ait nesneleri fiilen oluşturur. Örneğin `ModernMobilyaFabrikasi` sadece modern tarzdaki sandalyeleri ve masaları üretir.
- Soyut Ürünler (`AbstractProduct`), ürün ailesindeki her bir temel ürün türü için bir arayüz tanımlar.Tüm sandalyelerin sahip olması gereken ortak özellikleri (Örnek olarak `otur()`) belirler.
- Somut Ürünler (`ConcreteProduct`), soyut ürünlerin farklı varyasyonlarını (ailelerini) temsil eden gerçek sınıflardır. `ModernSandalye` veya `AntikaSandalye` gibi sınıflar burada yer alır.

Bu deseni kodlayalım;

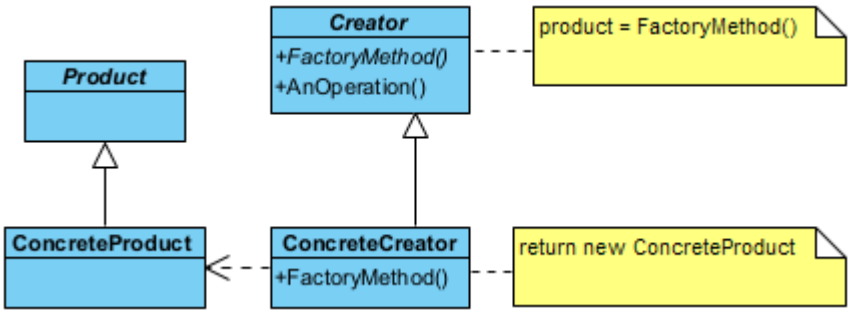
```
//C++ 14 ve üzeri ile derlenmeli
#include <iostream>
#include <string>
#include <memory> // unique_ptr için gerekli
#include <vector>

//1. Products (Ürünler)
--- Abstract Products ---
class AbstractProductA {
```

```
public:
        AbstractProductA(std::string pAdi) : productName(pAdi) {}
        virtual ~AbstractProductA() = default; // Sanal yıkıcı (Virtual Destructor) çok önemli!
         ↪ unique_ptr'nin alt sınıfları doğru silmesi için gereklidir.
        virtual std::string getUrunAdi() const { return productName; }
private:
        std::string productName;
};
class ConcreteProductA1 : public AbstractProductA {
public:
        using AbstractProductA::AbstractProductA; //Temel sınıfın TÜM yapıcılarını içeri aktarır!
};
class ConcreteProductA2 : public AbstractProductA {
public:
        using AbstractProductA::AbstractProductA; //Temel sınıfın TÜM yapıcılarını içeri aktarır!
};
// --- Abstract Product B ---
class AbstractProductB {
public:
        AbstractProductB(std::string pAdi) : productName(pAdi) {}
        virtual ~AbstractProductB() = default;
        virtual std::string getUrunAdi() const { return productName; }
        // Etkileşim için raw pointer (AbstractProductA*) kullanabiliriz. Çünkü sahiplik
         ↪ (ownership) almıyoruz, sadece gözlemliyoruz.
        void interact(const AbstractProductA* a) const {
                std::cout << this->productName << " ürünü " << a->getUrunAdi() << " ile etkileşim
                 ↪ halinde..." << std::endl;
        }
private:
        std::string productName;
};
class ConcreteProductB1 : public AbstractProductB {
public:
        using AbstractProductB::AbstractProductB; //Temel sınıfın TÜM yapıcılarını içeri aktarır!
};

class ConcreteProductB2 : public AbstractProductB {
public:
        using AbstractProductB::AbstractProductB; //Temel sınıfın TÜM yapıcılarını içeri aktarır!
};

//2. Factory (Fabrika)
--- Abstract Factory ---
class AbstractFactory {
public:
        virtual ~AbstractFactory() = default;
        // unique_ptr döndürerek nesnenin sahipliğini çağırana devrediyoruz
        virtual std::unique_ptr<AbstractProductA> createProductA() = 0;
        virtual std::unique_ptr<AbstractProductB> createProductB() = 0;
};
class ConcreteFactory1 : public AbstractFactory {
public:
        std::unique_ptr<AbstractProductA> createProductA() override {
                return std::make_unique<ConcreteProductA1>("A1 ürünü");
        }
        std::unique_ptr<AbstractProductB> createProductB() override {
                return std::make_unique<ConcreteProductB1>("B1 Ürünü");
        }
};

class ConcreteFactory2 : public AbstractFactory {
public:
```

```
        std::unique_ptr<AbstractProductA> createProductA() override {
                return std::make_unique<ConcreteProductA2>("A2 ürünü");
        }
        std::unique_ptr<AbstractProductB> createProductB() override {
                return std::make_unique<ConcreteProductB2>("B2 Ürünü");
        }
};

// 3. Client (İstemci)
int main() {
        // Fabrikaları unique_ptr ile oluşturuyoruz
        auto factory1 = std::make_unique<ConcreteFactory1>();
        auto factory2 = std::make_unique<ConcreteFactory2>();
        // Ürünleri oluşturuyoruz - Bellek yönetimi artık otomatik!
        auto productA = factory1->createProductA();
        std::cout << "Fabrika 1 ürünü: " << productA->getUrunAdi() << std::endl;
        auto productB = factory2->createProductB();
        std::cout << "Fabrika 2 ürünü: " << productB->getUrunAdi() << std::endl;
        // .get() ile akıllı işaretçinin içindeki ham adresi geçici olarak kullanabiliriz
        productB->interact(productA.get());
        // delete yazmamıza gerek yok; main biterken her şey otomatik temizlenir.
}
/*Program Çıktısı:
Fabrika 1 ürünü: A1 ürünü
Fabrika 2 ürünü: B2 Ürünü
B2 Ürünü ürünü A1 ürünü ile etkileşim halinde...
*/
```

§17.2.2 Fabrika Yöntemi

Fabrika yöntemi deseninde (factory method pattern) amaç, imal edilecek nesnenin sınıfını kesin olarak belirtmeden nesne yaratma işleminin gerçekleştirilmesidir. Bu desenin görevi, nesne üretim işini alt sınıflara



Şekil 17.2: Fabrika Yöntemi Deseni UML Diyagramı

bırakır. Hangi nesnenin üretileceğine çalışma zamanında karar verilir.

- Üretici (`Creator`), fabrika yöntemini tanımlayan soyut sınıftır.
- Ürün (`Product`), üretilecek nesnelerin ortak arayüzüdür.
- Somut Üretici (`ConcreteCreator`), ihtiyaca göre "A Ürünü" veya "B Ürünü" üreten gerçek fabrikadır.

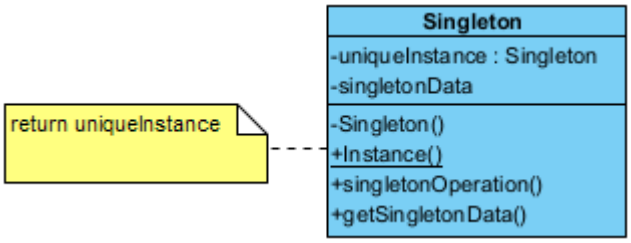
Bu deseni kodlayalım;

```
//C++ 14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>
#include <memory> // unique_ptr için şart

//1. Products (Ürünler)
// --- Ürün Sınıfları ---
class Product {
public:
        Product(string pAdi) : productName(pAdi) {}
        virtual ~Product() = default; // Alt sınıfların güvenli silinmesi için sanal yıkıcı
        virtual string getUrunAdi() const {
```

```
                return productName;
        }
        private:
        string productName;
};
class ConcreteProduct : public Product {
public:
        using Product::Product; // Üst sınıfın yapıcısını miras al
};

//2. Creators (İmalatçılar)
// --- Creator (İmalatçı) Sınıfları ---
class Creator {
public:
        virtual ~Creator() = default;
        virtual unique_ptr<Product> factoryMethod() = 0; // Sahipliği devreden Fabrika Yöntemi
        virtual void anOperation() = 0;
};
class ConcreteCreator : public Creator {
public:
        unique_ptr<Product> factoryMethod() override {
                // C++14 kullanıyorsan make_unique, C++11 ise unique_ptr<T>(new T())
                return make_unique<ConcreteProduct>("Fabrika Yöntemi Ürünü");
        }
        void anOperation() override {
                cout << "Fabrika Yöntemini Kullanan Sınıf Başka İşlemler de yapabilir..." << endl;
        }
};

//3. Client (İstemci)
int main() {
        // 1. Creator'ı akıllı gösterici ile oluştur
        unique_ptr<Creator> imalatci = make_unique<ConcreteCreator>();
        // 2. Ürünü oluştur (Sahiplik imalatçıdan 'urun' değişkenine geçer)
        unique_ptr<Product> urun = imalatci->factoryMethod();
        cout << "İmalatçı tarafından imal edilen ürün: " << urun->getUrunAdi() << endl;
        imalatci->anOperation();
        // Manuel 'delete' yapmaya GEREK YOK. main fonksiyonu bittiğinde urun ve imalatci
        ↪ otomatik olarak silinir.
}
/*Program Çıktısı:
İmalatçı tarafından imal edilen ürün: Fabrika Yöntemi Ürünü
Fabrika Yöntemini Kullanan Sınıf Başka İşlemler de yapabilir...
*/
```

§17.2.3 Tekil Nesne

Tekil Nesne deseninde (singleton pattern) amaç, bir sınıftan yalnızca bir örnek (instance) imal etmektir. Bir sınıfın yalnızca bir nesnesinin olmasına izin verilir, birden fazla olmasına izin verilmez.



Şekil 17.3: Tekil Deseni UML Diyagramı

Modern C++'ta (C++11 ve sonrası) bu desen, "Meyers' Singleton" olarak bilinen çok daha zarif, güvenli ve performanslı bir yapıya dönüşmüştür. Şimdi Singleton sınıfını kodlayalım;

```
1 //C++11 ve üzeri ile derlenmelidir
```

```
2  #include <iostream>
3
4  //1. Singletone (Tekil Nesne) Sınıfı
5  class Singleton {
6  private:
7          // Yapıcı yöntem gizli
8          Singleton() : singletonData(10) {
9                  std::cout << "Nesne ilk kez olusturuldu.\n";
10         }
11         int singletonData;
12 public:
13         // Kopyalama ve atama engellenir
14         Singleton(const Singleton&) = delete;
15         Singleton& operator=(const Singleton&) = delete;
16         // En modern erişim noktası
17         static Singleton& Instance() {
18                 // Statik yerel değişken: Sadece ilk çağrıda oluşturulur,
19                 // program sonlanırken otomatik silinir. Thread-safe'dir.
20                 static Singleton instance;
21                 return instance;
22         }
23         void anOperation() const {
24                 std::cout << "Modern tekil nesne gorev basinda...\n";
25         }
26         int getSingletonData() const { return singletonData; }
27 };
28
29 //2. Client (İstemci)
30 int main() {
31         // Nesneye referans üzerinden erişmek daha güvenlidir
32         Singleton& s1 = Singleton::Instance();
33         Singleton& s2 = Singleton::Instance();
34
35         if (&s1 == &s2) {
36                 std::cout << "s1 ve s2 AYNI nesnedir.\n";
37         }
38         s1.anOperation();
39 }
40 /*Program Çıktısı:
41 Nesne ilk kez olusturuldu.
42 s1 ve s2 AYNI nesnedir.
43 Modern tekil nesne gorev basinda...
44 */
```

Yukarıda kodlaması yapılan bu desenin amacı, bir sınıftan (`Singleton`) uygulama ömrü boyunca sadece bir adet nesne üretilmesini garanti etmektir.

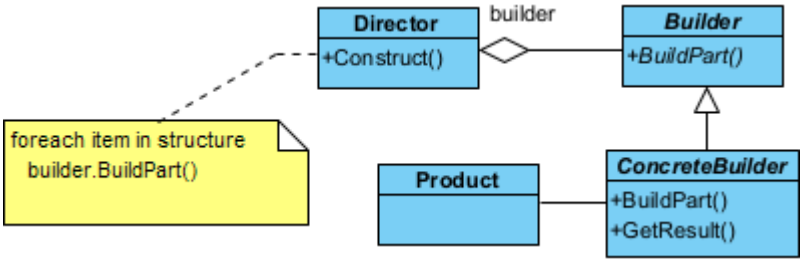
- Mahrem Yapıcı (private constructor), dışarıdan rastgele nesne üretilmesini engeller. 7. Satırda tanımlanmıştır.
- Statik Tekil Örnek (static unique instance), oluşturulan tek nesneyi kendi içinde saklar. 19. Satırda tanımlanmıştır.
- Statik Erişim Yöntemi (static access method), nesneye her yerden erişim sağlayan kapıdır. 16. Satırda tanımlanmıştır.

§17.2.4 Kurucu

Kurucu deseninde (builder pattern) amaç, çok karmaşık bir nesnenin imal edilmesiyle ilgili işlemleri bir başka sınıfa bırakmaktır. Böylece nesnenin gösterimi ve kullanılması ilişkin kodlar ile imalatına ilişkin kodlar birbirinden ayrılmış olur.

Bu deseni bir fast-food restoranındaki "Menü Hazırlama" sürecine benzetebilirsin. Hamburgerin içine turşu konulup konulmayacağı, içeceğin boyutu veya yan ürünün ne olacağı gibi adımlar tek tek belirlenir, ancak en sonunda ortaya tek bir "Menü" nesnesi çıkar.

- Ürün (`Product`), inşa edilmeye çalışılan karmaşık nesnedir. Genellikle çok sayıda parçadan veya kon-



Şekil 17.4: Kurucu Deseni UML Diyagramı

figürasyondan oluşur.

- Soyut Kurucu (`Builder`), ürünün parçalarını oluşturmak için gerekli olan tüm adımları (yöntemleri) arayüz olarak tanımlar. "Ekmek koy", "Köfte ekle", "Sos dök" gibi soyut inşa adımlarını belirler.
- Somut Kurucu (`ConcreteBuilder`), soyut kurucudaki adımları fiilen gerçekleştirir ve ürünün bir örneğini saklar. "Vejetaryen Burger" veya "Double Burger" gibi farklı ürün varyasyonlarını nasıl inşa edeceğini bilir.
- Yönetici (`Director`), inşa adımlarının hangi sırayla çağrılacağını kontrol eder. İstemciden (`Client`) aldığı kurucu nesnesini kullanarak, doğru sırayla (Örn: Önce temel, sonra yan ürünler) inşa sürecini yürütür.

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <vector>
#include <string>
#include <memory>   // std::unique_ptr için
#include <utility>  // std::move için

//1. Ürün Sınıfı (Product)
class Product {
public:
        explicit Product(std::string name) : productName(std::move(name)) {}
        void addPart(const std::string& part) {
                parts.push_back(part);
        }
        void showParts() const {
                std::cout << productName << " Urununun parcalari:\n";
                // Modern C++: Range-based for loop ve const kullanımı
                for (const auto& part : parts) {
                        std::cout << "- " << part << "\n";
                }
        }
private:
        std::string productName;
        std::vector<std::string> parts;
};

//2. Kurucu (Builder) Sınıfları
//Kurucu Arayüzü (Abstract Builder)
class Builder {
public:
        virtual ~Builder() = default; // Sanal yıkıcı şarttır
        virtual void buildPartKisim1() = 0;
        virtual void buildPartKisim2() = 0;
        virtual void buildPartKisim3() = 0;
        // Ürünün mülkiyetini (ownership) dışarıya devreden yöntem
        virtual std::unique_ptr<Product> getResult() = 0;
};
//Somut Kurucu (Concrete Builder)
class ConcreteBuilder : public Builder {
public:
        ConcreteBuilder() {
                // make_unique ile güvenli oluşturma
```

```
                product = std::make_unique<Product>("Modern C++ Kitabi");
        }
        void buildPartKisim1() override { product->addPart("Bolum 1: Akilli Gösterişciler"); }
        void buildPartKisim2() override { product->addPart("Bolum 2: Lamda Ifadeleri"); }
        void buildPartKisim3() override { product->addPart("Bolum 3: Tasarim Desenleri"); }
        std::unique_ptr<Product> getResult() override {
                // Sahipliği dışarıya (Client'a) taşıyoruz. Builder artık buna sahip değil.
                return std::move(product);
        }
private:
        std::unique_ptr<Product> product;
};

//3. Yönetici (Director)
class Director {
public:
        // Builder'ı referans olarak almak sahiplik karmaşasını önler
        void construct(Builder& builder) {
                builder.buildPartKisim1();
                builder.buildPartKisim2();
                builder.buildPartKisim3();
        }
};

//4. Client (İstemci)
int main() {
        // Nesneleri stack üzerinde veya unique_ptr ile oluşturuyoruz
        Director director;
        auto builder = std::make_unique<ConcreteBuilder>();
        // İnşa süreci
        director.construct(*builder);
        // Ürünü teslim al (Sahiplik builder'dan main'e geçti)
        std::unique_ptr<Product> product = builder->getResult();
        if (product) {
                product->showParts();
        }
        // Manuel 'delete' yok! Kapsam bitince her şey otomatik temizlenir.
}
/*Program Çıktısı:
Modern C++ Kitabi Urununun parcalari:
- Bolum 1: Akilli Isaretciler
- Bolum 2: Lambda Ifadeleri
- Bolum 3: Tasarim Desenleri
*/
```

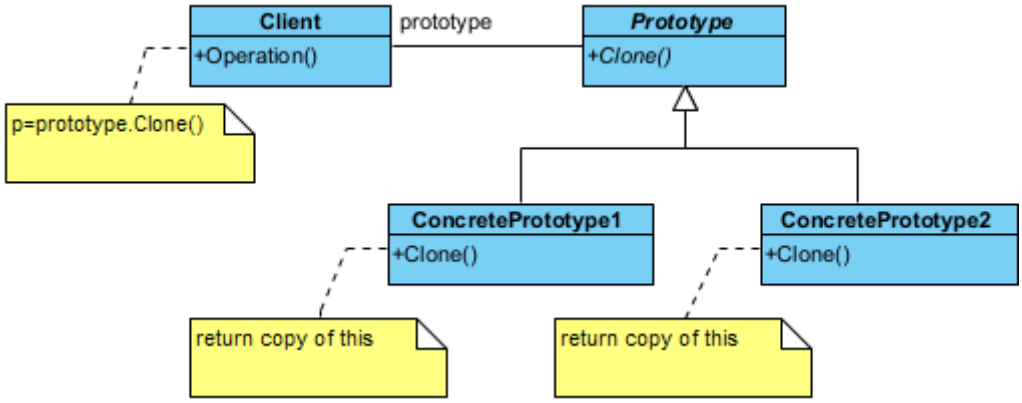
§17.2.5 Prototip

Prototip deseni (prototype pattern), mevcut bir nesnenin kopyasını (klonunu) oluşturmak için kullanılan imalatla ilgili bir desendir. Bu desenin temel amacı, bir nesneyi oluşturmanın maliyeti (zaman, bellek veya işlem gücü açısından) çok yüksek olduğunda, sıfırdan `new` işleci ile üretmek yerine mevcut bir örneği klonlayarak yeni nesneler elde etmektir.

- Soyut Prototip (`Prototype`), nesnenin kendisini kopyalayabilmesi için gerekli olan arayüzü tanımlar. Genellikle sadece bir `clone()` yönteminin imzasını barındırır.
- Somut Prototip (`ConcretePrototype`), soyut prototip arayüzünü uygular ve asıl kopyalama işlemini gerçekleştirir. Kendi verilerini yeni bir nesneye aktarır. C++ dünyasında bu işlem genellikle Kopya yapıcı (copy constructor) kullanılarak yapılır.
- İstemci (`Client`), yeni bir nesneye ihtiyaç duyduğunda, sınıfa gidip "yeni bir tane üret" demek yerine, elindeki prototip nesnesine "kendini klonla" der.

Şimdi bu deseni kodlayalım;

```
#include <iostream>
#include <memory>
#include <string>
```



Şekil 17.5: Prototip Deseni UML Diyagramı

```
// 1. Prototip (Prototype) Sınıfları
//--- Abstract Prototype ---
class Prototype {
protected: // Miras alan sınıfların erişebilmesi için protected
        std::string data_;
public:
        virtual ~Prototype() = default;
        // Nesneyi kopyalayıp akıllı işaretçi olarak döndürür
        virtual std::unique_ptr<Prototype> clone() const = 0;
        void showData() const {
                std::cout << "Nesne Verisi: " << data_ << std::endl;
        }
};
// --- Concrete Prototype ---
class ConcretePrototype : public Prototype {
        public:
        // Yapıcı fonksiyon (Constructor)
        ConcretePrototype(std::string data) {
                data_ = std::move(data);
        }
        // Kopyalama yapıcı (Copy Constructor) clone() için gerekli
        std::unique_ptr<Prototype> clone() const override {
                // Mevcut nesnenin (*this) kopyasını oluşturur
                return std::make_unique<ConcretePrototype>(*this);
        }
};

// 2. Client (İstemci)
void operation() {
        // 1. Orijinal nesneleri unique_ptr ile oluşturalım (Güvenli yol)
        std::unique_ptr<Prototype> object1 = std::make_unique<ConcretePrototype>("40.30");
        object1->showData();
        // 2. Klonlama işlemi
        // clone() bir unique_ptr döndürür, bu yüzden alıcı da unique_ptr olmalı
        std::unique_ptr<Prototype> clone1 = object1->clone();
        clone1->showData();
        std::unique_ptr<Prototype> object2 = std::make_unique<ConcretePrototype>("Mehmet");
        object2->showData();
        std::unique_ptr<Prototype> clone2 = object2->clone();
        clone2->showData();
        // Akıllı göstericiler (unique_ptr) sayesinde manuel 'delete' yapmaya gerek kalmaz!
}
int main() {
        operation();
}
```

```
/*Program Çıktısı:
Nesne Verisi: 40.30
Nesne Verisi: 40.30
Nesne Verisi: Mehmet
Nesne Verisi: Mehmet
*/
```

Nesneleri kopyalama söz konusu olduğunda, genel olarak üç tür kavramdan bahsedilir;

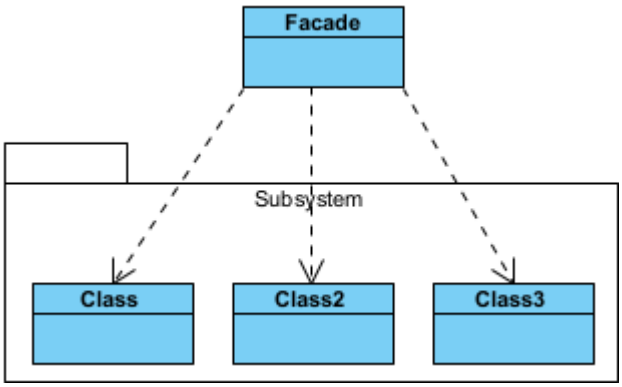
1. **Üstünkörü kopyalamada** (shallow copy) yeni nesne eskisini referans gösterir. Bu durumda birinci nesnenin bir alanı (field) değiştiğinde ikincisinin ki de değişmiş olur. Yani birinde olan değişiklik diğerini etkiler. Bu kopyalamanın diğer adı da bit düzeyi kopyalamadır (bitwise copy).
2. **Derinlemesine kopyalamada** (deep copy) ise birinci nesnenin aynısı bir başka bellek bölgesinde oluşturulur ve ikinci nesne yeni bellek bölgesini referans gösterir. Bu kopyalamanın diğer adı da üye düzeyi kopyalamadır (memberwise copy). Kopyalama sonrasında bir nesne üzerinde olan değişiklik yeni nesneyi etkilemez.
3. Üçüncü tip bir kopyalama ise **tembel kopyalamadır** (lazy copy). Yukarıda bahsi geçen iki kopyalamanın iç içe girmiş şeklidir

17.3 Yapısal Desenler

Yapısal desenler (structural pattern), sınıflar ve nesnelerin çok olduğu daha karmaşık programlarda getirilen yapısal çözüm yöntemleridir.

§17.3.1 Önyüz

Önyüz deseninde (facade pattern) karmaşık bir alt sistemdeki (subsystem) çok sayıda sınıfı ve arayüzü, kullanımı kolay tek bir üst düzey arayüz arkasında gizlemek için kullanılan yapısal bir desendir. Bu deseni bir restoranın "Garsonu" gibi düşünebilirsin. Sen sadece sipariş verirsin; garson ise mutfağın, bulaşıkhanenin ve kasanın karmaşıklığıyla senin adına ilgilenir.



Şekil 17.6: Önyüz Deseni UML Diyagramı

- Önyüz (`Facade`), karmaşık alt sistemleri bilen ve onları yöneten "ana giriş kapısı"dır. İstemciden (`Client`) gelen basit isteği, alt sistemdeki doğru nesnelere yönlendirir ve onları uygun sırayla çalıştırır.
- Alt Sistem Sınıfları (`SubSystem`), işin asıl detaylarını yapan çok sayıda sınıftır (Örn: SesSistemi, IsikDenetleyici, Projeksiyon). `Facade` nesnesinden tamamen habersizdirler. Kendi işlerini yaparlar. `Facade` sadece bunları koordine eder.
- İstemci (`Client`), sistemi kullanan son kullanıcı veya başka bir kod parçasıdır. Alt sistemin karmaşık detaylarıyla (hangi sınıf hangi sırayla çağrılacak gibi) uğraşmaz; sadece `Facade`'in sunduğu basit yöntemleri çağırır.

Şimdi desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir.
#include <iostream>
#include <memory> // std::unique_ptr için

//1. Alt Sistemler (Subsystems)
// Bu sınıflar karmaşık iş mantığını barındırır.
class SubSystem1 {
public:
        void operation11() { std::cout << "Subsystem1: Yöntem 1 calistirildi." << std::endl; }
```

```
};
class SubSystem2 {
public:
        void operation22() { std::cout << "Subsystem2: Yöntem 2 calistirildi." << std::endl; }
};
class SubSystem3 {
public:
        void operation31() { std::cout << "Subsystem3: Yöntem 1 calistirildi." << std::endl; }
        void operation32() { std::cout << "Subsystem3: Yöntem 2 calistirildi." << std::endl; }
        void operation33() { std::cout << "Subsystem3: Yöntem 3 calistirildi." << std::endl; }
};

//2. Facade (Önyüz) Sınıfı
//İstemciye (Client) basit bir arayüz sunar, karmaşayı yönetir.
class Facade {
private:
        // Akıllı göstericiler ile otomatik bellek yönetimi
        std::unique_ptr<SubSystem1> subsystem1;
        std::unique_ptr<SubSystem2> subsystem2;
        std::unique_ptr<SubSystem3> subsystem3;
public:
        Facade() : subsystem1(std::make_unique<SubSystem1>()),
         ↪  subsystem2(std::make_unique<SubSystem2>()),
         ↪  subsystem3(std::make_unique<SubSystem3>()) {
                // Yapıcıda üye ilklendirme listesi (member initializer list) kullanımı daha
                 ↪  performanslıdır.
        }

        // Yıkıcı (Destructor) yazmaya gerek yok, unique_ptr otomatik temizler.
        void executeComplexOperation() {
                std::cout << "--- Facade Karma sik Islemi Baslatiyor ---" << std::endl;
                subsystem1->operation11();
                subsystem2->operation22();
                subsystem3->operation31();
                subsystem3->operation33();
                subsystem3->operation32();
                std::cout << "--- Islem Tamamlandi ---" << std::endl;
        }
};

//3. Client (İstemci)
int main() {
        // Client tarafında Facade nesnesini doğrudan Stack üzerinde oluşturuyoruz.
        Facade facade;
        facade.executeComplexOperation();

}
/*Program Çıktısı:
--- Facade Karma sik Islemi Baslatiyor ---
Subsystem1: Yöntem 1 calistirildi.
Subsystem2: Yöntem 2 calistirildi.
Subsystem3: Yöntem 1 calistirildi.
Subsystem3: Yöntem 3 calistirildi.
Subsystem3: Yöntem 2 calistirildi.
--- Islem Tamamlandi ---
*/
```

Aşağıda bu desene gerçek dünya örneği olarak alt sistemleri olan bir bilgisayarın başlatılması modellenmiştir;

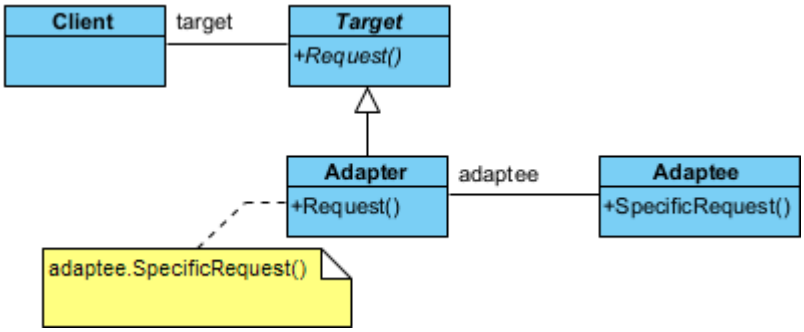
```
//C++17 ile derlenmelidir.
#include <iostream>
#include <string>
```

```
#include <string_view> // C++17: Bellek kopyalamayı azaltan hafif metin görünümü
#include <iomanip>     // Çıktı biçimlendirme (hex) için

// --- Sabit Tanımlamalar ---
namespace BootConstants {
        constexpr long ADDRESS = 0x7C00;
        constexpr int SECTOR = 0;
        constexpr int SECTOR_SIZE = 512;
}

//1. --- Alt Sistemler (Subsystems) ---
class CPU {
        public:
        void freeze() const {
                std::cout << "CPU: Durduruldu (Freeze)." << std::endl;
        }
        void jump(long position) const {
                std::cout << "CPU: Adrese atlandi: 0x" << std::hex << std::uppercase << position
                ↪  << std::endl;
        }
        void execute() const {
                std::cout << "CPU: Komutlar calistiriliyor..." << std::dec << std::endl;
        }
};
class Memory {
        public:
        // const char* yerine string_view kullanarak daha güvenli veri aktarımı sağlıyoruz
        void load(long position, std::string_view data) const {
                std::cout << "Memory: 0x" << std::hex << position
                << " adresine veri yuklendi: [" << data << "]" << std::endl;
        }
};
class HardDrive {
        public:
        std::string_view read(long lba, int size) const {
                std::cout << "HardDrive: Sector " << std::dec << lba << " uzerinden "
                << size << " byte veri okundu." << std::endl;
                return "OS_KERNEL_DATA";
        }
};

//2. --- Facade (Görünüş / Önyüz) ---
class ComputerFacade {
        private:
        // Alt sistemleri üyeler olarak tutuyoruz.
        // Eğer bu alt sistemler çok ağırsa unique_ptr kullanılabilir.
        CPU cpu_;
        Memory memory_;
        HardDrive hardDrive_;
        public:
        void start() {
                std::cout << "Bilgisayar baslatiliyor...\n" << std::string(35, '-') << std::endl;
                // Facade, karmaşık adımları bir orkestra şefi gibi yönetir:
                cpu_.freeze();
                // Diski oku ve belleğe aktar
                auto data = hardDrive_.read(BootConstants::SECTOR, BootConstants::SECTOR_SIZE);
                memory_.load(BootConstants::ADDRESS, data);
                // İşlemciyi yeni adrese yönlendir ve çalıştır
                cpu_.jump(BootConstants::ADDRESS);
                cpu_.execute();
                std::cout << std::string(35, '-') << "\nSistem hazir!" << std::endl;
        }
```

```
};

//3. --- Client (İstemci) ---
int main() {
        // İstemci (Kullanıcı) sadece "başlat" komutunu bilir.
        ComputerFacade computer;
        computer.start();
}
/*Program Çıktısı:
Bilgisayar baslatiliyor...
-----------------------------------
CPU: Durduruldu (Freeze).
HardDrive: Sector 0 uzerinden 512 byte veri okundu.
Memory: 0x7c00 adresine veri yuklendi: [OS_KERNEL_DATA]
CPU: Adrese atlandi: 0x7C00
CPU: Komutlar calistiriliyor...
-----------------------------------
Sistem hazir!
*/
```

§17.3.2 Adaptör

Adaptör deseni (adapter pattern), yabancı bir ara yüzü istemcinin anlayacağı bir ara yüze dönüştürür. Yani farklı ara yüzlere sahip sınıfların, iletişim ve etkileşim kurabilecekleri ortak bir nesne oluşturarak birlikte çalışmalarına izin verir. Bu desenin amacı, uyumsuz iki arayüzün (interface) birlikte çalışmasını sağlar. Burada



Şekil 17.7: Adaptör Deseni UML Diyagramı

yabancı ara yüze yaptırılacak işin bir şekilde yapıldığı ve zaten yapılan bu işlemin istemcinin (`Client`) isteğine uygun hale getirildiği düşünülmelidir. Bir nevi elektrik prizi dönüştürücüsü görevi görür.

- İstemci (`Client`), mevcut sistemle çalışan ana kod.
- Uyumsuz Nesne (`Adaptee`), sisteme dahil edilmek istenen ama arayüzü farklı olan nesne.
- Hedef (`Target`), istemcinin (`Client`) beklediği arayüz.
- Uyarlayıcı (`Adapter`), uyumsuz nesneyi sarmalayarak, istemcinin beklediği biçime çeviren aracı sınıf.

Şimdi de desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <memory> // std::unique_ptr için

// 1. Adaptee (Uyarlanan Uyumsuz Arayüz): Mevcut olan ama arayüzü uyumsuz olan sınıf.
class Adaptee {
public:
        void specificRequest() const {
                std::cout << "Adaptee: Ozel istek (specificRequest) yontemi calistirildi." <<
                ↪  std::endl;
        }
};

// 2. Target (Hedef): İstemcinin (Client) beklediği arayüz.
class Target {
public:
```

```
        // ÖNEMLİ: Soyut sınıflarda sanal yıkıcı (virtual destructor) şarttır!
        virtual ~Target() = default;
        virtual void request() = 0;
};

// 3. Adapter (Uyarlayıcı): Target arayüzünü Adaptee'ye bağlar.
class Adapter : public Target {
private:
        // Akıllı işaretçi ile otomatik ömür yönetimi (RAII)
        std::unique_ptr<Adaptee> adaptee;
public:
        // make_unique ile güvenli oluşturma
        Adapter() : adaptee(std::make_unique<Adaptee>()) {}
        void request() override {
                std::cout << "Adapter: Standart istek alindi, Adaptee'ye yonlendiriliyor..." <<
                ↪  std::endl;
                adaptee->specificRequest();
        }
};

// 4. Client (İstemci)
int main() {
        // Target nesnesini akıllı işaretçi ile yönetiyoruz.
        // Nesne kapsam dışına çıktığında delete işlemi otomatik yapılır.
        std::unique_ptr<Target> target = std::make_unique<Adapter>();
        target->request();
}
/*Program Çıktısı:
Adapter: Standart istek alindi, Adaptee'ye yonlendiriliyor...
Adaptee: Ozel istek (specificRequest) yontemi calistirildi.
*/
```

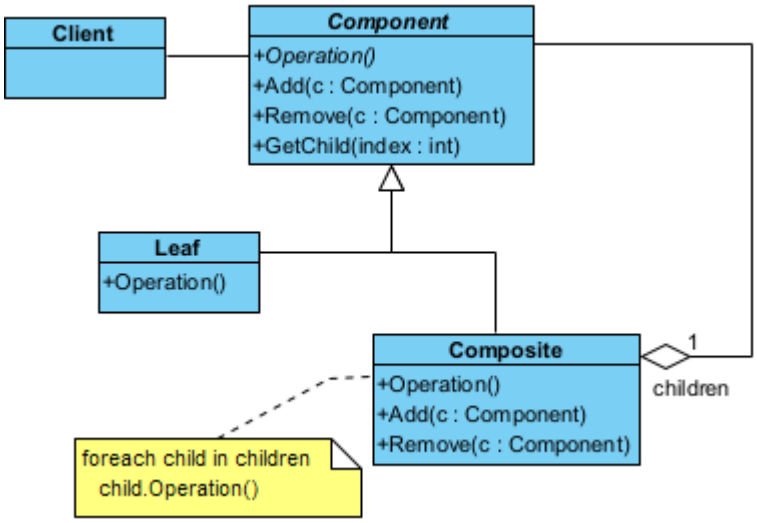
Bu desen gerçek dünya örneği olarak aşağıdaki iz kaydı (log) örneği verilebilir;

```
//C++ 14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>
#include <memory>

//1. --- Adaptee (Uyarlanan Sınıf) ---
// Değiştiremediğimiz veya eski (Legacy) olan sınıf
class OldLogger {
        public:
        void logMessage(const char* msg) {
                std::cout << "[Eski Kayit Sistemi]: " << msg << std::endl;
        }
};

//2. --- Target Interface (Hedef Arayüz) ---
// Yeni sistemin beklediği standart arayüz
class ILogger {
        public:
        virtual ~ILogger() = default;
        virtual void log(const std::string& msg) = 0;
};

//3. --- Adapter (Adaptör Sınıfı) ---
// Eski sınıfı yeni arayüze sarmalayan dönüştürücü
class LoggerAdapter : public ILogger {
        private:
        std::unique_ptr<OldLogger> oldLogger_; // Nesne ömrünü yönetmek için akıllı işaretçi
        public:
        // Adaptör oluşturulurken eski sınıfın bir örneğini alır
```

```
        LoggerAdapter(std::unique_ptr<OldLogger> logger) : oldLogger_(std::move(logger)) {}
        // Yeni arayüz metodu çağrıldığında, arka planda eski metodu uygun parametreyle çalıştırır
        void log(const std::string& msg) override {
                // std::string'i eski sistemin beklediği const char*'a çeviriyoruz
                oldLogger_->logMessage(msg.c_str());
        }
};

//4. --- Client (İstemci) ---
// Sadece ILogger arayüzünü tanır
void appLog(ILogger& logger) {
        logger.log("Sistem basariyla calisti.");
}
int main() {
        // 1. Eski sistemden bir logger nesnesi oluşturalım
        auto oldSys = std::make_unique<OldLogger>();
        // 2. Bu nesneyi yeni sistemde kullanamayız çünkü metod isimleri ve tipleri farklı.
        // Bu yüzden Adaptör devreye giriyor:
        std::unique_ptr<ILogger> modernLogger =
        std::make_unique<LoggerAdapter>(std::move(oldSys));
        // 3. İstemci (appLog), arka planda eski bir sistemin çalıştığını bilmeden
        // standart log() metodunu çağırır.
        std::cout << "Modern uygulama log gönderiyor..." << std::endl;
        appLog(*modernLogger);
}
/* Programın Çıktısı:
Modern uygulama log gönderiyor...
[Eski Kayit Sistemi]: Sistem basariyla calisti.
*/
```

§17.3.3 Bileşik

Bileşik deseni (composite pattern), özyinelemeli (recursive) ya da hiyerarşik yapıya sahip nesne oluşturmak gerektiğinde yapısal olarak nasıl bir kodlama yöntemi izleneceğini söyler. Bu desen, her nesnenin bağımsız olarak veya aynı ara yüz üzerinden iç içe geçmiş nesneler kümesi olarak ele alınabileceği nesne hiyerarşilerinin oluşturulmasını kolaylaştırır.



Şekil 17.8: Bileşik Deseni UML Diyagramı

Bu desenin görevi, nesneleri "parça-bütün" hiyerarşisinde (ağaç yapısı) düzenler. Tekil nesne ile nesne gruplarına aynı şekilde davranılmasını sağlar. Bir klasör örneği üzerinden ilerlersek;

- Bileşen (`Component`), hem dosyaların hem klasörlerin ortak özelliklerini tanımlayan arayüz.
- Yaprak (`Leaf`), En küçük birim (Örnek olarak Dosya).
- Bileşik (`Composite`), içinde başka birimler barındırabilen yapı (Örnek olarak Dosya içerebilen Klasör).

Şimdi de desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <list>
#include <memory>  // unique_ptr ve make_unique için
#include <string>
#include <algorithm>

//1.--- Abstract Component ---
class Component {
protected:
        Component* ebeveyn = nullptr; // Sadece gözlem amaçlı ham pointer (sahiplik içermez)
public:
        virtual ~Component() = default;
        void setEbeveyn(Component* pEbeveyn) {
                this->ebeveyn = pEbeveyn;
        }
        Component* getEbeveyn() const {
                return this->ebeveyn;
        }
        virtual bool IsComposite() const {
                return false;
        }
        // Sahipliği devralmak için unique_ptr kullanıyoruz
        virtual void addChild(std::unique_ptr<Component> component) {}
        virtual void removeChild(Component* component) {}
        virtual std::string displayOperation() const = 0;
};

//2.--- Leaf (Yaprak) ---
class Leaf : public Component {
public:
        std::string displayOperation() const override {
                return "Yaprak";
        }
};

//3.--- Composite (Dal) ---
class Composite : public Component {
private:
        // Çocukların gerçek sahibi bu listedir
        std::list<std::unique_ptr<Component>> cocuklar;
public:
        void addChild(std::unique_ptr<Component> pComponent) override {
                pComponent->setEbeveyn(this);
                this->cocuklar.push_back(move(pComponent)); // Sahiplik listeye geçer
        }
        void removeChild(Component* pComponent) override {
                // Listede bu adrese sahip olan unique_ptr'yi bul ve sil
                cocuklar.remove_if([pComponent](const std::unique_ptr<Component>& ptr) {
                        return ptr.get() == pComponent;
                });
        }
        bool IsComposite() const override {
                return true;
        }
        std::string displayOperation() const override {
                std::string result;
                for (auto it = cocuklar.begin(); it != cocuklar.end(); ++it) {
                        if (it != cocuklar.begin()) result += "+";
                        result += (*it)->displayOperation();
                }
                return "Dal(" + result + ")";
```

```
        }
};

// 4.--- Client (İstemci) ---
int main() {
        // 1. Sadece yaprak örneği
        auto yaprak = std::make_unique<Leaf>();
        std::cout << "Sadece Yapraktan Olusan Bilesik Nesne:" ;
        std::cout << yaprak->displayOperation() << std::endl;
        // 2. Karmaşık ağaç yapısı
        auto kok = std::make_unique<Composite>();
        auto dal1 = std::make_unique<Composite>();
        // dal1'i kok'e eklemeden önce yaprakları dal1'e ekleyelim
        dal1->addChild(std::make_unique<Leaf>()); // yaprak1
        // Daha sonra silmek istediğimiz bir yaprak varsa adresini saklamalıyız
        auto yaprak2_raw = new Leaf(); // Geçici ham pointer (veya make_unique sonrası .get())
        dal1->addChild(std::unique_ptr<Leaf>(yaprak2_raw));
        kok->addChild(std::move(dal1)); // dal1'in sahipliği kok'e geçti
        std::cout << "Yapraktan ve Dallardan Olusan Bilesik Nesne:" ;
        std::cout << kok->displayOperation() << std::endl;
        // 3. Silme işlemi (yaprak2_raw üzerinden güvenli bir şekilde)
        // Not: kok içindeki dal1'e erişmek için hiyerarşiyi bilmek gerekir,
        // gerçek projelerde daha dinamik yöntemler kullanılır.
        // Burada basitlik adına doğrudan kok üzerinden gösterim yapıyoruz.
        // 4. Dal2 ekleme
        auto dal2 = std::make_unique<Composite>();
        dal2->addChild(std::make_unique<Leaf>()); // yaprak3
        kok->addChild(std::move(dal2));
        std::cout << "Bir dal daha eklenmis Bilesik Nesne:" ;
        std::cout << kok->displayOperation() << std::endl;
        // Hiçbir delete işlemine gerek yok!
        // 'kok' silindiğinde tüm dallar ve yapraklar sırayla otomatik silinir.
}
/* Programın Çıktısı:
Sadece Yapraktan Olusan Bilesik Nesne:Yaprak
Yapraktan ve Dallardan Olusan Bilesik Nesne:Dal(Dal(Yaprak+Yaprak))
Bir dal daha eklenmis Bilesik Nesne:Dal(Dal(Yaprak+Yaprak)+Dal(Yaprak))
*/
```

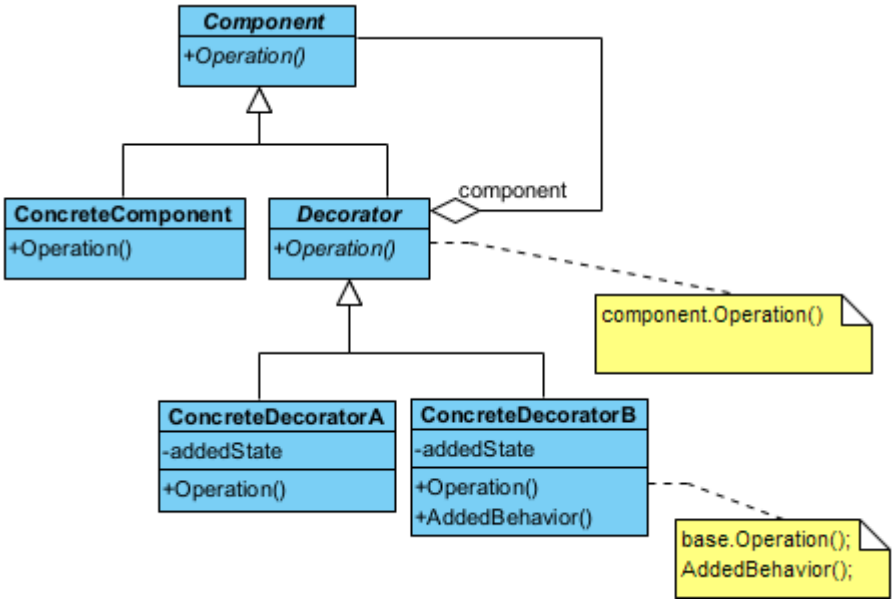
Bu desene gerçek dünya örneği olarak aşağıdaki dosya sistemini gösteren örnek verilebilir;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <utility>

//1. --- Component (Bileşen) ---
class FileSystemNode {
public:
        virtual ~FileSystemNode() = default;
        // Varsayılan parametreyi burada tanımlıyoruz
        virtual void showDetails(int indent = 0) const = 0;
        virtual long getSize() const = 0;
};

//2. --- Leaf (Yaprak) ---
class File : public FileSystemNode {
private:
        std::string name_;
        long size_;
public:
```

```
        File(std::string name, long size) : name_(std::move(name)), size_(size) {}
        // override kelimesiyle imzanın aynı olduğunu garantiliyoruz
        void showDetails(int indent) const override {
                std::string spacing(indent, ' ');
                std::cout << spacing << "- Dosya: " << name_ << " (" << size_ << " KB)" <<
                ↪  std::endl;
        }
        long getSize() const override { return size_; }
};

//3. --- Composite (Bileşik) ---
class Directory : public FileSystemNode {
private:
        std::string name_;
        std::vector<std::shared_ptr<FileSystemNode>> children_;
public:
        Directory(std::string name) : name_(std::move(name)) {}
        void add(std::shared_ptr<FileSystemNode> node) {
                children_.push_back(node);
        }
        // Dikkat: Burada 'indent = 0' yazmaya gerek yok, base class'tan devralınır.
        void showDetails(int indent) const override {
                std::string spacing(indent, ' ');
                std::cout << spacing << "+ Klasor: " << name_ << " [Toplam: " << getSize() << "
                ↪  KB]" << std::endl;
                for (const auto& child : children_) {
                        // Özyinelemeli çağrı
                        child->showDetails(indent + 4);
                }
        }
        long getSize() const override {
                long total = 0;
                for (const auto& child : children_) {
                        total += child->getSize();
                }
                return total;
        }
};

//4. --- Client (İstemci) ---
int main() {
        // Klasörleri oluşturalım
        auto root = std::make_shared<Directory>("C_Surucusu");
        auto docs = std::make_shared<Directory>("Belgelerim");
        // Dosyaları ekleyelim
        docs->add(std::make_shared<File>("Notlar.txt", 10));
        root->add(docs);
        root->add(std::make_shared<File>("Sistem.log", 500));
        // Bazı derleyiciler paylaşılan pointer (shared_ptr) üzerinden varsayılan parametreye
        ↪  erişirken sorun yaşayabilir. Açıkça 0 göndererek veya arayüzü doğru kullanarak
        ↪  çözeriz.
        root->showDetails(0);
}
/* Programın Çıktısı:
+ Klasor: C_Surucusu [Toplam: 510 KB]
+ Klasor: Belgelerim [Toplam: 10 KB]
- Dosya: Notlar.txt (10 KB)
- Dosya: Sistem.log (500 KB)
*/
```

§17.3.4 Süsleyici

Süsleyici deseninde (decorator pattern), bir nesneye dinamik olarak yeni durum ve davranışları eklemek mümkündür. Yani çalıştırma anında, bir nesnenin sahip olduğu yeteneklere, yeni yeteneklerin eklenmesini sağlar.



Şekil 17.9: Süsleyici Deseni UML Diyagramı

Bu tasarım deseni, bir nesneye dinamik olarak (çalışma zamanında) yeni sorumluluklar veya özellikler eklemek için kullanılan yapısal bir desendir. Kalıtımın (inheritance) aksine, özellikleri sınıf hiyerarşisini karmaşıklaştırmadan, nesneleri birbirinin içine "sarmalayarak" (wrapping) ekler. Bunu bir Rus matruşka bebeği veya bir kahve siparişine eklenen ekstralar (süt, şeker, karamel) gibi düşünebilirsin.

- Bileşen (`Component`), hem temel nesnenin hem de süsleyicilerin uyması gereken ortak soyut arayüzü tanımlar. Temel işlevin (Örnek olarak `ucretHesapla()`) imzasını tanımlar.
- Somut Bileşen (`ConcreteComponent`), üzerine ekleme yapılacak olan "temel" nesnedir. Fonksiyonun en yalın halini gerçekleştirir (Örnek olarak Sade Kahve).
- Soyut Süsleyici (`Decorator`), süsleyicilerin soyut temel yapısını kurar. İçinde bir `Component` referansı tutar. Bu referans sayesinde hem temel nesneyi hem de başka bir süsleyiciyi sarmalayabilir.
- Somut Süsleyiciler (ConcreteDecorator), gerçek özellikleri ekleyen sınıflardır.Temel nesnenin metodunu çağırır ve üzerine kendi eklemesini yapar (Örnek olarak Kahve ücretine +5 TL süt bedeli eklemek).

Şimdi desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenmeli
#include <iostream>
#include <string>
#include <memory> // std::unique_ptr için

// 1. Component (Bileşen): Ortak arayüz
class Component {
public:
        virtual ~Component() = default; // Kritik: Sanal yıkıcı
        virtual void operation() const = 0;
};


// 2. ConcreteComponent (Somut Bileşen): Temel nesne
class ConcreteComponent : public Component {
public:
        void operation() const override {
                std::cout << "Temel Urun Islemi" << std::endl;
        }
};
```

```
// 3. Decorator (Süsleyici): Sarmalayıcı temel sınıf
class Decorator : public Component {
protected:
        std::unique_ptr<Component> component; // Sahiplik dekoratördedir
public:
        Decorator(std::unique_ptr<Component> c) : component(std::move(c)) {}
        void operation() const override {
                if (component) component->operation();
        }
};

// 4. ConcreteDecorator1: Yeni Durum Ekleyen Süsleyici
class ConcreteDecorator1 : public Decorator {
private:
        int addedState;
public:
        ConcreteDecorator1(std::unique_ptr<Component> c, int state)
        : Decorator(std::move(c)), addedState(state * 100) {}
        void operation() const override {
                Decorator::operation(); // Önceki nesnenin davranışını çağır
                std::cout << "-> Decorator 1: Eklendi (State: " << addedState << ")" << std::endl;
        }
};

// 5. ConcreteDecorator2: Yeni Davranış Ekleyen Süsleyici
class ConcreteDecorator2 : public Decorator {
public:
        ConcreteDecorator2(std::unique_ptr<Component> c) : Decorator(std::move(c)) {}
        std::string addedBehavior() const {
                return "Bu metin Yeni Davranistan Geliyor...";
        }
        void operation() const override {
                Decorator::operation(); // Önceki nesnenin davranışını çağır
                std::cout << "-> Decorator 2: Yeni Davranis Tetiklendi: " << addedBehavior() <<
                 ↪  std::endl;
        }
};

// 6. Client (İstemci)
int main() {
        // Nesneleri iç içe sararak (stacking) oluşturuyoruz
        std::cout << "--- Dekorator Zinciri Olusturuluyor ---" << std::endl;
        // 1. Temel ürün oluştur
        auto myProduct = std::make_unique<ConcreteComponent>();
        // 2. Ürünü Decorator 1 ile süsle
        auto decorated1 = std::make_unique<ConcreteDecorator1>(std::move(myProduct), 2);
        // 3. Ortaya çıkan nesneyi Decorator 2 ile tekrar süsle
        auto decorated2 = std::make_unique<ConcreteDecorator2>(std::move(decorated1));
        // Tek bir çağrı tüm zinciri tetikler
        decorated2->operation();
    // Her şey otomatik silinir (unique_ptr sayesinde)
}
/*Program Çıktısı:
--- Dekorator Zinciri Olusturuluyor ---
Temel Urun Islemi
-> Decorator 1: Eklendi (State: 200)
-> Decorator 2: Yeni Davranis Tetiklendi: Bu metin Yeni Davranistan Geliyor...
*/
```

Aşağıda gerçek dünya örneği olarak sipariş edilen kahveyi süsleyen bir örnek verilmiştir;

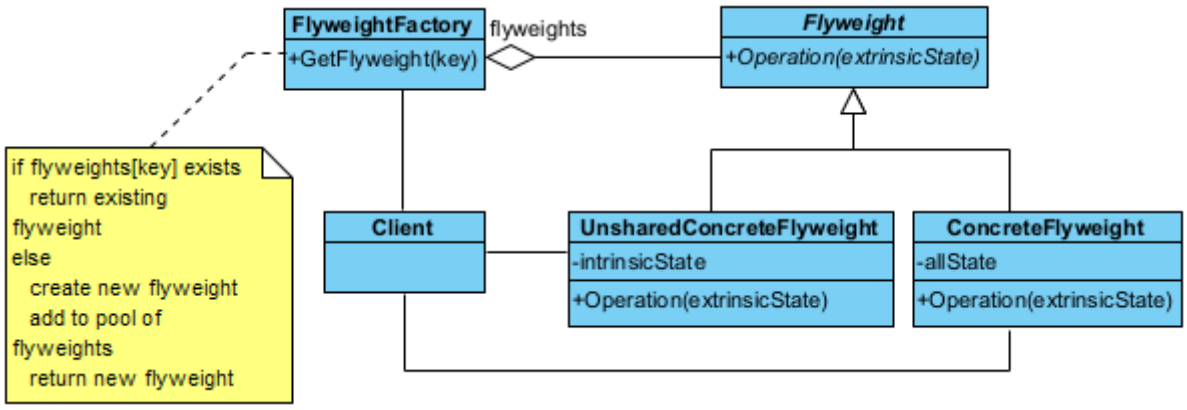
```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
```

```
#include <string>
#include <memory>  // unique_ptr ve move için gerekli
#include <utility> // std::move için

//1. --- Base Component (Temel Bileşen) ---
class Coffee {
public:
        virtual ~Coffee() = default;
        virtual double cost() const = 0;
        virtual std::string description() const = 0;
};

//2. --- Concrete Component (Somut Bileşen) ---
class SimpleCoffee : public Coffee {
public:
        double cost() const override { return 5.0; }
        std::string description() const override { return "Sade Kahve"; }
};

//3. --- Base Decorator (Temel Süsleyici) ---
class CoffeeDecorator : public Coffee {
protected:
        std::unique_ptr<Coffee> coffee_;
public:
        CoffeeDecorator(std::unique_ptr<Coffee> coffee) : coffee_(std::move(coffee)) {}
};

//4. --- Concrete Decorators (Somut Süsleyiciler) ---
class MilkDecorator : public CoffeeDecorator {
public:
        using CoffeeDecorator::CoffeeDecorator;
        double cost() const override {
                return coffee_->cost() + 1.5; // Mevcut fiyata süt ekle
        }
        std::string description() const override {
                return coffee_->description() + ", süt";
        }
};

class SugarDecorator : public CoffeeDecorator {
public:
        using CoffeeDecorator::CoffeeDecorator;
        double cost() const override {
                return coffee_->cost() + 0.5; // Mevcut fiyata şeker ekle
        }
        std::string description() const override {
                return coffee_->description() + ", seker";
        }
};

//5. --- Client (İstemci) ---
int main() {
        // 1. Sade bir kahve oluşturuyoruz
        std::unique_ptr<Coffee> kahvem = std::make_unique<SimpleCoffee>();
        // 2. Kahveye süt ekliyoruz (Sade kahveyi MilkDecorator ile sarmalıyoruz)
        kahvem = std::make_unique<MilkDecorator>(std::move(kahvem));
        // 3. Kahveye şeker ekliyoruz (Sütlü kahveyi SugarDecorator ile sarmalıyoruz)
        kahvem = std::make_unique<SugarDecorator>(std::move(kahvem));
        // Sonuçları yazdıralım
        std::cout << "Siparis: " << kahvem->description() << std::endl;
        std::cout << "Toplam Fiyat: " << kahvem->cost() << " TL" << std::endl;
}
```

```
/*
Siparis: Sade Kahve, süt, seker
Toplam Fiyat: 7 TL
*/
```

§17.3.5 Sinek Sıklet

Sinek Sıklet deseninde (flyweight pattern),çok sayıda benzer nesnenin bulunduğu durumlarda bellek kullanımını minimize etmek için kullanılan yapısal (structural) bir desendir.



Şekil 17.10: Sinek Sıklet Deseni UML Diyagramı

Bu desenin temel stratejisi, nesnelerin ortak verilerini (intrinsic state) paylaşarak bellekte tek bir kopyasını tutmak, her nesneye özgü değişen verileri (extrinsic state) ise dışarıdan sağlamaktır.

- Soyut Sinek Sıklet (`Flyweight`), paylaşılan nesnelerin uyması gereken arayüzü tanımlar. Nesneye özgü verilerin (extrinsic state) parametre olarak geçildiği yöntemin imzasını barındırır.
- Somut Sinek Sıklet (`ConcreteFlyweight`), paylaşılan veriyi (intrinsic state) kendi içinde saklayan nesnedir. Bu nesneler "değişmez" (immutable) olmalıdır. Örneğin; bir oyun için "Ağaç" modelinin 3D verisi burada bir kez tutulur.
- Sinek Sıklet Fabrikası (`FlyweightFactory`), `Flyweight` nesnelerini yönetir ve paylaşılmasını sağlar. İstendiğinde, eğer nesne daha önce oluşturulmuşsa mevcut olanı verir; yoksa yenisini oluşturup havuza ekler.
- Bağlam (`Context`), her nesneye özgü, sürekli değişen verileri (konum, renk gibi) tutar. Somut `ConcreteFlyweight` nesnesini bu verilerle birlikte kullanarak asıl işlemi gerçekleştirir.

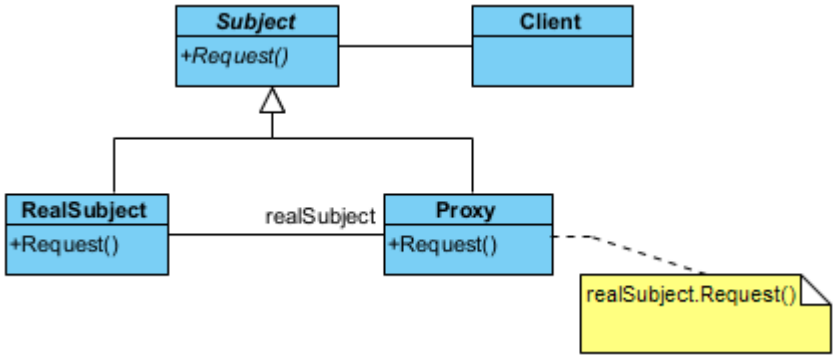
Şimdi desenimizi kodlayalım;

```
//C++17 ve üzeri ile derlenecek
#include <iostream>
#include <string>
#include <vector>
#include <unordered_map>
#include <memory>

// 1. Flyweight Arayüzü
class Flyweight {
        public:
        virtual ~Flyweight() = default;
        virtual void operation(const std::string& extrinsicState) const = 0;
        protected:
        std::string intrinsicState; // Ortak olan, değişmez durum
        Flyweight(std::string state) : intrinsicState(std::move(state)) {}
};
// 2. Somut Paylaşılan Flyweight (Concrete Flyweight)
class ConcreteFlyweight : public Flyweight {
        public:
        explicit ConcreteFlyweight(std::string state) : Flyweight(std::move(state)) {}

        void operation(const std::string& extrinsicState) const override {
                std::cout << "[Paylasilan] Ic Durum: " << intrinsicState
                << " | Dis Durum: " << extrinsicState << std::endl;
```

```
        }
};
// 3. Flyweight Factory (Fabrika)
class FlyweightFactory {
private:
        // Hızlı erişim için map kullanıyoruz. Nesne ömrünü shared_ptr yönetir.
        std::unordered_map<std::string, std::shared_ptr<Flyweight>> flyweights;
public:
        std::shared_ptr<Flyweight> getFlyweight(const std::string& key) {
                // Eğer bu anahtara sahip nesne zaten varsa onu döndür
                if (flyweights.find(key) != flyweights.end()) {
                        std::cout << "-> Fabrika: " << key << " zaten var, mevcut olan
                        ↪ döndürülüyor.\n";
                        return flyweights[key];
                }

                // Yoksa yeni bir tane oluştur ve depola
                std::cout << "-> Fabrika: " << key << " bulunamadi, yeni olusturuluyor.\n";
                auto flyweight = std::make_shared<ConcreteFlyweight>(key);
                flyweights[key] = flyweight;
                return flyweight;
        }
        void listFlyweights() const {
                std::cout << "\nFabrikadaki toplam nesne sayisi: " << flyweights.size() << "\n";
                for (const auto& [key, value] : flyweights) {
                        std::cout << "- " << key << "\n";
                }
        }
};
// 4. Client (İstemci)
int main() { // Client (İstemci)
        FlyweightFactory factory;
        // Aynı nesneleri tekrar tekrar isteyelim
        auto v1 = factory.getFlyweight("Binek Arac");
        auto v2 = factory.getFlyweight("Kamyon");
        auto v3 = factory.getFlyweight("Binek Arac"); // Fabrikadan mevcut olan gelecek
        // Dışsal durumları (konum, plaka gibi) işlem sırasında gönderiyoruz
        v1->operation("Konum: 41.00, 28.97");
        v2->operation("Konum: 39.93, 32.85");
        v3->operation("Konum: 40.71, -74.00");
        factory.listFlyweights();
        // shared_ptr sayesinde manuel delete gerekmez.
}
/*Program Çıktısı:
-> Fabrika: Binek Arac bulunamadi, yeni olusturuluyor.
-> Fabrika: Kamyon bulunamadi, yeni olusturuluyor.
-> Fabrika: Binek Arac zaten var, mevcut olan döndürülüyor.
[Paylasilan] Ic Durum: Binek Arac | Dis Durum: Konum: 41.00, 28.97
[Paylasilan] Ic Durum: Kamyon | Dis Durum: Konum: 39.93, 32.85
[Paylasilan] Ic Durum: Binek Arac | Dis Durum: Konum: 40.71, -74.00

Fabrikadaki toplam nesne sayisi: 2
- Kamyon
- Binek Arac
*/
```

§17.3.6 Vekil

Vekil deseni (proxy pattern), kullanılacak ve erişilecek bir nesnenin yerine geçen bir başka nesneye ihtiyaç duyulduğunda kullanılır. Bu desen, başka bir nesneye erişimi kontrol etmek için kullanılan yapısal bir desendir. Bir nesnenin önüne "vekil" bir katman koyarak, asıl nesneye erişmeden önce güvenlik kontrolü yapabilir, nesnenin yüklenmesini geciktirebilir (lazy loading) veya işlem sonuçlarını önbelleğe alabilirsiniz.



Şekil 17.11: Vekil Deseni UML Diyagramı

Bu deseni bir şirketteki "Yönetici Asistanı" gibi düşünebilirsin. Yöneticiyle doğrudan görüşemezsin; önce asistanla (`Proxy`) görüşürsün, asistan uygun görürse seni yöneticiye (`RealSubject`) yönlendirir.

- Bağlam (`Subject`), hem asıl nesnenin hem de vekilin (`Proxy`) uyması gereken ortak soyut arayüzü tanımlar. İstemcinin (`Client`) her iki nesneye de aynı şekilde davranabilmesini sağlar.
- Gerçek Bağlam (RealSubject), asıl işi yapan, asıl mantığı barındıran nesnedir. Genellikle oluşturulması maliyetli veya erişimi kısıtlı olan nesnedir.
- Vekil (`Proxy`), gerçek nesneye olan referansı kendi içinde saklar. İstekleri gerçek nesneye iletmeden önce araya girer. Erişim kontrolü yapabilir, gerçek nesne henüz oluşturulmamışsa onu oluşturabilir veya gelen isteği reddedebilir.
- İstemci (`Client`), nesnenin doğrudan kendisine değil, vekil (`Proxy`) nesnesine ulaşır ve bu nesneye isteklerini iletir.

Şimdi desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <memory> // std::unique_ptr için

// 1. Subject (Özne Arayüzü): Gerçek nesne ve vekilin ortak arayüzü
class Subject {
public:
        virtual ~Subject() = default; // Kritik: Modern C++ sanal yıkıcı
        virtual void request() const = 0;
};

// 2. RealSubject (Gerçek Özne): Asıl işi yapan nesne
class RealSubject : public Subject {
private:
        int state;
public:
        explicit RealSubject(int pState) : state(pState) {}
        void request() const override {
                std::cout << "RealSubject: Istek isleniyor (Durum: " << state << ")" << std::endl;
        }
};

// 3. Proxy (Vekil): Gerçek nesneye erişimi yönetir
class Proxy : public Subject {
private:
        // Mülkiyet vekilin içindedir (RAII)
        std::unique_ptr<RealSubject> realSubject;
        bool checkAccess() const {
                std::cout << "Proxy: Erisim yetkisi kontrol ediliyor..." << std::endl;
                return true;
        }
        void logAccess() const {
                std::cout << "Proxy: Islem gunlugu (log) kaydedildi." << std::endl;
        }
```

```
public:
        // Senaryo 1: Vekil kendi nesnesini oluşturur (Lazy Initialization örneği)
        Proxy() : realSubject(std::make_unique<RealSubject>(20)) {}
        // Senaryo 2: Mevcut bir nesnenin kopyası üzerinden vekillik yapar
        explicit Proxy(const RealSubject& source) :
         ↪  realSubject(std::make_unique<RealSubject>(source)) {}
        void request() const override {
                if (checkAccess()) {
                        realSubject->request();
                        logAccess();
                }
        }
};

// 4. Client (İstemci)
int main() { // Client (İstemci)
        // 1. Doğrudan erişim
        std::cout << "--- Dogrudan RealSubject Kullanimi ---" << std::endl;
        auto gercekOzne = std::make_unique<RealSubject>(10);
        gercekOzne->request();
        // 2. Kendi nesnesini yöneten vekil (Default Proxy)
        std::cout << "--- Proxy (Yeni Nesne Ile) ---" << std::endl;
        auto vekil1 = std::make_unique<Proxy>();
        vekil1->request();
        // 3. Mevcut nesneyi kopyalayan vekil
        std::cout << "--- Proxy (Mevcut Nesne Kopyasi Ile) ---" << std::endl;
        auto vekil2 = std::make_unique<Proxy>(*gercekOzne);
        vekil2->request();
        // Not: Manuel 'delete' yok! unique_ptr sayesinde her şey otomatik temizlenir.
}
/*Program Çıktısı:
--- Dogrudan RealSubject Kullanimi ---
RealSubject: Istek isleniyor (Durum: 10)
--- Proxy (Yeni Nesne Ile) ---
Proxy: Erisim yetkisi kontrol ediliyor...
RealSubject: Istek isleniyor (Durum: 20)
Proxy: Islem gunlugu (log) kaydedildi.
--- Proxy (Mevcut Nesne Kopyasi Ile) ---
Proxy: Erisim yetkisi kontrol ediliyor...
RealSubject: Istek isleniyor (Durum: 10)
Proxy: Islem gunlugu (log) kaydedildi.
*/
```

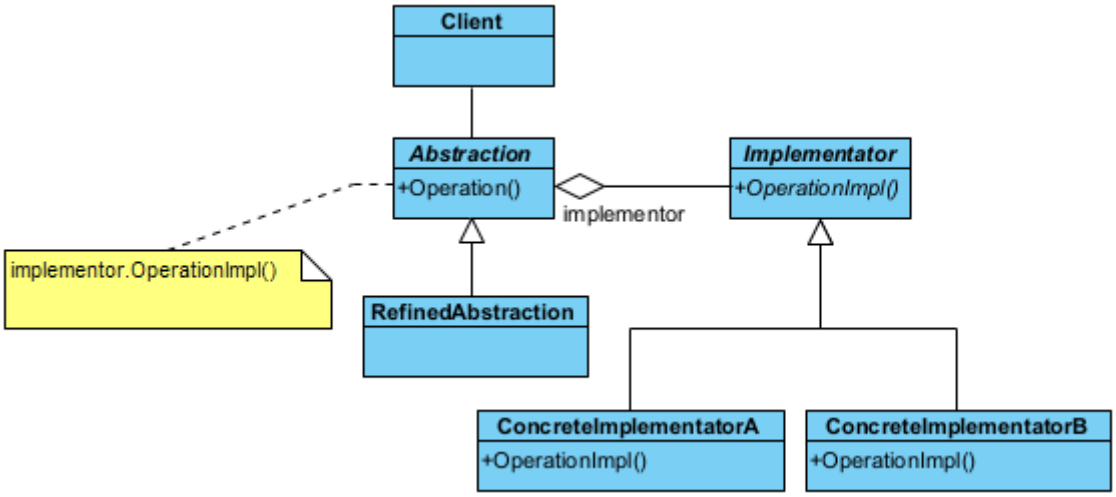
Bu desenle ilgili şu kavramlardan bahsetmek gerekir;

- Bir nesne, bulunduğu yerden uzaktaki bir nesneyi kullanacaksa uzak vekil (remote proxy) kullanılır. Gerçek nesne çok ağırsa (örneğin büyük bir resim), nesneyi sadece gerçekten ihtiyaç duyulduğunda (lazy loading) oluşturur.
- Bir nesneye başka nesneler tarafından erişim kısıtlanacaksa koruma vekili (protection proxy) olarak adlandırılır. Yetkilendirme (authentication) kontrolü yapar.
- Bir nesneye erişim sırasında başka işlemler yapılacaksa vekil, akıllı referans (smart reference) olarak adlandırılır. Böyle durumda vekil, nesneye bir erişim yapıldığında, diğer nesnelerin erişmemesi için kullanılacak nesneye kilit koyabilir. Akıllı referans, akıllı gösterici (smart pointer) olarak da adlandırılır. Nesne başka bir sunucudaysa ağ trafiğini yönetir.

§17.3.7 Köprü

Köprü deseni (bridge pattern), işi yapan alt yüklenici veya taşeron (implementator) nesne ile işi yaptıran nesne (client) arasındaki bağımlılığı soyutlama ile ortadan kaldırır. Yani birbiriyle oldukça iç içe olan kodlama ile soyutlamayı ortak ara yüz ile birbirinden uzaklaştırır. Sistemin genişlemesine izin verir. Bu desen, yazılımlarda oldukça çok kullanılan bir desendir. Çünkü taşeronlar değişse de yeni taşeron eklense de istemci tarafında bir şey değişmez.

Bu tasarım deseni, bir nesnenin soyutlamasını (abstraction), onun uygulamasından (implementation)



Şekil 17.12: Köprü Deseni UML Diyagramı

ayırarak her ikisinin de birbirinden bağımsız olarak geliştirilmesini sağlayan yapısal bir desendir. Bu desen, özellikle **sınıf patlaması** (class explosion) dediğimiz, her yeni özellik için onlarca alt sınıf türetmek zorunda kaldığımız durumlarda imdadımıza yetişir. Kalıtım yerine bileşimi (composition) kullanır.

- Soyutlama (`Abstraction`), istemcinin (`Client`) kullandığı üst düzey kontrol arayüzüdür. İçinde bir `Implementor` referansı tutar. İstemciden gelen komutları bu referansa iletir.
- Geliştirilmiş Soyutlama (`RefinedAbstraction`),soyutlamayı genişleten alt sınıflardır. Temel arayüzü kullanarak daha spesifik kontrol mantıkları ekler (Örn: Standart Kumanda -> Gelişmiş Akıllı Kumanda).
- Taşeron Arayüzü (`Implementor`), tüm somut uygulama sınıfları için ortak arayüzü tanımlar. Genellikle düşük seviyeli (ilkel) operasyonları barındırır (Örn: `cihaziAc()`, `sesAyari()`).
- Somut Taşeronlar (`ConcreteImplementor`), `Implementor` arayüzünü gerçekleyen asıl cihazlardır. Platforma veya cihaza özgü gerçek kodları barındırır (Örn: Televizyon, Radyo).

Şimdi desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenmeli
#include <iostream>
#include <memory> // Akıllı işaretçiler için
#include <utility>

//1.--- Implementor (Uygulayıcı/Taşeron Arayüzü) ---
class Implementor {
public:
        virtual ~Implementor() = default; // Güvenli temizlik için sanal yıkıcı
        virtual void operationImpl() = 0;
};

//2.--- Concrete Implementors ---
class ConcreteImplementorA : public Implementor {
public:
        void operationImpl() override {
                std::cout << "A Taseronu tarafindan yapilan is..." << std::endl;
        }
};
class ConcreteImplementorB : public Implementor {
public:
        void operationImpl() override {
                std::cout << "B Taseronu tarafindan yapilan is..." << std::endl;
        }
};

//3.--- Abstraction (Soyut Yüklenici) ---
class Abstraction {
protected:
```

```
        // Taşeronun mülkiyeti (sahipliği) yükleniciye ait olsun veya olmasın,
        // burada shared_ptr veya unique_ptr seçimi stratejiktir.
        // Taşeronların çalışma anında değişebilirliği için unique_ptr ile sahipliği yönetiyoruz.
        std::unique_ptr<Implementor> implementor;
public:
        Abstraction(std::unique_ptr<Implementor> pImpl) : implementor(std::move(pImpl)) {}
        virtual ~Abstraction() = default;
        // Çalışma anında taşeronu güvenli bir şekilde değiştiriyoruz
        void setImplementor(std::unique_ptr<Implementor> pImpl) {
                implementor = move(pImpl);
        }
        void operation() {
                if (implementor) {
                        std::cout << "Yuklenici Taserona is yaptiriyor:" << std::endl;
                        implementor->operationImpl();
                }
        }
};

//4.--- Refined Abstraction (Geliştirilmiş Soyutlama) ---
class RefinedAbstraction : public Abstraction {
public:
        using Abstraction::Abstraction; // Üst sınıfın constructor'ını kullan
};

// 5.--- Client (İstemci) ---
int main() {
        // 1. Yükleniciyi ilk taşeronla (A) başlatıyoruz
        auto yuklenici =
        ↪  std::make_unique<RefinedAbstraction>(std::make_unique<ConcreteImplementorA>());
        yuklenici->operation();
        std::cout << "--- Taseron Degistiriliyor ---" << std::endl;
        // 2. Çalıştırma anında yeni bir taşerona (B) geçiyoruz
        // Eski taşeron (A), unique_ptr'nin doğası gereği otomatik olarak hafızadan silinir.
        yuklenici->setImplementor(std::make_unique<ConcreteImplementorB>());
        yuklenici->operation();
        // Hiçbir delete işlemine gerek yok! 'yuklenici' kapsam dışına çıktığında içindeki
        ↪  taşeronla birlikte silinecektir.
}
/*Program Çıktısı:
Yuklenici Taserona is yaptiriyor:
A Taseronu tarafindan yapilan is...
--- Taseron Degistiriliyor ---
Yuklenici Taserona is yaptiriyor:
B Taseronu tarafindan yapilan is...
*/
```

Köprü deseni aşağıdaki durumlarda kullanılır;

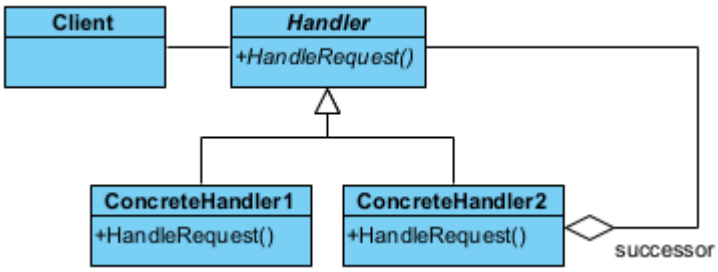
- Taşeron nesneye kalıcı bir bağımlılık istenmediği durumlar. Çalışma anında (run-time) taşeronlar arasında geçiş yapmayı sağlar.
- Birden çok taşeron kullanarak sistemin genişlemesine ihtiyaç olduğu durumlar.
- Taşeronlar üzerindeki değişiklikler, istemciyi etkilemez. İstemci tarafında derleme olmadan yazılımın değiştirilmesi sağlanır.
- Projede aynı anda farklı birden çok taşeron tasarımı yapılabildiğinden, proje parçalara ayrılarak kendi içinde hızla sınıf tasarımı yapılabilir. Bu durum *Rumbaugh* tarafından iç içe genelleştirme (nested generalization) olarak adlandırılmıştır.

17.4 Davranışla İlgili Desenler

Davranışla ilgili desenler (behavioral patterns) nesnelerin çeşitli olaylar karşısında gösterecekleri davranışlara çözüm getirmektedirler.

§17.4.1 Sorumluluk Zinciri

Bazı durumlarda bir nesne, kendinin yapmayacağı bir işlemi halefine (successor) devredebilir. Ya da nesnenin kendi sorumluluğunu aşan durumlarda ilgili diğer nesneye işlem yaptırılır. İşte bu durumda **sorumluluk zinciri deseni** (chain of responsibility pattern) kullanılır.



Şekil 17.13: Sorumluluk Zinciri Deseni UML Diyagramı

Bu desenin temel amacı, isteği gönderen ile isteği işleyen arasındaki bağı gevşetmektir (decoupling). İstek, zincir boyunca ilerler ve her bir işleyici ya isteği çözer ya da bir sonraki işleyiciye paslar.

- İşleyici (`Handler`), İsteklerin nasıl işleneceğine dair ortak bir arayüz tanımlar. İşleyici, bir sonraki işleyiciyi tutan bir referans barındırır ve zincirin halkalarını birbirine bağlar.
- Somut İşleyiciler (`ConcreteHandler`), kendisine gelen isteği kontrol eder. Eğer istek kendi sorumluluk alanındaysa işi bitirir. Eğer değilse (veya işin bir kısmını yapıp devam etmesi gerekiyorsa), isteği zincirdeki bir sonraki halkaya iletir.
- İstemci (`Client`), zinciri oluşturur (halkaları birbirine bağlar) ve ilk isteği zincirin başına gönderir. İsteğin kim tarafından, nasıl çözüleceğiyle ilgilenmez; sadece "çözülmesini" bekler.

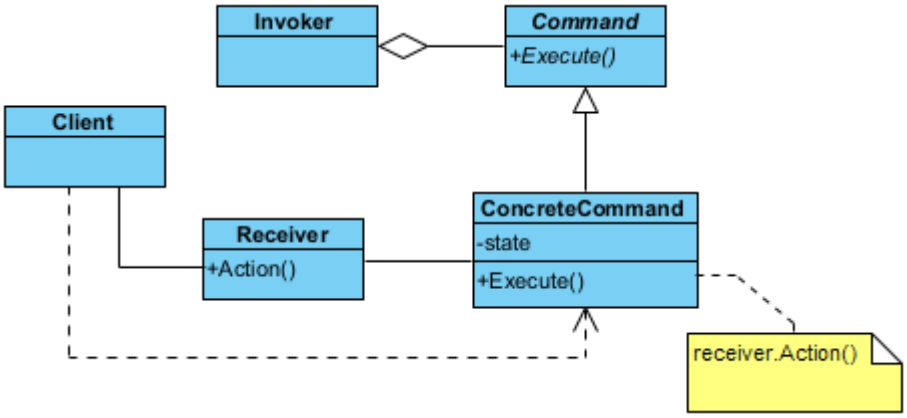
Şimdi desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenecek
#include <iostream>
#include <memory>

//1. Soyut İşleyici (Handler)
class Handler {
protected:
        std::unique_ptr<Handler> successor; // Zincirin bir sonraki halkası
public:
        virtual ~Handler() = default;
        // Zincire yeni bir halka eklemek için yardımcı metod
        void setSuccessor(std::unique_ptr<Handler> next) {
                successor = std::move(next);
        }
        virtual void handleRequest(int request) = 0;
};

//2. Somut İşleyici 1: Küçük Talepler (Örn: Memur)
class ConcreteHandler1 : public Handler {
public:
        void handleRequest(int request) override {
                if (request < 100) { //Birinci Halka: Kendisi
                        std::cout << "Memur: " << request << " birimlik talebi onayladi." <<
                        ↪ std::endl;
                } else if (successor) { //İkinci Halka: Halefi
                        std::cout << "Memur: Talep limitimi asiyor, Sefe devrediliyor..." <<
                        ↪ std::endl;
                        successor->handleRequest(request);
                } else {
                        std::cout << "Hata: Talebi karsilayacak kimse bulunamadi!" << std::endl;
                }
        }
};

//3. Somut İşleyici 2: Büyük Talepler (Örn: Sef)
```

```
class ConcreteHandler2 : public Handler {
public:
        void handleRequest(int request) override {
                if (request >= 100 && request < 500) {
                        std::cout << "Sef: " << request << " birimlik talebi onayladi." <<
                        ↪ std::endl;
                } else if (successor) {
                        std::cout << "Sef: Talep limitimi asiyor, Mudure devrediliyor..." <<
                        ↪ std::endl;
                        successor->handleRequest(request);
                } else {
                        std::cout << "Sef: Bu kadar buyuk bir yetkim yok ve baska amir tanimli
                        ↪ degil!" << std::endl;
                }
        }
};

//4. Client (İstemci)
int main() {
        // Zinciri oluşturuyoruz
        auto memur = std::make_unique<ConcreteHandler1>();
        auto sef = std::make_unique<ConcreteHandler2>();
        // Memurun halefi Sef olsun
        memur->setSuccessor(std::move(sef));
        std::cout << "--- Talep 1 (50) ---" << std::endl;
        memur->handleRequest(50);
        std::cout << "--- Talep 2 (250) ---" << std::endl;
        memur->handleRequest(250);
        std::cout << "--- Talep 3 (1000) ---" << std::endl;
        memur->handleRequest(1000);
        // unique_ptr sayesinde tüm zincir otomatik olarak silinir.
}
/*Program Çıktısı:
--- Talep 1 (50) ---
Memur: 50 birimlik talebi onayladi.
--- Talep 2 (250) ---
Memur: Talep limitimi asiyor, Sefe devrediliyor...
Sef: 250 birimlik talebi onayladi.
--- Talep 3 (1000) ---
Memur: Talep limitimi asiyor, Sefe devrediliyor...
Sef: Bu kadar buyuk bir yetkim yok ve baska amir tanimli degil!
*/
```

§17.4.2 Komut

Bazı durumlarda bir nesne diğer bir nesneye ileti (message) verip bir işlem başlattığında, bu işlem bitmeden başka işlemler yapmak isteyebilir. İşte bu tür durumlarda **komut deseni** (command pattern) kullanılır.



Şekil 17.14: Komut Deseni UML Diyagramı

Bu tasarım deseni, bir isteği veya eylemi kendi başına bir nesneye dönüştüren davranışsal bir desendir. Bu dönüşüm sayesinde eylemleri parametre olarak geçebilir, kuyruğa alabilir (queue), iz kaydını tutabilir (log) veya geri alabilirsiniz (undo). Kısacası; "ne yapılacağı" bilgisini, "kimin yapacağı" bilgisinden tamamen ayırır.

- Komut (`Command`), tüm komutların uyması gereken soyut arayüzü tanımlar. Genellikle tek bir yöntem barındırır (Örnek olarak `execute()`). Bazı durumlarda geri alma işlemi için `undo()` yöntemini de içerir.
- Somut Komut (`ConcreteCommand`), soyut komutu gerçek bir eylemle birleştirir. Alıcı (`Receiver`) nesnesini tanır ve `execute()` çağrıldığında alıcının ilgili metodunu tetikler.
- Alıcı(Receiver), işin asıl mutfağıdır. Gerçek mantığı ve operasyonları barındırır. Komut nesnesi ona "çalış" dediğinde, o işi nasıl yapacağını bilir (Örnek olarak Işığı açmak, dosyayı kaydetmek).
- Çağırıcı(`Invoker`), komutun ne zaman çalışacağına karar verir. Komutu saklar ve kullanıcı bir düğmeye bastığında veya bir olay gerçekleştiğinde komutun `execute()` metodunu çağırır. Alıcıyı doğrudan tanımaz.

Şimdi desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenecek
#include <iostream>
#include <string>
#include <vector>
#include <memory>

//1. Receiver (Alıcı): Asıl işi yapan sınıf
class Receiver {
public:
        void action(const std::string& pParam) const {
                std::cout << "Receiver: [" << pParam << "] komutu icin temel islemler
                ↪  yapiliyor.\n";
        }
        void actionA(const std::string& pParam) const {
                std::cout << "Receiver: [" << pParam << "] icin A islemi.\n";
        }
        void actionB(const std::string& pParam) const {
                std::cout << "Receiver: [" << pParam << "] icin B islemi.\n";
        }
};

//2. Command (Komut Arayüzü)
class Command {
public:
        virtual ~Command() = default;
        virtual void execute() const = 0;
};

//3. Concrete Commands (Somut Komutlar)
class SimpleCommand : public Command {
private:
        std::string state;
        Receiver& receiver; // Komut, mevcut bir alıcıya referansla bağlanır
public:
        SimpleCommand(std::string pState, Receiver& r) : state(std::move(pState)), receiver(r) {}
        void execute() const override {
                receiver.action(state);
        }
};
class ComplexCommand : public Command {
private:
        std::string state;
        Receiver& receiver;
public:
        ComplexCommand(std::string pState, Receiver& r) : state(std::move(pState)), receiver(r)
        ↪  {}
        void execute() const override {
                receiver.actionA(state);
```

```
                receiver.actionB(state);
        }
};

//4. Invoker (Çağırıcı): Komutları saklar ve tetikler
class Invoker {
private:
        std::vector<std::unique_ptr<Command>> history; // Komut geçmişini saklar
public:
        void storeAndExecute(std::unique_ptr<Command> cmd) {
                cmd->execute();
                history.push_back(std::move(cmd)); // Komutu sakla (Undo işlemleri için
                ↪  gerekebilir)
        }
};

//5. Client (İstemci)
int main() {
        // Tek bir alıcı (Receiver) oluşturulur
        Receiver receiver;
        Invoker invoker;
        std::cout << "--- Komutlar Tetikleniyor ---\n";
        // Komutlar oluşturulur ve Invoker aracılığıyla çalıştırılır
        invoker.storeAndExecute(std::make_unique<SimpleCommand>("Yazdir", receiver));
        invoker.storeAndExecute(std::make_unique<ComplexCommand>("Logla ve Gönder", receiver));
        // Her şey otomatik temizlenir
}
/*Program Çıktısı:
--- Komutlar Tetikleniyor ---
Receiver: [Yazdir] komutu icin temel islemler yapiliyor.
Receiver: [Logla ve Gönder] icin A islemi.
Receiver: [Logla ve Gönder] icin B islemi.
*/
```

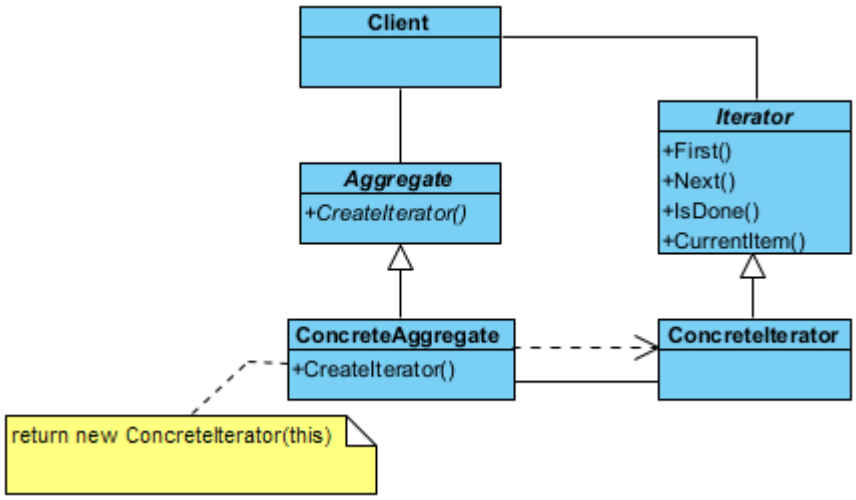
Aşağıda komut desenine uygun bir ışığım kapatılıp açılması örneği verilmiştir;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <vector>
#include <memory>
#include <string>

// --- Receiver (Alıcı) ---
class Light {
public:
        void on()  { std::cout << "Isik ACILDI." << std::endl; }
        void off() { std::cout << "Isik KAPATILDI." << std::endl; }
};

// --- Command (Soyut Komut) ---
class Command {
public:
        virtual ~Command() = default;
        virtual void execute() = 0;
        virtual void undo() = 0;
};

// --- Concrete Commands (Somut Komutlar) ---
class LightOnCommand : public Command {
private:
        Light& light_; // Referans kullanımı: Işık nesnesi var olmak zorundadır.
public:
        explicit LightOnCommand(Light& light) : light_(light) {}
```

```
        void execute() override { light_.on(); }
        void undo() override    { light_.off(); }
};
class LightOffCommand : public Command {
private:
        Light& light_;
public:
        explicit LightOffCommand(Light& light) : light_(light) {}
        void execute() override { light_.off(); }
        void undo() override    { light_.on(); }
};

// --- Invoker (Çağırıcı) ---
class RemoteControl {
private:
        // Geçmişi tutan yığın (LIFO - Last In First Out)
        std::vector<std::unique_ptr<Command>> history_;
public:
        void executeCommand(std::unique_ptr<Command> cmd) {
                cmd->execute();
                history_.push_back(std::move(cmd));
        }
        void undoLast() {
                if (!history_.empty()) {
                        // back() ile son komutu alıyoruz
                        std::cout << "[UNDO] ";
                        history_.back()->undo();
                        // Komut işini bitirdi, artık hafızadan silebiliriz
                        history_.pop_back();
                } else {
                        std::cout << "[UYARI] Geri alinacak islem yok!" << std::endl;
                }
        }
};

// --- Client (İstemci) ---
int main() {
        Light oturmaOdasiIsigi;
        RemoteControl kumanda;
        std::cout << "--- Islemler Basliyor ---" << std::endl;
        // Işığı açalım
        kumanda.executeCommand(std::make_unique<LightOnCommand>(oturmaOdasiIsigi));
        // Işığı kapatalım
        kumanda.executeCommand(std::make_unique<LightOffCommand>(oturmaOdasiIsigi));
        std::cout << std::endl <<"--- Geri Alma (Undo) Islemleri ---" << std::endl;
        kumanda.undoLast(); // Kapatmayı geri alır -> Işığı Açar
        kumanda.undoLast(); // Açmayı geri alır    -> Işığı Kapatır
        kumanda.undoLast(); // Liste boş uyarısı verir
}
/*Program Çıktısı:
--- Islemler Basliyor ---
Isik ACILDI.
Isik KAPATILDI.

--- Geri Alma (Undo) Islemleri ---
[UNDO] Isik ACILDI.
[UNDO] Isik KAPATILDI.
[UYARI] Geri alinacak islem yok!
*/
```

§17.4.3 Yineleyici

Yineleyici deseninde (iterator pattern), birden fazla elemandan oluşan küme (aggregate) içindeki her bir elemana sırayla erişim (iteration) sağlar. Erişim işleminde, istemcinin kümenin nasıl yapılandırıldığı konusunda bilgi sahibi olması gerekmez. Bu desende, istemci tarafından veri kümesi soyut (abstract) olarak kullanılmış olur. Yani istemciye, veri yapısından bağımsız bir şekilde verilere erişim sağlayan bir ara yüz (interface) sunulur. Bu tasarım deseni, bir veri grubunun (liste, ağaç, yığın vb.) iç yapısını dışarıya açmadan, o



Şekil 17.15: Yineleyici Deseni UML Diyagramı

grubun elemanları üzerinde sırayla gezinebilmemizi sağlayan davranışsal bir desendir. Bu desen sayesinde, verilerin bellekte nasıl tutulduğunu (bir dizi mi yoksa birbirine bağlı halkalar mı?) bilmenize gerek kalmadan standart bir şekilde tüm elemanları dolaşabilirsiniz.

- Yineleyici (`Iterator`), elemanlar üzerinde gezinmek için gerekli olan standart arayüzü tanımlar. Genellikle `next()` (sonraki eleman), `hasNext()` (başka eleman var mı?) ve `current()` (mevcut eleman) gibi yöntemleri barındırır.
- Somut Yineleyici (`ConcreteIterator`), belirli bir veri yapısı (örneğin bir Vektör veya Liste) için gezinme mantığını uygular. O an hangi elemanda olduğunu (indis veya işaretçi bazlı) takip eder.
- Soyut Koleysiyon (`Aggregate`), koleksiyonun (container) kendisi için ortak bir arayüz sağlar. Kendi elemanlarını gezmek için kullanılacak olan uygun `Iterator` nesnesini oluşturan `createIterator()` adlı bir yönteme sahitir.
- Somut Koleksiyon (`ConcreteAggregate`), verileri asıl saklayan sınıftır. İstendiğinde, kendi iç yapısını en verimli şekilde gezecek olan somut yineleyiciyi (`ConcreteIterator`) geri döndürür.

Şimdi desenimizi kodlayalım;

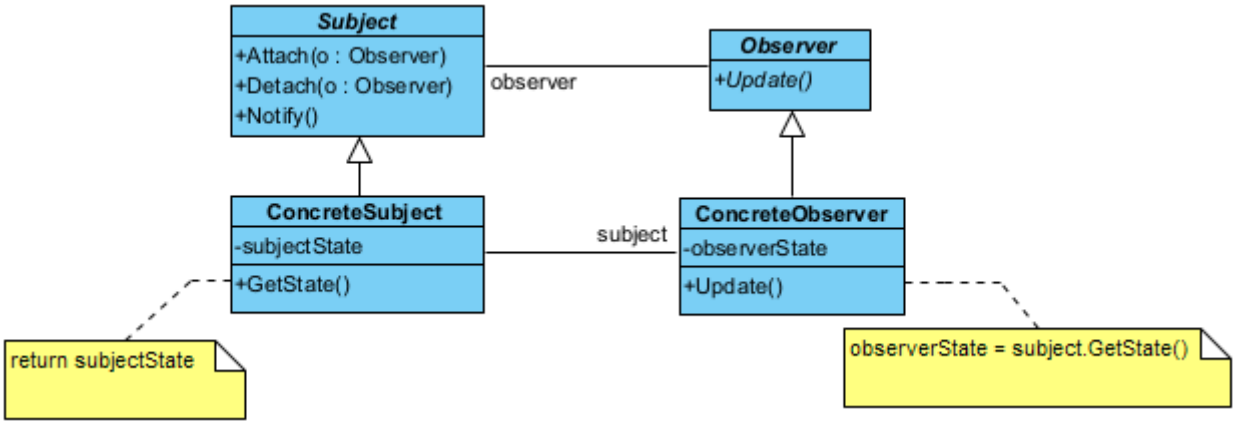
```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <vector>
#include <memory> // std::unique_ptr için

//1. Soyut Iterator Arayüzü
class Iterator {
        public:
        virtual ~Iterator() = default;
        virtual double first() = 0;
        virtual double next() = 0;
        virtual bool hasNext() const = 0; // isDone yerine daha yaygın isimlendirme
        virtual double currentItem() const = 0;
};

//2. Soyut Koleksiyon (Aggregate) Arayüzü
class Aggregate {
public:
        virtual ~Aggregate() = default;
        virtual void addItem(double item) = 0;
        virtual size_t size() const = 0;
```

```
        virtual double getItem(size_t index) const = 0;
        virtual std::unique_ptr<Iterator> createIterator() = 0;
};

//3. Somut Iterator (Concrete Iterator)
class ConcreteIterator : public Iterator {
private:
        Aggregate* aggregate; // Koleksiyona sadece referans/işaretçi tutar, mülkiyet almaz
        size_t current = 0;
public:
        explicit ConcreteIterator(Aggregate* pAggregate) : aggregate(pAggregate) {}
        double first() override {
                current = 0;
                return currentItem();
        }
        double next() override {
                if (hasNext()) {
                        return aggregate->getItem(current++);
                }
                return -1.0; // Veya hata fırlatılabilir
        }
        bool hasNext() const override {
                return current < aggregate->size();
        }
        double currentItem() const override {
                return aggregate->getItem(current);
        }
};

//4. Somut Koleksiyon (Concrete Aggregate)
class ConcreteAggregate : public Aggregate {
private:
        std::vector<double> items;
public:
        void addItem(double item) override {
                items.push_back(item);
        }
        size_t size() const override {
                return items.size();
        }
        double getItem(size_t index) const override {
                return items.at(index); // out_of_range kontrolü için .at() daha güvenlidir
        }
        std::unique_ptr<Iterator> createIterator() override {
                // factory method: mülkiyeti istemciye (unique_ptr ile) devreder
                return std::make_unique<ConcreteIterator>(this);
        }
};

//5. Client (İstemci)
int main() {// Client (İstemci)
        auto aggregate = std::make_unique<ConcreteAggregate>();
        aggregate->addItem(10.5);
        aggregate->addItem(20.7);
        aggregate->addItem(30.9);
        auto iterator = aggregate->createIterator();
        std::cout << "Koleksiyon elemanlari geziliyor:" << std::endl;
        // Standart döngü yapısı
        for (iterator->first(); iterator->hasNext(); ) {
                std::cout << "Deger: " << iterator->next() << std::endl;
        }
    // Manuel delete gerekmez, her şey otomatik temizlenir
```

```
}
/*Program Çıktısı:
Koleksiyon elemanlari geziliyor:
Deger: 10.5
Deger: 20.7
Deger: 30.9
*/
```

§17.4.4 Gözlemci

Gözlemci deseni (observer pattern), sistemdeki diğer nesnelerin durum değişiklikleri hakkında bir veya daha fazla nesnenin bilgilendirilmesini sağlar.



Şekil 17.16: Gözlemci Deseni UML Diyagramı

Bu tasarım deseni, nesneler arasında bire-çok (one-to-many) bir bağımlılık kurmak için kullanılır. Bir nesnenin durumu değiştiğinde, ona bağlı olan tüm diğer nesnelerin (abonelerin) bu değişiklikten otomatik olarak haberdar edilmesini ve güncellenmesini sağlar. Bu desen, modern yazılımlardaki "olay güdümlü" (event-driven) sistemlerin ve MVC (Model-View-Controller) mimarisinin kalbidir.

- Bağlam (`Subject`), takip edilen asıl yayıncı nesnedir. Durumu değiştikçe etrafa haber salar. İçinde bir "Gözlemci Listesi" tutar. Yeni gözlemci eklemek (`attach()`), çıkarmak (`detach()`) ve hepsine haber vermek (`notify()`) için yöntemler barındırır.
- Gözlemci (`Observer`), güncelleme mesajlarını alabilmek için gerekli olan standart abone arayüzünü tanımlar. Genellikle sadece tek bir `update()` metodu içerir.
- Somut Yayıncı (`ConcreteSubject`), gerçek veriyi saklayan sınıftır (Örnek olarak bir borsa takip sistemi veya haber sitesi). Kendi içindeki veri değiştiğinde, miras aldığı `notify()` metodunu çağırarak tüm aboneleri tetikler.
- Somut Gözlemci (`ConcreteObserver`), haberi alan ve buna göre tepki veren sınıftır. `update()` yöntemi tetiklendiğinde, yayıncıdan gelen yeni veriyi alır ve kendi içinde işler (Ekrana yazar, veritabanına kaydeder vb.).

Şimdi desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <vector>
#include <string>
#include <memory>
#include <algorithm>

//1. Gözlemci: Observer Arayüzü
class Observer {
public:
        virtual ~Observer() = default; // Kritik: Sanal yıkıcı
        virtual void update(int state) = 0; // Durumu doğrudan parametre olarak alabilir (Push
        ↪ model)
};

//2. Bağlam/Yayıncı: Taban Sınıfı
class Subject {
```

```
        private:
        // weak_ptr kullanımı: Gözlemci silinmişse Subject onu zorla hayatta tutmaz.
        std::vector<std::weak_ptr<Observer>> observers;
        public:
        virtual ~Subject() = default;
        void attach(std::shared_ptr<Observer> observer) {
                observers.push_back(observer);
        }
        void detach(std::shared_ptr<Observer> observer) {
                observers.erase(std::remove_if(observers.begin(), observers.end(),
                [&observer](const std::weak_ptr<Observer>& wp) {
                        return wp.expired() || wp.lock() == observer;
                }), observers.end());
        }
        void notify(int state) {
                for (auto it = observers.begin(); it != observers.end(); ) {
                        if (auto obj = it->lock()) { // Nesne hala hayattaysa
                                obj->update(state);
                                ++it;
                        } else { // Nesne silinmişse listeden temizle
                                it = observers.erase(it);
                        }
                }
        }
};

//3. Somut Bağlam/Yayıncı (Concrete Subject)
class NewsAgency : public Subject {
private:
        int newsId = 0;
public:
        void setNews(int id) {
                newsId = id;
                std::cout << "Haber Ajansi: Yeni haber yayinlandi (ID: " << id << ")\n";
                notify(newsId); // Otomatik bildirim
        }
};

//4. Somut Gözlemci (Concrete Observer)
class NewsChannel : public Observer {
private:
        std::string name;
public:
        explicit NewsChannel(std::string n) : name(std::move(n)) {}
        void update(int state) override {
                std::cout << " -> " << name << " kanali bilgiyi aldi. Yeni haber ID: " << state
                ↪  << std::endl;
        }
};

//5. Client (İstemci)
int main() { // İstemci (Client)
        // Nesneleri paylaşımlı akıllı işaretçiler ile oluşturuyoruz
        auto agency = std::make_shared<NewsAgency>();
        auto channel1 = std::make_shared<NewsChannel>("TRT Haber");
        auto channel2 = std::make_shared<NewsChannel>("NTV");
        auto channel3 = std::make_shared<NewsChannel>("CNN Turk");
        agency->attach(channel1);
        agency->attach(channel2);
        agency->attach(channel3);
        // Bir durumu değiştirelim, bildirimler otomatik gidecek
        agency->setNews(101);
```

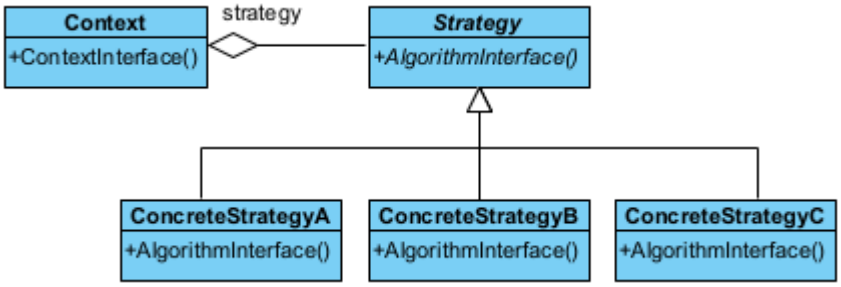
```
        std::cout << "-----------------------------------" << std::endl;
        agency->detach(channel2); // NTV ayrılıyor
        agency->setNews(102);
        // Manuel 'delete' yok, bellek sızıntısı imkansız.
}
/*Program Çıktısı:
Haber Ajansi: Yeni haber yayinlandi (ID: 101)
-> TRT Haber kanali bilgiyi aldi. Yeni haber ID: 101
-> NTV kanali bilgiyi aldi. Yeni haber ID: 101
-> CNN Turk kanali bilgiyi aldi. Yeni haber ID: 101
-----------------------------------
Haber Ajansi: Yeni haber yayinlandi (ID: 102)
-> TRT Haber kanali bilgiyi aldi. Yeni haber ID: 102
-> CNN Turk kanali bilgiyi aldi. Yeni haber ID: 102
*/
```

Bu desen aşağıdaki durumlarda kullanılır;

- Bir soyutlama (abstraction) sonucu ortaya çıkan bağımsız iki farklı yönün (aspect) ortaya çıktığı durumlarda, bu yönler birbirinden bağımsız olarak geliştirilebilir ve değiştirilebilir.
- Bir değişikliğin başka değişiklikleri gerektirdiği durumlar.
- Bir nesnenin, diğer nesnelerdeki değişiklikleri kendine vazife etmemesini sağlayan durumlar.

§17.4.5 Strateji

Strateji deseni (strategy pattern), belirli bir davranışı gerçekleştirmek için değiştirilebilen kapsüllenmiş (encapsulated) algoritmalar kümesini tanımlar.



Şekil 17.17: Strateji Deseni UML Diyagramı

Bu desenin görevi, bir işi yapmanın birden fazla yolu varsa, bu yolları (algoritmaları) sınıflara ayırır ve birbirinin yerine kullanılabilir hale getirir.

- Bağlam (`Context`), işi yapacak olan sınıf. Ancak işi "nasıl" yapacağını bir strateji nesnesine sorar.
- Strateji (`Strategy`): Tüm algoritmaların uyması gereken ara yüzdür.
- Somut Strateji (`ConcreteStrategy`), farklı algoritmalar ile tanımlı stratejilerdir (Örnek olarak "Hızlı Sıralama", "Kabarcık Sıralama" veya "Kredi Kartıyla Öde", "Nakit Öde").

Şimdi desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlnmelidir
#include <iostream>
#include <memory>
#include <vector>

//1. Strateji Arayüzü
class Strategy {
public:
        virtual ~Strategy() = default; // Kritik: Sanal yıkıcı
        virtual void execute() const = 0;
};

//2. Somut Stratejiler
class ConcreteStrategyA : public Strategy {
public:
        void execute() const override {
                std::cout << "Algoritma A: Hizli ama daha fazla bellek kullaniyor.\n";
```

```
        }
};
class ConcreteStrategyB : public Strategy {
public:
        void execute() const override {
                std::cout << "Algoritma B: Yavas ama bellek dostu.\n";
        }
};
class ConcreteStrategyC : public Strategy {
public:
        void execute() const override {
                std::cout << "Algoritma C: Dengeli bir yaklasim.\n";
        }
};

//3. Baglam (Context)
class Context {
private:
        // Stratejinin mulkiyetini almaz, sadece referans/isaretci tutar (Aggregation)
        Strategy* strategy_ = nullptr;
public:
        explicit Context(Strategy* strategy) : strategy_(strategy) {}
        void setStrategy(Strategy* strategy) {
                strategy_ = strategy;
        }
        void run() const {
                if (strategy_) {
                        strategy_->execute();
                } else {
                        std::cout << "Hata: Henuz bir strateji belirlenmedi!\n";
                }
        }
};

//4. Client (İstemci)
int main() {
        // Stratejileri akilli isaretcilerle yonetiyoruz
        auto s1 = std::make_unique<ConcreteStrategyA>();
        auto s2 = std::make_unique<ConcreteStrategyB>();
        auto s3 = std::make_unique<ConcreteStrategyC>();
        // Baglam (Context) nesnesini olustur ve baslangic stratejisini ata
        Context context(s1.get());
        std::cout << "--- Durum 1 ---" << std::endl;
        context.run();
        // Calisma zamaninda stratejiyi degistir
        context.setStrategy(s2.get());
        std::cout << "--- Durum 2 ---" << std::endl;
        context.run();
        context.setStrategy(s3.get());
        std::cout << "--- Durum 3 ---" << std::endl;
        context.run();
        // Manuel 'delete' yok, s1-s2-s3 otomatik temizlenir.
}
/*Program Çıktısı:
--- Durum 1 ---
Algoritma A: Hizli ama daha fazla bellek kullaniyor.
--- Durum 2 ---
Algoritma B: Yavas ama bellek dostu.
--- Durum 3 ---
Algoritma C: Dengeli bir yaklasim.
*/
```

Bu desenin **bağlam** nesnesinin, diğer tasarım desenlerinin biri ile iç içe kullanılması halinde, bu desen **sıfır**

desen (null pattern) olarak adlandırılır. Sıfır desende, `Context` nesnesi akıllıdır ve her zaman ne yapacağını bilir. Bu desen aşağıdaki durumda kullanılır;

- Aralarında ilişki bulunan birçok sınıfın davranışlara bağlı olarak anlaşmazlığa düşmesini önlemek için,
- Birden çok algoritmanın olması durumunda,
- Algoritma, istemcinin bilmeyeceği bir veri yapısına sahip olduğu durumlarda,
- Bir sınıf kendi yöntemlerinde çok fazla sayıda duruma göre seçim (conditional choice) kullanıyorsa.

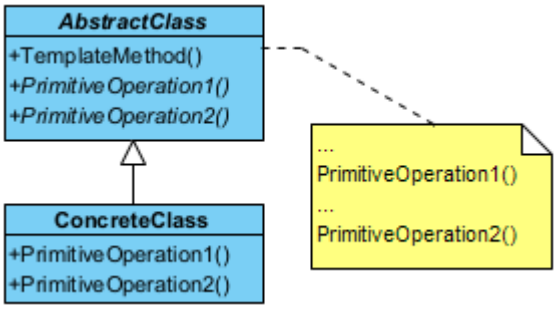
Aşağıda bu desene ilişkin sıralama algoritmalarının seçildiği gerçek bir örnek verilmiştir;

```
//C++ 14 ve üzeri ile derlenmelidir.
#include <iostream>
#include <vector>
#include <memory>
#include <algorithm> // std::sort için

// --- Abstract Strategy ---
class SortStrategy {
public:
        virtual ~SortStrategy() = default;
        virtual void sort(std::vector<int>& data) = 0;
};
// --- Concrete Strategy A: Quick Sort ---
class QuickSort : public SortStrategy {
public:
        void sort(std::vector<int>& data) override {
                std::cout << "QuickSort algoritmasi kullaniliyor..." << std::endl;
                // Gerçek QuickSort yerine std::sort kullanalım
                // (genelde introsort/quicksort türevidir)
                std::sort(data.begin(), data.end());
        }
};

// --- Concrete Strategy B: Merge Sort ---
class MergeSort : public SortStrategy {
public:
        void sort(std::vector<int>& data) override {
                std::cout << "MergeSort algoritmasi kullaniliyor..." << std::endl;
                // Örnek amaçlı standart stable_sort (merge sort benzeri garantiler sunar)
                std::stable_sort(data.begin(), data.end());
        }
};

// --- Context: Sorter ---
class Sorter {
private:
        std::unique_ptr<SortStrategy> strategy_;
public:
        // Stratejiyi çalışma zamanında değiştirmemizi sağlar
        void setStrategy(std::unique_ptr<SortStrategy> strategy) {
                strategy_ = std::move(strategy);
        }
        void sort(std::vector<int>& data) {
                if (!strategy_) {
                        std::cout << "Hata: Once bir strateji belirlenmelidir!" << std::endl;
                        return;
                }
                strategy_->sort(data);
        }
};
// Yardımcı fonksiyon: Verileri ekrana basmak için
void printVector(const std::vector<int>& data) {
        for (int n : data)
                std::cout << n << " ";
```

```
        std::cout << std::endl;
}
int main() {
        std::vector<int> sayilar = {34, 12, 5, 78, 1, 45};
        Sorter siralayici;
        std::cout << "Orijinal liste: ";
        printVector(sayilar);
        // 1. QuickSort Stratejisini Seçelim
        siralayici.setStrategy(std::make_unique<QuickSort>());
        siralayici.sort(sayilar);
        printVector(sayilar);
        // Listeyi tekrar karıştıralım
        sayilar = {99, 2, 56, 11, 0, 7};
        std::cout << "\nYeni liste: ";
        printVector(sayilar);
        // 2. MergeSort Stratejisine Geçelim (Çalışma zamanında değişim)
        siralayici.setStrategy(std::make_unique<MergeSort>());
        siralayici.sort(sayilar);
        printVector(sayilar);
}
/* Programın Çıktısı:
Orijinal liste: 34 12 5 78 1 45
QuickSort algoritmasi kullaniliyor...
1 5 12 34 45 78

Yeni liste: 99 2 56 11 0 7
MergeSort algoritmasi kullaniliyor...
0 2 7 11 56 99
*/
```

§17.4.6 Şablon Yöntem

Şablon yöntem deseninde (template method pattern), bir algoritmanın çerçevesini belirler ve uygulayıcı sınıfların gerçek davranışı tanımlamasına olanak tanır.



Şekil 17.18: Şablon Yöntem Deseni UML Diyagramı

Bu desen, nesne yönelimli programlamada "algoritma iskeleti" oluşturmak için kullanılır. Bir işin adımlarını (sırasını) belirler ancak bu adımların bazılarının nasıl yapılacağını alt sınıflara bırakır. Kod tekrarını önlemek ve bir işlemin temel yapısını korumak için mükemmeldir.

- Soyut Sınıf (`AbstractClass`), algoritmanın ana iskeletini ve Şablon yöntemi (`TemplateMethod()`) barındırır. Değişmez adımları kendi içinde tanımlar, değişebilecek adımları ise soyut (virtual) olarak işaretler. Şablon yöntem genellikle `final` olarak tanımlanır ki algoritmanın sırası bozulmasın.
- Somut Sınıf (`ConcreteClass`); Soyut sınıftaki "boşlukları" doldurur. Algoritmanın değişken adımlarını kendi ihtiyacına göre özelleştirir. Algoritmanın genel akışına müdahale edemez, sadece kendisine izin verilen adımları yazar.

Şimdi desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir.
#include <iostream>
#include <memory> // std::unique_ptr için

//1. Soyut Sınıf (Abstract Class)
```

```
class AbstractClass {
public:
        // Kritik: Polimorfik sınıflarda sanal yıkıcı şarttır.
        virtual ~AbstractClass() = default;
        // Şablon Yöntem: Algoritmanın iskeletini sabitler.
        // Alt sınıflar bu sırayı değiştiremez.
        void templateMethod() {
                baseOperation();          // Ortak adım
                requiredOperation1();     // Özelleştirilebilir adım
                hook();                   // İsteğe bağlı adım (Hook)
                requiredOperation2();     // Özelleştirilebilir adım
        }
protected:
        // Temel operasyon: Tüm alt sınıflar için aynıdır.
        void baseOperation() const {
                std::cout << "Sistem: Algoritmanin ortak temel adimi calistiriliyor.\n";
        }
        // Alt sınıfların mutlaka uygulaması gereken adımlar (Saf sanal)
        virtual void requiredOperation1() = 0;
        virtual void requiredOperation2() = 0;
        // Hook (Kanca): Alt sınıflar isterse ezer, istemezse boş kalır.
        virtual void hook() {}
};

//2. Somut Sınıf (Concrete Class)
class ConcreteClass : public AbstractClass {
        protected:
        // override anahtar kelimesi okunabilirliği ve güvenliği artırır.
        void requiredOperation1() override {
                std::cout << "ConcreteClass: Adim 1 ozel olarak uygulandi.\n";
        }
        void requiredOperation2() override {
                std::cout << "ConcreteClass: Adim 2 ozel olarak uygulandi.\n";
        }
        // İsteğe bağlı: Hook'u ezebiliriz.
        void hook() override {
                std::cout << "ConcreteClass: Hook (Kanca) devreye girdi.\n";
        }
};

//3. Client (İstemci)
int main() {
        // Akıllı işaretçi kullanarak bellek yönetimini otomatiğe bağlıyoruz.
        std::unique_ptr<AbstractClass> algoritma = std::make_unique<ConcreteClass>();
        std::cout << "--- Algoritma Baslatiliyor ---\n";
        algoritma->templateMethod();
        // Kapsam dışına çıkınca bellek otomatik temizlenir.
}
/*Program Çıktısı:
--- Algoritma Baslatiliyor ---
Sistem: Algoritmanin ortak temel adimi calistiriliyor.
ConcreteClass: Adim 1 ozel olarak uygulandi.
ConcreteClass: Hook (Kanca) devreye girdi.
ConcreteClass: Adim 2 ozel olarak uygulandi.
*/
```

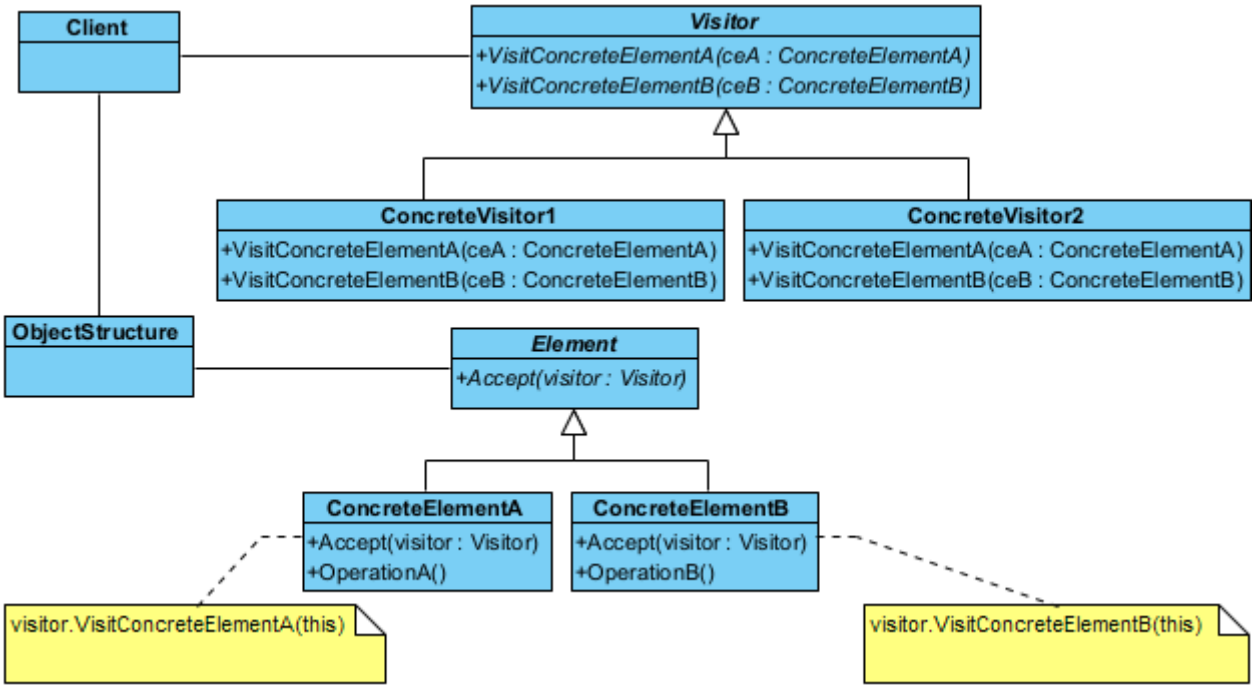
Bu desende, kullanılacak algoritmalardan hangilerinin standart, hangilerinin somut sınıflara özgü olacağı belirlenmelidir;

- İçinde "Bizi çağırmayın! Biz sizi çağıracağız." sloganını taşıyan yöntemler olan bir soyut bir taban sınıf tasarlanmalıdır.
- Taban sınıf içinde algoritmanın kabuğunu oluşturacak standart adımları içeren şablon yöntem tanımlanır.

- Somut sınıflarda detaylandırılacak yöntemlere ilişkin sanal yöntemler taban sınıfta tanımlanır.
- Şablon yöntem içinde algoritmayı oluşturan yöntemler çağrılır.
- Şablon yöntem içinde yer almayan tüm detay kodlamalar somut sınıflar için de yapılır.

§17.4.7 Ziyaretçi

Ziyaretçi deseni (visitor pattern), bir nesne grubunun (koleksiyonun) yapısını değiştirmeden, bu nesneler üzerinde uygulanacak yeni işlemleri dışarıdan eklememizi sağlayan davranışsal bir desendir. Bu desen, "Veri Yapısı" ile "Algoritma"yı birbirinden ayırır. Eğer elinizde sabit bir sınıf hiyerarşisi varsa (örneğin bir dokümandaki paragraflar, resimler, tablolar) ve sürekli bu sınıflar üzerinde yeni işlemler (PDF'e dönüştür, HTML'e dönüştür, kelime say) tanımlamanız gerekiyorsa, her seferinde o sınıfların içine girip kod yazmak yerine ziyaretçi desenini kullanırsınız.



Şekil 17.19: Ziyaretçi Deseni UML Diyagramı

- Ziyaretçi (`Visitor`), koleksiyondaki her bir somut nesne türü için bir ziyaret yöntemi (`visit()`) tanımlayan soyut bir sınıftır. "Eğer bir Paragraf nesnesine gidersem ne yapacağım?", "Eğer bir Resim nesnesine gidersem ne yapacağım?" sorularının imzasını belirler.
- Somut Ziyaretçi (`ConcreteVisitor`), gerçek operasyonu gerçekleştirir. Algoritmanın kendisidir. Örneğin ExportToPDFVisitor sınıfı, her nesne türünün PDF'e nasıl dönüştürüleceğini bilir.
- Eleman (`Element`), bir ziyaretçiyi kabul edebilmek için standart bir "kapı" açan soyut bir sınıftır. Ziyaretçi kabul etmek için `accept(Visitor* v)` yöntemini barındırır.
- Somut Elemanlar (`ConcreteElement`), veri yapısının asıl parçalarıdır. `accept()` yöntemi çağrıldığında, ziyaretçiye "Ben bir [NesneTürü]'yüm, beni ziyaret et" der (`v->visit(this)`). Buna yazılım dünyasında **çifte sevkiyat** (double dispatch) denir.

Şimdi desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <vector>
#include <memory>
#include <string>

// 1. İleriye Dönük Bildirimler (Forward Declarations): Böyle sınıflar olacak!
class ConcreteElementA;
class ConcreteElementB;

// 2. Visitor (Ziyaretçi) Arayüzü
class Visitor {
public:
        virtual ~Visitor() = default; // Kritik: Sanal yıkıcı
```

```
        virtual void visit(ConcreteElementA* element) = 0;
        virtual void visit(ConcreteElementB* element) = 0;
};

// 3. Element (Eleman) Arayüzü
class Element {
public:
        virtual ~Element() = default;
        // "Double Dispatch" mekanizmasının kalbi: accept metodu
        virtual void accept(Visitor* visitor) = 0;
};

// 4. Somut Elemanlar
class ConcreteElementA : public Element {
public:
        void accept(Visitor* visitor) override {
                visitor->visit(this);
        }
        void operationA() const {
                std::cout << "ConcreteElementA: Ozel islem calistiriliyor.\n";
        }
};
class ConcreteElementB : public Element {
public:
        void accept(Visitor* visitor) override {
                visitor->visit(this);
        }
        void operationB() const {
                std::cout << "ConcreteElementB: Ozel islem calistiriliyor.\n";
        }
};

// 5. Somut Ziyaretçiler
class ConcreteVisitor1 : public Visitor {
public:
        void visit(ConcreteElementA* element) override {
                std::cout << "Visitor 1: ElementA ziyaret edildi.\n";
                element->operationA();
        }
        void visit(ConcreteElementB* element) override {
                std::cout << "Visitor 1: ElementB ziyaret edildi.\n";
                element->operationB();
        }
};
class ConcreteVisitor2 : public Visitor {
public:
        void visit(ConcreteElementA* element) override {
                std::cout << "Visitor 2: ElementA üzerinde farkli bir islem yapiliyor.\n";
        }
        void visit(ConcreteElementB* element) override {
                std::cout << "Visitor 2: ElementB üzerinde farkli bir islem yapiliyor.\n";
        }
};

// 6. Nesne Yapısı (Object Structure)
class ObjectStructure {
        private:
        std::vector<std::unique_ptr<Element>> elements;
        public:
        void add(std::unique_ptr<Element> e) {
                elements.push_back(std::move(e));
        }
```

```
        void accept(Visitor* v) {
                for (const auto& e : elements) {
                        e->accept(v);
                }
        }
};

// 7. Client (İstemci)
int main() {
        ObjectStructure structure;
        // Elemanları ekliyoruz (Sahiplik structure nesnesine geçiyor)
        structure.add(std::make_unique<ConcreteElementA>());
        structure.add(std::make_unique<ConcreteElementB>());
        // Ziyaretçileri oluşturuyoruz
        auto v1 = std::make_unique<ConcreteVisitor1>();
        auto v2 = std::make_unique<ConcreteVisitor2>();
        std::cout << "--- Ziyaretci 1 Basliyor ---\n";
        structure.accept(v1.get());
        std::cout << "--- Ziyaretci 2 Basliyor ---\n";
        structure.accept(v2.get());
        // Her şey otomatik ve güvenli bir şekilde silinir.
}
/*Program Çıktısı:
--- Ziyaretci 1 Basliyor ---
Visitor 1: ElementA ziyaret edildi.
ConcreteElementA: Ozel islem calistiriliyor.
Visitor 1: ElementB ziyaret edildi.
ConcreteElementB: Ozel islem calistiriliyor.
--- Ziyaretci 2 Basliyor ---
Visitor 2: ElementA üzerinde farkli bir islem yapiliyor.
Visitor 2: ElementB üzerinde farkli bir islem yapiliyor.
*/
```

Bu desen aşağıdaki durumlarda kullanılır;

- Nesneler birden fazla olup birçok ara yüze sahip olduğunda, bu nesnelerin somut sınıfları ile bir işlem yapmak gerektiğinde,
- Sınıflar üzerinde yapılacak işlemlerin, sınıflar üzerinde bir iz bırakmamasını sağlamak amacıyla,
- Nadir değişen sınıf yapılarına, sınıf yapısını değiştirmeden, yeni işlemler eklemek gerektiğinde.

Bu deseni kullanmanın aşağıda sıralanan üstünlükleri sağlar;

- Yeni işlemleri (operation) sisteme eklemeyi kolaylaştırır.
- Birbiriyle ilişkili olan işlemleri ilişkili olmayandan ayırmaya yardımcı olur.
- Yeni `ConcreteElement` nesnelerinin eklenmesini zorlaştırır.
- Bütün nesne hiyerarşisi baştanbaşa ziyaret edilebilir. Sınıflardaki tüm durumlar (state) biriktirilebilir.
- Bazı durumlarda kapsülleme (encapsulation) işlemini engeller. Yani nesnenin iç durumlarından hareketle herkese açık işlemler tanımlanabilir.

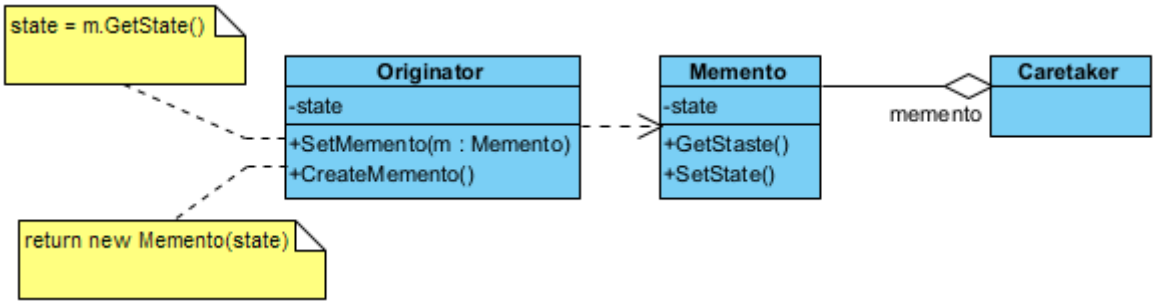
§17.4.8 Hatıra

Hatıra deseni (memento pattern), nesnelerin bir andaki durumlarını (state) saklayıp bir başka zaman da bu duruma geri dönmek gerektiğinde kullanılır. Bir nesnenin iç durumunun yakalanmasına ve dışarıda saklanmasına olanak tanır, böylece daha sonra geri yüklenebilir fakat tüm bunlar **kılıflamayı** (encapsulation) ihlal etmeden yapılır.

Hatıra deseninin amacı, nesnenin iç durumunu (state) bozmadan saklamak ve daha sonra geri yüklemektir.

- Yaratıcı (`Originator`), durumu değişen asıl nesnedir. Kendi "anlık görüntüsünü" (snapshot) oluşturur ve geri yükler.
- Hatıra (`Memento`), sadece veriyi tutan pasif nesnedir. Dışarıdan değiştirilemez; sadece `Originator` içeriğini okuyabilir.
- Bakıcı (`Caretaker`), hatıraları güvenli bir yerde (liste veya yığın) saklar. İçeriği asla değiştirmez, sadece saklar ve geri verir.

Şimdi desenimizi kodlayalım;



Şekil 17.20: Hatıra Deseni UML Diyagramı

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>

//1. Memento: Durumu saklayan pasif nesne
class Memento {
private:
        // Sadece Originator bu veriye erişmeli (Encapsulation)
        friend class Originator;
        int state;
        explicit Memento(int pState) : state(pState) {}
        int getState() const { return state; }
};

// 2. Originator: Durumu değişen ve "hatıra" üreten asıl nesne
class Originator {
private:
        int state;
public:
        void setState(int pState) {
                state = pState;
                std::cout << "Originator: Durum degistirildi -> " << state << std::endl;
        }
        // Mevcut durumu bir Memento nesnesine paketler
        std::unique_ptr<Memento> save() {
                std::cout << "Originator: Durum yedekleniyor... (" << state << ")" << std::endl;
                return std::unique_ptr<Memento>(new Memento(state));
        }
        // Bir Memento nesnesinden durumu geri yukler
        void restore(const Memento* memento) {
                if (memento) {
                        state = memento->getState();
                        std::cout << "Originator: Durum geri yuklendi -> " << state << std::endl;
                }
        }
};

// 3. Caretaker: Hatıraları saklayan bekçi
class Caretaker {
private:
        std::vector<std::unique_ptr<Memento>> history;
        Originator& originator;
public:
        explicit Caretaker(Originator& o) : originator(o) {}
        void backup() {
                history.push_back(originator.save());
        }
        void undo() {
```

```
                if (history.empty()) return;
                auto memento = std::move(history.back());
                history.pop_back();
                originator.restore(memento.get());
        }
};

// 4. İstemci (Client)
int main() {
        Originator player;
        Caretaker saveManager(player);
        player.setState(100); // Bölüm 1
        saveManager.backup(); // Kayıt noktası
        player.setState(50);  // Can azaldı
        player.setState(0);   // Öldü!
        std::cout << "--- Geri Yukleniyor ---" << std::endl;
        saveManager.undo();   // Bölüm 1'e geri dön
}
/*Program Çıktısı:
Originator: Durum degistirildi -> 100
Originator: Durum yedekleniyor... (100)
Originator: Durum degistirildi -> 50
Originator: Durum degistirildi -> 0
--- Geri Yukleniyor ---
Originator: Durum geri yuklendi -> 100
*/
```

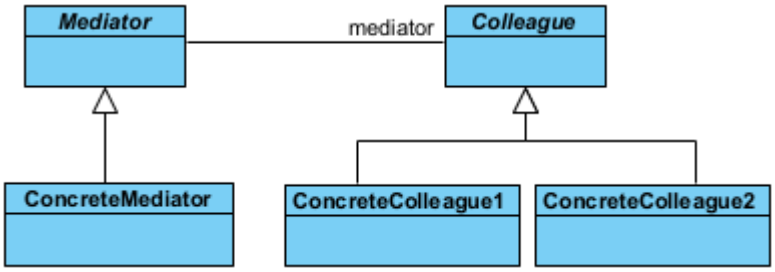
Aşağıdaki örnekte bir metin editörünün geçmiş özelliği modellenmiştir;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>
#include <vector>
#include <memory>

// --- Memento ---
// Sadece saklanacak veriyi tutar.
class EditorMemento {
private:
        std::string content;
public:
        EditorMemento(std::string text) : content(text) {}
        std::string getSavedContent() const { return content; }
};

// --- Originator ---
// Durumu kaydedilecek olan asıl nesne.
class TextEditor {
private:
        std::string text;
public:
        void type(std::string newText) { text += newText; }
        std::string getText() const { return text; }
        // Mevcut durumu bir Memento içine paketler.
        std::unique_ptr<EditorMemento> save() {
                return std::make_unique<EditorMemento>(text);
        }
        // Bir Memento'dan durumu geri yükler.
        void restore(const EditorMemento& memento) {
                text = memento.getSavedContent();
        }
};
```

```
// --- Caretaker ---
// Geçmişi (Memento listesini) yönetir.
class History {
private:
        std::vector<std::unique_ptr<EditorMemento>> mementos;
public:
        void push(std::unique_ptr<EditorMemento> m) {
                mementos.push_back(std::move(m));
        }
        std::unique_ptr<EditorMemento> pop() {
                if (mementos.empty()) return nullptr;
                auto m = std::move(mementos.back());
                mementos.pop_back();
                return m;
        }
};

int main() {
        TextEditor editor;
        History history;
        // 1. Yazmaya başla ve ilk durumu kaydet
        editor.type("Merhaba ");
        history.push(editor.save());
        std::cout << "Guncel Metin: " << editor.getText() << std::endl;
        // 2. Yazmaya devam et ve ikinci durumu kaydet
        editor.type("Dunya!");
        history.push(editor.save());
        std::cout << "Guncel Metin: " << editor.getText() << std::endl;
        // 3. Bir hata yapalım
        editor.type(" Yanlis bir ekleme.");
        std::cout << "Hata Sonrasi: " << editor.getText() << std::endl;
        // 4. Geri al (Undo)
        auto undo = history.pop(); // Son kaydedilen "Merhaba Dunya!" idi, onu alalım.
        auto redo = history.pop(); // Bir önceki "Merhaba " idi.
        if (redo) {
                editor.restore(*redo);
                std::cout << "Geri Alindi: " << editor.getText() << std::endl;
        }
}
/*Program Çıktısı:
Guncel Metin: Merhaba
Guncel Metin: Merhaba Dunya!
Hata Sonrasi: Merhaba Dunya! Yanlis bir ekleme.
Geri Alindi: Merhaba
*/
```

§17.4.9 Ara Bulucu

Ara Bulucu deseni (mediator pattern), sınıfların arasındaki iletişimi basitleştirir. Bir iletişim nesnesi tanımlanır, diğer nesneler bu nesne üzerinden birbirleriyle haberleşir. Diğer nesnelerin birbiriyle doğrudan iletişim kurmasına izin verilmez. Amaç, iletişimin kontrol altına alınmasıdır. Bu tasarım deseni, bir dizi nesne



Şekil 17.21: Ara Bulucu Deseni UML Diyagramı

arasındaki karmaşık iletişim ağını basitleştirmek için kullanılır. Nesnelerin birbirleriyle doğrudan konuşmasını engeller ve tüm iletişimi tek bir "merkezi aracı" üzerinden yürütmelerini sağlar. Bu desen, özellikle nesne sayısı arttığında ortaya çıkan "örümcek ağı" gibi karmaşık bağlantıları (tight coupling) çözmek için birebirdir.

- Soyut Aracı (`Mediator`), nesneler arasındaki iletişim protokolünü (arayüzünü) tanımlar. "Hangi olay gerçekleştiğinde kimlere haber verilecek?" sorusunun genel çerçevesini çizer.
- Somut Aracı (`ConcreteMediator`), iletişim trafiğini yöneten asıl "santral" binasıdır. tüm meslektaşları (`Colleague`) tanır ve aralarındaki koordinasyonu sağlar. Bir bileşenden mesaj geldiğinde, mantık çerçevesinde diğerlerine iletir.
- Meslektaş (`Colleague`) Sınıflar, kendi işlevini yerine getiren bağımsız sınıflardır. Diğer sınıfların varlığından haberdar değildirler. Sadece "Aracı"ya (`Mediator`) haber verirler. "Ben düğmeye bastım, gerisini aracı halletsin" mantığıyla çalışırlar.

Şimdi desenimizi kodlayalım;

```
#include <iostream>
#include <vector>
#include <string>
#include <memory>
#include <algorithm>

// 1. İleriye Dönük Bildirim: İleride böyle bir sınıf olacak!
class Colleague;

// 2. Mediator (Ara bulucu) Arayüzü
class Mediator {
public:
        virtual ~Mediator() = default;
        virtual void registerColleague(std::shared_ptr<Colleague> colleague) = 0;
        virtual void relayMessage(const std::string& message, Colleague* sender) = 0;
};

// 3. Colleague (Meslektaş) Taban Sınıfı
class Colleague {
protected:
        std::string name;
        Mediator* mediator; // Döngüsel bağımlılığı önlemek için ham işaretçi veya weak_ptr
public:
        Colleague(std::string pName, Mediator* pMediator) : name(std::move(pName)),
        ↪  mediator(pMediator) {}
        virtual ~Colleague() = default;
        std::string getName() const { return name; }
        void send(const std::string& message) {
                mediator->relayMessage(message, this);
        }
        void receive(const std::string& message, const std::string& senderName) {
                std::cout << "[" << name << "] " << senderName
                << " kisisinden mesaj aldi: " << message << std::endl;
        }
};

// 4. Somut Ara bulucu (Concrete Mediator)
class ChatRoom : public Mediator {
private:
        std::vector<std::shared_ptr<Colleague>> colleagues;
public:
        void registerColleague(std::shared_ptr<Colleague> colleague) override {
                colleagues.push_back(colleague);
        }
        void relayMessage(const std::string& message, Colleague* sender) override {
                std::cout << "--- LOG: " << sender->getName() << " bir mesaj yayinladi ---" <<
                ↪  std::endl;
```

```
                for (const auto& colleague : colleagues) {
                        // Mesajı gönderen kişiye geri göndermiyoruz
                        if (colleague.get() != sender) {
                                colleague->receive(message, sender->getName());
                        }
                }
        }
};

// 5. Somut Meslektaş (Concrete Colleague)
class User : public Colleague {
public:
        using Colleague::Colleague; // Yapıcıyı miras al
};

// 6. İstemci (Client)
int main() {
        // Arabulucu nesnesi
        auto oda = std::make_shared<ChatRoom>();
        // Meslektaşlar (Kullanıcılar)
        auto ilhan = std::make_shared<User>("Ilhan", oda.get());
        auto mehmet = std::make_shared<User>("Mehmet", oda.get());
        auto ayse = std::make_shared<User>("Ayse", oda.get());
        // Kayıt işlemleri
        oda->registerColleague(ilhan);
        oda->registerColleague(mehmet);
        oda->registerColleague(ayse);
        // İletişim başlıyor
        ilhan->send("Selam millet!");
        mehmet->send("Merhabalar Ilhan.");
        ayse->send("Nasılsınız?");
        // Her şey otomatik temizlenir.
}
/*Program Çıktısı:
--- LOG: Ilhan bir mesaj yayinladi ---
[Mehmet] Ilhan kisisinden mesaj aldi: Selam millet!
[Ayse] Ilhan kisisinden mesaj aldi: Selam millet!
--- LOG: Mehmet bir mesaj yayinladi ---
[Ilhan] Mehmet kisisinden mesaj aldi: Merhabalar Ilhan.
[Ayse] Mehmet kisisinden mesaj aldi: Merhabalar Ilhan.
--- LOG: Ayse bir mesaj yayinladi ---
[Ilhan] Ayse kisisinden mesaj aldi: Nasılsınız?
[Mehmet] Ayse kisisinden mesaj aldi: Nasılsınız?
*/
```

Bu desende, sistemde yalnızca bir adet ara bulucu olacaksa soyut `Mediator` nesnesi tanımlanmayabilir. Ayrıca ara bulucu ile meslektaşlar arasındaki iletişim iki türlü yapılır;

- Birisinde bu iletişim gözlemci deseni (observer pattern) ile sağlanır. `Colleague` sınıfı gözlemci desenindeki bağlam `Subject` sınıfı gibi davranır.
- Diğerinde ise `Mediator` nesnesine, görüşecekleri meslektaşları belirterek doğrudan ileti gönderirler.

Aşağıda havaalanındaki kule ile uçaklar arasındaki iletişimi modelleyen bir örnek verilmiştir;

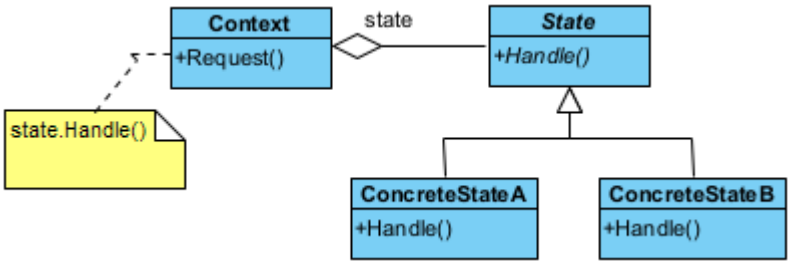
```
#include <iostream>
#include <string>
#include <vector>
#include <memory>

// Forward declaration
class Airplane;

// --- Mediator Arayüzü ---
class IAirTrafficControl {
```

```
public:
        virtual ~IAirTrafficControl() = default;
        virtual void sendMessage(const std::string& message, Airplane* originator) = 0;
        virtual void registerAirplane(Airplane* airplane) = 0;
};

// --- Colleague (Meslektaş/Bileşen) ---
class Airplane {
protected:
        IAirTrafficControl* mediator_;
        std::string callSign_;
public:
        Airplane(IAirTrafficControl* mediator, std::string callSign) : mediator_(mediator),
        ↪  callSign_(callSign) {}
        virtual ~Airplane() = default;
        virtual void send(const std::string& message) {
                std::cout << "[" << callSign_ << "] Kuleye mesaj gonderiyor: " << message <<
                ↪  std::endl;
                mediator_->sendMessage(message, this);
        }
        virtual void receive(const std::string& message) {
                std::cout << "[" << callSign_ << "] Mesaj aldi: " << message << std::endl;
        }
        std::string getCallSign() const { return callSign_; }
};

// --- Concrete Mediator (Somut Aracı) ---
class AtcTower : public IAirTrafficControl {
private:
        std::vector<Airplane*> airplanes_;
public:
        void registerAirplane(Airplane* airplane) override {
                airplanes_.push_back(airplane);
        }
        void sendMessage(const std::string& message, Airplane* originator) override {
                for (auto airplane : airplanes_) {
                        // Mesajı gönderen uçak hariç herkese ilet (veya kurala göre işle)
                        if (airplane != originator) {
                                airplane->receive("Kule'den " + originator->getCallSign() + "
                                ↪  bilgisi: " + message);
                        }
                }
        }
};

int main() {
        // 1. Aracı (Kule) oluşturulur
        auto tower = std::make_unique<AtcTower>();
        // 2. Meslektaşlar (Uçaklar) oluşturulur ve kuleye bildirilir
        auto boeing = std::make_unique<Airplane>(tower.get(), "THY-123");
        auto airbus = std::make_unique<Airplane>(tower.get(), "PGS-456");
        auto cessna = std::make_unique<Airplane>(tower.get(), "OZEL-789");
        tower->registerAirplane(boeing.get());
        tower->registerAirplane(airbus.get());
        tower->registerAirplane(cessna.get());
        // 3. İletişim merkezi üzerinden yürütülür
        boeing->send("Pist 1'e inis izni istiyorum.");
        std::cout << "-----------------------------------------" << std::endl;
        cessna->send("Pistten cikis yapildi.");
}
/* Program Çıktısı:
[THY-123] Kuleye mesaj gonderiyor: Pist 1'e inis izni istiyorum.
```

```
[PGS-456] Mesaj aldi: Kule'den THY-123 bilgisi: Pist 1'e inis izni istiyorum.
[OZEL-789] Mesaj aldi: Kule'den THY-123 bilgisi: Pist 1'e inis izni istiyorum.
-----------------------------------------
[OZEL-789] Kuleye mesaj gonderiyor: Pistten cikis yapildi.
[THY-123] Mesaj aldi: Kule'den OZEL-789 bilgisi: Pistten cikis yapildi.
[PGS-456] Mesaj aldi: Kule'den OZEL-789 bilgisi: Pistten cikis yapildi.
*/
```

§17.4.10 Durum

Durum deseni (state pattern), nesnenin durumunu davranışına bağlar, nesnenin içsel durumuna göre farklı şekillerde davranmasına olanak tanır.



Şekil 17.22: Durum Deseni UML Diyagramı

Bu desenin amacı, nesnenin iç durumu değiştiğinde davranışını da değiştirmek. Nesne sanki sınıfını değiştiriyormuş gibi görünür.

- Bağlam (`Context`), istemcinin (`Client`) etkileşime girdiği ana sınıftır. Mevcut durum nesnesini bir referans olarak tutar.
- Soyut Durum (`State`), tüm somut durumların uyması gereken arayüzü (Interface) tanımlar.
- Somut Durumlar (`ConcreteState`), Her sınıf, belirli bir durumdaki davranışı temsil eder (Örn: Oynatılıyor, Duraklatıldı). Bir durumdan diğerine geçiş mantığını genellikle bu sınıflar yönetir.

Şimdi desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <memory>

// 1. State Arayüzü (Soyut Durum)
class Context; // Forward declaration
class State {
public:
        virtual ~State() = default;
        virtual void handle(Context* context) = 0;
};

// 2. Context (Bağlam)
class Context {
private:
        std::unique_ptr<State> currentState;
public:
        explicit Context(std::unique_ptr<State> state) : currentState(std::move(state)) {}
        void transitionTo(std::unique_ptr<State> state) {
                std::cout << "Context: Durum degistiriliyor...\n";
                currentState = std::move(state);
        }
        void request() {
                currentState->handle(this);
        }
};

// 3. Somut Durumlar
class ConcreteStateB : public State {
public:
```

```
        void handle(Context* context) override; // İleride tanımlanacak
};
class ConcreteStateA : public State {
public:
        void handle(Context* context) override {
                std::cout << "ConcreteStateA: Talep isleniyor. B durumuna gecis tetikleniyor.\n";
                context->transitionTo(std::make_unique<ConcreteStateB>());
        }
};
void ConcreteStateB::handle(Context* context) {
        std::cout << "ConcreteStateB: Talep isleniyor. A durumuna geri donuluyor.\n";
        context->transitionTo(std::make_unique<ConcreteStateA>());
}

// 4. İstemci (Client)
int main() {
        // Başlangıç durumu A ile Context oluşturuluyor
        auto context = std::make_unique<Context>(std::make_unique<ConcreteStateA>());
        // İstekler gönderildikçe durumlar kendi içlerinde birbirini tetikler
        context->request(); // A -> B geçişi
        context->request(); // B -> A geçişi
        context->request(); // A -> B geçişi
        // Manuel delete gerekmez.
}
/*Program Çıktısı:
ConcreteStateA: Talep isleniyor. B durumuna gecis tetikleniyor.
Context: Durum degistiriliyor...
ConcreteStateB: Talep isleniyor. A durumuna geri donuluyor.
Context: Durum degistiriliyor...
ConcreteStateA: Talep isleniyor. B durumuna gecis tetikleniyor.
Context: Durum degistiriliyor...
*/
```

Bu desenin bağlam (`Context`) nesnesinin, diğer tasarım desenleri ile iç içe kullanılması halinde, bu desen, **sıfır desen** (null pattern) olarak adlandırılır. Ayrıca bu desenle ilgili olarak iki önemli şeyden bahsetmek gerekir:

- Sahip olunan duruma göre icra edilecek davranış, tanımlanan `State` sınıfına bağlıdır. Yani ilgili davranış somut sınıflardan biri tarafından icra edilir. Bu nedenle somut state (`ConcreteState`) sınıfları içinde tanımlanan yöntemler, durumu kullanan bağlam (`Context`) sınıf içinde tanımlanmak zorundadır.
- Durumu kullanan bağlam (`Context`) sınıf içinde hangi duruma geçildiğinin anlaşılması için ilgili duruma ilişkin tanımlanan özellik (property) içinde `set` yöntemi kullanılmak zorundadır.

Bu desen netice olarak, durum geçişlerini (state transition) açığa çıkarır. Bu deseni kullanırken aşağıda verilen durumlar özellikle irdelenmelidir;

- **Durum geçişleri kim tarafından tanımlanmalıdır?**: Bu desen, hangi şartlarda durumların ayrıştırılacağını belirtmez. Eğer şartlar belirgin ise bu işlem bağlam (`Context`) nesnesi tarafından yapılır. Bu şartlar belirgin değil ve karmaşık ise bu işlem `State` nesnesi ve halefi (successor) tarafından yapılır.
- **Tablo tabanlı alternatif**: Bu durumda durum geçişleri bir tablo aracılığı ile yapılır. Tablo üzerine sağlanacak her bir durum yazılır. Bu durumda, durum geçişlerini sağlayacak değişiklikler yeniden kodlama gerektirmez. Ancak bu yöntemde hız düşer. Durum geçişleri tekdüze (uniform) tanımlanmak zorundadır.
- **`State` nesnelerini yaratma ve yok etme**: Bu nesneler çalıştırma anında yaratılıp yok edildiklerinden, bu süre zarfında yapılacak işlemler için bir çözüm bulunmalıdır. İki yaklaşım vardır. Birincisi, State nesnesini ihtiyaç olduğunda yaratmak kullanmak ve öldürmektir. İkincisi ise bu nesneleri baştan yaratıp hiç öldürmemektir. Genellikle birinci yöntem tercih edilir.
- **Dinamik kalıtım (dynamic inheritance) kullanma**: Bazı durumlarda nesne, ait olduğu sınıfı çalıştırma anında değiştirir. Bu durumda talep edilen isteğin yerine getirilmesi tam olarak başarılamaz. Ancak birçok nesne yönelimli programlama dili bunu desteklemez.

Bu desen için en gerçekçi örneklerden biri **otomat makineleridir** (vending machine). Makinenin davranışı; içinde para olup olmamasına, ürünün kalıp kalmamasına veya ürün seçilip seçilmediğine göre tamamen değişir. Eğer para yoksa "Ürün Seç" düğmesi çalışmaz; para varsa çalışır;

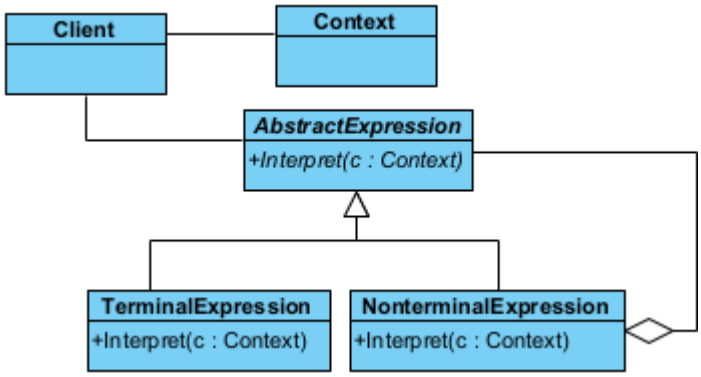
```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <memory>
#include <string>

// --- Forward Declarations ---
class VendingMachine;

// --- Abstract State ---
class State {
public:
        virtual ~State() = default;
        virtual void insertMoney() = 0;
        virtual void ejectMoney() = 0;
        virtual void selectProduct() = 0;
        virtual void dispense() = 0;
};

// --- Concrete States ---
// 1. Para Yok Durumu
class NoMoneyState : public State {
        VendingMachine& machine;
public:
        NoMoneyState(VendingMachine& m) : machine(m) {}
        void insertMoney() override;
        void ejectMoney() override { std::cout << "Hata: Zaten para yok.\n"; }
        void selectProduct() override { std::cout << "Hata: Once para yuklemelisiniz.\n"; }
        void dispense() override { std::cout << "Hata: Odeme yapilmadan urun verilmez.\n"; }
};

// 2. Para Var Durumu
class HasMoneyState : public State {
        VendingMachine& machine;
public:
        HasMoneyState(VendingMachine& m) : machine(m) {}
        void insertMoney() override { std::cout << "Bilgi: Zaten para yuklu.\n"; }
        void ejectMoney() override;
        void selectProduct() override;
        void dispense() override { std::cout << "Bilgi: Once urun secimi yapmalisiniz.\n"; }
};

// 3. Ürün Satıldı/Teslimat Durumu
class SoldState : public State {
        VendingMachine& machine;
public:
        SoldState(VendingMachine& m) : machine(m) {}
        void insertMoney() override { std::cout << "Lutfen bekleyin: Urununuz hazirlaniyor.\n"; }
        void ejectMoney() override { std::cout << "Hata: Islem geri alinamaz, urun veriliyor.\n";
        ↪  }
        void selectProduct() override { std::cout << "Hata: Islem zaten devam ediyor.\n"; }
        void dispense() override;
};

// 4. Stok Yok Durumu
class SoldOutState : public State {
public:
        void insertMoney() override { std::cout << "Hata: Stok bitti, para yukleyemezsiniz.\n"; }
        void ejectMoney() override { std::cout << "Hata: Para yok.\n"; }
        void selectProduct() override { std::cout << "Hata: Stok bitti.\n"; }
        void dispense() override { std::cout << "Hata: Stok yok.\n"; }
};
```

```
// --- Context: VendingMachine ---
class VendingMachine {
private:
        std::unique_ptr<State> currentState;
        int count = 0;
public:
        VendingMachine(int productCount) : count(productCount) {
                if (count > 0)
                currentState = std::make_unique<NoMoneyState>(*this);
                else
                currentState = std::make_unique<SoldOutState>();
        }
        void setState(std::unique_ptr<State> state) {
                currentState = std::move(state);
        }
        // Proxy metodlar
        void insertMoney() { currentState->insertMoney(); }
        void ejectMoney() { currentState->ejectMoney(); }
        void selectProduct() { currentState->selectProduct(); }
        void dispense() { currentState->dispense(); }
        void releaseProduct() {
                if (count > 0) {
                        std::cout << ">>> URUN VERILDI. Afiyet olsun!\n";
                        count--;
                }
        }
        int getCount() const { return count; }
};

// --- Yöntem Tanımlamaları (Durum Geçiş Mantığı) ---
void NoMoneyState::insertMoney() {
        std::cout << "Para kabul edildi.\n";
        machine.setState(std::make_unique<HasMoneyState>(machine));
}
void HasMoneyState::ejectMoney() {
        std::cout << "Para iade ediliyor...\n";
        machine.setState(std::make_unique<NoMoneyState>(machine));
}
void HasMoneyState::selectProduct() {
        std::cout << "Urun secildi, hazirlaniyor...\n";
        machine.setState(std::make_unique<SoldState>(machine));
        machine.dispense(); // Otomatik teslimat tetiklemesi
}
void SoldState::dispense() {
        machine.releaseProduct();
        if (machine.getCount() > 0) {
                machine.setState(std::make_unique<NoMoneyState>(machine));
        } else {
                std::cout << "Dikkat: Son urun satildi!\n";
                machine.setState(std::make_unique<SoldOutState>());
        }
}

int main() {
        // 2 ürünlük bir otomat oluşturalım
        VendingMachine machine(2);
        std::cout << "--- Senaryo 1: Basarili Alim ---\n";
        machine.insertMoney();
        machine.selectProduct();
        std::cout << "--- Senaryo 2: Para Iadesi ---\n";
        machine.insertMoney();
        machine.ejectMoney();
```

```
        std::cout << "--- Senaryo 3: Son Urunu Alma ---\n";
        machine.insertMoney();
        machine.selectProduct();
        std::cout << "--- Senaryo 4: Stok Bittiginde Deneme ---\n";
        machine.insertMoney();
}
/* Programın Çıktısı:
--- Senaryo 1: Basarili Alim ---
Para kabul edildi.
Urun secildi, hazirlaniyor...
*/
```

§17.4.11 Yorumlayıcı

Yorumlayıcı deseni (interpreter pattern), programlama dili yorumlama özelliğini uygulamalarımıza katmanın yolunu gösterir. Yani bir dil bilgisi için bir temsili ve aynı zamanda dil bilgisini anlayıp ona göre hareket etmeyi sağlayacak bir mekanizmayı tanımlar.



Şekil 17.23: Yorumlayıcı Deseni UML Diyagramı

Bu desenin amacı, belirli bir gramer yapısına sahip ifadeleri (matematiksel işlemler, SQL komutları vb.) çözümlemektir.

- Soyut İfade (`AbstractExpression`), tüm düğümlerin sahip olduğu `interpret()` yöntemini tanımlayan ara yüzdür.
- Uç İfade (`TerminalExpression`), gramerin en küçük birimleridir (Sayılar veya değişkenler). Daha fazla parçalanamazlar.
- Uç Olmayan İfade (`NonTerminalExpression`), gramerin kurallarını (Toplama, Çarpma vb.) temsil eder. Genellikle içinde diğer ifadeleri (uç yada veya uç olmayan) barındırır.
- Bağlam (`Context`), yorumlama sırasında ihtiyaç duyulan küresel bilgileri (Değişken değerleri, sembol tabloları) taşır.

Şimdi desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <map>

// 1. Context (Bağlam): Yorumlama sırasında kullanılan küresel bilgiler (Değişken değerleri vb.)
class Context {
private:
        std::map<std::string, int> variables;
public:
        void setVariable(const std::string& name, int value) { variables[name] = value; }
        int getVariable(const std::string& name) const { return variables.at(name); }
};

// 2. AbstractExpression (Soyut İfade)
class AbstractExpression {
public:
```

```
        virtual ~AbstractExpression() = default;
        virtual int interpret(const Context& context) const = 0;
};

// 3. TerminalExpression (Uç İfade): Sayılar veya değişkenler gibi atomik birimler
class NumberExpression : public AbstractExpression {
private:
        int number;
public:
        explicit NumberExpression(int n) : number(n) {}
        int interpret(const Context&) const override { return number; }
};

class VariableExpression : public AbstractExpression {
private:
        std::string name;
public:
        explicit VariableExpression(std::string n) : name(std::move(n)) {}
        int interpret(const Context& context) const override { return context.getVariable(name);
        ↪ }
};

// 4. NonterminalExpression (Uç Olmayan İfade): Operatörler (+, -, *) gibi karmaşık yapılar
class AddExpression : public AbstractExpression {
private:
        std::unique_ptr<AbstractExpression> left;
        std::unique_ptr<AbstractExpression> right;
public:
        AddExpression(std::unique_ptr<AbstractExpression> l, std::unique_ptr<AbstractExpression>
        ↪ r) : left(std::move(l)), right(std::move(r)) {}
        int interpret(const Context& context) const override {
                return left->interpret(context) + right->interpret(context);
        }
};

// 5. İstemci (Client)
int main() {
        Context context;
        context.setVariable("x", 10);
        context.setVariable("y", 5);
        // İfade: (x + y) + 20
        // Ağaç yapısı oluşturuluyor
        auto expression = std::make_unique<AddExpression>(
        std::make_unique<AddExpression>( std::make_unique<VariableExpression>("x"),
        ↪ std::make_unique<VariableExpression>("y") ), std::make_unique<NumberExpression>(20)
        ↪ );
        std::cout << "Sonuc: " << expression->interpret(context) << std::endl;
        // unique_ptr sayesinde tüm ağaç otomatik temizlenir
}
/*Program Çıktısı:
Sonuc: 35
*/
```

Bu deseni uygularken aşağıda verilen durumlar irdelenmelidir;

- **Soyut söz dizimin oluşturulması**: Bu desen yorumlanacak dile ilişkin soyut bir söz dizimin nasıl olması gerektiğini açıklamaz. Tabloya dayalı, ağaç yapısı şeklinde el becerisi ile veya doğrudan istemci tarafından yapılabilir.
- **Yorumlayacak işlemin tanımlanması**: Eğer işlemler ortak bir yapıdaysa yeni bir `Expression` nesnesi tanımlanarak yapılabilir. Daha karmaşık durumlarda ziyaretçi deseni kullanılır. Programlama dillerinin çoğu soyut bir söz dizimine sahiptir ve yorumlanacak öğeler ayrı bir Visitor nesnesi tarafından yorumlanır.
- **Cümle sonlarını bitiren ortak semboller için sineksıklet deseni kullanılabilir**: Cümle sonlarındaki

semboller genellikle bilgi saklamazlar. Yorumlamaya gerek duyulmazlar. Bu nedenle sineksıklet deseni (flyweight pattern) yorumlanacak yerler için bir eleme yapar.

Aşağıda bir metindeki ifadeyi hesaplayan bir uygulama örneği verilmiştir;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <sstream>
#include <stack>

// --- Önceki Örnekteki Sınıflar (Aynen Kalıyor) ---
class Context {};
class Expression {
public:
        virtual ~Expression() = default;
        virtual int interpret(Context& context) const = 0;
};
class NumberExpression : public Expression {
        int number;
public:
        explicit NumberExpression(int n) : number(n) {}
        int interpret(Context&) const override { return number; }
};
class AdditionExpression : public Expression {
        std::unique_ptr<Expression> left, right;
public:
        AdditionExpression(std::unique_ptr<Expression> l, std::unique_ptr<Expression> r)
        : left(std::move(l)), right(std::move(r)) {}
        int interpret(Context& c) const override { return left->interpret(c) +
        ↪  right->interpret(c); }
};
class MultiplicationExpression : public Expression {
        std::unique_ptr<Expression> left, right;
public:
        MultiplicationExpression(std::unique_ptr<Expression> l, std::unique_ptr<Expression> r)
        : left(std::move(l)), right(std::move(r)) {}
        int interpret(Context& c) const override { return left->interpret(c) *
        ↪  right->interpret(c); }
};

// --- Dinamik Interpreter Sınıfı ---
class Interpreter {
public:
        int interpret(const std::string& rawExpression) {
                Context context;
                auto expressionTree = buildExpressionTree(rawExpression);
                return expressionTree->interpret(context);
        }
private:
        // String ifadeyi parçalara (tokens) ayıran yardımcı metod
        std::vector<std::string> tokenize(const std::string& str) {
                std::vector<std::string> tokens;
                std::stringstream ss(str);
                std::string temp;
                while (ss >> temp) tokens.push_back(temp);
                return tokens;
        }
        // Gerçek Dinamik Ağaç Oluşturucu (Shunting-yard benzeri basitleştirilmiş mantık)
        std::unique_ptr<Expression> buildExpressionTree(const std::string& expression) {
                auto tokens = tokenize(expression);
```

```
                // İşlem önceliği için iki yığın (stack)
                std::stack<std::unique_ptr<Expression>> nodes;
                std::stack<char> operators;
                auto applyOp = [&]() {
                        auto right = std::move(nodes.top()); nodes.pop();
                        auto left = std::move(nodes.top()); nodes.pop();
                        char op = operators.top(); operators.pop();
                        if (op == '+')
                                nodes.push(std::make_unique<AdditionExpression>(std::move(left),
                                ↪  std::move(right)));
                        else if (op == '*')
                                nodes.push(std::make_unique<MultiplicationExpression>(std::move(left),
                                ↪  std::move(right)));
                };
                for (const std::string& token : tokens) {
                        if (isdigit(token[0])) {
                                nodes.push(std::make_unique<NumberExpression>(std::stoi(token)));
                        } else {
                                char currentOp = token[0];
                                // İşlem önceliği kontrolü: Çarpma (+) dan önce yapılır
                                while (!operators.empty() &&
                                ((currentOp == '+' && (operators.top() == '+' || operators.top()
                                ↪  == '*')) || (currentOp == '*' && operators.top() == '*'))) {
                                        applyOp();
                                }
                                operators.push(currentOp);
                        }
                }
                while (!operators.empty()) applyOp();
                return std::move(nodes.top());
        }
};

int main() {
        Interpreter parser;
        // Farklı kombinasyonları test edelim:
        std::string expr1 = "2 + 3 * 4";      // Beklenen: 14
        std::string expr2 = "10 * 2 + 5 * 3"; // Beklenen: 35
        std::string expr3 = "1 + 2 + 3 + 4";  // Beklenen: 10
        std::cout << expr1 << " = " << parser.interpret(expr1) << std::endl;
        std::cout << expr2 << " = " << parser.interpret(expr2) << std::endl;
        std::cout << expr3 << " = " << parser.interpret(expr3) << std::endl;
}
/*Program Çıktısı:
2 + 3 * 4 = 14
10 * 2 + 5 * 3 = 35
1 + 2 + 3 + 4 = 10
*/
```

17.5 MVC Mimari Deseni

MVC (Model-View-Controller), bir yazılım projesinde kodları görevlerine göre üç ana parçaya ayıran bir mimari desendir. Temel amacı, veriyi (mantık), kullanıcı arayüzünü ve bu ikisi arasındaki iletişimi birbirinden bağımsız hale getirerek kodun yönetimini kolaylaştırmaktır.

- **Veri ve Mantık** (model): Uygulamanın beynidir. Veritabanı işlemleri, hesaplamalar ve kurallar burada döner. "Veri nedir?" sorusuna cevap verir.
- **Arayüz/Görünüm** (view): Kullanıcının gördüğü ekrandır (HTML, CSS, butonlar). Sadece veriyi gösterir, arka planda ne olduğuyla ilgilenmez. "Veri nasıl görünmeli?" sorusuna cevap verir.
- **Aracı/Kontrolcü** (controller): Kullanıcıdan gelen istekleri karşılar. `Model`'den veriyi alır ve hangi `View`'da gösterileceğine karar verir. "Hangi veri nereye gitmeli?" sorusuna cevap verir.

Basit bir restoran senaryosunda MVC'yi bir restorana benzetebiliriz;

- Model (Aşçı): Mutfağın içindedir. Yemeği hazırlar, malzemeleri kontrol eder. Müşteriyi görmez, sadece siparişe göre yemeği (veriyi) üretir.
- View (Yemek Masası/Sunum): Müşterinin önüne gelen tabak ve sunumdur. Müşteri yemeğin nasıl piştiğini görmez, sadece sunulan tabağı görür.
- Controller (Garson): Müşteriden siparişi alır, aşçıya iletir. Yemek piştiğinde ise tabağı alır ve müşteriye servis eder.

Bir başka örnek olarak kullanıcı profili verilebilir. Diyelim ki bir kullanıcının profil bilgilerini ekrana getireceksiniz:

- Kullanıcı (User): Profil sayfasına tıklar (istek/request).
- Controller: Bu isteği yakalar. "Kullanıcı verisi lazım" diyerek Model'e seslenir.
- Model: Veritabanına gider, "Ahmet, 25 yaşında, Ankara" bilgilerini bulur ve Controller'a geri verir.
- Controller: Aldığı bu "Ahmet" verisini alır ve "Profil_Ekrani.html" adlı View dosyasına gönderir.
- View: Gelen veriyi şık bir tasarıma yerleştirir ve kullanıcıya gösterir.

MVC kullanmalıyız çünkü;

- **Kod Karmaşasını Önle**r: Tasarımcı sadece `View` ile, yazılımcı sadece `Model`/`Controller` ile ilgilenir.
- **Bakım Kolaylığı**: Görünümü değiştirmek istediğinizde veritabanı koduna dokunmanız gerekmez.
- **Yeniden Kullanılabilirlik**: Aynı `Model` verisini (örneğin kullanıcı listesi), hem bir web sayfasında hem de bir mobil uygulamada farklı `View`'lar ile gösterebilirsiniz.

C++ nesne yönelimli bir dil olduğu için MVC yapısını sınıflar arasındaki ilişkiyle kurgulamak, mantığı anlamak açısından çok faydalıdır. Bir "Öğrenci Bilgi Sistemi" üzerinden, bileşenlerin birbirine bağımlılığını (coupling) minimize edecek şekilde bir örnek verilebilir. Öğrenci Bilgi Sisteminde her bileşen kendi sorumluluğunu üstlenir: `Model` veriyi tutar, `View` ekrana basar, `Controller` ise akışı yönetir.

1. Model (Veri ve İş Mantığı): Sadece veriyi ve bu veriye erişim yöntemlerini içerir. Görselleştirme ile hiç ilgilenmez.
```
#include <string>
class OgrenciModel {
private:
        std::string isim;
        std::string numara;
public:
        OgrenciModel(std::string i, std::string n) : isim(i), numara(n) {}
        // Getter ve Setter yöntemleri
        void setIsim(std::string i) { isim = i; }
        std::string getIsim() const { return isim; }
        void setNumara(std::string n) { numara = n; }
        std::string getNumara() const { return numara; }
};
```
2. View (Görünüm): Verinin kullanıcıya nasıl sunulacağını belirler. Genellikle sadece "Yazdır" veya "Göster" gibi yöntemleri vardır.
```
#include <iostream>
class OgrenciView {
public:
        void ogrenciDetaylariniBas(std::string isim, std::string numara) {
                std::cout << "--- Ogrenci Bilgileri ---" << std::endl;
                std::cout << "Ad: " << isim << std::endl;
                std::cout << "No: " << numara << std::endl;
                std::cout << "-------------------------" << std::endl;
        }
};
```
3. Controller (Denetleyici): Model ve View arasındaki köprüdür. Kullanıcıdan gelen güncellemeleri Model'e yansıtır ve View'u güncellenmeye zorlar.
```
class OgrenciController {
private:
        OgrenciModel& model;
        OgrenciView& view;
public:
        OgrenciController(OgrenciModel& m, OgrenciView& v) : model(m), view(v) {}
        void setOgrenciIsim(std::string i) { model.setIsim(i); }
        std::string getOgrenciIsim() { return model.getIsim(); }
```

```
        void setOgrenciNumara(std::string n) { model.setNumara(n); }
        std::string getOgrenciNumara() { return model.getNumara(); }
        // View'u güncelleme komutu
        void gorunumuGuncelle() {
                view.ogrenciDetaylariniBas(model.getIsim(), model.getNumara());
        }
};
```

4. Sistemin Çalıştırılması:

```
int main() {
        // 1. Model'i (Veriyi) oluştur
        OgrenciModel model("Ahmet Yilmaz", "2026001");
        // 2. View'u (Arayüzü) oluştur
        OgrenciView view;
        // 3. Controller'i bağla
        OgrenciController controller(model, view);
        // İlk hali ekrana bas
        controller.gorunumuGuncelle();
        // Veriyi Controller üzerinden güncelle
        controller.setOgrenciIsim("Mehmet Demir");
        // Güncel hali tekrar bas
        std::cout << "\nVeri guncellendikten sonra:\n";
        controller.gorunumuGuncelle();
}
/* Program Çıktısı:
--- Ogrenci Bilgileri ---
Ad: Ahmet Yilmaz
No: 2026001
-------------------------

Veri guncellendikten sonra:
--- Ogrenci Bilgileri ---
Ad: Mehmet Demir
No: 2026001
-------------------------
*/
```

Buradaki mimari ile ilgili şunlar söylenebilir;

- **Bağımsızlık**: Dikkat edersen `OgrenciModel` sınıfının içinde ne bir `cout` var ne de `OgrenciView` referansı. Bu sayede yarın bir gün veriyi ekrana değil de bir dosyaya yazmak istersen (yeni bir `View`), Model koduna dokunman gerekmez.
- **Bellek Yönetimi**: Örnekte referans (&) kullandım. Ancak büyük projelerde nesne ömürlerini yönetmek için `std::shared_ptr` veya `std::unique_ptr` kullanmak daha sağlıklı olacaktır.
- **Gözlemci Tasarım Deseni İlişkisi**: Gelişmiş MVC yapılarında, `Model` değiştiğinde `Controller`'a haber vermesi için genellikle **gözlemci tasarım deseni** ile birlikte kullanılır.

17.6 Düşün ve Çöz: Tasarım Desenleri Antrenmanı

Bu bölümdeki sorular, desenlerin sadece "nasıl" yazıldığını değil, "neden" ve "nerede" kullanılması gerektiğini sorgulamanız için tasarlanmıştır.

§17.6.1 Yaratımsal (Creational) Desenler

1. **Singleton & Thread Safety:** Bir `Singleton` sınıfının `getInstance()` metodunda `static` yerel değişken kullanmak (Meyers' Singleton) neden C++11 sonrası için güvenli kabul edilir?
2. **Factory Method vs. Abstract Factory:** Bir oyun motorunda sadece "Düşman" üretmek istiyorsanız hangi deseni, "Düşman + Düşmanın Silahı + Düşmanın Zırhı"ndan oluşan uyumlu bir set üretmek istiyorsanız hangi deseni seçersiniz? Neden?
3. **Builder & Immutability:** Bir nesne inşa edildikten sonra durumunun (state) değiştirilmesini istemiyorsanız, `Builder` deseni size nasıl bir avantaj sağlar?
4. **Prototype & Performance:** Veritabanından 50 farklı tabloyu join ederek oluşturulan ağır bir nesneyi 1000 kez kopyalamanız gerekiyorsa, neden `new` yerine `clone()` tercih edilmelidir?
5. **Dependency Injection:** Bir sınıfın içinde başka bir sınıfı `new` ile oluşturmak yerine, onu kurucu fonksiyon üzerinden dışarıdan almanın "Test Edilebilirlik" açısından önemi nedir?

§17.6.2 Yapısal (Structural) Desenler

6. **Adapter & Legacy Code:** Eski bir kütüphanedeki `draw_circle(int, int, int)` fonksiyonunu, yeni sisteminizdeki `IDrawable` arayüzüne nasıl entegre edersiniz?
7. **Bridge & Class Explosion:** 5 farklı işletim sistemi ve 3 farklı dosya formatı (PDF, XML, JSON) için çıktı veren bir sistemde Bridge kullanılmazsa toplam kaç sınıf oluşur? Bridge ile bu sayı kaça düşer?
8. **Decorator & Composition:** Bir nesneye çalışma zamanında (runtime) özellik eklerken neden kalıtım yerine sarmalama (wrapping) tercih edilir?
9. **Facade & Clean Code:** Bir kütüphanenin 20 farklı sınıfını kullanmak zorunda olan bir istemciye (client) neden doğrudan bu sınıfları değil de bir `Facade` sınıfını sunmalısınız?
10. **Proxy & Security:** Bir nesneye erişmeden önce kullanıcının yetkisini kontrol etmek istiyorsanız, bu mantığı asıl nesnenin içine mi yoksa bir `Proxy` içine mi yazmalısınız? Neden?
11. **Flyweight & RAM:** Bir metin düzenleyicideki her bir karakter için ayrı birer nesne oluşturmak belleği nasıl etkiler? Paylaşılan (intrinsic) veri ne olmalıdır?

§17.6.3 Davranışsal (Behavioral) Desenler

12. **Chain of Responsibility:** Bir HTTP isteğinin önce "Güvenlik", sonra "Loglama", en son "Veri İşleme" aşamalarından geçmesi gerekiyorsa, zincirin sırasını çalışma zamanında değiştirmek bize ne kazandırır?
13. **Command & Undo:** Bir "Geri Al" (undo) mekanizması kurarken neden komutun sadece kendisini değil, operasyon öncesindeki durumu da saklaması gerekir?
14. **Iterator & Encapsulation:** Bir `List` nesnesinin içindeki `std::vector` yapısını `std::list` olarak değiştirirsek, `Iterator` kullanan istemci kodları bundan etkilenir mi?
15. **Mediator & Air Traffic:** 10 uçağın birbirinin konumunu bilmesi yerine neden sadece kulenin (`Mediator`) tüm uçakları bilmesi daha güvenlidir? Nesneler arasındaki bağımlılık (coupling) nasıl değişir?
16. **Observer & Memory Leaks:** Bir nesne (`Subject`) silindiğinde, ona abone olan (`Observer`) nesnelerin listeden çıkarılmaması ne gibi sorunlara yol açar?
17. **State Pattern:** Bir ATM makinesinin durumlarını yönetirken devasa bir `switch-case` bloğu yerine `State` desenini kullanmanın en büyük avantajı nedir?
18. **Strategy & Open-Closed:** Bir sıralama algoritmasını çalışma zamanında değiştirmek istediğinizde `Strategy` deseni "Open-Closed" prensibini nasıl sağlar?
19. **Template Method vs. Strategy:** Algoritmanın iskeletini sabit tutup adımlarını alt sınıfa bırakmakla, algoritmanın tamamını bir nesne olarak değiştirmek arasındaki temel fark nedir?
20. **Visitor & Double Dispatch:** Bir sınıf hiyerarşisine yeni bir işlem eklemek için neden sınıfların kodunu değiştirmek yerine `Visitor` kullanmalıyız?

18. Bölüm

Değişken Argümanlı Fonksiyonlar

18.1 Değişken Sayıda Parametre Tanımı

Belirsiz sayıda argüman alabilen fonksiyona **değişken sayıda argüman alan fonksiyon** (variadic function) denir. C dilindeki güçlü ancak çok nadiren kullanılan özelliklerden biridir. C dilindeki `printf()` gibi fonksiyonlardan tanıdığımız klasik kullanımdır.

C Dilindeki gibi C++ dilinde de, belirsiz sayıda bağımsız değişkeni olan bir fonksiyon; **en az bir sabit bağımsız değişkene** sahip olacak şekilde tanımlanır ve ardından derleyicinin değişken sayıda bağımsız değişkeni ayrıştırmasını sağlayan bir üç nokta (`...`) eklenir. **Birinci parametre kendisinden sonra gelebilecek parametre sayısı hakkında bilgi verir.** Aşağıda buna ilişkin örnek verilmiştir;

```
#include <iostream>
#include <cstdarg>
//ÖNERİLMEZ! TİP GÜVENLİĞİ YOKTUR!
int argumanlarinHepsiniTopla(int kacAdet,...) {
        va_list argumanlar;
        int sayac, toplam = 0;
        va_start(argumanlar, kacAdet);
        for (sayac = 0; sayac < kacAdet; sayac++)
        toplam += va_arg(argumanlar, int);
        va_end(argumanlar);
        return toplam;
}
int main(){
        std::cout << "3 Argüman Toplamı = " << argumanlarinHepsiniTopla(3, 10, 20, 30) <<
         ↪  std::endl;
        std::cout << "5 Argüman Toplamı = " << argumanlarinHepsiniTopla(5, 10, 20, 30,40,50) <<
         ↪  std::endl;
}
/*Program Çıktısı:
3 Argüman Toplamı = 60
5 Argüman Toplamı = 150
*/
```

Değişken sayıda parametreleri işlemek için kodunuza `<cstdarg>` başlığını dahil etmeniz gerekir. Bu başlıktaki fonksiyonlar aşağıda verilmiştir;

Fonksiyon	Açıklama
`va_start(va_list ap, arg)`	Bu fonksiyon, üç nokta ile verilen argümanları va_list değişkenine aktarır.
`va_arg(va_list ap, type)`	Her seferinde, üç nokta ile temsil edilen değişken listesindeki bir sonraki argümanı `va_list` üzerinden işler ve listenin sonuna ulaşana kadar onu type ile verilen veri tipine dönüştürür.
`va_copy(va_list dest, va_list src)`	`va_list`'teki argümanların bir kopyasını oluşturur.
`va_end(va_list ap)`	Bu, `va_list` değişkenlerine erişimi sonlandırır.

Tablo 18.1: Stdarg.h Fonksiyonları

Modern C++'ta bu yöntem tip güvenliği (type safety) olmadığı için kesinlikle önerilmez!

Bu, günümüz C++ dünyasında en çok tercih edilen yöntem, C++17 ile hayatımıza giren değişken şablonlar (variadic templates) ve katlama ifadeleridir (fold expressions). Bu konu 15.3 başlığında anlatılmıştır. Tip güvenliği tamdır. Yani sadece `int` değil, her şeyi toplayabilir ve "kaç adet" olduğunu belirtmemize gerek kalmaz; derleyici bunu senin için hesaplar.

```
#include <iostream>
// Değişken sayıda argüman alan modern şablon fonksiyonu
template<typename... Args>
auto argümanlarinHepsiniTopla(Args... args) {
        // C++17 Fold Expression (Katlama İfadesi): Paketteki tüm elemanları toplar
        return (... + args);
}
int main() {
        // Adet belirtmeye gerek yok!
        std::cout << "3 Arguman Toplami = " << argümanlarinHepsiniTopla(10, 20, 30) << std::endl;
        std::cout << "5 Arguman Toplami = " << argümanlarinHepsiniTopla(10, 20, 30, 40, 50) <<
         ↪  std::endl;
}
```

Bir de eski C++11 yöntemi olan liste yöntemi (`std::initializer_list`) bunun için kullanılabilir. Eğer gönderilecek tüm argümanların türü aynıysa (hepsi `int` ise), bu yöntem çok temizdir. Argümanları süslü parantez içinde göndermemiz gerekir.

```
#include <iostream>
#include <initializer_list>
#include <numeric> // std::accumulate için
int argümanlarinHepsiniTopla(std::initializer_list<int> liste) {
        // Listedeki tüm elemanları başlangıç değeri 0 olacak şekilde toplar
        return std::accumulate(liste.begin(), liste.end(), 0);
}
int main() {
        std::cout << "3 Arguman Toplami = " << argümanlarinHepsiniTopla({10, 20, 30}) <<
         ↪  std::endl;
        std::cout << "5 Arguman Toplami = " << argümanlarinHepsiniTopla({10, 20, 30, 40, 50}) <<
         ↪  std::endl;
}
```

18.2 Değişken Sayıda Argüman Alan Ana Fonksiyon

Ana fonksiyon (main function) programın icra edilmeye başladığı fonksiyondur. Şu ana kadar argümansız ana fonksiyonu gördük. C++ Dilinde yazdığımız programlar da çalıştırılırken konsoldan argüman alabilir.

> Ana fonksiyon;
>
> ◇ Satır içi (inline) fonksiyon olarak bildirilemez!
> ◇ Adresi alınamaz!
> ◇ Programın başka hiçbir yerinden çağrılamaz (call)!

Yazdığımız programlar da çalıştırılırken konsoldan argüman alabilir. Argüman alan ana fonksiyon (main function) aşağıdaki gibi iki şekilde tanımlanır;

```
int main(int argc, char* argv[]) {
 // Ana fonksiyon Gövdesi
 // ...
}
//Ya da aşağıdaki şekilde tanımlanır;
int main(int argc, char* argv[], char ** envp) {
 // Ana fonksiyon Gövdesi
 // ...
}
```

Aşağıda konsoldan çalıştırılırken alınan argümanları ve işletim siteminin ortam değişkenleri (environment variable) konsola yazan bir program örneği verilmiştir;

```
#include <iostream>
#include <vector>
#include <string_view>
int main(int argc, char* argv[], char* envp[]) {
        // 1. Argümanları İşleme (Command Line Arguments)
        std::cout << "--- Konsoldan Girilen Argümanlar ---\n";
        // argv bir dizidir, modern for döngüsü için bir vektöre kopyalanabilir
```

```
        // veya doğrudan indislerle erişilebilir.
        for (int i = 0; i < argc; ++i) {
                std::cout << "argv[" << i << "]: " << argv[i] << "\n";
        }
        // 2. Ortam Değişkenlerini İşleme (Environment Variables)
        std::cout << "\n--- Ortam (Environment) Degiskenleri ---\n";
        // envp, sonu NULL olan bir işaretçi dizisidir.
        for (char** env = envp; *env != nullptr; ++env) {
                std::string_view envVar(*env); // Bellek kopyalamadan veriyi oku
                std::cout << envVar << "\n";
        }
        return 0;
}
```

Programın derlenmesi ve çalıştırılması sonrası çıktı aşağıda verilmiştir;

```
C:\Users\ILHANOZKAN>notepad main.cpp
C:\Users\ILHANOZKAN>gcc main.cpp -o main.exe
C:\Users\ILHANOZKAN>main 10 20 otuz elli

Konsoldan Girilen Argümanlar:
argv[0]   main.exe
argv[1]   10
argv[2]   20
argv[3]   otuz
argv[4]   elli

Ortam Değişkenleri:
ALLUSERSPROFILE=C:\ProgramData
APPDATA=C:\Users\ILHANOZKAN\AppData\Roaming
CommonProgramFiles=C:\Program Files\Common Files
CommonProgramFiles(x86)=C:\Program Files (x86)\Common Files
CommonProgramW6432=C:\Program Files\Common Files
COMPUTERNAME=ILHANOZKAN
ComSpec=C:\WINDOWS\system32\cmd.exe
configsetroot=C:\WINDOWS\ConfigSetRoot
DriverData=C:\Windows\System32\Drivers\DriverData
HOMEDRIVE=C:
HOMEPATH=C:\Users\ILHANOZKAN
LOCALAPPDATA=C:\Users\ILHANOZKAN\AppData\Local
LOGONSERVER=\\DAMLANURO
NUMBER_OF_PROCESSORS=8
OneDrive=C:\Users\ILHANOZKAN\OneDrive
OneDriveConsumer=C:\Users\ILHANOZKAN\OneDrive
OS=Windows_NT Path=C:\WINDOWS\system32;C:\Users\ILHANOZKAN\Downloads\codeblocks\MinGW\bin;
PATHEXT=.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC
PROCESSOR_ARCHITECTURE=AMD64
PROCESSOR_IDENTIFIER=Intel64 Family 6 Model 142 Stepping 12, GenuineIntel
PROCESSOR_LEVEL=6
PROCESSOR_REVISION=8e0c
ProgramData=C:\ProgramData
ProgramFiles=C:\Program Files
ProgramFiles(x86)=C:\Program Files (x86)
ProgramW6432=C:\Program Files
PROMPT=$P$G
PSModulePath=C:\Program Files\WindowsPowerShell\Modules;
PUBLIC=C:\Users\Public
SESSIONNAME=Console
SystemDrive=C:
SystemRoot=C:\WINDOWS
TEMP=C:\Users\ILHANOZKAN\AppData\Local\Temp
TMP=C:\Users\ILHANOZKAN\AppData\Local\Temp
```

```
USERDOMAIN=DAMLANURO
USERDOMAIN_ROAMINGPROFILE=DAMLANURO
USERNAME=ILHANOZKAN
USERPROFILE=C:\Users\ILHANOZKAN
windir=C:\WINDOWS
ZES_ENABLE_SYSMAN=1

C:\Users\ILHANOZKAN>
```

19. Bölüm
Ön İşlemci Yönergeleri

19.1 Ön İşlemci Yönergesi Nasıl Çalışır?

C dilinde olduğu gibi C++ dilinde de **ön işlemci** yönergeleri (preprocessor directive), derleyicinin bir parçası değildir, ancak derleme sürecinde ayrı bir adımdır. Ön işlemci, yalnızca bir metin değiştirme aracıdır ve gerçek derlemeden önce gerekli ön işlemeyi yapmasını sağlar. Derleme süeci, Şekil 19.1'de gösterilmiştir. Kısaca derleme önce bul-değiştir mantığı ile kodumuzda değişiklikler yapar.

- Ön işleme bir C kodunun derlenmesi öncesindeki ilk adımdır.
- Kodu belirteçlere ayırma (tokenization) adımından önce gerçekleşir.
- Ön işlemcinin önemli işlevlerinden biri de programda kullanılan kütüphane işlevlerini içeren başlık dosyalarını koda dahil etmek için kullanılmasıdır.
- Ön işlemci ayrıca değişmezleri (literal) tanımlar ve makro kullanımını sağlar.

Ön işlemci ifadelerine, **yönerge** (directive) denir. Programda ön işlemci bölümü her zaman C kodunun en üstünde görünür. Her ön işlemci ifadesi, kare (hash) (`#`) sembolüyle başlar. Aşağıda ön işlemci yönergelerinin listesi verilmiştir;

Yönerge	Açıklama
`#define`	Ön işlemci makrosunu değiştirmek ya da ilk defa tanımlamak için kullanılır.
`#include`	Bir başlık dosyasını diğer ile dahil etmek için kullanılır.
`#undef`	Tanımlanmış bir makroyu tanımsız hale getirmek için kullanılır.
`#ifdef`	Bir makro tanımlı ise doğru/true değerini döndürür.
`#ifndef`	Bir makro tanımlı değilse doğru/true değerini döndürür.
`#if`	Derleme zamanında bir durumun kontrolü için kullanılır.
`#else`	`#if` Yönergesinde alternatif durumu ifade eder.
`#elif`	`#else` ve `#if` yönergelerini tek bir şekilde ifade etmek için kullanılır.
`#endif`	Şart ön işlemcilerini (`#if`, `#else`, `#elif`) bitirir.
`#error`	stderr standart dosyasına hata mesajını yazar.
`#pragma`	Derleyiciye özel komutlar vermek için kullanılır.

Tablo 19.1: Ön İşlemci Yönergeleri

19.2 Ön işlemci Makroları

`#define` yönergesi ile yeni bir makro tanımlanabileceği gibi tanımlanmış bir makro `#undef` ile ortadan kaldırabilir veya yenisini tanımlayabiliriz. Aşağıdaki örnekte `MAX_STUDENTS` adlı bir makro tanımlanmış ve `100` tam sayısını vermektedir. Daha önce `MAX_STUDENTS` adlı bir makro tanımlanmış ise derleyici hata verir. Aşağıda buna bir örnek verilmiştir;

```
#include <iostream>

#ifndef MAX_STUDENTS
#define MAX_STUDENTS 100
#endif

int main() {
        std::cout << "Ilk MAX_STUDENTS: " << MAX_STUDENTS << std::endl;
#ifdef MAX_STUDENTS
#undef MAX_STUDENTS
#define MAX_STUDENTS 20
#endif
        std::cout << "Yeni MAX_STUDENTS: " << MAX_STUDENTS << std::endl;
}
```

Ön işlemci yönergeleriyle koşullu derleme yapılır. `#ifndef` (if not defined) veya `#ifdef` (if defined) ile kodun belirli bölümlerini duruma göre derleme dışı bırakabilir veya dahil edebilir.

Ön işlemci makrolarının (`#define`) veri tipi yoktur ve kapsam (scope) tanımazlar. Bu nedenle C++ dilinde makro kullanımı **sıkıntılıdır!**
Makroların, tip kontrolü yoktur. Bu nedenle hatalara açıktır.
Hata ayıklayıcılar (debugger) makroyu görmez. Bu nedenle makrolarda hata ayıklama zordur.
(Header guard) gibi koşullu derleme için makrolar kullanılmak zorundadır!

Modern C++'ta sabit tanımlamak için `constexpr` tercih edilir. Bu konu **??** başlığında işlenmiştir.

```
// sabitler için makro yerine:
#define PI 3.1415
// Modern C++ ta sabit ifadeler kullanılır:
constexpr double PI = 3.1415; // Tip güvenli ve derleme zamanı sabiti
```

19.3 Parametreli Ön İşlemci Makroları

Ön işlemci yönergeleriyle tanımlanan makrolar derlemeye geçilmeden önce işlem görür. Dolayısıyla bu aşamada bazen parametrelerle işlem yapılması gerekebilir.

```
#define kare(x) ((x) * (x))
#define kup(x) ((x) * (x) *(x))
#define buyugu(x,y) ((x) > (y) ? (x) : (y))
```

Burada kullanılacak parametreler veri tipi tanımlanmaz. Dolayısıyla kullanılacağı yerlere buna dikkat edilerek parametreli makro tanımlanmalıdır. Eğer makro birkaç satırdan oluşacak ise **ters bölü** (back slash) (\) karakteri kullanılır.

```
#include <iostream>
#define SAYIYAZ(num, str) {  \
        /* 1.Satir */     \
        std::cout << str;    \
        /* 2.Satir */     \
        std::cout << "=" << num << std::endl;  \
        /* Son satırda terbölü olmaz! */    \
        }
#define MY_MULTI_LINE_MACRO(arg1, arg2) do { \
        /* Line 1 of the macro */   \
        std::cout << "Arg1:" << arg1 << std::endl;  \
        /* Line 2 of the macro */    \
        std::cout << "Arg2:" << arg2 << std::endl;   \
        /* Last line (no backslash) */   \
        } while (0);
int main() {
        SAYIYAZ(4,"Dort")
        MY_MULTI_LINE_MACRO(1,2)
}
/*Program Çıktısı:
Dort=4
Arg1:1
Arg2:2
*/
```

C++ dilinde makro kullanımı sıkıntılıdır. Eski kodlarda karşılaşırsınız diye bu konu anlatılmıştır.

19.4 Ön Tanımlı Ön İşlemci Makroları

Her C derleyicisi için standart olarak tanımlı ön işlemci yönergesi ile makrolar bulunmaktadır;

Makro	Açıklama
`__DATE__`	Mevcut tarihi `"MMM DD YYYY"` biçiminde dizgi (string) olarak tanımlıdır.
`__TIME__`	Mevcut saat `"HH:MM:SS"` biçiminde dizgi (string) olarak tanımlıdır.
`__FILE__`	Dosya adı biçiminde dizgi (string) olarak tanımlıdır.
`__LINE__`	Dosyadaki satır numarası tam sayı (integer) olarak tanımlıdır.
`__STDC__`	ANSI standardında derleme yapılıyorsa 1 olarak tanımlıdır.

Tablo 19.2: Standart Ön İşlemci Makroları

Standart makroları kullanan örnek program aşağıda verilmiştir;

```
#include <iostream>
int main() {
        std::cout << __FILE__ << std::endl; //File: main.c
        std::cout << __DATE__ << std::endl; //Date: Mar 22 2025
        std::cout << __TIME__ << std::endl; //Time: 15:32:08
        std::cout << __LINE__ << std::endl; //Line: 6
        std::cout << __STDC__ << std::endl; //ANSI: 1
}
```

Bu makrolar, özellikle büyük projelerde hata ayıklama (debug) ve iz kaydı (log) tutma işlemleri için hayat kurtarıcıdır. Modern C++ dünyasında bu bilgiler hâlâ aynı isimlerle kullanılır, ancak çıktı yöntemleri ve standartlar biraz daha gelişmiştir.

Modern C++ (C++11 ve sonrası) bu makrolara ek olarak `__func__` (veya derleyiciye göre `__FUNCTION__`) makrosunu da getirmiştir. Bu makro, hatanın hangi fonksiyonun içinde gerçekleştiğini anlamamızı sağlar.

```
#include <iostream>
void hataLogu(const std::string& mesaj, const char* dosya, int satir, const char* fonk) {
        std::cerr << "[HATA] " << dosya << ":" << satir << " (" << fonk << ") -> " << mesaj <<
        ↪  std::endl;
}
int main() {
        // Örnek bir hata simülasyonu
        hataLogu("Veritabanı baglantisi koptu!", __FILE__, __LINE__, __func__);
}
```

Eğer C++20 kullanıyorsanız, bu makroların yerine çok daha nesne yönelimli ve "temiz" bir yol olan `std::source_location` sınıfını kullanabilirsiniz:

```
#include <iostream>
#include <source_location>
void bilgiYazdir(const std::source_location location = std::source_location::current()) {
        std::cout << "Dosya: " << location.file_name() << "\n" << "Fonksiyon: " <<
        ↪  location.function_name() << "\n" << "Satir: " << location.line() << std::endl;
}
int main() {
        bilgiYazdir();
}
```

19.5 Başlık Koruması

Ön işlemci direktiflerinin en yaygın ve profesyonel kullanım alanı başlık (header) koruması (header guard) yapısıdır. Bir başlık dosyasının (.h) kazara birden fazla kez projeye dahil edilip yeniden tanımlama (redefinition) hatası vermesini engeller.

```
// Ogrenci.h dosyası içeriği:
#ifndef OGRENCI_H  // Eğer OGRENCI_H tanımlı değilse
#define OGRENCI_H  // OGRENCI_H tanımla

class Ogrenci {
        // Sınıf tanımları
};

#endif // OGRENCI_H sonu
```

Başlık dosyasının bir koda birden fazla dahil (include) edilmesini önlemek için `OGRENCI_H` makrosu, şartlı (conditional) olarak tanımlanmıştır. Bu başlık dosyası bir kod projesinde birden fazla dahil olması halinde, `OGRENCI_H` tanımlanmaz ise, başlık içindeki değişken ve fonksiyonlar birden fazla aynı kimlikle tanımlanacağından derleme hatası alınacaktı.

Kendi hazırlamış olduğumuz `Ogrenci.h` başlık dosyasını kodumuza dahil ettiğimizde artık başlık dosyamızdaki fonksiyon ve değişkenleri kullanabilir hale geliriz. Aynı klasörde/dizinde bulunacak şekilde bu başlık dosyasını kullanan `main.c` örnek programı aşağıda verilmiştir;

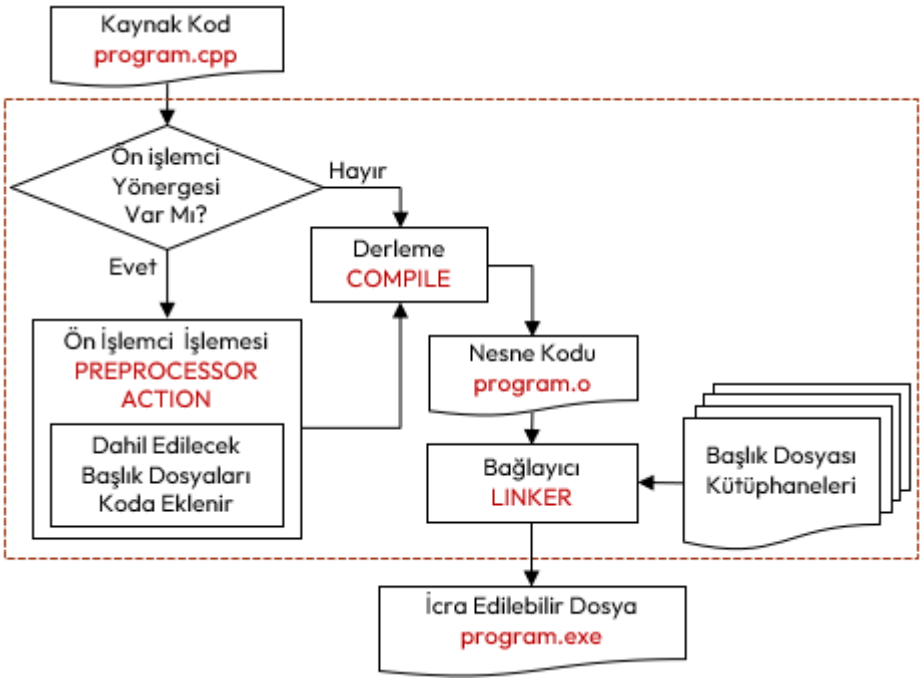
```
#include <iosreram>
#include "Ogrenci.h"
```

```
int main() {
        Ogrenci ali;
        //...
}
```

19.6 Derleme Sürecinin Detayı

C++ dilinde derleme sürecinin iki aşamadan oluştuğu daha önce anlatılmıştı. Aşağıda tüm süreç verilmiştir;

1. Birinci aşama:
   (a) Kaynak koddaki tüm açıklamalar silinir.
   (b) `#define`, `#if`, `#include`, gibi tüm yönergeler ile verilen işlemler yapılır.
   (c) Bunlar arasında başlık dosyalarının koda dahil edilmesi de vardır.
   (d) Bu duruma gelen kod, `g++ -E program.cpp` komutu ile dosya haline getirilebilir.
2. İkinci aşama:
   (a) Kaynak kod montaj koda (assembly code) çevrilir.
   (b) Bu duruma gelen kod, `g++c -S program.cpp` komutu dosya haline getirilebilir.
   (c) Assembly kod derlenerek makine diline çevrilir ve amaç dosya (object file) oluşur.
   (d) Bu duruma gelen kod, `g++ -c program.cpp` komutu ile dosya haline getirilebilir.
   (e) Amaç dosya kodda belirtilen başlık dosyalarına uygun kütüphaneler (library) ile birbirine bağlayıcı (linker) ile bağlanır ve icra edilebilir dosya (executable file) elde edilir. Burada bahsedilen kütüphaneler her işletim sistemi için ayrı olarak oluşturulur ve derleyici kurulumunda bir klasörde tutulur.
   (f) Bu aşamaya gelen derleme `g++ program.cpp -o program.exe` komutuyla icra edilebilir dosya oluşturulmaktadır.



Şekil 19.1: Derleme Süreci

19.7 Düşün ve Çöz

- **Düşün**: `#define` ile tanımlanan makroların, normal fonksiyonlara göre avantajları ve riskleri (yan etkileri) nelerdir?
- **Düşün**: Başlık dosyalarında kullanılan `#ifndef`, `#define`, `#endif` **başlık güvenliği** (header guard) bloklarının mükerrer dahil etmeyi nasıl önlediğini açıklayınız.
- **Düşün**: Koşullu derleme (`#ifdef DEBUG`) kullanarak, programın sadece geliştirme aşamasında ekrana teknik loglar basmasını nasıl sağlarsınız?
- **Çöz**: Bir sayının karesini alan bir makro (`#define KARE(x)`) tanımlayınız.`KARE(3+2)` ifadesinin sonucunu makronun nasıl açıldığını düşünerek analiz ediniz.
- **Çöz**: İki sayının küçüğünü bulan bir fonksiyonel makro (`#define MIN(a,b)`) yazınız ve makro **yan etkilerini** (side effects) önlemek için neden parantez kullanılması gerektiğini gösteriniz.

◇ **Çöz**: C'nin ön tanımlı makrolarını (`__DATE__`, `__TIME__`, `__FILE__`) kullanarak programın derlendiği anı ve dosya adını ekrana basan bir "Sürüm Bilgisi" fonksiyonu yazınız.

Dizin

+ operator, 102
= operator, 102
== operator, 102
=default specifier, 138
=delete specifier, 134

abstract, 124, 126, 146
abstract class, 149, 168
abstract factory pattern, 228
abstract method, 168
abstraction, 37, 124, 173
access, 25
access modifier, 119, 126, 168
accumulator, 3
actual parameter, 12
adapter pattern, 239
address operator, 32, 97
aggregation, 168, 172, 173
algotrithm, 213
American Standard Code for Information Interchange, 102
analysis pattern, 227
anti-pattern, 227
architectural pattern, 227
argument, 12
argument dependent lookup, 113
aritmetic functor, 216
aritmetic logic unit, 3
aritmetic operator, 31
array, 13, 79, 97
array decay, 88
array size, 29, 79
assembly, 19
assembly code, 5
assignment, 35
assignment constructor, 138
assignment operator, 34
associated container, 196
association, 168, 172
asynchronous, 177
auto keyword, 158
auto storage class, 70, 72
automatic type conversion, 154

base case, 74, 186, 187
base class, 130, 141, 150, 169
behavior, 116, 117, 119, 125, 126, 129, 146
behavioral patterns, 227, 253
bidirectional iterators, 215
binary, 11, 12, 24, 27
binary file, 222
BIOS, 6
bit field, 109
bitwise, 36
bitwise copy, 236
bitwise functors, 216
bitwise operator, 33
body, 160
bool literal, 27
break statement, 58
bridge pattern, 251
bubble sort, 88
buffer overflow, 88, 102, 217
buffered, 218
builder pattern, 232
bus, 1, 3
byte, 2

call, 12
call by reference, 97
call by reference over pointer, 96
call by value, 74, 78
callback, 99
calling convention, 69
calling overhead, 37
camel case, 25
capture clause, 160
catch an exception, 179
chain of responsibility pattern, 254
change management, 37
character literal, 27
child class, 130, 150
circular dependency, 193
class, 116, 168
class diagram, 129, 168
class templates, 188
classifier, 116
clock, 2
code bloat, 185
code block, 21
code segment, 70
comma operator, 54, 64, 188
command pattern, 255
comment, 23
comparision functors, 216
compile, 16, 17
compile-time, 29, 149, 155, 181, 185
compile-time error, 18, 19, 144
compile-time warning, 19
compiler, 5, 16
component, 126
composite pattern, 241
composition, 168, 172
computer program, 5
concrete, 124, 228
concrete class, 149
condition, 43, 52
conditional choice, 42, 43
conditional code, 43
conditional code register, 3
conditional tenary operator, 50
console, 37
const, 28, 191
const cast, 156
const method, 144
const qualifier, 28
const specifier, 156
const variable, 26, 28
const_cast operator, 156
constant, 26
constant expression, 29
constexpr, 29, 76, 293
constructor, 119, 122, 164, 191
container adapter, 197
continue statement, 58
control abstraction, 124–126
control expression, 48
control operation, 42, 44
control structure, 13, 24
control unit, 3
controller, 284
copy constructor, 133, 134, 137, 139
creational patterns, 227
current instruction code, 4
current instruction register, 3
cursor, 37

dangling else, 47
dangling pointer, 192
data, 1
data abstraction, 124, 126, 146
data memory register, 69
data pattern, 227
data segment, 70–72, 98, 143, 209
data structure, 13, 24, 79
data type, 10, 12, 24, 32, 89
data type modifier, 26
debugger, 18, 293
decay of array, 209
declaration, 25, 77
decltype operator, 159
decode, 3, 4
decorator pattern, 245
decrement, 32
deep copy, 236

default, 70, 119
default argument, 77, 140
default constructor, 120, 135, 138
definition, 24, 25
delegating constructor, 121, 136
delete operator, 92, 100, 122
dependency, 169, 170
dependency as a parameter, 169
dependency as an instantiation, 169
dependency injection, 170
Dependency Inversion Principle, 175
dereference operator, 89, 92
derived class, 130, 141, 150, 169
derived type, 89
design pattern, 143, 227
destructor, 133, 141, 183, 191
deviation, 81
directive, 292
do-while statement, 52
dot operator, 104, 117, 122
double pointer, 91
dynamic binding, 149
dynamic cast, 155
dynamic data structure, 89, 108, 196
dynamic initialization, 122
dynamic_cast operator, 155

else code, 45
emty product, 74
encapsulation, 124, 126, 173
engineer, 1
entry, 266
enumaration class, 151
equality, 35
event-driven programming, 99
exception, 177
exception object, 177
executable, 17
execute, 3, 4, 17
explicit specifier, 164
explicit type casting, 154, 155
expression, 42, 43, 48, 159
extern specifier, 71
extern storage class, 71
extern variable, 71
extraction operator, 38, 143, 211, 218, 219

facade pattern, 236
factory method pattern, 230
fetch, 3
field, 116–119, 122, 135
file, 218
final, 150
first in first out, 197
flag, 3
flag register, 69
floating point number, 11, 24
flowchart, 42
flyweight pattern, 248
fold expressions, 187
for statement, 53
foreach loop, 88
forward iterator, 215
framework, 126
free format, 21, 22
friend, 133, 142
function, 12
function block, 21
function call, 64
function call operator, 196
function declaration, 64, 66
function definition, 64, 66, 67, 77
function name, 64
function object, 215
function operator, 215
function pointer, 99, 103
function prototype, 64
function templates, 185
functor, 215

garbage, 72
garbage collector, 191
general class, 130, 150
general purpose registers, 70
generalization, 169
generic pointer, 90
generic programming, 185, 191, 196
global, 72
global variable, 72
goto, 51

hash, 204, 292
hash map, 204
header, 37, 61, 69, 111, 294
header guard, 69, 293, 294
heap, 192, 213
heap segment, 70, 91, 99
hexadecimal, 27
high cohesion, 173
high level programming language, 9

identifier, 24
identifier definition, 25, 42, 51
if statement, 43
if-else statement, 45
imlpicit type casting, 154
immutable, 144
immutable object, 162
imperative programming, 14, 21, 116
implementation, 17
implementation pattern, 227
implicit type casting, 154
include, 37
increment, 32
index, 79
indirection operator, 105, 122
infinitive loop, 59
information, 1
information hiding, 124, 126
inheritance, 129, 131, 139, 148, 155, 169, 173
initial value, 69
initialize, 27, 28, 30, 79, 104
inline, 160
inline function, 76
inline specifier, 76
input iterator, 215
input stream, 38
input-output, 37
input-output operation, 42
insertion operator, 37, 143, 218, 219
instance, 116, 119, 149, 168
instruction, 1, 3
instruction code, 4, 5, 16, 22
instruction cycle, 3
instruction set, 4
integer, 24, 79
integer literal, 27, 48, 151
integral literal, 151
integrated circuit, 1
intellisense, 17, 18
interface, 124, 133, 146, 168
interface imlementation, 146
interface realization, 146, 168
Interface Segregation Principle, 175
interpreter pattern, 281
interrupt, 26
invoke function, 64
iteration, 58, 74
iterator, 213, 214
iterator pattern, 259
itertation, 211

jump, 42, 57

key-value, 196, 201
keywords, 22, 36

label, 51
lambda expression, 159
last in first out, 70, 197
lazy copy, 236
leaf class, 150
library, 61, 126
link, 25, 67
link-time, 64
link-time error, 19
linked list, 13
linker, 61, 67
Liskov Substitution Principle, 175

literal, 26–28, 150
local variable, 72
locale, 40
logical error, 19, 73
logical functors, 216
logical operator, 33
logical sequence, 9, 22, 23, 42, 52–54, 59
logical sequences, 22
loop, 74
loop code, 52, 58
loose coupling, 173
low level programming, 8, 13

machine code, 4
machine language, 5
main function, 13, 21, 60, 61, 289
manipulator, 39, 221
mediator pattern, 273
memberwise copy, 236
memento pattern, 270
memory, 1
memory data register, 4
memory leak, 192
memory stream, 221
message-passing, 116
method, 117, 134
method chaining, 153
mnemonic, 5, 16
model, 284
Model-View-Controller, 261, 284
modular programming, 37
modulus, 31
move assignment, 191
move constructor, 140, 191
multiple inheritance, 133, 146
mutable keyword, 144

namespace, 38, 111
namespace keyword, 111
narrowing casting, 155
nested if, 44
nested loop, 56
nested structs, 104, 107, 115
new operator, 92, 99, 122
noexcept operator, 181
noexcept specifier, 181, 182
NULL Character, 27, 101, 208, 209
null pattern, 265
NULL Pointer, 95
null pointer, 90

object, 116, 119
object oriented programming, 14, 42, 116
object slicing, 181
observer pattern, 261
one definition rule, 69
opcode, 4
Open Closed Principle, 174
operand, 4, 30, 31
operating system, 6, 16
operator, 19, 42, 94, 134, 218
operator overloading, 134
operator precedence, 34
operator", 28
out of bounds, 79, 88
output iterator, 215
output stream, 37
overloading, 77, 225
override, 146, 148, 182
override specifier, 145
overriding, 145
ownership, 170, 172

pair, 201
parameter, 160
parameter pack, 186, 187
parameterized constructor, 120, 136
parent class, 130, 150
pattern, 227
pattern oriented programming, 227
physical sequence, 42, 52–54, 59
placeholder, 185
plug-in, 18
pointer, 79, 140
pointer type, 89
polymorphism, 148, 155, 173
pop, 74
postfix, 35
preprocessor directive, 29, 37, 61, 109, 292
primitive data type, 13, 24
private, 119, 132, 150, 168, 172
private association, 170, 172
private class, 130
procedure, 68
processor, 1
program counter, 3
programming, 5
property, 125, 129
protected, 119, 132, 168
prototype, 61
prototype pattern, 234
proxy pattern, 249
pseudocode, 15
public, 117, 119, 132, 168, 172
public association, 170, 172
punc card, 8
pure virtual method, 146, 149

random access iterator, 215
recursion, 74
recursive, 75, 187, 241
referans type, 89
reference, 95
reference counter, 193
register, 3
register specifier, 72
register storage class, 72
reinterpret cast, 157
reinterpret_cast operator, 157
relational, 35
relational loop, 42, 52
relationship, 168
reserved words, 22
Resource Acquisition Is Initialization, 184, 191
return, 12
return type, 160
reusing, 37, 185, 227
role, 146
Role Segregation Principle, 175
root class, 150
row-major, 88
run, 17
run-time, 28, 29, 62, 91, 149, 155, 185
run-time error, 19
rvalue reference, 140

samart pointers, 192
scope, 69, 160, 191, 192
scope resolution operator, 38, 112, 118, 152
scopped enumaration, 151
sealed class, 150
segment, 70
segmentation violation, 198
self-referetial struct, 108
sentinel value, 57
seperation of concern, 173
sequential container, 196
sequential operation, 42, 44, 50
set, 79
shallow copy, 236
shared code, 131
short-circuit, 36
short-circuiting, 188
sign, 11
signed, 26
Single Responsibility Principle, 174
singleton class, 150
singleton pattern, 143, 231
size, 79
size_t, 32
sizeof... operator, 187
snap change, 131
software engineering, 167
SOLID principles, 174
solution pattern, 227
spaceship operator, 32
spaghetti code, 10
special class, 150
stack overflow, 74, 78
stack pointer, 69

stack register, 69
stack segment, 70, 72–74, 76, 91, 98, 99, 209
stack unwinding, 183
Standart Template Library, 196
standart type casting, 154
state, 116, 117, 119, 125, 135
state pattern, 277
statement, 9, 13, 19, 22, 30
statements, 14
static, 168
static binding, 149
static linking, 19
static specifier, 70
static storage class, 70
static variable, 70
static_cast operator, 155
storage classes, 69
stream, 142, 218
stream states, 221
string, 101, 103, 208, 221
string concatenate, 102
string literal, 38, 101
string streams, 221
struct, 109, 110, 144
structural pattern, 227, 236
structural programming, 13, 52
structure, 13
structure padding, 108
switch statement, 48
synchronous, 177
syntax, 19, 21

tab, 23
technician, 1
template, 161, 185, 191, 196
test pattern, 227
text segment, 70
this pointer, 123
throw an exception, 178, 179
tight coupling, 170
trace, 17
trace method, 150
two-dimentional array, 83
type aliasing, 165
type inference, 158, 160
type safety, 185, 288
typedef, 164
typedef keyword, 165

unary, 35
unary cast operator, 155
unary operator, 31
Unified Modeling Language, 167
Unified Modelling Language, 129
union, 110
unique, 82
unordered map, 204
unsigned, 26
user defined function, 61, 63
user defined literal, 28
user interface, 18
user-defined-class, 116
using, 169
using keyword, 114, 141, 165, 216
using namespace, 114
utility class, 150
utility method, 143

value type, 89
variable, 11, 13, 24
variable declaration, 25
variable scope, 25
variable type, 12
variadic function template, 186
variadic template, 186
variance, 81
version control, 18
view, 284
virtual method, 148
virtual specifier, 145
visibility, 119
visitor pattern, 268
void, 24, 64, 90
volatile specifier, 156
voletile, 26

while statement, 52
white space, 21, 29, 218
wide, 24
wide character, 210
widening casting, 154
wrapper, 192